
Deep Learning Club Handouts

发行版本 0.0.1

UCS Deep Learning Club

2026 年 06 月 17 日

1 写在前面：关于深度学习	1
1.1 深度学习是什么	1
1.2 黑盒与炼丹	2
1.3 你需要的心态	3
1.4 如何使用这份材料	4
1.5 与鱼书配合阅读	5
2 深度学习核心数学基础	9
2.1 引言	9
2.2 计算图	10
2.3 学习任务的形式	14
2.4 激活函数	19
2.5 损失函数	28
2.6 反向传播算法	33
2.7 梯度下降与优化算法	41
2.8 总结与展望	55
3 翻越两座山：CNN 三十年攀登史	59
3.1 引言：因为山在那里	59
3.2 上篇：练习峰	61
3.3 拆开看看：CNN 消融研究	127
3.4 下篇：真正的未登峰	147
3.5 架构心法：从会用装备到会设计装备	202
3.6 下一座山	259
4 PyTorch 实践：把理论变成代码	261
4.1 引言：从理论到代码	261
4.2 从 NumPy 到 PyTorch：张量的本质	265

4.3	张量操作：数据在网络中的流动	273
4.4	神经网络模块：搭建计算图	282
4.5	自动微分：PyTorch 的核心魔法	291
4.6	优化器：用梯度更新参数	300
4.7	完整训练流程	306
4.8	调试与可视化技巧	320
4.9	工程最佳实践	332
4.10	使用训练框架	347
4.11	结语：从入门到实践	360
5	模型部署与服务：从训练到生产	365
5.1	引言：训练完模型之后	365
5.2	ONNX：模型的中立语言	368
5.3	模型优化：量化与剪枝	376
5.4	服务架构：从单机到分布式	394
5.5	部署实践：用 Ferrinx 服务模型	399
5.6	结语：完整的学习链路	405
6	迁移学习与微调：站在巨人的肩膀上	409
6.1	导论与核心思想	409
6.2	迁移学习分类体系	414
6.3	基于模型的迁移	418
6.4	实操指南	427
6.5	结语：站在巨人的肩膀上	442
7	U-Net：图像分割的革命	447
7.1	引言：图像分割问题	447
7.2	U 形架构详解	450
7.3	损失函数设计	458
7.4	完整实现	462
7.5	数据增强与训练技巧	467
7.6	总结与展望	473
8	序列建模：从 RNN 到 Transformer 再到 Mamba	477
8.1	引言：从空间到时间	477
8.2	循环神经网络：让网络拥有"记忆"	478
8.3	LSTM：给记忆装上"门"	484
8.4	从 RNN 到注意力：一个思想的跳跃	492
8.5	Transformer：注意力的极致	498
8.6	Mamba 简述：对 RNN 思想的回归	506
8.7	总结与展望	515
9	附录	521
9.1	环境配置	521

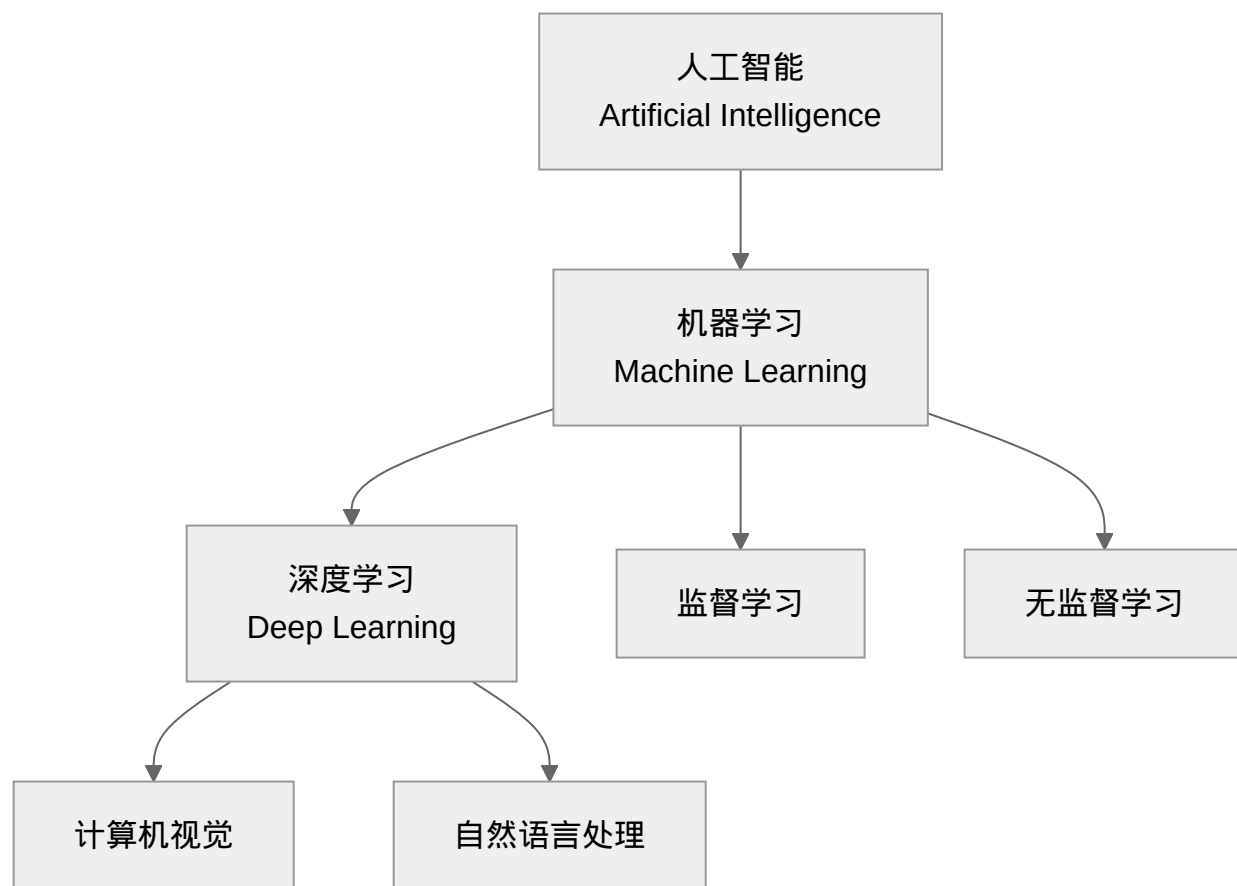
9.2	Sphinx Markdown 语法参考	560
9.3	编译文档	566
10	写在后面	577
10.1	可能的 FAQ:	578
	Bibliography	581

写在前面：关于深度学习

1.1 深度学习是什么

深度学习是机器学习的一个分支，而机器学习又是人工智能（AI）的一部分。可以这样理解它们的关系：

- **人工智能**：让机器展现出需要人类智能才能完成的能力（如识别图像、理解语言）
- **机器学习**：让机器从数据中自动学习规律，而不是通过显式编程
- **深度学习**：使用多层神经网络进行机器学习的方法



深度学习之所以“深”，是因为它的网络结构有很多层。每一层都会从数据中提取不同层次的特征：浅层识别边缘和纹理，深层识别物体部件和语义概念。这种层次化的特征提取，让深度学习在图像、语音、文本等领域取得了突破性进展。

1.2 黑盒与炼丹

1.2.1 为什么叫炼丹

老一代深度学习研究者常自嘲说自己在“炼丹”。这个说法虽然调侃，但确实抓住了深度学习的某些特点：

- **配方复杂**：学习率、批大小、网络结构、正则化参数...每个选择都可能影响结果
- **过程不可控**：同样的代码跑两次，结果可能略有不同（随机初始化、数据打乱）
- **结果难预测**：理论上合理的改动，实践中可能完全无效甚至适得其反
- **经验重于理论**：很多时候靠的是“试试就知道”，而不是严密的数学推导

1.2.2 但炼丹也有科学方法

虽然我们自嘲炼丹，但这并不意味着深度学习完全是玄学。好的“炼丹师”会：

- **做对照实验**：改变一个变量，保持其他不变，观察影响
- **记录实验日志**：详细记录每次实验的配置和结果，建立可复现性
- **理解原理**：知道为什么要用某个技巧（如 BatchNorm 是为了解决内部协变量偏移）
- **系统性调参**：不是随机尝试，而是按重要性排序（学习率 > 网络结构 > 优化器 > ...）

1.2.3 黑盒不等于不可理解

深度学习模型常被批评为“黑盒”——输入进去，输出出来，中间发生了什么不知道。但实际上，我们有越来越多的工具来理解模型：

- **特征可视化**：看卷积层在关注图像的哪些部分
- **注意力热图**：看 Transformer 在处理文本时关注了哪些词（详见 [Transformer：注意力的极致](#)）
- **消融实验**：去掉某个模块，看性能下降多少，从而判断其重要性
- **对抗样本分析**：通过构造特殊输入，理解模型的决策边界

这些方法让我们能够“打开”黑盒，虽然不能完全理解每一个细节，但至少知道模型在做决策时考虑了哪些因素。

1.3 你需要的心态

1.3.1 理论与实践并重

深度学习是一个理论与实践紧密结合的领域。只懂理论不会写代码，你无法验证想法；只写代码不懂理论，你只能在已知范围内打转。

建议的学习方式是：

- **先理解原理**：知道某个技术为什么有效（如：Dropout 是集成学习的近似）
- **再动手实现**：用自己的代码复现一遍，加深理解
- **最后做实验**：在自己的数据集上验证，观察实际效果

1.3.2 培养直觉

在深度学习领域，有时候**直觉比公式更重要**。当你看到一个新的架构时，能够凭直觉判断“这个设计会有效吗”比能够推导反向传播公式更有价值。

培养直觉的方法：

- **多看架构**：了解经典网络的设计思路和演进历程

- **多做实验**：亲手调参，感受不同选择的影响
- **多思考“为什么”**：不是记住“这样做有效”，而是理解“为什么有效”

记住，深度学习领域很多“真理”都是暂时的。五年前的最佳实践，今天可能已经被淘汰。只有培养出好的直觉，才能在这个快速变化的领域保持竞争力。

1.3.3 保持怀疑精神

深度学习领域充斥着各种说法：

- “这个架构在 ImageNet 上达到了 SOTA”
- “我们的方法比基线提高了 5%”
- “这个技巧对所有任务都有效”

作为学习者，你需要保持怀疑：

- **质疑结果**：5%的提升是统计显著的吗？测试集有没有泄露？
- **质疑结论**：在一个数据集上的有效，能推广到其他数据集吗？
- **质疑论文**：作者有没有隐藏负面结果？对比的基线是不是太弱？
- **质疑权威**：即使是大佬说的，也要有独立判断

怀疑精神不是为了抬杠，而是为了建立真正可靠的知识。当你能够分辨哪些是可靠的结论、哪些是过度宣传时，你就成为了合格的深度学习从业者。

1.3.4 耐心与迭代

最后，学习深度学习需要耐心。不要指望看一遍材料就能完全理解，也不要因为第一次实验失败就气馁。

- **反复阅读**：第一遍可能只看懂 30%，第二遍 60%，第三遍 90%
- **多次实验**：调参很少一次成功，记录每次结果，逐步逼近最优
- **接受失败**：很多想法最终证明是无效的，但这本身就是学习过程

记住，即使是顶级研究者，大部分想法也是失败的。区别在于，他们能够从失败中提取信息，快速迭代到下一个想法。

1.4 如何使用这份材料

这份材料的设计遵循了上述学习理念：

1. **从直觉出发**：每个概念都先建立直觉理解，再深入数学细节
2. **理论与实践结合**：每个理论点都有对应的代码实现和实验

3. **强调对比和怀疑**：鼓励你质疑、实验、形成自己的判断

4. **渐进式深入**：从基础概念逐步过渡到前沿技术

建议的学习方式是：**先通读建立整体认知，再精读理解细节，最后动手实验验证**。遇到不懂的地方，可以暂时跳过，很多时候后面的内容会帮助理解前面的概念。

1.5 与鱼书配合阅读

如果你正在读斋藤康毅的《深度学习入门：基于 Python 的理论与实现》（俗称“鱼书”），那么恭喜你——你手里拿的是深度学习领域口碑最好的入门书之一。本书和鱼书不是替代关系，而是**互补关系**。

1.5.1 鱼书讲什么

鱼书共 8 章，从 Python 基础出发，**不依赖任何外部库**，只用 NumPy 从零开始手写：

章节	内容	鱼书的做法
第 1 章	Python 入门	复习基础，熟悉 NumPy
第 2 章	感知机	用 Python 实现逻辑门，理解“权重+偏置”
第 3 章	神经网络	手写 sigmoid、softmax、3 层网络
第 4 章	神经网络的学习	梯度下降、损失函数、数值微分
第 5 章	误差反向传播法	计算图 ——鱼书最精彩的部分
第 6 章	与学习相关的技巧	参数更新（SGD/Momentum/Adam）、BatchNorm、正则化
第 7 章	卷积神经网络	im2col、卷积/池化的手工实现
第 8 章	深度学习	深层网络、历史（AlexNet/VGG/ResNet）、应用

鱼书核心理念是：“**只有做出来才能真正理解**”。它强迫你亲手写每一行代码，让你明白 PyTorch 里那些 `backward()`、`nn.Conv2d` 背后到底发生了什么。

鱼书第 1~2 章与本材料的衔接：鱼书第 1 章（Python/NumPy）和第 2 章（感知机）非常基础。本材料默认读者**已经会用 Python 和 NumPy**，所以如果你 Python 完全不熟，建议先快速过一遍鱼书第 1 章；如果你已经能熟练写 NumPy 数组操作，这两章可以跳过，直接从第 3 章开始配着本材料读。

1.5.2 怎么配合读

场景	建议
你完全零基础	先读鱼书第 1~3 章，再读本材料的数学基础和神经网络基础
你懂 Python 但不懂反向传播	鱼书第 5 章（计算图）+ 本材料 数学基础 ，两手抓
你会跑 PyTorch 但不懂原理	鱼书第 4~6 章（梯度下降、更新技巧）+ 本材料对应理论，填坑
你想参加比赛/做项目	鱼书第 7 章（CNN 手写实现）+ 本材料 CNN 消融研究+迁移学习，实战起飞
你卡在某个概念	鱼书用代码讲，本材料用直觉和类比讲，两边对照

一个简单的阅读策略：

- 1. 并行推进：鱼书读一章，本材料读对应章节
 - 鱼书第 3 章（神经网络）→ 本材料[翻越两座山：CNN 三十年攀登史](#)
 - 鱼书第 5 章（反向传播）→ 本材料[深度学习核心数学基础](#)
 - 鱼书第 7 章（CNN）→ 本材料[拆开看看：CNN 消融研究](#)
- 2. 交叉验证：鱼书教会你"怎么造轮子"，本材料教会你"怎么开车"+ “为什么这样设计车”。当你在本材料看到 `nn.Conv2d` 时，鱼书第 7 章让你知道它内部是怎么做 `im2col` 和矩阵乘法的。

1.5.3 典型学习路径示例（按周规划）

如果你完全没有自学规划，这里有一个 8 周并行推进方案。每周大约投入 6~10 小时：

周次	鱼书章节	本材料章节	核心目标
第 1 周	第 1 章 (Python 复习) 第 2 章 (感知机)	深度学习核心数学基础	补齐 NumPy/Python, 建立数学工具箱
第 2 周	第 3 章 (神经网络) 第 4 章 (神经网络的学习)	深度学习核心数学基础 (续)	前向传播、损失函数、梯度下降 —— 数学打底
第 3 周	第 5 章 (误差反向传播)	深度学习核心数学基础 (反向传播)	计算图 + 链式法则 —— 全书最难, 单独一周消化
第 4 周	—	PyTorch 实践: 把理论变成代码	前 3 周数学储备已够, 正式上手 PyTorch: 张量、自动微分、训练循环
第 5 周	第 6 章 (学习技巧)	上篇: 练习峰	优化器、BatchNorm、正则化 → 用 PyTorch 写出第一个 LeNet
第 6 周	第 7 章 (CNN 手写实现)	拆开看看: CNN 消融研究	对照鱼书 im2col 内部机制 → 拆开 LeNet 验证每个组件贡献
第 7 周	第 8 章 (深度学习历史)	下篇: 真正的未登峰	AlexNet→Inception→ResNet→MobileNet→EfficientNet 全景
第 8 周	—	模型部署与服务: 从训练到生产 或 迁移学习与微调: 站在巨人的肩膀上	二选一: 端到端部署上线, 或迁移学习做新任务 —— 完成第一个项目

灵活调整: 如果你已经懂 Python, 第 1 周可以压缩到 2 天; 第 8 周如果想两个都学, 可以延长到第 9 周。

1.5.4 一句话总结

鱼书是“从泥土里把神经网络种出来”, 本材料是“站在山上看清森林的地图”。两本对照, 既知道根扎多深, 也看得见枝伸多远。

现在, 你已经对深度学习有了初步的认识, 也了解了需要什么样的心态, 还知道如何把手里的材料与经典教材配合。接下来, 让我们从数学基础开始, 一步步揭开深度学习的面纱。

祝学习愉快, 炼丹顺利。

深度学习核心数学基础

2.1 引言

2.1.1 学习目标与核心概念

在深入探讨计算图、反向传播和梯度下降之前，我们先明确本章的学习目标：

学习目标

- **计算图直觉**：把数学运算看作"数据流动"的管道系统
- **激活函数直觉**：理解非线性如何将线性不可分的数据"折叠"成可分
- **损失函数直觉**：理解不同目标如何塑造优化问题的几何结构
- **反向传播直觉**：把最终误差"分摊"给每个参数（信用分配机制）
- **梯度下降直觉**：理解损失曲面的几何形状如何影响优化路径
- **代码连接**：在 PyTorch 中观察这些机制的实际表现

2.1.2 机器学习的基本问题

机器学习的目标是让计算机从数据中自己发现规律，而不是我们一条条写规则。核心挑战是如何自动调整模型参数，让它能更好地拟合数据、做出准确预测。

机器学习的核心挑战

- **模型复杂度**：现代深度学习模型可能有数百万甚至数十亿个参数
- **优化难度**：在如此高维的空间里找到最优解，就像在迷宫里找出口
- **计算效率**：需要又快又省的算法处理海量数据和复杂模型
- **泛化能力**：模型不仅要在训练数据上表现好，还要在没见过的新数据上靠谱

2.1.3 为什么需要这些核心机制？

神经网络训练涉及几个关键环节，每个环节都需要特定的数学工具：

1. **计算图**：描述数据如何在网络中流动，为自动求导提供结构基础
2. **激活函数**：引入非线性，让网络能够划分复杂的决策边界
3. **损失函数**：定义"预测好坏"的度量，将训练转化为优化问题
4. **反向传播算法**：高效计算梯度，完成误差的"信用分配"
5. **梯度下降与优化算法**：利用梯度信息迭代优化参数

这些概念相互配合，构成了现代深度学习框架的核心机制 [GBC16]。

2.1.4 本章预览

本章我们将从**计算图**开始，建立"数据流"的直觉视角。然后理解**激活函数**如何用非线性变换在空间中划分决策边界。接着探讨**损失函数**如何定义优化目标、塑造损失曲面的几何形状。之后理解**反向传播算法**如何高效地完成"信用分配"——把最终误差分摊给每个参数。最后探索**梯度下降与优化算法**如何在损失曲面上寻找最优解。

掌握这些核心机制后，**总结与展望**将回顾本章要点，然后下一章我们将用 PyTorch 构建实际的神经网络，在 MNIST 任务上观察这些理论如何转化为训练动态。

2.2 计算图

2.2.1 什么是计算图？

计算图是一种用图来表示数学运算的方法。你可以把它想象成**数据流的管道系统**：数据从输入节点流入，经过各种操作节点（如加法、乘法、激活函数），最终到达输出节点。

这种表示方式有两个核心价值 [Wer90]:

1. **可视化**: 复杂的数学表达式变成清晰的流程图
2. **自动求导**: 为反向传播提供结构基础, 让梯度计算自动化

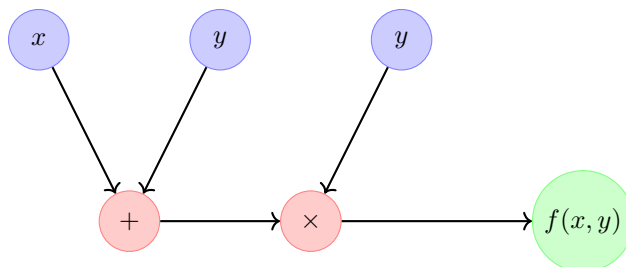


图 1: 计算图示例: $f(x, y) = (x + y) \times y$

在这个图中:

- **蓝色节点**: 输入变量 (x, y)
- **红色节点**: 操作 (加法 +、乘法 $\times$)
- **绿色节点**: 最终输出
- **箭头**: 数据流动方向

2.2.2 前向传播: 数据如何流动

前向传播就是数据从输入节点流向输出节点的过程。按照拓扑顺序 (先计算依赖的节点), 每个操作节点接收输入、计算输出。

示例: 计算 $f(x, y, z) = (x + y) \times z$

设 $x = 2, y = 3, z = 4$:

1. **输入节点**: $x = 2, y = 3, z = 4$
2. **加法节点**: $a = x + y = 2 + 3 = 5$
3. **乘法节点**: $f = a \times z = 5 \times 4 = 20$

这就是计算图在前向传播中的工作流程。

2.2.3 反向传播: 梯度如何回流

计算图的真正威力在于**反向传播算法**。梯度 (误差信号) 沿着边的反方向流动, 从输出节点传回输入节点。这使得我们可以高效计算每个参数对最终损失的贡献。

关键洞察

计算图让"信用分配"变得系统化：

- 前向传播：数据沿着箭头方向流动，计算输出
- 反向传播：梯度沿着箭头反方向流动，分摊误差

2.2.4 PyTorch 中的动态计算图

PyTorch 使用**动态计算图** (define-by-run)：每次前向传播时实时构建计算图，这提供了极大的灵活性。

```
import torch

# 创建张量并启用梯度跟踪
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)
z = torch.tensor(4.0, requires_grad=True)

# 构建计算图（前向传播）
a = x + y      # 加法操作
f = a * z      # 乘法操作

print(f"前向传播: f = {f}") # 输出: 20.0

# 反向传播：自动计算梯度
f.backward()

print(f"\n 梯度计算:")
print(f"∂f/∂x = {x.grad}") # 输出: 4.0 (z 的值)
print(f"∂f/∂y = {y.grad}") # 输出: 4.0 (z 的值)
print(f"∂f/∂z = {z.grad}") # 输出: 5.0 (a 的值)
```

梯度计算的解释

为什么梯度是这些值？根据链式法则：

- $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial x} = z \cdot 1 = 4$
- $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial y} = z \cdot 1 = 4$
- $\frac{\partial f}{\partial z} = a = 5$

PyTorch 的 `backward()` 自动完成了这些计算。

2.2.5 计算图的优势

- **可视化复杂计算**: 深度网络可能有数百万个操作，计算图将其分解为可理解的模块。
- **自动微分**: 无需手动推导梯度公式，计算图结构让自动求导成为可能。
- **优化执行**:

通过分析依赖关系，可以：

- 确定最优计算顺序
- 识别可并行执行的操作
- 避免重复计算

2.2.6 从计算图到神经网络

神经网络的计算图具有以下特点：

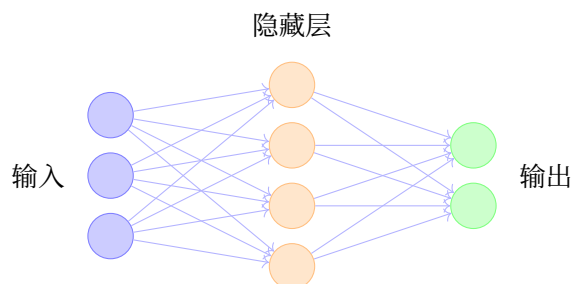


图 2: 简化神经网络计算图

- **多层结构**: 输入层 → 隐藏层 → 输出层
- **权重参数**: 每条边代表一个可学习的权重
- **激活函数**: 每个节点通常包含非线性变换

在下一节中，我们将深入探讨**激活函数**——计算图中的非线性操作如何让神经网络拥有划分复杂决策边界的能力。

2.2.7 总结

计算图是深度学习的核心数据结构：

- **前向传播**: 数据沿着边流动，计算预测结果
- **反向传播**: 梯度沿着边反向流动，为**反向传播算法**奠定基础
- **自动求导**: 计算图结构让**反向传播算法**的梯度计算自动化

理解计算图后，我们将探索激活函数如何用非线性变换在空间中划分决策边界。之后损失函数会将预测误差量化，反向传播算法通过计算图高效计算梯度，最后梯度下降与优化算法利用这些梯度优化参数。

2.3 学习任务的形式

计算图中介绍的计算图描述了数据如何在模型中流动。但数据流向什么样的目标？不同类型的“规律”需要不同的数学建模方式。

2.3.1 机器学习在找什么规律？

机器学习的本质是从数据中发现规律，然后用这些规律做预测 [Bis06]。但“规律”有不同形式：

规律类型	核心问题	例子
类别边界	这类事物有什么共同特征？	识别猫和狗
数值映射	输入输出是什么关系？	根据面积预测房价
序列依赖	下一步应该是什么？	续写句子

这三种规律对应三种任务类型：**分类**、**回归**和**自回归**。它们共享相同的计算图计算机制，但目标的数学形式截然不同。

2.3.2 分类任务：发现类别边界的规律

想象你在教小孩认动物。你不会给他看一张 checklist，而是指着图片说：“这个有四条腿、有毛、会汪汪叫，所以是狗。”

分类的本质：找到区分不同类别的**边界**——边界这边是 A 类，那边是 B 类。

在特征空间中，每个样本是一个点，分类就是画一条**决策边界**把空间切开：**关键洞察**：

- 边界这边聚集着同类样本
- 边界是“模糊”的（软边界），不是一刀切的硬线
- 新样本落在哪边，就预测为哪类

数学形式：分类模型输出类别概率分布 $p(y = k|x)$ ，其中 $k \in \{1, 2, \dots, K\}$ 。决策规则是选择概率最大的类别：

$$\hat{y} = \arg \max_k p(y = k|x)$$

决策边界是两个类别概率相等的位置： $p(y = i|x) = p(y = j|x)$ 。

激活函数中讨论的非线性激活函数，正是为了让神经网络能够学习**非线性的决策边界**。没有非线性激活，无论多少层网络都只能学习线性边界。

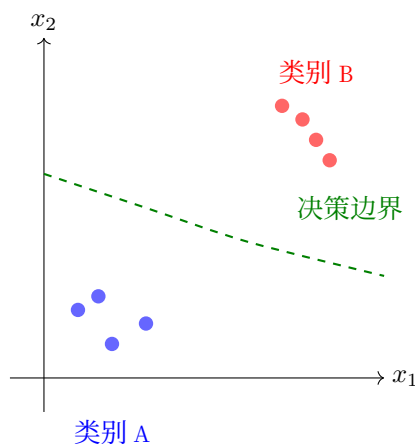


图 3: 分类任务的决策边界

2.3.3 回归任务：发现数值映射的规律

想象你在做物理实验：测量不同拉力下弹簧的伸长量。你发现“拉力越大，弹簧伸得越长”。这不是分类问题，因为你不是把拉力分成“大”或“小”两类，而是要找出**具体的数值关系**。

回归的本质：学习一个**函数映射**，输入一个数值，输出另一个数值。

最简单的回归是**线性回归**：假设输入和输出之间是直线关系。类比：你收集了若干房屋的面积和价格数据，在纸上画散点图，然后画一条最“拟合”这些点的直线。

数学形式：对于单变量线性回归

$$\hat{y} = wx + b$$

- x : 输入特征（如房屋面积）
- w : 权重（斜率，表示“每平方米值多少钱”）
- b : 偏置（截距，表示“基础价格”）
- $\hat{y}$: 预测值（预测房价）

几何意义

在二维平面上，线性回归就是找一条直线，让所有数据点到直线的**垂直距离之和最小**。扩展到多元：现实中的房价不只由面积决定，还取决于位置、房龄、楼层等。多元线性回归：

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = \mathbf{w}^T \mathbf{x} + b$$

这不再是二维平面上的直线，而是**高维空间中的超平面**。

非线性回归

线性回归假设关系是直线，但很多规律是曲线。神经网络通过堆叠非线性层，可以学习**任意复杂的函数映射**。

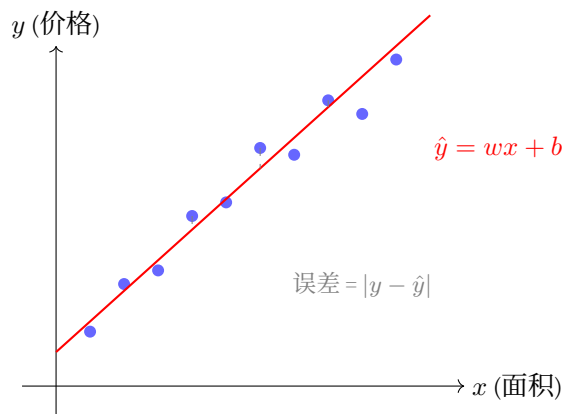


图 4: 线性回归的几何意义

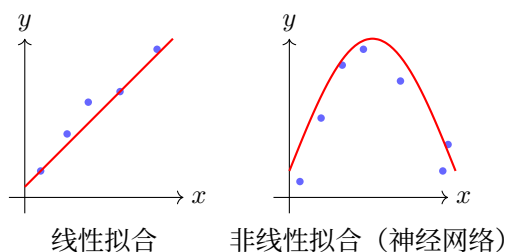


图 5: 从线性到非线性回归

回归 vs 分类的本质区别

维度	分类	回归
规律形式	类别边界 (划分区域)	函数映射 (输入 $\rightarrow$ 输出)
输出空间	离散有限集 $\{1, \dots, K\}$	连续无穷集 $\mathbb{R}$
预测内容	“这是哪一类”	“数值是多少”
误差度量	分类错误率	数值偏差 $ y - \hat{y} $
几何形象	空间中的分界曲面	空间中的拟合曲面

一句话总结：分类是“切蛋糕”（划分空间），回归是“画曲线”（拟合关系）。

2.3.4 自回归任务：发现序列依赖的规律

想象你在听故事：“从前有座山，山里有座... “你自然会接”庙”。为什么不是“城堡”或“学校”？因为你学过，“山里有座”后面最常接的是“庙”。

自回归的本质：基于已生成的内容预测下一个内容，一步一步“续写”。

概率分解：序列的联合概率可以用链式法则分解。对于两个事件， $p(A, B) = p(A) \cdot p(B|A)$ —— 先发生 A，再

在 A 发生的条件下发生 B。扩展到序列：

$$p(y_1, y_2, y_3) = p(y_1) \cdot p(y_2|y_1) \cdot p(y_3|y_1, y_2)$$

一般形式：

$$p(y_1, y_2, \dots, y_T) = \prod_{t=1}^T p(y_t|y_{<t})$$

其中 $y_{<t}$ 表示 y_1 到 y_{t-1} 的所有历史内容。

为什么这样分解有用？ 想象你要生成句子“猫坐在垫子上”。直接学习整个句子的概率很难，但分解后每步都是简单的分类：

- $p(\text{猫}|\text{<s>})$ ：句首是“猫”的概率
- $p(\text{坐}|\text{<s>猫})$ ：看到“猫”后接“坐”的概率
- $p(\text{在}|\text{<s>猫坐})$ ：看到“猫坐”后接“在”的概率

因果性约束： 自回归只能依赖过去，不能“偷看”未来：

$$p(y_t|y_1, \dots, y_{t-1}, \underbrace{y_{t+1}, \dots}_{\text{不能用}})$$

这就像写作文——你不能用还没写的句子来决定现在写什么。这种“只看左边”的特性称为**因果掩码**。

训练 vs 推理：

- **训练时：** 已知完整序列“猫坐在垫子上”，模型同时学习每个位置的条件概率。可以并行计算所有位置的损失。
- **推理时：** 从<start>开始，生成第一个词，然后用这个词作为输入生成第二个词，依此类推。必须**串行**进行。

序列生成的树状结构： 在每一步，模型输出一个概率分布，然后采样（或选择最大概率）得到下一个词。

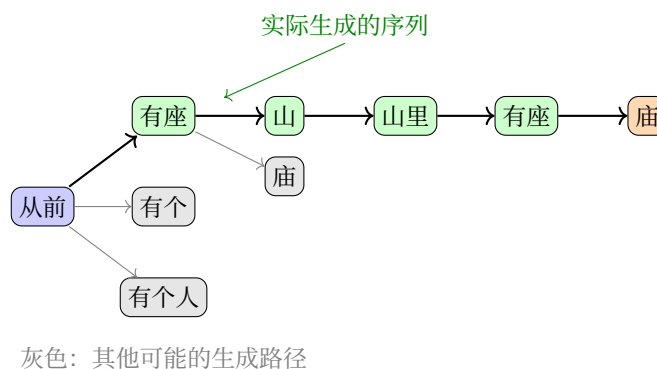


图 6: 自回归的序列生成

与分类的联系：自回归的每一步本质上都是分类——从词汇表中选择一个词。区别在于：普通分类的每次预测独立进行；而自回归的每次预测**依赖所有历史选择**，是“**有记忆的分类**”。评估时也不只看单步准确率，而是看整个序列的质量（用困惑度衡量）。

2.3.5 三种任务的统一与差异

三类任务都遵循相同的计算框架（见**计算图**）：输入 → 变换 → 输出 → 损失计算。差异在于**输出层的设计**和**损失函数的选择**。

任务	规律形式	输出层	典型损失	几何直观
分类	类别边界	Softmax	交叉熵	空间区域划分
回归	函数映射	线性	MSE/MAE	曲面拟合
自回归	条件概率链	Softmax（每步）	累计交叉熵	树状展开

同一输入在不同任务中的处理流程：

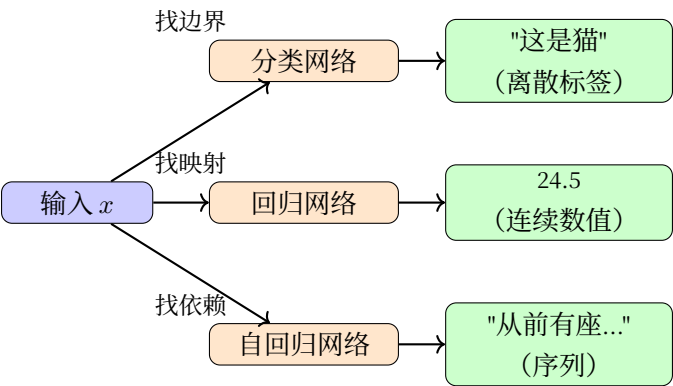


图 7: 三种任务的处理流程对比

2.3.6 总结

概念	实践	联系
分类	发现类别边界	用 激活函数 学习非线性边界
回归	学习函数映射	线性回归是基础，神经网络扩展非线性
自回归	建模序列依赖	每步是分类，整体是序列

三类任务的区别不在于网络的内部结构（都可以用相同的层堆叠），而在于：

- 1. **输出层的设计**（Softmax vs 线性 vs 每步 Softmax）

2. 损失函数的选择（交叉熵 vs MSE vs 累计交叉熵）

3. 评估的方式（准确率 vs 误差大小 vs 序列质量）

理解了三种任务的数学形式后，**损失函数**将详细介绍如何用损失函数**量化**预测的好坏——分类用交叉熵，回归用 MSE，自回归用累计交叉熵。这些损失函数塑造了不同的**梯度下降与优化算法**优化曲面。

2.4 激活函数

2.4.1 激活函数的本质：划分特征空间

从线性回归到逻辑回归

你可能熟悉**线性回归**： $y = wx + b$ ，它用一条直线拟合数据，输出连续值。但很多时候我们需要做**分类**——不是预测数值，而是判断“这是 A 类还是 B 类”。

问题：如果用线性回归做二分类会怎样？

假设我们想区分“垃圾邮件”和“正常邮件”。如果用线性回归输出 $y = wx + b$ ，我们得到一个实数值。但分类需要的是一个概率——“这封邮件是垃圾邮件的可能性有多大”。

逻辑回归解决了这个问题。它在线性回归的输出上叠加一个 **Sigmoid** 函数：

$$\hat{y} = \sigma(wx + b) = \frac{1}{1 + e^{-(wx+b)}}$$

Sigmoid 将任意实数压缩到 $(0, 1)$ 区间，恰好可以解释为概率。

逻辑回归的决策规则：

- 如果 $\sigma(wx + b) \geq 0.5$ ，预测为类别 1
- 如果 $\sigma(wx + b) < 0.5$ ，预测为类别 0

这等价于判断 $wx + b \geq 0$ 还是 $wx + b < 0$ ——在特征空间中，就是用一条**直线**（或超平面）划分两个区域。逻辑回归只能学习**线性决策边界**。

为什么需要激活函数

逻辑回归解决了二分类问题，但它有一个根本限制：**只能学习线性边界**。现实中很多分类问题的决策边界是复杂的非线性形状——比如异或（XOR）问题，逻辑回归就无法解决。

多层感知机（MLP）突破了这个限制：

- 第一层：多个神经元并行，各自学习不同的线性边界
- **激活函数**：引入非线性，将空间“折叠”
- 后续层：组合这些折叠后的特征，学习复杂边界

什么是"空间"?

在深度学习中，**空间**指的是输入数据的特征空间：

- 一个样本对应空间中的一个点
- 每个特征对应一个**坐标轴**
- MNIST 图像（784 维）就是 784 维空间中的点

神经网络的每一层都在**变换这个空间**：

- 线性变换 ($Wx + b$)：旋转、缩放、平移
- **非线性激活**：将空间"折叠"，使线性不可分的数据变得可分

决策边界的形成

没有激活函数，无论堆叠多少层，网络只能学习线性变换 —— 因为多个线性变换的组合仍然是线性的： $f(W_2(W_1x)) = W_2W_1x = W'x$ 。激活函数引入非线性，使网络能够在空间中划分**复杂的决策边界**。

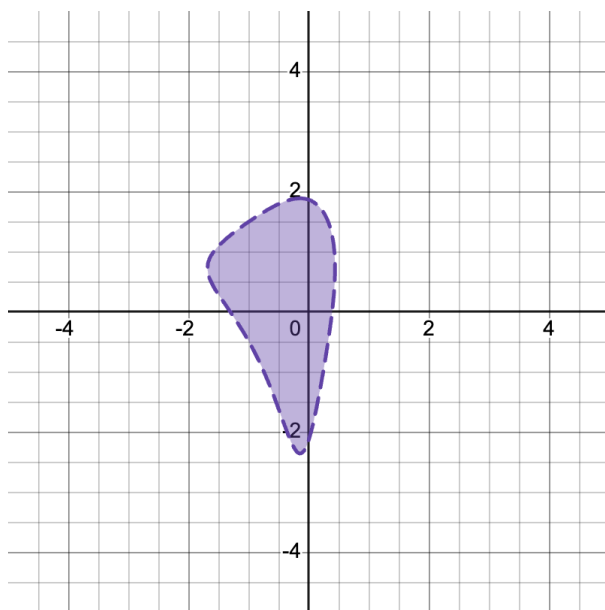
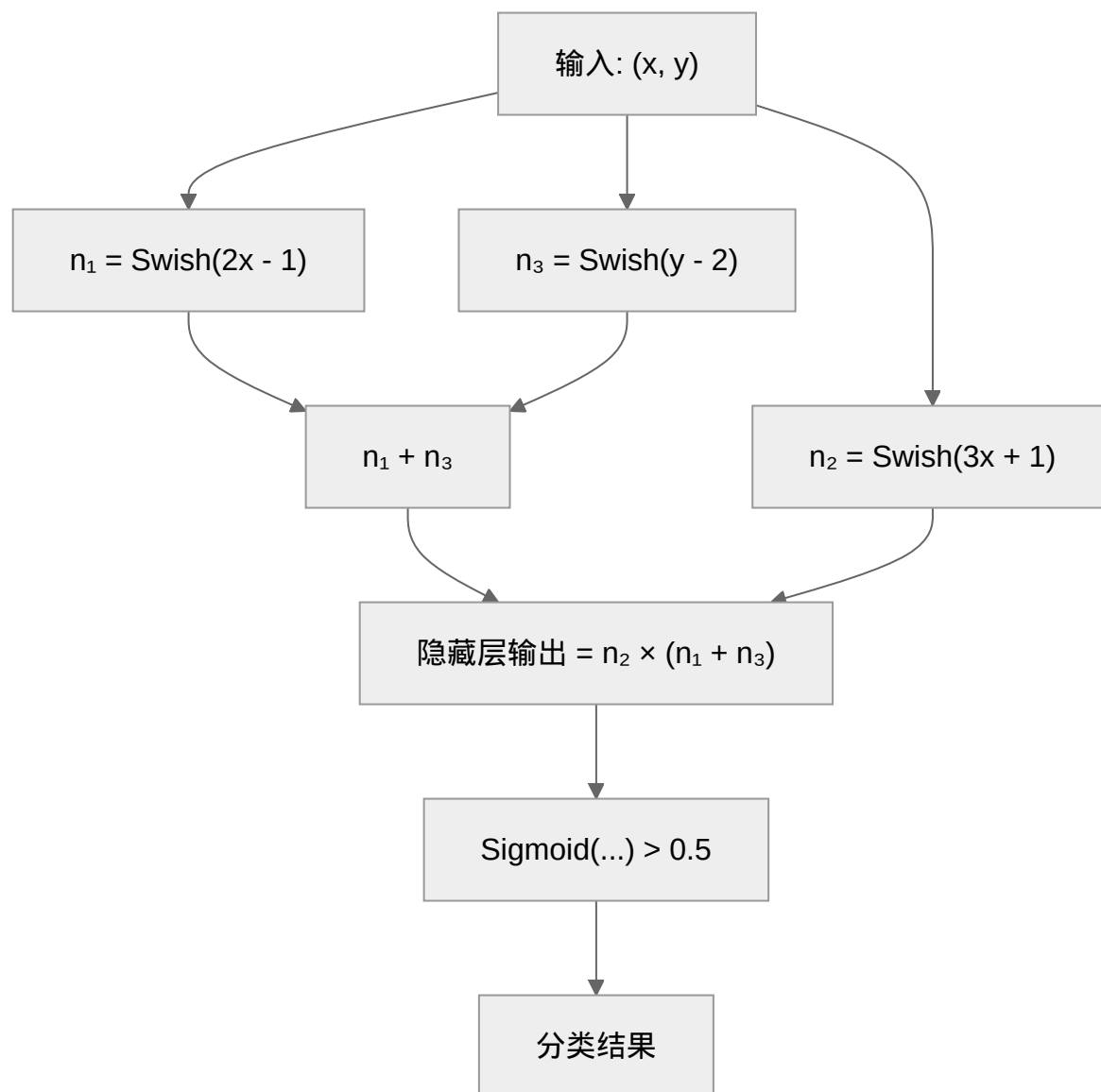


图 8: 3 神经元网络形成的复杂决策边界

网络结构解析：

这个网络只有 3 个隐藏层神经元，却能形成如此复杂的决策边界。让我们逐层解析：



数学表达式:

基础激活函数:

- **Swish**: 平滑的 ReLU 变体

$$r(x) = \frac{x}{1 + e^{-x}}$$

- **Sigmoid**: 将值压缩到 0-1

$$s(x) = \frac{1}{1 + e^{-x}}$$

隐藏层神经元（3 个独立的线性变换 + Swish）：

$$\begin{aligned} n_1 &= r(2x - 1) \quad [\text{对 } x \text{ 敏感的神经元}] \\ n_2 &= r(3x + 1) \quad [\text{另一个 } x \text{ 方向的神经元}] \\ n_3 &= r(y - 2) \quad [\text{对 } y \text{ 敏感的神经元}] \end{aligned}$$

输出决策边界：

$$\text{决策} = s\left(\underbrace{n_2}_{\text{门控}} \times \underbrace{(n_1 + n_3)}_{\text{特征组合}}\right) > 0.5$$

为什么能形成复杂形状？

1. 每个 Swish 神经元定义一个"软"半平面：

- $n_1 = r(2x - 1)$ ：当 $2x - 1 > 0$ （即 $x > 0.5$ ）时显著激活
- $n_3 = r(y - 2)$ ：当 $y > 2$ 时显著激活
- 注意 Swish 不是简单的 0/1 开关，而是平滑过渡

2. 特征组合 $n_1 + n_3$ ：

- 将两个方向的激活"叠加"
- 在 $x > 0.5$ 且 $y > 2$ 的区域最强

3. 门控机制 $n_2 \times (\dots)$ ：

- n_2 充当"开关"，控制哪些区域可以输出
- 当 n_2 很小时，无论 $(n_1 + n_3)$ 多大，输出都接近 0

4. Sigmoid 最终决策：

- 将连续值转换为概率式判断
- > 0.5 判定为类别 A（紫色区域）

直观理解：这个网络像是在空间中"雕刻"——先用 n_1, n_3 定义一个粗略的区域，再用 n_2 进行精细修剪，最终得到不规则的形状。

交互式探索：[Desmos¹](https://www.desmos.com/calculator/gnixzkljaz)（可以拖动滑块调整参数，观察形状变化）

核心洞察：

从这个 3 神经元的例子，我们可以看到神经网络划分决策边界的关键机制：

1. 每个神经元定义一个"软"边界：

- Swish 激活将线性边界 $wx + b = 0$ 变成平滑的过渡区域

¹ <https://www.desmos.com/calculator/gnixzkljaz>

- 不同于 ReLU 的硬开关, Swish 提供渐变的激活强度
- 这允许网络学习更平滑、更复杂的边界

2. 非线性组合创造复杂性:

- 乘法操作 $n_2 \times (n_1 + n_3)$ 是关键
- 没有激活函数的非线性, 这样的组合无法实现
- 这展示了**非线性激活的本质作用**: 让简单线性边界的组合产生复杂形状

3. "门控"与"特征"的配合:

- n_1, n_3 提供基础特征 (对 x 和 y 的响应)
- n_2 充当门控, 决定哪些区域可以"通过"
- 这种配合模式在深层网络中反复出现

4. 少量神经元就能实现复杂分类:

- 仅需 3 个神经元就能形成非凸、非线性的决策区域
- 这说明神经网络的表达能力来自**非线性组合**, 而非单纯增加神经元数量
- 深度 (多层非线性组合) 比宽度 (单层神经元数量) 更重要

关键结论: 非线性激活函数让神经网络能够将简单的线性决策边界**组合、变换、叠加**成任意复杂的形状。没有非线性激活, 无论多少层都只能学习线性分类器: $f(W_2(W_1x)) = W_2W_1x = W'x$ 。

2.4.2 Sigmoid 函数

Sigmoid 将任意实数映射到 $(0, 1)$, 提供平滑的非线性变换:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

导数: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, 在 $x = 0$ 处最大值为 0.25。这个导数形式在[反向传播算法](#)中非常重要。

优点:

- 输出可解释为概率
- 平滑可导

局限性:

- **梯度消失:** $|x|$ 较大时梯度接近 0, 这会导致[反向传播算法](#)时梯度衰减
- **非零中心:** 输出始终为正

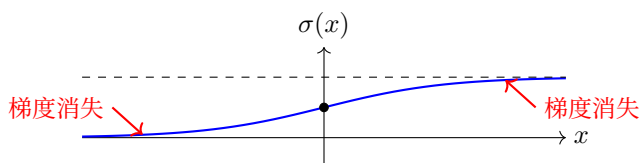


图 9: Sigmoid 函数

2.4.3 Tanh 函数（双曲正切）

Tanh 是 Sigmoid 的"零中心"版本：

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

导数： $\tanh'(x) = 1 - \tanh^2(x)$ ，在 $x = 0$ 处最大值为 1

相比 Sigmoid 的优势：

- 零中心输出（-1 到 1）
- 更强梯度信号（最大梯度是 Sigmoid 的 4 倍）

局限：仍然存在梯度消失问题

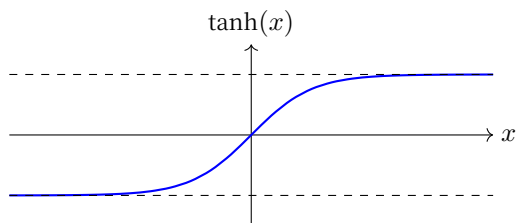


图 10: Tanh 函数

2.4.4 ReLU 函数（Rectified Linear Unit）

ReLU [NH10] 于 2012 年 AlexNet 的成功后成为主流：

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

导数： $\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$

核心优势：

- 解决梯度消失：正区间梯度恒为 1

- 计算高效：仅需比较操作
- 稀疏激活：约 50% 神经元输出为 0

问题：“死亡神经元”——若输入始终为负，梯度永远为 0，神经元永久失活

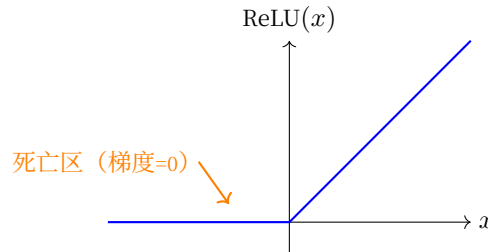


图 11: ReLU 函数

2.4.5 Leaky ReLU 与变体

为解决死亡神经元问题，Leaky ReLU 给负区间一个小斜率：

$$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

其中 α 通常取 0.01。

优势：

- 负区间有非零梯度，防止神经元死亡
- 保持 ReLU 的计算效率

进阶变体：

- PReLU： α 作为可学习参数
- ELU：负区间使用指数函数，更平滑

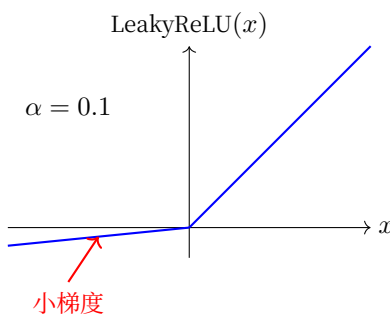


图 12: Leaky ReLU 函数

2.4.6 Swish 函数

由 Google Brain 团队的 Prajit Ramachandran 等人于 2017 年通过神经架构搜索发现 [RZL17]。其设计动机体现了深度学习研究的新趋势：**通过自动化方法发现新的激活函数**

$$\text{Swish}(x) = x \cdot \sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}$$

其中 β 是可选的可学习参数（通常取 1.0）。

性质：

1. **自门控机制**：Swish 将输入 x 作为“门”，将 $\sigma(\beta x)$ 作为“门控信号”，实现 x 与 $\sigma(\beta x)$ 的逐元素相乘
2. **平滑非单调**：与 ReLU 不同，Swish 是非单调的，在负区间有小的负值
3. **可学习参数**： β 可以设置为可学习的参数，让网络自动调整门控强度
4. **实验验证**：通过大规模实验验证，Swish 在多个任务上优于 ReLU

与 ReLU 的关系

当 $\beta \rightarrow \infty$ 时， $\sigma(\beta x) \rightarrow \text{step}(x)$ ，Swish 趋近于 ReLU：

$$\lim_{\beta \rightarrow \infty} \frac{x}{1 + e^{-\beta x}} = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

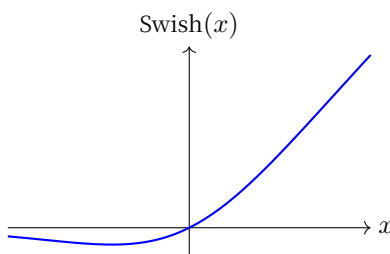


图 13: Swish 函数

2.4.7 Softmax 函数

Softmax 是多分类问题的标准输出层激活函数，将 logits 转换为概率分布：

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^C e^{x_j}}$$

关键性质：

- 输出和为 1，可解释为概率
- 指数放大差异：高分更高，低分更低

- 类别之间应该是"竞争"的，一个类别的概率增加意味着其他类别概率降低

与交叉熵损失配合：

- 梯度形式简洁： $\frac{\partial L}{\partial x_i} = \text{softmax}(x_i) - y_i$

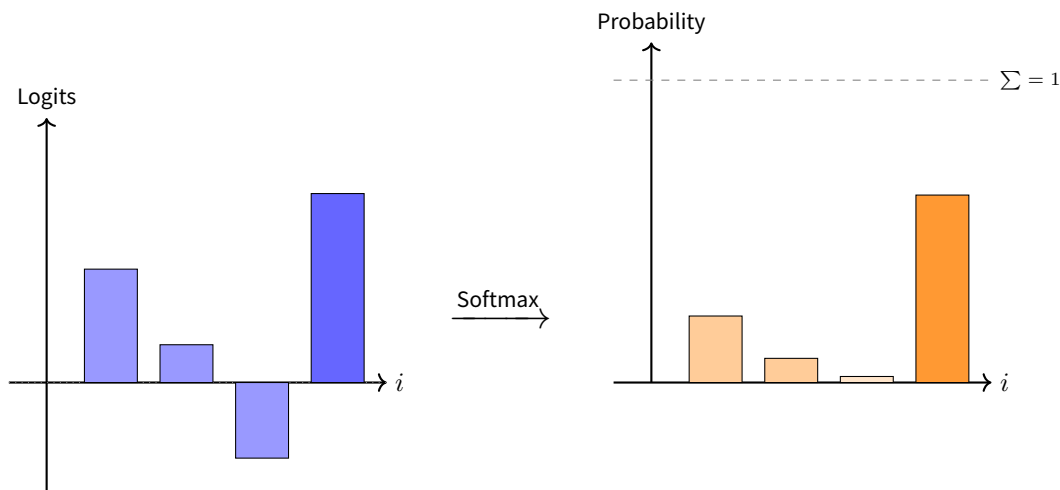


图 14: Softmax 变换示意

2.4.8 激活函数选择指南

表 1: 激活函数选择参考

场景	推荐选择	理由	注意事项
隐藏层（默认）	ReLU	计算快、梯度好	注意学习率，避免死亡神经元
隐藏层（深度网络）	Leaky ReLU / ELU / Swish	避免梯度消失和死亡神经元	ELU 和 Swish 计算稍慢
二分类输出	Sigmoid	输出为概率	不适用于隐藏层
多分类输出	Softmax	输出概率分布	配合交叉熵损失

2.4.9 总结

激活函数是神经网络非线性表达能力的来源：

1. **几何视角：** 每个激活函数定义了输入空间的划分方式
2. **梯度视角：** 激活函数的导数决定了反向传播时的梯度流动
3. **实践视角：**

- ReLU 是隐藏层的默认选择

- Softmax 是多分类输出的标准选择
- 梯度消失问题推动了 ReLU 及其变体的发展

理解激活函数如何引入非线性后，我们将探讨**损失函数**——如何量化模型预测的好坏，将训练转化为可通过梯度下降与优化算法求解的优化问题。

2.5 损失函数

学习任务的**形式**中我们讨论了分类、回归、自回归三类任务的数学形式。不同任务需要不同的方式量化"预测好坏"——这就是损失函数的作用。

2.5.1 损失函数的本质：定义优化目标

损失函数衡量模型预测与真实值之间的差异，将训练转化为**优化问题**。不同的任务类型对应不同的损失函数，塑造了不同的**损失曲面几何**，影响**梯度下降与优化算法**优化的难易程度。**核心作用**：

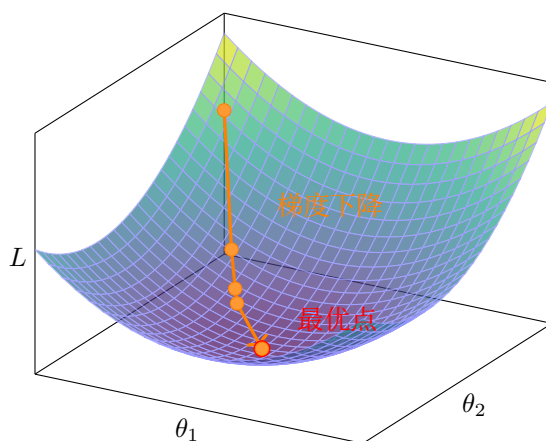


图 15: 损失函数定义误差曲面

- **量化误差**：将预测好坏转化为可计算的数值
 - **指导优化**：通过梯度告诉模型"如何调整"（详见**梯度下降与优化算法**）
 - **塑造曲面**：不同损失函数对应不同的**梯度下降与优化算法**优化难度
-

2.5.2 回归问题的损失函数

均方误差 (MSE)

对大错误"零容忍"

想象你在玩飞镖。如果你投得离靶心很近，误差很小，这没问题。但如果你投偏了很远，平方误差会让这个错误显得**特别大**——就像老师批改作业时，小错误扣 1 分，但大错误要扣 10 分。

为什么平方? 因为平方让大的偏差被放大。误差为 2 时，惩罚是 4；误差为 4 时，惩罚是 16。这种“零容忍大错误”的特性让模型特别关注那些预测很离谱的样本。

代价: 对异常值太敏感。如果数据里有一个极端异常点，MSE 会为了讨好它而牺牲其他大部分样本的准确性。

数学形式

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

特点:

- 对大误差惩罚更重（平方增长）
- 处处可导，便于梯度计算
- 对异常值敏感

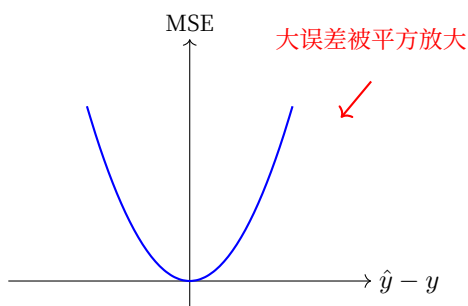


图 16: MSE 损失函数形状

平均绝对误差 (MAE)

一视同仁的“公平裁判”

MSE 像是一个严厉的老师——小错误扣分少，大错误扣分特别多。而 MAE 更像是一个公平的裁判：偏差 1 就扣 1 分，偏差 10 就扣 10 分，一视同仁。

优势: 不会被个别极端值带偏。如果数据里有噪声或异常点（比如房价数据里混入了一个标价 10 亿的豪宅），MAE 不会为了这一个点而牺牲对其他正常房价的预测准确性。

代价: 在误差为 0 的点不可导（像是一个尖角而非平滑的曲线），优化时会稍微麻烦一些。而且，因为它对所有错误一视同仁，对小错误的“惩罚力度”不够，收敛可能比 MSE 慢。

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

特点:

- 对异常值更鲁棒（线性惩罚）
- 在零点不可导（可用次梯度）

Huber 损失 (Smooth L1)

分情况的讨论

Huber 损失 [Hub64] 像一个有智慧的老师，懂得**因材施教**：

- **小错误时** (误差 $\leq \delta$)：像 MSE 一样严厉 —— "这点小错都不能犯？给我好好学！" 平方惩罚让模型对小误差敏感，快速收敛。
- **大错误时** (误差 $> \delta$)：像 MAE 一样理性 —— "已经错这么远了，再骂也没用。" 线性惩罚避免异常值主导整个训练。

参数 δ 的作用：设定一个"红线"。偏差在红线内用平方（严格），超过红线改用线性（宽容）。目标检测任务常用 Huber 损失，因为位置偏差小时要精确定位，偏差大时可能是标注噪声，不必过分纠结。

数学形式

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2 & |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

特点：

- 小误差：MSE（平滑）
- 大误差：MAE（鲁棒）
- 常用于目标检测等任务

2.5.3 分类问题的损失函数

交叉熵损失 (Cross-Entropy)

从编码角度理解

想象你要传递一条消息：“这是一张猫的图片”。如果只有 10 个可能的类别，最直接的编码方式是给每个类别分配一个编号 (0-9)。但这种方式有问题：**如果模型预测"狗"的概率是 80%而"猫"是 20%，我们丢失了预测信心信息。**

更好的方式是使用**概率分布**作为编码。模型输出 [0.1, 0.7, 0.05, ...] 表示对每个类别的信心。

One-Hot 编码：将类别标签转换为向量，对应类别的位置为 1，其余为 0。**为什么叫"One-Hot"？**

- **One**：只有一个位置是 1
- **Hot**：这个位置"激活"（热），其余都是"冷"（0）

类别	One-Hot 编码
猫	$[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]$
狗	$[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]$
鸟	$[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$

10 个类别中只有 1 个位置是 1

图 17: One-Hot 编码示例

交叉熵公式

交叉熵衡量用预测分布 Q 来编码真实分布 P 所需的平均信息量:

$$\text{CE}(P, Q) = - \sum_i P(i) \log Q(i)$$

对于分类问题 (P 是 one-hot 编码), 这简化为:

$$\text{CE} = -\log(\hat{y}_{\text{正确类别}})$$

关键性质:

- 预测越准 (概率接近 1), 损失越低
- 对错误预测惩罚很大 (预测接近 0 时损失 $\rightarrow \infty$)
- 与 softmax 配合, 梯度形式简洁

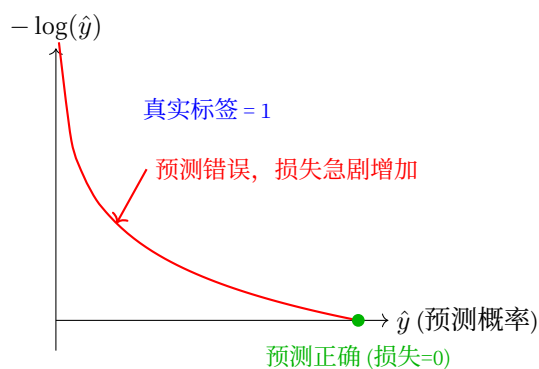


图 18: 交叉熵损失: 惩罚错误预测

为什么不用 MSE 做分类?

- MSE 在预测概率接近 0 或 1 时梯度很小, 学习缓慢
- 交叉熵在错误预测时提供 stronger 的梯度信号

KL 散度 (Kullback-Leibler Divergence)

直觉：额外的编码代价

想象你有一个完美的天气预测模型 P (知道每天实际下雨的概率)，但你只能用另一个模型 Q 来做预测。KL 散度衡量的是：因为使用了不完美的 Q 而不是真实的 P ，你需要多传输多少信息。

$$D_{KL}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

直观例子：

- 如果 $Q = P$ (预测完美)，额外代价为 0
- 如果预测经常出错，代价就高
- 非对称：用 Q 近似 P 的代价 $\neq$ 用 P 近似 Q 的代价

与交叉熵的关系

$$\underbrace{\text{CrossEntropy}(P, Q)}_{\text{总编码长度}} = \underbrace{H(P)}_{\text{最优编码长度}} + \underbrace{D_{KL}(P\|Q)}_{\text{额外代价}}$$

其中 $H(P)$ 是分布 P 的熵 (信息论中最小可能编码长度)。因为真实分布 P 是固定的， $H(P)$ 是常数。因此：

最小化交叉熵 = 最小化 KL 散度 = 让预测分布尽可能接近真实分布

应用场景

- 变分自编码器 (VAE)：约束潜在变量 z 的分布接近标准正态分布
- 知识蒸馏：让小模型学习大模型的"软预测" (概率分布)，而非硬标签
- 强化学习：限制策略更新幅度，防止模型突变

2.5.4 损失函数选择指南

表 2: 损失函数选择参考

问题类型	推荐损失	优势	注意事项
回归问题	MSE / MAE	MSE 光滑可导，MAE 鲁棒	数据有异常值时选 MAE
回归 (需鲁棒性)	Huber 损失	结合两者优点	需要调超参数 δ
二分类	二元交叉熵	梯度强、收敛快	输出需 sigmoid 激活
多分类	交叉熵	配合 softmax 效果最佳	PyTorch 的 CE 包含 softmax
分布匹配	KL 散度	衡量分布差异	注意方向性 $P\ Q$

2.5.5 PyTorch 实现示例

```
import torch
import torch.nn as nn

# 回归损失
mse_loss = nn.MSELoss()
mae_loss = nn.L1Loss() # MAE
huber_loss = nn.SmoothL1Loss(beta=1.0)

# 分类损失
ce_loss = nn.CrossEntropyLoss() # 包含 softmax
bce_loss = nn.BCELoss() # 二分类, 需手动 sigmoid
kl_loss = nn.KLDivLoss(reduction='batchmean')

# 使用示例
predictions = model(inputs)
loss = ce_loss(predictions, targets) # 多分类
loss.backward() # 计算梯度
```

2.5.6 总结

损失函数定义了"什么是好的预测", 将训练转化为优化问题:

1. **回归问题**: MSE 光滑易优化, MAE 鲁棒抗异常值
2. **分类问题**: 交叉熵提供强梯度, 是深度学习的主流选择
3. **分布匹配**: KL 散度衡量概率分布差异, VAE 和知识蒸馏的核心工具

理解损失函数后, 我们将探讨[反向传播算法](#)——如何高效计算梯度, 完成误差的"信用分配"。

2.6 反向传播算法

2.6.1 反向传播的本质: 信用分配

神经网络训练时, 我们根据最终损失来调整参数。核心问题是: **损失是由很多参数共同造成的, 每个参数该"背多少锅"?**

反向传播 (Backpropagation) [RHW86] 就是解决这个 **"信用分配问题"** 的高效算法。

类比：团队项目的责任分摊

想象一个团队项目失败了（损失很大），需要找出每个人的责任：

- **前向传播**：项目执行过程，每个人完成自己的任务
- **反向传播**：从失败结果倒推，计算每个人对失败的责任（梯度）
- **链式法则**：如果 A 的工作影响了 B，B 的责任要按贡献度传递给 A

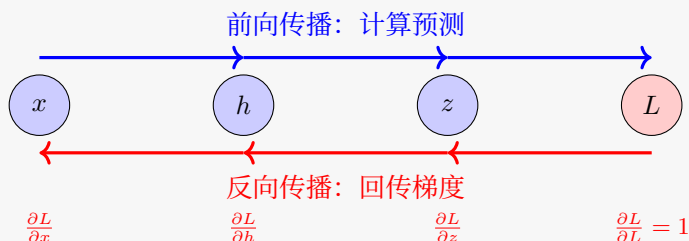


图 19: 反向传播：梯度从输出流回输入

2.6.2 链式法则：梯度的传递机制

反向传播算法的核心是**链式法则**（Chain Rule），这是微积分中复合函数求导的基本法则。想象你改变了输入 x 一点点，这个变化会层层传递，最终影响损失 L 。链式法则告诉我们：**总的影响是各层影响的乘积**。

回顾 [计算图](#) 中讨论的**计算图**概念：神经网络可以被看作是一个由许多简单操作（加法、乘法、激活函数等）组成的图。反向传播正是沿着这个图的边，将梯度从输出传递回输入。

对于复合函数 $z = f(g(x))$ ，如果 x 变化了 Δx ：

- 首先影响 g ： $\Delta g \approx \frac{\partial g}{\partial x} \Delta x$
- 然后影响 z ： $\Delta z \approx \frac{\partial z}{\partial g} \Delta g$
- 综合起来： $\Delta z \approx \frac{\partial z}{\partial g} \cdot \frac{\partial g}{\partial x} \cdot \Delta x$

所以：

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial g} \cdot \frac{\partial g}{\partial x}$$

直觉：影响会层层叠加——就像多米诺骨牌，第一块倒下的角度（ $\frac{\partial g}{\partial x}$ ）乘以第二块倒下的角度（ $\frac{\partial z}{\partial g}$ ），就是最后一块倒下的总角度。

从标量到向量：Jacobian 矩阵

上述链式法则适用于**标量函数**。但在神经网络中，每一层的输入和输出都是**向量**（几百甚至几千个神经元的值），而不是单个数字。所以我们需要一个推广版的导数——**Jacobian 矩阵**。

从标量导数到 Jacobian

概念	数学表示	输入维度	输出维度	矩阵形状
标量导数	$\frac{df}{dx}$	1	1	1×1
梯度	$\nabla f = [\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots]$	N	1	$1 \times N$ 行向量
Jacobian 矩阵	$\frac{\partial \mathbf{h}}{\partial \mathbf{x}}$	N	M	$M \times N$ 矩阵

每个元素 (i, j) 的意思：“第 j 个输入变一点点，第 i 个输出变多少”。

形状：(输出维度) $\times$ (输入维度)。

定义：Jacobian 矩阵

对于向量值函数 $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，其 Jacobian 矩阵 $\mathbf{J} \in \mathbb{R}^{m \times n}$ 定义为：

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

每个元素 $J_{ij} = \frac{\partial f_i}{\partial x_j}$ 的含义是：第 j 个输入变化一点点，第 i 个输出变化多少。

维度规则：Jacobian 的形状是 (输出维度) $\times$ (输入维度)。

神经网络中的例子：

一个全连接层 $\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{b}$ ：

- 输入 $\mathbf{x} \in \mathbb{R}^{100}$ (100 维)
- 输出 $\mathbf{h} \in \mathbb{R}^{64}$ (64 维)
- 权重 $\mathbf{W} \in \mathbb{R}^{64 \times 100}$

其 Jacobian $\frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \mathbf{W}$ ，正好是一个 64×100 的矩阵。

多层网络的 Jacobian 连乘与梯度消失

对于深层网络，损失 L 对第 l 层参数的梯度等于每一层 Jacobian 的连乘：

$$\frac{\partial L}{\partial \mathbf{h}_l} = \underbrace{\frac{\partial L}{\partial \mathbf{h}_L}}_{\text{输出层梯度}} \cdot \underbrace{\frac{\partial \mathbf{h}_L}{\partial \mathbf{h}_{L-1}} \cdot \frac{\partial \mathbf{h}_{L-1}}{\partial \mathbf{h}_{L-2}} \dots \frac{\partial \mathbf{h}_{l+1}}{\partial \mathbf{h}_l}}_{L-1 \text{ 个 Jacobian 矩阵相乘}}$$

关键洞察：Jacobian 矩阵的连乘是梯度消失/爆炸的数学根源。

如何理解 Jacobian 对梯度的影响？

想象 Jacobian 矩阵是一个"放大器"——它把输入的梯度变换成输出的梯度。我们用 γ 表示这个**放大倍数**²:

- 如果 $\gamma < 1$: 梯度被"缩小"了
- 如果 $\gamma = 1$: 梯度大小不变
- 如果 $\gamma > 1$: 梯度被"放大"了

对于简单的标量 (单个数字), γ 就是导数的绝对值 $\left| \frac{\partial h_{k+1}}{\partial h_k} \right|$ 。

对于向量 (多个数字组成的层), 这个放大倍数可以用**行列式** (determinant) 来理解——行列式的绝对值告诉我们: 经过这个线性变换后, 空间的体积被放大或缩小了多少倍。虽然 Jacobian 不一定是方阵, 但核心直觉是一样的: 每层都会对梯度进行"缩放"。

假设每层 Jacobian 的放大倍数为 γ , 经过 n 层连乘后, 梯度的总体放大倍数为 γ^n :

层数 n	$\gamma = 0.9$ (消失)	$\gamma = 1.0$ (稳定)	$\gamma = 1.1$ (爆炸)
10 层	$0.9^{10} \approx 0.35$	1.0	$1.1^{10} \approx 2.6$
50 层	$0.9^{50} \approx 0.005$	1.0	$1.1^{50} \approx 117$
100 层	$0.9^{100} \approx 2.6 \times 10^{-5}$	1.0	$1.1^{100} \approx 13,780$

梯度消失 ($\gamma < 1$):

- 当 $\gamma < 1$ 时, 梯度呈**指数级衰减**
- 50 层后衰减到 0.5%, 几乎收不到梯度信号
- 浅层参数无法更新, 网络退化为"浅层网络"

梯度爆炸 ($\gamma > 1$):

- 当 $\gamma > 1$ 时, 梯度呈**指数级增长**
- 100 层后放大 13,780 倍, 数值溢出
- 参数更新剧烈, 损失震荡发散甚至变成 NaN

实际神经网络中的 γ 值:

² 这里是简化的版本, 实际应为**最大奇异值** (maximum singular value)。对向量 (多层梯度) 而言, 局部放大倍数由 Jacobian 矩阵 $\frac{\partial h_{k+1}}{\partial h_k}$ 的最大奇异值给出, 它决定了梯度模长经过这一层映射后被放大或缩小的最大倍数。

激活函数/技术	典型 γ 值	可训练深度	原因
Sigmoid	≤ 0.25	< 10 层	最大梯度在 0 处也只有 0.25
Tanh	≤ 1.0	< 15 层	比 Sigmoid 稍好，但仍会饱和
ReLU	≈ 0.5	20-30 层	负区间梯度为 0，正区间为 1（见 激活函数 ）
BatchNorm	≈ 1.0	50+ 层	标准化梯度分布，稳定数值
跳跃连接	≈ 1.0	100+ 层	提供梯度捷径，减少连乘长度（见 ResNet：高原反应补给站 ）

关键结论：

- 只有当 $\gamma \approx 1$ 时，深层网络才能稳定训练
- 从 Sigmoid $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ ResNet，每次突破都是让 γ 更接近 1

这就是为什么深层网络训练困难，以及残差连接和跳跃连接能缓解这个问题的数学原理。

与后面内容的衔接

下面“常见问题”部分会从更直观的角度解释梯度消失和爆炸，包括：

- 具体场景（什么时候会发生）
- 后果（对训练的影响）
- 解决方案（如何应对）

两部分内容互补：这里侧重**数学原理**（Jacobian 连乘），后面侧重**实践指导**。

示例：简单计算图

考虑 $f(x, y) = (x + y) \times y$ ，设 $x = 2, y = 3$ ：

前向传播：

- $a = x + y = 2 + 3 = 5$
- $f = a \times y = 5 \times 3 = 15$

反向传播（假设损失对 f 的梯度为 1）：

- $\frac{\partial f}{\partial f} = 1$
- $\frac{\partial f}{\partial a} = y = 3$ （乘法规则）
- $\frac{\partial f}{\partial y} = a = 5$ （乘法规则）
- $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial x} = 3 \times 1 = 3$ （链式法则）
- $\frac{\partial f}{\partial y}$ 有两条路径： $= 5 + 3 = 8$

2.6.3 PyTorch 中的自动微分

PyTorch 通过 autograd 自动完成反向传播：

```
import torch

# 创建张量，启用梯度跟踪
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)

# 前向传播
a = x + y      # a = 5
f = a * y      # f = 15

# 反向传播
f.backward()

print(f"∂f/∂x = {x.grad}") # 输出: 3.0
print(f"∂f/∂y = {y.grad}") # 输出: 8.0
```

关键机制：

- `requires_grad=True`：告诉 PyTorch 需要跟踪这个张量的计算图
- PyTorch 自动构建动态计算图（参见计算图）
- `.backward()`：自动执行反向传播算法计算所有叶节点的梯度

2.6.4 神经网络中的反向传播

对于多层神经网络，反向传播从输出层逐层回传：流程：

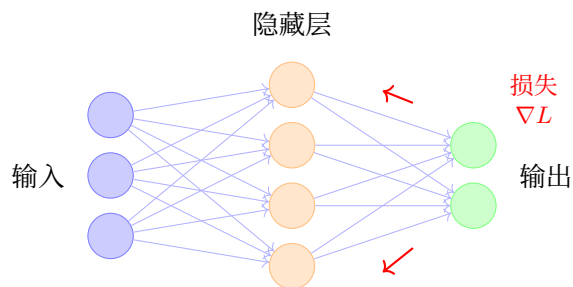


图 20: 神经网络反向传播示意

1. **前向传播**：数据从输入层 → 隐藏层 → 输出层，计算预测和损失
2. **反向传播**：梯度从输出层 ← 隐藏层 ← 输入层，计算每个参数的梯度

3. **参数更新**：使用梯度下降更新权重

2.6.5 常见操作的梯度

反向传播算法的实现依赖于定义每个基本操作的梯度规则。下面是神经网络中常用操作的梯度：

操作	前向	反向梯度
加法 $z = x + y$	$z = x + y$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}, \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}$
乘法 $z = x \times y$	$z = x \times y$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot y, \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot x$
ReLU $z = \max(0, x)$	$z = \max(0, x)$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}$ (if $x > 0$), else 0
Sigmoid $z = \sigma(x)$	$z = \sigma(x)$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot z(1 - z)$

更多激活函数的梯度定义，参见 [激活函数](#)。

2.6.6 反向传播的优势

1. 计算效率

反向传播算法的时间复杂度是 $O(n)$ ，其中 n 是计算图中边的数量。相比之下，数值差分需要 $O(n \times p)$ ，其中 p 是参数数量。

关键：通过复用中间计算结果，避免重复求导。

2. 模块化设计

每个操作只需要定义：

- 前向：如何计算输出
- 反向：如何计算梯度

这使得添加新操作变得简单。

2.6.7 常见问题

梯度消失 (Vanishing Gradient)

想象一个 100 层的神经网络。当你训练时，发现靠近输出层的参数更新正常，但靠近输入层的参数几乎不动——就像前面几十层“冻住”了一样。这就是**梯度消失**。

为什么会发生？回顾链式法则：梯度是**各层影响的乘积**。如果每一层的梯度都小于 1（比如 Sigmoid 在饱和区的梯度只有 0.01），100 层连乘后：

$$0.01 \times 0.01 \times \dots \times 0.01 \quad (100\text{次}) = 10^{-200}$$

这个数小到计算机都当成 0 处理了！具体场景是：使用 Sigmoid/Tanh 激活函数时，输入值很大或很小会让函数进入“饱和区”，梯度接近 0；网络层数越深，问题越严重。

后果：前面的层学不到东西，模型退化成“浅层网络”——只有后面几层在学习。对于图像数据，前面的层本应学习边缘、纹理等基础特征，但这些特征学不到。

解决方案：

- **ReLU 激活函数**：正区间的梯度恒为 1，不会衰减（见 [激活函数](#)）
 - **批归一化 (BatchNorm)**：控制每层的数值范围，避免进入饱和区
 - **残差连接 (ResNet)**：让梯度有“捷径”可以直达前面层（见 [ResNet：高原反应补给站](#) 中的数学推导）
 - **跳跃连接 (U-Net)**：在编码器-解码器架构中保留梯度路径（见 [U 形架构详解](#)）
-

梯度爆炸 (Exploding Gradient)

训练过程中，损失突然变成 NaN（不是数字），或者参数变得极其巨大（比如从 0.1 变成 1000000），模型完全失控。这就是**梯度爆炸**。

为什么会发生？和梯度消失相反，如果每层的梯度都大于 1（比如权重初始化过大，梯度为 2），100 层连乘后：

$$2 \times 2 \times \dots \times 2 \quad (100\text{次}) = 2^{100} \approx 10^{30}$$

这是一个天文数字！参数会被更新得极其剧烈，完全偏离最优解。常见于权重初始化不当，或循环神经网络（RNN，详见 [RNN 的两个核心问题](#)）处理长序列时的梯度爆炸。

后果：参数更新过大跳出最优解区域，损失函数震荡发散，甚至数值溢出（NaN）导致训练彻底失败。

解决方案：

- **梯度裁剪**：设定梯度上限，超过就截断
- **权重正则化 (L2 正则)**：惩罚过大的权重
- **更好的初始化**：如 Xavier 初始化、He 初始化，控制初始梯度规模
- **更小的学习率**：让参数更新更温和

2.6.8 总结

反向传播是深度学习的核心算法：

1. **信用分配**：将最终损失“分摊”给每个参数
2. **链式法则**：梯度从输出层逐层回传（基于 [计算图](#) 的结构）
3. **高效计算**：复用中间结果，时间与网络规模线性相关

反向传播计算出的梯度用于更新模型参数。下一节 [梯度下降与优化算法](#) 将详细介绍如何利用这些梯度进行参数优化。

关于反向传播在实际神经网络训练中的应用，参见 [训练基础：登山纪律](#)。

2.7 梯度下降与优化算法

2.7.1 沿着最陡方向下山

想象你在山上迷路了，想要下到山谷：

- **当前位置**：参数当前值 θ_t
- **最陡下降方向**：负梯度方向（你脚下最陡的下坡方向）
- **步长**：学习率 η （你每一步迈多大）
- **目标**：到达山谷最低点（损失 L 最小）

神经网络训练转化为优化问题后，我们需要找到损失函数的**最小值点**。梯度下降（Gradient Descent）就是解决这个问题的迭代算法。

核心思想：在当前位置，沿着**梯度下降方向**（损失减小最快的方向）迈出一小步。

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta)$$

其中 η 是**学习率**（步长）， $\nabla_{\theta} J(\theta)$ 是损失函数对参数的梯度。

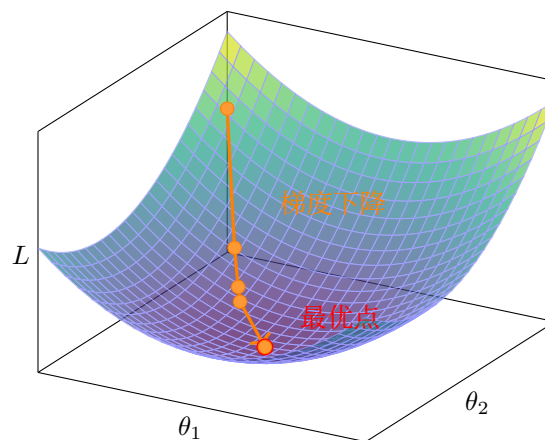


图 21: 梯度下降：从初始点逐步逼近最小值

学习率的重要性

学习率的选择至关重要：

学习率	效果	类比
太小	收敛极慢	婴儿学步
合适	快速收敛	正常行走
太大	震荡/发散	大跳, 可能跳过山谷

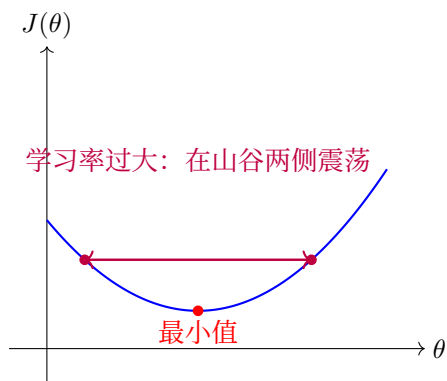


图 22: 学习率过大导致震荡

2.7.2 梯度下降的变体

在梯度下降与优化算法中，我们根据反向传播算法计算的梯度更新参数。根据使用多少数据计算梯度，有三种变体：

批量梯度下降（Batch GD）

使用**全部数据**计算梯度：

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} J(\theta; x_i, y_i)$$

特点：

- 梯度估计准确，收敛稳定
- 每次迭代计算量大，不适合大数据集

随机梯度下降（SGD）

每次使用一个**样本**计算梯度：

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta; x_i, y_i)$$

特点：

- 计算快，适合在线学习

- 梯度有噪声，收敛不稳定
- 噪声可能帮助跳出局部最优

小批量梯度下降 (Mini-batch GD)

使用一小批样本（通常 32-256 个）计算梯度：

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} J(\theta; x_i, y_i)$$

特点：

- 平衡计算效率和稳定性
- 可以利用 GPU 并行计算
- 深度学习中最常用

2.7.3 高级优化算法

Loss Landscape：损失函数塑造的优化地形

Loss Landscape（损失景观）描述了损失函数在参数空间中的形状。想象一个高维的山地地形：山谷代表损失低的区域，山峰代表损失高的区域。

在深度学习中，这个景观通常是**非凸**的，包含：

- **局部最优**：局部山谷（浅坑）
- **全局最优**：最深的山谷（最优解）
- **鞍点**：马鞍形状的点（高维空间中非常常见）

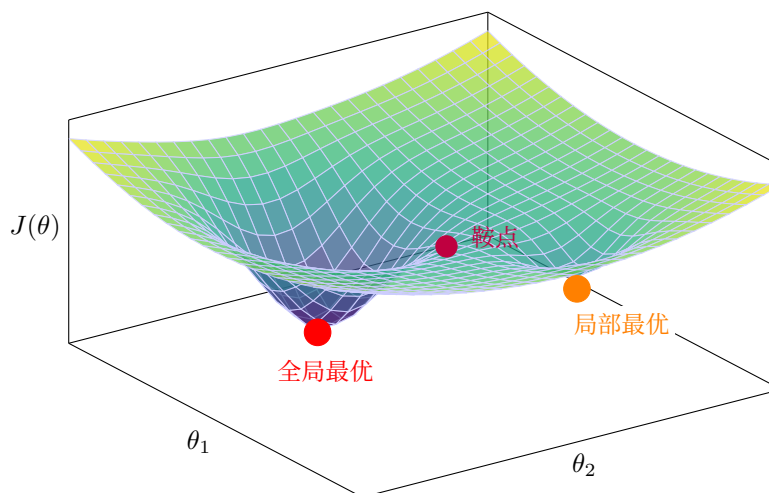


图 23: Loss Landscape：局部最优 vs 全局最优

关键观察：

- 全局最优比局部最优**明显更深**（红色点远低于橙色点）
- 两个最优之间由**鞍点**（紫色）分隔
- 优化算法需要“翻过”鞍点才能从局部最优到达全局最优

鞍点的详细展示：

鞍点像一个马鞍 —— 沿一个方向是极小值，沿另一个方向是极大值：**深度学习的特殊情况：**

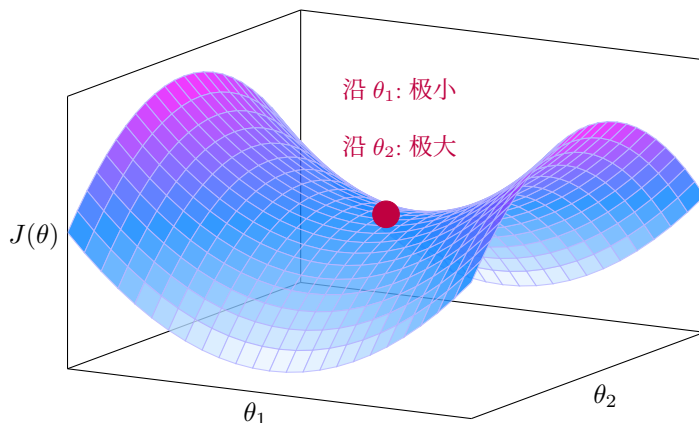


图 24: 鞍点：马鞍形状

- 高维空间中，**鞍点比局部最优更常见**
- 随机梯度下降的噪声有助于跳出局部最优和鞍点
- 现代网络通常能找到足够好的解，即使不是全局最优

动量法：积累速度翻越障碍

直觉：想象滚雪球下山，球会积累动量，越滚越快，同时可以滚过小的坑洼。在 loss landscape 中，动量帮助我们冲过鞍点和平坦区域。

原理：引入速度变量，累积历史梯度：

$$v_{t+1} = \gamma v_t + \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta \cdot v_{t+1}$$

逐步理解动量法：**第一步：理解速度变量 v_t**

在基础 SGD 中，我们直接用梯度更新参数： $\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta_t)$

这相当于每走一步都**完全重新决定方向**，没有记忆之前走过的路径。

动量法引入速度变量 v_t ，相当于给优化过程添加了"记忆"——记住之前累积的运动趋势。

第二步：解读速度更新公式

$$v_{t+1} = \gamma v_t + \nabla_{\theta} J(\theta_t)$$

这行公式的含义是：**新速度 = 保留的旧速度 + 当前梯度**

公式详解：

符号	含义	维度	直觉理解
v_{t+1}	第 $t+1$ 步的速度	与参数相同	下一步要迈的"步伐向量"
γ	动量系数	标量 (0~1)	惯性大小，通常 0.9（保留 90% 旧速度）
v_t	第 t 步的速度	与参数相同	之前累积的运动趋势
$\nabla_{\theta} J(\theta_t)$	当前梯度	与参数相同	当前位置最陡的下降方向

核心洞察：

- $\gamma = 0$ （无动量）： $v_{t+1} = \nabla_{\theta} J$ ，退化成基础 SGD
- $\gamma = 0.9$ （有动量）：新速度 = 90% 旧速度 + 10% 新梯度

第三步：解读参数更新公式

$$\theta_{t+1} = \theta_t - \eta \cdot v_{t+1}$$

与 SGD 的区别：

- **SGD**: $\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J$ （只用当前梯度）
- **Momentum**: $\theta_{t+1} = \theta_t - \eta \cdot v_{t+1}$ （用累积的速度）

为什么这样能减少震荡？

想象你在峡谷中行走：

- **SGD**：每次只看脚下，向左跨一步，发现右边更低，又向右跨一步，在峡谷两侧来回震荡
- **Momentum**：你有了惯性，向左走时积累了向左的速度，即使右边略低，惯性也会带你继续向左，直到左边明显比右边高，才会慢慢转向

展开公式看本质：

如果我们展开 v_t 的递归定义：

$$\begin{aligned}v_t &= \gamma v_{t-1} + g_t \\&= \gamma(\gamma v_{t-2} + g_{t-1}) + g_t \\&= \gamma^2 v_{t-2} + \gamma g_{t-1} + g_t \\&= \dots \\&= \sum_{i=0}^t \gamma^{t-i} g_i\end{aligned}$$

其中 $g_i = \nabla_{\theta} J(\theta_i)$ 是第 i 步的梯度。

关键发现：当前速度是**历史梯度的加权平均**，越早的梯度权重越小（按 γ^k 指数衰减）。

类比：

- 基础 SGD：每走一步都重新决定方向，容易在峡谷两侧来回震荡
- Momentum：像滚下山坡的球，有惯性，同方向会加速，反方向会减速，自然减少震荡

效果：

- 加速收敛（尤其在梯度方向一致时）
- 减少震荡（梯度方向变化时动量抵消）
- 帮助跳出局部最优

Adam（自适应矩估计）

Adam [KB15] 是目前最常用的优化算法之一，结合了 Momentum 和 RMSprop 的优点。

直觉：为每个参数单独调整学习率。频繁更新的参数用较小学习率，稀疏更新的用较大学习率。

原理：结合一阶矩估计（动量）和二阶矩估计（自适应学习率）：

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\\hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\\theta_{t+1} &= \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}\end{aligned}$$

逐步理解 Adam：

第一步：一阶矩估计（momentum 部分）

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

这与动量法类似，计算梯度的指数移动平均：

符号	含义	维度	直觉理解
m_t	第 t 步的一阶矩 (均值)	与参数相同	梯度的"平均趋势", 平滑后的梯度方向
β_1	一阶矩衰减率	标量 (0~1)	通常是 0.9, 保留 90% 历史信息
m_{t-1}	上一步的一阶矩	与参数相同	之前累积的梯度趋势
g_t	当前梯度	与参数相同	当前计算出的梯度值
$(1 - \beta_1)$	新梯度权重	标量	新梯度占 10% 的权重

与动量法的区别:

- **动量法**: $v_{t+1} = \gamma v_t + \nabla_{\theta} J$ (新梯度权重为 1)
- **Adam**: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ (新梯度权重为 $1 - \beta_1$)

Adam 的形式使 m_t 更接近真正的**梯度均值** (数学期望)。

展开看本质:

$$m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i$$

这是历史梯度的**指数加权平均**, 越近的梯度权重越大。

第二步: 二阶矩估计 (自适应学习率部分)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

注意这里的 g_t^2 是**逐元素平方** (element-wise square):

$$g_t^2 = (g_t^{(1)})^2, (g_t^{(2)})^2, \dots, (g_t^{(d)})^2$$

符号	含义	维度	直觉理解
v_t	第 t 步的二阶矩 (未中心化的方差)	与参数相同	梯度"变化幅度"的历史平均
β_2	二阶矩衰减率	标量 (0~1)	通常是 0.999, 比 β_1 更大
g_t^2	梯度逐元素平方	与参数相同	梯度的幅度信息

核心洞察: v_t 记录了每个参数梯度的"活跃程度":

- 梯度经常很大 $\rightarrow v_t$ 很大
- 梯度经常很小 $\rightarrow v_t$ 很小

为什么用平方?

平方消除了符号 (正负), 只保留幅度信息。我们想知道的是梯度"有多大", 而不是"朝哪个方向"。

第三步：偏差修正 (Bias Correction)

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

为什么需要修正？

问题：初始时 $m_0 = 0, v_0 = 0$

前几步的偏差：

- 第 1 步： $m_1 = (1 - \beta_1)g_1$ ，比真实均值小了约 $(1 - \beta_1)$ 倍
- 第 2 步： $m_2 = \beta_1(1 - \beta_1)g_1 + (1 - \beta_1)g_2$ ，仍有偏差

修正原理：

- 如果 m_t 估计的是 $E[g]$ ，那 $E[m_t] = (1 - \beta_1^t)E[g]$
- 除以 $(1 - \beta_1^t)$ 就得到了无偏估计

符号	含义	计算方式
$\hat{m}_t$	修正后的一阶矩	$m_t / (1 - \beta_1^t)$
$\hat{v}_t$	修正后的二阶矩	$v_t / (1 - \beta_2^t)$
t	当前步数	训练迭代次数

修正效果：

- t 很小（初期）： $1 - \beta_1^t$ 很小，除数很小，放大效果明显
- t 很大（后期）： $\beta_1^t \approx 0$ ， $1 - \beta_1^t \approx 1$ ，几乎不修正

举例 ($\beta_1 = 0.9$)：

- $t = 1$ ：修正系数 $1/(1 - 0.9) = 10$ ，放大 10 倍
- $t = 10$ ：修正系数 $1/(1 - 0.9^{10}) \approx 1.5$ ，放大 1.5 倍
- $t = 100$ ：修正系数 ≈ 1.0 ，几乎不修正

第四步：参数更新 (自适应学习率的精髓)

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

逐元素解读：

对于第 i 个参数 $\theta^{(i)}$ ：

$$\theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta \cdot \frac{\hat{m}_t^{(i)}}{\sqrt{\hat{v}_t^{(i)}} + \epsilon}$$

符号	含义	作用
$\hat{m}_t^{(i)}$	第 i 个参数的平滑梯度	决定更新方向
$\sqrt{\hat{v}_t^{(i)}}$	第 i 个参数的梯度 RMS	决定步长缩放
ϵ	数值稳定性常数	防止除以 0, 通常 10^{-8}
η	全局学习率	整体步长控制

核心洞察 - 自适应学习率机制:

假设有两个参数:

参数 A (梯度波动大):

- 历史梯度: [10, -8, 12, -10, 9] (大起大落)
- $\hat{m}_t \approx 2.6$ (平均梯度)
- $\hat{v}_t \approx 100$ (平均平方梯度)
- $\sqrt{\hat{v}_t} \approx 10$
- 更新步长: $\eta \cdot 2.6/10 = 0.26\eta$ (步长被缩小)

参数 B (梯度波动小):

- 历史梯度: [0.01, -0.02, 0.015, -0.01, 0.012] (小幅波动)
- $\hat{m}_t \approx 0.009$ (平均梯度)
- $\hat{v}_t \approx 0.0002$ (平均平方梯度)
- $\sqrt{\hat{v}_t} \approx 0.014$
- 更新步长: $\eta \cdot 0.009/0.014 = 0.64\eta$ (步长相对较大)

结果:

- 梯度波动大的参数 → 步长自动变小 (谨慎更新)
- 梯度波动小的参数 → 步长相对较大 (大胆更新)

为什么用 $\sqrt{\hat{v}_t}$ 而不是 $\hat{v}_t$?

- 梯度平方 g_t^2 的量纲是原梯度的平方
- 除以 $\sqrt{\hat{v}_t}$ (RMS, 均方根) 可以抵消量纲, 使更新步长与原梯度同量纲
- 类比: 标准差 $\sigma = \sqrt{\text{方差}}$, 与原始数据同量纲

超参数总结:

参数	推荐值	作用
β_1	0.9	一阶矩衰减率，控制 momentum 强度
β_2	0.999	二阶矩衰减率，控制自适应灵敏度
ϵ	10^{-8}	数值稳定性
η	0.001 (默认)	全局学习率

特点：

- 结合动量和自适应学习率
- 对超参数不敏感，默认首选
- 适合大多数深度学习任务

表 3: 优化算法选择建议

算法	适用场景	特点
SGD + Momentum	图像分类、大模型	泛化性能好，需调学习率
Adam	默认首选、NLP、推荐系统	自适应、收敛快、易用
AdamW	Transformer、大模型	改进权重衰减，效果更好

2.7.4 学习率调度：为什么需要调整学习率？

核心问题

训练初期，参数远离最优，需要大学习率快速接近目标。训练后期，参数接近最优，需要小学习率精细调整，避免在最优解附近震荡。

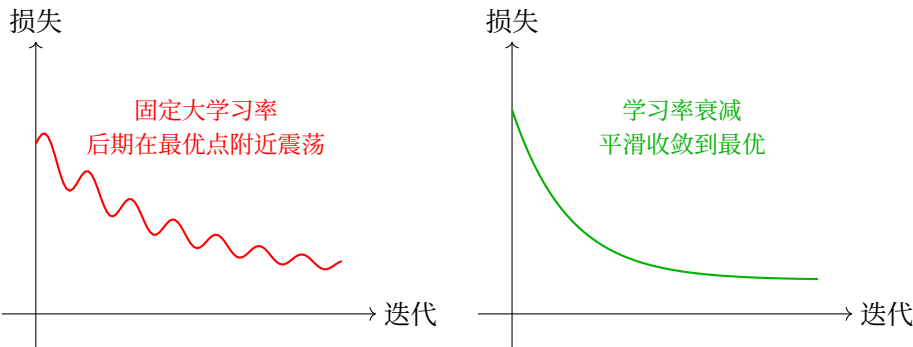


图 25: 固定学习率 vs 学习率调度

常见学习率调度策略

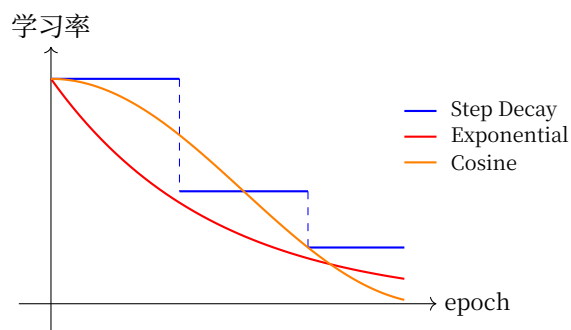


图 26: 学习率调度策略对比

1. 步长衰减 (Step Decay)

策略: 每 N 个 epoch, 学习率乘以衰减系数 (如 0.1)

适用场景:

- 传统图像分类任务 (ImageNet 训练标配)
- 当验证集损失不再下降时手动降低学习率

```
# 每 30 个 epoch, 学习率乘以 0.1
```

```
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.1)
```

特点: 阶梯式下降, 简单有效, 但需要预设衰减时机。

2. 指数衰减 (Exponential Decay)

策略: 每个 epoch 学习率按指数衰减

$$\eta_t = \eta_0 \cdot \gamma^t$$

适用场景:

- 需要平滑连续衰减
- 训练轮数较多时

```
# 每个 epoch 学习率乘以 0.95
```

```
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)
```

3. 余弦退火 (Cosine Annealing)

策略：学习率按余弦函数从初始值降到接近 0

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t}{T_{max}}\pi))$$

适用场景：

- 现代 Transformer 模型 (BERT、GPT 等)
- 配合 Warmup 使用效果更佳

```
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)
```

4. 预热 (Warmup)

问题：训练初期参数随机初始化，梯度可能很大且不稳定，大学习率会导致震荡。

策略：从很小的学习率开始，线性增加到目标学习率。

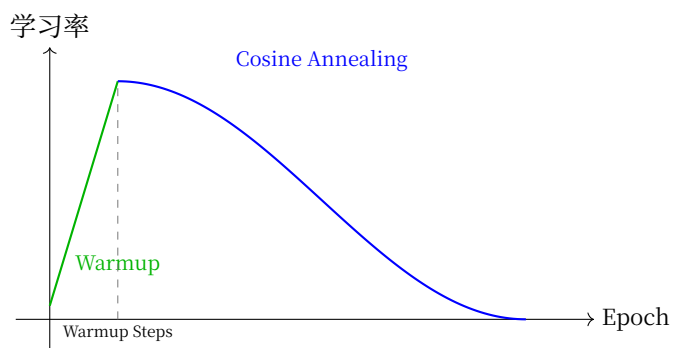


图 27: Warmup + Cosine Annealing

```
# 组合使用: Warmup + Cosine Annealing
scheduler1 = optim.lr_scheduler.LinearLR(
    optimizer,
    start_factor=0.01,      # 从 1% 的目标学习率开始
    end_factor=1.0,
    total_iters=5           # 前 5 个 epoch 预热
)
scheduler2 = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=95                # 剩余 95 个 epoch 余弦退火
)
scheduler = optim.lr_scheduler.SequentialLR(
    optimizer,
```

(续下页)

(接上页)

```

schedulers=[scheduler1, scheduler2],
milestones=[5]
)

```

选择建议

策略	推荐场景	优点	缺点
Step Decay	图像分类、CNN	简单有效	需预设衰减时机
Exponential	长周期训练	平滑连续	衰减过快
Cosine	Transformer、NLP	效果通常最好	相对复杂
Warmup + Cosine	大模型训练	稳定+效果好	需调两个参数

2.7.5 PyTorch 实践

· 基本训练循环

```

import torch
import torch.nn as nn
import torch.optim as optim

# 定义模型、损失函数、优化器
model = MyModel()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练循环
for epoch in range(num_epochs):
    for data, target in dataloader:
        # 前向传播
        output = model(data)
        loss = criterion(output, target)

        # 反向传播
        optimizer.zero_grad() # 清空梯度
        loss.backward()        # 计算梯度
        optimizer.step()       # 更新参数

```

· 学习率调度

```
# 组合调度: 预热 + 余弦退火
scheduler1 = optim.lr_scheduler.LinearLR(optimizer, start_factor=0.1, total_iters=5)
scheduler2 = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=45)
scheduler = optim.lr_scheduler.SequentialLR(optimizer, [scheduler1, scheduler2],
→ milestones=[5])

for epoch in range(num_epochs):
    train(...)
    scheduler.step() # 更新学习率
```

2.7.6 训练技巧

1. 梯度裁剪

防止梯度爆炸:

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

2. 批归一化 (BatchNorm)

稳定训练、加速收敛:

```
self.bn = nn.BatchNorm1d(num_features)
```

3. 早停 (Early Stopping)

验证集损失不再下降时停止训练, 防止过拟合。

2.7.7 总结

梯度下降是深度学习训练的基石:

1. **基本原理**: 沿反向传播算法计算的负梯度方向迭代更新参数
2. **学习率**: 最关键的超参数, 过大震荡, 过小收敛慢
3. **Mini-batch**: 平衡效率和稳定性, 深度学习标配
4. **Adam**: 默认首选优化器, 自适应、易用
5. **学习率调度**: 训练过程中调整学习率能提升效果

理解损失函数和优化算法后，你就掌握了深度学习训练的全流程。接下来我们将通过总结与展望回顾本章内容，然后进入实践章节。

2.8 总结与展望

2.8.1 本章核心回顾

本章我们建立了深度学习的数学基础，形成了完整的训练流程：

1. 计算图：描述"数据如何流动"

- 直觉：像工厂流水线，数据从输入到输出的处理流程
- 作用：让复杂计算可视化，为反向传播算法提供结构

2. 学习任务的形式：定义"找什么规律"

- 直觉：分类找边界、回归找映射、自回归找序列依赖
- 作用：不同任务类型对应不同的输出形式和损失函数

3. 激活函数：引入非线性表达能力

- 直觉：在特征空间中划分决策边界，将线性不可分数据"折叠"成可分
- 关键：ReLU 是隐藏层默认选择，解决梯度消失问题

4. 损失函数：定义"什么是好的预测"

- 直觉：衡量预测与真实的差距，塑造梯度下降与优化算法的优化地形
- 关键：交叉熵为分类提供强梯度，MSE 适合回归

5. 反向传播算法："信用分配"机制

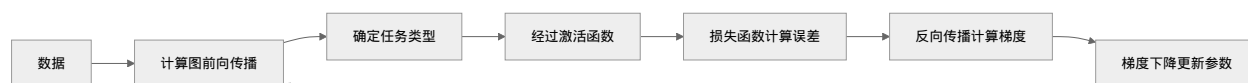
- 直觉：项目失败后倒推责任，按贡献度分配
- 核心：链式法则让梯度在计算图中高效回传

6. 梯度下降与优化算法：在 Loss Landscape 中下山

- 直觉：沿着反向传播算法计算的最陡方向一步步接近山谷底部
- 要点：学习率是关键，Adam 是默认首选

2.8.2 训练流程全景

本章各节内容的关联：



这就是深度学习训练的核心循环：

- 1. 数据流入计算图，根据任务类型（分类/回归/自回归）得到预测
- 2. 损失函数评估预测好坏
- 3. 反向传播计算每个参数的梯度
- 4. 梯度下降调整参数，让损失减小
- 5. 重复直到收敛

2.8.3 关键点速查

概念	核心直觉	实践建议
计算图	数据流水线	PyTorch 自动构建，可可视化调试
任务形式	分类找边界、回归找映射、自回归找依赖	先确定任务类型，再选损失函数和输出层
激活函数	在空间中划分决策边界	隐藏层用 ReLU，输出层按任务选
损失函数	塑造优化地形	分类用交叉熵，回归用 MSE/MAE
反向传播	信用分配	框架自动完成，理解即可
梯度下降	下山策略	默认 Adam，大模型用 Warmup+Cosine
学习率	步长大小	最关键超参，常用 0.001-0.1

2.8.4 常见问题速答

Q: 为什么需要激活函数？

A: 没有激活函数，多层网络等价于单层线性变换，无法学习复杂模式。

Q: 反向传播和梯度下降的关系？

A: 反向传播计算梯度，梯度下降使用梯度更新参数。两者配合完成训练。

Q: 损失函数和优化算法的关系？

A: 损失函数定义"去哪里"（目标），优化算法决定"怎么去"（路径）。

Q: 局部最优真的是大问题吗？

A: 在高维深度网络中，鞍点比局部最优更常见。随机梯度的噪声通常能帮助逃离。

2.8.5 通往下一章

本章的数学基础将在 [上篇：练习峰](#) 中付诸实践：

- **感知机与 MLP**：用计算图搭建实际网络
- **卷积神经网络**：处理图像的空间特征
- **循环神经网络**：处理序列的时间特征（详见 [序列建模：从 RNN 到 Transformer 再到 Mamba](#)）
- **训练技巧**：批归一化、Dropout、早停等

准备好了吗?

现在你已经理解了深度学习"为什么能工作"。下一章我们将学习"如何让它工作得更好"——从理论走向实践，搭建和训练真实的神经网络。

记住：数学只是工具，直觉才是向导，实践是检验真理的唯一标准。

2.8.6 推荐资源**英文资源****快速复习：**

- 3Blue1Brown 神经网络系列³（直观可视化）

深入理解：

- 《深度学习》(Goodfellow) 第 4-6 章
- CS231n 斯坦福课程⁴（李飞飞等，计算机视觉+深度学习）

动手实践：

- PyTorch 60 分钟入门⁵（官方教程）
- fast.ai 课程⁶（实用导向，Top-down 教学）

中文资源**视频课程：**

- 李宏毅机器学习⁷（台大教授，中文讲解清晰，B 站/YouTube）

文档教程：

- PyTorch 官方中文文档⁸（含中文教程）
- PyTorch 深度学习实践⁹（中文教程合集）

本章完。下一步：**上篇：练习峰**

³ https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

⁴ <https://cs231n.stanford.edu/>

⁵ https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html

⁶ <https://course.fast.ai/>

⁷ <https://www.youtube.com/@HungyiLeeNTU>

⁸ <https://pytorch.org/docs/stable/index.html>

⁹ <https://github.com/fendouai/PyTorchDocs>

翻越两座山：CNN 三十年攀登史

3.1 引言：因为山在那里

1924 年，登山家乔治·马洛里被问到：“为什么要登珠峰？”他答：“**因为山在那里。**”

深度学习面前也有一座山。一座没有山顶的山——**完美的泛化**。无论投入多少算力、数据、层数，我们永远在接近，永远到不了。

那为什么还要攀登？因为每一个“爬不上去”的困境，都逼出了一个新的设计。**攀登本身，就是理解的过程。**

这一章不是百科全书

这一章讲的不是“CNN 有哪些层”或“ResNet 有几条连接”。

这一章讲的是：一群人如何从**赤手空拳**出发，一步步发现更好的工具、更聪明的路线、更可靠的生存法则——直到站在离山顶最近的地方，回望来路，然后把旗帜递给下一个人。

每一个架构都不是凭空设计的。每一个突破都是对前一代“为什么摔倒”的深刻回应。

已经熟悉 CNN？这里有快速路线

这一章非常长——占全书约 40%。如果你已经了解卷积、池化、LeNet 这些基础，不需要从头爬练习峰。

推荐路线：

1. 快速扫一遍 [登山上篇](#) 的引言，重点看 [归纳偏置](#) —— 它回答了"为什么 CNN 比全连接更适合图像"，是整个上篇的核心洞察
2. 翻翻 [消融研究](#) —— 拆开装备、验证每个组件的贡献，这套方法论在任何实验中都有用
3. 主力时间花在 [登山下篇](#) —— AlexNet、Inception、ResNet、MobileNet、EfficientNet，每一个都是对前代"为什么摔倒"的回应
4. 收尾读 [架构心法](#) —— 把具体技术升维为设计原则

可以跳过：全连接：赤手空拳、卷积：发现装备、数据准备：打包行囊（除非你想复习 MNIST 数据管线的细节）。

3.1.1 两座山

 登山上篇	 登山下篇
MNIST：练习峰	ImageNet：真正的未登峰
赤手空拳 → 发现装备 → 首登	AlexNet 首登 → 多尺度 → 高原反应 → 聚焦 → 预算学
学会攀登	征服极限

上篇，从赤手空拳出发，一步步发现冰镐和绳索，学会打包行囊、遵守纪律，完成第一次登顶。每一章都是对前一章"为什么爬不上去"的回应。

这背后的核心原理是 [归纳偏置](#) —— 把山的结构刻进装备里。全连接不看山体，20 万参数走不远；CNN 顺应山体，几百个参数就能爬更高。

两座山之间，[消融研究](#) 拆开装备 —— 不是破坏，是理解。把卷积层拆掉会跌多少？换掉激活函数呢？每个零件到底贡献了多少？

下篇，换一座真正的山。ImageNet 比 MNIST 大 1,000 倍。AlexNet 在 2012 年首登成功 —— 错误率从 28% 砸到 15%。此后八年：多尺度工具、高原反应补给站、学会聚焦的注意力、预算有限时的精打细算。练习峰教会了你怎么爬，登山下篇教会你在真正的山上怎么活。

3.1.2 攀登之后：把经验炼成心法

爬完两座山，你的装备包里塞满了具体技术 —— 卷积、跳跃连接、通道注意力、深度可分离卷积。但下次面对一座全新的山，你该怎么选装备？

[架构心法](#) 把经验升维为原则：感受野、深度连接、注意力、效率 —— 四个维度，四个旋钮。不是"加哪个模块"，而是"面对新任务时怎么判断该加什么"。它以登山者笔记收尾 —— 回望三十年的路，提炼出可以带走的智慧。

3.1.3 攀登路线



3.1.4 出发

深度学习核心数学基础 给了你理论 —— 计算图、反向传播、梯度下降。登上上篇是这些理论的**实战延伸**：

- 计算图 → PyTorch 自动求导
- 激活函数 → ReLU 和 Softmax 的实际表现
- 反向传播算法 → `loss.backward()` 的魔法
- 梯度下降与优化算法 → `optimizer.step()` 的参数更新

系紧鞋带。

上篇：练习峰 —— 走。

3.2 上篇：练习峰

3.2.1 引言：从公式到代码

理论你有了。打开编辑器的那一刻，你会愣住 —— 784 个像素，10 个数字，从哪开始？

深度学习核心数学基础 给了你登山手册 —— 计算图、反向传播、梯度下降。你读完了，理解了，甚至能徒手推导。

但打开一个空白的 `.py` 文件，面对 MNIST 的 784 个像素和 10 个数字 —— **第一行写什么？**

理论告诉你 CNN 更好。但“更好”这个词，在你亲眼看到全连接挣扎之前，只是一个抽象概念。**登山者不读登山手册 —— 他们爬山。摔了，才知道为什么需要更好的装备。**

为什么是 MNIST？

MNIST 是深度学习世界的“Hello World”。但和 Hello World 不同 —— 它真的能教会你东西。

	MNIST	CIFAR-10	ImageNet
样本数	70,000	60,000	1,400,000
图像尺寸	28×28 灰度	32×32 彩色	224×224 彩色
类别数	10	10	1,000
训练时间（单 GPU）	几分钟	几十分钟	几天到几周
预处理需求	几乎为零	需要标准化	复杂流水线

它之所以是完美的**练习峰**，有四个原因：

1. **小到可以快速迭代**——一次训练几分钟，摔了立刻重来。你一天可以跑几十个实验，每个实验都让你更理解一个参数的意义。
2. **干净到可以跳过干扰**——图像已标准化，背景纯净，数字居中。你不用操心数据清洗、类别不平衡、多尺度目标。**专注攀登技术本身**。
3. **直观到肉眼可验证**——模型学到了什么？把卷积核画出来——哦，这个在找竖线，那个在找圆圈。人类一眼能懂。这种可解释性在更大的模型上会迅速消失。
4. **历史意义**——1989 年 LeNet 在这座山上首登。三十年后，这座山依然是验证新想法的第一站。它小，但不小到毫无挑战——全连接网络在这座山上也会摔。

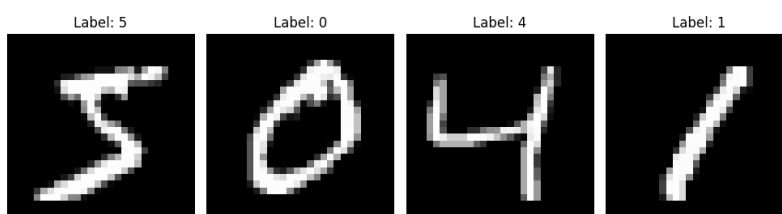


图 1: MNIST 手写数字样本

登山者的先验知识

你第一次攀岩时会怎么做？大概率是**徒手试探每一块岩石**——全身贴在岩壁上，手指抠进每一条裂缝，看看哪块石头能撑住体重、哪块会松动。

这就是全连接网络做的事。784 个像素，每一个都连到每一个神经元，不做任何假设，不信任任何结构。20 万个参数，就为了处理一张 28×28 的小图。

但经验丰富的登山者不会这样。他们看一眼岩壁就知道：

- **裂缝附近通常有稳固的支撑点**——相邻像素是相关的，不需要检查全部
- **同样的攀爬动作在不同位置都有效**——一个模式学会了，全图通用
- **岩壁的纹理是有方向的**——水平边缘和垂直边缘需要不同的“读法”

这，就是**归纳偏置**（Inductive Bias）。

归纳偏置

简单来说：把山的结构刻进装备里

归纳偏置是指学习算法对特定类型解决方案的偏好 [Mit80]。它不是代码里的 `if` 语句，而是架构设计隐含的“我先相信什么”。

攀法	归纳偏置	隐含假设
全连接（徒手）	弱/无	每一块岩石独立存在，和周围没关系
CNN（冰镐+绳索）	局部性 + 平移不变性	相邻岩石连在一起；同一技巧到处适用

为什么好的先验是作弊器？

1. **数据效率**：好的偏置让你用更少的尝试找到规律
 - 全连接：必须看到数字"7"出现在图像左上角、右下角、中间……每个位置都学一遍
 - CNN：学到一个位置的"7"，自动在所有位置认出它 —— 平移不变性
2. **参数效率**：好的偏置让每一克负重都值得
 - 全连接第一层： $784 \times 256 + 256 = 200,960$ 个参数
 - CNN 第一个卷积层（32 个 3×3 核）： $32 \times (3 \times 3 \times 1 + 1) = 320$ 个参数
 - 差距超过 **600 倍**。不是 CNN “更聪明” —— 是它“更信任山的结构”
3. **泛化能力**：好的偏置反映的是物理世界的客观规律，不是训练集的偶然噪声

贝叶斯视角

简单来说：先验就是信念

从贝叶斯统计的角度，归纳偏置就是**先验知识** —— 看到数据之前，你已经相信的东西：

概念	含义	在 CNN 中的体现
先验	看到数据前的信念	“相邻像素相关，卷积核应该局部连接”
似然	数据对假设的支持程度	训练样本告诉你哪些特征对分类有用
后验	看到数据后更新的信念	学到的具体卷积核权重 —— 先验 + 数据的融合

为什么先验重要？想象猜一枚硬币：

- **无先验（全连接）**：看到 3 次正面，就断言“这硬币 100% 正面”（过拟合了）
- **有先验（CNN）**：你本来就相信硬币大致公平；3 次正面只是随机波动（泛化好）

CNN 的归纳偏置就是给模型一个“登山老手的直觉” —— 还没开始爬，就知道该注意什么。好的先验让后验更快收敛到真相，且需要更少的数据。

$$P(\text{假设}|\text{数据}) \propto P(\text{数据}|\text{假设}) \times P(\text{假设})$$

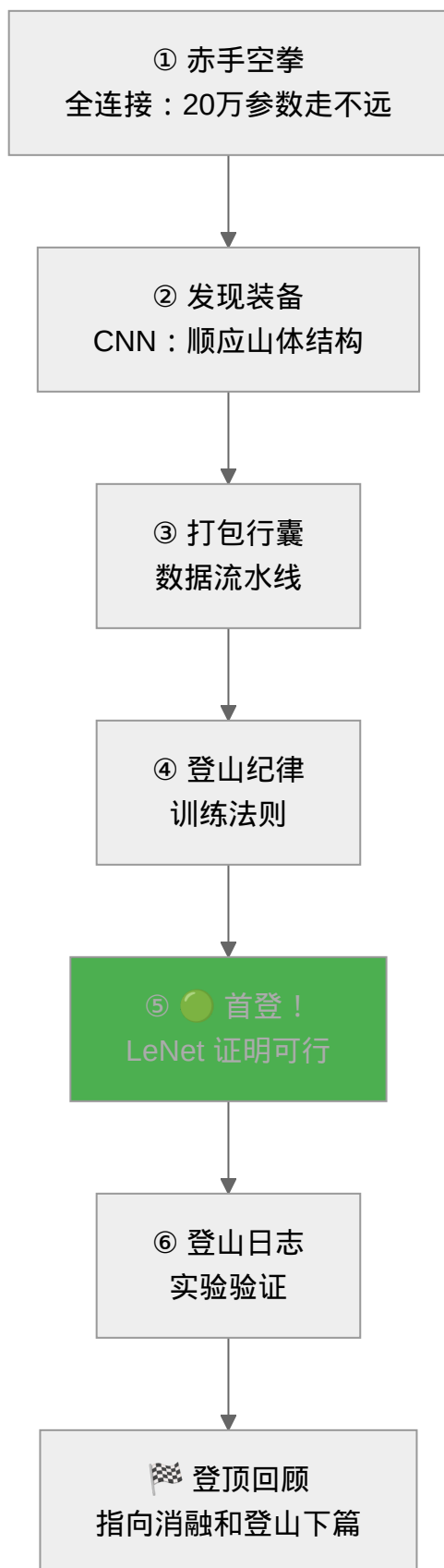
一句话

全连接是每遇到一个"7"都当全新符号教一遍。

CNN 是教一次"7"的特征，孩子自动在任何位置认出它。

好的架构，让学习事半功倍。因为你把山的形状，刻进了装备里。

登山上篇路线



核心问题驱动

你在哪里	你面对的问题	答案藏在
赤手空拳上阵	为什么 20 万参数走不远?	全连接: 赤手空拳
发现冰镐绳索	怎么用 1/4 参数爬更高?	卷积: 发现装备
打包行囊	数据怎么从硬盘飞到 GPU?	数据准备: 打包行囊
学习纪律	为什么装备好也会摔?	训练基础: 登山纪律
练习峰首登	能不能真正爬上去?	LeNet-5: 练习峰首登
记登山日志	装备好 = 爬得高? 数据说话	实验对比: 登山日志
登顶之后	每件装备贡献了多少? 换山怎么爬?	练习峰, 登顶

学完登上上篇, 你将能够

登上上篇目标

1. **理解归纳偏置**: 为什么 CNN 比全连接更适合图像 (归纳偏置 的胜利)
2. **实现**: 用 PyTorch 搭建全连接网络和 CNN
3. **训练**: 掌握调试技巧 —— 损失曲线、学习率、正则化
4. **验证**: 用实验数据检验理论 (山不会听你嘴硬, 山只看数据)
5. **登顶**: 完整走通从赤手空拳到 LeNet 首登的全过程

前置提醒

深度学习核心数学基础 是上一篇 —— 计算图、反向传播、梯度下降。本章是它的实战延伸:

- 计算图 → PyTorch 自动求导
- 激活函数 → ReLU 和 Softmax 的实际应用
- 反向传播算法 → `loss.backward()` 的魔法
- 梯度下降与优化算法 → `optimizer.step()` 的参数更新

3.2.2 全连接：赤手空拳

赤手空拳 —— 不看山体结构，20 万参数走不远。

赤手空拳出发 —— 每个像素连接每个神经元，20 万个参数，不做任何假设。很快就发现：**这山，爬不远。**

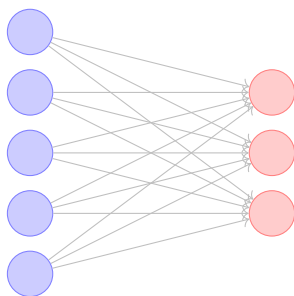
站在 MNIST 面前。用的是攀岩的技术 —— 每一块岩石亲手检查，不靠地形判断。784 个像素，全部连到 256 个神经元。200,960 个参数。不多想，先试试。

训练。看结果。**能跑，但吃力。**

不是参数不够。攀岩的思路放到山上，天然就错了 —— 你把岩壁当成了一堆互不相关的碎石。相邻的像素本来连在一起，偏要一个一个检查。同一个数字“7”出现在左边和右边，偏要学两遍。全连接不假设任何空间结构 —— 代价就是，20 万参数还爬不稳一座 28×28 的小山。

先看这个工具本身。

全连接层 (Fully Connected Layer)，源于感知机 (Perceptron) [Ros58]，是神经网络最基本的构建块。建议回顾[激活函数](#)中[线性回归](#) → [多层感知机](#)的演变。每个输入节点都与每个输出节点相连接 —— 赤手空拳，不留死角。



输入层 (5 个神经元)

输出层 (3 个神经元)

每个输入都连接到每个输出

图 2: 全连接层结构示意图

备注

全连接层的数学表达

对于输入向量 $\mathbf{x} \in \mathbb{R}^n$ 和输出向量 $\mathbf{y} \in \mathbb{R}^m$ ，全连接层的变换为：

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

这正是[激活函数](#)中提到的[线性变换](#) $\mathbf{W}\mathbf{x} + \mathbf{b}$ 的矩阵形式，其中：

- $\mathbf{W} \in \mathbb{R}^{m \times n}$ 是权重矩阵（所有连接）

· $\mathbf{b} \in \mathbb{R}^m$ 是偏置向量

正如我们在[激活函数](#)中看到的，如果没有非线性激活函数，多层全连接等价于单层。这就是为什么代码中需要使用 `F.relu()`。

为什么堆叠多层有用？

单层全连接 ($y = Wx + b$) 只能学习**线性决策边界**——在特征空间中画一条直线或超平面。但现实中大多数问题（手写数字识别、图像分类）的决策边界是高度非线性的。

多层堆叠 + 非线性激活解决了这个问题：

1. **每层变换空间**： $Wx + b$ 对特征空间做旋转、缩放、平移
2. **激活函数折叠空间**：[激活函数](#) 中我们看到，ReLU 将负区域"压平"到 0，实现了空间的非线性折叠
3. **层数越多，边界越复杂**：第一层学直线，第二层组合直线成简单形状，第三层组合形状成复杂边界

3 个神经元就能画出复杂边界

[决策边界的形成](#) 一节展示了一个惊人的例子：仅用 **3 个隐藏层神经元** 就能划分出任意复杂的不规则区域。每个神经元定义一个"软"半平面，通过乘法 ($n_2 \times (n_1 + n_3)$) 实现**门控机制**——一个神经元充当"开关"控制其他神经元的输出是否生效。少量神经元的非线性组合就能逼近几乎任何形状的决策边界。

这就是多层感知机 (MLP) 的核心能力：**用简单线性单元的层级组合表达任意复杂的函数**。

备注

全连接层的威力

$$3 \text{ 层全连接} = \text{ReLU}(W_3 \cdot \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 x + b_1) + b_2) + b_3)$$

每一层 $W_i x + b_i$ 做线性变换，每一层 ReLU 引入非线性折叠。三层叠在一起，理论上可以逼近任何连续函数（通用近似定理）。

这正是全连接网络的核心矛盾：**表达能力极强（可以学任何函数），但参数效率极低（什么先验都不假设）**。

参数数量分析

数一数装备的重量。全连接层从 n 维到 m 维的参数总数：

$$\text{Parameters} = m \times n + m = m(n + 1)$$

以 MNIST 为例，如果将 $28 \times 28 = 784$ 像素的图像直接输入到全连接层：

· 第一层：假设有 256 个神经元，参数数量为 $256 \times 784 + 256 = 200,960$

- 第二层：从 256 到 128 个神经元，参数数量为 $128 \times 256 + 128 = 32,896$
- 输出层：从 128 到 10 个类别，参数数量为 $10 \times 128 + 10 = 1,290$

总参数数量：235,146 个参数

参数爆炸：全连接的核心问题

235,146 个参数看似不多，但考虑以下问题：

问题	说明
空间信息丢失	784 维向量完全打乱了 28×28 的二维结构，相邻像素在向量中可能相距很远
参数效率低	每个像素都连接到每个神经元，无论距离多远，没有"局部性"概念
过拟合风险	参数量大，而 MNIST 是相对简单的 10 类分类任务

直觉理解：全连接把图像当作"表格"，而非"图片"。它不知道像素 A 和像素 B 是相邻的——就像登山者把岩壁当成散落的碎石，看不到它们本来的连接。这种空间关系必须从零学起。

PyTorch 实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class FullyConnectedNet(nn.Module):
    def __init__(self, input_size=784, hidden_size=256, num_classes=10):
        super(FullyConnectedNet, self).__init__()
        # 将 28x28 图像展平为 784 维向量
        self.flatten = nn.Flatten()

        # 全连接层堆叠
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, hidden_size // 2)
        self.fc3 = nn.Linear(hidden_size // 2, num_classes)

        # Dropout 层用于防止过拟合
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        # 展平：28x28 图像 → 784 维向量
        # 这就是"空间信息丢失"的发生点！二维结构变一维
        x = self.flatten(x)
```

(续下页)

(接上页)

```

# 第一层: 784 -> 256
x = self.fc1(x)      # 线性变换  $y = Wx + b$ 
x = F.relu(x)        # 非线性激活 (见 activation-functions)
x = self.dropout(x)  # 随机丢弃 50% 神经元, 防止过拟合

# 第二层: 256 -> 128
x = self.fc2(x)
x = F.relu(x)        # 再次激活, 增加表达能力
x = self.dropout(x)

# 输出层: 128 -> 10 (10 个数字类别)
# 注意: 最后一层不加激活, CrossEntropyLoss 内部会应用 Softmax
x = self.fc3(x)

return x

# 模型实例化
model = FullyConnectedNet()
print(f"模型总参数数量: {sum(p.numel() for p in model.parameters()):,}")

```

全连接层的优缺点

赤手空拳有赤手空拳的账本:

表 1: 全连接层优缺点对比

优点	缺点
实现简单直观	忽略空间结构信息
理论成熟完善	参数数量巨大
易于理解和调试	容易过拟合
计算效率高 (小模型)	对平移不具备鲁棒性
适用于非结构化数据	需要大量训练数据

核心问题：参数效率低下

235,146 个参数。爬一座 28×28 的小山，太重了。

什么是参数效率？

参数效率

参数效率可以粗略理解为 **准确率 / 参数量** —— 用更少的参数达到相同或更高的准确率，意味着更高的参数效率。这不是严格的数学公式，而是一个**设计哲学**。

指标	全连接网络	评价
参数量	235,146	过多
MNIST 准确率	~98%	尚可
参数效率	低	用 20 万参数"记住"数据，而非学会规律

问题本质：全连接缺乏**归纳偏置** —— 它没有利用"图像是二维的"、"相邻像素相关"这些先验。从贝叶斯角度看，全连接是**无信息先验** —— 上山前不做任何假设，每一块岩石都要亲自检查。这是攀岩的思路。登山需要另一种。

你用全连接爬了一次 MNIST。20 万参数。98%。

能用。但代价不对 —— 攀岩的技术放在山上，每一块岩石都要亲自检查。根源只有一个：全连接不看山体结构。

困境	摔在哪里	登山需要什么
每一块岩石单独检查	参数爆炸	局部感受野 —— 只看相邻的
山体被压成一条线	空间信息丢失	保留 2D 结构
同一模式每出现一次学一遍	"7"在左上和右下是两回事	权值共享 —— 一个模式，全图通用

这次摔跤不是失败。它是地图 —— 告诉你山的形状是什么，下一件装备要解决什么问题。

卷积：**发现装备** —— 走。换登山的思路。

3.2.3 卷积：发现装备

发现装备 —— 局部感受野 + 权值共享，顺应山体结构。

全连接：赤手空拳 中我们用全连接网络爬了一座 28×28 的小山 —— 攀岩的思路，每一块岩石亲手检查，不靠地形判断。结果：235,000 个参数，爬到山顶还吃力。

CNN 的设计来自对图像本质的**两个洞察**。这两个洞察分别引出了 CNN 的两个核心归纳偏置，合在一起就是卷积操作本身。

洞察一：只看邻居 —— 局部感受野

在照片里找猫。你不会同时盯着整张照片的每个像素 —— 你的视线会**聚焦在一个小区域**，看这里有没有猫耳朵的形状。然后移到下一个区域。你的判断基于局部特征，而非同时处理全部信息。

全连接网络却恰恰相反：每个神经元同时连接全部 784 个像素，相当于**每次判断都要看整张图**。这不仅浪费（每个连接都是一个参数），还让网络难以学到“边缘”、“纹理”这类天然是局部属性的特征。

CNN 的第一个洞察：**一个像素的含义主要由其周围像素决定**。因此，每个神经元只连接输入的一个小窗口 —— 比如 3×3 ，只看相邻 9 个像素。这就是**局部感受野**（Local Receptive Field）。

局部感受野的直观对比

- 全连接：每个神经元看全部 784 像素 → 第一层 $784 \times 128 = 100,352$ 参数（假设 128 个神经元）
- 局部感受野：每个神经元只看 $3 \times 3 = 9$ 像素 → 第一层 320 参数
- **减少 99.7% 的参数**，却能更好地捕获边缘、纹理等局部特征

洞察二：同一把尺子量遍全图 —— 权值共享

在地图上找十字路口。你不会为每个位置发明一个不同的“十字检测规则” —— 你会用一个**统一的规则**（“两条路垂直相交”），然后拿着它检查每个位置。左上角用这规则，右下角也用同一个规则。

全连接网络的做法又反了：检测“左上角有横线”的权重和检测“右下角有横线”的权重是**两组不同的参数**。网络必须浪费大量参数反复学习同一个模式在不同位置的样子。

CNN 的第二个洞察：**同一个视觉模式可以出现在图像的任何位置，检测它的规则应该共享**。因此，同一个卷积核在所有位置使用相同的权重 —— 这就是**权值共享**（Weight Sharing）。

权值共享的直观理解

一个检测“竖线”的卷积核滑过整张图 —— 不管竖线出现在左边还是右边，同一个核都能检测到。网络不需要为每个位置分别学习“什么是竖线”。

卷积：两个洞察的统一实现

局部感受野回答了"怎么看"：每次只看一小块。**权值共享**回答了"用什么看"：同一个工具反复用。

将两者结合，就得到了卷积操作：**用滑动窗口（局部感受野）在图像上移动，在每个位置用相同权重（权值共享）计算点积**。32 个卷积核就是 32 个不同的"检测器"——一个检测横线、一个检测竖线、一个检测弧线……它们各自滑过全图，各自寻找自己关注的特征。

这正是我们在 [归纳偏置](#) 中讨论的核心概念——CNN 将"图像是二维的"、“相邻像素相关”、“平移不变”这些先验知识**内置到架构中**。

参数效率：两个洞察的量化回报

参数效率

参数效率可以粗略理解为 **准确率 / 参数量**——用更少的参数达到相同或更高的准确率，意味着更高的参数效率。这不是严格的数学公式，而是一个**设计哲学**：在引入合理归纳偏置的前提下，用尽量少的参数完成任务。

架构	参数量	MNIST 准确率	归纳偏置	参数效率
全连接	~235K	~98%	弱	低
CNN	~60K	~99%	局部性+平移不变性	高

关键问题：CNN 如何用 1/4 的参数达到更好的效果？

答案就在那两个洞察里：

1. **局部感受野**： 3×3 卷积核只看相邻 9 个像素，而非全部 784 个
2. **权值共享**：同一个卷积核滑过整张图像，参数重复使用

感受野

感受野（Receptive Field）是理解 CNN 最重要的概念之一。它定义为输出特征图上的一个神经元对应到输入图像上的区域大小。简单说：一个神经元"看到"了输入图像上多大一个区域。

- 第 1 层卷积： 3×3 卷积核，感受野 = 3×3 （只看相邻像素）
- 第 2 层卷积：堆叠两个 3×3 ，感受野 = 5×5 （看到更广的区域）
- 第 3 层卷积：堆叠三个 3×3 ，感受野 = 7×7

递推公式：

$$RF_l = RF_{l-1} + (K_l - 1) \times \prod_{i=1}^{l-1} s_i$$

其中 RF_l 是第 l 层的感受野大小, K_l 是卷积核大小, s_i 是第 i 层的步长。

感受野随着网络深度线性增长。这也是为什么需要深层网络——**层数越多, 每个神经元能捕获的上下文信息越丰富**。注意, 感受野完全由网络架构(卷积核大小、步长、层数)决定, 训练不会改变它。

卷积操作的基本原理

卷积操作是 CNN 的核心, 它通过滑动窗口的方式在输入图像上应用(通过点积)滤波器(卷积核)来提取特征。这种思想最早由 LeCun 等人在 1989 年提出 [LBD+89], 并在后续的 LeNet-5 工作中 [LBBH98] 得到完善。

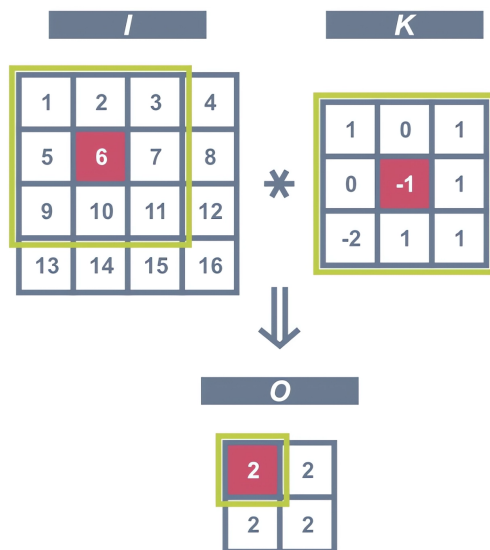


图 3: 卷积操作示意图

备注

卷积操作的数学定义

数学上严格定义的卷积包含核的翻转:

$$(f * g)[m, n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f[i, j] \cdot g[m - i, n - j]$$

但在深度学习中, 我们实际使用的是**互相关 (cross-correlation)**, 省略了翻转步骤:

$$Y[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X[i + u, j + v] \cdot K[u, v] + b$$

由于卷积核的数值是通过训练学习的, 翻转与否对结果没有影响——网络会自动学到合适的核值。因此, 本文后续提到的“卷积”均指互相关形式。

其中:

- X : 输入特征图

- K : 卷积核 (滤波器)
- b : 偏置项
- k : 卷积核大小

直觉：卷积核为什么能提取特征？

卷积核提取特征的原理其实很直观：**点积 = 相似度度量**。

回想一下两个向量的点积： $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ 。当两个向量方向一致时，点积最大；方向相反时，点积为负。换句话说，**点积衡量了两个向量的相似程度**。

卷积操作就是反复做点积：这个 3×3 的卷积核左边全是 -1、中间是 0、右边全是 1 —— 它检测的是**垂直边缘**：

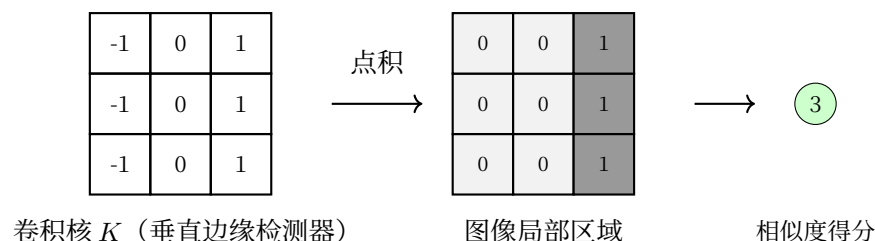


图 4: 卷积核=模式匹配器

- 当图像局部区域的**左边暗、右边亮** (如 $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$)，点积结果是**正数** (上图结果是 3)，说明"这里有垂直边缘"
- 如果图像区域是均匀的 (如全是 0)，点积结果是 0，说明"这里没有边缘"
- 如果左边亮右边暗，点积结果是负数，说明"这里有反方向的边缘"

不同的卷积核检测不同的模式：

卷积核	检测的特征	示例数值	响应条件
垂直边缘	左右亮度突变	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	左暗右亮 → 正响应
水平边缘	上下亮度突变	$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$	上暗下亮 → 正响应
角点检测	对角线变化	对角线模式	特定方向变化
纹理检测	重复的局部图案	训练得到	匹配特定纹理

训练过程中，卷积核的数值通过反向传播自动调整——网络自己“学会”了要检测什么特征。浅层学边缘、深层学语义，这就是归纳偏置中讨论的层次特征提取。

简单来说，卷积操作就是拿着一个个“模板”（卷积核）在图像上滑动，在每个位置计算模板和该区域的相似度。相似的区域得到高响应，不相似的得到低响应。32 个卷积核就是 32 个不同的模板，各自关注不同的模式。

特征图尺寸计算

对于输入尺寸为 $W \times H$ ，卷积核大小为 $K \times K$ ，步长为 S ，填充为 P 的情况，输出特征图的尺寸为：

$$W_{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

$$H_{out} = \left\lfloor \frac{H - K + 2P}{S} \right\rfloor + 1$$

MNIST 实例计算

对于 28×28 的 MNIST 图像，使用 3×3 卷积核，步长 $S=1$ ，填充 $P=1$ ：

- 输出宽度： $W_{out} = \left\lfloor \frac{28-3+2 \times 1}{1} \right\rfloor + 1 = 28$
- 输出高度： $H_{out} = \left\lfloor \frac{28-3+2 \times 1}{1} \right\rfloor + 1 = 28$
- 结论：输出特征图尺寸保持 28×28 不变

如果使用 2×2 池化，步长 $S=2$ ，无填充 $P=0$ ：

- 输出尺寸： $W_{out} = \left\lfloor \frac{28-2+0}{2} \right\rfloor + 1 = 14$
- 结论：池化将特征图尺寸减半

参数共享机制

洞察二 中我们建立了权值共享的直觉：同一个检测规则在所有位置反复使用。下面用数学语言精确描述这一机制。

卷积层的一个重要特性是参数共享。同一个卷积核在图像的不同位置重复使用，这大大减少了参数数量。

对于 C_{out} 个输出通道，每个通道使用独立的卷积核：

$$\text{Parameters} = C_{out} \times (K \times K \times C_{in} + 1)$$

其中 C_{in} 是输入通道数。

以 MNIST 为例（单通道输入，32 个 3×3 卷积核）：

$$\text{Parameters} = 32 \times (3 \times 3 \times 1 + 1) = 32 \times 10 = 320$$

相比全连接层的数万参数，卷积层的参数数量显著减少了 99% 以上。

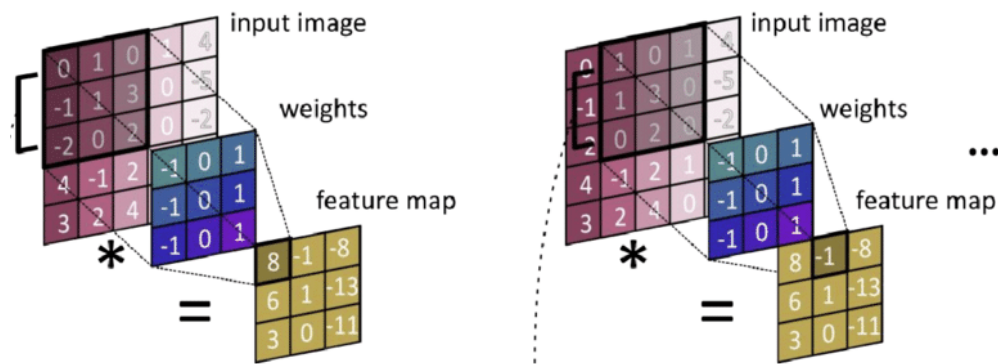


图 5: 参数共享机制示意图

卷积层的优势

回到找猫和十字路口的例子：局部连接让你每次聚焦一个小区域（洞察一），权值共享让你用同一个检测器扫遍全图（洞察二），平移不变性正是两者的自然推论。

卷积层的核心优势

1. **局部连接**：每个神经元只连接输入图像的局部区域
2. **权值共享**：同一卷积核在整个图像上滑动，大幅减少参数
3. **平移不变性**：相同特征在不同位置被同一卷积核检测
4. **层次特征提取**：浅层提取边缘等低级特征，深层提取语义等高级特征

池化层

直观理解

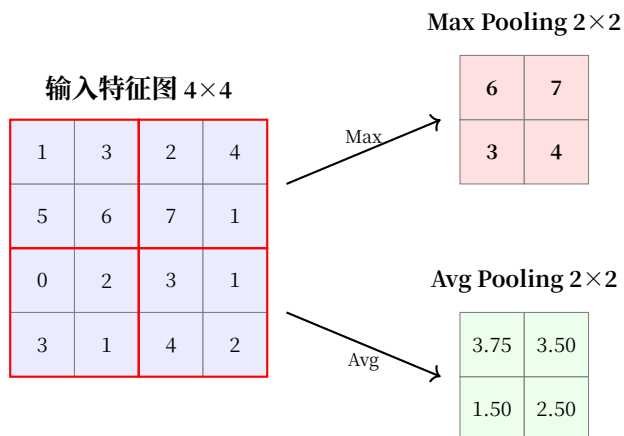
感受野告诉我们，卷积层通过堆叠来逐步扩大感受野。但有一个问题：如果只用卷积堆叠，特征图的空间尺寸始终不变（比如 28×28 ），后面的全连接层要处理的参数量会非常庞大。

我们需要一种“压缩”机制——在保留关键特征的同时，减小空间尺寸。

池化层就是 CNN 的“压缩包”：

- **最大池化 (Max Pooling)**：保留最显著的特征，丢弃细节。就像做摘要：只记最重要的论点
- **平均池化 (Average Pooling)**：保留整体信息，平滑噪声。就像取平均成绩，反映整体水平

图解：池化怎么压缩特征图

图 6: 2×2 池化操作示意

Max Pooling: 每个 2×2 窗口只保留最大值 —— 这代表该区域最强的特征响应。原来 $4 \times 4 = 16$ 个数字，压缩后只剩 4 个。

Avg Pooling: 每个 2×2 窗口取平均值 —— 保留了该区域的整体信息强度。

为什么 Max Pooling 更常用？

在实际 CNN 中，Max Pooling 远比 Avg Pooling 常用，原因与归纳偏置有关：

- 卷积层学出的特征响应本身就是“这个位置有没有检测到某个模式”
- Max Pooling 保留的是**最强的检测信号**，丢弃了“这个模式在这个区域不是最强位置”
- 这个过程本质上是一种**软性平移不变性**：只要特征出现在窗口内某个位置，Max Pooling 就能捕获它

Max Pooling 的直觉

想象你在一张照片里找猫。Max Pooling 的操作是：把照片分成 2×2 的小格子，每个格子里只保留**最像猫的证据**（最高响应），丢掉其他像素。这样无论猫出现在格子的哪个位置，你都能找到它 —— 这就是**平移不变性**的直觉来源。

数学定义

对于输入特征图 $X \in \mathbb{R}^{C \times H \times W}$ ，池化操作在每个通道上独立进行，不改变通道数：

最大池化：

$$Y[c, i, j] = \max_{(m, n) \in \mathcal{R}_{ij}} X[c, m, n]$$

平均池化：

$$Y[c, i, j] = \frac{1}{|\mathcal{R}_{ij}|} \sum_{(m,n) \in \mathcal{R}_{ij}} X[c, m, n]$$

其中 $\mathcal{R}_{ij}$ 是输出位置 (i, j) 对应的输入区域（如 2×2 窗口）。

池化与卷积的关键区别

池化操作**没有可学习参数**，它的计算是固定的（取最大值或平均值）。这意味着：

- 池化不会增加模型的表达能力
- 但池化能有效减少计算量（空间尺寸减半 = 计算量减半）
- 池化层在反向传播时，梯度直接通过最大值位置回传，无需更新权重

池化层尺寸计算

池化层的尺寸计算公式与卷积层一致：

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1$$

其中 K 是池化窗口大小， S 是步长， P 是填充。最常用配置是 $K = 2, S = 2, P = 0$ ，此时输出尺寸恰好减半：

$$28 \xrightarrow{2 \times 2} 14 \xrightarrow{2 \times 2} 7 \xrightarrow{2 \times 2} 4 \quad (\text{不足窗口大小时, 按 floor 处理})$$

PyTorch 实现

```
import torch.nn as nn

# 最常用：2x2 最大池化，步长=2
# 每次调用，空间尺寸减半，通道数不变
pool = nn.MaxPool2d(kernel_size=2, stride=2)

# 输入：batch_size × channels × height × width
x = torch.randn(1, 32, 28, 28) # 32 通道，28x28
y = pool(x)
print(y.shape) # torch.Size([1, 32, 14, 14]) —— 32 通道不变，空间减半

# 平均池化
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)

# 自适应池化：输入可变，输出固定尺寸
# 这在 ResNet 中用于替代全连接层，称为全局平均池化（GAP）
```

(续下页)

(接上页)

见本文后续“池化选型指南”表格中的说明

adaptive_pool = nn.AdaptiveAvgPool2d((1, 1))

池化选型指南

池化类型	PyTorch 类	典型配置	输出特点	适用场景
最大池化	nn.MaxPool2d	kernel_size=2, stride=2	保留最强响应, 空间减半	CNN 卷积层后降维 (LeNet/ResNet 标准配置)
平均池化	nn.AvgPool2d	kernel_size=2, stride=2	保留平均信息, 空间减半	特征统计, 轻量降维, 辅助 Max Pooling
全局平均池化	nn.AdaptiveAvgPool2d((1, 1))	输出固定 1×1	无参数, 完全压缩	替代 FC 层, 大幅减少参数, 见 ResNet: 高原反应补给站
自适应池化	nn.AdaptiveAvgPool2d((H, W))	输出固定 H×W	输入可变, 输出固定	多尺度输入、部署时输入尺寸不固定

全局平均池化 (GAP): ResNet 的标志设计

全局平均池化将整个特征图压缩成 1 个数字 (每个通道取平均), 然后直接接分类层。这避免了全连接层的巨大参数量。在 [ResNet: 高原反应补给站](#) 中, 我们将看到 ResNet 如何用 GAP 替代传统 FC 层, 使网络参数减少 90% 以上。

这种设计体现了 CNN 的另一种归纳偏置: **特征的空间位置信息在深层已不重要, 重要的是每个通道的整体激活强度**——这与归纳偏置中讨论的“层次特征提取”一脉相承。

反向传播中的梯度流动

池化层没有可学习参数, 但梯度仍然需要通过它回传。反向传播的规则很简单:

- **Max Pooling**: 梯度只回传给最大值所在位置, 其他位置梯度为 0
- **Average Pooling**: 梯度均匀回传给窗口内的所有位置

这个机制在后面讨论**梯度消失 (Vanishing Gradient)**时会再次出现——Max Pooling 的梯度“门控”特性, 某种程度上帮助缓解了梯度在深层网络中的稀释问题。

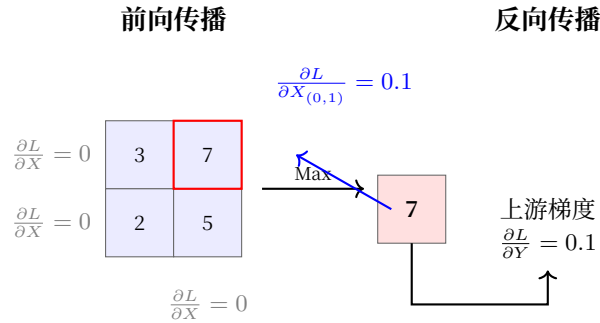


图 7: Max Pooling 反向传播示意

验证实验

```
import torch
import torch.nn as nn

# 创建一个简单的卷积+池化块
conv = nn.Conv2d(1, 32, kernel_size=3, padding=1) # 保持尺寸
pool = nn.MaxPool2d(2, 2)

x = torch.randn(1, 1, 28, 28)
print(f"输入尺寸: {x.shape}")           # [1, 1, 28, 28]

x = conv(x)
print(f"卷积后: {x.shape}")             # [1, 32, 28, 28] —— 尺寸不变 (padding=1)

x = pool(x)
print(f"池化后: {x.shape}")             # [1, 32, 14, 14] —— 空间减半, 通道不变

x = pool(x)
print(f"再池化: {x.shape}")             # [1, 32, 7, 7] —— 继续减半

# 注: 实际 CNN 中, 两次池化之间会有卷积层增加通道数 (见下方 SimpleCNN)

# 参数量对比: 有无池化
# 无池化: fc_input = 32 × 28 × 28 = 25,088 → fc1(25088, 128) = 3,211,264 参数
# 有池化: fc_input = 32 × 7 × 7 = 1,568 → fc1(1568, 128) = 200,832 参数
# 池化减少全连接层参数约 94%!
```

一句话总结: 池化层是 CNN 的"空间压缩机"——它不学习任何参数, 但通过下采样让后续层的计算量大幅减少, 同时通过取最大值提供了一种软性的平移不变性。

PyTorch 实现 CNN

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()

        # 第一个卷积块: 1 -> 32 通道
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)

        # 第二个卷积块: 32 -> 64 通道
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)

        # 池化层 (见{ref}`pooling`)
        self.pool = nn.MaxPool2d(2, 2)

        # 全连接层
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, num_classes)

        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        # 第一个卷积块: 28x28 -> 14x14
        # conv1: 输入 1 通道, 输出 32 通道, 3x3 卷积核, 参数量 32*(3*3*1+1)=320
        x = self.conv1(x)      # {ref}`computational-graph`中的卷积节点
        x = self.bn1(x)        # 批归一化, 缓解内部协变量偏移, 加速收敛 {cite}`ioffe2015batch`
        x = F.relu(x)          # 非线性激活 (见{ref}`activation-functions`)
        x = self.pool(x)       # 下采样: 28x28 -> 14x14

        # 第二个卷积块: 14x14 -> 7x7
        # conv2: 输入 32 通道, 输出 64 通道, 参数量 64*(3*3*32+1)=18,496
        x = self.conv2(x)
        x = self.bn2(x)
        x = F.relu(x)
        x = self.pool(x)      # 14x14 -> 7x7

```

(续下页)

(接上页)

```

# 展平: 64 通道 × 7x7 = 3,136 维向量
# 对比全连接: 直接从 784 展平, CNN 先提取特征再展平
x = x.view(x.size(0), -1)

# 全连接层: 3,136 -> 128 -> 10
x = self.fc1(x)
x = F.relu(x)
x = self.dropout(x)      # 正则化, 防止过拟合
x = self.fc2(x)          # 输出 logits, CrossEntropyLoss 内部 Softmax

return x

def get_param_count(self):
    """计算各层参数数量, 验证参数效率"""
    conv1_params = 32 * (3*3*1 + 1)    # 320
    conv2_params = 64 * (3*3*32 + 1)   # 18,496
    fc1_params = 64*7*7 * 128 + 128    # 401,536
    fc2_params = 128 * 10 + 10         # 1,290
    total = conv1_params + conv2_params + fc1_params + fc2_params
    return {
        'conv1': conv1_params,
        'conv2': conv2_params,
        'fc1': fc1_params,
        'fc2': fc2_params,
        'total': total
    }

# 模型实例化
model = SimpleCNN()
param_info = model.get_param_count()

print("=" * 50)
print("CNN 参数效率分析")
print("=" * 50)
print(f"卷积层参数:")
print(f"  Conv1 (32 通道): {param_info['conv1']:,} 参数")
print(f"  Conv2 (64 通道): {param_info['conv2']:,} 参数")
print(f"全连接层参数:")
print(f"  FC1 (128 神经元): {param_info['fc1']:,} 参数")

```

(续下页)

(接上页)

```
print(f" FC2 (10 类别): {param_info['fc2'],} 参数")
print(f"-" * 50)
print(f"总参数数量: {param_info['total'],}")
print(f"对比全连接: 235,146 参数")
print(f"参数减少: {235146 - param_info['total'],} ({(1 - param_info['total']/235146)*100:.1f}%)
↪")
print(f"-" * 50)
```

特征图: CNN 的"眼睛"

每层卷积都在学习什么? 通过可视化特征图, 我们可以"看到"CNN 的感知过程:

层级	特征图内容	人类可理解性	作用
Conv1 (32 通道)	边缘、纹理、颜色梯度	☆☆☆ 高	像边缘检测器, 识别笔画边界
Conv2 (64 通道)	笔画、弧线、角点	☆☆☆ 中	组合低级特征, 识别数字部件
池化后	抽象的位置信息	☆☆☆ 低	降维同时保留关键特征
FC 层	数字类别的高层表示	☆☆☆ 低	完全抽象, 人类难以理解

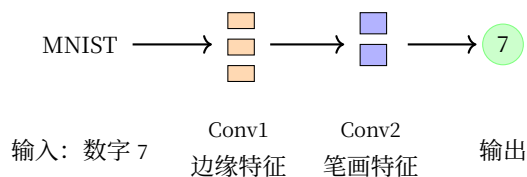


图 8: CNN 分层特征提取示意

分层学习的启示:

- CNN 自动学习从**简单到复杂**的特征层次, 无需人工设计
- 这与人类视觉系统类似: 视网膜 → 边缘检测 → 形状识别 → 物体认知
- 全连接网络没有这种层次结构, 必须同时学习所有复杂度

如何"看到"特征图

在前面的 SimpleCNN 中, 我们可以用 PyTorch 的 hook 机制提取中间层的特征图:

```
import torch
import matplotlib.pyplot as plt

# 注册 hook 捕获 conv1 的输出
```

(续下页)

(接上页)

```

features = {}
def get_activation(name):
    def hook(model, input, output):
        features[name] = output.detach()
    return hook

model = SimpleCNN()
model.conv1.register_forward_hook(get_activation('conv1'))
model.conv2.register_forward_hook(get_activation('conv2'))

# 前向传播
x = torch.randn(1, 1, 28, 28)
output = model(x)

# 查看特征图形状
print(f"Conv1 特征图: {features['conv1'].shape}") # [1, 32, 28, 28] — 32 个边缘检测器
print(f"Conv2 特征图: {features['conv2'].shape}") # [1, 64, 14, 14] — 64 个笔画检测器

# 可视化前 8 个通道的 Conv1 特征图
fig, axes = plt.subplots(2, 4, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(features['conv1'][0, i].numpy(), cmap='viridis')
    ax.axis('off')
    ax.set_title(f'Channel {i}')
plt.suptitle('Conv1 特征图 (32 通道中的前 8 个) ')
plt.show()

```

运行这段代码，你会看到：前几个通道捕获不同方向的边缘，后面通道捕获纹理和颜色梯度——这正是归纳偏置 中层次特征提取的直观证据。

CNN 的优缺点

表 2: 卷积神经网络优缺点对比

优点	缺点
保留空间结构信息	感受野固定，无法自适应调整关注区域
参数数量少	空间信息随池化逐层丢失
平移不变性	对小物体不友好（深层感受野大，小物体特征被稀释）
局部连接	卷积核大小固定，难以捕获长距离依赖
分层特征提取	对旋转/缩放等变换敏感（需数据增强弥补）

从全连接到卷积的演进

卷积神经网络的设计直接针对全连接网络的局限性：

1. **参数爆炸** → 参数共享大幅减少参数
2. **空间信息丢失** → 局部连接保留空间结构
3. **平移不变性差** → 平移不变性提高泛化能力
4. **计算效率低** → 分层特征提取提高计算效率

这种演进体现了深度学习架构设计中的**归纳偏置**思想：通过引入适合任务结构的先验知识，让模型更高效地学习。从找猫（局部感受野）到找十字路口（权值共享），CNN 的设计始终遵循一个原则：**架构应该顺应数据的结构**。

装备有了。但出发前还有一件事——**数据准备：打包行囊**：打包行囊。数据不会自己飞到 GPU 上。

3.2.4 数据准备：打包行囊

打包行囊——数据怎么从硬盘飞到 GPU？

装备有了。出发前最后一件事：**打包行囊**。数据不会自己变成张量。**卷积：发现装备**教会了 CNN 如何提取特征，但数据从哪里来、怎么从硬盘飞到 GPU——这是登山前必须解决的。

本节回答这个问题，打造一条完整的数据流水线：从原始数据 → 预处理 → Dataset → DataLoader → GPU。

数据流水线概览



从右往左看：PyTorch 不需要你知道文件存在哪里，它只需要一个能按索引返回（图片，标签）的对象——这就是 Dataset。DataLoader 负责在这个对象上自动打包成 batch、打乱顺序、多进程预加载。

第一步：数据从哪里来

内置数据集（最省心）

对于 MNIST、CIFAR-10、ImageNet 等经典基准数据集，PyTorch 通过 `torchvision.datasets` 一行代码下载并准备好：

```
import torchvision
```

(续下页)

(接上页)

```
# 下载 MNIST (第一次自动下载, 之后复用本地缓存)
train_data = torchvision.datasets.MNIST(
    root='./data',      # 本地缓存目录
    train=True,         # 训练集
    download=True       # 第一次运行时设为 True
)
```

常用内置数据集

数据集	PyTorch 类	任务	规模
MNIST	<code>torchvision.datasets.MNIST</code>	手写数字分类 (10 类)	60K 训练 / 10K 测试
Fashion-MNIST	<code>torchvision.datasets.FashionMNIST</code>	服装分类 (10 类)	60K 训练 / 10K 测试
CIFAR-10	<code>torchvision.datasets.CIFAR10</code>	通用物体分类 (10 类)	50K 训练 / 10K 测试
CIFAR-100	<code>torchvision.datasets.CIFAR100</code>	通用物体分类 (100 类)	50K 训练 / 10K 测试
ImageNet	<code>torchvision.datasets.ImageNet</code>	大规模分类 (1000 类)	1.2M 训练 / 50K 验证

自定义数据集 (真实项目)

真实场景中你的数据通常是以文件夹形式组织的:

```
data/
├── train/
│   ├── cat/          # 按类别分文件夹
│   │   ├── cat_001.jpg
│   │   ├── cat_002.jpg
│   │   └── ...
│   ├── dog/
│   └── ...
└── test/
    ├── cat/
    ├── dog/
    └── ...
```

对于这种结构, `torchvision.datasets.ImageFolder` 直接读取:

```
from torchvision.datasets import ImageFolder
```

(续下页)

(接上页)

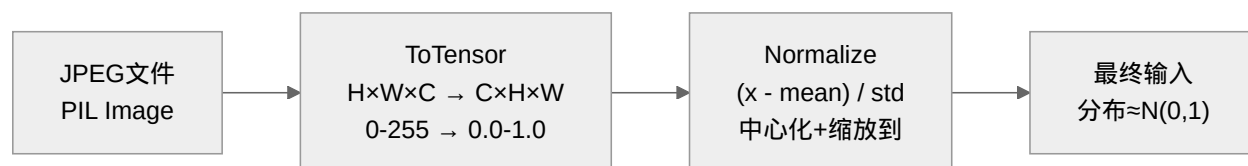
```
# 自动按文件夹名分配标签: cat-0, dog-1, ...
dataset = ImageFolder(root='data/train')
print(dataset.classes)      # ['cat', 'dog', ...]
print(dataset.class_to_idx) # {'cat': 0, 'dog': 1, ...}
```

警告

ImageFolder 要求数据按**文件夹名 = 类别名**组织。如果你的数据是 CSV/JSON/X 光片等非图像格式，就需要自己写 Dataset 类（见第六步）。

第二步：Transform —— 从图像到张量

原始图像是 JPEG/PNG 文件（像素值 0~255），不能直接喂给神经网络，需要**三步标准处理**：

**ToTensor：格式转换 + 归一化**

```
transforms.ToTensor() # 做了两件事：
# 1. PIL Image (H×W×C) → Tensor (C×H×W)
# 2. [0, 255] → [0.0, 1.0]
```

Normalize：标准化

图像数据中，像素强度差异很大（背景暗、文字亮）。标准化让每个通道的均值为 0、方差为 1，**帮助梯度下降更稳定地收敛**。

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    # (x - mean) / std, mean 和 std 是数据集的统计值
    # MNIST 是灰度图，所以只有 1 个通道
    transforms.Normalize((0.1307,), (0.3081,))
])
```

这里的 $(0.1307,)$ 和 $(0.3081,)$ 是从整个 MNIST 训练集计算出来的**全局均值和标准差**。它们的作用是让模型输入具有统一的数值范围，不受亮度、对比度的影响。

对于彩色图像（如 CIFAR-10）每个通道分别标准化：

```
# CIFAR-10 的 RGB 三通道统计值
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        mean=(0.4914, 0.4822, 0.4465), # R/G/B 各通道均值
        std=(0.2023, 0.1994, 0.2010)  # R/G/B 各通道标准差
    )
])
```

训练集与验证集的 transform 不同

这是实践中最常犯的错误之一：**验证集和测试集不能做数据增强**，只能用标准的 ToTensor + Normalize。

```
# 训练集 transform: 可做数据增强
train_transform = transforms.Compose([
    transforms.RandomRotation(10),          # 随机旋转 ±10°
    transforms.RandomAffine(0, translate=(0.1, 0.1)), # 随机平移 ±10%
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# 验证/测试集 transform: 只做标准化
val_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

为什么验证集不能做数据增强？

数据增强（旋转、平移、翻转）本质上是在制造“假样本”来扩增训练数据。但在验证时，我们需要评估模型在真实数据上的表现——如果验证集也被增强了，评估结果就不反映真实性能了。就像你平时用模拟题练习（数据增强），但考试时题目必须是原题（无增强）。

常见数据增强操作一览

操作	transforms 类	典型参数	效果
随机旋转	RandomRotation	degrees=10	小幅旋转，增加旋转不变性
随机平移	RandomAffine	translate=(0.1, 0.1)	小幅平移，增加平移不变性
随机翻转	RandomHorizontalFlip	p=0.5	水平镜像，增加翻转不变性
随机裁剪	RandomCrop / RandomResizedCrop	size=224	局部特征，增加尺度不变性
色彩抖动	ColorJitter	brightness=0.2	调整亮度/对比度，增加光照不变性

Albumentations：更强大的图像增强库

上面表格中的操作可以应付 MNIST/CIFAR 等基础任务，但在真实项目中（特别是竞赛和生产环境），你可能需要**更强、更快、更丰富的增强策略**——这就是完整训练流程中提到的工程最佳实践里会再次讨论的 Albumentations。

为什么用 Albumentations？

对比维度	torchvision.transforms	Albumentations
增强种类	~20 种	~70 种，涵盖像素级、空间级、噪声模拟
速度	一般	快 2~5 倍（基于 OpenCV 实现）
API 一致性	每个类参数风格不同	统一的 always_apply / p=0.5 概率控制
边界框/掩码同步增强	✗ 不支持	☑ 目标检测/分割的标注同步变换
pip install	随 torchvision 自带	pip install albumentations 单独安装

安装

```
pip install albumentations
```

基本用法

Albumentations 的 API 与 transforms.Compose 类似，但参数更一致，并且所有操作都支持统一的概率控制：

```
import albumentations as A
from albumentations.pytorch import ToTensorV2
```

(续下页)

(接上页)

```

# 训练集增强
# 注意每个操作都有 p=... 指定概率, 默认 p=1
# A.Compose 按顺序应用, 与 torchvision 的 transforms.Compose 相同
train_transform = A.Compose([
    A.HorizontalFlip(p=0.5),          # 50% 概率翻转
    A.Rotate(limit=15, p=0.8),        # 80% 概率旋转 ±15°
    A.RandomBrightnessContrast(p=0.5), # 50% 概率调整亮度对比度
    A.Normalize(mean=(0.1307,), std=(0.3081,)),
    ToTensorV2(),
])

# 验证集: 只做标准化 (与 torchvision 一致)
val_transform = A.Compose([
    A.Normalize(mean=(0.1307,), std=(0.3081,)),
    ToTensorV2(),
])

```

与 PyTorch Dataset 集成

Albumentations 的输入输出格式与 torchvision 不同, 需要在 Dataset 的 `__getitem__` 中做适配:

```

from torch.utils.data import Dataset
import cv2

class AlbumentationsDataset(Dataset):
    """支持 Albumentations 增强的 Dataset"""

    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # 1. 读取图像 (Albumentations 需要 numpy 数组格式)
        image = cv2.imread(self.image_paths[idx])
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        label = self.labels[idx]

```

(续下页)

(接上页)

```
# 2. 应用增强 (传入 dict, 返回 dict)
if self.transform:
    augmented = self.transform(image=image)
    image = augmented['image']

return image, label
```

Albumentations 的独家增强操作

以下是一些 torchvision 没有、但在竞赛中非常常用的增强：

操作	类	效果
网格失真	A.GridDistortion	模拟镜头畸变，增加几何鲁棒性
光学畸变	A.OpticalDistortion	模拟鱼眼镜头效果
Cutout / CoarseDropout	A.CoarseDropout	随机遮罩部分区域，防止过拟合到特定纹理
像素级噪声	A.GaussNoise	模拟传感器噪声，增加抗噪能力
锐化/模糊	A.Sharpen / A.Blur / A.MotionBlur	模拟对焦不准或运动模糊
通道偏移	A.ChannelShuffle	随机打乱颜色通道，增加色彩鲁棒性

何时用 torchvision，何时用 Albumentations？

场景	推荐	理由
MNIST/CIFAR 学习入门	torchvision	零额外依赖，够用
ImageNet 级分类	torchvision 或 Albumentations	两者均可
目标检测 / 语义分割	Albumentations	标注（边界框、掩码）同步变换是关键能力
比赛 / 生产调优	Albumentations	70+ 增强策略，精细控制，可反复组合
自定义数据加载逻辑	两者结合	torchvision 处理基础，Albumentations 扩展增强

数据增强哲学：适度优于过度

数据增强不是越多越好。过多的增强（如同时做旋转+畸变+Cutout+噪声）可能导致：

1. 样本失真：增强后的图像人类都认不出，模型更不可能学会
2. 训练不稳定：输入分布变化过大，梯度难以收敛
3. 性价比递减：前 3~5 种增强效果最显著，超过 8 种几乎没有额外收益

一个实用的起点是：**旋转 + 翻转 + 颜色抖动 + Cutout**，根据任务逐步追加。

第三步：Dataset —— 数据结构的统一接口

Dataset 是 PyTorch 数据系统的核心抽象。它定义了一个简单的协议：“给我一个索引，返回一个 (数据, 标签) 对”。

```
from torch.utils.data import Dataset

# 任何 Dataset 都只需要实现两个方法
class MyDataset(Dataset):
    def __len__(self):
        """返回数据集总大小"""
        pass

    def __getitem__(self, idx):
        """返回第 idx 个样本的 (数据, 标签) """
        pass
```

对于内置数据集（如 MNIST），PyTorch 已经实现了这两个方法，你不需要自己写。

Dataset 只负责“有什么数据”，不负责“怎么组织成 batch”——那是 DataLoader 的事。

Dataset vs DataLoader 的职责边界

组件	职责	不负责
Dataset	索引 → (数据, 标签)	打乱顺序、打包 batch、多进程
DataLoader	Dataset → batch	文件读取、预处理

第四步：DataLoader —— 从单样本到批量

有了 Dataset 之后，DataLoader 把它变成一个可迭代的批量数据源：

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=64,      # 每批 64 个样本
    shuffle=True,       # 每 epoch 打乱顺序
    num_workers=2,      # 2 个子进程预加载数据
```

(续下页)

(接上页)

```
pin_memory=True    # 加速 GPU 数据传输
)
```

关键参数详解

batch_size: 每批打包多少个样本。模型是**每批更新一次参数**，所以 batch size 直接影响：

- 内存占用：batch size × 单个样本大小
- 梯度稳定性：batch 越大，梯度估计越准确
- 泛化能力：过大的 batch 可能降低泛化性能

shuffle: 每轮训练前打乱数据。如果不打乱，模型会“记住”数据顺序而非真正学习特征。

num_workers: 用几个子进程预加载下一批数据。设置为 2~8 可以显著加速数据加载，但过大反而增加开销。

pin_memory: 将数据固定在 CPU 内存的特定区域，**加速 CPU → GPU 传输**。设为 True 几乎总是更好。

实际效果演示

```
# 假设 train_dataset 有 60,000 张 MNIST 图片
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=2
)

# 每次迭代返回一个 batch
for batch_idx, (images, labels) in enumerate(train_loader):
    # images.shape: [64, 1, 28, 28] — 64 张灰度图
    # labels.shape: [64] — 64 个标签
    # batch_idx: 0, 1, 2, ..., 937 — 共 938 个 batch

    if batch_idx == 0:
        print(f"image shape: {images.shape}") # [64, 1, 28, 28]
        print(f"label shape: {labels.shape}") # [64]
        print(f"labels: {labels}") # tensor([3, 7, 1, ...])
        print(f"total batches per epoch: {len(train_loader)}") # 938
        break
```

GPU 数据传输路径

流式加载机制保证了训练循环不会因为 I/O 而停滞 —— GPU 在算第 N 批时，CPU 已经在加载第 N+1 批了。

第五步：数据集拆分 —— train / val / test

模型训练不能只看训练集，需要三组数据各司其职：

数据集	用途	使用频率	核心要求
训练集	更新模型参数（权重+偏置）	每个 epoch 反复使用	数据量大，覆盖多样性
验证集	选择超参数、检测过拟合	每 epoch 后评估一次	与训练集独立分布，但不可参与训练
测试集	最终报告模型性能	只在全部调参完成后用一次	完全隔离，绝不可泄露

最常用：随机拆分（train_test_split）

很多内置数据集只给了训练/测试两份，需要自己从中分出验证集。

```
from torch.utils.data import random_split

# MNIST 标准划分：60K 训练 / 10K 测试
# 从 60K 训练集中分出 10% 作为验证集
train_dataset = torchvision.datasets.MNIST(root='./data', train=True)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False)

val_size = int(0.1 * len(train_dataset)) # 6,000 张
train_size = len(train_dataset) - val_size # 54,000 张

# random_split 按比例随机拆分，不会破坏类别平衡
train_subset, val_subset = random_split(
    train_dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(42) # 固定种子，确保可复现
)

print(f"训练集: {len(train_subset)}") # 54,000
print(f"验证集: {len(val_subset)}") # 6,000
print(f"测试集: {len(test_dataset)}") # 10,000
```

为什么需要固定 random_split 种子?

不固定种子的话，每次运行程序验证集会不同——今天在 A 组上验证好，明天可能在 B 组上不好，实验结果不可复现。固定 `manual_seed(42)` 确保每次拆分结果一致。

分层拆分（保持类别比例）

`random_split` 是随机抽取，在小数据集上可能破坏类别平衡。如果原始数据分布不均（如猫 90%，狗 10%），随机抽取的验证集可能恰好没有狗。

```
from sklearn.model_selection import train_test_split
import numpy as np

# 获取所有数据的标签
labels = [train_dataset[i][1] for i in range(len(train_dataset))]

# 分层拆分：按标签比例分配
train_idx, val_idx = train_test_split(
    np.arange(len(train_dataset)),
    test_size=0.1,
    stratify=labels,    # 按标签分层
    random_state=42
)

from torch.utils.data import Subset
train_subset = Subset(train_dataset, train_idx)
val_subset   = Subset(train_dataset, val_idx)
```

scikit-learn vs torch 的选择

- 数据量大且类别均衡 → `random_split` 就够了（简单、无额外依赖）
- 小数据集或类别不均衡 → `train_test_split(stratify=...)`（需要 sklearn）
- 做交叉验证 → `KFold` / `StratifiedKFold`（也需要 sklearn）

完整数据流水线模板

把上述所有步骤组合成一个可复用的函数：

```
def prepare_mnist_data(batch_size=64, val_split=0.1, num_workers=2):
    """
```

(续下页)

(接上页)

```

返回 train_loader, val_loader, test_loader

Args:
    batch_size: 每批样本数
    val_split: 从训练集中分出的验证集比例
    num_workers: 并行加载进程数
"""
# 1. 定义 transform (训练集有增强, 验证/测试集无增强)
train_transform = transforms.Compose([
    transforms.RandomRotation(10),
    transforms.RandomAffine(0, translate=(0.1, 0.1)),
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

val_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# 2. 加载原始数据
full_train = torchvision.datasets.MNIST(
    root='./data', train=True, download=True
)
test_dataset = torchvision.datasets.MNIST(
    root='./data', train=False, transform=val_transform
)

# 3. 拆分 train / val
val_size = int(val_split * len(full_train))
train_size = len(full_train) - val_size

train_dataset, val_dataset = random_split(
    full_train, [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

# 4. 分别应用 transform
# 注意: random_split 之后, 每个子集需要独立设置 transform。
# 简化做法是提前在原始 Dataset 初始化时就传入 transform,

```

(续下页)

(接上页)

```

# 但 train 和 val 需要不同的 transform (训练集增强, 验证集不增强),
# 因此实际工程中常用 Subset + 自定义 Dataset wrapper。
#
# 为简洁起见, 这里省略 Subset wrapper 的实现细节 ——
# 框架中已封装好此逻辑, 直接调用 prepare_mnist_data() 即可。

# 5. 创建 DataLoader
train_loader = DataLoader(
    train_dataset, batch_size=batch_size,
    shuffle=True, num_workers=num_workers, pin_memory=True
)
val_loader = DataLoader(
    val_dataset, batch_size=batch_size,
    shuffle=False, num_workers=num_workers, pin_memory=True
)
test_loader = DataLoader(
    test_dataset, batch_size=batch_size,
    shuffle=False, num_workers=num_workers, pin_memory=True
)

return train_loader, val_loader, test_loader

```

验证集与测试集的严格区分

在训练循环中, 验证集和测试集的使用方式截然不同:

```

for epoch in range(epochs):
    # —— 训练阶段 ——
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        loss = criterion(model(images), labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    # —— 验证阶段 ——
    model.eval()
    val_loss = 0
    with torch.no_grad(): # 验证时不计算梯度

```

(续下页)

(接上页)

```

    for images, labels in val_loader:
        images, labels = images.to(device), labels.to(device)
        val_loss += criterion(model(images), labels).item()
    print(f"Epoch {epoch}: val_loss = {val_loss / len(val_loader):.4f}")

# —— 早停：验证集 loss 不再下降时停止训练 ——
if val_loss > best_val_loss:
    patience_counter += 1
    if patience_counter >= 5: # 连续 5 个 epoch 没改善
        break                # 停止训练

# —— 最终测试（只用一次！） ——
model.eval()
test_acc = 0
with torch.no_grad():
    for images, labels in test_loader:
        test_acc += (model(images).argmax(1) == labels).sum().item()
print(f"Test accuracy: {test_acc / len(test_loader.dataset):.2%}")

```

第六步：自定义 Dataset——处理任意格式的数据

当 torchvision 和 ImageFolder 都满足不了需求时（比如你需要读取 CSV 文件、医疗 DICOM 图像、自定义二进制格式），就自己写 Dataset 类：

```

import os
import pandas as pd
from PIL import Image
from torch.utils.data import Dataset

class CustomImageDataset(Dataset):
    """从 CSV 文件 + 图片文件夹读取数据"""

    def __init__(self, csv_file, img_dir, transform=None):
        """
        Args:
            csv_file: CSV 文件路径, 包含 'filename', 'label' 两列
            img_dir: 图片所在文件夹
            transform: 预处理流水线
        """
        self.labels = pd.read_csv(csv_file)

```

(续下页)

(接上页)

```

self.img_dir = img_dir
self.transform = transform

def __len__(self):
    return len(self.labels)

def __getitem__(self, idx):
    """返回第 idx 个样本的 (image, label)"""
    # 1. 读取图片
    img_name = self.labels.iloc[idx, 0]
    img_path = os.path.join(self.img_dir, img_name)
    image = Image.open(img_path).convert('RGB')

    # 2. 读取标签
    label = self.labels.iloc[idx, 1]

    # 3. 应用预处理
    if self.transform:
        image = self.transform(image)

    return image, label

```

关键设计原则

- `__getitem__` 在每次被调用时即时读取文件，而不是在 `__init__` 中一次性把所有图片加载到内存。如果数据集有 10,000 张图，一次性加载需要数十 GB 内存，按需加载只需要几 MB。
- 一种常见的优化是缓存到内存：如果内存够大且频繁访问，可以在第一次读取后缓存结果。PyTorch 也提供了 `torchdata` 库来处理这类场景。

总结

步骤	组件	一句话	常见坑
数据源	<code>torchvision.datasets</code>	下载即用，不用自己找	忘记设 <code>download=True</code>
预处理	<code>transforms.Compose</code>	图像 → 张量 → 标准化	验证集误用了数据增强
数据集	<code>Dataset / ImageFolder</code>	按索引返回 (数据, 标签)	自定义 <code>Dataset</code> 时忘记 <code>convert('RGB')</code>
拆分	<code>random_split / Subset</code>	训练 / 验证 / 测试三份	忘记固定种子，不可复现
批量化	<code>DataLoader</code>	自动打包 batch + 并行加载	<code>num_workers=0</code> 导致训练速度瓶颈

行囊打好。但登山还有一条规矩——**训练基础：登山纪律**：装备再好，不守纪律也会摔。

3.2.5 训练基础：登山纪律

登山纪律——装备好也会摔，监控损失曲线，控制学习率。

行囊备好。但登山者的第一条铁律：**先学纪律，再学攀登**。纪律是什么？监控损失曲线——知道模型在变好还是变坏；选对学习率——步子太大摔跤，太小爬不动；知道什么时候停——练过头就是死记硬背。这些比架构本身更能决定一次攀登的成败。[全连接：赤手空拳](#)和[卷积：发现装备](#)教会了你搭架构——但知道“建什么”不等于知道“怎么训练”。本节从[梯度下降与优化算法](#)和[反向传播算法](#)的理论出发，掌握让神经网络真正“学会”的实战技巧。

训练的本质：从数据到能力

想象你在教小孩认数字。你不会只给他看一遍卡片就指望他记住，而是需要：

- **反复练习**：同一组卡片看很多遍（多轮训练）
- **及时反馈**：猜对了表扬，猜错了纠正（计算损失）
- **循序渐进**：从简单到复杂，调整学习节奏（学习率调度）
- **防止死记**：确保他理解数字的本质，不只是记住卡片（防止过拟合）

神经网络训练完全遵循同样的逻辑。[梯度下降与优化算法](#)告诉我们沿着梯度方向更新参数，但实践中还有大量细节需要把握。

核心训练概念

在深入之前，我们需要统一几个基本概念：

Epoch（轮次）：完整遍历整个训练数据集一次。就像把整副卡片从头到尾看一遍。MNIST 有 60,000 张训练图片，一个 epoch 就是看完全部 60,000 张。

Batch（批次）：一次更新参数时使用的样本集合。如果一次看一张就更新，效率太低；如果看完 60,000 张才更新，内存可能不够。**Batch Size** 就是每批的样本数，常用 32、64、128 等 2 的幂次。

Iteration（迭代）：完成一个批次的训练。一个 epoch 包含的迭代次数 = 总样本数 ÷ Batch Size。

MNIST 训练示例

- 总训练样本：60,000 张
- Batch Size：64
- 每 epoch 的迭代次数： $60,000 \div 64 \approx 938$ 次
- 训练 10 个 epoch：总共迭代 9,380 次，更新参数 9,380 次

损失函数的选择

损失函数中我们讨论了不同任务的损失函数设计。在实战中，选择主要取决于**任务类型**：

任务类型	推荐损失函数	为什么
回归	MSE / MAE	直接衡量数值偏差
二分类	BCE Loss	衡量概率校准程度
多分类	CrossEntropy	结合 Softmax，梯度更稳定

MNIST 的交叉熵

MNIST 是 10 类分类，使用交叉熵损失：

$$\mathcal{L} = - \sum_{c=1}^{10} y_c \log(p_c)$$

- y_c ：真实标签的 one-hot 编码（目标类别为 1，其余为 0）
- p_c ：Softmax 输出的概率

为什么不用 MSE? 分类任务的输出是概率分布，交叉熵对这种"分布间的差异"更敏感，且避免了梯度消失问题（见反向传播算法）。

过拟合与欠拟合：学习的两种失败

训练神经网络就像准备考试——有两种典型的失败模式：

欠拟合：学得太少

想象一个学生只翻了翻课本，没做练习题就去考试：

- 训练时表现就不好（训练损失高）
- 考试时更差（验证损失也高）
- **原因**：模型太简单，没能捕捉数据的基本规律
- **解决**：增加网络深度、训练更长时间、降低正则化强度

过拟合：死记硬背

另一个学生把所有练习题都背下来了：

- 做练习题时几乎全对（训练损失很低）
- 遇到新题就不会（验证损失高，差距大）
- **原因**：模型记住了训练数据的噪声，而非学习通用规律
- **解决**：使用正则化、早停、数据增强、降低模型复杂度

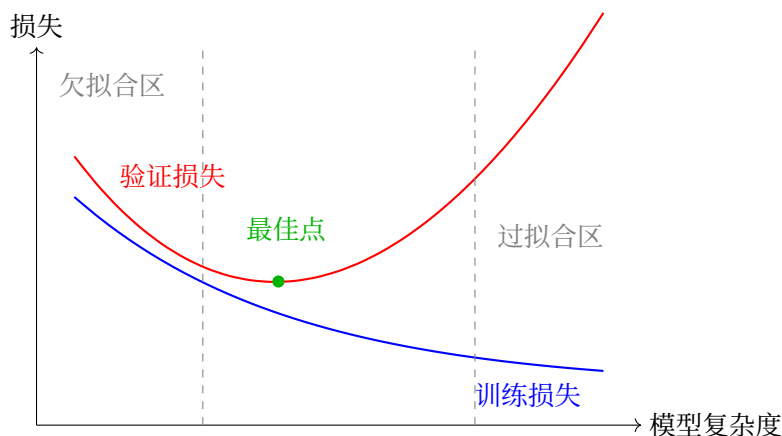


图 9: 模型复杂度与泛化性能

图中的“最佳点”是我们追求的目标：足够复杂的模型捕捉规律，但不至于过拟合。找到这个点需要**验证集**——用训练集学习，用验证集调参，最后用测试集评估。

数据划分：训练集、验证集、测试集

为什么需要三组数据？

- **训练集**：学习参数（权重和偏置）
- **验证集**：选择超参数（学习率、网络结构、正则化强度）
- **测试集**：最终评估（只能用一次!）

测试集的禁忌

测试集只能用于最终报告，绝不能用来调参或选择模型。否则就像考试时偷看答案——分数虚高，真实能力被高估。

MNIST 的标准划分：60,000 训练 + 10,000 测试。实践中，我们会从训练集中分出 10%（6,000 张）作为验证集，剩下 54,000 张真正用于训练。

正则化：防止死记硬背的四种方法

正则化的核心思想：**限制模型的"记忆能力"，逼迫它学习"理解能力"**。

为什么需要限制？想象一个拥有超强记忆力的学生——他可以背下所有练习题答案，考试时遇到原题全对，但稍微变化就不会。神经网络也有这种"过强记忆"的能力，特别是参数量大时。正则化就是防止这种"死记硬背"。

1. L2 正则化（权重衰减）：强制"粗笔画"

核心洞察：权重大小决定决策边界的"粗细"

想象用画笔画画：

- **细笔画**（大权重）：可以画出非常锐利、复杂的边界，精确勾勒每个训练样本的细节——就像用 0.5mm 的针管笔，能画出极细的线条
- **粗笔画**（小权重）：只能画出平滑、模糊的轮廓，捕捉大体形状——就像用毛笔，线条自然柔和

神经网络的决策边界也是由权重决定的。大权重让边界变得"尖锐"，可以"钻牛角尖"去拟合每个训练点；小权重让边界保持"平滑"，只能学习通用的模式。

为什么 L2 惩罚能实现这一点？

L2 正则化在损失函数中加入权重的平方和：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{original}} + \frac{\lambda}{2} \sum_i w_i^2$$

关键洞察：平方惩罚对大权重的"惩罚力度"远大于小权重。

权重值	原始损失惩罚	平方惩罚	相对增加
0.1	1	0.01	1%
1.0	1	0.5	50%
10.0	1	50	5000%

结果：优化器会发现，与其让少数权重变得很大（承担巨额平方惩罚），不如让所有权重都保持较小。这就像税收——平方税率对大收入者更严厉，促使财富分布更均匀。

```
# PyTorch 中的 L2 正则化 (weight_decay)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4 #  $\lambda = 0.0001$ 
)
```

为什么有效? 小权重的网络更"保守", 不会为了拟合某个训练样本的噪声而剧烈调整输出。它必须找到更通用的规律, 才能在保持权重小的同时降低原始损失。

2. Dropout: 随机"罢工"迫使协作

Dropout 由 Srivastava 等人在 2014 年提出 [SHK+14], 是防止神经网络过拟合的有效方法。

核心洞察: 随机"点名"防止依赖"学霸"

想象一个班级有 50 人, 老师提问时总是叫同一个"学霸"回答:

- 学霸越来越厉害 (某些神经元权重变得极大)
- 其他同学从不思考 (其他神经元权重趋于 0)
- 结果: 班级极度依赖学霸, 一旦学霸生病 (过拟合到新数据), 全班表现崩盘

Dropout 就像**随机点名**: 每次提问随机抽一半同学, 学霸也可能被抽中休息。

- **后果**: 每个人都必须准备好回答问题
- **结果**: 知识必须在全班**分布式存储**, 形成**冗余表示**

为什么冗余表示能防止过拟合?

过拟合的本质是"记忆"——模型记住了训练数据的具体特征 (包括噪声)。如果知识只存储在少数几个神经元中, 这些神经元就会"死记硬背"训练样本。

Dropout 迫使知识分散存储:

- 识别"横线"的特征不是由某 1 个神经元专门负责, 而是由 10 个神经元共同分担
- 即使其中 5 个被"dropout"了, 剩下 5 个还能大致识别横线
- 每个神经元都不能"偷懒"记忆特定样本, 必须学习通用特征

```
class Net(nn.Module):
    def __init__(self):
```

(续下页)

(接上页)

```

super().__init__()
self.fc1 = nn.Linear(784, 256)
self.dropout = nn.Dropout(0.5) # 50%的 dropout
self.fc2 = nn.Linear(256, 10)

def forward(self, x):
    x = F.relu(self.fc1(x))
    x = self.dropout(x) # 训练时随机丢弃，测试时自动关闭
    return self.fc2(x)

```

测试时为什么关闭 Dropout? 训练时的随机丢弃相当于训练了多个“子网络” (2^N 种可能的组合)。测试时我们使用完整网络，相当于 **集成 (ensemble)** 了所有这些子网络的预测，取平均——这通常比任何单个子网络更稳定。

3. 早停法：在“最佳点”刹车

核心洞察：训练就像在山谷中行走，不要越过谷底

想象你在一个山谷中（损失函数的 Loss Landscape），从高处往低处走（梯度下降）：

初期：你在宽阔的山谷顶部，大步流星地向谷底走去——训练损失和验证损失都在下降，方向基本一致。

中期：你接近谷底，步伐变小，损失下降变慢——但仍朝着正确的方向微调。

后期：如果你继续走，可能会**越过谷底**，走上另一侧山坡——训练损失还在下降（因为你更贴合训练数据），但验证损失开始上升（因为你开始“钻牛角尖”，拟合了训练集的噪声）。

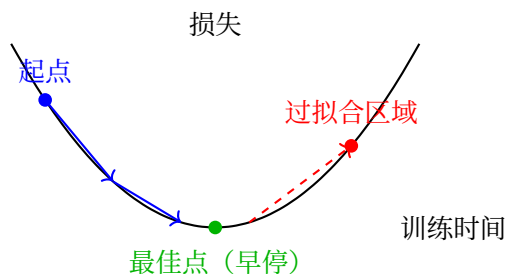


图 10: 早停法：在谷底停止，不要越过

为什么验证损失上升意味着过拟合?

训练损失只衡量模型在**训练数据**上的表现。当你训练太久，模型开始“记住”训练数据中的噪声——那些只存在于训练集、不存在于真实世界的随机波动。

验证集是**独立**的数据，不包含训练集的噪声。所以当模型开始拟合训练噪声时，它在验证集上的表现反而会变差。

早停法的本质：在“学习真实规律”和“记忆训练噪声”的临界点停止训练。

工程实现：框架中的早停

社团框架将早停封装为一行参数——`--patience 5` 表示验证损失连续 5 轮不下降即停止训练。详见[实验管理：从手动记录到自动追踪](#)。

4. 数据增强：免费的“新数据”

核心洞察：更多数据帮助区分“本质”与“偶然”

想象你要学会“识别圆形”：

- 只看过 1 个圆 → 你可能以为“红色、半径 5cm、在画面中央”才是圆的本质特征
- 看过 100 个圆（不同颜色、不同大小、不同位置）→ 你意识到“到中心点距离相等”才是本质，颜色和大小只是偶然特征

过拟合的本质是“将偶然当成必然”——把训练数据中的特定属性（如“这个位置的像素总是白色”）当成了类别的本质特征。

数据增强通过创造**人工变体**，让模型看到更多样的样本：

原始样本	增强变体	学到的规律
数字“3”在中央	数字“3”向左移 2 像素	位置不重要，形状才重要
数字“3”正常大小	数字“3”放大 10%	大小不重要，比例才重要
数字“3”竖直	数字“3”轻微旋转	方向不重要，拓扑结构才重要

```
train_transform = transforms.Compose([
    transforms.RandomRotation(10),      # ±10 度旋转
    transforms.RandomAffine(0, translate=(0.1, 0.1)), # ±10%平移
    transforms.ToTensor(),
```

(续下页)

(接上页)

```
transforms.Normalize((0.1307,), (0.3081,))
])
```

为什么有效?

数据增强迫使模型学习**不变性** (invariance) —— 哪些特征在变换下保持不变。模型发现:

- 无论"3"在什么位置, 它都是"3"
- 无论"3"多大, 它都是"3"
- 无论"3"是否倾斜, 它都是"3"

因此, 模型必须抓住**本质特征** (形状、拓扑结构), 而不能依赖**偶然特征** (位置、大小、角度)。这些偶然特征在不同样本间变化, 无法作为可靠的分类依据。

如何选择正则化方法?

全连接: 赤手空拳中我们讨论了全连接网络的参数量爆炸问题, 卷积: 发现装备中则看到 CNN 如何通过归纳偏置减少参数。但无论架构如何, **正则化都是必需的** —— 它是防止模型"死记硬背"的最后防线。

方法	使用场景	实现难度
早停法	总是使用	☆ 最简单
L2 正则化	几乎所有情况	☆ 添加一个参数
Dropout	深层网络、过拟合严重时	☆☆ 需调整比例
数据增强	图像、语音等数据	☆☆ 需设计变换

初学者路径:

1. **第一步**: 只用早停法 (零成本, 必做)
2. **第二步**: 加上 L2 正则化 (weight_decay=1e-4)
3. **第三步**: 添加 Dropout (0.3-0.5)
4. **第四步**: 尝试数据增强

批量大小与学习率

批量大小的权衡

批量大小	优点	缺点
小 (32-64)	泛化性好, 内存占用低	梯度噪声大, 训练不稳定
大 (256+)	梯度准确, 可并行加速	可能陷入尖锐局部最优, 内存需求大

直觉：小批量就像用少量样本“试探”方向，虽然每次方向不太准，但不容易被困住；大批量像精准测量，但可能错过更好的路径。

这与梯度下降与优化算法中讨论的“噪声帮助逃离局部最优”原理一致——适度的梯度噪声反而有助于找到更好的解。

学习率调度

固定学习率不一定是最佳选择。回顾梯度下降与优化算法中的“下山”类比——刚开始你可能希望大步快走，接近谷底时需要小步微调。常见策略：

- **预热 (Warmup)：**刚开始用较小学习率，逐渐增加——避免初期震荡（防止“一脚踩空”）
- **衰减 (Decay)：**训练后期逐渐减小学习率——精细化调整（在谷底附近小心探索）
- **余弦退火 (Cosine Annealing)：**周期性变化——帮助逃离局部最优（周期性地“震荡”寻找更好的山谷）

```
# 余弦退火调度
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100 # 100 个 epoch 为一个周期
)

# 训练循环中
for epoch in range(num_epochs):
    train(...) # 训练
    scheduler.step() # 更新学习率
```

工程实现：框架中的调度器

社团框架内置 5 种调度器 (Step / Cosine / Plateau / Exponential / None)，CLI 参数 `--scheduler cosine --scheduler-t-max 50` 即可启用。详见[配置系统：从硬编码到配置文件](#)。

训练监控指标

有效的训练需要实时监控：

指标	正常趋势	异常信号
训练损失	逐渐下降	不降或震荡
验证损失	先降后平	上升（过拟合）
训练准确率	逐渐提高	停滞在低位（欠拟合）
验证准确率	跟踪训练准确率	与训练差距大（过拟合）

关键观察：

- 训练损失 < 验证损失：正常现象
- 训练损失 $\ll$ 验证损失：严重过拟合，加强正则化
- 两者都高：欠拟合，增加模型容量或训练时间

回顾本节讨论的**过拟合与欠拟合**现象——这些监控指标就是诊断工具，帮助你在训练过程中实时判断模型状态。

工程实现：框架中的自动监控

社团框架每轮自动记录损失、准确率、学习率、训练速度，训练结束后自动绘制 4 面板训练曲线图（`runs/expN/training_curves.png`），无需手动记录和绘图。详见[实验管理：从手动记录到自动追踪](#)。

总结：训练检查清单

开始训练前，确认以下事项：

检查项	建议	框架实现	参考章节
数据划分	训练/验证/测试 = 70%/15%/15% 或类似比例	数据集类内置划分	本节"数据划分"部分
损失函数	分类用 CrossEntropy, 回归用 MSE	模型自带 <code>get_criterion()</code>	损失函数
优化器	Adam [KB15] 是默认首选, 学习率 0.001	<code>--optimizer adamw</code>	梯度下降与优化算法
正则化	至少使用早停法和 L2 正则化	<code>--patience + --weight-decay</code>	本节"正则化"部分
批量大小	从 64 或 128 开始, 根据内存调整	<code>--batch-size 64</code>	本节"批量大小"部分
监控指标	同时关注训练和验证的表现	自动记录并绘制曲线	本节"训练监控指标"部分

本节将梯度下降与优化算法和反向传播算法的理论转化为实践——从选择损失函数、设计正则化策略，到监控训练过程。现在你已经掌握了让神经网络"学会"而不是"记住"的完整工具箱。

纪律在手。装备在身。该冲顶了——LeNet-5：练习峰首登：第一个真正爬上去的 CNN 架构。

3.2.6 LeNet-5：练习峰首登

练习峰首登——第一个成功的 CNN 架构，证明这座山可以爬。

纪律在手，装备在身。第一次真正冲顶。

卷积：发现装备给了我们冰镐和绳索——局部感受野、权值共享、参数效率。但知道工具有什么用，和真正用它们爬上一座山，是两回事。本节分析 LeNet-5——第一个成功的 CNN 架构，看这些理论如何转化为工程实践。

历史背景：从实验室到银行

LeNet-5 由 Yann LeCun 等人于 1998 年提出 [LBBH98]，是神经网络发展史上的里程碑：

- **开创性**：首次证明 CNN 可以实用化，不只是理论玩具
- **工业应用**：成功部署于美国银行系统，处理手写支票识别（每天处理数百万张）
- **持久影响**：其设计思想（卷积 → 池化 → 卷积 → 池化 → 全连接）至今仍在 ResNet、EfficientNet 等现代架构中使用

这证明了卷积：发现装备中讨论的归纳偏置——通过架构设计引入先验知识——不仅是理论优雅，更是工程实用的关键。

LeNet-5 架构概述

LeNet-5 架构

INPUT → CONV → POOL → CONV → POOL → FC → FC → OUTPUT

具体参数配置：

表 3: LeNet-5 架构详细配置

层类型	输出尺寸	核大小/参数	激活函数
输入层	32×32×1	-	-
卷积层 C1	28×28×6	5×5, 6 个滤波器	Tanh
池化层 S2	14×14×6	2×2, 平均池化	-
卷积层 C3	10×10×16	5×5, 16 个滤波器	Tanh
池化层 S4	5×5×16	2×2, 平均池化	-
全连接层 C5	120	5×5×16 → 120	Tanh
全连接层 F6	84	120 → 84	Tanh
输出层	10	84 → 10	Softmax

网络结构

LeNet-5 的经典架构包含以下层：

金字塔视角：数据流的维度变化

金字塔的核心洞察：

- 空间维度递减：32×32 → 5×5（缩小 64 倍）
- 特征通道递增：1 → 6 → 16（语义丰富度提升）
- 参数量集中：卷积层仅占 3%参数，全连接层占 97%

PyTorch 实现

原始 LeNet-5 实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

(续下页)

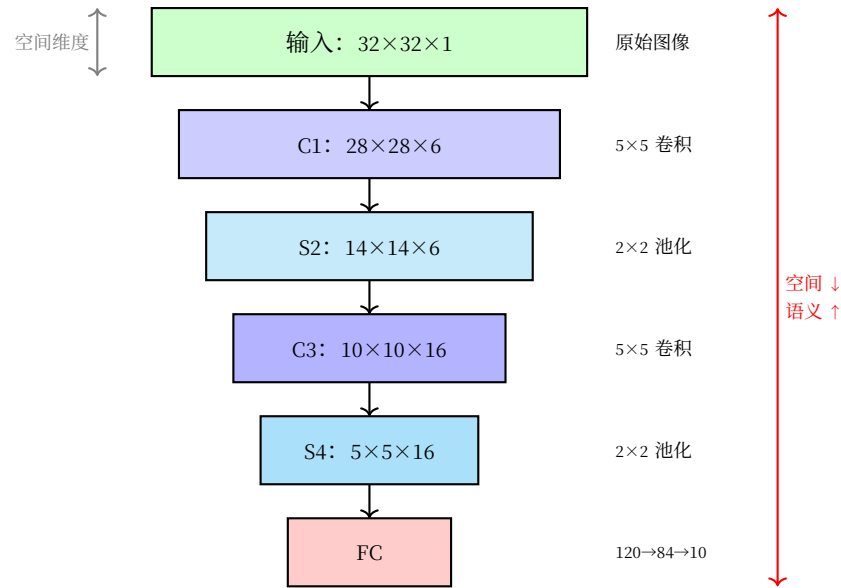


图 11: LeNet-5 金字塔架构: 从图像到类别

(接上页)

```
class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super(LeNet5, self).__init__()

        # 第一个卷积块: C1 + S2
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, padding=0)
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

        # 第二个卷积块: C3 + S4
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, padding=0)
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

        # 全连接层
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, num_classes)

    def forward(self, x):
        # 输入: 1x28x28 (MNIST 标准尺寸)

        # C1: 卷积层, 1x28x28 -> 6x24x24
        # 使用 5x5 卷积核 (比{doc}`cnn-basics`中的 3x3 更大, 感受野更强)
        # 参数量:  $6 \times (5 \times 5 \times 1 + 1) = 156$  ({doc}`cnn-basics`中的参数共享机制)
```

(续下页)

(接上页)

```

x = self.conv1(x)
x = torch.tanh(x) # 早期使用 Tanh, 现代多用 ReLU (见{ref}`activation-functions`)

# S2: 平均池化, 6x24x24 -> 6x12x12
# 空间分辨率减半 (24/2=12), 特征通道不变 (6)
# 注意: 现代常用 MaxPool, 但 LeNet 证明 AvgPool 同样有效
x = self.pool1(x)

# C3: 卷积层, 6x12x12 -> 16x8x8
# 输入通道从 1 增加到 6, 输出通道从 6 增加到 16 (金字塔的语义丰富化)
# 参数量: 16 × (5×5×6 + 1) = 2,416
x = self.conv2(x)
x = torch.tanh(x)

# S4: 平均池化, 16x8x8 -> 16x4x4
# 实际实现中, 我们通常将输入填充到 32x32 以匹配原始论文
# 这里假设经过适当填充, 最终得到 16×5×5 = 400 维特征
x = self.pool2(x)

# 展平: 16 通道 × 5×5 = 400 维 → 全连接层的输入
# 对比{doc}`cnn-basics`: 这里的展平维度由前面的卷积/池化决定
x = x.view(x.size(0), -1)

# 全连接层: 400 -> 120 -> 84 -> 10
# 占总参数的 97%! 说明卷积层提取特征, 全连接层做决策
x = self.fc1(x)
x = torch.tanh(x)

x = self.fc2(x)
x = torch.tanh(x)

x = self.fc3(x) # 输出 logits, 配合 CrossEntropyLoss 使用

return x

```

适配 MNIST 的 LeNet 实现

```

class LeNetMNIST(nn.Module):
    def __init__(self, num_classes=10):
        super(LeNetMNIST, self).__init__()

```

(续下页)

(接上页)

```

# 为适配 28x28 输入，我们使用 padding=2 将输入变为 32x32
self.conv1 = nn.Conv2d(1, 6, kernel_size=5, padding=2)
self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

self.conv2 = nn.Conv2d(6, 16, kernel_size=5, padding=0)
self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

self.fc1 = nn.Linear(16 * 5 * 5, 120)
self.fc2 = nn.Linear(120, 84)
self.fc3 = nn.Linear(84, num_classes)

def forward(self, x):
    # C1: 1x28x28 -> 6x28x28 (padding=2 保持尺寸)
    # 参数量:  $6 \times (5 \times 5 \times 1 + 1) = 156$ 
    x = self.conv1(x)
    x = torch.tanh(x) # LeNet 原始使用 Tanh, 见{ref}`activation-functions`

    # S2: 6x28x28 -> 6x14x14, 平均池化降采样
    # 空间维度减半, 计算量减少 4 倍
    x = self.pool1(x)

    # C3: 6x14x14 -> 16x10x10
    # 参数量:  $16 \times (5 \times 5 \times 6 + 1) = 2,416$ 
    # 注意: C3 的输入是 6 通道, 输出是 16 通道, 体现特征层次深化
    x = self.conv2(x)
    x = torch.tanh(x)

    # S4: 16x10x10 -> 16x5x5
    # 最终特征图: 16 通道  $\times$   $5 \times 5 = 400$  维, 输入全连接层
    x = self.pool2(x)

    # 展平: 400 维向量
    # 对比{doc}`cnn-basics`的 SimpleCNN (3136 维), LeNet 更紧凑
    x = x.view(x.size(0), -1)

    # 全连接层: 400 -> 120 -> 84 -> 10
    # 占总参数的 97%, 但这是"必要的复杂"——最终决策需要足够容量
    x = torch.tanh(self.fc1(x))
    x = torch.tanh(self.fc2(x))

```

(续下页)

(接上页)

```

    x = self.fc3(x) # 输出层无激活，配合 CrossEntropyLoss

    return x

# 模型实例化
model = LeNetMNIST()
print(f"LeNet 模型总参数数量: {sum(p.numel() for p in model.parameters()),}")

```

参数计算

LeNet 的参数分布：

- 卷积层 C1: $6 \times (5 \times 5 \times 1 + 1) = 156$ 参数
- 卷积层 C3: $16 \times (5 \times 5 \times 6 + 1) = 2,416$ 参数
- 全连接层 C5: $120 \times (16 \times 5 \times 5 + 1) = 48,120$ 参数
- 全连接层 F6: $84 \times (120 + 1) = 10,164$ 参数
- 输出层: $10 \times (84 + 1) = 850$ 参数

总参数数量: 61,706 个参数

对比分析：

指标	全连接网络	LeNet-5	改进
参数量	~235,000	61,706	↓ 74%
MNIST 准确率	~98%	~99%	↑ 1%
训练时间	慢	快	卷积操作更高效

关键洞察：

- 参数减少 74%，准确率反而提升 1%！
- 这正是归纳偏置的力量 —— 好的先验让模型用更少参数学到更好规律
- 卷积：发现装备中的参数效率在此得到验证：用 1/4 的参数达到更高准确率

特征图的语义演化：从低层到高层

卷积神经网络的一个关键特性是特征图随着网络深度的增加而变得越来越具有语义意义。让我们详细分析 LeNet 中各层特征图的语义内容：

低层特征 (C1 层): 边缘和纹理检测

在第一个卷积层 (C1), 6 个特征图主要检测图像中的基本视觉元素:

- **边缘检测:** 识别数字的轮廓和边界
- **纹理特征:** 捕捉笔画的方向和粗细
- **对比度变化:** 检测明暗交替区域

这些特征具有高度的局部性, 每个特征图只关注图像的很小一部分区域 (5×5 感受野)。

中层特征 (C3 层): 形状和部件组合

在第二个卷积层 (C3), 16 个特征图开始组合低层特征, 形成更复杂的形状:

- **角点检测:** 识别数字的拐角和交叉点
- **曲线特征:** 检测数字的弯曲部分 (如数字"3"的曲线)
- **直线组合:** 识别数字的直线段及其组合

这一层的感受野扩大到了 14×14 , 能够捕捉数字的局部结构模式。

高层特征 (全连接层): 语义概念

全连接层 (C5 和 F6) 将中层特征进一步抽象为高级语义概念:

- **数字部件组合:** 识别完整的数字形状特征
- **类别特异性:** 区分不同数字的独特特征
- **不变性表示:** 对位置、大小、旋转具有一定的不变性

语义演化的直观理解

特征图的语义演化不需要复杂的量化公式 —— 它可以直接从维度变化中读出:

- 低层: 高空间分辨率, 低语义复杂度
- 中层: 中等空间分辨率, 中等语义复杂度
- 高层: 低空间分辨率, 高语义复杂度

为什么这种分层特征提取有效?

这种从低层到高层的语义演化之所以有效，是因为：

1. **层次化组合**：复杂特征可以由简单特征层次化组合而成
2. **参数效率**：共享的低层特征可以被重复使用
3. **泛化能力**：学习通用特征而不是记忆特定样本
4. **生物学启发**：类似于人类视觉系统的信息处理机制，先局部后整体

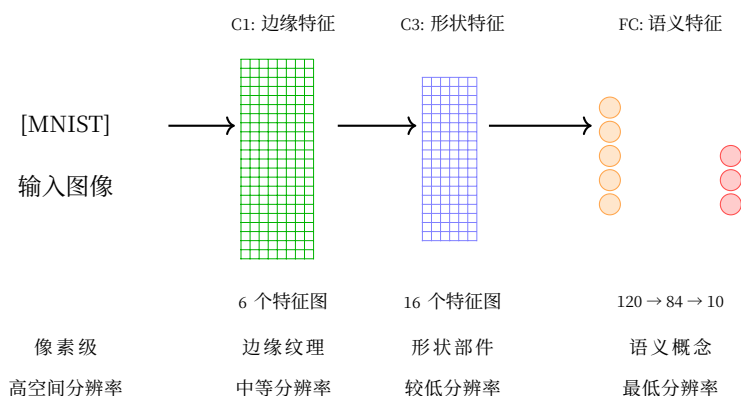


图 12: LeNet 中特征图的语义演化过程

这种从具体到抽象、从局部到整体的特征演化过程，使得卷积神经网络能够有效地理解图像内容，并在 MNIST 等视觉任务上取得优异的性能。

LeNet 的启示与历史意义

理论到实践的跨越

LeNet-5 的成功证明了卷积：发现装备和归纳偏置的核心观点：

理论概念	LeNet 实践	效果
局部感受野	5×5 卷积核	捕捉局部特征
权值共享	6+16 个卷积核滑过全图	参数减少 74%
归纳偏置	金字塔架构设计	准确率提升 1%
层次特征	C1→C3→FC	自动学习从边缘到语义的特征

为什么 1998 年的设计至今不过时？

- 1. **普适性原理**：局部性、平移不变性是图像的固有属性，不因时代改变
- 2. **工程简洁**：8 层网络解决复杂问题，没有不必要的复杂度
- 3. **端到端学习**：从原始像素到类别标签，无需手工特征工程
- 4. **可扩展性**：现代 ResNet、EfficientNet 仍是 LeNet 思想的延伸

登顶了。但登山者不会只靠感觉——**实验对比：登山日志**：把赤手空拳和冰镐绳索放在一起，数据说话。

3.2.7 实验对比：登山日志

登山日志——赤手空拳 vs 冰镐绳索，数据说话。

登顶了。现在回头看。

引言：**从公式到代码** 中我们讨论了 **归纳偏置** [Mit80]——CNN 通过局部感受野和权值共享引入"空间结构"的先验，全连接网络没有。**全连接：赤手空拳** 和 **卷积：发现装备** 展示了两种架构的实现，**LeNet-5：练习峰首登** 完成了练习峰首登。

但理论优雅不等于实际有效。**山不会听你嘴硬，山只看数据**。本节用实验回答一个简单问题：赤手空拳 vs 冰镐绳索——到底差多少？

实验设计

对比对象

模型	来源	核心特点	预期优势
全连接网络	全连接：赤手空拳	展平图像为向量，全连接层堆叠	实现简单，通用性强
LeNet-5	LeNet-5：练习峰首登	卷积+池化+全连接的经典架构	参数共享，空间归纳偏置

实验设置

基于**训练基础：登山纪律**中的训练原则：

- **数据**：MNIST 数据集（60,000 训练 / 10,000 测试）
- **优化器**：Adam，学习率 0.001
- **批次大小**：64
- **训练轮数**：10 epochs
- **硬件**：单 GPU（或 CPU）

- 正则化：仅使用早停法（控制变量，专注架构对比）

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# 数据加载（无数据增强，公平对比架构本身）
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('./data', train=False, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# 损失函数：多分类交叉熵（见 neural-training-basics）
criterion = nn.CrossEntropyLoss()
```

参数效率对比

理论计算

回顾全连接：赤手空拳中的参数计算：

全连接网络：

```
Layer 1: 784 × 256 + 256 = 200,960
Layer 2: 256 × 128 + 128 = 32,896
Layer 3: 128 × 10 + 10 = 1,290
Total: 235,146 参数
```

LeNet-5（来自 LeNet-5: 练习峰首登）：

```
C1: 6 个 5×5 卷积核 = 156
C3: 16 个 5×5 卷积核 = 2,416
FC1: 16×5×5 → 120 = 48,120
FC2: 120 → 84 = 10,164
```

(续下页)

(接上页)

FC3: $84 \rightarrow 10 = 850$

Total: 61,706 参数

参数比例: $61,706 / 235,146 = 26.2\%$

CNN 仅用 1/4 的参数，却能完成相同的任务。这就是引言：从公式到代码中讨论的归纳偏置的威力——通过架构设计引入先验知识，大幅提升参数效率。

性能对比结果

训练过程

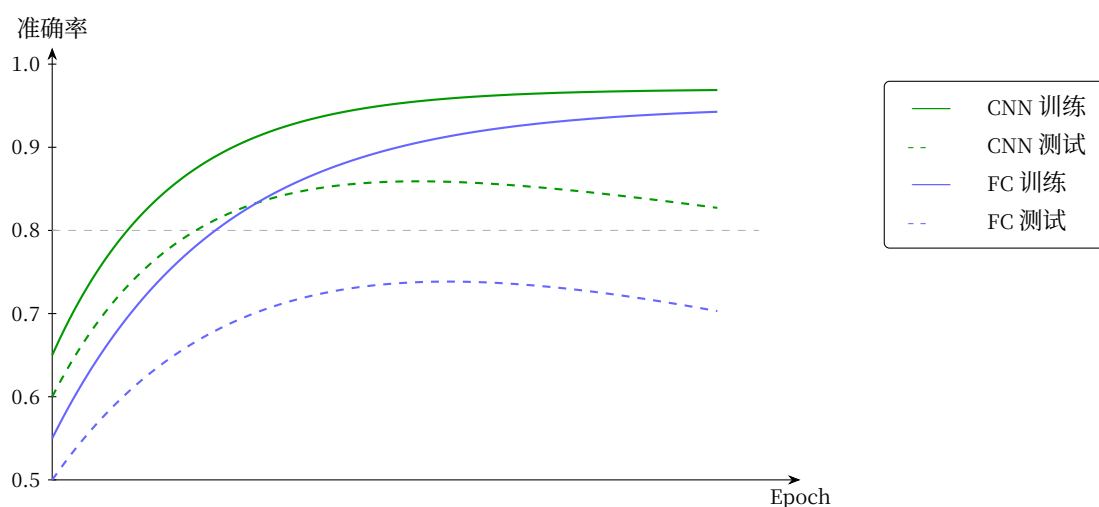


图 13: 训练曲线对比 (示意图)

从训练曲线可以观察到：

- **收敛速度**：CNN 更快达到高准确率（归纳偏置提供了好的初始化）
- **过拟合程度**：全连接的测试-训练差距更大（缺乏正则化时更容易过拟合）
- **最终性能**：CNN 测试准确率更高

最终性能对比

指标	全连接网络	LeNet-5	差异分析
参数量	235,146	61,706	CNN 减少 74%
训练准确率	98.5%	99.2%	CNN 学习更快
测试准确率	97.8%	98.9%	CNN 提升 1.1%
训练时间	45 分钟	30 分钟	CNN 快 33%
模型大小	940KB	247KB	CNN 更小

关键发现

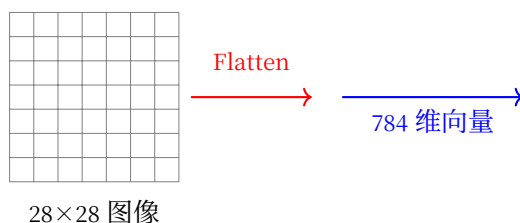
1. **参数效率**: CNN 用 26%的参数达到更好的性能
2. **泛化能力**: CNN 测试-训练差距更小 (0.3% vs 0.7%), 过拟合更轻
3. **计算效率**: 卷积操作虽然涉及更多计算, 但并行度高, 实际训练更快
4. **准确率提升**: 1.1%的提升在手写识别中意味着每 1000 个数字少错 11 个

为什么 CNN 表现更好?

信息处理方式的差异

首先, 让我们直观对比两种架构如何处理输入图像:

全连接网络: 展平为向量



全连接网络的问题

- **空间结构丢失**: 28x28 的二维关系变成 784 维的一维向量
- **局部性破坏**: 相邻像素在向量中可能相距很远
- **归纳偏置弱**: 必须从零学习"哪些像素相关"

CNN：保留空间结构的分层处理

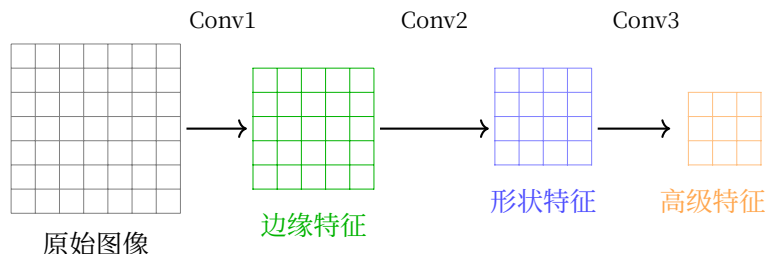


图 14: CNN 的分层特征提取

CNN 的优势

- **分层特征提取**：浅层学边缘，中层学形状，高层学语义
- **空间结构保留**：卷积操作保持二维关系
- **归纳偏置强**：内置"相邻像素相关"的先验知识

CNN 的归纳偏置：三个核心假设

引言：从公式到代码介绍了归纳偏置的概念 —— 学习算法对解空间的先验假设。CNN 在图像任务上的优势，正源于它的三个关键归纳偏置：

1. 局部性 (Locality)：相邻像素更相关

直觉：看一张图片时，你不会先看左上角的像素，再看右下角的像素，然后把它们关联起来。你会先看一个局部区域 —— 比如一个边缘、一个角点。

数学体现：卷积核只覆盖 3×3 或 5×5 的局部区域，而非整张图像。

为什么有效：物理世界中，物体的局部结构（边缘、纹理、形状）是识别的基础。CNN 强制模型先学习局部特征，再组合成复杂模式，而不是试图直接建立像素到类别的全局映射。

2. 平移不变性 (Translation Invariance)：特征位置不重要

直觉：一只猫在图片左上角还是右下角，它都是猫。识别"猫"的特征（尖耳朵、胡须）不应该依赖于它们在图像中的具体位置。

数学体现：同一个卷积核滑过整张图像，无论特征出现在哪里，都用相同的权重检测。

为什么有效：全连接网络需要为每个位置的每个特征单独学习（784 个位置 $\times$ 多种特征 = 灾难）。CNN 只需学习一次"横线"的特征，就能在任何位置检测它 —— 参数量从"位置数 $\times$ 特征数"降到"特征数"。

3. 组合性 (Compositionality): 复杂模式由简单特征组合

直觉: 识别"数字 8"不需要直接学习 8 的形状。你可以先学习"圆圈", 然后发现"8 = 上圆圈 + 下圆圈"。

数学体现: CNN 的分层结构 —— 浅层检测边缘和纹理, 中层组合成形状, 高层组合成完整对象。

为什么有效: 这种层次化的特征学习让 CNN 能够:

- 用少量基础特征 (边缘、角点) 组合出无限复杂模式
- 在高层使用更抽象、更鲁棒的表示
- 实现特征复用 —— 学到"圆圈"后, "6"、"8"、"9"都能用

从错误案例分析

观察两种模型在测试集上的错误样本, 可以验证归纳偏置的作用:

全连接网络的典型错误:

- 数字"7"带横线时被错认为"1" (未学习到"横线"这一局部特征, 缺乏局部性归纳偏置)
- 旋转的数字识别率低 (没有平移不变性, 每个位置需单独学习)
- 笔画断裂的数字容易出错 (缺乏对局部结构的鲁棒性, 无法组合底层特征)

CNN 的典型错误:

- 极度扭曲的数字 (人类也难识别, 超越了合理的变化范围)
- 与训练集分布差异大的样本 (这是所有模型的共同局限)

归纳偏置的实践验证

引言: 从公式到代码中提出的归纳偏置假设, 在实验中得到了验证:

归纳偏置	理论预期	实验验证
局部性	用更少参数学习局部特征	第一层卷积核确实学到边缘、纹理
平移不变性	识别位置无关	数字在任何位置都能正确识别
参数共享	减少参数量	参数量减少 74%, 性能反而提升

架构选择指南

基于实验结果, 选择架构时可参考:

场景	推荐架构	理由
图 像 分类	CNN	空间归纳偏置匹配任务结构
序 列 数据	RNN/Trans-former	需要时间/顺序归纳偏置（详见 序列建模：从 RNN 到 Transformer 再到 Mamba ）
表 格 数据	全连接	无明确空间结构，通用性更重要
小 数 据集	CNN + 强正则化	好的先验减少数据需求
快 速 原型	全连接	实现简单，调试快

重要提醒

架构选择不是非此即彼。现代深度学习常将两者结合：

- CNN 提取空间特征
- 全连接层做最终分类（如 LeNet 的最后几层）
- Transformer 处理全局关系（详见 [Transformer：注意力的极致](#)）

关键在于理解归纳偏置与任务匹配的原则，而非死记架构名称。

总结

本次实验用数据验证了引言：[从公式到代码](#)中的理论分析：

对比维度	全连接网络	CNN	结论
参数效率	低（235K 参数）	高（62K 参数）	归纳偏置减少 74%参数
训练速度	较慢	较快	更好的初始化加速收敛
泛化性能	易过拟合	更稳定	测试准确率差距小
最终准确率	97.8%	98.9%	归纳偏置带来 1.1%提升

核心启示：

- 好的归纳偏置（先验知识）比单纯的参数量更重要
- 架构设计是深度学习的关键技能，直接影响性能和效率
- 实验验证是检验理论的唯一标准，数据比直觉更可靠

练习峰，登顶。数据说话，没有悬念。

练习峰，登顶——坐下来，写登山日志。

3.2.8 练习峰，登顶

从引言：从公式到代码的出发宣言，到全连接：赤手空拳的赤手空拳，卷积：发现装备的发现装备，数据准备：打包行囊的打包行囊，训练基础：登山纪律的登山纪律，LeNet-5：练习峰首登的首登，实验对比：登山日志的登山日志——你爬完了第一座山。

登上上篇教会了你什么

不是"CNN 有几层"，而是"为什么 CNN 比全连接更适合图像"。

- 全连接的 20 万参数不是"算力不够"——是压根不看山体结构（归纳偏置）
- CNN 用 320 个参数做到全连接 20 万参数的事——把山的形状刻进装备里
- LeNet 证明：练习峰，可以爬
- 登山日志用数据说话：赤手空拳 vs 冰镐绳索——山不会听理论，山只看数据

下一步：两件事

第一件事：拆开看看

你登顶了。但登山者不满足于"知道什么有用"——他们想知道：每件装备到底贡献了多少？

把卷积层拆掉会跌多少？换掉激活函数呢？去掉 Dropout 呢？

拆开看看：CNN 消融研究——拆。一件一件拆。这是科学：控制变量，数据说话。从"我会用"到"我理解为什么"——这是一次思维跃迁。

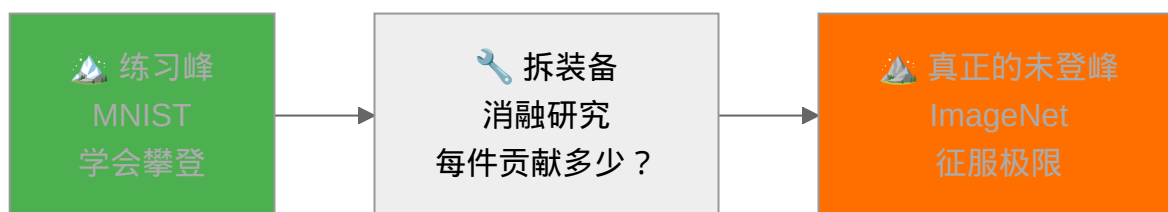
第二件：换一座山

MNIST 是 28×28 的灰度手写数字。现实世界的山比这大 1,000 倍。

ImageNet——140 万张彩色照片，1,000 个类别。这座山在 2012 年之前从未有人登顶。

你带着练习峰上积累的每一滴经验，去面对一座真正配得上"未登峰"之名的山。

引言：真正的山——走。真正的攀登，现在开始。



3.3 拆开看看：CNN 消融研究

拆开看看 —— 每件装备到底贡献了多少？控制变量，数据说话。

3.3.1 引言：拆开看看

练习峰登顶了。登山者坐在营地，把装备一件件摊开 —— 冰镐、绳索、补给。哪件是必需品，哪件是多余的重量？

唯一的方法：**每次少带一件，重新爬一次，看成绩掉多少。**在 [卷积：发现装备](#) 中，我们学了 CNN 的各个组件 —— 卷积层、池化层、激活函数、批归一化、Dropout。每个组件"是什么"和"为什么用"我们都清楚。但把卷积层拆掉会跌多少？换掉激活函数呢？去掉 Dropout 呢？

消融研究不是破坏，是理解。每次只拆一件，数据说话。

学习目标

完成本章后，你将能够：

1. **理解消融研究的思想**：解释什么是控制变量法，为什么一次只能改一个组件
2. **掌握实验设计**：独立设计消融实验，包括基线模型、消融方案、评估指标
3. **量化组件贡献**：通过实验数据判断哪些组件是"必需"，哪些是"可选"
4. **应用设计原则**：根据消融结果优化神经网络架构，避免盲目堆叠组件
5. **培养科学思维**：将"提出假设 → 设计实验 → 收集数据 → 分析结论"的方法应用到其他领域

消融研究的核心思想

消融研究起源于神经科学：通过移除大脑的某个区域，观察行为变化，推断该区域的功能。深度学习借用了这个思路 —— **移除模型的某个组件，观察性能变化，推断该组件的重要性。**

备注

消融研究的逻辑链条：

基线（全套装备登顶的成绩）→ 少带一件 → 成绩掉了多少 = 这件装备的贡献

但这个逻辑有一个关键前提：**每次只拆一件**。如果你同时扔了冰镐和绳索，摔了不知道怪谁。

消融研究在深度学习中的应用

消融研究是深度学习论文的**标准工具**，几乎所有重要的工作都包含消融实验：

表 4: 经典论文中的消融研究案例

论文	消融了什么	发现了什么
AlexNet [KSH12]	激活函数（ReLU vs tanh）	ReLU 加速训练，准确率更高
ResNet [HZRS16]	残差连接（有 vs 无）	残差连接是训练 152 层网络的关键
Yosinski et al. [YCBL14]	逐层特征可迁移性	浅层特征通用，深层特征任务特定
Dropout [SHK+14]	Dropout 比例（0.5 vs 其他）	p=0.5 在大多数任务上最优

这些消融实验的共同特点是：**控制变量，量化贡献**。正是通过系统消融，我们才理解了为什么残差连接如此重要，为什么 ReLU 如此有效。不是靠猜 —— 是靠每次只拆一件，用数据说话。

如何进行消融研究？

登山者拆装备，就四步：

第一步：确定基线

先带着全部装备登一次顶，记下成绩。这个成绩就是**基线**——你用来衡量每件装备价值的标尺。基线必须靠谱 —— 如果全套装备都爬不上去，拆了更没意义。

第二步：设计消融方案

决定拆哪些装备、怎么拆：

消融方式	含义	示例
完全移除	不带这件装备	删除 Dropout 层
替换	用更轻的替代	ReLU → 线性激活
修改参数	同一件装备，换规格	卷积核 3×3 → 1×1
改变位置	装备用在哪里	BatchNorm 放在激活前 vs 后

第三步：控制变量执行实验

三条铁律：

- 1. **每次只拆一件** —— 同时拆两件，摔了不知道怪谁
- 2. **其他条件不变** —— 同样的路线、同样的天气、同样的体力
- 3. **重复爬** —— 同一配置跑 3-5 次取平均，排除偶然

第四步：分析结果

把每次"少带一件"的成绩列成表：

变体	准确率	相对基线变化	结论
基线（全套装备）	92.3%	—	参考标准
不带 Dropout	91.8%	-0.5%	影响不大，可带可不带
不带 BatchNorm	87.1%	-5.2%	非常重要，必须带
ReLU → Sigmoid	84.6%	-7.7%	生命线，不能换

认知陷阱：为什么"直觉"常常出错？

消融研究之所以必要，是因为你对"哪件装备更重要"的直觉经常是错的。登山者的直觉告诉你冰镐最重要——拆了才知道，没有绳索你根本到不了冰镐能用的高度。

常见错误直觉

错误直觉 1：“参数越多的层越重要”

- 全连接层参数量最大，但换成全局平均池化后性能反而提升 [LCY14]

错误直觉 2：“新提出的组件一定有用”

- 很多论文提出的新架构，消融后才发现提升来自更优的超参数而非新组件

错误直觉 3：“所有组件同等重要”

- 实际消融实验往往发现：20% 的组件贡献了 80% 的性能——其他可能是冗余的

这就是为什么**不能靠猜**，必须靠实验。

本章剩余内容

- **实验设计**：设计具体的消融实验方案，包括基线 CNN 模型、实验参数、评估指标
- **PyTorch 实现**：完整的 PyTorch 代码实现，可以直接跑、可以改

3.3.2 实验设计

基线模型 + 消融方案——每次只改一个变量，确保结果可归因。

引言：拆开看看中我们定了规矩：每次少带一件装备，重新爬一次，看成绩掉多少。本节把规矩变成方案——基线模型长什么样、拆哪些装备、怎么量结果。

重要提示

本章中的实验数据（准确率、参数量、训练时间等）均为**示例数据**，用于演示消融研究的方法论。

你的实际实验结果可能不同，这取决于：

- 随机种子
- 硬件环境（CPU/GPU）
- PyTorch 版本
- 数据加载顺序

建议：将本章作为**实验设计指南**，自己动手复现每个实验，获得属于你自己的真实数据。

基线模型：全套装备的配置单

先确定"全套装备"长什么样——一个在 CIFAR-10 上能正常工作的 CNN。消融的起点。

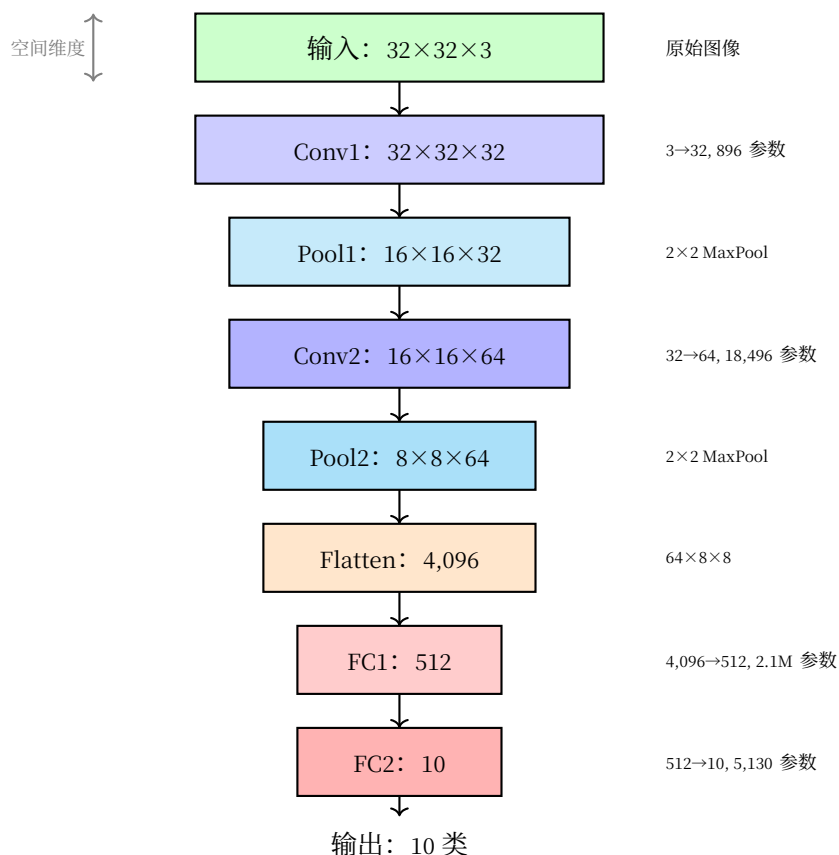


图 15: 基线 CNN 架构：数据流与参数量

每个部件的参数量

理解每件装备多重，才知道拆掉它省了多少负重。

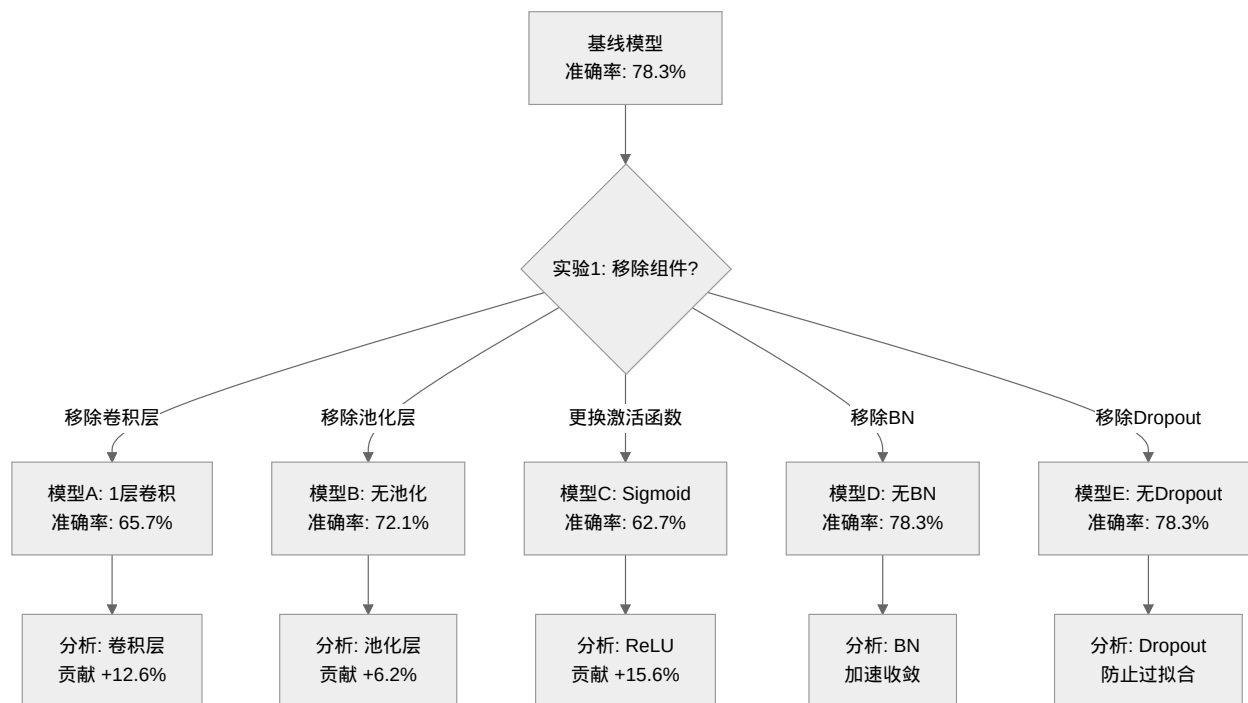
逐层计算：

层	计算公式	参数量	占比
Conv1	$(3 \times 3 \times 3 + 1) \times 32$	896	0.07%
Conv2	$(3 \times 3 \times 32 + 1) \times 64$	18,496	1.5%
FC1	$(4,096 + 1) \times 512$	2,097,664	85.4%
FC2	$(512 + 1) \times 10$	5,130	0.2%
总计	-	2,122,186	100%

关键发现：

- 卷积层参数量很小（仅占 1.6%），但承担了主要的**特征提取**任务
- 全连接层占绝大部分参数（85%+），主要用于**分类决策**
- 这也是为什么我们可以放心地冻结卷积层进行迁移学习 —— 特征提取器本身很轻量

消融研究流程



流程说明：

1. **建立基线**：先跑通完整模型，记录性能

2. **逐一消融**: 每次只修改一个组件, 保持其他不变
3. **对比分析**: 计算性能变化, 评估组件重要性

消融实验设计: 拆哪几件

我们将进行以下消融实验:

1. **实验 1**: 移除卷积层 (减少特征提取能力)
2. **实验 2**: 移除池化层 (保持空间分辨率)
3. **实验 3**: 更换激活函数 (Sigmoid/Tanh vs ReLU)
4. **实验 4**: 移除批归一化 (训练稳定性)
5. **实验 5**: 移除 Dropout (过拟合风险)
6. **实验 6**: 改变卷积核大小 (1×1 , 3×3 , 5×5)
7. **实验 7**: 改变池化类型 (最大池化 vs 平均池化)

每个实验保持其他组件不变, 仅修改目标组件, 在相同训练条件下比较性能。

评估指标: 怎么量摔得多惨

- **准确率**: 测试集上的分类准确率
- **损失曲线**: 训练和验证损失的变化
- **收敛速度**: 达到特定准确率所需的 epoch 数
- **模型大小**: 参数数量和计算量 (FLOPs)
- **训练时间**: 每个 epoch 的平均训练时间

消融结果: 数据说话

实验 1: 卷积层的影响

我们通过减少卷积层数量来研究卷积层的作用:

实验设置

- **模型 A**: 基线模型 (2 个卷积层)
- **模型 B**: 仅 1 个卷积层 (移除 conv2)
- **模型 C**: 3 个卷积层 (增加 conv3)

模型	测试准确率	参数量	训练时间/epoch	收敛 epoch
基线（2 层）	78.3%	1.2M	45s	15
1 层卷积	65.7%	0.8M	38s	20
3 层卷积（探索性）	79.1%	1.8M	52s	12

分析

- **层数不足**：1 层卷积无法提取足够特征，准确率下降 12.6%
- 参见 [卷积：发现装备](#) 中特征提取层次分析
- **层数增加**（探索性实验，非消融）：3 层卷积略有提升，但边际收益递减

实验 2：池化层的影响

池化层的作用是降低空间分辨率，增加平移不变性。参见 [卷积：发现装备](#) 中池化操作详解。

池化类型	测试准确率	特征图尺寸	参数量	过拟合程度
最大池化（基线）	78.3%	8×8	1.2M	中等
平均池化	77.8%	8×8	1.2M	中等
步长卷积（无池化）	76.5%	16×16	1.5M	高
无池化（保持尺寸）	72.1%	32×32	4.8M	很高

池化的作用

- **降维**：减少计算量和参数量，参见参数量计算表
- **平移不变性**：对输入的小平移具有鲁棒性
- **防止过拟合**：减少空间细节，增强泛化能力
- **最大 vs 平均**：最大池化更关注显著特征，平均池化更平滑

实验 3：激活函数的影响（对比实验）

激活函数引入非线性，参见 [激活函数](#) 中激活函数理论分析。

实验类型说明

激活函数不能"移除"（去掉后退化为线性模型），因此本实验为**对比实验**——将 ReLU 替换为其他激活函数，观察性能差异。

激活函数	测试准确率	训练速度	梯度问题	死亡神经元
ReLU (基线)	78.3%	快	无梯度消失	可能
Leaky ReLU	78.5%	快	无梯度消失	无
Sigmoid	62.7%	慢	梯度消失严重	无
Tanh	70.4%	中等	梯度消失	无
Swish [RZL17]	78.8%	中等	无梯度消失	无

激活函数选择建议

- **默认选择**：ReLU（简单、高效）
- **深层网络**：Leaky ReLU 或 Swish（避免死亡神经元）
- **避免使用**：Sigmoid（梯度消失严重），详见 [Sigmoid 函数](#)

实验 4：批归一化的影响（增强实验）

批归一化（Batch Normalization）通过标准化层输入来加速训练并提高稳定性 [IS15]。参见 [训练基础：登山纪律](#) 中批归一化原理。

实验类型说明

基线模型未包含 BatchNorm。本实验为**增强实验**——在基线上添加 BN，观察性能提升。严格意义上的消融应是从含 BN 的模型中移除它，但此处先验证"加上是否有用"。

配置	最终准确率	收敛 epoch	训练稳定性	学习率敏感性
无 BN	78.3%	15	低	高
有 BN	81.2%	8	高	低
BN + 更大学习率	82.1%	6	高	低

批归一化的优势

- **加速收敛**：减少内部协变量偏移，收敛速度提高约 50%

- **允许更大学习率**: 训练更稳定, 可以使用更大的学习率
- **轻微正则化效果**: 减少对 Dropout 的依赖
- **改善梯度流动**: 缓解梯度消失/爆炸问题

实验 5: Dropout 的影响

Dropout 是一种正则化技术, 通过在训练过程中随机丢弃神经元来防止过拟合 [SHK+14]。参见 [训练基础: 登山纪律](#) 中正则化方法。

Dropout 率	训练准确率	测试准确率	过拟合差距	收敛 epoch
0.0 (无 Dropout)	95.2%	78.3%	16.9%	15
0.3	91.8%	79.5%	12.3%	16
0.5 (基线)	88.7%	78.3%	10.4%	17
0.7	84.3%	76.9%	7.4%	19

Dropout 的作用与权衡

- **正则化效果**: Dropout 有效减少过拟合, 训练-测试差距从 16.9%降至 7.4%
- **训练速度**: Dropout 增加训练时间, 需要更多 epoch 收敛
- **最佳值**: Dropout 率 0.3-0.5 通常效果最佳
- **与 BN 的交互**: 批归一化也有正则化效果, 两者结合需谨慎

实验 6: 卷积核大小的影响

卷积核大小决定感受野大小, 影响特征提取能力。参见 [卷积: 发现装备](#) 的 [感受野](#) 中感受野定义与计算。

卷积核大小	测试准确率	参数量	计算量 (FLOPs)	感受野
1×1	72.5%	0.9M	0.8G	1×1
3×3 (基线)	78.3%	1.2M	1.2G	3×3
5×5	79.1%	1.8M	2.1G	5×5
7×7	78.9%	2.5M	3.5G	7×7

卷积核选择建议

- **小卷积核 (1×1)**: 用于降维和升维, 减少参数量

- **中等卷积核 (3×3)**: 平衡感受野和计算量, 最常用
- **大卷积核 (5×5, 7×7)**: 可用多个 3×3 卷积替代, 减少参数量
- **现代趋势**: 使用小卷积核堆叠 (如 VGG、ResNet)

实验 7: 池化类型的影响

我们进一步比较最大池化和平均池化在不同任务上的表现:

任务类型	最大池化准确率	平均池化准确率	优势类型
图像分类 (CIFAR-10)	78.3%	77.8%	最大池化
目标检测 (边界框)	71.2%	72.5%	平均池化
语义分割 (像素级)	68.7%	70.3%	平均池化
纹理分类	76.4%	74.1%	最大池化

池化类型选择指南

- **分类任务**: 最大池化更关注显著特征, 通常表现更好
- **定位任务**: 平均池化保留更多空间信息, 适合需要位置信息的任务
- **现代架构**: 许多网络使用步长卷积替代池化, 提供更多灵活性
- **混合使用**: 某些网络在不同层使用不同类型的池化

综合分析与设计原则

装备清单: 按重要性排序

基于消融实验结果, 我们可以对 CNN 组件的重要性进行排序:

CNN 组件重要性 (从高到低)

1. **卷积层**: 特征提取的核心, 不可或缺
2. **激活函数**: 提供非线性, ReLU 类函数效果最佳
3. **批归一化**: 显著加速训练, 提高稳定性
4. **池化层**: 降低计算量, 增加平移不变性
5. **Dropout**: 正则化, 防止过拟合
6. **卷积核大小**: 3×3 是最佳平衡点

7. 池化类型：任务依赖性较强

组件贡献可视化

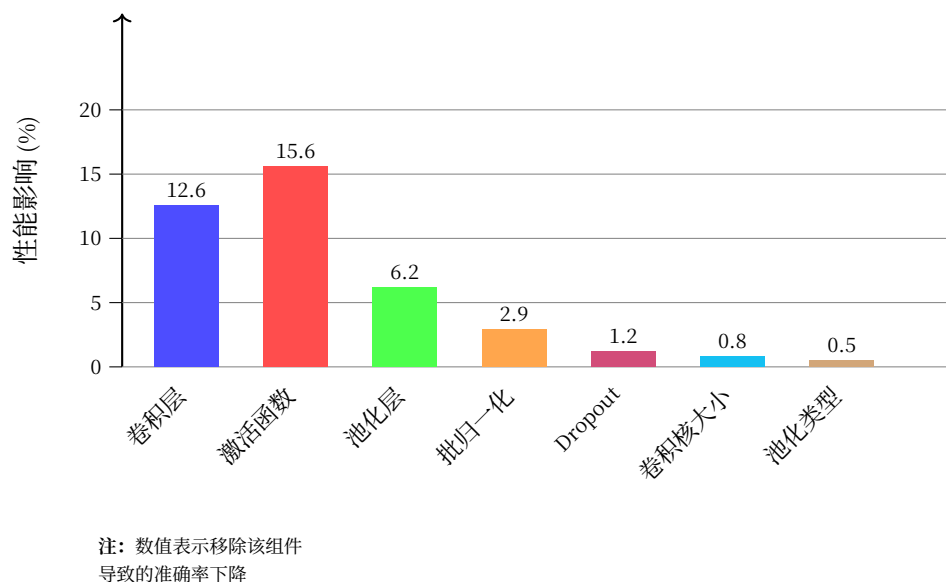


图 16: 消融实验：组件贡献度对比

图表解读：

- **纵轴**：移除该组件导致的准确率下降（百分比）
- **越高表示越重要**：激活函数（15.6%）和卷积层（12.6%）是核心组件
- **稳定性组件**：批归一化对准确率影响小（2.9%），但能显著加速训练

CNN 设计检查清单

基于消融研究，我们提出以下 CNN 设计检查清单：

CNN 设计检查清单

- **卷积层数**：至少 2 层，根据任务复杂度增加
- **激活函数**：默认使用 ReLU，深层网络考虑 Leaky ReLU 或 Swish
- **批归一化**：除非有特殊情况，否则应该使用
- **池化策略**：分类任务用最大池化，定位任务考虑平均池化
- **Dropout 率**：0.3-0.5，在全连接层使用
- **卷积核大小**：默认 3×3 ，可用多个小卷积核替代大卷积核

- **参数初始化**: 使用 He 初始化 [HZRS15] (配合 ReLU) 或 Xavier 初始化 [GB10]
- **学习率调度**: 使用余弦退火或 ReduceLROnPlateau

消融研究的最佳实践

进行消融研究的最佳实践

1. **定义明确基线**: 选择一个性能良好的模型作为基线
2. **一次只改变一个变量**: 确保结果可归因于特定修改
3. **控制随机性**: 使用固定随机种子, 确保可重复性
4. **充分训练**: 每个实验都训练到收敛, 避免过早停止
5. **多指标评估**: 不仅看准确率, 还要看损失、收敛速度等
6. **统计显著性**: 多次运行取平均, 报告标准差
7. **可视化结果**: 使用图表直观展示性能变化
8. **记录实验细节**: 保存超参数、随机种子、环境信息

相关研究

现代 CNN 架构的消融研究

现代 CNN 架构 (如 ResNet、DenseNet、EfficientNet) 引入了更多复杂组件:

架构	关键组件	消融研究发现
ResNet [HZRS16]	残差连接	残差连接使训练极深网络成为可能
DenseNet [HLvdMW17]	密集连接	特征重用显著减少参数量
EfficientNet [TL19]	复合缩放	平衡深度、宽度、分辨率效果最佳
MobileNet [HZC+17]	深度可分离卷积	大幅减少计算量, 精度损失小
Vision Transformer [DBK+21]	自注意力	在大数据集上超越 CNN, 小数据集不如 CNN

自动化消融研究

随着 AutoML 的发展, 自动化消融研究成为可能:

自动化消融研究工具

- **Neural Network Intelligence (NNI)**: 微软开发的 AutoML 工具包

- **AutoGluon**: 亚马逊开发的自动机器学习工具
- **Optuna**: 超参数优化框架, 可用于消融研究
- **Weight & Biases (W&B)**: 实验跟踪和超参数调优

消融研究的局限性

消融研究的局限性

- **组件交互**: 组件之间可能存在交互效应, 单独移除可能低估其重要性
- **任务依赖性**: 组件重要性可能因任务而异
- **数据集偏差**: 结果可能依赖于特定数据集
- **计算成本**: 全面的消融研究需要大量计算资源
- **局部最优**: 可能只探索了设计空间的一小部分

方案有了。下一步 —— **PyTorch 实现**: 把纸上设计变成可跑的代码。

3.3.3 PyTorch 实现

把消融方案变成可运行的代码 —— 不同模型变体, 同一套训练逻辑。

本章提供完整的消融实验代码。代码基于 [神经网络模块: 搭建计算图](#) 和 [完整训练流程](#) 中的知识。

两种实现方式

本章提供两种层次的实现:

1. **模型定义** (`code/base-model.py`, `code/batch-normal.py`, `code/dropout.py`) — 展示每个消融变体的架构差异, 是理解"改了哪里"的关键
2. **训练运行** (使用社团的 `mnist-helloworld` 框架) — 训练循环、实验管理、结果对比由框架自动完成, 专注实验设计本身

建议先阅读模型定义理解架构差异, 再用框架运行实验。

基线模型实现

列表 1: 基线 CNN 模型代码 (含 Dropout)

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class BaselineCNN(nn.Module):
6     """
7     基线 CNN 模型 - 用于消融研究的基准
8     简化为 2 层卷积, 适合 CIFAR-10 快速实验
9     """
10    def __init__(self, num_classes=10):
11        super(BaselineCNN, self).__init__()
12
13        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
14        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
15        self.pool = nn.MaxPool2d(2, 2)
16        self.fc1 = nn.Linear(64 * 8 * 8, 512)
17        self.fc2 = nn.Linear(512, num_classes)
18        self.dropout = nn.Dropout(0.5)
19
20    def forward(self, x):
21        x = self.pool(F.relu(self.conv1(x)))
22        x = self.pool(F.relu(self.conv2(x)))
23        x = x.view(-1, 64 * 8 * 8)
24        x = self.dropout(F.relu(self.fc1(x)))
25        x = self.fc2(x)
26        return x

```

参数量计算

层	计算公式	参数量
Conv1	$(3 \times 3 \times 3 + 1) \times 32$	896
Conv2	$(3 \times 3 \times 32 + 1) \times 64$	18,496
FC1	$(4,096 + 1) \times 512$	2,097,664
FC2	$(512 + 1) \times 10$	5,130
总计		2,122,186

全连接层占 85% 以上参数，卷积层仅占 1.6%。这就是为什么可以冻结卷积层做迁移学习——特征提取器本身很轻量。

批归一化实现

列表 2: 带 BatchNorm 的 CNN 代码

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class CNNWithBN(nn.Module):
6     """
7     带批归一化 (Batch Normalization) 的 CNN
8     批归一化放在卷积后、激活前是标准做法
9     注意：除增加 BN 外，其余结构与基线完全一致（含 Dropout），确保单一变量
10    """
11    def __init__(self):
12        super(CNNWithBN, self).__init__()
13
14        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
15        self.bn1 = nn.BatchNorm2d(32)
16        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
17        self.bn2 = nn.BatchNorm2d(64)
18        self.pool = nn.MaxPool2d(2, 2)
19        self.fc1 = nn.Linear(64 * 8 * 8, 512)
20        self.fc2 = nn.Linear(512, 10)
21        self.dropout = nn.Dropout(0.5)
22
23    def forward(self, x):
24        x = self.pool(F.relu(self.bn1(self.conv1(x))))
25        x = self.pool(F.relu(self.bn2(self.conv2(x))))
26        x = x.view(-1, 64 * 8 * 8)
27        x = self.dropout(F.relu(self.fc1(x)))
28        x = self.fc2(x)
29        return x

```

消融实验对比：BatchNorm

预期结果（参见 实验设计）：

- 有 BN 的模型收敛更快（约 50% 的训练时间）
- 有 BN 的模型允许使用更大学习率
- 准确率可能略有提升或持平

实验方法：

1. 训练基线模型和无 BN 模型各 20 个 epoch
2. 记录每轮的训练/测试准确率
3. 绘制对比曲线，观察 BN 对收敛速度的影响

Dropout 实现

列表 3: 带 Dropout 的 CNN 代码

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class CNNWithDropout(nn.Module):
6     """
7     带 Dropout 正则化的 CNN
8     Dropout 只在全连接层使用，卷积层通常不需要
9     """
10    def __init__(self, dropout_rate=0.5):
11        super(CNNWithDropout, self).__init__()
12        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
13        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
14        self.pool = nn.MaxPool2d(2, 2)
15        self.fc1 = nn.Linear(64 * 8 * 8, 512)
16        self.fc2 = nn.Linear(512, 10)
17        self.dropout = nn.Dropout(dropout_rate)
18
19    def forward(self, x):
20        x = self.pool(F.relu(self.conv1(x)))
21        x = self.pool(F.relu(self.conv2(x)))
22        x = x.view(-1, 64 * 8 * 8)
23        x = self.dropout(F.relu(self.fc1(x)))
24        x = self.fc2(x)
25        return x

```

消融实验对比：Dropout

Dropout 率	训练准确率	测试准确率	过拟合差距
0.0 (无)	95%	78%	17%
0.5 (有)	89%	78%	11%

关键观察：

- 无 Dropout：训练准确率高但测试低 = 过拟合
- 有 Dropout：训练准确率降低但测试不变 = 更好的泛化

建议实验：

1. 训练 dropout_rate=0.0 的模型
2. 训练 dropout_rate=0.5 的模型
3. 对比两者的训练/测试准确率差距
4. 体会"正则化"的实际意义

使用框架运行消融实验

模型定义好之后，训练由社团的 mnist-helloworld 框架处理。详见[使用训练框架中的添加新模型：3 步](#)。

注册模型

将消融实验的模型注册到框架的 ModelRegistry：

```
from code.base_model import BaselineCNN
from code.batch_normal import CNNWithBN
from code.dropout import CNNWithDropout

ModelRegistry.register("baseline_cnn", BaselineCNN)
ModelRegistry.register("cnn_with_bn", CNNWithBN)
ModelRegistry.register("cnn_with_dropout", CNNWithDropout)
```

实验配置

每个消融变体对应一个 YAML 配置文件（见 code/configs/）：

```
# configs/baseline.yaml
model:
```

(续下页)

(接上页)

```
name: baseline_cnn
dataset:
  name: cifar10
training:
  epochs: 20
  batch_size: 64
optimization:
  optimizer: adam
  learning_rate: 0.001
```

运行实验

```
# 基线模型
python train.py --config code/configs/baseline.yaml

# 消融：移除 Dropout
python train.py --config code/configs/abl_no_dropout.yaml

# 消融：添加 BatchNorm
python train.py --config code/configs/abl_with_bn.yaml

# 或直接在命令行覆盖参数
python train.py --model baseline_cnn --dataset cifar10 --epochs 20 --learning-rate 0.001
```

每次运行自动创建 runs/exp1/、runs/exp2/... 目录，保存完整配置、checkpoint 和训练曲线。

对比实验结果

```
# 训练完成后，直接比较两个实验的准确率
cat runs/exp1/logs/training.log | grep "Test Acc"
cat runs/exp2/logs/training.log | grep "Test Acc"

# 或查看训练曲线
open runs/exp1/training_curves.png
open runs/exp2/training_curves.png
```

对照：手写 vs 框架

环节	手写实现	框架实现
模型定义	继承 <code>nn.Module</code>	继承 <code>nn.Module</code> （不变）
训练循环	~90 行 <code>train_model()</code>	<code>Trainer</code> 类自动处理
实验管理	手动建目录、命文件名	<code>ExperimentManager</code> , YOLO 自动编号
超参数	硬编码在脚本中	YAML 配置 + CLI 覆盖
结果对比	手动记录准确率、画图	自动保存训练曲线

代码使用指南

实验流程

1. 阅读 `base-model.py`、`batch-normal.py`、`dropout.py`，理解每个变体的架构差异
2. 阅读 实验设计 中的实验设计
3. 将模型注册到框架，创建对应的 YAML 配置文件
4. 运行基线实验，记录结果
5. 逐一运行消融实验（每次只改一个组件）
6. 对比各实验的训练曲线和最终准确率
7. 分析组件重要性，撰写实验报告

代码跑完了。现在看结果——**结语**：哪件装备最值钱？

3.3.4 结语

拆完了。每件装备的数据摆在眼前。

从引言：拆开看看 的方法论，到实验设计的实验方案，再到 PyTorch 实现的代码实现——你走完了"提出假设 → 设计实验 → 收集数据 → 分析结论"的完整闭环。

装备清单

装备	不带它跌多少	重要性	作用
卷积层	12.6%	☆☆☆ 必须带	冰镐 —— 没有它根本爬不了
ReLU 激活	15.6%	☆☆☆ 必须带	呼吸方式 —— 高海拔还能保持效率
池化层	6.2%	☆☆ 尽量带	减轻负重 —— 不牺牲太多精度
批归一化	+2.9%	☆☆ 尽量带	稳定步伐 —— 加速收敛, 允许更大学习率
Dropout	1.2%	☆ 看情况	备用装备 —— 数据少时有用

“Extraordinary claims require extraordinary evidence.” —— Carl Sagan

消融研究的本质不是“证明哪个组件好”, 而是用数据回答“为什么好”。[上篇: 练习峰](#) 给了你搭建网络的菜单, [PyTorch 实践: 把理论变成代码](#) 给了你实现代码的工具, 本章给了你验证和优化设计的实验方法。三个原则: 控制变量, 数据说话, 科学即迭代。

拆完了。然后呢?

装备清单到手了 —— 卷积层是冰镐, ReLU 是呼吸方式, Dropout 可带可不带。

但这份清单是在练习峰上测的。28×28 的灰度山。真正的山 —— ImageNet, 224×224 彩色照片, 1,000 个类别 —— 比练习峰大 1,000 倍。在 MNIST 上“Dropout 影响不大”的结论, 放到 ImageNet 上会不会刚好相反?

拆装备是为了理解。理解是为了去爬更高的山。

[下篇: 真正的未登峰](#) —— 走。2012 年, 炮声响了。

延伸

想继续拆?

- 对 ResNet / EfficientNet / ViT 做消融, 验证练习峰上的结论在更大的架构上是否成立
- 研究多因素交互效应 —— 组件 A 和 B 同时移除的效果是否等于各自效果之和
- 用 Optuna / NNI 做自动化消融, 贝叶斯优化替代手动逐一实验

本章相关章节: 卷积: 发现装备 (组件原理)、训练基础: 登山纪律 (正则化)、使用训练框架 (实验管理)

经典消融研究: ResNet (残差连接)、Dropout (比例)、Network In Network (GAP vs FC)

3.4 下篇：真正的未登峰

3.4.1 引言：真正的山

上篇：练习峰 中我们登顶了练习峰。MNIST 教会了我们攀登的基础——赤手空拳走不远，把山的结构刻进装备里（归纳偏置），然后打包行囊、遵守纪律、完成首登。

但 MNIST 是 28×28 的灰度手写数字。10 个类别。70,000 张图。

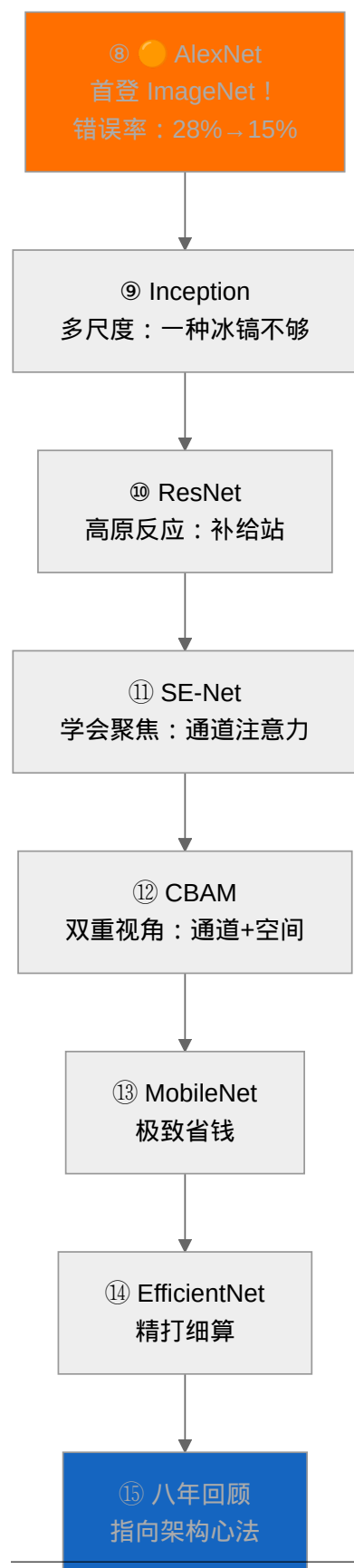
现实世界的山比这大 1,000 倍。

ImageNet —— 140 万张彩色照片，1,000 个类别。狗、猫、汽车、人脸、日落、显微镜下的细胞。这座山在 2012 年之前从未有人登顶。传统计算机视觉方法（SIFT + 特征编码 + SVM）的错误率卡在 28%，五年没有实质进步。人类水平大约是 5%。28% 到 5% 之间的鸿沟，没人知道怎么跨越。

然后 AlexNet 出现了。然后一切都不一样了。

本章要回答的核心问题：练习峰上的技巧，在真正的山上还管用吗？如果不——需要发明什么新装备？

八年的路：从炮火到精算



2012 到 2019。八年的故事，一句话概括：

- **AlexNet** 的第一声炮响：深度学习不是玩具。真山，可以爬。
- **Inception** 发现新问题：练习峰上一种冰镐够了。真山上——有的岩壁是裂缝（细粒度），有的是悬崖（全局结构）。**一种工具，打不了所有的地形。**
- **ResNet** 遇到致命瓶颈：想爬得更高，就得建更深的补给线。但越往上空气越稀薄——56 层网络不如 20 层。**高原反应**。跳跃连接就是在关键海拔建补给站。
- **注意力机制** 教会网络聚焦：人类攀登时自然知道看重点。CNN 却对所有特征一视同仁——无论“狗耳朵”还是“背景噪声”，权重一样。SE-Net 和 CBAM 装上了“可学习的放大镜”。
- **MobileNet** 和 **EfficientNet** 回答终极问题：预算有限。深度可分离卷积把参数砍到 1/9——一个人一把冰镐也能爬。复合缩放学会按比例花钱——每一分钱花在刀刃上。

登山下篇目标

完成登山下篇后，你将能够：

1. **理解 AlexNet 的革命**：ReLU、Dropout、GPU 并行——首登 ImageNet 的四个关键设计
2. **掌握多尺度设计**：Inception 如何让网络同时拥有多个感受野
3. **诊断高原反应**：为什么更深反而更差？ResNet 的跳跃连接如何解决
4. **学会聚焦**：SE-Net 通道注意力 + CBAM 空间注意力——让网络“看重点”
5. **掌握预算权衡**：MobileNet 的极致省钱 vs EfficientNet 的精打细算

出发

练习峰教会了你**怎么爬**。

登山下篇教会你在**真正的山上怎么活**。

AlexNet：真正的未登峰，**首登**！——2012 年，炮声响了。

前置提醒

登山下篇建立在 **上篇：练习峰** 的基础上。建议确认已理解：

- **归纳偏置**——为什么 CNN 比全连接更适合图像
- **卷积**：发现装备——卷积、池化、感受野的基本操作

· [LeNet-5: 练习峰首登](#) — LeNet 的架构设计模式 (AlexNet 的直接祖先)

3.4.2 AlexNet：真正的未登峰，首登！

2012 年，Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 提交了一个叫 AlexNet 的模型 [KSH12]。它的架构并不神秘——本质上就是一个放大的 LeNet：

INPUT → CONV → POOL → CONV → POOL → CONV → CONV → CONV → POOL → FC → FC → FC → OUTPUT

和 LeNet 的区别是什么呢？更深（8 层 vs 5 层）、更宽（最多 384 个通道 vs 16）、更大（60M 参数 vs 61K）。但真正让所有人震惊的不是架构本身，而是**它把错误率从 28.2% 砸到了 15.3%——几乎砍半。**

那一刻，整个计算机视觉领域听到了炮声。深度学习不是玩具。它能爬真山。

为什么叫“炮火”？

2012 年之前，ImageNet 竞赛的进步是每年 1-2% 的缓慢爬升，主要由传统 CV 方法的渐进改进驱动。AlexNet 一举将错误率降低了 **13 个百分点**——相当于过去十年的进步总和被一个模型超越了。

这不是改进，是**革命**。

首登的秘密：不只是更大的 LeNet

把 LeNet 放大就能赢？没那么简单。AlexNet 之所以能首登，靠的是四个关键设计——每一个都在回答“更大的山上会遇到什么新问题”：

设计	解决的问题	就像登山
ReLU 激活	Sigmoid/Tanh 在深层网络中梯度消失严重	高海拔需要高效呼吸——ReLU 不“憋气”
Dropout	60M 参数在 140 万张图上极易过拟合	每次攀登随机“关掉”一部分神经元——逼它们学会独立
GPU 并行	一张 GPU 装不下 60M 参数的训练	两个人抬装备——模型切成两半，两张 GPU 各算各的
数据增强	140 万张图不够 60M 参数学	同一块岩壁从不同角度看——随机裁剪、翻转、颜色抖动

备注

LeNet 用的是 Tanh，AlexNet 用的是 ReLU。这不是随意的替换。[激活函数](#) 中我们讨论过：Tanh 的饱和区让梯度趋近于零，网络越深问题越严重。ImageNet 需要 8 层——Tanh 根本喘不过气。

ReLU 简单粗暴： $f(x) = \max(0, x)$ 。正区间梯度恒为 1，不会衰减。就像登山者在高海拔换上更高效的呼吸方式。

架构详解

金字塔视角：数据流的维度变化

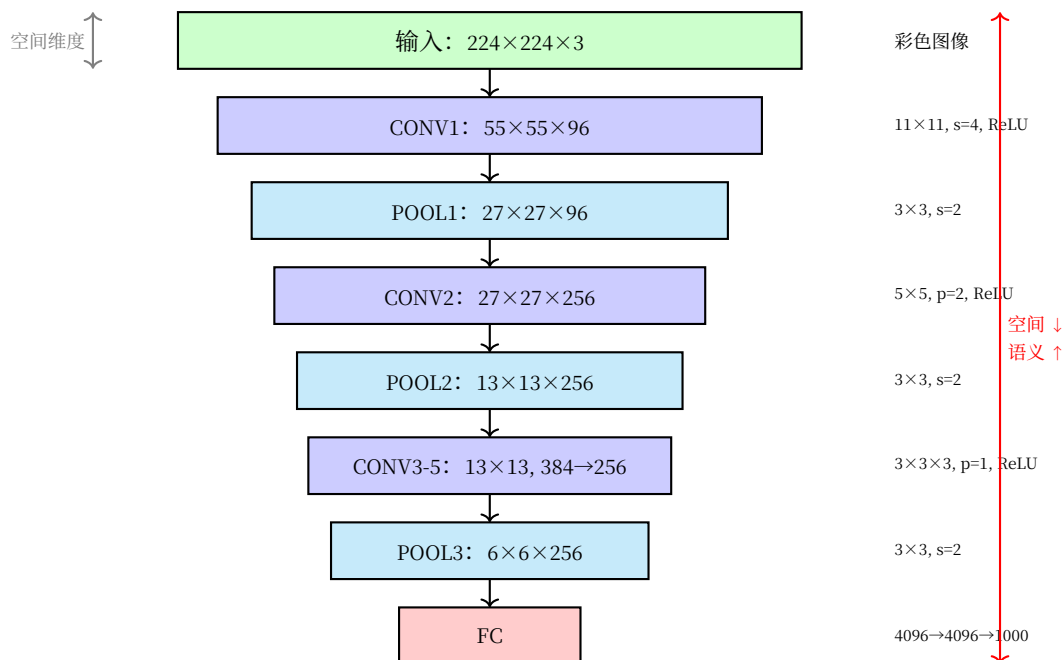


图 17: AlexNet 金字塔架构：从图像到 1,000 类

金字塔的核心洞察：

- 空间维度递减： $224 \times 224 \rightarrow 6 \times 6$ （缩小约 1,400 倍 —— 比 LeNet 的 64 倍剧烈得多）
- 特征通道递增： $3 \rightarrow 96 \rightarrow 256 \rightarrow 384 \rightarrow 256$ （语义丰富度先升后调）
- 参数量分布：卷积层约 3.7M（占 6%），全连接层约 57M（占 94%）—— 与 LeNet 的 97% 全连接占比一脉相承
- 比 LeNet 更深更宽：8 层 vs 5 层，最多 384 通道 vs 16 通道，对应更大的山需要更大的容量

和 LeNet 的对比

对比维度	LeNet-5	AlexNet
输入	32×32 灰度	224×224 彩色
层数	5	8
参数	61K	60M (×1,000)
激活函数	Tanh	ReLU
正则化	无	Dropout
训练硬件	CPU	2× GPU
首层卷积核	5×5	11×11 (更大的山需要更广的视野)

ReLU：高海拔的呼吸方式

AlexNet 是第一个大规模使用 ReLU 的深度网络。为什么这很重要？

卷积：发现装备中的 LeNet 用的是 Tanh。Tanh 的问题是：输入稍微大一点，输出就饱和在 ± 1 ，梯度趋近于零。8 层 Tanh，梯度连乘 8 次——前面的层几乎收不到信号。

ReLU 换了一种呼吸方式：

$$\text{ReLU}(x) = \max(0, x)$$

ReLU 的直觉：在正区间，梯度永远是 1——不会衰减。在负区间，梯度是 0——神经元“关掉”。这种“要么全开要么全关”的方式，在深层网络中意外地高效。

首登之后：暴露的新问题

AlexNet 证明了 ImageNet 可以爬。但成功的攀登者会立刻看到新的障碍：

1. **11×11 的卷积核太大**——一次看太多，浪费参数且容易过拟合（Inception：多尺度工具和 VGG 会回答：用更小的核堆叠）
2. **只有一条前向路径**——如果某一层“学废了”，整条路就断了（ResNet：高原反应补给站 会回答：加跳跃连接）
3. **对所有特征一视同仁**——无论这个通道是“狗耳朵”还是“背景噪声”，权重一样（SE-Net：学会聚焦 会回答：动态加权）

首登永远不是终点。它只是告诉你：这座山更高，新问题更多。

代码实践

```
import torch
import torch.nn as nn
```

(续下页)

(接上页)

```

class AlexNet(nn.Module):
    """AlexNet 的 PyTorch 实现

    架构: 5 层卷积 + 3 层全连接
    关键设计: ReLU (高海拔呼吸)、Dropout (防止过拟合)

    参数量: 约 60M (LeNet 的 1,000 倍)
    """
    def __init__(self, num_classes=1000):
        super(AlexNet, self).__init__()

        # 特征提取部分 (5 层卷积)
        self.features = nn.Sequential(
            # CONV1: 11×11 大核 —— ImageNet 需要更大的初始感受野
            # 输出: (96, 55, 55)
            nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True), # {ref}`activation-functions` —— 高海拔呼吸方式
            nn.MaxPool2d(kernel_size=3, stride=2), # 降采样

            # CONV2: 5×5 核 —— 缩小感受野, 提取更精细特征
            # 输出: (256, 27, 27)
            nn.Conv2d(96, 256, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            # CONV3-CONV5: 连续 3×3 卷积 —— 小核堆叠 = 感受野扩大 + 参数更少
            nn.Conv2d(256, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(384, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )

        # 分类器部分 (3 层全连接)
        self.classifier = nn.Sequential(
            nn.Dropout(p=0.5), # 每次随机"关掉"一半神经元 —— 逼它们独立学习

```

(续下页)

(接上页)

```

nn.Linear(256 * 6 * 6, 4096), # 6×6 来自输入 224×224 经过 5 层卷积+3 次池化
nn.ReLU(inplace=True),

nn.Dropout(p=0.5),
nn.Linear(4096, 4096),
nn.ReLU(inplace=True),

nn.Linear(4096, num_classes), # ImageNet: 1,000 类
)

def forward(self, x):
    """
    Args:
        x: 输入图像 (batch_size, 3, 224, 224)
    Returns:
        logits: (batch_size, num_classes)
    """
    x = self.features(x) # 卷积特征提取
    x = torch.flatten(x, 1) # 展平 → 全连接层输入
    x = self.classifier(x) # 分类
    return x

```

参数量计算

```

model = AlexNet(num_classes=1000)
total_params = sum(p.numel() for p in model.parameters())
print(f"AlexNet 总参数量: {total_params:,}") # 约 60,965,000
print(f"LeNet-5 总参数量: 61,706")
print(f"倍数: {total_params / 61706:.0f}x") # 约 988x

```

总结

维度	内容
历史意义	2012 年 ImageNet 首登 —— 错误率从 28.2% → 15.3%，深度学习革命的第一声炮响
核心创新	ReLU（高海拔呼吸）+ Dropout（独立训练）+ GPU 并行 + 数据增强
参数量	60M —— LeNet 的 1,000 倍。但爬的是一座大 1,000 倍的山
暴露的问题	大卷积核浪费参数、无跳跃连接、对所有特征一视同仁

首登成功的那一刻，欢呼声还没落地，真正的攀登者已经在看下一段岩壁了。

下一步

AlexNet 证明了 ImageNet 能爬。但它用 11×11 的大卷积核一次性看太多——浪费参数，且只能看一个尺度。

山比想象的更复杂。不同的岩壁需要不同的工具。

Inception: 多尺度工具——走。

3.4.3 Inception: 多尺度工具

多尺度工具——一种冰镐打不了所有岩壁，同时备好放大镜和广角镜。

AlexNet: 真正的未登峰，首登! 用 11×11 的大卷积核首登了 ImageNet。一种冰镐打天下——太粗暴了。

有的岩壁是裂缝（细粒度），有的是悬崖（全局结构）。**一种冰镐，打不了所有的岩壁。**但传统 CNN 有一个隐藏假设：所有特征都需要相同大小的感受野。

仔细想想，这个假设并不成立：

- 识别一只蚂蚁，只需要看局部几个像素（小感受野）
- 识别一头大象，需要看到整个身体结构（大感受野）
- 一张图片里可能同时有蚂蚁和大象

感受野告诉我们，感受野随深度增加而扩大。但问题是——**每一层只有一个固定大小的感受野**。如果第 3 层的感受野是 5×5 ，那它就只看 5×5 的区域，不管这个区域里的是蚂蚁还是大象。

能不能让网络**同时拥有多个大小的感受野**，然后自己选择用哪个？这就是 Inception 的核心洞察。

固定感受野的困境

想象你在看一张照片：

- 用放大镜（小感受野）：能看清蚂蚁的触角细节，但看不到大象的全貌
- 用广角镜（大感受野）：能看到大象的轮廓，但蚂蚁只是一个模糊的点

传统的 CNN 就像一台固定焦距的相机——要么一直用放大镜，要么一直用广角镜。无论哪种选择，都会丢失另一种尺度的信息。

从信息论的角度看，这就是**信息瓶颈**问题：

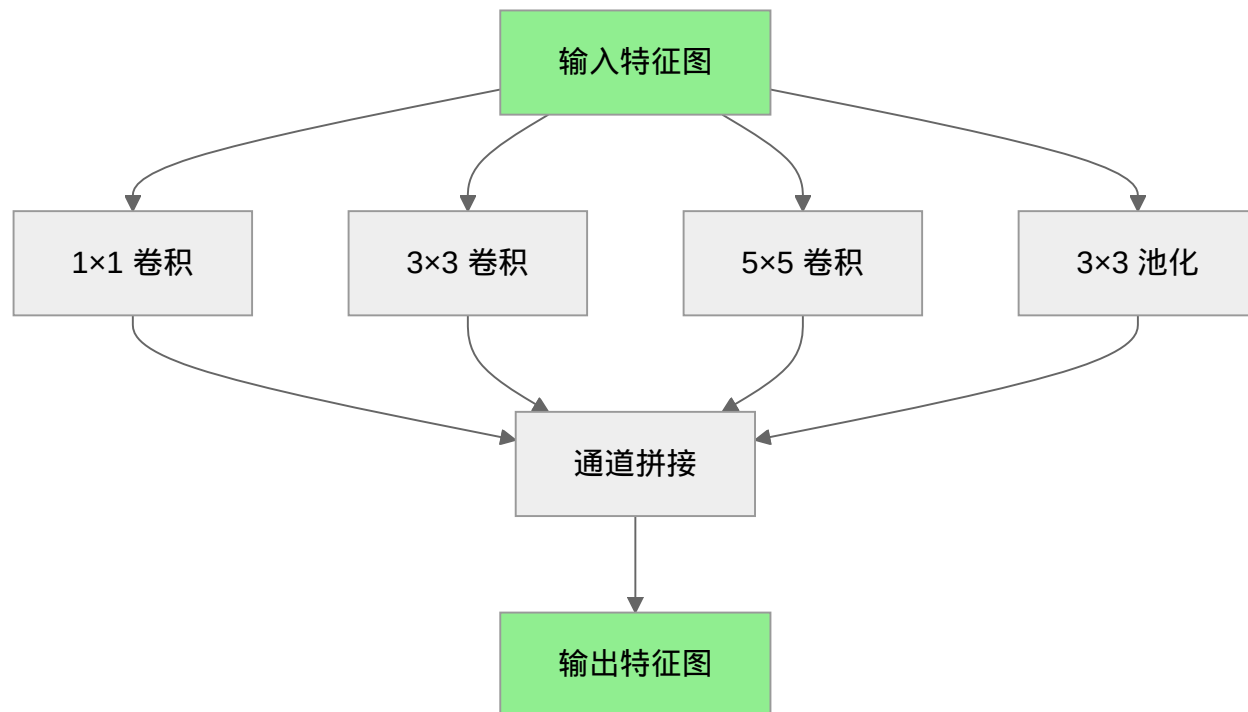
- 小卷积核（ 3×3 ）：丢失了大于 3×3 的结构信息
- 大卷积核（ 5×5 或更大）：丢失了精细的空间细节，还增加参数量

有没有办法**同时保留不同尺度的信息**？

Inception 的解决方案：多分支并行

Inception (GoogLeNet) [SLJ+15] 的解决方案非常直接：不要选择，全都要。

在每个 Inception 模块中，输入同时通过多个分支处理：



四个分支并行工作：

分支	卷积核	感受野	捕捉什么
1×1	1×1	1×1	逐像素的精细特征
3×3	3×3	3×3	局部边缘、纹理
5×5	5×5	5×5	更大的部件结构
池化	3×3	可变	平滑后的显著特征

最后把四个分支的输出在**通道维度拼接**——相当于同时保留了四种"焦距"的信息，让网络自己学习什么时候用哪个。

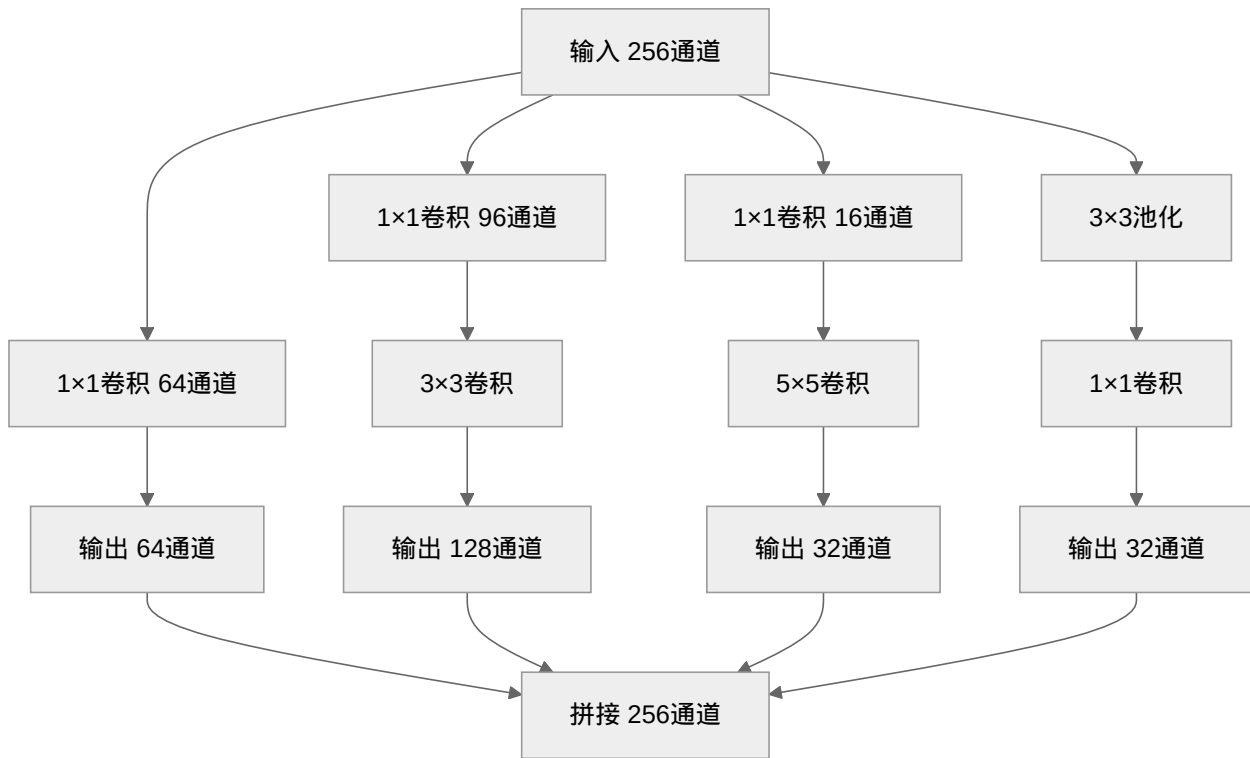
1×1 卷积的智慧：降维与信息压缩

如果你细心算一下参数量，会发现一个问题：

- 输入：256 通道
- 3×3 卷积输出 256 通道：参数量 = $256 \times (3 \times 3 \times 256) \approx 590\text{K}$
- 5×5 卷积输出 256 通道：参数量 = $256 \times (5 \times 5 \times 256) \approx 1.6\text{M}$

四个分支加起来，参数量爆炸了！

Inception 的第二个洞察：先用 1×1 卷积降维，再上大卷积核。



1×1 卷积的作用：

1. **降维**：256 通道 $\rightarrow$ 64/96/16 通道，大幅减少后续计算
2. **跨通道信息融合**：每个 1×1 卷积核看所有通道，学习通道间的组合关系
3. **增加非线性**： 1×1 卷积后接 ReLU，在不增加感受野的情况下提升表达能力

这个设计后来被许多后续架构采用，成为**降维与特征融合**的标准技巧。

信息论视角 —— 为什么要"全都要"

为什么 Inception 坚持"不要选择，全都要"？这背后有一个深层的信息论原理。

从"筛子"说起

想象你有一堆混合的沙子、石子和石块，需要把它们分类：

- **小筛子**：能筛出细沙，但石子和石块都留在上面
- **大筛子**：能筛出细沙和石子，但石块留在上面
- **无论用哪种筛子，你都会漏掉一些东西**

卷积核就像筛子：

- **小卷积核** (3×3): 只捕捉局部细节 (纹理、边缘), 看不到更大结构
- **大卷积核** (5×5): 能捕捉更大结构, 但会模糊掉细节

图像中的"频率"

图像里其实藏着不同"粗细"的信息:

信息类型	类比	用什么捕捉
细节 (高频)	蚂蚁的触角、树叶的脉络	小感受野 (3×3)
局部结构 (中频)	车轮、窗户、眼睛	中等感受野 (5×5)
整体布局 (低频)	天空、草地、建筑物的轮廓	大感受野或池化

关键洞察: 如果只用一种卷积核, 就像只用一种筛子 —— 总会漏掉某些信息。

Inception 的信息保留策略

Inception 的多分支设计, 相当于**同时用多个筛子筛选**:

- 小卷积核保留细节信息
- 中等卷积核保留局部结构
- 大卷积核保留整体布局
- 最后把所有结果拼在一起 —— **没有信息被丢弃**

用更专业的术语说, 这就是**最大化输入与输出的互信息** —— 让网络尽可能多地保留原始图像的信息 (这一概念将在 [诊断入门: 信息论听诊器](#) 中详细展开)。单分支会"丢失"某些尺度的信息, 而多分支确保了**每个尺度都有专门的通道来处理**。

为什么这比单分支更好?

假设图片里有一只猫:

- 只看**胡须细节** (小卷积核): 你知道这里有东西, 但不知道是什么
- 只看**整体轮廓** (大卷积核): 你知道这是一个动物, 但看不清细节
- **两者结合**: 你同时知道"这是猫" (轮廓) 和"猫有胡须" (细节)

Inception 让网络**同时掌握多个尺度的信息**, 就像人眼看东西时既看到整体又看清细节一样。

历史背景：2014 年的突破

Inception (GoogLeNet) 由 Google 团队于 2014 年提出 [SLJ+15], 是 ImageNet 2014 的冠军:

模型	年份	层数	参数量	ImageNet Top-5 错误率
AlexNet	2012	8	60M	16.4%
VGG-16	2014	16	138M	7.4%
GoogLeNet	2014	22	6.8M	6.7%

注意参数量: GoogLeNet 只有 6.8M 参数, 是 VGG 的 1/20, 但性能更好。这证明了精巧的结构设计比暴力堆参数更有效。

局限与演进

Inception 虽然强大, 但也有一些局限:

1. **结构复杂**: 需要手工设计每个分支的通道数
2. **超参数多**: 降维到多少通道? 几个分支? 需要反复实验
3. **训练困难**: 22 层在当时已经是深层网络, 优化有挑战

这些挑战促使研究者探索更简洁高效的设计。但 Inception 的核心思想——**多尺度处理**——从未过时, 而是以新的形式不断演进:

- **空洞卷积 (Atrous Convolution)**: 不增加参数就能扩大感受野, 是 Inception 多尺度思想的另一种实现
- **EfficientNet**: 复合缩放 (深度、宽度、分辨率) + 移动版 Inception 模块
- **RFB Net [LHW18]**: 用空洞卷积模拟人类视觉皮层的感受野结构

RFB Net: 向生物视觉学习

RFB (Receptive Field Block) Net [LHW18] 是 Inception 思想的延伸, 但更进一步——它不只是“多个固定大小”, 而是**模拟人类视觉皮层的感受野分布**。

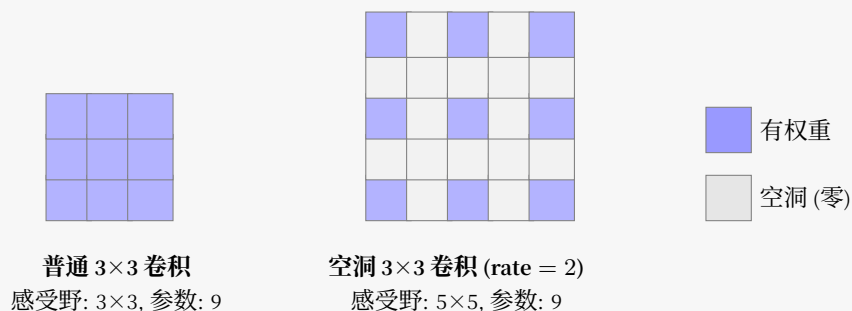
人类视觉有个特点:

- **中央凹 (fovea)**: 视野中心, 感受野小、密度高、分辨率精细
- **周边视觉**: 视野边缘, 感受野大、密度低、分辨率粗糙

空洞卷积: 不增加参数就能看更远

在介绍 RFB 之前, 我们需要先理解**空洞卷积 (Dilated/Atrous Convolution)**——一种在不增加参数量的情况下扩大感受野的技术。

核心思想：在卷积核的像素之间插入"空洞"（零），让卷积核能覆盖更大的区域，而实际的参数量不变。



- **普通 3×3 卷积：**看 $3 \times 3 = 9$ 个像素，感受野 3×3
- **空洞 3×3 (rate=2)：**在像素间插入 1 个空洞，实际看 $5 \times 5 = 25$ 个像素，感受野 5×5 ，但参数量仍是 9！

这就像用稀疏的网捕鱼——网眼变大（空洞），覆盖范围变广，但网的"线"（参数）数量不变。

RFB 用空洞卷积实现人类视觉的分布：

- 小卷积核 + 小空洞率 (rate=1)：中心密集采样
- 中等卷积核 + 中等空洞率 (rate=3)：中间区域中等密度
- 大卷积核 + 大空洞率 (rate=5)：外围稀疏采样

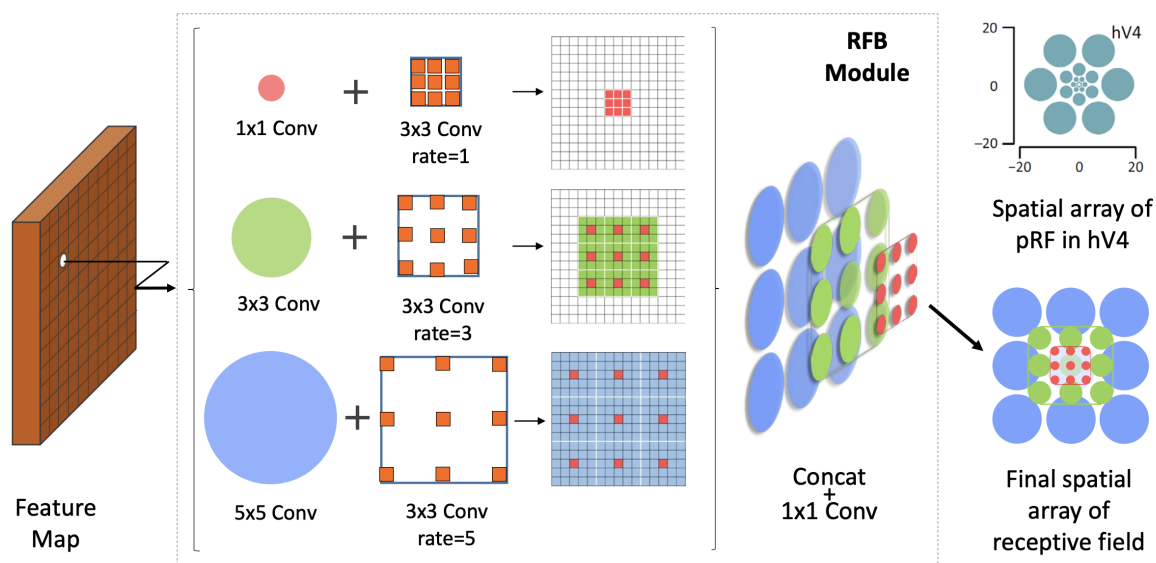


图 18: RFB 模块结构：三个分支并行，分别使用不同大小的初始卷积（ 1×1 、 3×3 、 5×5 ）配合不同空洞率的 3×3 卷积（rate=1、3、5），最后拼接并通过 1×1 卷积融合，形成多尺度感受野。图片来自 RFB Net 论文。

空洞卷积的妙处：

- 普通 5×5 卷积：看 $5 \times 5 = 25$ 个像素，参数量 25
- 空洞 3×3 (rate=2)：看 $5 \times 5 = 25$ 个像素，但参数量只有 9！

通过调整空洞率，RFB 可以在**不增加参数量**的情况下**扩大感受野**，同时保持 Inception 的多尺度特性。

RFB Net 在目标检测任务上取得了很好的效果，证明了**生物视觉的启发**依然是架构设计的重要源泉。

总结

Inception 通过**多分支并行**让网络同时看到不同尺度的信息：

视角	核心洞察	实现方式
感受野	不同物体需要不同大小的感受野	1×1 、 3×3 、 5×5 卷积并行
信息论	多频段采样保留更完整信息	通道拼接，让网络自动选择
工程	降维减少计算，升维恢复表达能力	1×1 卷积压缩 → 大卷积核处理 → 通道拼接

Inception 回答了"感受野应该多大"——答案是：**别选，全要**。但 22 层的网络已经是当时的极限。再往上爬呢？

ResNet：高原反应补给站——走。海拔越高，空气越稀薄。

3.4.4 ResNet：高原反应补给站

高原反应补给站——越深越缺氧，跳跃连接让梯度跳过中间层，直接"呼吸"到浅层。

Inception：多尺度工具用多尺度工具赢得了 ImageNet 2014——22 层。很自然的想法：堆更深，比如 50 层、100 层，会不会更好？

感受野中讨论过：层数越多，感受野越大，每个神经元能捕获的上下文越丰富。理论上，100 层应该比 22 层更强。

但现实很骨感——退化问题：

网络深度	训练集准确率	测试集准确率
20 层	85%	82%
56 层	75%	70%

更深反而更差！而且这不是过拟合——注意训练集准确率也在下降。过拟合的特征是训练好但测试差，而这里训练本身就已经崩了。这种现象被称为**退化问题**：随着网络深度增加，准确率饱和后迅速下降，且无法通过更多训练迭代解决。

直到 2015 年，ResNet [HZRS16] 的出现才解决了这个难题，它用 152 层网络赢得了 ImageNet 竞赛，证明了"深"确实可以"更好"。更惊人的是，他们在 CIFAR-10 上成功训练了 **1000 层** 的网络——这在之前是完全不可想象的。

深层网络的困境

回顾 [梯度消失 \(Vanishing Gradient\)](#) 中讨论的问题：梯度是各层影响的乘积。在深层网络中，这个乘积链太长了。

假设每层梯度都是 0.9（已经不错了），100 层连乘后：

$$0.9^{100} \approx 0.0000266$$

这意味着靠近输入层的参数几乎收不到梯度信号——前面几十层“冻住”了，只有后面几层在更新。

[Sigmoid 函数](#) 中提到 Sigmoid 会加剧这个问题，但即使换成 ReLU，单纯堆叠层数仍然会遇到优化困难。问题的本质是：**网络需要学习一个复杂的映射函数，但梯度传播的路径太长太曲折。**

方法一：几何视角——建立“抄近路”通道

想象一个传话游戏：100 个人排成一列，第一个人说一句话，传到最后一个人时，信息已经严重失真了。深层网络就像这个游戏——输入信号经过几十层变换后，梯度信号“传”不回前面的层。

ResNet 的核心洞察很简单：**如果某一层学不到有用的东西，不如让它“抄近路”，把输入原封不动地传过去。**

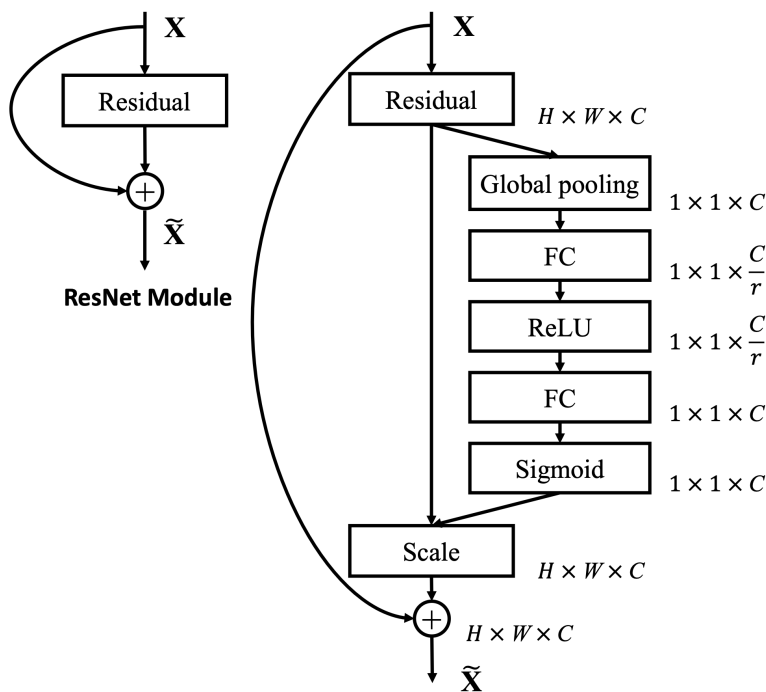


图 19: 左图 (ResNet 基础模块)：输入 x 通过残差路径后与恒等映射相加得到输出 $\tilde{x}$ 。曲线箭头即为跳跃连接，让梯度可以直接回传。这是本节的核心结构。

右图 (SE-ResNet 模块)：在残差连接基础上增加了 Squeeze-and-Excitation 分支，用于学习通道注意力——自动判断“哪些特征通道更重要”。这是 [SE-Net](#)：学会聚焦的内容，展示了 ResNet 架构的扩展性。图片来自 SE-Net 论文 [HSS18]。

关键区别：

- **普通网络**：梯度必须一层一层地“传”，每传一层就衰减一点

- **残差网络**：梯度可以通过"捷径"直接回传到前面的层，绕过了中间的衰减

这就像爬山：普通网络只能沿着山路蜿蜒而上，残差网络则允许你直接坐缆车从山顶滑到山脚。

方法二：信息论视角——只学"差异"

从互信息的角度看，深层网络面临一个**信息瓶颈**问题：数据每经过一层，都可能丢失一些原始输入的信息。100 层之后，网络可能已经"忘记"了输入到底是什么。

ResNet 的解决方案很巧妙：与其让网络学完整的映射 $H(x)$ ，不如让它学"需要改什么"——即残差 $F(x) = H(x) - x$ 。

用生活场景类比：

- **普通网络**：老师让你"画一只猫"，你只能凭空想象猫长什么样
- **残差网络**：老师给你一张狗的图片，让你"改成一只猫"——你只需要画"猫和狗的不同之处"（耳朵、尾巴、胡须）

显然，后者的任务更简单。同样，让网络学 $F(x)$ （残差）比学 $H(x)$ （完整映射）更容易——如果输入和输出已经很接近， $F(x) \approx 0$ 即可。

恒等映射：两种视角的数学统一

现在我们用数学语言统一上述两种视角。

恒等映射（Identity Mapping）指的是：输入 x 原封不动地作为输出。数学上就是 $f(x) = x$ 。

普通网络的层学习目标：

$$y = H(x)$$

ResNet 的层学习目标：

$$y = F(x) + x$$

其中 $F(x)$ 是残差函数， x 通过**跳跃连接**（Skip Connection）直接加到输出上。

核心优势：

1. **梯度传播**：反向传播时，梯度可以通过恒等映射的 x 分支直接回传，不受 $F(x)$ 的影响
2. **优化容易**：如果某一层不需要做任何变换，让 $F(x) = 0$ 即可恢复恒等映射，网络总能"退回到"较简单的状态

梯度流动的数学原理

为什么跳跃连接能解决梯度消失？回顾 [反向传播算法](#) 中的分析：深层网络的梯度等于多层 [定义：Jacobian 矩阵](#) 的连乘，当每层 Jacobian 的奇异值 < 1 时，梯度会指数级衰减（详见 [多层网络的 Jacobian 连乘与梯度消失](#)）。

ResNet 的核心突破在于为梯度提供了**第二条路径**：

ResNet 的梯度传播

残差块： $h_{i+1} = h_i + F(h_i, W_i)$

反向传播时：

$$\frac{\partial \mathcal{L}}{\partial h_i} = \frac{\partial \mathcal{L}}{\partial h_{i+1}} \cdot \frac{\partial h_{i+1}}{\partial h_i} = \frac{\partial \mathcal{L}}{\partial h_{i+1}} \cdot \left(1 + \frac{\partial F}{\partial h_i}\right)$$

关键洞察：梯度分成两条路径

1. **跳跃路径**： $\frac{\partial \mathcal{L}}{\partial h_{i+1}} \cdot 1 = \frac{\partial \mathcal{L}}{\partial h_{i+1}}$ （直接回传，不衰减！）
2. **残差路径**： $\frac{\partial \mathcal{L}}{\partial h_{i+1}} \cdot \frac{\partial F}{\partial h_i}$ （可能衰减，但不影响主路径）

对于 n 个残差块的堆叠：

$$\frac{\partial \mathcal{L}}{\partial h_i} = \frac{\partial \mathcal{L}}{\partial h_n} \prod_{k=i}^{n-1} \left(1 + \frac{\partial F_k}{\partial h_k}\right)$$

展开前几项：

$$\prod_{k=i}^{n-1} (1 + a_k) = 1 + \sum a_k + \sum_{k \neq j} a_k a_j + \dots$$

其中 $a_k = \frac{\partial F_k}{\partial h_k}$ 。

保底机制：即使所有 $a_k \approx 0$ （残差分支没学到东西），梯度依然可以通过：

$$\frac{\partial \mathcal{L}}{\partial h_i} \approx \frac{\partial \mathcal{L}}{\partial h_n} \cdot 1 = \frac{\partial \mathcal{L}}{\partial h_n}$$

这就是 +1 的魔力 —— 梯度至少为 1，不会衰减到零。

数值验证

注意

以下为假设每层梯度因子 = 0.9 的数值模拟，用于直观对比数量级差异 —— 并非实测数据。

假设残差函数的梯度 $\frac{\partial F}{\partial h} \sim \mathcal{N}(0, 0.01)$ （均值为 0，方差 0.01）：

网络深度	普通网络梯度	ResNet 梯度	改善倍数
10 层	$0.9^{10} \approx 0.35$	≈ 0.95	$2.7\times$
50 层	$0.9^{50} \approx 0.005$	≈ 0.78	$156\times$
100 层	$0.9^{100} \approx 2.6 \times 10^{-5}$	≈ 0.61	$23,000\times$

梯度不再消失，深层网络终于可以训练了！

历史背景：ImageNet 2015 的突破

ResNet 由微软亚洲研究院的 Kaiming He 等人于 2015 年提出 [HZRS16]，是深度学习发展史上的里程碑：

模型	年份	层数	ImageNet Top-5 错误率
VGG-19	2014	19	7.3%
GoogLeNet	2014	22	6.7%
ResNet-152	2015	152	3.57%

ResNet-152 的 3.57% 是**模型 ensemble** 的结果，单模型约为 4.5%。但即便如此，这也是前所未有的突破。更重要的是，ResNet 不仅在 ImageNet 分类任务夺冠，还在检测、定位、分割等任务中横扫所有对手——证明了残差架构的普适价值。

152 层——这是此前认为“不可能训练”的深度。ResNet 不仅证明了深层网络可以训练，还证明了“深”是提升性能的关键。

更惊人的是，ResNet 的设计极其简洁：

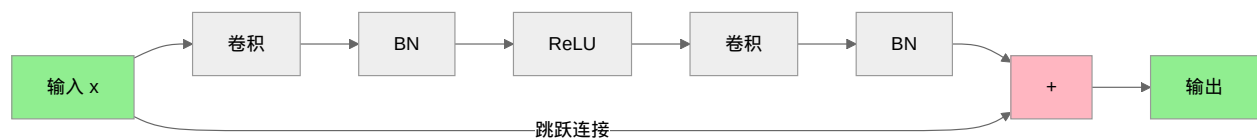
- 没有复杂的并行结构（不像 GoogLeNet 的 Inception 模块）
- 没有精心设计的超参数（VGG 需要反复实验确定深度）
- 只是简单地加上了跳跃连接，就解决了困扰学界多年的难题

这再次印证了深度学习核心数学基础中的核心思想：**好的设计往往源于对问题本质的深刻洞察，而非复杂的技术堆砌。**

ResNet 架构详解

基础构建块：残差块

ResNet 由多个**残差块**（Residual Block）堆叠而成。每个残差块包含两条路径：



Bottleneck 设计：对于更深的网络（ResNet-50+），使用 1×1 卷积先降维、再 3×3 卷积、再 1×1 升维的三层结构：

层	卷积核	输入通道	输出通道	作用
	1×1	256	64	降维，减少计算
	3×3	64	64	特征提取（低维）
	1×1	64	256	升维，恢复维度

这种"先压缩再处理再扩张"的思路，将计算量减少了约 50%，同时保持表达能力。

维度匹配问题

当输入输出维度不一致时（比如通道数从 64 变为 128），简单的 $x + y$ 无法计算。解决方案：

1. **投影 shortcut**：用 1×1 卷积调整 x 的维度（带额外参数）
2. **填充 shortcut**：用零填充增加通道（无额外参数）

实践中，投影 shortcut 效果更好，因为它允许网络学习如何调整维度。

PyTorch 实现

基础残差块

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    """ResNet 基础残差块（用于 ResNet-18/34）"""
    expansion = 1

    def __init__(self, in_channels, out_channels, stride=1, downsample=None):
        super(BasicBlock, self).__init__()

        # 第一个卷积层，可能下采样 (stride=2)
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)

        # 第二个卷积层，保持尺寸
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # 维度不匹配时的调整（投影 shortcut）
        self.downsample = downsample

    def forward(self, x):
        # 保存输入用于跳跃连接
        identity = x
```

(续下页)

(接上页)

```

# 主路径：卷积 → BN → ReLU → 卷积 → BN
out = self.conv1(x)
out = self.bn1(out)
out = F.relu(out)

out = self.conv2(out)
out = self.bn2(out)

# 维度匹配处理
if self.downsample is not None:
    identity = self.downsample(x)

# 残差连接：F(x) + x
out += identity
out = F.relu(out)

return out

```

Bottleneck 块（用于 ResNet-50+）

```

class Bottleneck(nn.Module):
    """Bottleneck 残差块（用于 ResNet-50/101/152）"""
    expansion = 4 # 输出通道是输入的 4 倍

    def __init__(self, in_channels, out_channels, stride=1, downsample=None):
        super(Bottleneck, self).__init__()

        # 1×1 降维：in_channels → out_channels
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)

        # 3×3 卷积：空间特征提取
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # 1×1 升维：out_channels → out_channels×4
        self.conv3 = nn.Conv2d(out_channels, out_channels * self.expansion,
                                kernel_size=1, bias=False)
        self.bn3 = nn.BatchNorm2d(out_channels * self.expansion)

```

(续下页)

(接上页)

```

        self.downsample = downsample

    def forward(self, x):
        identity = x

        # 1×1 降维
        out = self.conv1(x)
        out = self.bn1(out)
        out = F.relu(out)

        # 3×3 卷积
        out = self.conv2(out)
        out = self.bn2(out)
        out = F.relu(out)

        # 1×1 升维（无激活，在残差连接后再激活）
        out = self.conv3(out)
        out = self.bn3(out)

        # 维度匹配
        if self.downsample is not None:
            identity = self.downsample(x)

        # 残差连接
        out += identity
        out = F.relu(out)

    return out

```

完整 ResNet 架构

```

class ResNet(nn.Module):
    """ResNet 完整实现"""

    def __init__(self, block, layers, num_classes=1000):
        """
        Args:
            block: BasicBlock 或 Bottleneck
            layers: 每个阶段的残差块数量，如 [3,4,6,3] 对应 ResNet-50

```

(续下页)

(接上页)

```

        num_classes: 分类类别数
    """
    super(ResNet, self).__init__()
    self.in_channels = 64

    # 初始卷积: 7×7, stride=2, 下采样 4 倍
    self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
    self.bn1 = nn.BatchNorm2d(64)
    self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

    # 四个阶段, 每个阶段包含多个残差块
    self.layer1 = self._make_layer(block, 64, layers[0]) # 输出: 56×56
    self.layer2 = self._make_layer(block, 128, layers[1], stride=2) # 28×28
    self.layer3 = self._make_layer(block, 256, layers[2], stride=2) # 14×14
    self.layer4 = self._make_layer(block, 512, layers[3], stride=2) # 7×7

    # 全局平均池化 + 全连接
    self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
    self.fc = nn.Linear(512 * block.expansion, num_classes)

    # 权重初始化 (He 初始化, 适合 ReLU)
    self._initialize_weights()

def _make_layer(self, block, out_channels, num_blocks, stride=1):
    """构建一个阶段的残差块"""
    # 第一个块可能需要下采样
    downsample = None
    if stride != 1 or self.in_channels != out_channels * block.expansion:
        downsample = nn.Sequential(
            nn.Conv2d(self.in_channels, out_channels * block.expansion,
                      kernel_size=1, stride=stride, bias=False),
            nn.BatchNorm2d(out_channels * block.expansion),
        )

    layers = []
    layers.append(block(self.in_channels, out_channels, stride, downsample))

    # 后续块保持尺寸
    self.in_channels = out_channels * block.expansion
    for _ in range(1, num_blocks):

```

(续下页)

(接上页)

```

        layers.append(block(self.in_channels, out_channels))

    return nn.Sequential(*layers)

def _initialize_weights(self):
    """He 初始化"""
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, 1)
            nn.init.constant_(m.bias, 0)

def forward(self, x):
    # 初始卷积: 224×224 → 112×112 → 56×56
    x = self.conv1(x)
    x = self.bn1(x)
    x = F.relu(x)
    x = self.maxpool(x)

    # 四个残差阶段
    x = self.layer1(x) # 56×56
    x = self.layer2(x) # 28×28
    x = self.layer3(x) # 14×14
    x = self.layer4(x) # 7×7

    # 分类头
    x = self.avgpool(x) # 7×7 → 1×1
    x = torch.flatten(x, 1)
    x = self.fc(x)

    return x

def resnet18(num_classes=1000):
    """ResNet-18: [2,2,2,2] = 8 个残差块 + 初始层 = 18 层 (含全连接) """
    return ResNet(BasicBlock, [2, 2, 2, 2], num_classes)

def resnet50(num_classes=1000):

```

(续下页)

(接上页)

```
"""ResNet-50: [3,4,6,3] = 16 个 Bottleneck = 48 层 + 初始层 = 50 层"""
return ResNet(Bottleneck, [3, 4, 6, 3], num_classes)
```

效果验证：残差连接的威力

ResNet 论文中的关键对比实验：

网络	层数	Top-1 错误率	训练情况
Plain-34	34	28.54%	困难
ResNet-34	34	25.03%	正常
Plain-50	50	训练失败	无法收敛
ResNet-50	50	23.85%	正常
ResNet-152	152	21.3%	正常

关键发现：

- 34 层普通网络（Plain）比 18 层更差（退化问题）
- 34 层 ResNet 不仅解决了退化，还比 18 层更好
- 50+ 层的普通网络完全无法训练，但 ResNet 可以训练到 152 层甚至 1000+ 层

CIFAR-10 上的极限测试

论文作者在 CIFAR-10 上进行了更激进的实验 —— 训练超过 100 层的网络：

网络深度	普通网络	ResNet
20 层	训练成功	训练成功
56 层	退化（训练失败）	训练成功
110 层	完全无法收敛	训练成功
1202 层	—	训练成功

1202 层 —— 这个数字本身就令人震惊。它不仅证明了残差连接可以训练极深网络，还表明深度本身不是障碍，如何训练才是关键。

与 LeNet 的对比

特性	LeNet-5 (1998)	ResNet-50 (2015)	演进逻辑
深度	8 层	50 层	深度增加了 6 倍
核心问题	如何设计 CNN	如何训练深层网络	问题域的扩展
关键组件	卷积 + 池化	残差连接	解决新问题的创新
归一化	无	BatchNorm	稳定训练的关键
激活函数	Tanh	ReLU	更强的梯度信号
参数量	60K	25M	规模增大了 400 倍
应用场景	手写数字	通用图像分类	从特定到通用

演进脉络：

- 1. LeNet 证明了 CNN 的可行性（“能做”）
- 2. AlexNet 证明了深度的重要性（“要深”）
- 3. ResNet 解决了训练难题（“能深”）

总结

ResNet 通过**残差连接**解决了深层网络的训练难题，其核心思想可以概括为：

视角	核心洞察	实现方式
几何	建立梯度高速公路	跳跃连接绕过衰减层
信息论	只学差异，保留原始信息	$F(x)$ 代替 $H(x)$
优化	恒等映射作为保底方案	学不好就退化成 x

为什么 ResNet 重要：

- 它是现代深度学习的基石，几乎所有后续架构（DenseNet、EfficientNet、Transformer）都借鉴了残差思想
- 它证明了**网络深度是性能的关键因素**，开启了"越深越好"的时代
- 它的设计极其简洁优雅，是工程与理论完美结合的典范

ResNet 建好了补给站。152 层不再是梦。但还有一个问题 —— 补给站让信息能流通，却没有告诉网络**哪些信息更重要**。CNN 对所有特征一视同仁：“狗耳朵”和“背景噪声”，权重一样。

SE-Net：学会聚焦 —— 走。学会聚焦。

3.4.5 SE-Net：学会聚焦

ResNet：高原反应补给站 建好了补给站，解决了高原反应。但还有一个更深的问题：

CNN 对所有特征一视同仁。无论这个通道编码的是"狗耳朵"还是"背景噪声"，卷积核给的权重一样。就像登山者盯着每一块岩石——包括那些根本不值得抓的碎石。

登山老手不会这样。他们知道什么时候聚焦裂缝，什么时候忽略光滑的岩面。**不是所有特征都值得抓。**

SE-Net (Squeeze-and-Excitation Networks) [HSS18] 给 CNN 装上了"可学习的放大镜"——让网络自己判断"哪些通道重要"，然后动态分配权重。

它的核心洞察只有三行代码：**压缩** → **激励** → **缩放**。

想想你在管理一个团队：你有很多个员工（通道），每个员工都有不同的专长。但你的资源有限，需要决定谁更重要。

SE-Net 的三步直觉

1. **压缩 (Squeeze)**：你让每个员工交一份"工作总结"——用一句话概括他这周做了什么。对应到网络：把每个通道的整个特征图压缩成一个数字（全局平均池化）。
2. **激励 (Excite)**：你看了所有总结，判断谁的工作更重要——给每个员工分配一个"重要性分数"。对应到网络：用两个全连接层学习通道间的依赖关系，输出每个通道的权重。
3. **缩放 (Scale)**：按重要性分配资源——重要员工获得更多支持，不重要员工减少资源。对应到网络：把权重乘回原始特征图，重要通道被增强，不重要通道被抑制。

关键洞察：SE-Net 不改变网络的结构（通道数不变），而是改变"流经每个通道的信息量"——就像调节音量旋钮，而不是换音箱。

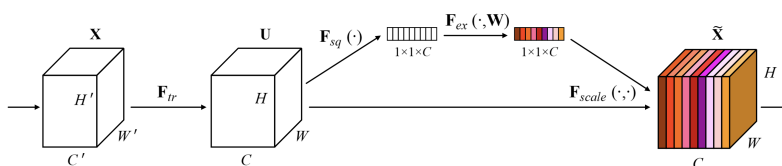


图 20: SE-Net 模块结构：Squeeze → Excite → Scale

三步详解

1. Squeeze：压缩

输入是一个特征图 $X \in \mathbb{R}^{C \times H \times W}$ (C 个通道，每个 $H \times W$)。Squeeze 操作把每个通道压缩成单个数字：

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_c(i, j)$$

这其实就是全局平均池化 (Global Average Pooling)。为什么用平均? 因为我们想获取每个通道的"全局统计信息"——不只看局部响应, 而是看整个特征图的平均激活强度。

备注

直觉: 如果一个通道的平均激活值很高, 说明该通道编码的特征在整张图片中都很"活跃", 可能很重要。反之, 平均激活值低的通道可能对当前任务贡献不大。

2. Excitation: 学习权重

有了每个通道的"工作总结" $z \in \mathbb{R}^C$, 接下来要学习通道间的依赖关系:

$$s = \sigma(W_2 \cdot \delta(W_1 \cdot z))$$

其中:

- $W_1 \in \mathbb{R}^{C/r \times C}$: 降维层, 先把 C 维压缩到 C/r 维
- δ : ReLU 激活函数
- $W_2 \in \mathbb{R}^{C \times C/r}$: 升维层, 恢复回 C 维
- σ : Sigmoid 激活函数, 输出 $(0, 1)$ 之间的权重

为什么用瓶颈结构 (先降维再升维)?

瓶颈结构的设计动机

1. **减少参数量:** 直接学习 $C \times C$ 的变换需要 C^2 个参数。用瓶颈结构只需要 $2C^2/r$ 个参数, 当 $r = 16$ 时减少了 8 倍。
2. **引入非线性:** 降维 $\rightarrow$ ReLU $\rightarrow$ 升维的结构比单层线性变换有更强的表达能力。
3. **学习通道间关系:** 瓶颈迫使信息通过一个低维"瓶颈", 这迫使网络学到通道间的紧凑表示。

压缩比 r 控制瓶颈的宽度: r 越大参数越少, 但表达能力也越弱。通常取 $r = 16$ 。

3. Scale: 加权

把学习到的权重 $s_c \in (0, 1)$ 乘回原始特征图的对应通道:

$$\tilde{x}_c = s_c \cdot x_c$$

这就是一个逐通道的缩放操作。权重接近 1 的通道被保留甚至增强, 权重接近 0 的通道被抑制。

SE-ResNet 集成

SE 模块通常插入到残差块中、残差连接之前：

列表 4: SE-ResNet 基础块实现

```

1 import torch
2 import torch.nn as nn
3 from .se_block import SEBlock
4
5
6 class SEBasicBlock(nn.Module):
7     """
8     带有 SE 模块的 ResNet 基础块
9
10    SE 模块插入在第二个卷积之后、残差连接之前：
11    Conv1 → BN → ReLU → Conv2 → BN → SE → + → ReLU
12                                ↑
13                        残差连接 └─┘
14
15    输入: (B, C, H, W)
16    输出: (B, planes*expansion, H/stride, W/stride)
17    """
18    expansion = 1
19
20    def __init__(self, inplanes, planes, stride=1, downsample=None, reduction=16):
21        super(SEBasicBlock, self).__init__()
22
23        self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=3,
24                                stride=stride, padding=1, bias=False)
25        self.bn1 = nn.BatchNorm2d(planes)
26
27        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3,
28                                stride=1, padding=1, bias=False)
29        self.bn2 = nn.BatchNorm2d(planes)
30
31        # SE 模块: 在第二个卷积后, 残差连接前
32        self.se = SEBlock(planes, reduction)
33
34        self.downsample = downsample
35        self.stride = stride
36        self.relu = nn.ReLU(inplace=True)

```

(续下页)

(接上页)

```

37
38     def forward(self, x):
39         identity = x
40
41         out = self.conv1(x)
42         out = self.bn1(out)
43         out = self.relu(out)
44
45         out = self.conv2(out)
46         out = self.bn2(out)
47
48         # 注意力：在特征融合前重新校准通道重要性
49         out = self.se(out)
50
51         # 残差连接
52         if self.downsample is not None:
53             identity = self.downsample(x)
54
55         out += identity
56         out = self.relu(out)
57
58         return out
59
60 if __name__ == "__main__":
61     x = torch.randn(2, 64, 32, 32)
62     block = SEBasicBlock(64, 64)
63     y = block(x)
64     print(f"Input shape: {x.shape}")
65     print(f"Output shape: {y.shape}")
66     print(f"Block params: {sum(p.numel() for p in block.parameters())}")

```

参数量与计算开销

SE 模块的参数量为：

$$\text{Params} = \frac{C}{r} \times C + C \times \frac{C}{r} = \frac{2C^2}{r}$$

当 $C = 256, r = 16$ 时，参数数量为 $2 \times 256^2 / 16 = 8,192$ —— 相对于 ResNet-50 的 2500 万参数可以忽略不计。计算开销增加也小于 1%。

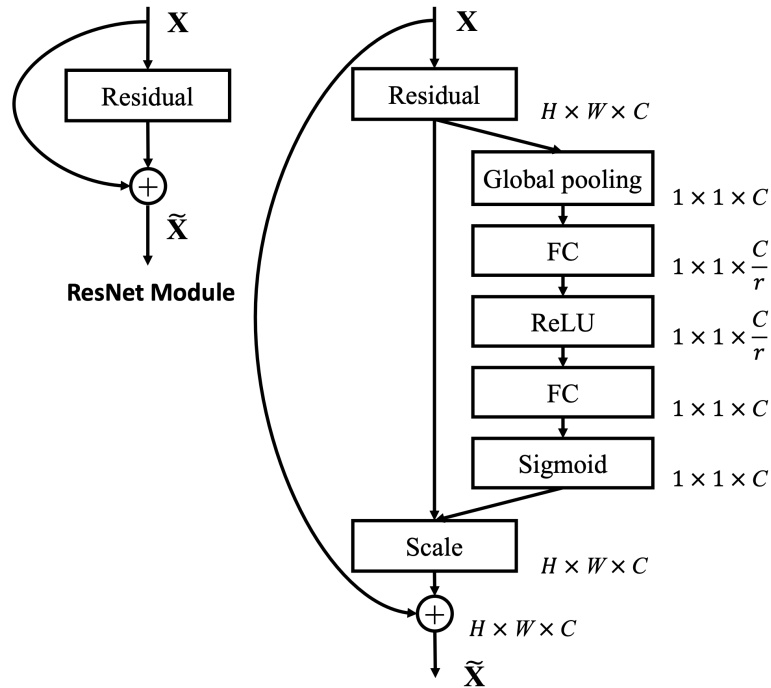


图 21: SE-ResNet: 在残差块中插入 SE 模块

SE-Net 的关键贡献

- 仅增加约 1% 计算量, 提升 1-2% 准确率
- 即插即用: 可集成到任何 CNN 架构中 (ResNet、MobileNet、Inception 等)
- 在 ImageNet 上, SE-ResNet-50 达到 77.62% Top-1 准确率, 比基线提升 +1.47%

PyTorch 实现

列表 5: SE 模块的 PyTorch 实现

```

1 import torch
2 import torch.nn as nn
3
4 class SEBlock(nn.Module):
5     """
6     Squeeze-and-Excitation 模块
7
8     三步核心操作:
9     1. Squeeze: 全局平均池化, 将 CxHxW 压缩为 Cx1x1
10    2. Excitation: 两个全连接层学习通道权重 (瓶颈结构)

```

(续下页)

(接上页)

3. Scale: 权重乘回原始特征图

参数量: $2 * C * (C/r) = 2C^2/r$

输入: (B, C, H, W)

输出: (B, C, H, W)

"""

def __init__(self, channels, reduction=16):

super(SEBlock, self).__init__()

Squeeze: 全局平均池化, 每个通道压缩为一个标量

输入 (B, C, H, W) → 输出 (B, C, 1, 1)

self.avg_pool = nn.AdaptiveAvgPool2d(1)

Excitation: 瓶颈结构, 压缩比 r=16 减少参数量

 # $C \rightarrow C/r \rightarrow C$, 参数: $C*(C/r) + (C/r)*C = 2C^2/r$

self.fc = nn.Sequential(

nn.Linear(channels, channels // reduction, bias=False), # 降维

nn.ReLU(inplace=True), # 非线性

nn.Linear(channels // reduction, channels, bias=False), # 升维

nn.Sigmoid() # 输出 (0,1) 权重

)

def forward(self, x):

b, c, _, _ = x.size()

Squeeze: (B, C, H, W) → (B, C, 1, 1) → (B, C)

y = self.avg_pool(x).view(b, c)

Excitation: (B, C) → (B, C/r) → (B, C) → (B, C, 1, 1)

y = self.fc(y).view(b, c, 1, 1)

Scale: 逐通道乘法, 广播到 H×W

每个通道乘以对应的标量权重

return x * y.expand_as(x)**if** __name__ == "__main__":

x = torch.randn(2, 64, 32, 32)

se = SEBlock(64, reduction=16)

y = se(x)

print(f"Input shape: {x.shape}")

(续下页)

(接上页)

```

51 print(f"Output shape: {y.shape}")
52 print(f"SEBlock parameters: {sum(p.numel() for p in se.parameters())} (2*64^2/16 = 512)")

```

代码要点:

- `AdaptiveAvgPool2d((1, 1))` 实现 Squeeze: 把任意尺寸的特征图压缩到 1×1
- `Linear` 层的输入输出维度由压缩比 r 控制
- Sigmoid 保证输出在 $(0, 1)$ 范围内
- `x * self.scale` 实现逐通道加权

本章小结

- SE-Net 通过 **Squeeze** → **Excitation** → **Scale** 三步实现通道注意力
- 核心是让网络学习"每个通道的重要性权重", 然后据此重新校准特征
- 结构简单、计算开销小、即插即用

通道注意力解决了"什么特征重要"。但还有一个问题: **同一个特征, 出现在画面的不同位置, 意义完全不同。**
"耳朵"在画面中央是猫, 在角落是噪声。

CBAM: **哪里重要 + 什么重要** —— 走。不只选频道, 还要调区域。

3.4.6 CBAM: 哪里重要 + 什么重要

SE-Net: **学会聚焦** 让 CNN 学会了"看什么" (通道注意力)。但只看"什么"不够 —— **同一个特征在不同空间位置的意义完全不同。**

一张猫的图片中, "耳朵"特征出现在画面中央和出现在背景角落, 含义天差地别。登山者不会在整个山壁上均匀用力 —— 他们先看"抓什么" (裂缝还是岩棱), 再看"抓哪里" (这个裂缝的中间段还是末端)。

CBAM (Convolutional Block Attention Module) [WPLK18] 把这两个问题合并成一个答案: **先选频道, 再调区域。**

空间注意力

空间注意力 (Spatial Attention) 的目标是生成一个**空间注意力图** —— 与输入特征图同等宽高的二维权重图, 重要的区域更亮, 不重要的更暗。

空间注意力的直觉

通道注意力是"给每个频道调音量", 空间注意力则是"给屏幕的每个区域调亮度"。

输入特征图 $F \in \mathbb{R}^{C \times H \times W}$ ，空间注意力模块生成注意力图 $M_s \in \mathbb{R}^{1 \times H \times W}$ ：

$$\tilde{F} = F \odot M_s$$

关键问题：如何从 C 个通道生成 1 个通道的空间注意力图？答案是**沿通道维度聚合信息**。

最常用的方法来自 CBAM [WPLK18]：

1. **通道池化：**对每个空间位置，分别计算该位置在所有通道上的平均值和最大值
2. **拼接：**把两个聚合图拼成 $2 \times H \times W$
3. **卷积：**用 7×7 卷积将 2 个通道压缩回 1 个通道
4. **激活：**Sigmoid 映射到 $(0, 1)$

$$M_s = \sigma(f^{7 \times 7}([\text{AvgPool}^c(F); \text{MaxPool}^c(F)]))$$

为什么同时用平均池化和最大池化？

平均池化捕获"该位置的整体激活强度"，最大池化捕获"该位置的最强响应"。两者互补。

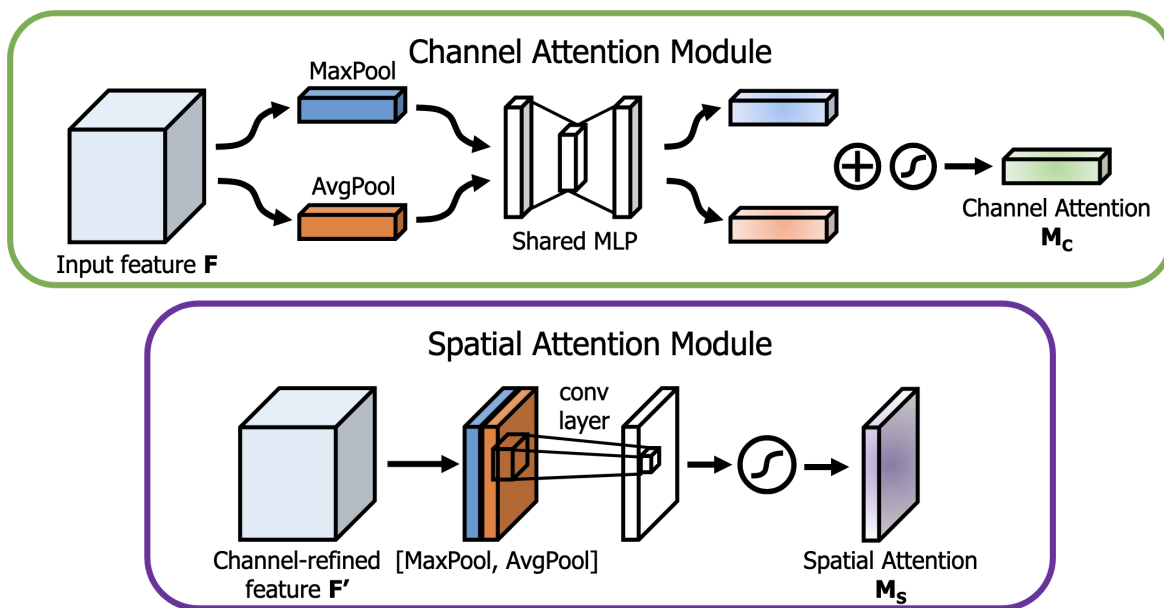


图 22: 通道注意力（选频道）与空间注意力（调区域）的输入输出对比

CBAM: 通道 + 空间, 两个都要

SE-Net: 学会聚焦 解决了"什么特征重要" (通道维度)。空间注意力解决了"哪里重要" (空间维度)。那么问题来了: **能不能两个都要?**

CBAM (Convolutional Block Attention Module) [WPLK18] 就是答案 —— 它把通道注意力和空间注意力串联起来, 形成一个更强大的注意力模块。

核心理念: 先"选频道"再"调区域"

CBAM 的工作流程很直观: 先判断哪些通道重要 (通道注意力), 再判断这些重要通道中的哪些空间位置最关键 (空间注意力)。

CBAM 的直觉

想象你在看一张照片找猫:

1. **通道注意力**: 首先, 你决定看"颜色"这个维度 (而不是"纹理"或"亮度"), 因为猫的颜色信息最有用。
2. **空间注意力**: 然后, 你在"颜色"维度下, 聚焦到照片中特定的区域 —— 猫所在的位置。

这就是 CBAM 的两步走: 先"选对频道", 再"看对位置"。

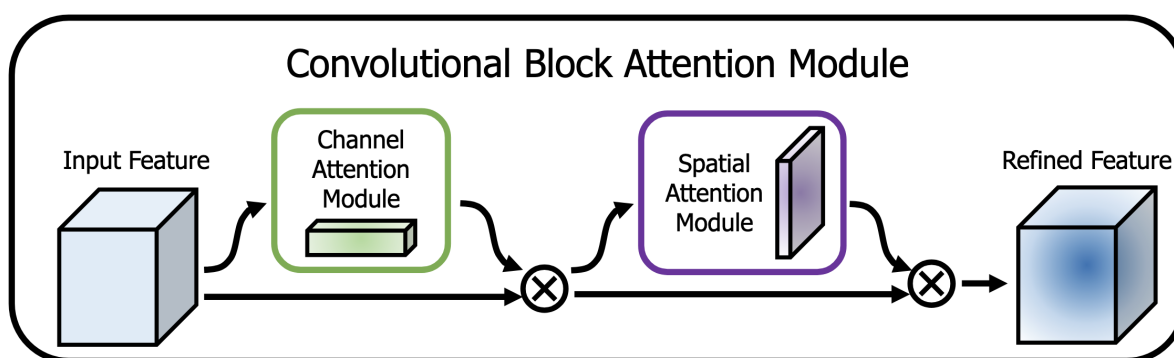
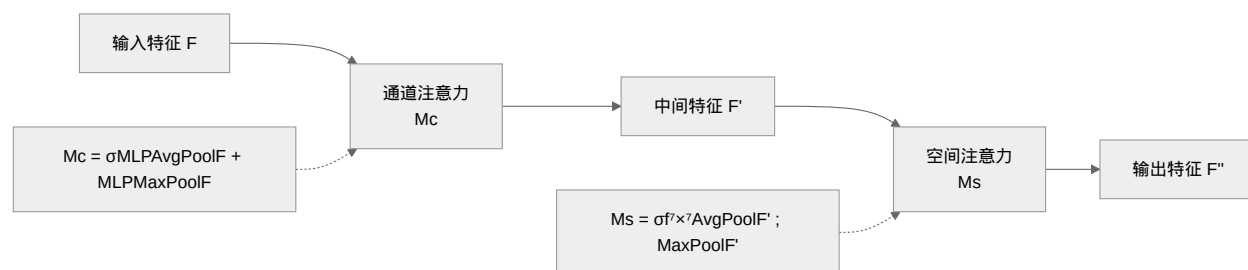


图 23: CBAM 模块结构: 通道注意力 → 空间注意力串联



数学形式

CBAM 的完整计算过程为：

$$F' = M_c(F) \otimes F$$

$$F'' = M_s(F') \otimes F'$$

其中 $\otimes$ 表示逐元素乘法。

通道注意力部分

CBAM 的通道注意力与 SE-Net 类似，但增加了一个改进：**同时使用平均池化和最大池化**：

$$M_c(F) = \sigma(\text{MLP}(\text{AvgPool}(F)) + \text{MLP}(\text{MaxPool}(F)))$$

列表 6: CBAM 通道注意力模块实现

```

1 import torch
2 import torch.nn as nn
3
4 class ChannelAttention(nn.Module):
5     """
6     CBAM 的通道注意力模块
7
8     与 SE-Net 的区别：同时使用平均池化和最大池化，然后相加融合
9     - 平均池化：捕获全局统计信息
10    - 最大池化：捕获最显著响应
11    - 两者互补，提供更丰富的通道描述
12
13    使用 1×1 Conv2d 替代 Linear，保持 4D 张量格式（兼容性更好）
14    参数量：2 * C * (C/r) = 2C²/r
15    输入：(B, C, H, W)
16    输出：(B, C, 1, 1)
17    """
18    def __init__(self, in_channels, reduction=16):
19        super(ChannelAttention, self).__init__()
20
21        self.avg_pool = nn.AdaptiveAvgPool2d(1)
22        self.max_pool = nn.AdaptiveMaxPool2d(1)
23
24        # 共享的 MLP，用 1×1 Conv2d 实现（等价于 Linear，但保持 4D）
25        self.mlp = nn.Sequential(
26            nn.Conv2d(in_channels, in_channels // reduction, 1, bias=False), # 降维

```

(续下页)

(接上页)

```

27         nn.ReLU(inplace=True),
28         nn.Conv2d(in_channels // reduction, in_channels, 1, bias=False) # 升维
29     )
30
31     self.sigmoid = nn.Sigmoid()
32
33     def forward(self, x):
34         # 两种池化分别通过共享 MLP, 然后逐元素相加
35         avg_out = self.mlp(self.avg_pool(x)) # (B, C, 1, 1)
36         max_out = self.mlp(self.max_pool(x)) # (B, C, 1, 1)
37         out = avg_out + max_out # 融合
38         return self.sigmoid(out)
39
40 if __name__ == "__main__":
41     x = torch.randn(2, 64, 32, 32)
42     ca = ChannelAttention(64, reduction=16)
43     y = ca(x)
44     print(f"Input shape: {x.shape}")
45     print(f"Attention shape: {y.shape}")
46     print(f"ChannelAttention params: {sum(p.numel() for p in ca.parameters())}")

```

空间注意力部分

与上一节描述的一致，使用通道池化 + 7×7 卷积生成空间注意力图。

列表 7: CBAM 空间注意力模块实现

```

1  import torch
2  import torch.nn as nn
3
4  class SpatialAttention(nn.Module):
5      """
6      CBAM 风格的空间注意力模块
7
8      沿通道维度聚合信息，生成空间注意力图 (1×H×W):
9      1. 通道平均池化 + 通道最大池化 → 2×H×W
10     2. 7×7 卷积压缩为 1×H×W
11     3. Sigmoid 激活
12
13     参数量: 2 * 7 * 7 = 98 (k=7 时)

```

(续下页)

(接上页)

```

14  输入: (B, C, H, W)
15  输出: (B, 1, H, W) — 空间注意力图
16  """
17  def __init__(self, kernel_size=7):
18      super(SpatialAttention, self).__init__()
19
20      assert kernel_size in (3, 7), 'kernel size must be 3 or 7'
21      padding = 3 if kernel_size == 7 else 1
22
23      # 输入 2 通道 (avg + max), 输出 1 通道 (注意力图)
24      # 参数量:  $2 * 1 * k * k = 2k^2$ 
25      self.conv = nn.Conv2d(2, 1, kernel_size, padding=padding, bias=False)
26      self.sigmoid = nn.Sigmoid()
27
28  def forward(self, x):
29      # 沿通道维度 (dim=1) 聚合
30      # avg_out: (B, 1, H, W), 每个位置的平均激活强度
31      avg_out = torch.mean(x, dim=1, keepdim=True)
32      # max_out: (B, 1, H, W), 每个位置的最强响应
33      max_out, _ = torch.max(x, dim=1, keepdim=True)
34
35      # 拼接: (B, 2, H, W)
36      x = torch.cat([avg_out, max_out], dim=1)
37      # 卷积压缩 + Sigmoid: (B, 1, H, W)
38      x = self.conv(x)
39      return self.sigmoid(x)
40
41  if __name__ == "__main__":
42      x = torch.randn(2, 64, 32, 32)
43      sa = SpatialAttention(kernel_size=7)
44      y = sa(x)
45      print(f"Input shape: {x.shape}")
46      print(f"Attention shape: {y.shape}")
47      print(f"SpatialAttention params: {sum(p.numel() for p in sa.parameters())} (2*7^2=98)")

```

完整 CBAM 模块

列表 8: 完整 CBAM 模块实现

```

1 import torch
2 import torch.nn as nn
3 from .channel_attention import ChannelAttention
4 from .spatial_attention import SpatialAttention
5
6
7 class CBAM(nn.Module):
8     """
9     完整的 CBAM 模块
10
11     先通道注意力 → 再空间注意力, 串联组合:
12     1. 通道注意力: 判断"什么特征重要" (C×1×1 权重)
13     2. 空间注意力: 判断"哪里重要" (1×H×W 权重)
14
15     输入: (B, C, H, W)
16     输出: (B, C, H, W)
17     """
18     def __init__(self, in_channels, reduction=16, kernel_size=7):
19         super(CBAM, self).__init__()
20
21         self.channel_attention = ChannelAttention(in_channels, reduction)
22         self.spatial_attention = SpatialAttention(kernel_size)
23
24     def forward(self, x):
25         # Step 1: 通道注意力 → 重新校准通道重要性
26         # x * channel_attention(x): 每个通道乘以其重要性权重
27         x = x * self.channel_attention(x)
28
29         # Step 2: 空间注意力 → 聚焦重要空间区域
30         # x * spatial_attention(x): 每个空间位置乘以其重要性权重
31         x = x * self.spatial_attention(x)
32
33         return x
34
35 if __name__ == "__main__":
36     x = torch.randn(2, 64, 32, 32)
37     cbam = CBAM(64, reduction=16)
38     y = cbam(x)

```

(续下页)

(接上页)

```

39 print(f"Input shape: {x.shape}")
40 print(f"Output shape: {y.shape}")
41 print(f"CBAM params: {sum(p.numel() for p in cbam.parameters())}")

```

列表 9: CBAM-ResNet 基础块实现

```

1 import torch
2 import torch.nn as nn
3 from .cbam import CBAM
4
5
6 class CBAMBasicBlock(nn.Module):
7     """
8     带有 CBAM 模块的 ResNet 基础块
9
10    CBAM 插入在第二个卷积之后、残差连接之前:
11    Conv1 → BN → ReLU → Conv2 → BN → CBAM → + → ReLU
12
13    残差连接 ————— ↑
14
15    与 SEBasicBlock 的区别: CBAM 同时做通道和空间注意力
16    输入: (B, C, H, W)
17    输出: (B, planes*expansion, H/stride, W/stride)
18    """
19    expansion = 1
20
21    def __init__(self, inplanes, planes, stride=1, downsample=None, reduction=16):
22        super(CBAMBasicBlock, self).__init__()
23
24        self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=3,
25                                stride=stride, padding=1, bias=False)
26        self.bn1 = nn.BatchNorm2d(planes)
27
28        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3,
29                                stride=1, padding=1, bias=False)
30        self.bn2 = nn.BatchNorm2d(planes)
31
32        # CBAM: 通道注意力 + 空间注意力
33        self.cbam = CBAM(planes, reduction)
34

```

(续下页)

(接上页)

```

35     self.downsample = downsample
36     self.stride = stride
37     self.relu = nn.ReLU(inplace=True)
38
39     def forward(self, x):
40         identity = x
41
42         out = self.conv1(x)
43         out = self.bn1(out)
44         out = self.relu(out)
45
46         out = self.conv2(out)
47         out = self.bn2(out)
48
49         # 同时进行通道和空间注意力重校准
50         out = self.cbam(out)
51
52         if self.downsample is not None:
53             identity = self.downsample(x)
54
55         out += identity
56         out = self.relu(out)
57
58         return out
59
60 if __name__ == "__main__":
61     x = torch.randn(2, 64, 32, 32)
62     block = CBAMBasicBlock(64, 64)
63     y = block(x)
64     print(f"Input shape: {x.shape}")
65     print(f"Output shape: {y.shape}")
66     print(f"Block params: {sum(p.numel() for p in block.parameters())}")

```

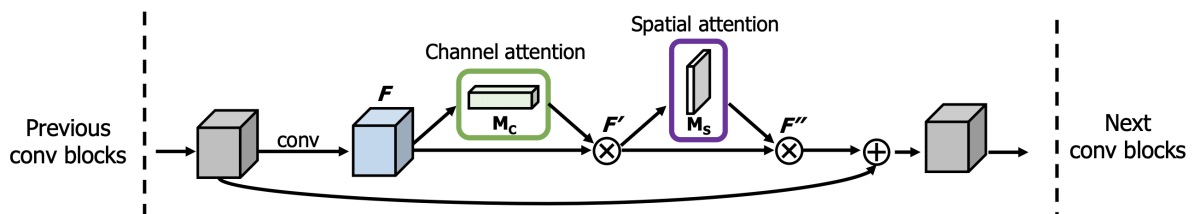


图 24: CBAM-ResNet: 在残差块中插入 CBAM 模块

SE-Net vs CBAM

对比项	SE-Net	CBAM
注意力维度	仅通道	通道 + 空间
通道池化方式	仅平均池化	平均 + 最大池化
参数量增加	$2C^2/r$	$2C^2/r + k^2$
计算开销	~1%	~2%
适用场景	分类任务	检测/分割等需要定位的任务

CBAM 的实验表明 [WPLK18]: 在 ImageNet 上, CBAM-ResNet-50 达到 **78.49%** Top-1 准确率, 比基线 ResNet-50 (76.15%) 提升 **+2.34%**, 比 SE-ResNet-50 (77.62%) 额外提升 **+0.87%**。

为什么要先通道再空间?

CBAM 的作者实验发现**先通道再空间**的效果优于先空间再通道或并行。直觉上: 先通过通道注意力突出重要的语义特征, 再通过空间注意力定位这些特征在图像中的具体位置 —— 这个顺序更符合人的认知习惯: “先知道看什么, 再知道看哪里”。

下一步

通道注意力、空间注意力 —— 你学会了让网络“看重点”。

但回头看这八年: AlexNet 首登、Inception 多尺度、ResNet 补给站、SE-Net/CBAM 聚焦 —— 每一件装备都在回答“怎么爬得更高”。山一直是同一座: ImageNet 上的准确率。

现在, 山变了。新的问题不是“怎么爬得更高”, 而是“怎么用最少的资源爬上去”。你的手机只有几瓦功耗, 你的无人机只有一颗嵌入式芯片。预算几乎为零 —— 还能登顶吗?

MobileNet: 一个人, 一把冰镐 —— 走。一个人, 一把冰镐。

3.4.7 MobileNet: 一个人, 一把冰镐

CBAM: 哪里重要 + 什么重要 让网络学会了看重点。但有一个残酷的现实: **不是每个团队都雇得起 1,000 人、买得起补给站。**

你的手机只有几瓦的功耗。你的无人机只有一颗嵌入式芯片。你的浏览器里跑模型不能超过几 MB。你不是 Google —— 你没有 1,024 张 TPU。

关键问题: 预算几乎为零, 还能爬吗?

历史背景：CNN 太"重"了

2017 年之前，CNN 的发展方向只有一条：**更深、更宽、更大**。AlexNet（60M 参数）→ VGG（138M）→ ResNet（60M，但 152 层）。准确率在涨，但模型大到塞不进手机。

Andrew Howard 和 Google 的研究团队问了一个逆向的问题：**如果准确率不是你唯一的追求呢？如果必须在准确率和速度之间做取舍——你该怎么设计？**

2017 年，MobileNet [HZC+17] 给出了答案。核心洞察只有一个，但彻底改变了轻量化设计的游戏规则。

核心洞察：把卷积"拆开"

卷积：发现装备 中我们学到：一个普通的 3×3 卷积同时做两件事——

1. **空间混合**：把 3×3 区域内的像素信息融合（“这块岩石的纹理是什么？”）
2. **通道混合**：把不同通道的信息融合（“纹理 + 颜色 + 边缘——这整体是什么？”）

MobileNet 说：**这两件事可以分开做。而且分开做，便宜 8-9 倍。**

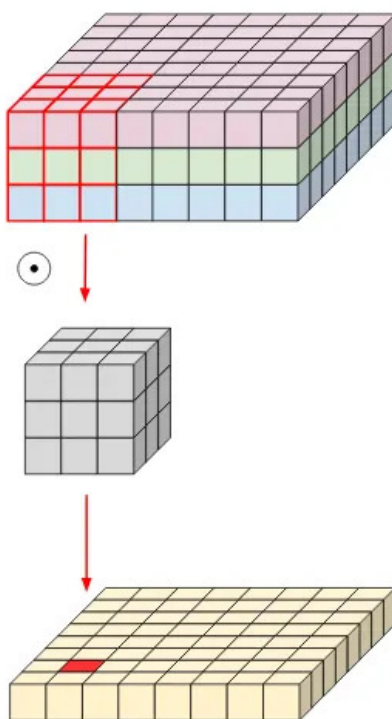


图 25: 标准卷积：空间混合与通道混合同时进行

参数量对比（假设 $M = N = 256$ ）：

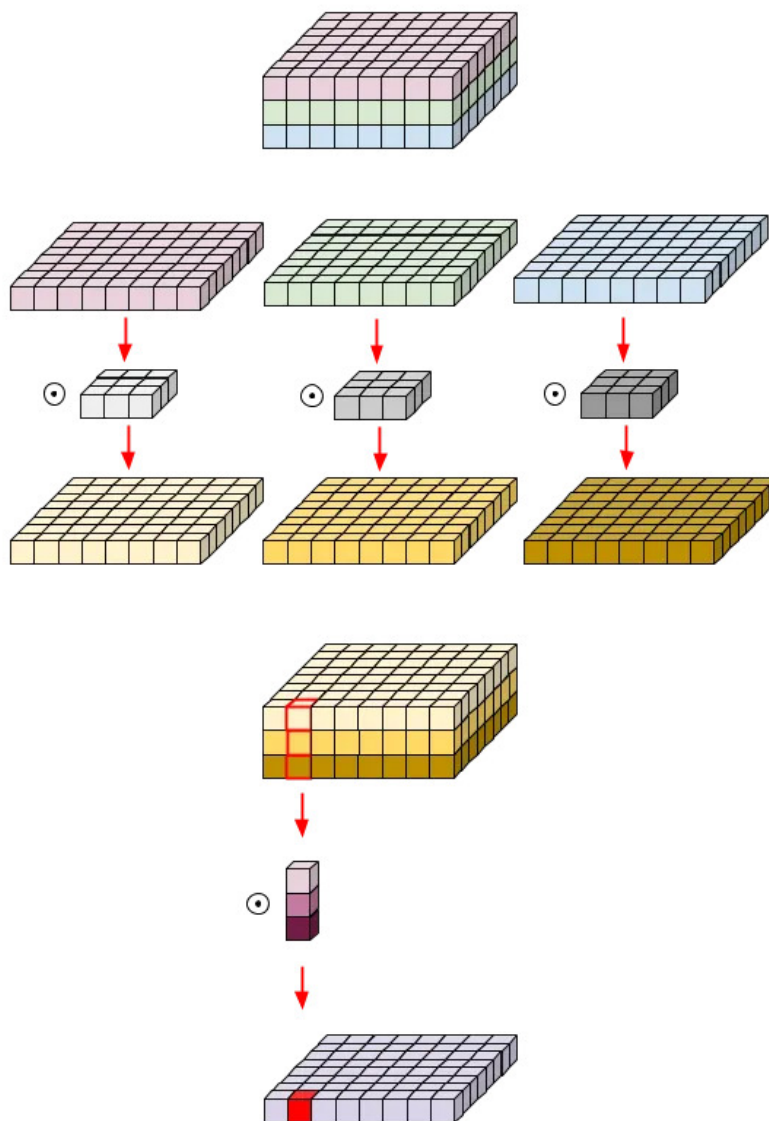


图 26: 深度可分离卷积: 先 Depthwise (空间混合), 再 Pointwise (通道混合)

操作	计算	参数数量
标准 3×3 卷积	$3 \times 3 \times 256 \times 256$	589,824
深度可分离	$3 \times 3 \times 256 + 256 \times 256$	67,840

减少了 8.7 倍。不是魔法的结果——是你把“空间混合”和“通道混合”拆开了，中间省掉了大量的交叉连接。

登山直觉

标准卷积 = 一个人既探路（空间）又做决策（通道），每一步都要综合考虑所有因素。累，慢，负重多。

深度可分离 = **两个人分工**：一个人只管脚下的岩壁（Depthwise: $3 \times 3 \times M$ ），另一个人只管“整体判断”（Pointwise: $1 \times 1 \times M \times N$ ）。各司其职，总负重少了近 9 倍。

两个超参数：贫穷的自由度

MobileNet 给了你两个旋钮，让你根据自己的预算灵活调整：

超参数	含义	取值	效果
宽度乘数 α	每层的通道数缩放	(0, 1], 通常 0.25/0.5/0.75/1.0	$\alpha = 0.5 \rightarrow$ 参数和计算量减少约 4 倍
分辨率乘数 ρ	输入图像分辨率缩放	(0, 1], 如 224/192/160/128	$\rho = 0.5 \rightarrow$ 计算量减少约 4 倍

这两个旋钮相乘，效果叠加： $\alpha = 0.5, \rho = 0.5 \rightarrow$ 计算量减少约 16 倍。

极端案例

MobileNet 的最轻版本 ($\alpha = 0.25$, 输入 128×128) 参数量不到 **0.5M**——是 AlexNet 的 1/120。在 ImageNet 上仍然能达到约 50% 的准确率。

50% 准确率听起来低，但记住：这个模型可以跑在 10 年前的手机上，功耗不到 1 瓦，推理时间不到 10 毫秒。对于“这张照片里有没有人？”这种简单任务，足够了。

这就像一个人、一把冰镐、没有补给站——他爬不到 8,000 米，但他能爬到 5,000 米。而他的全部装备只占一个背包。

架构详解

MobileNet 的骨架极其简单——每一层都是一个深度可分离卷积块。按空间分辨率分组，形成经典的金字塔结构：

金字塔视角：数据流的维度变化

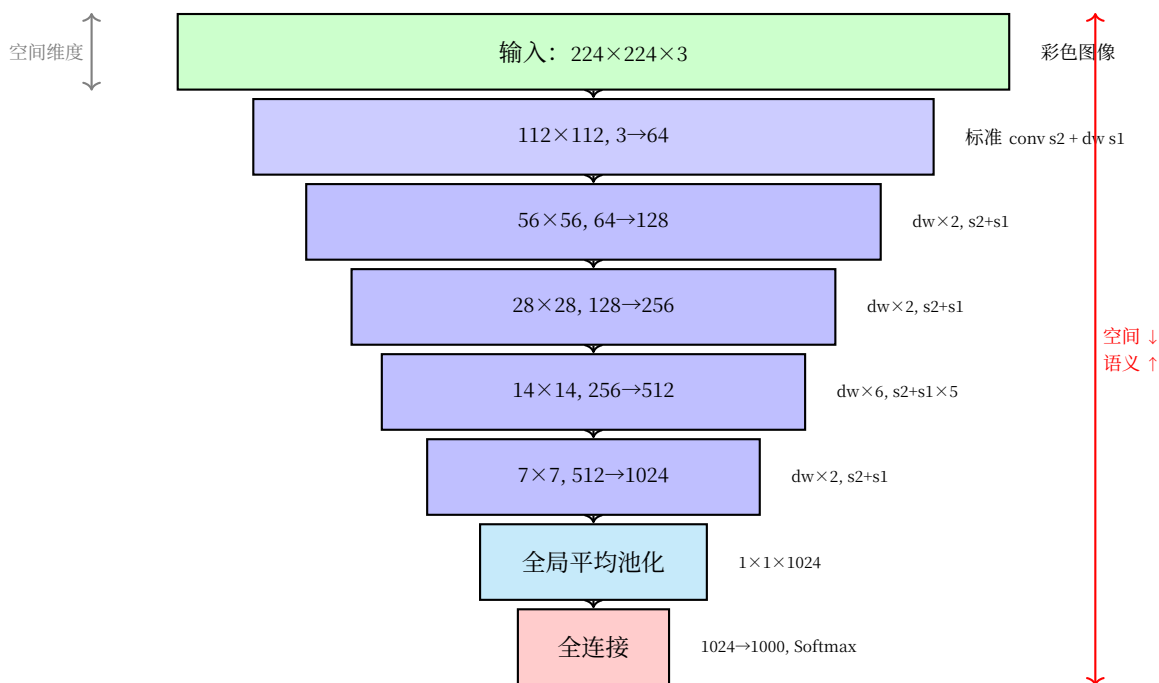


图 27: MobileNet 金字塔架构：用深度可分离卷积爬轻量路线

金字塔的核心洞察：

- **空间维度递减**：224 → 112 → 56 → 28 → 14 → 7 → 1（与 AlexNet 相同路径，但每一步都靠深度可分离卷积大幅省力）
- **特征通道递增**：3 → 64 → 128 → 256 → 512 → 1024（稳步翻倍，没有 AlexNet 的先升后调——更简洁的增长策略）
- **参数量分布**：约 4.2M 总参数——仅 AlexNet 的 7%，原因是每个卷积都拆成了 Depthwise+Pointwise，中间省掉了大量交叉连接
- **14×14 是中海拔大本营**：6 层堆叠在此——空间信息还够用，计算量又不至于太大，是性价比最高的深度投入点

为什么 14×14 处堆了 5 层？

这是 MobileNet 设计中最精妙的地方。14×14 是“中等分辨率”——既保留了足够空间信息，又不至于太大。在这个分辨率上堆深度，让网络在不增加太多计算量的前提下学到更复杂的特征组合。

就像一个登山者在中海拔建立大本营——这里氧气还够，但已经接近关键路线。在这里多待几个回合，为最后的冲顶做准备。

代码实践

```
import torch
import torch.nn as nn

def conv_dw(in_channels, out_channels, stride):
    """深度可分离卷积块

    两个操作：
    1. Depthwise: 3x3 逐通道卷积 —— 空间混合（见 cnn-basics）
    2. Pointwise: 1x1 卷积 —— 通道混合

    参数量: 3x3x $in + in \times out$  (vs 标准卷积的  $3 \times 3 \times in \times out$ )
    节省: 约  $(out-1)/(out)$  的参数
    """
    return nn.Sequential(
        # Depthwise: 每个通道独立做 3x3 卷积
        nn.Conv2d(in_channels, in_channels, kernel_size=3,
                  stride=stride, padding=1, groups=in_channels, bias=False),
        nn.BatchNorm2d(in_channels),
        nn.ReLU(inplace=True),

        # Pointwise: 1x1 卷积混合通道
        nn.Conv2d(in_channels, out_channels, kernel_size=1,
                  stride=1, padding=0, bias=False),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True),
    )

class MobileNet(nn.Module):
    """MobileNet V1 实现

    核心设计: 深度可分离卷积 —— 把空间混合和通道混合拆开

    超参数:
    - alpha: 宽度乘数 (0.25, 0.5, 0.75, 1.0)
    - 参数量  $\alpha=1.0$  时约 4.2M,  $\alpha=0.25$  时约 0.5M
    """
    def __init__(self, num_classes=1000, alpha=1.0):
        super(MobileNet, self).__init__()
```

(续下页)

(接上页)

```

def conv_layer(in_c, out_c, stride):
    """每一层的通道数都乘以 alpha"""
    in_c = int(in_c * alpha)
    out_c = int(out_c * alpha)
    return conv_dw(in_c, out_c, stride)

self.model = nn.Sequential(
    # 第一层用标准卷积（输入只有 3 通道，depthwise 没意义）
    nn.Conv2d(3, int(32 * alpha), 3, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(int(32 * alpha)),
    nn.ReLU(inplace=True),

    # 深度可分离卷积层
    conv_layer(32, 64, 1),      # 112×112
    conv_layer(64, 128, 2),     # 56×56
    conv_layer(128, 128, 1),
    conv_layer(128, 256, 2),     # 28×28
    conv_layer(256, 256, 1),
    conv_layer(256, 512, 2),     # 14×14

    # 在 14×14 处堆 5 层 —— 中等分辨率下的特征深化
    conv_layer(512, 512, 1),
    conv_layer(512, 512, 1),
    conv_layer(512, 512, 1),
    conv_layer(512, 512, 1),
    conv_layer(512, 512, 1),

    conv_layer(512, 1024, 2),    # 7×7
    conv_layer(1024, 1024, 1),

    # 全局平均池化替代全连接 —— 进一步省钱
    nn.AdaptiveAvgPool2d(1),
)

self.fc = nn.Linear(int(1024 * alpha), num_classes)

def forward(self, x):
    x = self.model(x)           # (B, C, 1, 1)
    x = x.view(x.size(0), -1)   # (B, C)

```

(续下页)

(接上页)

```

    x = self.fc(x)                # (B, num_classes)
    return x

# 参数量验证
model_full = MobileNet(alpha=1.0)
model_half = MobileNet(alpha=0.5)
model_quarter = MobileNet(alpha=0.25)

print(f"α=1.0: {sum(p.numel() for p in model_full.parameters()),} 参数")
print(f"α=0.5: {sum(p.numel() for p in model_half.parameters()),} 参数")
print(f"α=0.25: {sum(p.numel() for p in model_quarter.parameters()),} 参数")

```

总结

维度	内容
核心洞察	把卷积拆成 Depthwise（空间）+ Pointwise（通道）—— 参数量减少 8-9 倍
哲学	准确率不是唯一的目标。在给定的计算预算下最大化效果
两个旋钮	α （宽度）+ ρ （分辨率）—— 灵活适配从手机到服务器的任何设备
历史意义	开启了 CNN "瘦身"的整个方向 —— MobileNetV2/V3、ShuffleNet、EfficientNet 都受其启发

登山者笔记：不是每个攀登者都雇得起大队人马。一个人、一把冰镐 —— 但如果你知道哪些装备真正必要、哪些可以舍弃，你仍然能爬到足够高的地方。

下一步

MobileNet 告诉你：**极致的省钱，也能爬**。但如果你不是"几乎没钱"，而是"有一笔预算，想花在刀刃上"呢？

深度、宽度、分辨率 —— 三个维度不是独立的。能不能按一个最优比例一起调？

EfficientNet：每一分钱，花在刀刃上 —— 走。

3.4.8 EfficientNet：每一分钱，花在刀刃上

MobileNet：一个人，一把冰镐 教会了我们一件事：**极致省钱，也能爬**。深度可分离卷积把参数量砍到 1/9，一个人一把冰镐就能出发。

但大多数攀登者的处境不在两个极端 —— 既不是 Google 的无限预算，也不是嵌入式芯片的极限抠门。你有一笔预算：几个 GPU，几天的训练时间，几百 MB 的模型限制。**怎么把这笔钱花在刀刃上？**

关键问题：深度、宽度、分辨率 —— 三个旋钮，应该按什么比例一起拧？

历史背景：三个旋钮，各自为政

在 EfficientNet 之前，人们已经知道扩大模型可以提升性能：

- **更深** (ResNet-152 $\rightarrow$ ResNet-1000)：更多层 = 更强的表达能力
- **更宽** (MobileNet 的 α 参数)：更多通道 = 更丰富的特征
- **更大分辨率** ($224 \times 224 \rightarrow 380 \times 380 \rightarrow 600 \times 600$)：更精细的输入 = 更细节的信息

但问题是 —— **大家都是凭感觉调的**。有人加深度，有人加宽度，有人加大分辨率。没有一个系统性的原则告诉你在给定预算下，三者怎么组合最优。

2019 年，Mingxing Tan 和 Google 团队提出了 **EfficientNet** [TL19]。核心洞察只有一个，但它统一了三年的零散经验：

深度、宽度、分辨率不是独立的。它们应该按一个复合比例同时缩放。

复合缩放：为什么一起拧才对？

直觉

想象你要建一个更大的登山营地：

- **更宽** = 多雇人（更细的分工：有人专门探路，有人专门做饭）
- **更深** = 建更多补给站（更深的补给线：大本营 $\rightarrow$ C1 $\rightarrow$ C2 $\rightarrow$ C3）
- **更大分辨率** = 更精细的地图（能看到更小的冰裂缝）

如果你只加人不建补给站 —— 人挤在低海拔，到了高海拔没人能送物资。如果你只建补给站不加入 —— 空有路线，人手不够。如果你只换精细地图不加入也不建站 —— 看到了冰裂缝但没人能处理。

三者必须按比例一起加。

数学上，EfficientNet 用了一个复合系数 ϕ ：

$$d = \alpha^\phi \quad (\text{深度：补给站数量})$$

$$w = \beta^\phi \quad (\text{宽度：每站人数})$$

$$r = \gamma^\phi \quad (\text{分辨率：地图精细度})$$

其中 $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ —— 意味着每增大 ϕ 一个单位，总计算量翻倍。而 α, β, γ 的具体值是通过在**小模型上的网格搜索**找到的： $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$ 。

为什么比例是搜索出来的，不是推导出来的？

公平地说， $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$ 这三个数字没有任何理论必然性。它们是在一个 8 卡 GPU 上对小基线模型做网格搜索找到的最优组合。

但这不是 EfficientNet 的"弱点"——这恰恰是它的务实之处。就像登山者不会从理论推导出"氧气瓶应该带 3.2 升而不是 3.1 升"——你在低海拔试了不同组合，选最好的，然后在大山上按比例放大。

架构家族：从 B0 到 B7

基于复合缩放，EfficientNet 推出了一个模型家族。基线模型 EfficientNet-B0 是通过 NAS（神经架构搜索）找到的，然后通过增大 ϕ 得到 B1 到 B7：

模型	ϕ	参数	Top-1 准确率	相比 ResNet-50
EfficientNet-B0	0	5.3M	77.1%	$4.5\times$ 更小，同准确率
EfficientNet-B1	1	7.8M	79.1%	—
EfficientNet-B3	3	12M	81.6%	—
EfficientNet-B5	5	30M	83.6%	—
EfficientNet-B7	7	66M	84.3%	当时 SOTA，比 ResNet-152 还高

登山者的选择

你的预算	建议模型	比喻
一台手机	B0 (5.3M)	一个人、一天、基本装备
一台 GPU 服务器	B3 (12M)	小团队、一周
多卡集群	B5 (30M)	中型探险队
土豪	B7 (66M)	直升飞机空投补给

MobileNet vs EfficientNet：省钱的两条路

	MobileNet	EfficientNet
回答的问题	几乎没钱，怎么爬？	有一笔预算，怎么花？
核心操作	拆开卷积（深度可分离）	按比例缩放三个维度
哲学	极致瘦身——先砍到骨头，再往上加	系统优化——不瘦身，而是聪明地胖
就像登山	一个人、一把冰镐	有限团队、精细分工、每分钱花在刀刃上

二者并不矛盾

事实上，EfficientNet 的基线架构本身就用到了深度可分离卷积 —— MobileNet 的“拆开”思想。EfficientNet 的创新不在于“怎么省”，而在于“省完之后，多出来的预算怎么分配”。

MobileNet 回答的是“怎么省到极致”。EfficientNet 回答的是“省出了预算之后，往哪个方向花最值”。

代码实践

```
import torch
import torch.nn as nn
import math

class MBConvBlock(nn.Module):
    """Mobile Inverted Bottleneck Conv Block (EfficientNet 的基本构建块)

    这是 MobileNetV2 的倒置残差块 + SE 注意力 ({doc}`se-net`):
    1. 1×1 扩展（升维 —— 先给更多“呼吸空间”）
    2. Depthwise 3×3（空间混合）
    3. SE 注意力（自适应通道加权）
    4. 1×1 压缩（降维 —— 回到原来的通道数）
    5. 残差连接（如果 shape 匹配）

    为什么叫“倒置”？因为传统 bottleneck 是先压缩再扩展，这里反过来：先扩展再压缩。
    """
    def __init__(self, in_channels, out_channels, kernel_size,
                 stride, expand_ratio, se_ratio=0.25):
        super().__init__()

        # 是否使用残差连接
        self.use_residual = (stride == 1 and in_channels == out_channels)

        # 扩展后的通道数
        expanded_channels = in_channels * expand_ratio

        # 1. 1×1 扩展（如果有扩展）
        self.expand_conv = nn.Sequential(
            nn.Conv2d(in_channels, expanded_channels, 1, bias=False),
            nn.BatchNorm2d(expanded_channels),
            nn.SiLU() # Swish 激活 —— 比 ReLU 更平滑，EfficientNet 的默认选择
```

(续下页)

(接上页)

```

) if expand_ratio != 1 else nn.Identity()

# 2. Depthwise 3x3 卷积 —— {ref}`mobilenet` 的核心思想
self.depthwise_conv = nn.Sequential(
    nn.Conv2d(expanded_channels, expanded_channels, kernel_size,
              stride, padding=kernel_size//2,
              groups=expanded_channels, bias=False),
    nn.BatchNorm2d(expanded_channels),
    nn.SiLU()
)

# 3. SE 注意力模块 —— {doc}`se-net`
se_channels = max(1, int(in_channels * se_ratio))
self.se = nn.Sequential(
    nn.AdaptiveAvgPool2d(1),
    nn.Conv2d(expanded_channels, se_channels, 1),
    nn.SiLU(),
    nn.Conv2d(se_channels, expanded_channels, 1),
    nn.Sigmoid()
)

# 4. 1x1 压缩
self.project_conv = nn.Sequential(
    nn.Conv2d(expanded_channels, out_channels, 1, bias=False),
    nn.BatchNorm2d(out_channels)
)

def forward(self, x):
    residual = x
    x = self.expand_conv(x)
    x = self.depthwise_conv(x)
    x = x * self.se(x) # SE 注意力重标定
    x = self.project_conv(x)

    if self.use_residual:
        x = x + residual # 残差连接 —— {ref}`res-net` 的补给站
    return x

def efficientnet_b0(num_classes=1000):

```

(续下页)

(接上页)

```
"""EfficientNet-B0 的简化实现
```

架构原理：

- 基于 MBConv 块的倒置残差结构
- 复合缩放：如果要做 B1-B7，只需按比例调整深度/宽度/分辨率

参数量：约 5.3M

Top-1 准确率：77.1% (ImageNet)

```
"""
```

```
return nn.Sequential(
    # Stem: 标准卷积，快速降采样
    nn.Conv2d(3, 32, 3, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(32),
    nn.SiLU(),

    # Stage 1: 112×112
    MBConvBlock(32, 16, 3, stride=1, expand_ratio=1),

    # Stage 2: 112→56
    MBConvBlock(16, 24, 3, stride=2, expand_ratio=6),
    MBConvBlock(24, 24, 3, stride=1, expand_ratio=6),

    # Stage 3: 56→28
    MBConvBlock(24, 40, 5, stride=2, expand_ratio=6),
    MBConvBlock(40, 40, 5, stride=1, expand_ratio=6),

    # Stage 4: 28→14
    MBConvBlock(40, 80, 3, stride=2, expand_ratio=6),
    MBConvBlock(80, 80, 3, stride=1, expand_ratio=6),
    MBConvBlock(80, 80, 3, stride=1, expand_ratio=6),

    # Stage 5: 14×14
    MBConvBlock(80, 112, 5, stride=1, expand_ratio=6),
    MBConvBlock(112, 112, 5, stride=1, expand_ratio=6),
    MBConvBlock(112, 112, 5, stride=1, expand_ratio=6),

    # Stage 6: 14→7
    MBConvBlock(112, 192, 5, stride=2, expand_ratio=6),
    MBConvBlock(192, 192, 5, stride=1, expand_ratio=6),
    MBConvBlock(192, 192, 5, stride=1, expand_ratio=6),
```

(续下页)

(接上页)

```

        MBConvBlock(192, 192, 5, stride=1, expand_ratio=6),

        # Stage 7: 7×7
        MBConvBlock(192, 320, 3, stride=1, expand_ratio=6),

        # Head
        nn.Conv2d(320, 1280, 1, bias=False),
        nn.BatchNorm2d(1280),
        nn.SiLU(),
        nn.AdaptiveAvgPool2d(1),
        nn.Flatten(),
        nn.Dropout(0.2),
        nn.Linear(1280, num_classes),
    )

# 参数量验证
model = efficientnet_b0()
params = sum(p.numel() for p in model.parameters())
print(f"EfficientNet-B0: {params:,} 参数")

```

总结

维度	内容
核心洞察	深度、宽度、分辨率不是独立的 —— 复合缩放，按比例一起拧
哲学	在给定计算预算下，将每一分钱系统性地分配到三个维度
三系数	$\alpha = 1.2$ （深度）、 $\beta = 1.1$ （宽度）、 $\gamma = 1.15$ （分辨率） —— 小模型网格搜索得出
历史意义	统一了 ResNet（深度派）、MobileNet（宽度派）、高分辨率方法的经验

登山者笔记：MobileNet 告诉你一个人怎么爬。EfficientNet 告诉你有团队怎么爬。但底层道理是一样的 —— 不是“更多就一定更好”，而是“更多要加在对的地方”。

下一步

两座山都翻越了。从 1989 年 LeNet 的练习峰首登，到 2019 年 EfficientNet 的复合缩放 —— 三十年架构演化，一言以蔽之：

好的设计，是把山的形状刻进装备里。

但登山者不会在山顶停留。八年的路走完了——回头看，每一件装备都在回答同一个问题。

登山下篇回顾：八年，八件装备——坐下来，写登山笔记。

3.4.9 登山下篇回顾：八年，八件装备

AlexNet 的第一声炮响。Inception 的多尺度工具。ResNet 的补给站。SE-Net 和 CBAM 的聚焦。MobileNet 的一人一稿。EfficientNet 的每一分钱花在刀刃上。

2012 到 2019。从错误率 28% 到 15%，再到 84.3% 的 Top-1。八年间每一次“爬不上去”，都逼出了一件新装备。

八年级装备清单

年份	架构	解决的问题	核心洞察
2012	AlexNet	ImageNet 首登	ReLU + Dropout + GPU —— 深度学习能爬真山
2014	Inception	单一感受野不够	多分支并行 —— 不同尺度，同时捕捉
2015	ResNet	越深越差（退化）	跳跃连接 —— 梯度有捷径，深层能训练
2017	SE-Net	对所有通道一视同仁	Squeeze→Excite→Scale —— 动态通道加权
2018	CBAM	只看通道不够	通道 + 空间注意力 —— 先选频道，再调区域
2017	MobileNet	模型太大塞不进手机	深度可分离卷积 —— 空间/通道拆开，参数砍到 1/9
2019	EfficientNet	深度/宽度/分辨率各自调	复合缩放 —— 三个旋钮按比例一起拧

回过头看，这些装备不是孤立的。

Inception 的多尺度本质上是**操控感受野**。ResNet 的跳跃连接本质上是**操控信息通路**。SE-Net/CBAM 本质上是**操控注意力**。MobileNet/EfficientNet 本质上是**操控效率**。

同一件事，四个维度。八年的经验，可以炼成一条原则：**面对一座全新的山，不是加哪个模块——是判断哪个维度出了问题。**

这就是下一站要做的。

架构心法：**从会用装备到会设计装备——走。把经验炼成心法。**

3.5 架构心法：从会用装备到会设计装备

3.5.1 诊断入门：信息论听诊器

信息论听诊器——瓶颈在哪？容量不够还是信息传不过去？

下篇：真正的未登峰翻完了。AlexNet 的首登、Inception 的多尺度、ResNet 的补给站、SE-Net/CBAM 的聚焦、MobileNet/EfficientNet 的预算学——八件装备，八年。

回过头看，它们不是孤立的。Inception 本质上是**操控感受野**，ResNet 本质上是**操控信息通路**，SE-Net/CBAM 本质上是**操控注意力**，MobileNet/EfficientNet 本质上是**操控效率**。

同一件事，四个维度。本章把经验炼成原则——不是“加哪个模块”，而是“**面对一座全新的山，判断哪个维度出了问题**”。

学习目标

完成本章后，你将能够：

1. **用信息论视角诊断瓶颈**：区分“容量不够”和“信息传不过去”
2. **操控感受野**：根据目标大小选择合适的多尺度策略
3. **操控深度与连接**：用跳跃连接和特征融合保证信息流动
4. **操控注意力**：知道什么时候需要通道注意力、空间注意力或长程依赖
5. **操控效率**：在效果与速度之间做出合理权衡
6. **建立设计直觉**：面对任何模型，能自主判断需要改什么、怎么改

先想想我们目前积累的“零件库”：

零件	来源	解决什么问题
卷积（不同大小）	卷积：发现装备	提取局部特征
池化	LeNet-5：练习峰首登	降采样、增大感受野
跳跃连接	ResNet：高原反应补给站	让梯度能传回浅层
多分支并行	Inception：多尺度工具	同时看多个尺度
通道注意力	SE-Net：学会聚焦	知道哪个特征重要
空间注意力	CBAM：哪里重要 + 什么重要	知道哪里重要

现在的问题是——这些零件之间是什么关系？它们的设计思想能不能提炼成一套通用原则？

信息论：诊断架构瓶颈的工具

在开始改造之前，我们需要一个“听诊器”——能告诉你模型问题在哪的工具。

这个听诊器来自信息论：**神经网络本质上是把输入信息压缩、变换、提取的过程**。信息在层层传递中会面临两个问题：

信息漏斗：容量不够

想象一个沙漏——上宽下窄。数据从 3072 维压到 256 维，每压缩一次就丢失一些信息。如果网络太“窄”（通道

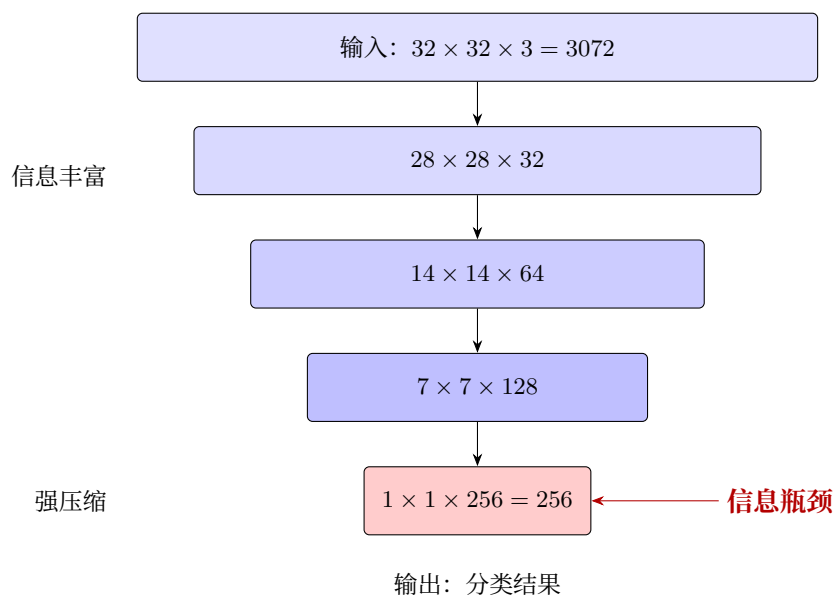


图 28: 信息漏斗：容量瓶颈

数太少)，就像漏斗的口太小——还没提取够就挤过去了。这就是**容量不够**。

梯度消失：传不过去

另一种问题更隐蔽：**信息量够，但传不回来**。

想象一个传话游戏——100 个人，每人说一遍。理论上每个人都能说清楚，但实际上第 1 个人的话传到第 100 个人时早已面目全非。

梯度消失 (Vanishing Gradient) 中讨论过：梯度是各层影响的乘积。层数一多，浅层的梯度就像在传话游戏中一样，衰减到几乎为零。这就是**信息传不过去**。

互信息

在深入讨论之前，我们需要一个信息论的核心概念：**互信息 (Mutual Information)**。

$$I(X;Y) = H(X) - H(X|Y)$$

直觉理解：互信息衡量的是“知道 Y 后， X 的不确定性减少了多少”。在神经网络中：

- X ：输入数据
- Y ：模型输出或中间层特征
- $I(X;Y)$ ：模型保留了多少输入信息

互信息越大，说明模型从输入中提取并保留了更多相关信息。

用互信息诊断网络：一个具体例子

想象你要设计一个分类网络，输入是 32×32 的 CIFAR-10 图像（3072 维），输出是 10 个类别。我们来看看互信息如何帮你诊断问题。

场景一：容量不够

你设计了一个"极简"网络：

- 1 个卷积层（ $3 \rightarrow 8$ 通道）
- 全局平均池化 $\rightarrow$ 10 维输出

计算互信息：

- 输入熵 $H(X)$ ：CIFAR-10 图像的信息量，约 3072 比特（假设每个像素独立）
- 中间表示：卷积后 $28 \times 28 \times 8 = 6272$ 个数值，看似比输入还大
- 但实际有效信息：只有 8 个通道，每个通道是一个"特征图"

问题在哪？8 个通道意味着网络只能用 8 个"模式"来描述所有图像。CIFAR-10 有 10 个类别，每个类别有不同的纹理、形状、颜色——8 个模式远远不够。

从互信息角度： $I(X; \hat{Y})$ 的上限是 8 比特（因为中间表示只有 8 个通道的特征），而分类 10 个类别需要 $\log_2(10) \approx 3.3$ 比特——看似够，但每个类别内部还有巨大变化（同一类别的不同图像差异很大）。

信息瓶颈的量化：如果瓶颈层只有 256 维（如 ResNet 的最后特征层），而你需要区分的类别有 1000 个（ImageNet），每个类别平均只能分到 0.256 维的"特征预算"——这显然不够。

场景二：信息传不过去

你设计了一个很深的网络：50 层，每层都是标准卷积+ReLU，没有跳跃连接。

前向传播时：

- 第 1 层能看到原始图像，信息丰富
- 第 10 层还能保留大部分信息
- 第 50 层：由于 ReLU 的截断效应和多次非线性变换，很多输入信息已经"丢失"了

反向传播时：

- 损失对第 50 层的梯度很清晰
- 传到第 40 层开始衰减
- 传到第 1 层几乎为零

互信息视角：

- $I(X; h_{50})$ ：输入与第 50 层特征的互信息——由于逐层变换，这个值可能很小

- $I(h_{50}; \hat{Y})$: 第 50 层与输出的互信息 —— 如果 h_{50} 已经丢失了关键信息, 这个值也上不去

这就是退化问题: 网络很深, 但浅层学不到东西, 深层也提取不到好特征。

互信息视角下的改造策略

改造策略	解决的问题	互信息解释	典型案例
加宽 (更多通道)	容量不够	增加 $I(X; h)$ 的上限	VGG 从 64→512 通道
加深 (更多层)	表征能力不足	学习更复杂的 $P(Y X)$	ResNet 152 层
跳跃连接	信息传不过去	保持 $I(X; h_L) \approx I(X; X)$	ResNet、U-Net
多尺度特征	丢失细节或全局	同时保留 $I(X_{local}; h)$ 和 $I(X_{global}; h)$	FPN、Inception
注意力机制	信息淹没	增强 $I(X_{task}; h)$, 抑制无关信息	SE-Net、CBAM
知识蒸馏	小模型容量不够	用小模型逼近大模型的 $I(X; Y)$	Teacher-Student

关键洞察

架构改造的本质是**控制信息流**:

- **扩大容量**: 让 $I(X; \hat{Y})$ 足够大 (能装下任务需要的信息)
- **保持通路**: 让 $I(X; h_L) \approx I(X; X)$ (信息能传到底层)
- **筛选信息**: 让 $I(X_{task}; h) \gg I(X_{noise}; h)$ (留下有用的, 去掉噪音)

所有改造技术 —— Inception、ResNet、注意力、DW 卷积 —— 都在解决这三个问题之一。

改造的四个维度

从信息论出发, 架构改造自然分为四个维度:

维度	核心问题	代表性策略	信息论映射
感受野	看什么?	多尺度、空洞卷积、FPN	信息入口 —— 多频段采样
深度与连接	怎么传?	跳跃连接、密集连接、特征融合	信息通路 —— 防止信息瓶颈
注意力	重点看哪?	通道注意力、空间注意力、自注意力	信息路由 —— 选择任务相关信息
计算效率	怎么更快?	DW 卷积、Bottleneck	信息压缩 —— 利用冗余降本

前三个是**效果维度** (让模型更好), 最后一个是**效率维度** (让模型更快)。

这四者不是孤立的。一个好的架构改造往往**同时考虑多个维度**。比如 ResNet 的跳跃连接不仅解决了梯度问题 (维度二), 也间接扩大了每层的有效感受野 (维度一)。

从本章开始

接下来的每一章聚焦一个维度。我们不会罗列所有技术（那是论文做的事），而是提炼**设计心法**——理解每种设计解决了什么信息论问题，什么时候该用。

学完本章后，你面对一个模型表现不佳时，不会再盲目地“加层、加参数、加注意力”，而是能系统地判断：

问题在哪？感受野不够、信息传不过去、还是没注意重点？然后对症下药。

第一件装备。望远镜——**操控感受野：网络应该看多大**。

3.5.2 操控感受野：网络应该看多大

第一件装备：望远镜。看裂缝还是看山脊？多尺度、空洞卷积、FPN。

感受野告诉我们：卷积层越深，感受野越大。但 **Inception：多尺度工具** 揭示了一个根本问题——**每一层只有一个固定大小的感受野**。

这就是操控感受野要解决的核心矛盾：**不同的目标需要不同大小的感受野，但传统 CNN 只有一个**。

什么时候需要改造感受野

先确认你是不是真的遇到了感受野问题：

症状	可能原因	是否感受野问题
大物体（占据半张图）识别差	感受野只能看到局部	是
同时有大物体和小物体的图效果差	单一感受野顾此失彼	是
所有物体识别都差	可能不是感受野问题	先检查训练
小物体识别差、大物体尚可	大感受野丢失细节	是

关键判断

如果加大卷积核尺寸（比如 $3 \rightarrow 7$ ）能提升效果，那大概率是感受野问题。

策略一：多尺度并行

这是 **Inception：多尺度工具** 的核心思想，但我们从**设计心法**而非架构本身来理解它。

本质：不说“用多大”，而说“都试试”。同时运行多个感受野的卷积，让网络自己选。**适用场景**：你有多个大小悬殊的目标类别（比如自然图像分类）。

代价与缓解：参数和计算量增加（多个分支同时跑）。**Inception：多尺度工具** 中的关键创新是用 1×1 卷积先降维——在昂贵的 3×3 和 5×5 卷积之前，先用 1×1 把通道数降下来：

心法：这是一种“信息保险”——不确定感受野应该多大时，把不同大小的都保留下来。 1×1 降维则是“用计算换信息”的精打细算。

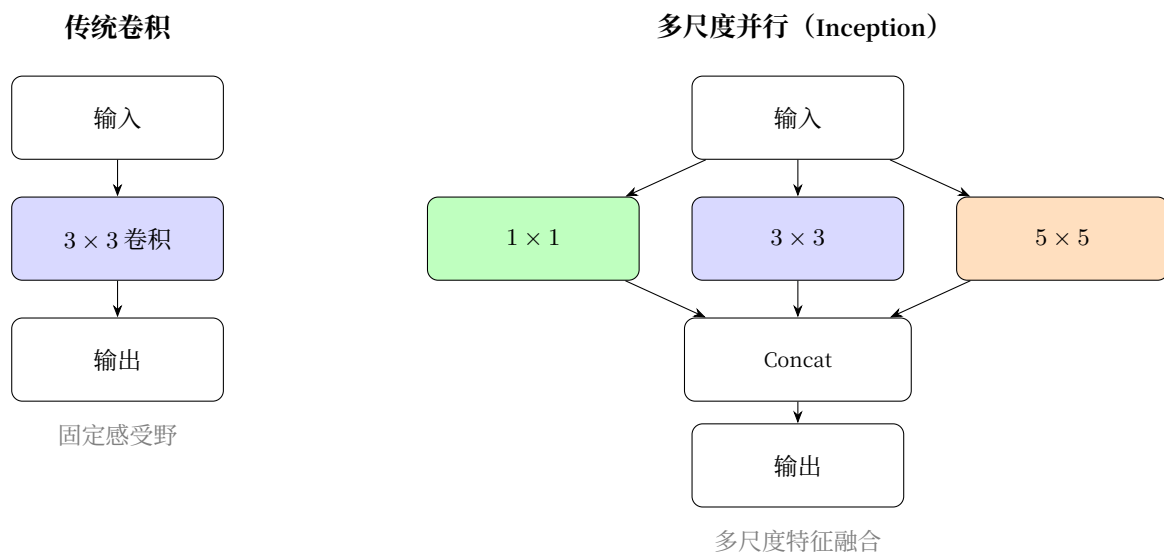


图 29: Inception 的多尺度并行

策略二：空洞卷积

诊断入门：信息论听诊器中我们简单介绍了空洞卷积——它在不增加参数的情况下扩大感受野。现在从设计心法来理解它。

本质：用稀疏采样替代密集采样。 3×3 卷积 + 空洞率 2 = 5×5 的感受野，但参数还是 9 个。**Inception：**多尺度工具中我们已用 TikZ 图展示了普通卷积与空洞卷积的直观对比，这里不再重复。

设计原则：

空洞率	3×3 卷积的感受野	相当于普通	参数量
1	3×3	3×3	9
2	5×5	5×5	9
3	7×7	7×7	9
4	9×9	9×9	9

心法：空洞卷积是“用稀疏换取广度”。你牺牲了每个卷积核的采样密度，换来不花钱的大感受野。

代价：网格效应 (Gridding Effect)

稀疏采样不是免费的午餐。当空洞率过大时，卷积核只采样输入的“离散点”，会丢失局部连续性信息，产生棋盘格状的伪影：**缓解策略：**

- 不要把空洞率设得太大（通常 ≤ 16 ）
- 不同空洞率的卷积并行（ASPP），互相弥补采样盲区

适用场景：需要大感受野但参数量、计算量受限（分割任务、轻量模型）。

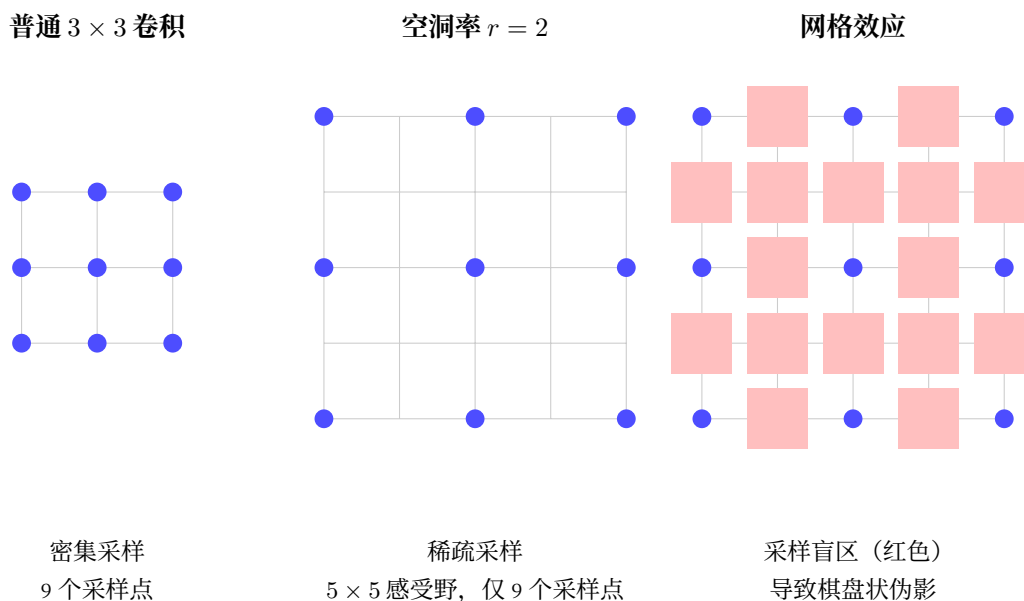


图 30: 空洞卷积的网格效应对比

典型应用 —— ASPP (Atrous Spatial Pyramid Pooling)

DeepLab 系列的核心模块, 同时并行 4 个不同空洞率的卷积 (空洞率=6, 12, 18, 24) + 1 个全局平均池化: ASPP

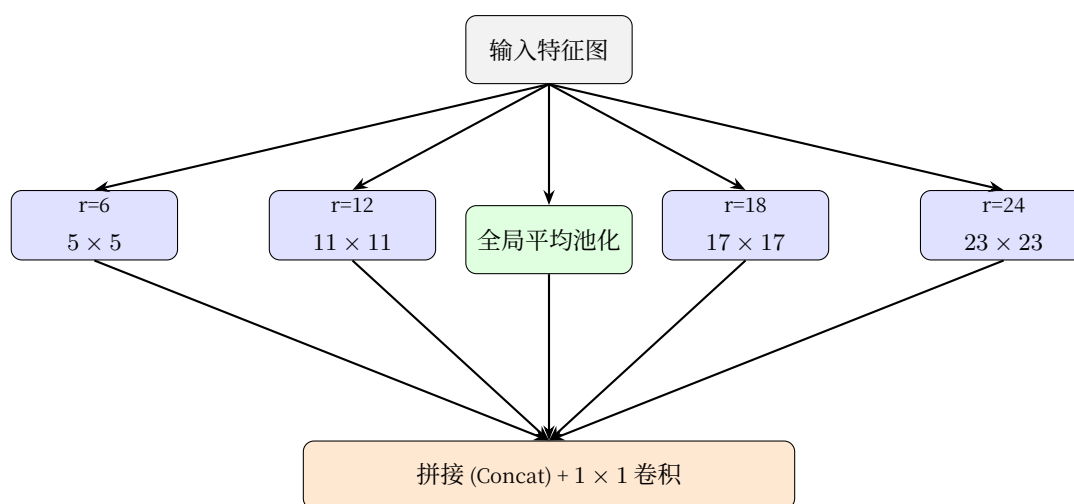


图 31: ASPP 结构: 多空洞率并行

与 Inception 的区别:

- Inception: 多个卷积核尺寸 (1×1 , 3×3 , 5×5)
- ASPP: 相同卷积核, 不同空洞率 (3×3 with rate=6,12,18,24)

前者增加参数, 后者不增加参数 —— 都是多尺度, 但代价不同。

策略三：特征金字塔（FPN）——不同深度、不同感受野

Inception 和空洞卷积解决的是同一层的多尺度问题。但 CNN 有一个天然的多尺度结构——不同深度的层自然拥有不同感受野。

浅层特征：分辨率高、感受野小 → 适合检测小物体
深层特征：分辨率低、感受野大 → 适合检测大物体

特征金字塔网络（FPN, Feature Pyramid Network）的洞察是：不给每一层单独分配目标大小，而是在不同深度的特征之间建立信息通道，让它们对话。

心法：FPN/PAN 的本质是在不同感受野的特征之间建立信息通道。信息不仅在层之间流动，还在不同感受野之间流动。

FPN 的结构：自上而下 + 横向连接

FPN 并不是简单的"把不同层的特征拼起来"，它有一个精妙的结构设计：为什么需要这种结构？

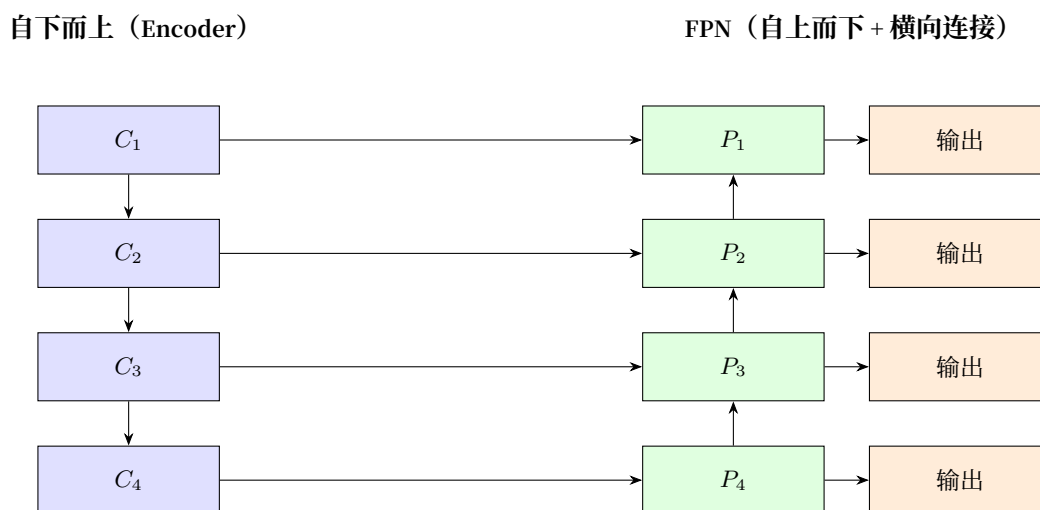


图 32: FPN 结构：自上而下路径与横向连接

单纯使用深层特征的问题：分辨率太低，小物体可能已经"消失"了。单纯使用浅层特征的问题：感受野太小，看不清全局上下文。

FPN 的解决方案：

1. **横向连接**：保留浅层的高分辨率细节
2. **自上而下路径**：把深层的强语义信息传递回浅层
3. **逐层融合**：每个输出层都同时具有"高分辨率 + 强语义"

融合机制的具体操作

横向连接和自上而下路径相遇时，不是简单相加，而是有一个精心设计的流程：关键设计选择：

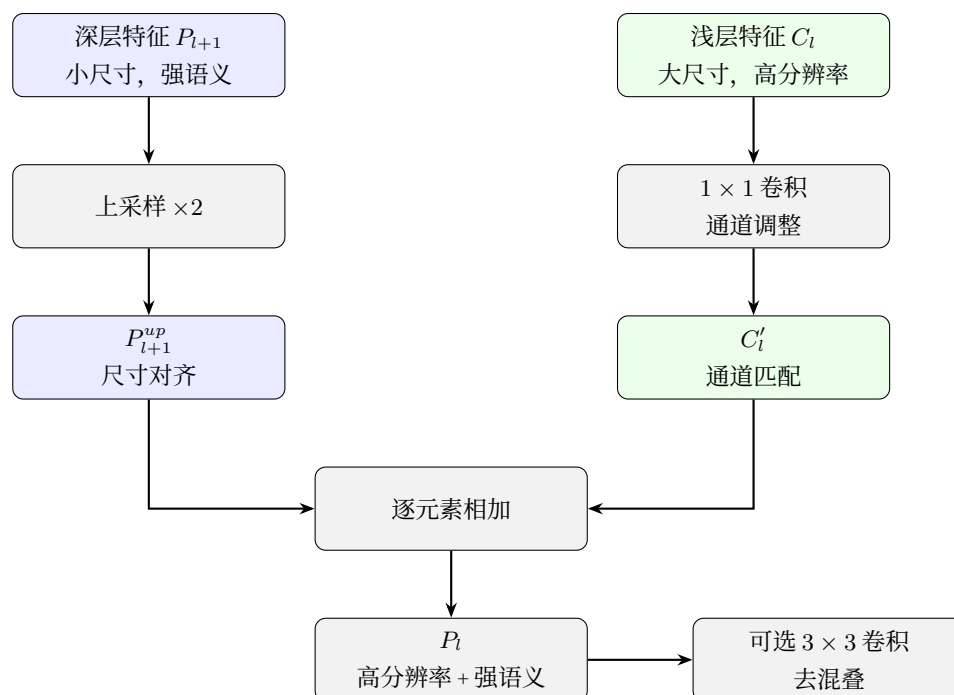


图 33: FPN 融合机制流程

操作	选项	FPN 的选择	原因
上采样	最近邻/双线性/转置卷积	最近邻或双线性	简单高效, 后续有 3×3 优化
通道调整	1×1 卷积/直接截断	1×1 卷积	可学习地选择重要通道
融合方式	相加/拼接	相加	保持通道数, 计算效率高
后处理	3×3 卷积/无	可选 3×3	消除上采样的混叠效应

为什么用**相加**而不是拼接?

- 相加: 通道数不变, 计算效率高, 适合"融合语义"
- 拼接: 通道数翻倍, 信息保留更完整但计算量大

FPN 选择相加, 是因为深层的强语义已经足够指导浅层检测 —— 目标是**融合**而非**堆叠**信息。

与 Inception 的本质区别

策略	多尺度来源	信息如何融合
Inception	同一层的多个卷积核	拼接 (concat)
FPN	不同深度的自然层次	自上而下 + 横向连接

Inception 是"横向并行", FPN 是"纵向融合"。两者可以结合使用 (如 PANet)。

三种策略的对比与选择

策略	感受野	计算代价	适用任务
多尺度并行 (Inception)	同一层多个固定大小	中等 (多分支)	分类 — 目标大小不确定
空洞卷积	不增参扩大感受野	低	分割 — 需要大感受野但计算受限
FPN/PAN	不同层不同感受野+融合	中高 (上采样+融合)	检测/分割 — 多尺度目标

设计心法

操控感受野的本质不是"选一个大小", 而是"让网络同时看到多个尺度"。

三种策略的区别仅在于实现方式: Inception 在同一层内并行多个卷积核、空洞卷积用稀疏采样换广度、FPN 利用网络深度天然的多尺度结构。

失败案例：什么情况下不该用

案例一：MNIST 上用 Inception

场景：MNIST 手写数字识别 (28×28 灰度图)，你在第一层就加了 Inception 模块 (1×1 、 3×3 、 5×5 、MaxPool 并行)。

结果：

- 训练时间翻倍
- 准确率从 99.2% 降到 99.0% (或没有提升)

为什么失败：

- MNIST 数字只占图像中心一小块，大小相对固定
- 5×5 卷积核几乎覆盖了整个数字，没有"多尺度"的必要
- 增加的分支引入了更多参数，但没有带来信息增益

教训：感受野策略的前提是"确实存在多尺度"。如果任务本身尺度单一 (如对齐良好的人脸、固定大小的字符)，多尺度就是浪费。

案例二：分割任务中空洞率过大

场景：医学图像分割（细胞分割），你用空洞率 16 的 3×3 卷积来扩大感受野。

结果：

- 大细胞分割还可以
- **小细胞大量丢失**——在特征图上彻底"消失"

为什么失败：

- 空洞率 16 意味着卷积核实际采样点间距为 16 像素
- 对于直径只有 20-30 像素的细胞， 3×3 空洞卷积只能看到 3 个采样点
- 信息严重欠采样，小目标被"跳过"

教训：空洞卷积的上限是目标最小尺寸。如果目标只有 20 像素，空洞率不应超过 5-7。

案例三：分类任务盲目用 FPN

场景：ImageNet 分类，你把 ResNet 改造成了 FPN 结构——多尺度输出、自上而下连接。

结果：

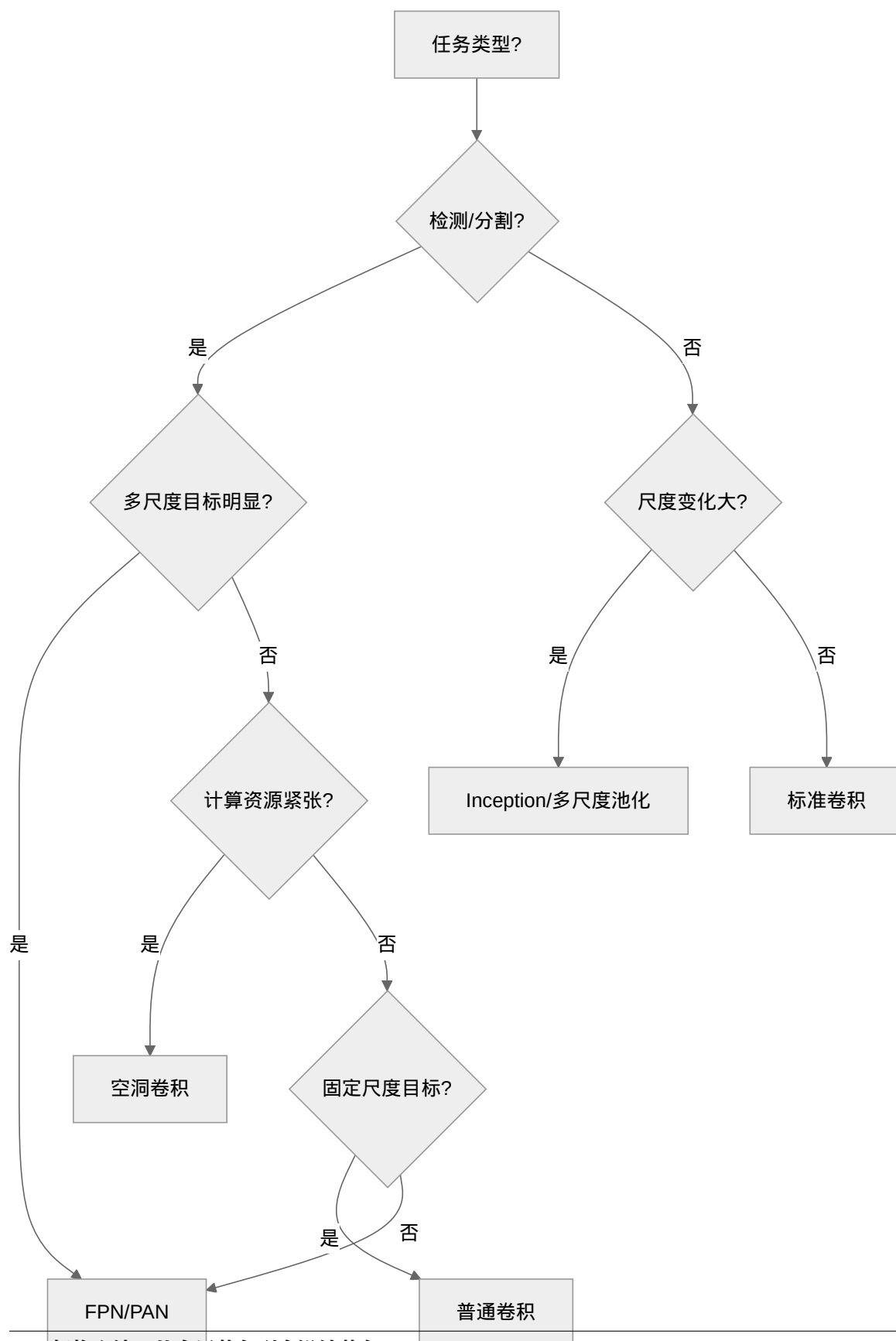
- 参数量和计算量显著增加
- Top-5 准确率没有提升，甚至略有下降

为什么失败：

- FPN 的核心价值是**多尺度目标的定位和分类**（检测/分割）
- 分类任务只需要一个全局判断，不需要"每层都输出"
- 额外的上采样和融合引入了计算负担，但没有对应的收益

教训：FPN 是为目标检测/分割设计的，**不是为纯分类**。分类任务用 FPN 是"杀鸡用牛刀"。

决策树：什么时候用什么



核心原则：感受野改造是为了解决**特定问题**，不是为了“看起来厉害”。如果 baseline 没有明显问题，不要引入复杂度。

信息论视角：多尺度 = 多频段采样

从信息论来看，感受野策略都是**多频段采样** [TPB00]：

- 自然界图像的互信息分布在不同的空间频率上
- 小感受野捕捉高频（细节），大感受野捕捉低频（全局结构）
- 单一感受野 = 信息窄带采样，丢失其他频段的信息
- 多尺度策略 = 宽带采样，**最大化** $I(X; Y)$

诊断入门：信息论听诊器 中讨论的信息漏斗问题，感受野的操控策略本质上是在**空间维度上拓宽漏斗的入口**——不止截取一个频段，而是同时截取所有频段。

望远镜调好了。但看到的東西，能传回来吗？第二件装备——**操控深度与连接：信息如何在网络中流动。**

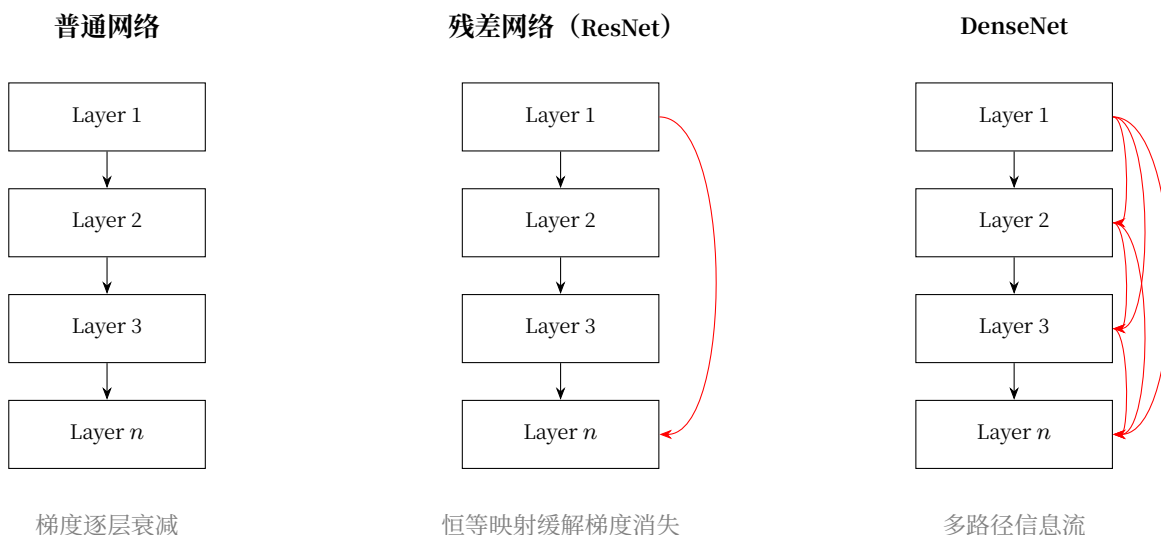
3.5.3 操控深度与连接：信息如何在网络中流动

第二件装备：补给线。高海拔还能呼吸吗？跳跃连接、密集连接、特征融合。

ResNet：高原反应补给站 揭露了一个惊人的事实：堆更多的层不一定更好，反而可能更差。根源在于**梯度消失**（Vanishing Gradient）——梯度在反向传播中逐层衰减，浅层收不到训练信号。

但 ResNet 只展示了跳跃连接这一种解法。本节从更根本的角度审视：**信息如何在网络中流动？有哪些方式保证信息不丢失？**

信息流动的三种通路



串行链——梯度消失

普通网络只有一个路径：Layer 1 → Layer 2 → ... → Layer n。信息在串行中丢失，梯度在乘法中归零。这就是诊断入门：信息论听诊器中讨论的**信息传不过去**。

跳跃连接——信息捷径

ResNet 的核心原理和梯度推导已在 [ResNet：高原反应补给站](#) 中详细介绍。从架构设计角度，我们提炼为一条核心原则：

每个模块都应该有一条"不做任何事"的退路。

数学上， $y = F(x) + x$ 中的 $+x$ 就是这条退路：

- 梯度可以沿跳跃路径直接回传（+1 项），不受残差分支影响
- 即使残差分支梯度消失，主梯度依然能保持（保底机制）

这解释了为什么 ResNet 能训练 100+ 层网络，而普通网络在 20 层就会退化。

密集连接——信息高速公路

DenseNet 把跳跃连接推到了极致：**每一层都连接到后面所有层**。这创建了一个密集的信息网络，梯度有多条路径可以回传，几乎不可能完全消失。

三种连接方式的对比：

连接方式	参数效率	信息安全	适用深度
串行	高	低——梯度消失	< 20 层
跳跃连接	中	中——有退路	20 ~ 200 层
密集连接	低——每层输入很多	高——多重保障	任意深度

DenseNet 为什么不流行

既然密集连接的信息安全度最高，为什么现实中 ResNet 比 DenseNet 更常见？三个原因：

1. **内存爆炸**：第 100 层的输入 = 前 99 层所有输出的拼接，通道数线性增长，显存消耗巨大
2. **特征冗余**：很多层的特征非常相似，全部保留存在大量浪费——这违背了**操控效率：效果、速度与硬件的三角权衡**中讨论的压缩原则
3. **跳跃连接已经够用**：对于绝大多数任务，每隔 1~2 层一跳就足够保证梯度流通，不需要每层接所有层

DenseNet 的价值在**思想的完整性**而非实用性——它证明了"连接密度"是一个可以调节的维度，你可以根据计算预算在 ResNet 和 DenseNet 之间做折衷。

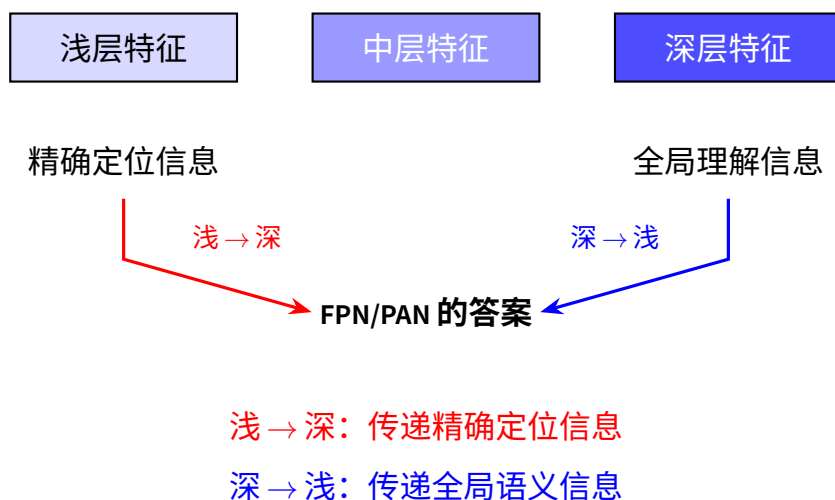
特征融合：不止是连接，而是对话

跳跃连接解决的是一根弦上的信息传递。但更深层的问题是：不同深度的特征如何互相配合？

操控感受野：网络应该看多大 中介绍了 FPN —— 从感受野角度看，它让不同深度的层各自负责不同大小的目标。但从信息流动角度看，FPN 做了一件更重要的事：在深层语义和浅层细节之间建立对话。

高分辨率 + 弱语义

低分辨率 + 强语义



设计心法：特征融合不是“把两个向量拼起来”，而是在不同抽象层次的特征之间建立双向信息流动——就像讨论问题时，从细节（浅层）和全局（深层）两个角度来回碰撞。

信息论视角：为什么连接方式决定了信息容量

从信息论来看，连接方式直接决定了每层能访问多大的信息空间：

连接方式	第 i 层的输入	信息量	瓶颈
串行	h_{i-1} 一个来源	逐层压缩	任一层的丢失永久不可恢复
跳跃连接	$h_{i-1} + x$ 两个来源	恒等映射保证下限	残差函数可能退化
密集连接	$[h_0, h_1, \dots, h_{i-1}]$ 全部来源	任意层的特征都可复用	计算量大
FPN 融合	同尺度 + 上采样特征	跨尺度信息互补	上采样可能引入噪声

关键洞察：多一条连接，就多一个信息通道。信息的可恢复性随连接数增加而提升，但代价是计算量。

设计心法

操控连接的黄金法则：

1. 每个模块都需要一条“退路”（恒等映射）——至少一层一跳

2. 特征融合不是简单的拼接，而是不同抽象层次的双向对话
3. 连接方式的选择取决于你的瓶颈：深度（加跳跃）还是尺度（做融合）

失败案例：过度连接的陷阱

案例一：浅层网络强行加残差连接

场景：你的网络只有 4 层，但每层都加了残差连接。

结果：

- 训练反而变慢了
- 最终准确率没有提升

为什么失败：

- 残差连接解决的是**深层网络**的梯度消失问题
- 4 层网络的梯度本来就能正常回传， γ^4 不会太小
- 额外的 $+x$ 操作引入了不必要的计算，反而可能破坏特征的逐层变换

教训：残差连接是“药”，不是“保健品”。10 层以下不需要，20 层以上才考虑。

案例二：DenseNet 的内存爆炸

场景：你设计了一个 100 层的 DenseNet，每层都接收之前所有层的输出。

结果：

- 训练时 GPU 内存溢出（OOM）
- 即使 batch size=1 也跑不动

为什么失败：

- DenseNet 的特征图数量随深度线性增长
- 第 L 层需要存储之前 $L - 1$ 层的所有特征图
- 100 层 DenseNet 的特征图数量是普通网络的 50 倍

教训：密集连接的信息收益有边际递减效应，但内存消耗是线性增长的。DenseNet 更适合 20-40 层，而不是 100+ 层。

案例三：跳跃连接位置放错

场景：你把 U-Net 的跳跃连接从"编码器 → 解码器同层"改成了"编码器最深层 → 解码器最浅层"。

结果：

- 分割边缘模糊
- 小目标检测不到

为什么失败：

- U-Net 跳跃连接的核心是**同尺度特征融合**（ 28×28 对 28×28 ）
- 编码器深层是 4×4 ，解码器浅层是 56×56 —— 尺度不匹配
- 上采样后的 $4 \times 4 \rightarrow 56 \times 56$ 丢失了精确定位信息

教训：特征融合的前提是**空间对齐**。跳跃连接不是"随便连"，而是需要精心设计融合点的位置。

案例四：FPN 中忘记横向连接

场景：你实现了 FPN，但只做了自上而下的上采样，没有横向连接（lateral connection）。

结果：

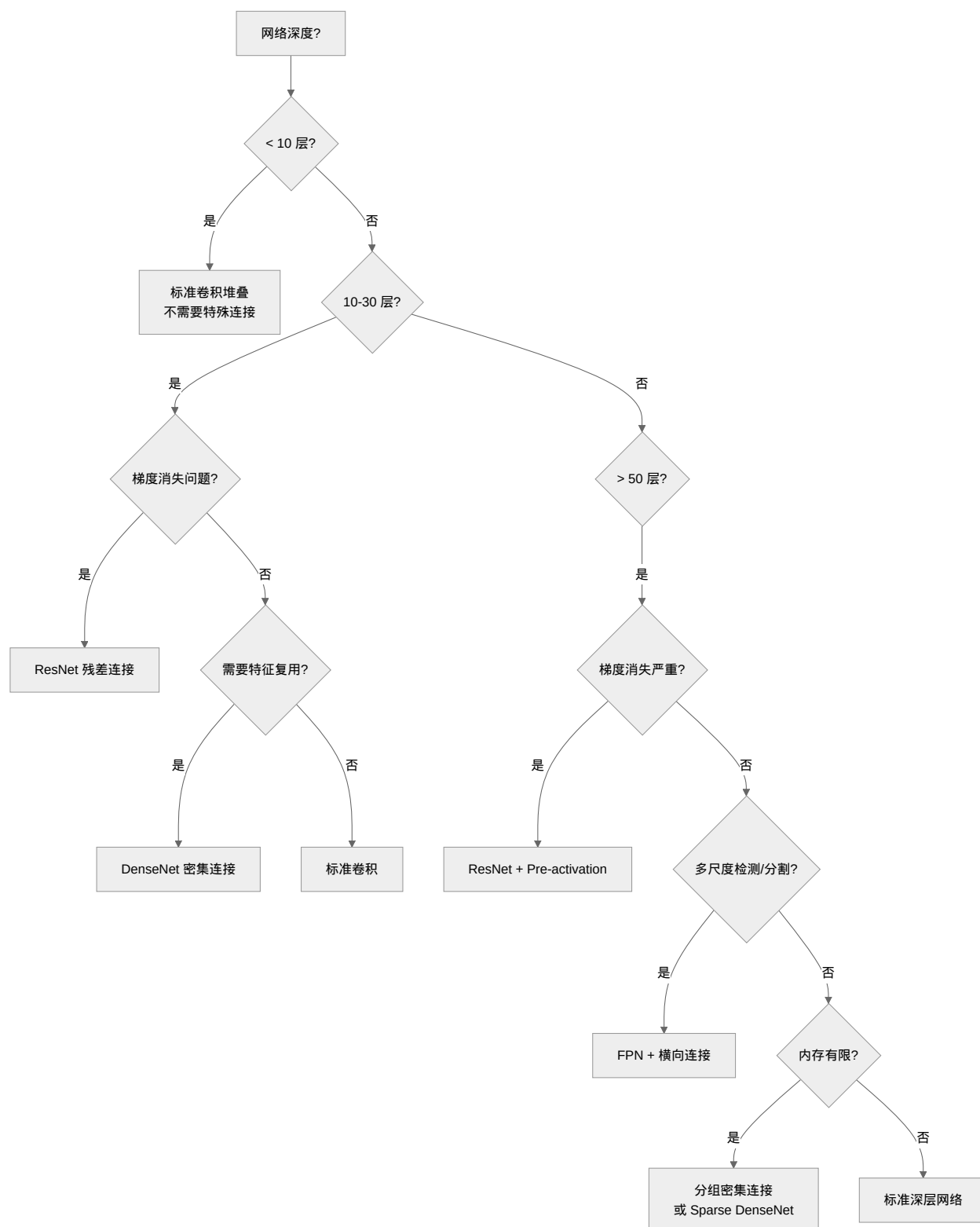
- 小目标检测率提升不明显
- 整体 mAP 比标准 FPN 低 2-3 个点

为什么失败：

- 纯上采样的深层特征虽然语义强，但空间精度差（混叠效应）
- 没有横向连接引入的浅层高分辨率特征，小目标定位不准
- 这正是 FPN 论文强调 lateral connection 的原因

教训：FPN 的价值在于**"双向融合"**，不是**单向传递**。少了横向连接，就退化成普通的上采样金字塔。

决策树：如何选择连接策略



核心原则：连接策略服务于信息流动问题。先诊断问题（梯度消失？信息丢失？多尺度？），再选策略，不要盲目堆叠连接。

通路畅通了。但网络知道该看哪吗？第三件装备——操控注意力与长程依赖：信息该走哪条路。

3.5.4 操控注意力与长程依赖：信息该走哪条路

第三件装备：聚焦。盯裂缝，忽略光滑岩面。通道注意力、空间注意力、长程依赖。

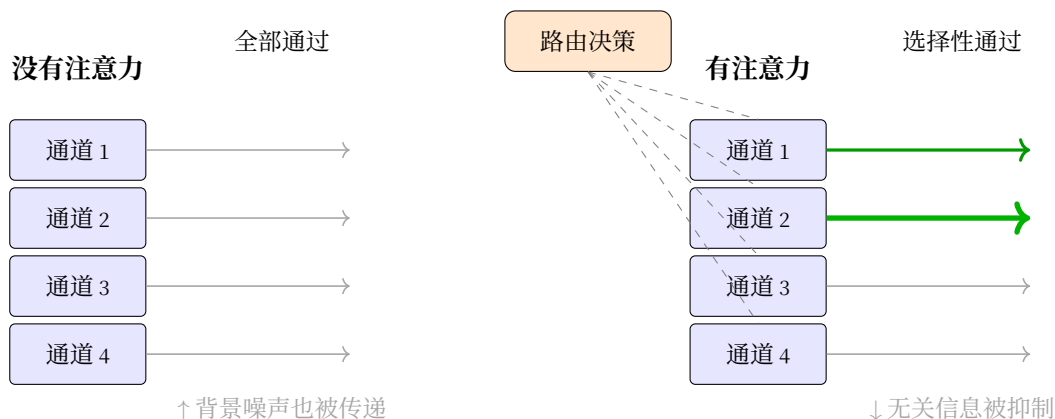
下篇：真正的未登峰 详细介绍了 SE-Net 和 CBAM 的具体实现。现在我们从设计心法的角度来理解——注意力不是一种“技巧”，而是神经网络中信息路由的通用机制。

重新理解注意力：从“模块”到“路由”

卷积：发现装备 告诉我们卷积有归纳偏置——平移等变性、局部感受野。这些归纳偏置让 CNN 很高效，但也带来了一个代价：所有输入位置被同等对待。

池化后，所有特征不管来自背景还是关键目标，都一视同仁地进入下一层。这就像高速公路所有车道的通行权一样——效率高，但不合理。

注意力的作用是：在信息通路上设置红绿灯，动态控制每个通道、每个位置的通行权。



三种注意力，三种路由策略

从路由的角度，注意力分为三个层次：

层次	路由问题	典型实现	信息论解释
通道注意力	哪个特征重要？	SE-Net、ECA-Net	在通道间分配信息权重
空间注意力	哪里重要？	CBAM 空间分支	在空间位置间分配信息权重
长程依赖	远处和近处能关联上吗？	Self-Attention、Non-local	全局信息路由，破除距离限制

通道注意力 —— 分配"什么"

通道注意力回答：这 256 个特征通道中，哪些对当前任务更重要？

SE-Net：学会聚焦 详细介绍了 SE-Net 的三步机制（压缩 → 激励 → 缩放）。从架构设计角度，通道注意力的核心价值在于：用极小的额外代价（<3% 参数），实现全局上下文对局部特征的动态调控。

架构层面的关键决策

在将 SE-Net 集成到你的网络时，需要考虑：

决策点	选项	影响	建议
降维比例 r	4/8/16/32	参数量 vs 表达能力	通常 16 是 sweet spot，轻量网络可用 32
插入位置	每个卷积后 / 仅深层	计算代价 vs 效果	只在深层（分辨率低）插入，节省计算
与残差连接的关系	残差内 / 残差外	梯度流动	建议放残差内（如 SE-ResNet），与跳跃连接协同

设计心法

通道注意力的本质是一个轻量化的信息路由决策器：

- **输入**：所有通道的全局统计信息（通过 Squeeze 获得）
- **决策**：学习通道间的重要性关系（通过 Excitation 实现）
- **输出**：动态调节各通道的信息流（通过 Reweight 执行）

这种"全局指导局部"的机制，让网络学会了根据输入内容自适应地重新分配计算资源——重要的特征被放大，无关的特征被抑制。

空间注意力 —— 分配"哪里"

空间注意力回答：图像中哪些位置更重要？看背景还是看目标？

CBAM：哪里重要 + 什么重要 展示了如何用通道压缩和 7×7 卷积生成空间注意力图。

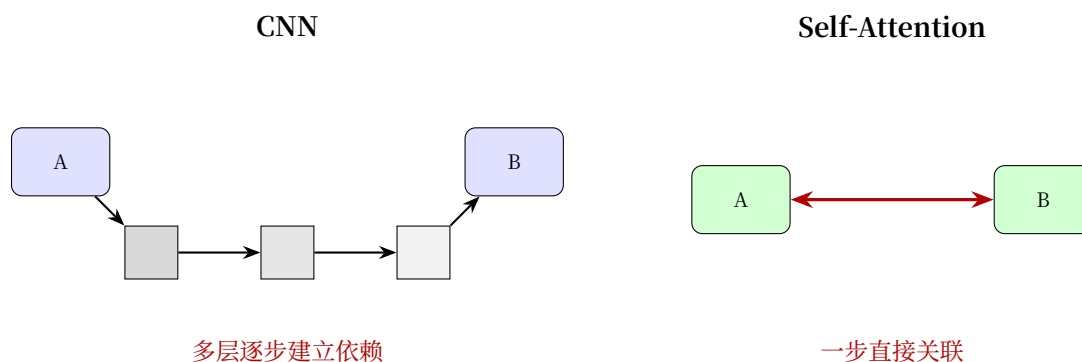
长程依赖 —— 破除距离的限制

这是传统 CNN 最微妙却最根本的一个局限。

问题：图像中两个相距较远的区域需要关联——比如篮球运动员的手和篮筐。卷积能看到吗？

答案：理论上能，但代价很大。

CNN 的感受野随深度线性增长。要让两个相距 100 像素的区域产生关联，需要堆很多层。而在层层传递中，远距离的关联信息很容易丢失。



Self-Attention 的方案：不管位置多远，直接计算关联。这彻底绕过了卷积的层层传递。

架构层面的权衡

Self-Attention 的核心机制是 **Query-Key-Value 计算**（详见 步骤四：分离"查询"和"被查询"——引入可学习的投影）：每个位置通过可学习的投影生成 Q/K/V，通过 QK^T 计算相似度，再对 v 加权求和。从架构设计角度，关键是理解它的**能力-代价权衡**：

维度	卷积	Self-Attention	影响
感受野	局部（随深度增长）	全局（一步到达）	远距离依赖建模能力
计算复杂度	$O(N \cdot C \cdot k^2)$	$O(N^2 \cdot d_k)$	图像尺寸平方增长 vs 线性增长
归纳偏置	强（平移等变性）	弱（需从头学习位置关系）	数据效率 vs 灵活性
并行性	高	极高（矩阵运算）	硬件友好度

关键洞察：对于 $H \times W$ 的特征图，Self-Attention 的 $N^2 = (H \times W)^2$ 复杂度意味着：

- 224×224 输入： $N^2 \approx 2.5 \times 10^9$ —— **不可行**
- 14×14 特征图： $N^2 = 38,416$ —— **可行**

因此 Self-Attention 在 CNN 中通常只用于**深层低分辨率特征**。

设计心法：何时使用 Self-Attention

推荐使用场景：

- 需要捕捉长距离依赖（如视频中的跨帧关联）
- 空间分辨率已经较低（如 14×14 的特征图）
- 计算资源充足，追求最佳性能

不推荐场景：

- 高分辨率输入（如 224×224 原始图像）
- 实时应用（如移动端实时检测）

- 简单任务（如 MNIST 分类）

缓解高复杂度的策略：

1. **限制感受野**：局部窗口注意力（如 7×7 邻域）
2. **下采样后再计算**：只在低分辨率特征图上使用
3. **稀疏注意力**：只计算特定模式（如轴向注意力）

什么时候需要长程依赖

场景	是否需要长程依赖	原因
分类猫 vs 狗	不太需要	局部特征（耳朵、鼻子）已足够
场景理解	中等需要	需要关联天空（上）和草地（下）
目标检测	需要	关联物体的不同部分
语义分割	非常需要	像素级一致性需要全局上下文
视频理解	必须	跨帧关联

设计心法

判断是否需要长程依赖的方法：问自己 —— “这张图里，一个像素的类别是否依赖于远处的像素？”

- 如果每一小块都能独立判断 → 不需要
- 如果需要全局结构推理 → 需要

注意力的代价

注意力不是免费午餐：

注意力类型	计算复杂度	显存	何时用
通道注意力	$O(C^2)$ —— 很小	几乎不增加	默认推荐，几乎所有模型
空间注意力	$O(k^2 \cdot H \cdot W)$ —— 中等	略增加	目标检测、分割
自注意力/Non-local	$O(C \cdot H^2 \cdot W^2)$ —— 极大	显著增加	长程依赖必不可少时
全局池化 attention	$O(C)$ —— 极小	忽略不计	轻量级通道注意力

失败案例：注意力的误用

案例一：MNIST 上加 SE 模块

场景：MNIST 分类网络（3 层 CNN），你在每层后加了 SE 通道注意力模块。

结果：

- 参数量从 50K 增加到 65K (+30%)
- 准确率从 99.1% 提升到 99.15% (几乎无提升)
- 训练时间增加 20%

为什么失败:

- MNIST 是灰度图, 只有 1 个输入通道, 特征通道本身也很少 (8-16 通道)
- SE 模块学习"哪个通道重要", 但通道数太少, 选择空间有限
- 任务太简单, 没有复杂特征需要筛选

教训: 注意力解决的是"选择困难", 不是"能力不够"。如果 baseline 本身就能很好地完成任务, 注意力只是徒增计算。

案例二: 高分辨率特征图用 Self-Attention

场景: 目标检测网络中, 你在 56×56 分辨率的特征图上用了标准的 Non-local Self-Attention。

结果:

- 训练 batch size 从 8 降到 2 (显存不足)
- 推理一张图从 50ms 变成 500ms
- mAP 提升 0.3% (几乎不可感知)

为什么失败:

- Self-Attention 复杂度是 $O(H^2 \times W^2 \times C)$, 对于 56×56 就是 $56^4 \approx 1000$ 万的计算量
- 高分辨率时空间像素太多, 全局关联的计算成本爆炸
- 大多数关联是局部的, 全局计算是浪费

教训: Self-Attention 只适用于低分辨率特征图 ($\leq 14 \times 14$)。高分辨率请用局部窗口注意力或直接不用。

案例三: 所有层都加注意力

场景: 你在 ResNet-50 的每一层 (包括浅层的 56×56 和 28×28) 都加了 CBAM (通道+空间注意力)。

结果:

- 训练不稳定, loss 震荡
- 最终准确率比不加注意力还低 1%

为什么失败:

- 浅层特征 (边缘、纹理) 本身就很基础, 不需要复杂的注意力筛选
- 过早的注意力会抑制低级特征的提取

- 深层才需要注意力来筛选高级语义

教训：注意力应该加在深层，不是每一层。通常只在最后 1-2 个 stage（分辨率最低的部分）添加。

案例四：注意力权重全部趋于相同

场景：你训练了一个带 SE 模块的网络，但发现所有样本的 attention weight 都差不多（接近均匀分布）。

结果：

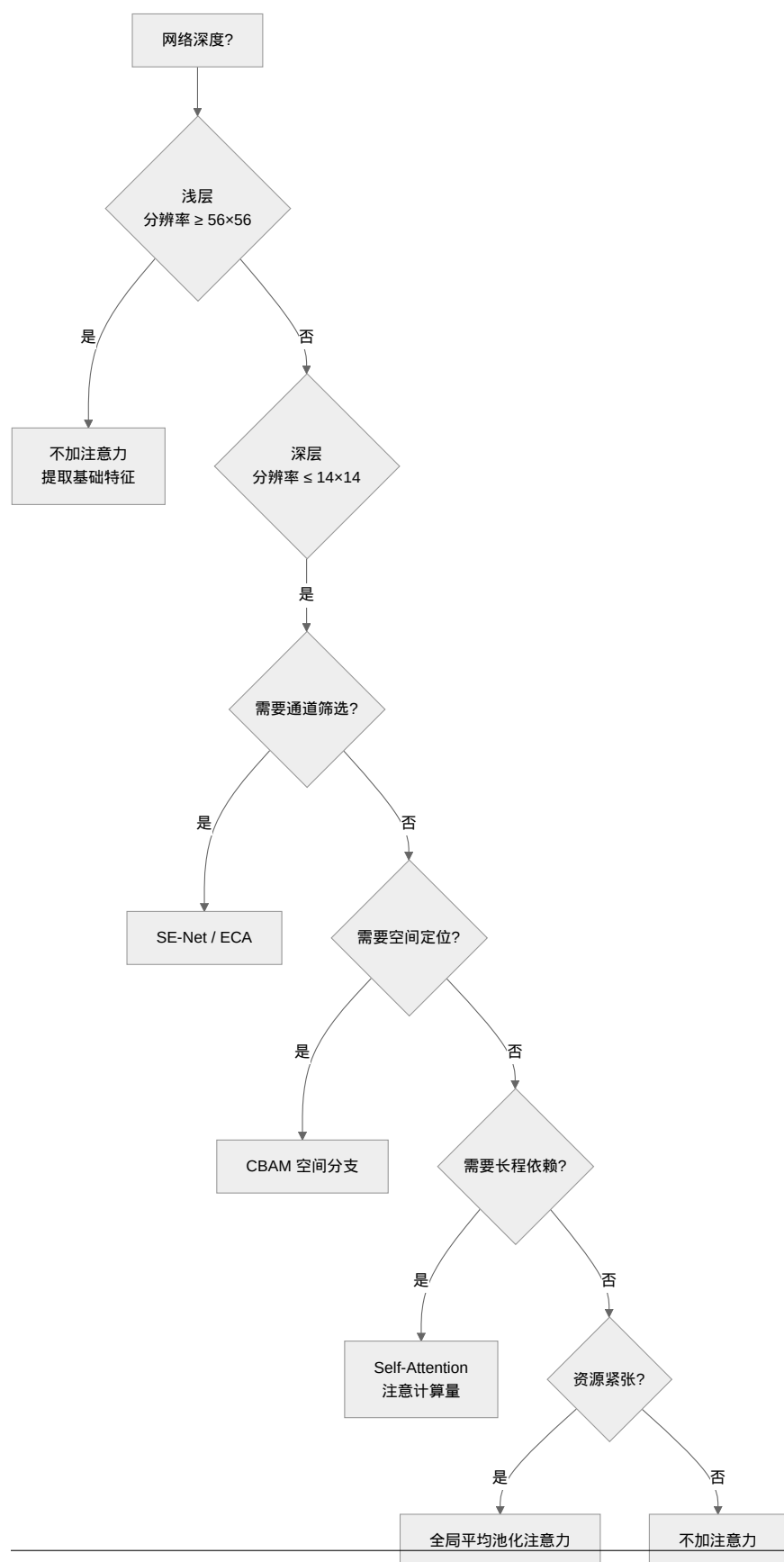
- SE 模块没有起到筛选作用
- 相当于加了一层无用的全连接

为什么失败：

- 可能是学习率太小，注意力没有被充分训练
- 可能是任务本身不需要通道筛选（如单通道输入的任务）
- 可能是 squeeze 操作的池化方式不对（全局平均池化丢失了重要信息）

教训：注意力需要被"训练出来"。如果权重趋于均匀，说明要么超参不对，要么任务本身不适合。

决策树：注意力使用指南



核心原则：

- 注意力是高级认知功能，浅层不需要
- 通道注意力几乎是"免费"的，可以默认加上
- 空间注意力和自注意力有显著代价，只在必要时使用

信息论视角：注意力 = 最大化任务相关互信息

从信息论来看，注意力解决的是 **诊断入门：信息论听诊器** 中讨论的**容量 vs 信息流**之外的第三个维度：**相关性**。

网络既要有足够容量（感受野）、信息要能流动（连接），还要懂得**区分有用信息和噪声**。

通道注意力在通道维度做信息选择，空间注意力在空间维度做信息选择，而长程依赖确保**信息不因距离而丢失**。

三种注意力协同工作，最大化的是**任务相关的互信息** $I(X; Y_{\text{task}})$ —— 不是保留所有信息，而是保留任务需要的信息。

知道看哪了。但背包太重，爬不动。第四件装备 —— **操控效率：效果、速度与硬件的三角权衡**。

3.5.5 操控效率：效果、速度与硬件的三角权衡

第四件装备：背包。哪些可以不带？DW 卷积、Bottleneck、复合缩放。

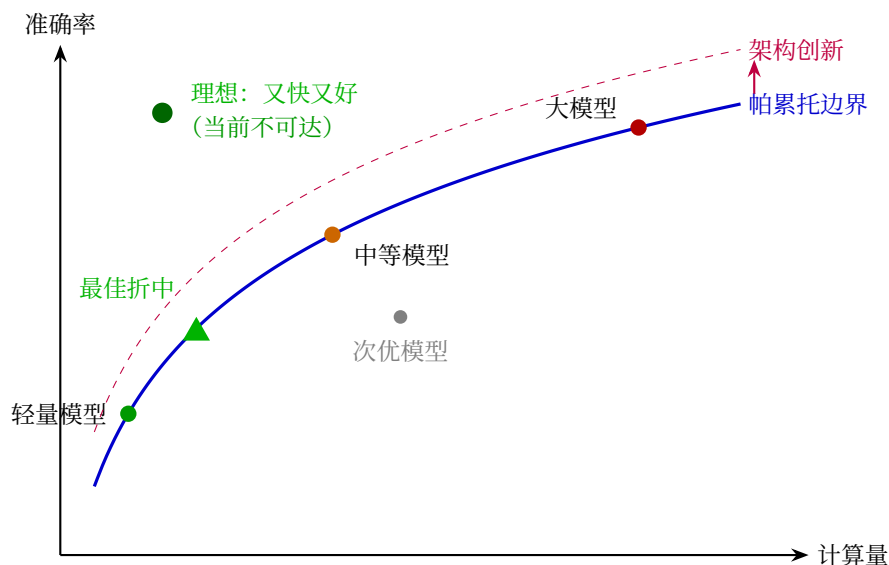
操控注意力与长程依赖：信息该走哪条路 教会了我们信息该往哪走。但信息走哪条路只是问题的一半 —— **每条路能承载多少车流**同样重要。本节我们从四个维度拆解效率瓶颈：**参数量、计算量、控制流、硬件利用**。

关键问题：同样的准确率，为什么一个模型比另一个慢 10 倍？参数量小就一定快吗？

帕累托边界：效果的物理上限

在讨论效率优化之前，需要先理解一个基本概念：**没有免费的午餐**。

对于给定的计算预算，存在一个**帕累托边界（Pareto Frontier）** —— 在这个边界上，任何效果的提升都必须以效率为代价，任何效率的提升都必须以效果为代价。



帕累托最优的意思是：如果一个模型在帕累托边界上，你没法让它在牺牲效果的前提下变得更快，也没法在不牺牲效率的前提下让它更好。

而**架构改造**的意义就是**推动帕累托边界向外移动**——不是在同一曲线上往返，而是在相同计算量下达到更好效果，或在相同效果下减少更多计算量。

这解释了为什么 Inception、ResNet、MobileNet 被认为是里程碑级别的贡献：它们不只是在已有曲线上滑动，而是**重新定义了什么算“最优”**。

但帕累托边界的横轴“计算量”其实掩盖了很多细节。两个模型 FLOPs 相同，一个可能比另一个慢好几倍——因为效率不仅仅是“算多少”的问题，更是“怎么算”的问题。

效率问题的四个源头

如果把神经网络比作一条高速公路，效率问题可以归结为四类：

表 5: 效率问题的四个维度

维度	问题表现	根本原因	就像...
维度一：参数量 —— 让模型“瘦身”	模型太大，存不下、传不动	权重有冗余，很多连接不重要	车上装了一堆空箱子
维度二：计算量 —— 让每一次乘法都有价值	训练/推理太慢，FLOPs 太高	乘法太多，很多可以合并或省略	每个路口都停下来称重
维度三：控制流 —— 让计算不再“排队等待”	GPU 利用率低，时快时慢	Python 循环和分支阻止了并行和编译	收费站一次只放一辆车
维度四：硬件利用 —— 让芯片“吃饱”	算力浪费，功耗高，发热大	数据搬运比计算还慢，芯片“吃不饱”	高速公路限速 120，但每辆车只能开 30

这四个维度相互独立又相互交织。接下来我们逐一拆解每个维度的直觉、问题和策略。

维度一：参数量——让模型"瘦身"

直觉：一个 3×3 卷积的权重数量是 $K^2 \cdot C_{in} \cdot C_{out}$ 。当 $C_{in} = C_{out} = 256$ 时，仅一层就有 $9 \times 256 \times 256 = 589,824$ 个参数。但 256 个通道之间真的需要全连接吗？**通道之间可能存在大量冗余**——很多通道捕捉的是几乎相同的信息。

Bottleneck：先压缩，再膨胀

Bottleneck 的核心思想来自 Inception：多尺度工具的 1×1 降维——先压缩再膨胀：



这种"沙漏结构"将最昂贵的 3×3 卷积放在低维空间执行：

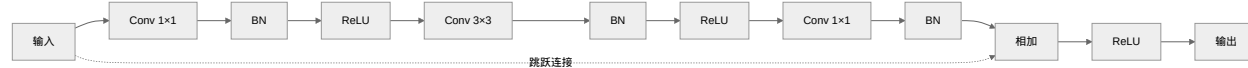
方式	参数分布	主要计算在哪
直接 3×3	$256 \times 3 \times 3 \times 256 = 589,824$	高维空间计算
Bottleneck	$256 \times 1 \times 1 \times 64 + 64 \times 3 \times 3 \times 64 + 64 \times 1 \times 1 \times 256 = 69,632$	3×3 在低维（64）执行

减少 88.2% 参数，而 3×3 卷积的执行维度从 256 降到 64。

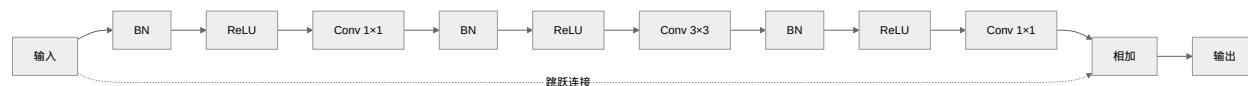
为什么 Bottleneck 有效： 1×1 卷积发现，256 个通道中的有效信息可以用更少的维度（64 维）表示——信息的**低维结构**保证了压缩不会丢失太多信息。用互信息的语言说： $I(X; Z) \approx I(X; Y)$ ，压缩后的特征 Z 保留了原始特征 Y 的几乎所有信息。

Pre-activation vs Post-activation

Bottleneck 的激活函数位置影响信息传递质量。原始的 ResNet Bottleneck 使用 **Post-activation**（卷积后激活）：



改进的 Pre-activation（激活在卷积前）[HZRS16]：



Pre-activation 的优势：

1. **梯度更畅通：**跳跃连接直接传递的是归一化后的特征，没有激活函数的"阻断"
2. **正则化效果更好：**每个卷积层前面都有 BN，训练更稳定
3. **理论上更优：**恒等映射 $y = x$ 不需要经过非线性变换

实际效果：在极深网络（1000+层）上，Pre-activation 优势明显；普通深度（50-200 层）两者差距不大。

参数量优化小结

策略	原理	减少幅度	适用场景
Bottleneck	通道降维 (256→64→256)	~3-4×	深层, $C \geq 128$
Pre-activation	让跳跃连接更"纯净"	间接 (提升收敛, 非直接减参)	极深网络 (1000+层)

(除了 Bottleneck, 还有其他减少参数量的方法如**分组卷积** (Group Convolution) —— 将通道分成若干组, 每组独立卷积。它是 DW 卷积的"中间态", 本节将在维度二中一并讨论。)

维度二：计算量——让每一次乘法都有价值

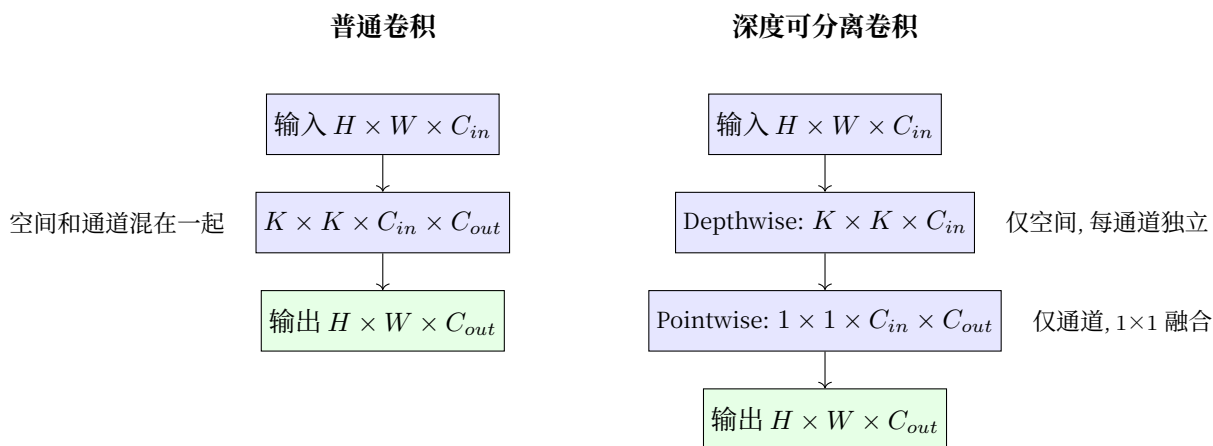
直觉：参数量小不等于计算量小。一个 1×1 卷积参数很少, 但如果特征图很大 (1024×1024), FLOPs 依然很高。计算量的公式是：

$$\text{FLOPs} = K_h \cdot K_w \cdot C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out}$$

关键观察：空间和通道混在一起计算。能不能分拆？

深度可分离卷积：空间与通道分离

深度可分离卷积 (Depthwise Separable Convolution, MobileNet 核心 [HZC+17]) 的洞察：**把空间和通道分开处理。**



两步拆分：

1. **Depthwise:** $K \times K$ 卷积, 但每个通道独立处理 (不跨通道) —— 计算量 $K^2 \cdot C_{in} \cdot H \cdot W$
2. **Pointwise:** 1×1 卷积, 只在通道间融合 —— 计算量 $C_{in} \cdot C_{out} \cdot H \cdot W$

普通卷积计算量: $K^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$ 分离卷积计算量: $(K^2 \cdot C_{in} + C_{in} \cdot C_{out}) \cdot H \cdot W$

$$\text{计算量减少倍数} = \frac{K^2 \cdot C_{out}}{K^2 + C_{out}}$$

当 $C_{out} = 256, K = 3$ 时, $\frac{9 \times 256}{9 + 256} \approx 8.7$, 约减少 **8.7 倍** 计算量 (参数量也减少相同倍数)。

分组卷积：分离度的调节旋钮

DW 卷积实质上是**分组卷积**的极端情况 —— 当 $\text{groups} = C_{\text{in}} = C_{\text{out}}$ 时，分组卷积退化为 DW 卷积。分组卷积可以看作一个“分离度旋钮”：

分组数	行为	相当于
$\text{groups}=1$	标准卷积	空间和通道完全耦合
$\text{groups}=g \ (1 < g < C)$	部分分离	空间和通道部分解耦
$\text{groups}=C$	DW 卷积	空间和通道完全分离

```
# 分组卷积：调节 groups 参数
# groups=1：标准卷积
nn.Conv2d(256, 256, 3, groups=1) # 空间和通道完全耦合

# groups=4：分成 4 组，每组 64 通道
nn.Conv2d(256, 256, 3, groups=4) # 部分解耦

# groups=256：DW 卷积（极端情况）
nn.Conv2d(256, 256, 3, groups=256) # 空间和通道完全分离
```

分组卷积是“标准卷积”与“DW 卷积”之间的连续谱 —— group 越大，分离度越高，计算量越低，但信息融合越弱。

MobileNetV2 的倒残差结构

MobileNetV2 结合了 DW 卷积和 Bottleneck，创造了**倒残差结构**（Inverted Residual）：**关键区别**：

特性	标准残差块 (ResNet)	倒残差块 (MobileNetV2)
形状	高 → 低 → 高（沙漏）	低 → 高 → 低（漏斗）
核心卷积	普通 3×3	DW 3×3
升维/降维	先降后升	先升后降
跳跃连接	高维连接	低维连接 （关键！）
激活位置	升维后激活	只在中间高维处激活

为什么倒残差更高效？

- DW 卷积在高维执行更高效**：DW 卷积计算量与通道数成正比，而标准卷积与通道数的平方成正比。在高维空间（192 通道）执行 DW 卷积，比在低维空间（32 通道）执行标准卷积更省 —— DW 节省 $32 \times$ ！
- 低维跳跃连接节省内存**：跳跃连接不需要存储高维特征图，减少显存占用。
- Linear Bottleneck（线性瓶颈）**：最后的 1×1 降维后**不接 ReLU** —— ReLU 在低维空间会破坏信息（一旦变负就永远为 0）。

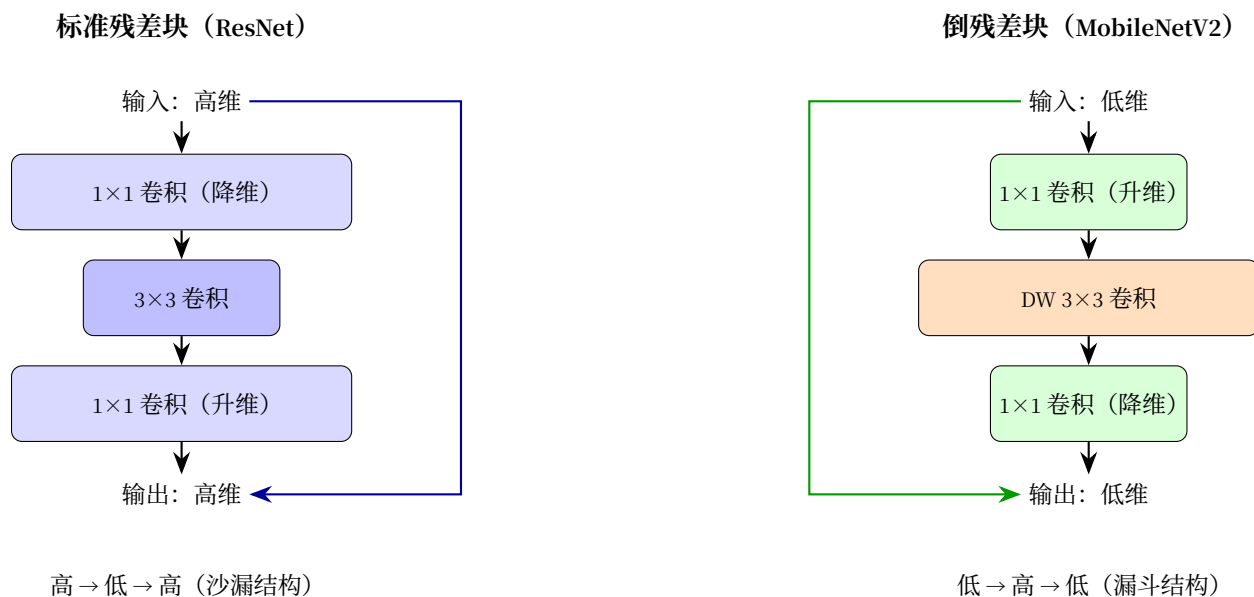


图 34: MobileNetV2 倒残差结构对比

设计心法：倒残差是"DW 卷积的 Bottleneck 版本"——先升维创造"计算空间"，用 DW 卷积高效处理，再降维回到紧凑表示。

维度一+二的统一心法

Bottleneck、DW 卷积、倒残差本质上是一回事——**利用信息冗余来降本**，只是切入角度不同：

- **Bottleneck**：利用通道间的冗余 (256→64→256)，主要降低**参数量**
- **DW 卷积**：利用空间和通道的可分离性 (先空间再通道)，主要降低**计算量**
- **倒残差**：结合两者 (低维 → 高维 → 低维 + DW)，同时降低参数和计算

理解了本质，你就能灵活组合这些策略，设计出适合你自己任务的模块。

计算量优化小结

策略	原理	减少幅度	适用场景
DW 卷积	空间通道分离计算	~8-10×	通道数 ≥ 32 ，移动端
分组卷积	部分分离 (调节 group 数)	~2-4×	需要平衡效果和效率
倒残差 (DW+Bottleneck)	低维跳连 + DW 在高维执行	~8-15×	移动端，极致效率

维度三：控制流——让计算不再“排队等待”

直觉：前两个维度关心“算多少”。但还有一个隐藏的杀手：**计算能不能并行**？如果 forward 函数里有 Python 的 for 循环和 if/else 分支，那么即使每个操作很快，整体速度也会被串行依赖拖垮。

关键问题：两个模型 FLOPs 完全相同，一个用 Python for 循环逐时间步计算，一个用全张量运算——前者可能慢 100 倍。为什么？

问题：Python 控制流的重重代价

在 forward 里写 Python for 循环或 if/else 分支，看似方便，实则付出三重代价：

```
# ❌ 看似正常，实则低效
class SlowRNN(nn.Module):
    def forward(self, x):
        # x: (batch, seq_len, hidden)
        outputs = []
        h = torch.zeros(x.size(0), self.hidden_size)
        for t in range(x.size(1)):          # ① 串行依赖：第 t 步必须等 t-1 步
            xt = x[:, t, :]                  # ② 每次取一个切片，无法融合
            if h.sum() > 0:                  # ③ 动态分支，阻止图优化
                h = torch.tanh(self.w_ih(xt) + self.w_hh(h))
            else:
                h = torch.tanh(self.w_ih(xt))
            outputs.append(h)
        return torch.stack(outputs, dim=1)  # stack 操作又引入一次内存搬运
```

三重代价分别是：

代价一：串行依赖 (Sequential Dependency)。循环体内的每一轮必须等上一轮结束。GPU 有上千个核心，但 `for t in range(seq_len)` 强行让这些核心排队——`t=0` 算完才能算 `t=1`。当 `seq_len = 1000` 时，意味着 999 个时间步在空转等待。

代价二：阻止计算图编译 (No Graph Optimization)。PyTorch 是动态图框架，每一步的 Python 控制流都会创建一个新的计算图节点。`torch.compile` 和 `torch.jit.script` 尝试将 Python 代码编译为优化的静态图，但 if 和 for 让编译器“看不懂”完整的计算模式，无法做算子融合和内存规划。

代价三：内核启动开销 (Kernel Launch Overhead)。每个 PyTorch 操作（如 `self.w_ih(xt)`）都涉及一次 GPU 内核 (kernel) 启动。当循环体里有 3 个操作、循环 1000 次时，就是 3000 次内核启动。每次启动的延迟约为 5-10 μ s——仅仅启动开销就累计了 15-30ms，而实际计算可能只需 5ms。

策略一：向量化——用张量运算代替循环

最直接的解法：把"逐个处理"变成"整批处理"。

☒ 向量化：一次处理整个序列

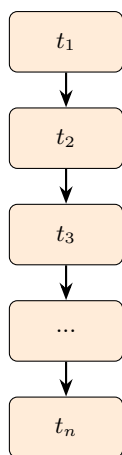
```
class FastRNN(nn.Module):
    def forward(self, x):
        # x: (batch, seq_len, hidden)
        batch, seq_len, hidden = x.shape
        h = torch.zeros(batch, hidden, device=x.device)
        outputs = []
        for t in range(seq_len):
            h = torch.tanh(x[:, t, :] @ self.w_ih.T + h @ self.w_hh.T)
            outputs.append(h)
        return torch.stack(outputs, dim=1)
```

但 RNN 的串行本质无法完全消除——只能缓解。真正从架构层面解决这个问题的是 Transformer。

策略二：从架构层面消除串行——Transformer 为何比 RNN 快

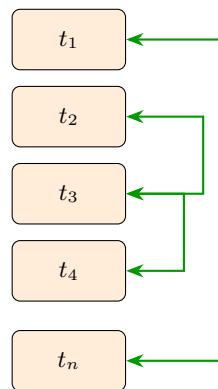
Transformer：注意力的极致 中介绍的 Transformer[VSP+17]，其核心突破不只是"注意力机制"——更本质的是将串行处理变为并行处理：

RNN（串行处理）



时间： $O(n)$ ，无法并行

Transformer（并行处理）



时间： $O(1)$ 信息路径，可完全并行

维度	RNN	Transformer
处理方式	逐个时间步串行	所有位置同时处理
信息路径	$O(n)$ (第 n 步才能看到第 1 步)	$O(1)$ (任意两位置直接相连)
GPU 利用率	低 (串行等待+内核启动开销)	高 (大规模矩阵乘法, GPU 最擅长)
训练速度	序列越长越慢	序列长度影响较小 ($O(n^2)$ 注意力量化)
长程依赖	梯度消失/爆炸, 难以学习	直接路径, 容易学习

Transformer 的效率秘诀：注意力矩阵乘法 QK^T 的维度是 $(n, d) \times (d, n) \rightarrow (n, n)$ 。这是 GPU 最喜欢的操作——**大矩阵乘法**，充分利用了 GPU 的并行计算能力。相比之下，RNN 的 step-by-step 计算用的是**小矩阵乘法** ($(1, d) \times (d, d)$ 重复 n 次)，GPU 每个核心都在“摸鱼”。

策略三：编译优化——torch.compile 与静态图

即使架构层面无法完全消除串行，编译技术也能大幅减少开销。PyTorch 2.0 引入的 `torch.compile` 通过以下优化提升效率：

- **算子融合 (Operator Fusion)：**将 `Conv` → `BN` → `ReLU` 融合为一个内核调用，消除中间结果的内存读写
- **内存规划 (Memory Planning)：**预分配计算所需内存，避免运行时频繁分配/释放
- **自动混合精度：**自动识别哪些操作可以用更低精度执行

PyTorch 2.0+：一行代码获得 10-50% 加速

```
model = torch.compile(model, mode="reduce-overhead")
```

对于包含 Python 控制流的模型，`mode="reduce-overhead"` 尤其有效——它会缓存编译后的图，避免每次 forward 都重新编译。

torch.compile 的本质

`torch.compile` 做的事情可以概括为：**把动态的 Python 逻辑“拍扁”成静态的计算图**。一旦控制流被“拍扁”，编译器就能看到全局计算模式，做全局优化。

但它不能完全“拍扁”动态分支——如果一个 `if` 的分支取决于输入数据（如 `if x.sum() > 0`），编译器无法预知走哪条路。这是静态图天生的局限，也是为什么 PyTorch 选择动态图作为默认：**灵活性有代价**。

控制流优化小结

策略	原理	加速幅度	适用场景
向量化	用张量操作代替 Python 循环	~10-100×	任何有显式循环的 forward
架构级并行	用 Transformer/CNN 代替 RNN	~10-50× (训练)	序列建模、长程依赖
torch.compile	动态图 → 静态图编译	~10-50%	PyTorch 2.0+, 任何模型

维度四：硬件利用 —— 让芯片"吃饱"

直觉：FLOPs 低 $\neq$ 速度快。有人把深度学习比作"搬砖" —— 计算（乘法）是你搬砖的速度，而数据搬运（内存读写）是你往返工地和仓库的时间。如果每次只搬一块砖，那 90% 的时间都花在路上。

这就是**内存带宽瓶颈**（Memory-Bandwidth Bound）：芯片的计算能力绰绰有余，但数据喂不进来，算力闲置。

问题：为什么"算得少"反而可能更"慢"

考虑两个极端操作：

操作	FLOPs	内存访问量 (Bytes)	算术强度 (FLOPs/Byte)
矩阵乘法 4096×4096	68.7 GFLOPs	0.10 GB	687
逐元素加法 4096×4096	0.016 GFLOPs	0.10 GB	0.16

矩阵乘法的算术强度是 687 —— 计算量远大于数据搬运量，受限于**计算能力**（Compute-Bound）。逐元素加法的算术强度只有 0.16 —— 数据搬运量远大于计算量，受限于**内存带宽**（Memory-Bound）。

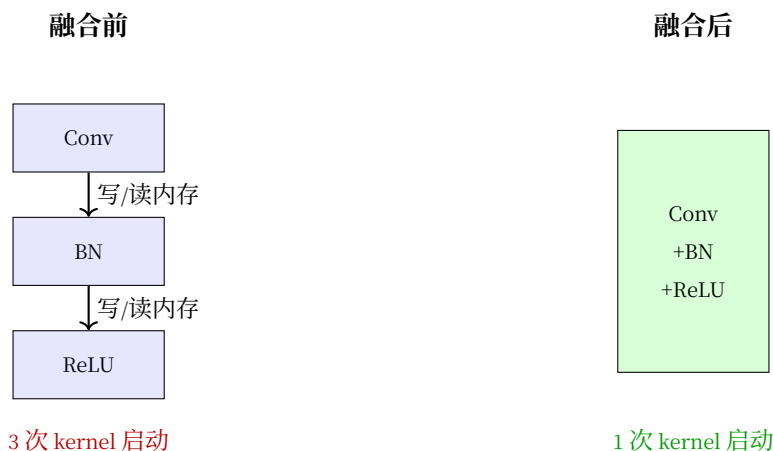
DW 卷积的尴尬：DW 卷积虽然 FLOPs 低，但算术强度也低 —— 3×3 的卷积核只做 9 次乘法，却要读取 9 个输入值和 1 个输出值。加上每个通道独立，无法组成大矩阵乘法。这就是为什么 DW 卷积"理论快 8 倍，实际可能只快 2-3 倍"。

策略一：算子融合 —— 减少"往返搬运"

深度学习中最常见的算子序列是 Conv $\rightarrow$ BN $\rightarrow$ ReLU。朴素实现需要三次内核启动和三次内存读写：

```
# ✗ 三次内核启动，两次中间结果的内存读写
x = self.conv(x)    # 写中间结果到内存
x = self.bn(x)      # 读中间结果，写中间结果
x = F.relu(x)       # 读中间结果，写最终结果
```

算子融合将这三个操作合并为一次内核调用：



融合后，中间结果不再写回内存——在 GPU 寄存器或共享内存中直接传递。这正是 `torch.compile` 和推理框架（TensorRT、ONNX Runtime）的核心优化之一。

策略二：内存布局——数据怎么"排列"影响访问效率

内存布局决定了数据在显存中的物理排列顺序。PyTorch 默认使用 **NCHW**（batch, channel, height, width），而某些 GPU 操作对 **NHWC**（batch, height, width, channel）更友好：

布局	含义	优势	劣势
NCHW	通道维度连续	逐通道操作（BN）更高效	逐像素操作效率低
NHWC	像素维度连续	逐像素操作 + Tensor Core 效率高	逐通道操作稍慢

```
# 转换为 channels_last (NHWC) 内存布局
x = x.to(memory_format=torch.channels_last)

# 后续卷积自动使用优化后的 NHWC 内核
x = self.conv(x) # cuDNN 自动选择最优实现
```

对于使用 `nn.Conv2d` 等标准算子的模型，切换为 `channels_last` 通常能带来 10-30% 的加速，且代码改动仅为一行。

策略三：混合精度——精度够用就好，不必 32 位

FP32（32 位浮点）提供了极高的数值精度，但多数深度学习操作并不需要——FP16（16 位）或 BF16（Brain Float 16）的精度已足够。

表 6: 精度方案对比

精度	位数	速度	显存占用	适用 GPU
FP32	32	1×（基准）	100%	所有
FP16 + FP32 混合	16/32	2-3×	~60%	V100+（Tensor Core）
BF16 + FP32 混合	16/32	2-3×	~60%	A100+（Ampere+）
INT8 量化	8	3-4×（推理）	~25%	T4+（TensorRT）

混合精度的核心做法：前向和反向传播用 FP16/BF16 计算（快），权重更新用 FP32 累积（稳）。

```
# PyTorch 混合精度训练的标准写法
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()
```

(续下页)

(接上页)

```
for data, target in dataloader:
    optimizer.zero_grad()

    with autocast():
        # 前向：自动用 FP16 计算
        output = model(data)
        loss = criterion(output, target)

    scaler.scale(loss).backward()
    # 反向：scale 防止梯度下溢
    scaler.step(optimizer)
    # 更新：FP32 精度累加
    scaler.update()
```

GradScaler 的作用：FP16 的数值范围远小于 FP32 ($6 \times 10^{-8} \sim 65,504$ vs $1.4 \times 10^{-45} \sim 3.4 \times 10^{38}$)。微小梯度在 FP16 下可能变成 0 (下溢)。scaler 将 loss 放大若干倍，让梯度也放大，反向传播后再缩小回来——避免下溢的同时保持精度。

工程最佳实践 中有更详细的混合精度实战指南。

硬件利用优化小结

策略	原理	加速幅度	代码改动
算子融合	Conv+BN+ReLU 合并为一次内核调用	~20-40%	torch.compile, 零改动
channels_last	NHWC 布局适配 Tensor Core	~10-30%	1 行: x.to(memory_format=...)
混合精度	FP16/BF16 计算 + FP32 更新	2-3× (训练)	~5 行包装 autocast+GradScaler

四个维度的策略全景

表 7: 效率优化策略全景

维度	最有效策略	减少幅度	主要代价	典型应用
维度一：参数量 —— 让模型"瘦身"	Bottleneck (先压缩再膨胀)	~3-4×	需要合适 Bottleneck 维度	ResNet-50/101/152
维度二：计算量 —— 让每一次乘法都有价值	DW 卷积 + 倒残差	~8-15×	可能损失少量精度 (<1%)	MobileNetV2/V3, EfficientNet
维度三：控制流 —— 让计算不再"排队等待"	向量化 + torch.compile	~10-100× (循环场景)	架构可能需重新设计 (RNN→Transformer)	序列建模, 长程依赖
维度四：硬件利用 —— 让芯片"吃饱"	混合精度 + 算子融合	2-3×	需要硬件支持 (V100+)	几乎所有模型

失败案例：效率优化的陷阱

案例一：Bottleneck 压缩率过大

场景：设计图像分类网络，你将 Bottleneck 的压缩比设为 $256 \rightarrow 8 \rightarrow 256$ （32 倍压缩）。

结果：

- 参数量确实大幅减少（约 15 $\times$ ）
- 但准确率从 75% 暴跌到 45%

为什么失败：

- 8 维的 Bottleneck 太小，无法承载 1000 类 ImageNet 的分类信息
- 信息瓶颈过于狭窄，有用的特征在压缩过程中被"挤掉"
- 从互信息角度： $I(X; Z)$ 太小， Z 无法保留足够区分 1000 类的信息

教训：Bottleneck 的维度有下限。对于 C 类分类任务，Bottleneck 维度至少应该是 C 的 2-4 倍。1000 类任务，Bottleneck 不应小于 256 维。

案例二：DW 卷积用于通道数少的层

场景：网络第一层（3 通道输入，16 通道输出），你用了 DW 卷积+ 1×1 pointwise，而不是标准 3×3 卷积。

结果：

- 计算量反而增加了！
- 准确率下降 2%

为什么失败：

- DW 卷积的优势在于"空间和通道分离"
- 但当输入通道只有 3 时，DW 卷积的"逐通道卷积"几乎等价于标准卷积（反正每个输入通道独立处理）
- 但 DW 卷积后还要加 1×1 pointwise 来融合通道，多了一步计算
- 标准卷积在 $3 \rightarrow 16$ 时本来就是 $3 \times 3 \times 3 \times 16$ ，没有冗余可省

教训：DW 卷积只适用于通道数 ≥ 32 的情况。浅层（通道数少）用标准卷积更高效。

案例三：盲目追求轻量导致无法收敛

场景：为了部署到嵌入式设备，你将 MobileNetV2 的宽度乘子（width multiplier）设为 0.1（所有通道数变为原来的 10%）。

结果：

- 模型大小只有 0.5MB，满足部署要求

- 但训练 loss 根本不下降，模型无法学习

为什么失败：

- 宽度乘子 0.1 意味着 Bottleneck 维度从 192 降到 19
- 19 维的特征空间无法表达 CIFAR-10 甚至 MNIST 的特征
- 模型容量被压缩到连简单任务都无法完成

教训：效率优化有极限。宽度乘子通常不应小于 0.25，否则模型会“饿死”。

案例四：倒残差用错激活函数位置

场景：实现倒残差块时，你在最后的 1×1 降维后加了 ReLU 激活。

结果：

- 准确率比标准 MobileNetV2 低 3-5%
- 特征可视化显示大量神经元“死亡”（恒为 0）

为什么失败：

- 倒残差的最后降维到低维空间（如 24 通道）
- ReLU 在低维空间是“信息杀手”——一旦输出为负，信息永久丢失
- MobileNetV2 论文明确建议在最后的 Bottleneck 用线性激活（Linear Bottleneck）

教训：倒残差的最后必须是线性激活。这是 MobileNetV2 的关键设计，不能省略。

案例五：Python 循环里的张量拼接

场景：为了处理变长序列，你用 Python 循环逐个样本做 padding 和拼接：

```
# ✗ Python 循环 + 逐个 cat
outputs = []
for x in batch:
    h = model(x)                # 每个样本单独 forward
    outputs.append(h)
result = torch.cat(outputs)     # 最后拼接
```

结果：

- 训练速度只有预期的 5-10%
- GPU 利用率徘徊在 20-30%

为什么失败：

- 每个样本单独 forward 意味着 *batch_size* 次内核启动（而不是 1 次）

- `torch.cat` 又触发一次内存重分配和拷贝
- GPU 被"碎片化"的小任务填满，无法发挥并行优势

教训：尽量在 **batch 维度上统一处理**。用 `pad_sequence + pack_padded_sequence`（或 `attention mask`）在单次 `forward` 中处理整个 `batch`。

案例六：DW 卷积搭配小特征图

场景：你在 MobileNetV2 的深层（特征图只有 7×7 ）大量使用 DW 卷积。

结果：

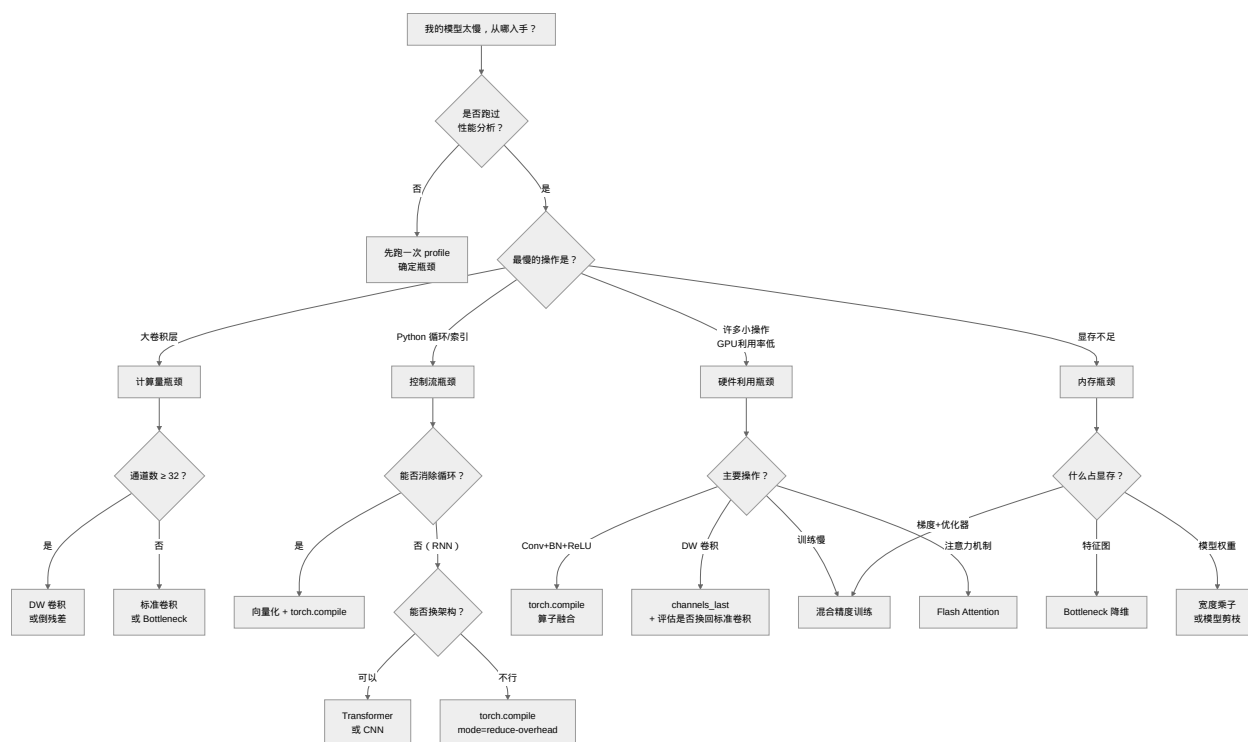
- 实际速度只有理论速度的 30%
- 换成标准卷积后反而更快

为什么失败：

- DW 卷积的算术强度极低（见[维度四：硬件利用 —— 让芯片"吃饱"](#)）
- 当特征图很小时，DW 卷积受限于内核启动开销和内存带宽，GPU 的并行能力完全浪费
- 标准卷积虽然 FLOPs 更高，但能组成大矩阵乘法，硬件利用率高

教训：深层小特征图考虑换回标准卷积。DW 卷积在特征图 $\geq 14 \times 14$ 时才最有效。

决策树：效率优化策略选择



核心原则：

- 效率优化是**权衡**，不是"免费午餐"——每个维度都有代价
- **先 profile，再优化**——不要猜瓶颈在哪，用数据说话
- 先确定 baseline 能正常工作，再逐步优化
- 每次优化后都要验证准确率，别只盯着模型大小和速度
- 四个维度可以叠加使用，但每次只加一个，测量效果后再加下一个

信息论视角：效率优化为什么可能

效率优化能成功的根本原因来自信息论：**自然数据（图像、文本、语音）的特征具有稀疏性和低维结构。**

通道之间不是独立的——256 个通道中，很多通道捕捉的是高度相关的信息。 1×1 卷积能发现这种相关性，将 256 维压缩到 64 维，保留的信息量基本相同。

同样，空间和通道的**可分离性**意味着：空间模式（边缘、纹理）和通道模式（哪种特征）的交互不是任意组合的——把它们分开处理，信息损失很小。

用**互信息**的语言说：

- Bottleneck 做了 $I(X; Z) \approx I(X; Y)$ 的信息压缩——压缩后信息几乎不变
- DW 卷积利用了 $I_{\text{space, channel}} \approx I_{\text{space}} + I_{\text{channel}}$ 的近似分离性
- 混合精度利用了"位数的冗余"——大多数梯度值和权重值不需要 32 位精度来表达
- 算子融合利用了"计算的局部性"——相邻操作之间的数据不需要写回全局内存

这就是为什么我们能"白嫖"效率——压缩掉的是冗余，保留的是信息。而控制流优化和硬件感知算法更进一步：它们压缩的是**时间维度的冗余**（消除不必要的等待）和**空间局部性的浪费**（数据尽量留在高速缓存中）。

四件装备齐了。但面对一座具体的山，先动哪件？**诊断与心法：当模型表现不佳时**——登山者的自救手册。

3.5.6 诊断与心法：当模型表现不佳时

登山者的自救手册——损失不降？梯度消失？逐项排查，对症下药。

前面四节我们分别学习了四个维度的改造策略。现在是最后一关：**面对一个实际表现不佳的模型，你怎么知道该用哪个策略？**

症状与对策

从训练和验证曲线的表现可以推断瓶颈:

症状	诊断	对应维度	改造策略
训练 loss 不降或降得很慢, 测试 loss 也不降	容量不够 (欠拟合)	感受野 / 深度与连接	加深度、加宽度、检查训练配置
深层网络训不动, 浅层梯度接近零	信息传不过去 (梯度消失)	深度与连接	加跳跃连接、密集连接
大物体差小物体好, 或相反	感受野策略不对	感受野	多尺度 / 空洞卷积 / FPN
训练/推理时间不可接受, 显存不足	效率瓶颈 (操控效率: 效果、速度与硬件的三角权衡)	计算效率	视瓶颈而定: DW 卷积、Bottle-neck、向量化、混合精度

如何量化判断: 诊断工具箱

1. 判断欠拟合 vs 过拟合

关键指标: 训练准确率与验证准确率的差距

```
# PyTorch 示例: 监控训练过程
def diagnose_model(model, train_loader, val_loader):
    train_acc = evaluate(model, train_loader)
    val_acc = evaluate(model, val_loader)
    gap = train_acc - val_acc

    if train_acc < 0.7 and gap < 0.05:
        return "欠拟合: 模型容量不够"
    elif train_acc > 0.95 and gap > 0.15:
        return "过拟合: 需要正则化"
    elif 0.05 < gap < 0.10:
        return "正常: 轻微过拟合, 可接受"
    else:
        return "需要进一步分析"
```

经验阈值 (分类任务):

训练准确率	验证准确率	差距	诊断	行动
< 70%	< 70%	< 5%	欠拟合	加容量、检查数据/代码
70-90%	65-85%	5-10%	轻微过拟合	可加正则化或更多数据
> 95%	< 80%	> 15%	严重过拟合	必须加强正则化
> 98%	> 95%	< 3%	可能收敛	尝试更大模型或更多数据

2. 可视化梯度分布

梯度消失/爆炸的定量判断：

```
def analyze_gradients(model):
    grad_stats = {}
    for name, param in model.named_parameters():
        if param.grad is not None:
            grad = param.grad.abs()
            grad_stats[name] = {
                'mean': grad.mean().item(),
                'max': grad.max().item(),
                'std': grad.std().item()
            }

    # 按层分组分析
    shallow_grads = [v['mean'] for k, v in grad_stats.items()
                     if 'layer1' in k or 'conv1' in k]
    deep_grads = [v['mean'] for k, v in grad_stats.items()
                  if 'layer4' in k or 'fc' in k]

    shallow_mean = sum(shallow_grads) / len(shallow_grads)
    deep_mean = sum(deep_grads) / len(deep_grads)

    if deep_mean / shallow_mean < 0.01:
        print("警告：深层梯度几乎为零，可能存在梯度消失")
    elif deep_mean / shallow_mean > 100:
        print("警告：深层梯度远大于浅层，可能存在梯度爆炸")
    else:
        print(f"梯度流动正常（比率：{deep_mean/shallow_mean:.3f}）")
```

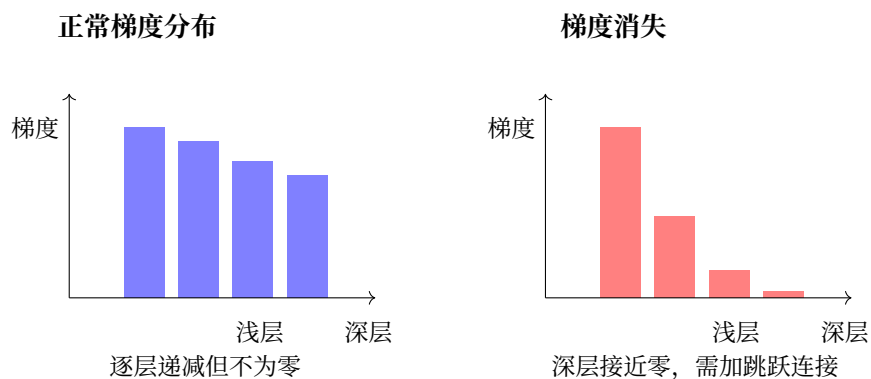


图 35: 梯度分布可视化示意

3. 分析各类别准确率

类别不平衡检测:

```
from sklearn.metrics import classification_report, confusion_matrix

def analyze_per_class(model, loader, class_names):
    y_true, y_pred = [], []
    for images, labels in loader:
        outputs = model(images)
        _, predicted = outputs.max(1)
        y_true.extend(labels.numpy())
        y_pred.extend(predicted.numpy())

    # 逐类别分析
    report = classification_report(y_true, y_pred,
                                   target_names=class_names,
                                   output_dict=True)

    for cls in class_names:
        precision = report[cls]['precision']
        recall = report[cls]['recall']
        f1 = report[cls]['f1-score']
        support = report[cls]['support']

        if f1 < 0.5 and support > 100: # 样本充足但表现差
            print(f"类别 {cls}: F1={f1:.3f} (差), 可能需要专门优化")
        elif f1 > 0.9:
            print(f"类别 {cls}: F1={f1:.3f} (优秀)")
```

诊断意义：

模式	可能原因	解决策略
某几类持续较差	特征不明显/类别相似	加注意力、数据增强特定类
大类过好，小类过差	类别不平衡	加权损失、过采样小类
随机几类差	可能是标注错误	检查数据标注
所有类都差但均匀	模型容量不够	整体加容量

4. 学习曲线分析

```
import matplotlib.pyplot as plt

def plot_learning_curves(history):
    """
    history: {'train_loss': [...], 'val_loss': [...],
             'train_acc': [...], 'val_acc': [...]}
    """
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))

    # Loss 曲线
    axes[0,0].plot(history['train_loss'], label='Train')
    axes[0,0].plot(history['val_loss'], label='Val')
    axes[0,0].set_title('Loss Curves')
    axes[0,0].legend()

    # 准确率曲线
    axes[0,1].plot(history['train_acc'], label='Train')
    axes[0,1].plot(history['val_acc'], label='Val')
    axes[0,1].set_title('Accuracy Curves')
    axes[0,1].legend()

    # 学习率变化（如果使用学习率调度）
    if 'lr' in history:
        axes[1,0].plot(history['lr'])
        axes[1,0].set_title('Learning Rate')

    # Gap 分析
    gap = [t - v for t, v in zip(history['train_acc'], history['val_acc'])]
    axes[1,1].plot(gap)
    axes[1,1].axhline(y=0.1, color='r', linestyle='--', label='过拟合阈值')
```

(续下页)

(接上页)

```

axes[1,1].set_title('Train-Val Gap')
axes[1,1].legend()

plt.tight_layout()
return fig

```

学习曲线诊断: 诊断决策树:

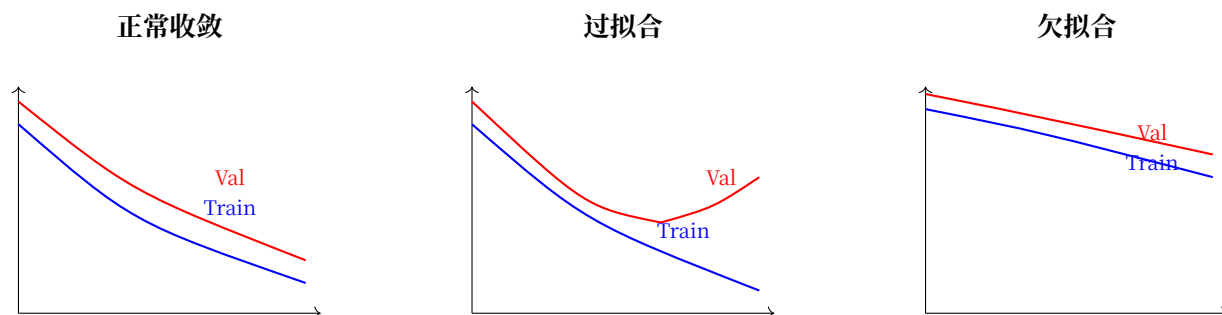
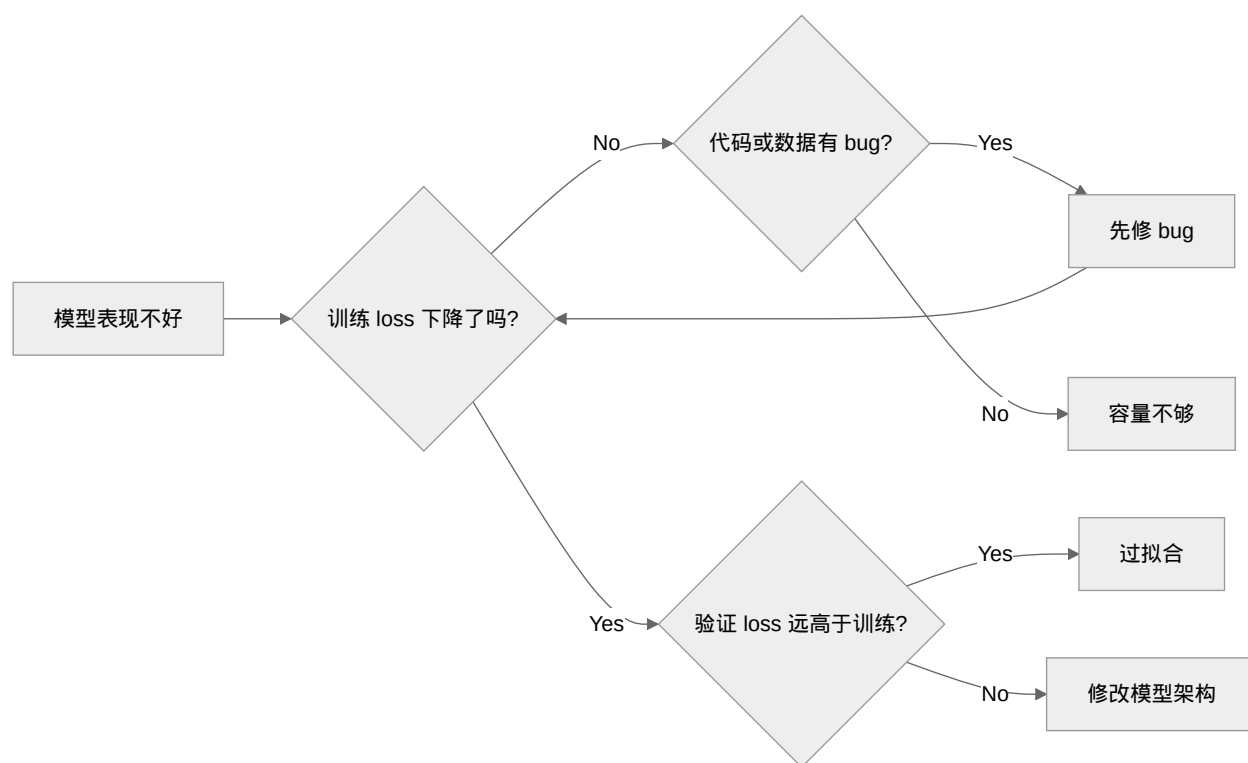


图 36: 典型学习曲线模式

曲线模式	诊断	优先级行动
验证集 loss 持续下降	还没收敛	继续训练
验证集 loss 平台期	可能收敛	降低学习率微调
验证集 loss 先降后升	过拟合	早停 + 加强正则化
训练集 loss 下降慢	学习率小/梯度问题	增大学习率/检查梯度
两条曲线都很高	欠拟合	加模型容量
Gap 越来越大	严重过拟合	立即干预

完整的诊断流程图



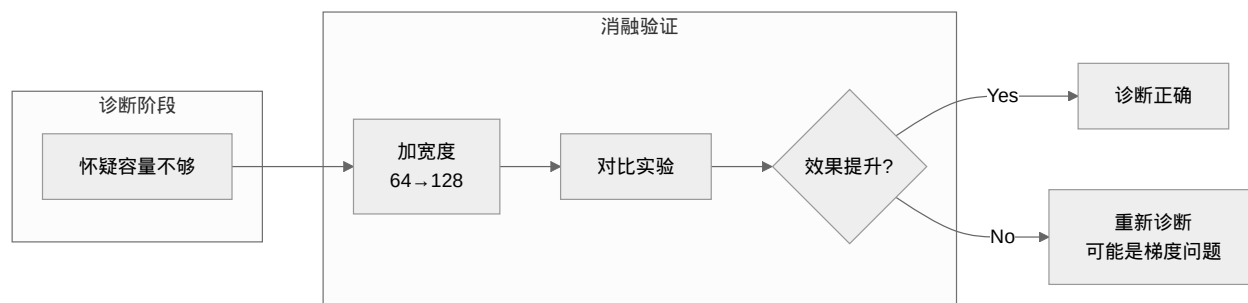
流程图卡在"修改模型架构"节点后，按下表判断具体改动：

症状	瓶颈在哪个维度	改造策略
容量不够（训练 loss 不降）	感受野 / 深度与连接	加宽度（通道数）、加深度（层数）、多尺度并行
过拟合（验证远差于训练）	正则化	数据增强、Dropout、权重衰减
深层训不动、浅层梯度接近零	深度与连接（操控深度与连接：信息如何在网络中流动）	跳跃连接、密集连接
大物体差小物体好，或相反	感受野（操控感受野：网络应该看多大）	多尺度并行、空洞卷积、FPN
背景干扰大	注意力（操控注意力与长程依赖：信息该走哪条路）	通道注意力（SE-Net）、空间注意力（CBAM）
推理速度太慢	计算效率（操控效率：效果、速度与硬件的三角权衡）	视瓶颈而定：DW 卷积、Bottleneck、向量化、混合精度
远距离关联失效	注意力（操控注意力与长程依赖：信息该走哪条路）	Self-Attention、Non-local
所有指标都还行但不理想	可能已达到瓶颈	更多数据、迁移学习与微调：站在巨人的肩膀上

验证诊断：与消融研究的结合

上述诊断流程给出了**假设**（“可能是容量不够”），但如何**验证**这个假设？这就是**拆开看看**：CNN 消融研究的价值所在。

诊断 → 消融的闭环



实用的消融策略

你的假设	消融实验设计	观察指标
感受野不够	在 baseline 上加一个 5×5 卷积分支	大物体 mAP 是否提升?
跳跃连接有帮助	移除 ResNet 的跳跃连接，训练对比	深层梯度是否消失? 收敛速度?
SE 模块有用	在相同位置分别加 SE / 不加 SE	准确率提升 vs 参数量增加
DW 卷积可接受	将某层换成 DW+Pointwise，对比效果	计算量下降 vs 准确率下降

关键原则（来自 [实验设计](#)）：

1. **一次只改一个变量** —— 如果你同时加宽度和加 SE 模块，就不知道是哪个起作用
2. **控制计算量** —— 消融不是为了刷分，是为了理解组件价值
3. **记录所有指标** —— 不只是最终准确率，还包括训练速度、收敛稳定性、梯度分布

诊断与消融的迭代

第一次迭代

- **诊断**：欠拟合，需要加容量
- **消融**：加宽度 $64 \rightarrow 128$
- **结果**：训练 loss 下降更快，但验证 loss 也上升 $\rightarrow$ 过拟合！

第二次迭代

- **诊断**：容量够了，但需要正则化
- **消融**：保持 128 宽度，加 Dropout 0.5

- **结果**: 验证 loss 下降, gap 缩小

第三次迭代

- **诊断**: 还有提升空间, 试试多尺度
- **消融**: 在第二、三 stage 加 FPN 融合
- **结果**: 小物体检测提升 3%, 但大物体下降 1%
- **结论**: 多尺度有帮助, 但融合策略需要调优

这个迭代过程就是架构设计的科学方法: **假设** → **实验** → **验证** → **修正**。没有消融研究的诊断只是猜测。

更多消融实验的方法论, 参见 [引言: 拆开看看](#) 和 [实验设计](#)。

反直觉案例

以下是一些容易让初学者走弯路的常见误区:

案例一: 为什么"加更多层"不一定好?

[ResNet](#): [高原反应补给站](#) 展示了退化问题: 56 层网络比 20 层更差。

直觉错误: 深度 = 表达力, 所以越深越好。

真相: 深度增加表达力, 但也增加优化难度。没有跳跃连接的 56 层网络信息早已丢失殆尽 —— 不是"没学到", 而是"根本谈不上学习"。

启示: 加深度之前, 先保证信息通路。

案例二: 为什么参数少反而更好?

[GoogLeNet](#) 6.8M 参数, 远少于 VGG 的 138M, 但效果更好。[Inception](#): [多尺度工具](#) 已经展示了这点。

直觉错误: 参数 = 能力, 所以参数多一定好。

真相: 参数多意味着搜索空间大, 更难优化。精巧的归纳偏置 (多尺度、跳跃连接) 比暴力堆参数更有效。

启示: 追求"优雅"而非"庞大"。

案例三: 注意力总是越多越好?

直觉错误: 注意力就是好, 越多越强。

真相: 注意力增加计算, 且不同类型的注意力适用场景不同。通道注意力 (SE-Net) 几乎通用 —— 因为"调整特征通道的权重"在分类、检测、分割中都有效, 代价极小。但空间注意力 (CBAM) 和自注意力 (Non-local) 就有取舍了: 对于 MNIST 这种信息密度高的简单任务, 几乎所有位置都很关键, 空间注意力几乎没有增益; 而在 COCO 等复杂场景中, 需要定位目标位置, 空间注意力的提升才显著。

启示: 通道注意力是"默认推荐" —— 代价极低、几乎稳定提升; 空间注意力和自注意力则是"按需使用" —— 只在信息过剩、需要选择性关注时才有意义。

改造十诫

架构设计心法十诫

1. **观察先行，再动刀。**看训练曲线、梯度分布、各类别准确率，不要凭感觉改。
2. **从简到繁，逐步验证。**每次只改一个变量，否则你永远不知道哪个改动真正有效。
3. **先修信息通路，再增信息容量。**先保证跳跃连接，再考虑加深度。
4. **感受野与目标大小匹配。**目标大 → 大感受野，目标小 → 小感受野，都有 → 多尺度。
5. **注意力解决“不知道看哪”，不是万能药。**先看信息有没有、通路通不通，再决定注意力。
6. **效率优先，延迟优化。**先跑出好结果，再考虑加速。
7. **多尺度是永恒主题。**无论感受野、FPN 还是注意力，核心都是“不要只用一个尺度”。
8. **不迷信新技术。**新技术 = 新归纳偏置，关键看它解决了什么信息论问题。
9. **外部变量先排除。**学习率、优化器、数据预处理出错的概率比架构问题大多了。
10. **保持怀疑。**论文里的 SOTA 架构可能不适用你的场景。没有“最好”的架构，只有“最适合”的架构。

心法实践：一个改造示例

下面是一个简短的示例 —— 在一个基础 CNN 上同时应用跳跃连接、SE 注意力和 DW 卷积：

```
import torch
import torch.nn as nn

class ModifiedBlock(nn.Module):
    """融合了三个维度改造策略的基础模块"""
    def __init__(self, in_ch, out_ch, stride=1, expansion=4):
        super().__init__()
        hidden_ch = out_ch // expansion

        # 维度四（效率）：Bottleneck 降维
        self.conv1 = nn.Conv2d(in_ch, hidden_ch, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(hidden_ch)

        # 维度四（效率）：DW 卷积 —— 空间和通道分离
        self.dwconv = nn.Conv2d(hidden_ch, hidden_ch, 3, stride,
                                padding=1, groups=hidden_ch, bias=False)
        self.bn2 = nn.BatchNorm2d(hidden_ch)
```

(续下页)

(接上页)

```

self.conv2 = nn.Conv2d(hidden_ch, out_ch, 1, bias=False)
self.bn3 = nn.BatchNorm2d(out_ch)

# 维度三（注意力）：SE 模块 —— 知道哪个通道重要
self.se = nn.Sequential(
    nn.AdaptiveAvgPool2d(1),
    nn.Conv2d(out_ch, out_ch // 4, 1), # r=4 降维
    nn.ReLU(inplace=True),
    nn.Conv2d(out_ch // 4, out_ch, 1), # 升维
    nn.Sigmoid()
)

# 维度二（深度与连接）：跳跃连接 —— 信息流动的退路
self.skip = nn.Identity() if stride == 1 and in_ch == out_ch else \
    nn.Sequential(nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
                  nn.BatchNorm2d(out_ch))

def forward(self, x):
    residual = self.skip(x)

    out = self.conv1(x)
    out = self.bn1(out)
    out = nn.ReLU(inplace=True)(out)

    out = self.dwconv(out)
    out = self.bn2(out)
    out = nn.ReLU(inplace=True)(out)

    out = self.conv2(out)
    out = self.bn3(out)

    out = out * self.se(out) # 通道注意力：每个通道乘一个 0~1 重要性系数

    out += residual          # 跳跃连接：从不衰减的梯度通道
    return nn.ReLU(inplace=True)(out)

```

心法与代码的对应：跳跃连接保证梯度通路 → `residual`；SE 注意力选择重要通道 → `se(out)`；DW 卷积降低计算量 → `groups=hidden_ch`；Bottleneck 降维执行 → `out_ch // 4`。

从"心法"到"实战"

掌握心法后，下一步就是实战。后面的 [迁移学习与微调：站在巨人的肩膀上](#) 将展示[预训练模型上的改造策略](#)——当你面对数据稀缺的场景时，如何站在巨人的肩膀上应用这些心法。

不需要从头训练的架构改造，才是工业界最常见的情况。

下一步

四件装备你已经会用，也知道怎么诊断了。

但现在——**坐下来，写登山者笔记**。回望这两座山、三十年路：感受野让你看对范围，深度连接让信息传得通，注意力让你学会聚焦，效率让你精打细算。每一条心法背后，都是一次"爬不上去"时的回应。

登山者笔记：四件装备，三十年——走。

3.5.7 登山者笔记：四件装备，三十年

两座山。四次冲顶。三十年的路。

从 [全连接：赤手空拳](#) 的赤手空拳，到 [EfficientNet](#)：每一分钱，花在刀刃上的复合缩放——你翻越了从 1989 到 2019 的整个卷积神经网络演化史。

但翻完了，然后呢？**坐下来，趁热写下笔记**。

四件装备

诊断入门：[信息论听诊器](#) 中我们问过：面对一座全新的山，你该怎么选装备？翻完两座山之后，答案不再是一堆零散的技术名称，而是**四个维度，四件装备**：

装备	核心问题	就像登山	关键策略
感受野	看多大范围？	望远镜——看裂缝还是看山脊？	多尺度、空洞卷积、FPN
深度连接	信息怎么传？	补给线——高海拔还能呼吸吗？	跳跃连接、密集连接、特征融合
注意力	重点看哪？	聚焦——盯裂缝，忽略光滑岩面	通道注意力、空间注意力、长程依赖
效率	怎么省力气？	背包——哪些装备可以不带？	DW 卷积、Bottleneck、复合缩放

[归纳偏置](#) 告诉我们：好的先验比暴力学习更高效。这四件装备教会我们另一件事：**好的设计，是把山的形状刻进每一件装备里**。

叙事弧线：一座山，三十年



每一个箭头背后，都是一次“爬不上去”时的回应——也是一件新装备的诞生。

登山装备演化史

把三十年摊开，每件装备解决了一个问题，也暴露了下一个。右边一列，标注了每件装备主攻的维度：

年份	装备	核心创新	解决了什么	暴露了什么	主攻维度
1989	LeNet	卷积+池化+全连接	证明端到端学习可行	只在 MNIST 上有效	感受野
2012	AlexNet	ReLU+Dropout+GPU	ImageNet 首登	大卷积核浪费参数	效率
2014	Inception	多分支并行	多尺度感受野	单分支信息瓶颈	感受野
2015	ResNet	跳跃连接	高原反应(梯度消失)	对所有特征一视同仁	深度连接
2018	SE-Net/CBAM	通道/空间注意力	学会聚焦	注意力 ≠ 效率	注意力
2017	MobileNet	深度可分离卷积	极致省钱	没有系统性缩放策略	效率
2019	EfficientNet	复合缩放	预算的最优分配	缩放需要搜索, 没有解析解	效率

每一条记录，都是一个维度的突破。伟大的架构（ResNet、EfficientNet）往往同时在多个维度上推动帕累托边界。

关键数字：策略性价比

四件装备不是免费午餐。每件都有自己的代价：

改造策略对比

策略	效果提升	效率影响	实施难度	对应维度
多尺度并行 (Inception)	+3-5%	中等增加	中等	感受野
空洞卷积	感受野扩大不增参	几乎零影响	低	感受野
FPN	+5-8% (检测)	中等增加	高	感受野
跳跃连接	使深层可能 (质变)	零影响	低	深度连接
SE 注意力	+1-2%	增加 <3% 参数量	低	注意力
DW 卷积	-1% 精度	减少 5-10× 计算量	低	效率

帕累托边界上的里程碑

模型	年份	核心贡献	推动边界的维度
Inception	2014	多尺度 + 1×1 降维	感受野 + 效率
ResNet	2015	跳跃连接	深度连接
MobileNet	2017	DW 卷积	效率
SE-Net	2018	通道注意力	注意力
EfficientNet	2019	复合缩放	效率（系统性）
ConvNeXt	2022	现代化 CNN	四维度精细优化

登山者关于规模的发现

翻越了足够多的山之后，一些规律浮现出来。

更大的团队，更细的分工，更深的补给线

观察	规律	山上的含义
更大的团队	参数量 ↑ → 成功率 ↑（但有边际递减）	人多覆盖面广，但协调成本也上升
更细的分工	通道数 ↑ → 每种模式有专人负责	有人专攻裂缝，有人专攻悬岩
更深的补给线	层数 ↑ → 但高原反应也 ↑	跳跃连接就是在关键海拔建补给站
更精细的地图	分辨率 ↑ → 能看到更小的冰裂缝	但也意味着每一步的计算量都更大

收益递减是山本身的形状

模型	年份	参数	ImageNet Top-1	边际收益
AlexNet	2012	60M	~63%	—
VGG-16	2014	138M	71.6%	+8.6%
Inception V1	2014	7M	69.8%	更小但差不多
ResNet-50	2015	25M	76.0%	+6.2%
ResNet-152	2015	60M	77.8%	+1.8%（更深但提升变小）
SE-ResNet-50	2018	28M	77.6%	+1.6%（几乎不增参数）
EfficientNet-B7	2019	66M	84.3%	复合缩放的效果

从 AlexNet 到 ResNet-152，参数量翻倍但准确率只从 63% 涨到 78%。**翻倍投入，收益却在递减。**这不是任何人的失败——**这是山本身的形状。**

在数学上，这种关系遵循幂律：

$$L(N) = L_{\infty} + A \cdot N^{-\alpha}$$

- L_∞ (不可约损失): 山的物理极限。即使无限大的模型、无限多的数据, 也无法突破
 - $N^{-\alpha}$: 增加投入带来的收益, 呈幂律递减
-

学会聚焦: 注意力的故事

ResNet 解决了高原反应。但还有一个更深的问题。

CNN 对所有特征一视同仁。无论这个通道编码的是"狗耳朵"还是"背景噪声", 卷积核给的权重一样。就像登山者盯着每一块岩石——包括那些根本不值得抓的碎石。

2018 年, SE-Net 装上了"可学习的放大镜": **通道注意力**——让网络自己判断"哪些特征重要"。只加了 8,000 个参数 (ResNet-50 的 0.03%), 准确率提升 1.5%。CBAM 再加一层**空间注意力**——让网络同时知道"哪里重要"。49 个参数, 又提升 1.1%。

这不是"更大更强"的故事。这是“**更聪明**”的故事。

归纳偏置 告诉我们好的先验比暴力学习更高效。注意力教会我们另一件事: **好的选择比好的先验更高效**。先验告诉你"该看哪里"——注意力让你根据实际情况**动态决定看哪里**。登山者不需要一直盯着每一块岩石。他们知道什么时候聚焦裂缝, 什么时候扫视全景。

从 ResNet 的"对所有特征一视同仁"到 SE-Net/CBAM 的"学会聚焦"——这不是技术改进, 是哲学转变。

四件装备在包里。三十年的路在脚下。笔记写完了。

推荐资源

必读论文

- [KSH12] — AlexNet: 深度学习革命的第一声炮响
- [SLJ+15] — Inception: 多尺度感受野
- [HZRS16] — ResNet: 跳跃连接与深层网络突破
- [HSS18] — SE-Net: 通道注意力
- [HZC+17] — MobileNet: 深度可分离卷积
- [TL19] — EfficientNet: 复合缩放

延伸阅读

- Tishby et al., “The Information Bottleneck Method”, 2000 — 信息瓶颈理论的数学基础
 - 迁移学习与微调: 站在巨人的肩膀上 — 站在预训练模型肩膀上改造
 - 序列建模: 从 RNN 到 Transformer 再到 Mamba — 序列建模的平行故事: 从 RNN 到 Transformer 到 Mamba
-

3.6 下一座山

四件装备在包里。三十年的路在脚下。

你翻越了练习峰，拆开了每一件装备，征服了真正的未登峰。MobileNet 把参数砍到 1/9，EfficientNet 按比例花钱——轻装攀登的答案，你已经有了。

但设计只是第一步。**设计好的模型，怎么让它真正跑起来？**

PyTorch 实践：把理论变成代码 是你面前的下一章——把架构设计变成可训练、可调试的代码。学完 CNN 架构再来写 PyTorch，你不再是“跟着教程敲”，而是“心里有图再敲”。

再往后，**模型部署与服务：从训练到生产** 会回答“怎么把训练好的模型部署到真实世界”——你的手机只有 2GB 内存，你的无人机需要在 10 毫秒内避障。这不是“怎么设计得更省”的问题，而是“已经设计好了，怎么让它跑在资源受限的设备上”的问题：

技术	做了什么	就像登山
剪枝	扔掉不重要的参数	登山归来，检查背包——没用上的装备留在营地
量化	用 8-bit 存储权重	钛合金替换钢铁——精度换空间

这和你刚学过的 MobileNet、EfficientNet 是互补的：架构设计从“怎么建得更省”的角度压缩模型，量化和剪枝从“建好后怎么再省一笔”的角度进一步瘦身。一个 MobileNet 经过 INT8 量化后，模型体积再缩小 4 倍、推理速度再提升 2-3 倍——架构省 + 部署省，效果相乘。

帕累托边界，永无止境

MobileNet 推动了边界。EfficientNet 推动了下一个边界。剪枝和量化推动的边界还在更前面。

设计让模型更好。PyTorch 让设计变成代码。部署让好模型能用。这座山，没有山顶。

PyTorch 实践：把理论变成代码——走。

3.6.1 关于马洛里

因为山在那里

1922 年，马洛里第二次冲击珠峰——失败了。一场雪崩卷走了七名搬运工的生命。他带着幸存者的内疚回到英国。

妻子露丝不想让他再去。父亲不想让他再去。他们有三个年幼的孩子。但他还是去了。

1924 年 6 月，他在珠峰北坡的暴风雪中消失。遗体在 1999 年被发现——75 年后，依然保持着向上攀爬的姿势。没人知道他是否登顶。

但所有人都知道：**他失败过，被至亲反对过，但他到死都在往上爬。**

因为山，它就在那里。

深度学习面前的这座山，也没有山顶。完美的泛化，物理上就不存在。

但马洛里不会因为山顶不可到达就放下冰镐。你也不会。

攀登本身，就是理解的过程。

Happy Climbing. 🧗

PyTorch 实践：把理论变成代码

4.1 引言：从理论到代码

还记得 [LeNet-5：练习峰首登](#) 中那 61,706 个参数吗？我们分析了每一层的维度、计算了参数量、讨论了为什么 CNN 比全连接更高效。

但有一个问题：这些数字是怎么存到电脑里的？那些矩阵乘法是怎么算的？反向传播到底怎么实现？

本章将回答这些问题。我们会用 PyTorch 把 [上篇：练习峰](#) 中的理论全部实现出来，让你真正理解从数学公式到可运行代码的转化过程。

学习目标

完成本章后，你将能够：

1. 用 PyTorch 实现 [深度学习核心数学基础](#) 中的计算图和反向传播
2. 搭建 [上篇：练习峰](#) 中的全连接网络和 CNN
3. 理解 PyTorch API 与理论概念的对应关系
4. 训练并调试一个完整的 MNIST 分类器
5. 掌握深度学习开发的工程实践技巧
6. 使用社团的训练框架进行高效的实验管理

4.1.1 为什么要用框架？

想象你要手写一个 LeNet

如果没有框架，你需要自己实现：

1. **张量存储**：如何存 32 个 5×5 卷积核？用 Python 列表？效率太低
2. **卷积运算**：嵌套循环实现滑动窗口？代码复杂还容易错
3. **反向传播**：手动推导每一层的梯度？LeNet 有 8 层，每层梯度公式都不一样
4. **GPU 加速**：想让计算快 10 倍？你得学 CUDA 编程

手动实现的痛苦

全连接：赤手空拳 中那个 3 层全连接网络，手写反向传播就需要：

- 推导每一层的梯度公式
- 小心矩阵维度匹配
- 调试数值稳定性问题
- 花了半天，最后发现是某个下标写错了

这还只是 3 层！想象一下 ResNet-152 的 152 层...

框架做了什么？

PyTorch 把上面这些痛苦都解决了：

问题	手写实现	PyTorch 方案
数据存储	Python 列表/NumPy 数组	<code>torch.Tensor</code> ：统一的数据结构
卷积运算	嵌套循环	<code>nn.Conv2d</code> ：一行代码
反向传播	手动推导梯度	<code>.backward()</code> ：自动计算
GPU 加速	写 CUDA 代码	<code>.to('cuda')</code> ：一键转移

核心洞察：PyTorch 不是新技术，而是 **深度学习核心数学基础** 中理论的**工程封装**——每个 API 都对应一个数学概念。

4.1.2 PyTorch 与前面章节的对应

让我们建立一个“理论 → 代码”的映射表：

深度学习核心数学基础 理论	PyTorch 实现	上篇：练习峰 应用
计算图	<code>torch.Tensor</code> + 运算	数据如何在网络中流动
反向传播算法	<code>.backward()</code>	梯度如何回传更新参数
梯度下降与优化算法	<code>optim.SGD/Adam</code>	参数如何一步步优化
激活函数	<code>nn.ReLU/Sigmoid</code>	引入非线性
损失函数	<code>nn.CrossEntropyLoss</code>	衡量预测好坏

学习策略：每学一个 PyTorch API，问自己“这对应哪个理论概念？”

4.1.3 PyTorch 的设计哲学

动态计算图：调试友好

PyTorch 采用**动态计算图** (define-by-run)：每次前向传播时实时构建计算图。

动态 vs 静态

静态计算图 (TensorFlow 1.x)：

- 先定义完整的计算图
- 然后在一个独立的 session 中运行
- **调试痛苦**：出错时不知道是哪一行

动态计算图 (PyTorch)：

- 代码按顺序执行
- 计算图在运行时构建
- **调试友好**：可以用 `print()` 随时查看中间结果

这对学习很重要 —— 你可以随时停下来检查张量的形状和内容，就像调试普通 Python 代码一样。

Pythonic：符合直觉

PyTorch 的 API 设计遵循 Python 的习惯：

```
# NumPy 风格的操作
import torch
x = torch.tensor([1.0, 2.0, 3.0])
y = x * 2 + 1 # 就像普通的 Python 运算
```

(续下页)

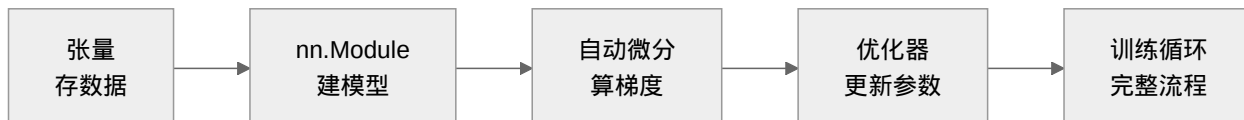
(接上页)

```
# 查看形状、设备、是否需要梯度
print(x.shape, x.device, x.requires_grad)
```

没有魔法：你看到的就是实际发生的。没有隐藏的图构建过程，没有复杂的 session 管理。

4.1.4 本章学习路线图

我们将按照"数据 → 模型 → 训练"的自然顺序学习：



与上篇：练习峰 的对应：

1. **从 NumPy 到 PyTorch：张量的本质**：理解 `torch.Tensor` 如何对应 计算图 中的节点
2. **张量操作：数据在网络中的流动**：数据如何在网络中流动（`reshape`、`transpose` 对应维度变换）
3. **神经网络模块：搭建计算图**：用 `nn.Module` 实现 全连接：赤手空拳 和 卷积：发现装备 中的架构
4. **自动微分：PyTorch 的核心魔法**：`.backward()` 就是 反向传播算法 的自动化
5. **优化器：用梯度更新参数**：`optimizer.step()` 实现 梯度下降与优化算法 的各种变体
6. **完整训练流程**：把 训练基础：登山纪律 中的流程代码化

4.1.5 核心认知：API 即理论

记住这个公式：

$$\text{PyTorch API} = \text{数学概念} + \text{工程优化}$$

例子：`nn.Conv2d` 的背后

- **数学**：卷积运算 $Y[i, j] = \sum_{u, v} X[i + u, j + v] \cdot K[u, v]$
- **工程**：CuDNN 优化的 GPU 实现，比手写快 100 倍

你的任务是理解**数学概念**，工程优化 PyTorch 已经帮你做了。

4.1.6 开始之前

本章的学习心法

1. **带着理论学代码**：每看到一个 API，问"这对应哪个数学概念？"
2. **动手实验**：修改参数看结果变化，比看书更有效

3. 善用帮助：help(torch.nn.Conv2d) 会告诉你数学公式和参数说明
4. 连接前后：随时回顾 [深度学习核心数学基础](#) 和 [上篇：练习峰](#) 的对应内容

一句话总结：本章不是学新东西，而是用 PyTorch **重新表达**你已经懂的理论。

准备好了吗？让我们从张量开始——这是 PyTorch 世界的“原子”。

4.2 从 NumPy 到 PyTorch：张量的本质

还记得 [计算图](#) 中那个简单的计算图吗？

$$z = (x + y) \times w$$

在数学上， x 、 y 、 w 、 z 只是符号。但在代码中，它们需要**存储在内存里**，需要**支持数学运算**，还需要**记录梯度信息**——这就是 **张量 (Tensor)** 的作用。

本章我们将看到：PyTorch 的 Tensor 不仅是 NumPy 数组的 GPU 版本，更是 [计算图](#) 中节点的**工程实现**。

4.2.1 回顾：NumPy 是什么？

如果你熟悉 Python 科学计算，一定用过 NumPy：

```
import numpy as np

# NumPy 数组：高效存储多维数据
x = np.array([[1.0, 2.0], [3.0, 4.0]])
y = np.array([[5.0, 6.0], [7.0, 8.0]])

# 矩阵乘法：对应数学中的  $Z = X \cdot Y$ 
z = np.dot(x, y)
print(z)
# [[19. 22.]
#  [43. 50.]
```

NumPy 解决了 Python 列表的两个问题：

1. **存储效率**：连续内存，比 Python 列表快 10-100 倍
2. **运算效率**：底层 C 实现，向量化操作避免 Python 循环

但这对于深度学习还不够。

4.2.2 为什么 NumPy 不够用？

问题 1：没有自动微分

在反向传播算法中，我们需要计算 $\frac{\partial L}{\partial w}$ 。用 NumPy，你必须手动推导和实现：

```
# NumPy 实现线性回归的梯度
# 手动推导：dL/dw = 2 * X^T @ (Xw - y) / n
def compute_gradient(X, y, w):
    n = len(y)
    predictions = X @ w
    error = predictions - y
    gradient = 2 * X.T @ error / n # 手动实现的梯度公式
    return gradient
```

问题：稍微复杂的网络（比如 LeNet），梯度公式有几十行，非常容易出错。

问题 2：不能用 GPU

现代 GPU 有数千个计算核心，比 CPU 快 10-100 倍。但 NumPy 只能在 CPU 上运行。

```
# 大规模矩阵乘法在 CPU 上可能要几分钟
large_matrix = np.random.randn(10000, 10000)
result = large_matrix @ large_matrix # CPU 计算，慢！
```

问题 3：与深度学习生态脱节

NumPy 是通用科学计算库，没有：

- 预定义的神经网络层（卷积、池化等）
- 优化器（SGD、Adam 等）
- 数据加载和预处理工具

需要一个新的数据结构：保持 NumPy 的易用性，同时解决上述问题。这就是 PyTorch 的 Tensor。

张量的维度层级：

4.2.3 PyTorch 张量：NumPy + 自动微分 + GPU

基础用法：和 NumPy 几乎一样

```
import torch

# 创建张量：语法和 NumPy 几乎相同
```

(续下页)

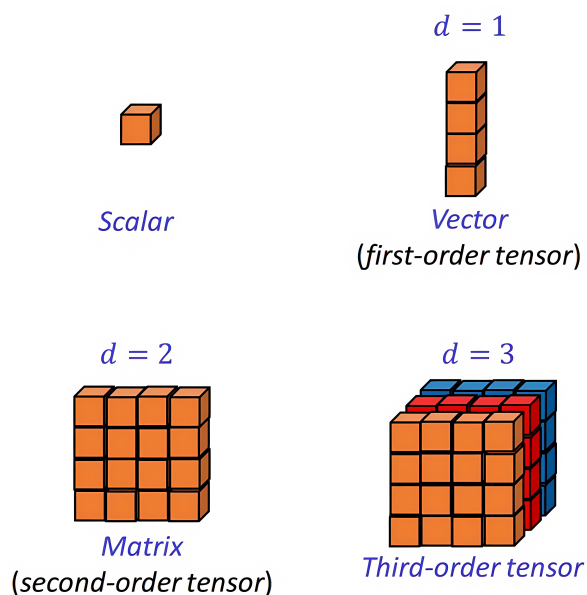


图 1: 张量维度从标量到高阶张量: 0 维标量 (Scalar)、1 维向量 (Vector)、2 维矩阵 (Matrix)、3 维张量 (Third-order tensor)。张量的阶数 (order) 等于索引它的下标数量。

(接上页)

```
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
y = torch.tensor([[5.0, 6.0], [7.0, 8.0]])

# 矩阵乘法: 语法也类似
z = torch.matmul(x, y)
print(z)
# tensor([[19., 22.],
#         [43., 50.]])
```

关键区别: `torch.Tensor` 是计算图中的节点, 而 `np.array` 只是数据存储。

张量作为计算图节点:

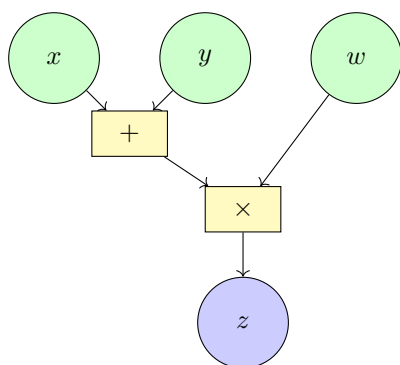
关键特性 1: 自动微分

这是 PyTorch 张量最核心的特性 —— 自动记录计算历史。

```
# 创建需要梯度的张量 (对应计算图中的参数节点)
x = torch.tensor(2.0, requires_grad=True)
w = torch.tensor(3.0, requires_grad=True)

# 前向计算: z = x * w
z = x * w
```

(续下页)



每个节点 = torch.Tensor

```

x: requires_grad=True
  data=2.0, grad=3.0
y: requires_grad=True
  data=3.0, grad=3.0
w: requires_grad=True
  data=4.0, grad=5.0
z: requires_grad=False
  data=20.0
  
```

图 2: 张量与计算图节点的对应

(接上页)

```

print(f"z = {z}") # z = 6.0

# 反向传播: 计算梯度
z.backward()

# 查看梯度
print(f"∂z/∂x = {x.grad}") # ∂z/∂x = w = 3.0
print(f"∂z/∂w = {w.grad}") # ∂z/∂w = x = 2.0
  
```

这对应什么理论?

这行代码 `z.backward()` 实现了 反向传播算法 的全部逻辑:

- 自动构建计算图
- 从输出节点回溯
- 应用链式法则计算每个参数的梯度

在 NumPy 中, 这需要几十行代码手动实现。PyTorch 只需要一行。

关键特性 2: GPU 加速

```

# 检查是否有 GPU
if torch.cuda.is_available():
    device = torch.device('cuda')
  
```

(续下页)

(接上页)

```

print(f"使用 GPU: {torch.cuda.get_device_name(0)}")
else:
    device = torch.device('cpu')
    print("使用 CPU")

# 把张量移到 GPU
x_gpu = x.to(device)
w_gpu = w.to(device)

# 现在在 GPU 上计算
z_gpu = x_gpu * w_gpu

```

性能对比

在 [实验对比：登山日志](#) 中，我们讨论了训练速度。使用 GPU 通常比 CPU 快 10-100 倍，这让训练大模型（如 ResNet）成为可能。

4.2.4 张量的属性：理解计算图节点

每个 PyTorch 张量都有三个关键属性，对应 [计算图](#) 中的节点特性：

属性	含义	对应计算图概念
data	存储的数值	节点的值
grad	梯度值	损失对该节点的偏导数
requires_grad	是否需要梯度	是否是参数节点
is_leaf	是否是叶子节点	是否是输入/参数

```

# 详细查看张量属性
x = torch.tensor(2.0, requires_grad=True)
print(f"值: {x.data}")           # 2.0
print(f"需要梯度: {x.requires_grad}") # True
print(f"是否叶子: {x.is_leaf}")   # True

# 经过运算后的张量
y = x * 3
print(f"y 是否叶子: {y.is_leaf}") # False (中间节点)

```

4.2.5 完整示例：线性回归的 NumPy vs PyTorch

让我们用同一个问题（线性回归）对比两种实现，看看 PyTorch 如何简化代码：

NumPy 实现（手动梯度）

```
import numpy as np

# 数据:  $y = 2x + 1 + \text{noise}$ 
np.random.seed(42)
X = np.random.randn(100, 1)
y = 2 * X + 1 + np.random.randn(100, 1) * 0.1

# 参数初始化
w = np.zeros((1, 1))
b = np.zeros(1)

# 训练 100 步
lr = 0.1
for epoch in range(100):
    # 前向传播
    y_pred = X @ w + b

    # 计算损失
    loss = np.mean((y_pred - y) ** 2)

    # 手动计算梯度（需要推导公式！）
    #  $\partial L / \partial w = 2 * X^T @ (y\_pred - y) / n$ 
    #  $\partial L / \partial b = 2 * \text{mean}(y\_pred - y)$ 
    grad_w = 2 * X.T @ (y_pred - y) / len(X)
    grad_b = 2 * np.mean(y_pred - y)

    # 参数更新（梯度下降）
    w -= lr * grad_w
    b -= lr * grad_b

print(f"学习到的参数: w={w[0,0]:.3f}, b={b[0]:.3f}")
# 应该接近 w=2, b=1
```

PyTorch 实现（自动微分）

```

import torch

# 数据
torch.manual_seed(42)
X = torch.randn(100, 1)
y = 2 * X + 1 + torch.randn(100, 1) * 0.1

# 参数: requires_grad=True 表示需要计算梯度
w = torch.zeros((1, 1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)

# 训练
lr = 0.1
for epoch in range(100):
    # 前向传播
    y_pred = X @ w + b

    # 计算损失
    loss = torch.mean((y_pred - y) ** 2)

    # 反向传播: 自动计算所有梯度!
    loss.backward()

    # 参数更新
    with torch.no_grad(): # 更新时不需要计算梯度
        w -= lr * w.grad
        b -= lr * b.grad
        w.grad.zero_() # 清空梯度, 准备下一轮
        b.grad.zero_()

print(f"学习到的参数: w={w.item():.3f}, b={b.item():.3f}")

```

对比分析

方面	NumPy 实现	PyTorch 实现
梯度计算	手动推导公式（易错）	backward() 自动计算
代码行数	需要额外 3-4 行梯度计算	一行搞定
扩展性	网络复杂后难以维护	自动处理任意复杂网络
对应理论	手动实现 梯度下降与优化算法	框架封装了优化算法

核心洞察：PyTorch 没有改变算法，只是自动化了机械性的梯度计算。

4.2.6 常见陷阱

陷阱 1：原地修改张量

```
x = torch.tensor([1.0, 2.0], requires_grad=True)

# 错误：原地修改会破坏计算图
# x += 1 # 这会报错！

# 正确：创建新张量
x = x + 1 # 这是安全的
```

陷阱 2：忘记清空梯度

```
# 错误：梯度会累加！
for epoch in range(100):
    loss = compute_loss()
    loss.backward()
    # 如果不清空，w.grad 会累加所有轮次的梯度
```

陷阱 3：在梯度计算图中更新参数

```
# 错误：这会在计算图中记录参数更新操作
w = w - lr * w.grad # 不要这样做！

# 正确：用 torch.no_grad() 上下文
with torch.no_grad():
    w -= lr * w.grad
```

4.2.7 下一步

现在你已经理解了张量是 **计算图** 的代码实现。接下来：

1. **张量操作：数据在网络中的流动：**学习张量的各种操作（reshape、transpose 等），理解它们对应的数据流变换
2. **神经网络模块：搭建计算图：**用张量搭建 全连接：赤手空拳 和 卷积：发现装备 中的网络

关键认知：PyTorch 开发的核心流程就是 ——

- 用 `torch.Tensor` 存储数据（对应计算图节点）

- 用张量运算实现前向传播（数据流动）
- 用 `.backward()` 自动计算梯度（反向传播）
- 用优化器更新参数（梯度下降）

4.3 张量操作：数据在网络中的流动

在 **计算图** 中，数据沿着计算图的边流动——从输入节点流向输出节点。但在实际代码中，这些数据需要**变形**以适应不同层的输入要求：

- 卷积层输出 `[batch, 32, 28, 28]`，但全连接层需要 `[batch, 25088]`——这需要 **reshape**
- 单张图片是 `[3, 224, 224]`，但批次需要 `[batch, 3, 224, 224]`——这需要 **unsqueeze**
- 两个不同形状的张量如何相加？——这需要**广播机制**

本章将学习 PyTorch 中改变张量"形状"的各种操作，理解它们如何对应神经网络中的数据流转。

4.3.1 形状操作：改变张量的"视图"

Reshape vs View：重新排列元素

Reshape 和 View 就像重新排列书架上的书——书还是那些书，但摆放方式变了。

```
import torch

# 原始张量：2 张 3×4 的特征图（如卷积层输出）
x = torch.randn(2, 3, 4)
print(f"原始形状: {x.shape}") # torch.Size([2, 3, 4])

# reshape/view: 改变形状，但不改变数据
# 想象把书架从 2×3×4 重新排列成 6×4
y = x.reshape(6, 4) # 或 x.view(6, 4)
print(f"reshape 后: {y.shape}") # torch.Size([6, 4])

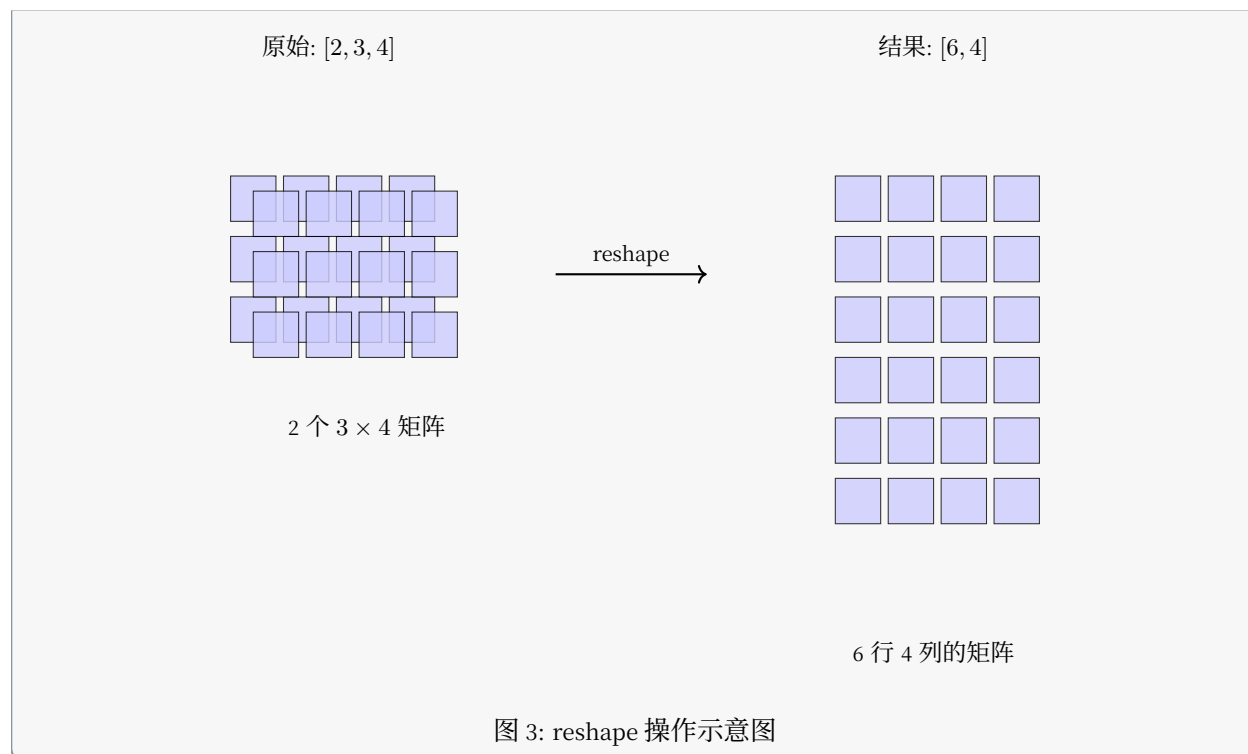
# 元素数量必须一致：2×3×4 = 6×4 = 24
print(f"元素数量: {x.numel()} == {y.numel()}") # True
```

reshape vs view 的区别

view：要求张量在内存中是**连续的**，速度更快但不灵活

reshape：可以处理非连续张量（必要时复制数据），更灵活但可能稍慢

建议：能用 view 就用 view，不确定时用 reshape **reshape 的可视化**：



在神经网络中的应用:

```
# CNN → 全连接的经典转换
conv_output = torch.randn(64, 32, 28, 28) # batch=64, 32 通道, 28x28 特征图

# 展平为全连接层的输入: 64 个样本, 每个 32x28x28 = 25088 维
fc_input = conv_output.reshape(64, -1) # -1 表示自动计算
print(f"展平后: {fc_input.shape}") # torch.Size([64, 25088])

# 这就是 {doc}`../cnn-expedition/practice-peak/le-net` 中 C5 层到 F6 层的操作!
```

squeeze 与 unsqueeze: 增减维度

直觉:

- **unsqueeze**: 在指定位置插入一个维度为 1 的轴 (增加一个"壳")
- **squeeze**: 移除所有维度为 1 的轴 (剥掉"壳")

squeeze/unsqueeze 的可视化:

```
# 单张图片: [3, 224, 224] — 没有 batch 维度
image = torch.randn(3, 224, 224)
print(f"原始: {image.shape}") # torch.Size([3, 224, 224])
```

(续下页)

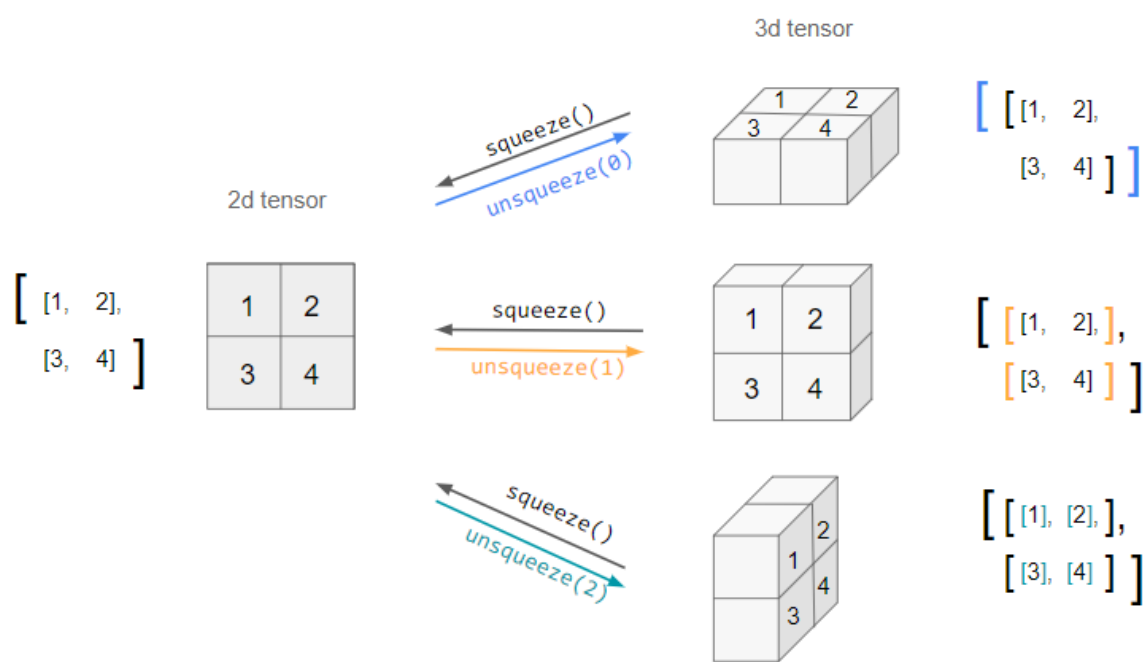


图 4: `squeeze` 和 `unsqueeze` 操作示意图: `squeeze` 移除维度为 1 的轴, `unsqueeze` 在指定位置添加维度为 1 的轴。图中展示了 2D 张量如何通过不同的 `unsqueeze` 操作变成不同形状的 3D 张量。

(接上页)

```
# unsqueeze(0): 在第 0 维添加 batch 维度
batch_image = image.unsqueeze(0)
print(f"加 batch 维度: {batch_image.shape}") # torch.Size([1, 3, 224, 224])

# 也可以在其他位置添加
channel_first = image.unsqueeze(1) # 在通道维度前添加
print(f"在中间添加: {channel_first.shape}") # torch.Size([3, 1, 224, 224])
```

squeeze 的用法:

```
# 模型输出的 logits: [batch, 1] —— 多余的维度
logits = torch.randn(64, 1)

# squeeze(): 移除所有维度为 1 的轴
predictions = logits.squeeze() # 或 logits.squeeze(1)
print(f"squeeze 后: {predictions.shape}") # torch.Size([64])

# 注意: squeeze 只移除维度为 1 的轴, 不会删除其他维度
x = torch.randn(2, 1, 3, 1, 4)
print(f"squeeze 后: {x.squeeze().shape}") # torch.Size([2, 3, 4])
```

什么时候用 squeeze/unsqueeze?

unsqueeze 常见场景:

- 单张图片 → 批次: 需要在前面加 batch 维度
- 调整广播: 让两个张量的维度对齐以便运算

squeeze 常见场景:

- 移除模型输出中多余的维度
- 计算损失前调整形状

transpose 与 permute: 重排维度顺序

直觉: 想象一个魔方, 你可以旋转它的面 —— 元素位置改变, 但数据不变。

```
# 原始: [batch, channels, height, width] —— PyTorch 默认格式
x = torch.randn(2, 3, 4, 5)
```

(续下页)

(接上页)

```
print(f"原始: {x.shape}") # torch.Size([2, 3, 4, 5])

# transpose: 交换两个维度
y = x.transpose(1, 2) # 交换 channels 和 height
print(f"transpose: {y.shape}") # torch.Size([2, 4, 3, 5])

# permute: 任意重排所有维度
z = x.permute(0, 2, 3, 1) # [batch, height, width, channels]
print(f"permute: {z.shape}") # torch.Size([2, 4, 5, 3])
```

实际应用：图像格式转换

```
# OpenCV 读取的图片是 [H, W, C] (通道在最后)
opencv_image = torch.randn(224, 224, 3)

# 转换为 PyTorch 格式 [C, H, W] (通道在最前)
pytorch_image = opencv_image.permute(2, 0, 1)
print(f"PyTorch 格式: {pytorch_image.shape}") # torch.Size([3, 224, 224])
```

flatten：完全展平

```
# 卷积层输出: [batch, 32, 7, 7]
conv_out = torch.randn(64, 32, 7, 7)

# flatten: 从指定维度开始展平
fc_input = conv_out.flatten(start_dim=1) # 保持 batch, 展平后面所有维度
print(f"flatten: {fc_input.shape}") # torch.Size([64, 1568])

# 等价于
fc_input = conv_out.reshape(64, -1)
```

4.3.2 广播机制：让不同形状的张量一起运算

什么是广播？

直觉：广播就像班级合影——小个子站在凳子上，大个子弯下腰，最终大家的脸在同一水平线上。

```
# 场景 1: 张量 + 标量
x = torch.tensor([[1, 2], [3, 4]])
y = x + 10 # 标量 10 被"广播"为 [[10, 10], [10, 10]]
print(y)
```

(续下页)

(接上页)

```
# tensor([[11, 12],
#         [13, 14]])
```

广播规则 (从右到左比较维度):

1. 维度相等: 可以广播
2. 其中一个为 1: 可以广播 (复制该维度)
3. 都不为 1 且不相等: **不能广播**

广播可视化:

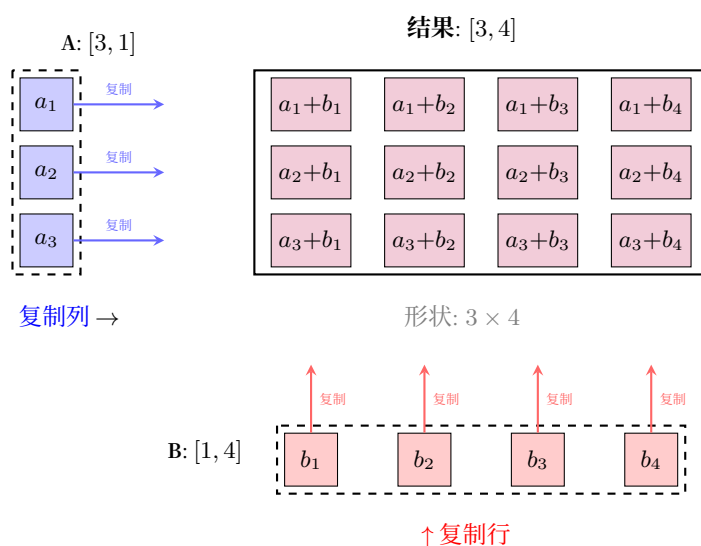


图 5: 广播机制示意图

广播示例

```
# 场景 2: 不同形状但兼容
# A: [3, 1] — 3 行 1 列
# B: [1, 4] — 1 行 4 列
A = torch.tensor([[1], [2], [3]]) # shape: [3, 1]
B = torch.tensor([[10, 20, 30, 40]]) # shape: [1, 4]

# A 被广播为 [3, 4]: 复制列
# B 被广播为 [3, 4]: 复制行
C = A + B
print(f"C 的形状: {C.shape}") # torch.Size([3, 4])
print(C)
```

(续下页)

(接上页)

```
# tensor([[11, 21, 31, 41],
#         [12, 22, 32, 42],
#         [13, 23, 33, 43]])
```

神经网络中的广播应用

```
# 批归一化中的均值减法
batch_data = torch.randn(64, 3, 224, 224) # batch=64, 3 通道

# 计算每个通道的均值: [3] —— 每个通道一个均值
channel_mean = batch_data.mean(dim=[0, 2, 3]) # 在 batch、height、width 上求平均
print(f"通道均值形状: {channel_mean.shape}") # torch.Size([3])

# 广播减法: channel_mean [3] - 自动广播为 [64, 3, 224, 224]
normalized = batch_data - channel_mean.view(1, 3, 1, 1)
print(f"归一化后: {normalized.shape}") # torch.Size([64, 3, 224, 224])
```

4.3.3 索引与切片：精准定位数据

基本索引

```
x = torch.randn(4, 5, 6) # 类比: 4 张图片, 每张 5×6

# 取第一张图片
first = x[0] # shape: [5, 6]

# 取所有图片的第 0 行
first_row = x[:, 0] # shape: [4, 6]

# 取子张量
patch = x[1:3, 2:4, :] # shape: [2, 2, 6]
```

高级索引

```
# 用索引张量选取
x = torch.randn(5, 3)
indices = torch.tensor([0, 2, 4]) # 选第 0、2、4 行
selected = x[indices] # shape: [3, 3]

# 布尔掩码
```

(续下页)

(接上页)

```
mask = x > 0
positive = x[mask] # 一维张量，包含所有正数
```

gather 与 scatter：复杂数据重排

```
# gather：按索引收集数据
src = torch.tensor([[1, 2], [3, 4], [5, 6]]) # [3, 2]
index = torch.tensor([[0, 0], [1, 0], [0, 1]]) # [3, 2]

# 按 index 从 src 中收集
dst = torch.gather(src, 1, index)
print(dst)
# tensor([[1, 1], # 第 0 行取 src[0,0], src[0,0]
#         [4, 3], # 第 1 行取 src[1,1], src[1,0]
#         [5, 6]]) # 第 2 行取 src[2,0], src[2,1]
```

4.3.4 内存与性能

视图 vs 复制

```
x = torch.randn(2, 3, 4)

# view/reshape：共享内存（视图）
y = x.view(6, 4)
y[0, 0] = 999
print(x[0, 0, 0]) # 999 —— x 也被修改了！

# clone()：创建副本
z = x.clone().view(6, 4)
z[0, 0] = 888
print(x[0, 0, 0]) # 999 —— x 不受影响
```

什么时候需要 clone?

需要 clone 的场景：

- 你想修改张量但保留原始数据
- 原地操作会导致梯度计算错误时
- 需要断开源张量的计算图

contiguous：内存连续性

```

x = torch.randn(2, 3, 4)

# transpose 后张量可能不连续
y = x.transpose(0, 1) # shape: [3, 2, 4]
print(y.is_contiguous()) # False

# view 要求连续内存，会报错
# z = y.view(6, 4) # RuntimeError!

# 方法 1: 先 contiguous
z = y.contiguous().view(6, 4)

# 方法 2: 直接用 reshape (自动处理)
z = y.reshape(6, 4)

```

4.3.5 操作速查表

操作	作用	神经网络场景	内存影响
view	改变形状	CNN → FC 转换	视图（共享）
reshape	改变形状（更灵活）	通用形状变换	可能复制
squeeze	移除维度为 1 的轴	移除多余维度	视图
unsqueeze	添加维度为 1 的轴	添加 batch 维度	视图
transpose	交换两个维度	图像格式转换	视图
permute	任意重排维度	NHWC → NCHW	视图
flatten	展平指定维度后所有轴	特征图 → 向量	视图
expand	广播（不复制数据）	匹配形状	视图
repeat	复制数据	重复张量	复制

4.3.6 下一步

掌握了张量操作后，我们可以开始构建神经网络了。在 [神经网络模块：搭建计算图](#) 中：

- 用 `nn.Linear` 实现 [全连接：赤手空拳](#) 中的全连接层
- 用 `nn.Conv2d` 实现 [卷积：发现装备](#) 中的卷积层
- 理解 `forward` 方法中的张量流动

核心认知：神经网络的每一层本质上都是**张量 → 张量**的映射 —— 理解形状变换，就理解了数据在网络中的流动。

4.4 神经网络模块：搭建计算图

在 [全连接：赤手空拳](#) 和 [卷积：发现装备](#) 中，我们学习了全连接层和卷积层的数学原理：

- **全连接层**： $y = Wx + b$ ，每个输入连接所有输出
- **卷积层**：局部感受野 + 权值共享，检测空间特征

本章将学习如何用 PyTorch 的 `nn.Module` 把这些数学公式变成可运行的代码。你会发现，每个层类 (`nn.Linear`、`nn.Conv2d`) 都直接对应一个数学运算。

4.4.1 从数学公式到 PyTorch 层

全连接层： `nn.Linear`

回顾 [全连接：赤手空拳](#)：

$$y_j = \sum_{i=1}^n W_{ji}x_i + b_j$$

PyTorch 实现：

```
import torch
import torch.nn as nn

# 输入：784 维（如展平后的 28×28 图像）
# 输出：256 维（隐藏层）
fc = nn.Linear(in_features=784, out_features=256)

# 查看参数
print(f"权重形状: {fc.weight.shape}") # torch.Size([256, 784])
print(f"偏置形状: {fc.bias.shape}")   # torch.Size([256])

# 前向传播
x = torch.randn(64, 784) # batch=64
y = fc(x)                 # y = x @ W^T + b
print(f"输出形状: {y.shape}") # torch.Size([64, 256])
```

参数量计算

全连接层参数量 = 输入维度 × 输出维度 + 输出维度（偏置）

- 权重： $784 \times 256 = 200,704$
- 偏置： 256

· 总计：200,960 参数

这与 全连接：赤手空拳 中的计算一致。

全连接层的可视化：

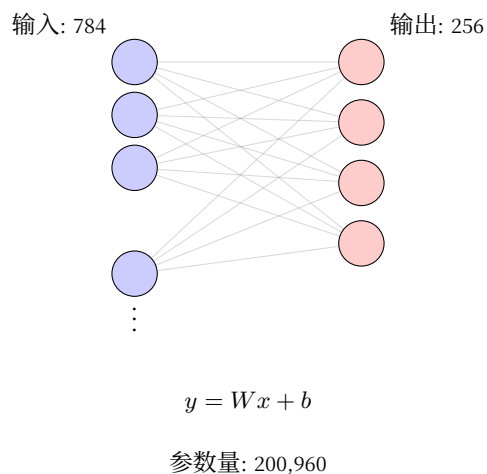


图 6: 全连接层结构示意图

卷积层：nn.Conv2d

回顾 卷积：发现装备：

$$Y[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X[i + u, j + v] \cdot K[u, v]$$

PyTorch 实现：

```
# 输入：3 通道图像，输出：32 个特征图
# 卷积核：5×5，步长：1，填充：0
conv = nn.Conv2d(
    in_channels=3,      # 输入通道（RGB）
    out_channels=32,    # 输出通道（32 个卷积核）
    kernel_size=5,      # 卷积核大小
    stride=1,           # 步长
    padding=0           # 填充
)

# 查看参数
print(f"卷积核形状: {conv.weight.shape}") # torch.Size([32, 3, 5, 5])
print(f"偏置形状: {conv.bias.shape}")     # torch.Size([32])
```

(续下页)

(接上页)

```
# 前向传播
x = torch.randn(64, 3, 32, 32) # batch=64, 3 通道, 32x32 图像
y = conv(x)                       # 卷积运算
print(f"输出形状: {y.shape}")    # torch.Size([64, 32, 28, 28])
# (32 - 5 + 0) / 1 + 1 = 28
```

参数量计算

卷积层参数量 = 输出通道 × 输入通道 × 卷积核高 × 卷积核宽 + 输出通道（偏置）

- 卷积核: $32 \times 3 \times 5 \times 5 = 2,400$
- 偏置: 32
- 总计: 2,432 参数

对比全连接层（200,960 参数），卷积层参数量少了 82 倍！这就是 **归纳偏置** 的威力——利用空间局部性减少参数。

卷积层的可视化：

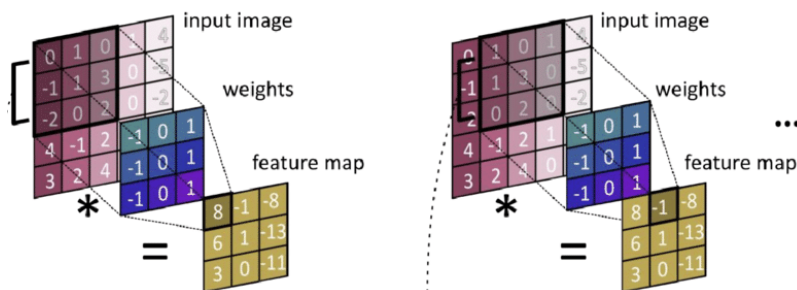


图 7: 卷积运算示意图：同一个卷积核（蓝色 3×3 权重矩阵）在输入图像上滑动，每次计算点积得到一个输出值。相同的权重被共享用于检测图像不同位置的特征。

4.4.2 nn.Module：构建网络的基石

什么是 nn.Module?

`nn.Module` 是 PyTorch 中所有神经网络组件的基类。它提供了：

- 参数管理（自动注册可学习参数）
- 前向传播定义（`forward` 方法）
- 设备转移（`.to('cuda')`）

自定义网络类

模板:

```
import torch.nn as nn

class MyNetwork(nn.Module):
    def __init__(self):
        super(MyNetwork, self).__init__()

        # 1. 定义层 (在__init__中创建)
        self.fc1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        # 2. 定义前向传播 (数据如何流动)
        x = self.fc1(x)      # 线性变换
        x = self.relu(x)     # 激活函数 (见{ref}`activation-functions`)
        x = self.dropout(x)  # 正则化 (见{doc}`neural-training-basics`)
        x = self.fc2(x)      # 输出层
        return x

# 创建实例
model = MyNetwork()
print(model)
```

完整示例: 实现 LeNet-5

让我们用 PyTorch 实现 LeNet-5: 练习峰首登 中的经典架构:

```
class LeNet5(nn.Module):
    """
    LeNet-5 的 PyTorch 实现
    对应 {doc}`../cnn-expedition/practice-peak/le-net` 中的架构分析
    """
    def __init__(self, num_classes=10):
        super(LeNet5, self).__init__()

        # C1: 卷积层, 1-6 通道, 5×5 卷积核, 输出 28×28
        # 参数量: 6×1×5×5 + 6 = 156
```

(续下页)

(接上页)

```

self.c1 = nn.Conv2d(1, 6, kernel_size=5, padding=2)

# S2: 2×2 平均池化, 输出 14×14
self.s2 = nn.AvgPool2d(kernel_size=2, stride=2)

# C3: 卷积层, 6→16 通道, 5×5 卷积核, 输出 10×10
# 参数量: 16×6×5×5 + 16 = 2,416
self.c3 = nn.Conv2d(6, 16, kernel_size=5)

# S4: 2×2 平均池化, 输出 5×5
self.s4 = nn.AvgPool2d(kernel_size=2, stride=2)

# C5: 卷积层 (实为全连接), 16→120, 输出 1×1
# 参数量: 120×16×5×5 + 120 = 48,120
self.c5 = nn.Conv2d(16, 120, kernel_size=5)

# F6: 全连接层, 120→84
# 参数量: 120×84 + 84 = 10,164
self.f6 = nn.Linear(120, 84)

# 输出层, 84→10 (类别数)
# 参数量: 84×10 + 10 = 850
self.output = nn.Linear(84, num_classes)

def forward(self, x):
    # C1: [batch, 1, 28, 28] → [batch, 6, 28, 28]
    x = torch.tanh(self.c1(x))

    # S2: [batch, 6, 28, 28] → [batch, 6, 14, 14]
    x = self.s2(x)

    # C3: [batch, 6, 14, 14] → [batch, 16, 10, 10]
    x = torch.tanh(self.c3(x))

    # S4: [batch, 16, 10, 10] → [batch, 16, 5, 5]
    x = self.s4(x)

    # C5: [batch, 16, 5, 5] → [batch, 120, 1, 1]
    x = torch.tanh(self.c5(x))

```

(续下页)

(接上页)

```

# 展平: [batch, 120, 1, 1] → [batch, 120]
x = x.view(x.size(0), -1)

# F6: [batch, 120] → [batch, 84]
x = torch.tanh(self.f6(x))

# 输出: [batch, 84] → [batch, 10]
x = self.output(x)
return x

# 验证参数量
model = LeNet5()
total_params = sum(p.numel() for p in model.parameters())
print(f"总参数量: {total_params:,}") # 61,706 (与论文一致!)

```

维度追踪技巧

写 forward 时，建议在每个操作后注释输出形状：

```

def forward(self, x):
    x = self.conv1(x) # [64, 3, 32, 32] → [64, 32, 28, 28]
    x = self.pool(x) # [64, 32, 28, 28] → [64, 32, 14, 14]
    x = self.conv2(x) # [64, 32, 14, 14] → [64, 64, 10, 10]
    # ...

```

这样出错时可以快速定位维度不匹配的位置。

4.4.3 容器：组织多个层

nn.Sequential：顺序执行

当层按顺序连接时，用 nn.Sequential 简化代码：

```

# 方式 1：逐个添加
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 10)
)

```

(续下页)

(接上页)

```
# 方式 2: 带名称 (便于访问)
from collections import OrderedDict

model = nn.Sequential(OrderedDict([
    ('fc1', nn.Linear(784, 256)),
    ('relu1', nn.ReLU()),
    ('dropout', nn.Dropout(0.2)),
    ('fc2', nn.Linear(256, 10))
]))

# 访问特定层
print(model.fc1) # 或 model[0]
```

nn.ModuleList: 动态层数

当层数不固定 (如根据配置决定深度) 时使用:

```
class DynamicNet(nn.Module):
    def __init__(self, layer_sizes):
        super(DynamicNet, self).__init__()

        self.layers = nn.ModuleList()
        for i in range(len(layer_sizes) - 1):
            self.layers.append(nn.Linear(layer_sizes[i], layer_sizes[i+1]))
            if i < len(layer_sizes) - 2: # 最后一层不加激活
                self.layers.append(nn.ReLU())

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

# 使用
model = DynamicNet([784, 256, 128, 64, 10])
```

4.4.4 参数管理

查看和访问参数

```
model = LeNet5()
```

(续下页)

(接上页)

```
# 查看所有参数
for name, param in model.named_parameters():
    print(f"{name}: {param.shape} ({param.numel()} 参数)")

# 输出示例:
# c1.weight: torch.Size([6, 1, 5, 5]) (150 参数)
# c1.bias: torch.Size([6]) (6 参数)
# c3.weight: torch.Size([16, 6, 5, 5]) (2400 参数)
# ...

# 只查看可训练参数
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"可训练参数量: {trainable_params:,}")
```

参数初始化

良好的初始化对训练很重要:

```
def initialize_weights(m):
    """自定义初始化"""
    if isinstance(m, nn.Linear):
        # Xavier 初始化 (适合 tanh/sigmoid)
        nn.init.xavier_uniform_(m.weight)
        nn.init.zeros_(m.bias)
    elif isinstance(m, nn.Conv2d):
        # He 初始化 (适合 ReLU)
        nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
        nn.init.zeros_(m.bias)

# 应用到整个模型
model.apply(initialize_weights)
```

冻结参数

迁移学习中，常常需要冻结预训练层的参数:

```
# 冻结卷积层，只训练全连接层
for param in model.c1.parameters():
    param.requires_grad = False

for param in model.c3.parameters():
```

(续下页)

(接上页)

```
param.requires_grad = False

# 验证
for name, param in model.named_parameters():
    print(f"{name}: requires_grad={param.requires_grad}")
```

4.4.5 常见层详解

激活函数层

```
# ReLU (最常用)
relu = nn.ReLU() # max(0, x)

# Sigmoid (二分类输出)
sigmoid = nn.Sigmoid() # 1 / (1 + exp(-x))

# Tanh (早期网络)
tanh = nn.Tanh()

# Softmax (多分类输出)
softmax = nn.Softmax(dim=1) # 沿类别维度
```

归一化层

```
# BatchNorm (最常用)
bn = nn.BatchNorm2d(num_features=64) # 对 64 个通道分别归一化

# LayerNorm (适合序列数据)
ln = nn.LayerNorm(normalized_shape=[64, 28, 28])
```

正则化层

```
# Dropout (防止过拟合)
dropout = nn.Dropout(p=0.5) # 50%概率丢弃

# 注意：训练时生效，推理时自动关闭 (model.eval())
```


4.4.6 下一步

现在你已经学会了如何搭建网络架构。接下来：

- 1. **自动微分：PyTorch 的核心魔法**：理解 `.backward()` 如何自动计算梯度（对应 反向传播算法）
- 2. **优化器：用梯度更新参数**：用优化器更新参数（对应 梯度下降与优化算法）

核心认知：`nn.Module` 的本质是封装了参数的数学运算——每个层类都实现了特定的前向传播公式，并自动处理反向传播的梯度计算。

4.5 自动微分：PyTorch 的核心魔法

神经网络模块：搭建计算图中我们学会了构建神经网络结构。现在的问题是：**如何让网络"学习"？**

在 反向传播算法 中，我们学习了反向传播算法的原理——通过链式法则将损失梯度从输出层传回输入层。但如果手动实现这个算法，代码会非常复杂且容易出错。

PyTorch 的自动微分（autograd）就是解决方案：它自动构建计算图并执行反向传播，让我们只需关注前向计算，梯度会自动计算。

4.5.1 直觉理解：自动微分是什么？

类比：自动记账系统

想象你经营一家连锁餐厅：

- **手动反向传播**：每道菜卖了多少钱，你需要手动追踪每一笔成本（食材、人工、租金），然后计算每个分店应该调整什么——极其繁琐且容易出错。
- **自动微分**：你只需记录每笔交易（前向传播），系统自动生成财务报表（梯度），告诉你每个分店该如何调整。

核心洞察：自动微分 = 自动构建计算图 + 自动执行反向传播 + 自动存储梯度

4.5.2 从理论到代码

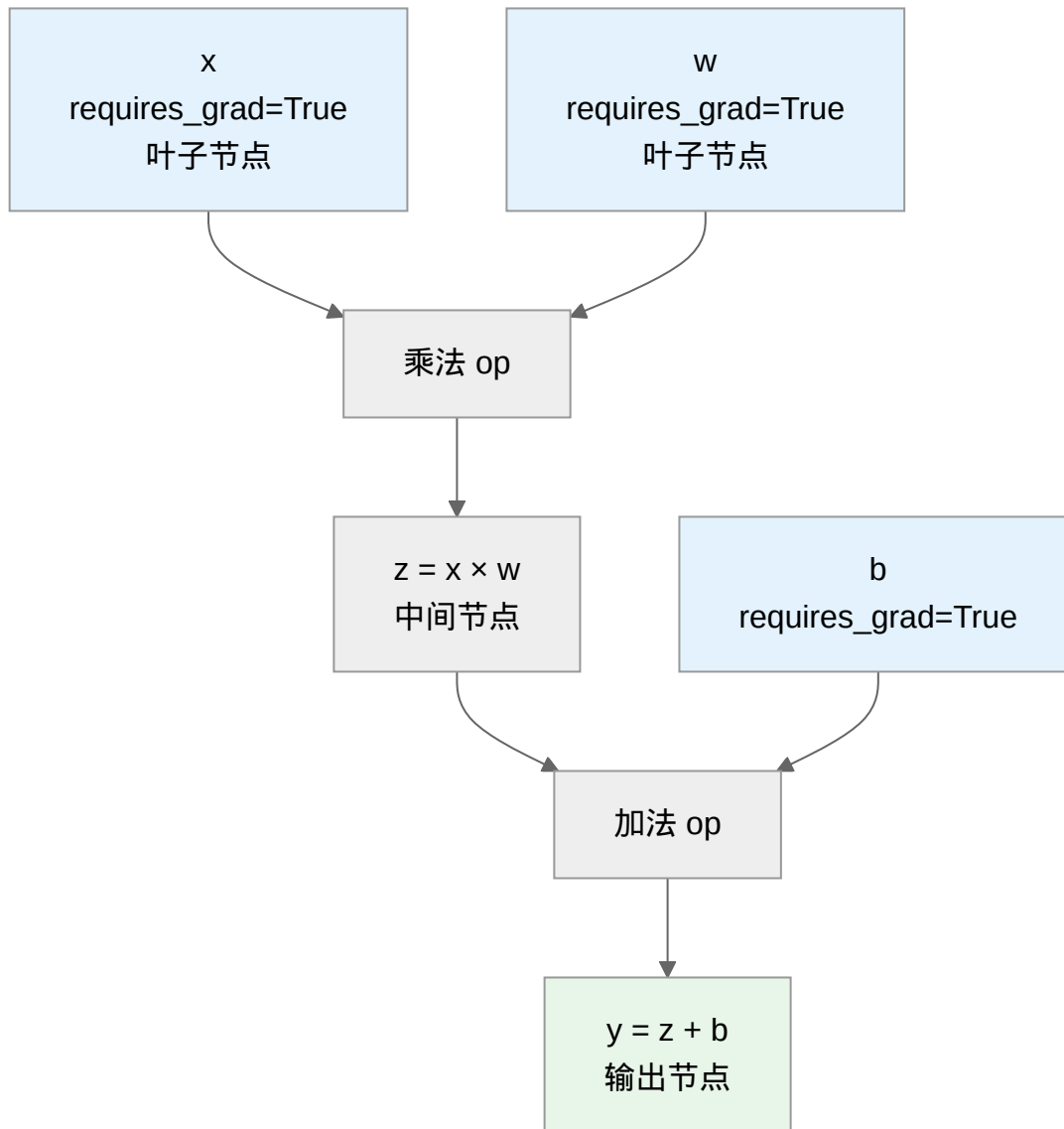
反向传播算法 理论	PyTorch 实现	作用
计算图	<code>requires_grad=True</code>	标记需要计算梯度的张量
链式法则	<code>.backward()</code>	自动回传梯度
梯度存储	<code>.grad</code>	存储计算得到的梯度
梯度清零	<code>.zero_()</code>	清除旧梯度，避免累积

4.5.3 计算图的自动构建

什么是动态计算图？

PyTorch 使用**动态计算图** (Dynamic Computational Graph)，这意味着：

1. **图在运行时构建**：每次前向传播都会重新构建图
2. **图结构可以变化**：支持 Python 控制流 (if、for、while)
3. **内存高效**：反向传播后可以释放中间结果



关键区别：

- **叶子节点 (Leaf Node)**：用户创建的张量 (`requires_grad=True`)，梯度会保存
- **中间节点**：运算产生的张量，默认不保存梯度 (除非设置 `retain_graph=True`)

创建可追踪的张量

```

1 import torch
2
3 # 叶子节点: requires_grad=True 告诉 PyTorch"我要跟踪这个张量"
4 x = torch.tensor([2.0, 3.0], requires_grad=True) # 输入特征
5 w = torch.tensor([1.0, 2.0], requires_grad=True) # 权重
6 b = torch.tensor(0.5, requires_grad=True)        # 偏置
7
8 print(f"x.is_leaf: {x.is_leaf}") # True: 用户创建的叶子节点
9 print(f"w.is_leaf: {w.is_leaf}") # True
10
11 # 中间节点: 通过运算产生
12 z = x * w          # 逐元素乘法, 形状 [2]
13 y = z.sum() + b    # 求和后加偏置, 标量
14
15 print(f"z.is_leaf: {z.is_leaf}") # False: 运算产生
16 print(f"y.is_leaf: {y.is_leaf}") # False

```

4.5.4 梯度计算: .backward() 的魔力

标量输出的梯度计算

```

1 import torch
2
3 # 创建叶子节点
4 x = torch.tensor(2.0, requires_grad=True) # 对应 back-propagation 中的示例
5 y = torch.tensor(3.0, requires_grad=True)
6
7 # 前向传播: f = (x + y) × y
8 a = x + y      # a = 5
9 f = a * y      # f = 15
10
11 print(f"前向结果: f = {f.item()}") # 15.0
12
13 # 反向传播: 自动计算梯度
14 f.backward()
15
16 # 查看梯度
17 print(f"∂f/∂x = {x.grad}") # 3.0 (与理论一致!)
18 print(f"∂f/∂y = {y.grad}") # 8.0 (与理论一致!)
19

```

(续下页)

(接上页)

```

20 # 梯度验证 (手工计算):
21 #  $\partial f / \partial x = \partial f / \partial a \times \partial a / \partial x = y \times 1 = 3 \checkmark$ 
22 #  $\partial f / \partial y = \partial f / \partial a \times \partial a / \partial y + \partial f / \partial y$  (直接)  $= y + a = 3 + 5 = 8 \checkmark$ 

```

代码解释:

- `requires_grad=True`: 告诉 PyTorch 跟踪这个张量的所有操作
- `.backward()`: 从输出节点开始, 沿计算图反向传播, 计算所有叶子节点的梯度
- `.grad`: 存储计算得到的梯度值

非标量输出的处理

当输出不是标量时, 需要指定权重向量:

```

import torch

# 输出是向量而非标量
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x * 2 # y = [2, 4, 6]

# 错误: y 是向量, 不能直接 backward()
# y.backward() # RuntimeError!

# 正确: 提供一个权重向量 (相当于计算  $v^T \times J$ )
# 这里我们计算 y 每个元素对 x 的梯度的加权和
v = torch.tensor([1.0, 1.0, 1.0]) # 权重向量
y.backward(v) # 等价于计算  $\text{sum}(y)$  的梯度

print(f"x.grad = {x.grad}") # [2, 2, 2]

# 实际应用: 通常我们会将损失降为标量
loss = y.sum() # 标量
loss.backward() # 可以直接调用

```

4.5.5 梯度控制技巧

梯度累积与清零

关键问题: PyTorch 的 `.backward()` 默认会累积梯度!

```

1 import torch
2
3 x = torch.tensor(2.0, requires_grad=True)
4
5 # 第一次前向+反向
6 y1 = x ** 2      # y1 = 4
7 y1.backward()
8 print(f"第一次反向后 x.grad = {x.grad}") # 4.0 (dy1/dx = 2x = 4)
9
10 # 第二次前向+反向 (不清零)
11 y2 = x ** 3      # y2 = 8
12 y2.backward()
13 print(f"第二次反向后 x.grad = {x.grad}") # 16.0! (4 + 12, 累积了!)
14
15 # 正确做法: 每次反向前清零
16 x.grad.zero_()   # 清零梯度
17 y3 = x ** 2
18 y3.backward()
19 print(f"清零后 x.grad = {x.grad}") # 4.0

```

训练循环中的标准模式:

```

optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

for batch in dataloader:
    # 1. 清零旧梯度
    optimizer.zero_grad() # 或 model.zero_grad()

    # 2. 前向传播
    output = model(batch.input)
    loss = criterion(output, batch.target)

    # 3. 反向传播
    loss.backward()

    # 4. 更新参数
    optimizer.step()

```

禁用梯度计算

在某些情况下，我们不需要计算梯度：

```

1 import torch
2
3 x = torch.tensor(2.0, requires_grad=True)
4
5 # 方式 1: 使用 torch.no_grad() 上下文管理器 (推荐)
6 with torch.no_grad():
7     y = x * 2
8     print(f"y.requires_grad: {y.requires_grad}") # False
9     # y.backward() # 错误! 无法反向传播
10
11 # 方式 2: 使用 .detach() 从计算图中分离
12 z = x * 3
13 z_detached = z.detach()
14 print(f"z_detached.requires_grad: {z_detached.requires_grad}") # False
15
16 # 方式 3: 设置 requires_grad=False (全局)
17 w = torch.tensor(2.0, requires_grad=False)
18 output = w * x
19 print(f"output.requires_grad: {output.requires_grad}") # True (x 需要梯度)

```

何时禁用梯度?

场景	原因	代码
模型推理	不需要更新参数	<code>with torch.no_grad()</code>
特征提取	冻结预训练模型	<code>param.requires_grad = False</code>
数值计算	避免梯度开销	<code>.detach()</code>
保存张量	避免保留计算图	<code>tensor.detach().cpu().numpy()</code>

4.5.6 实际应用：神经网络训练

完整示例：线性回归

```

import torch
import torch.nn as nn

# 生成数据:  $y = 2x + 1 + \text{噪声}$ 
torch.manual_seed(42)

```

(续下页)

(接上页)

```

X = torch.randn(100, 1)          # 100 个样本, 1 个特征
y_true = 2 * X + 1 + 0.1 * torch.randn(100, 1)

# 定义模型: 对应 ../math-fundamentals/gradient-descent 中的线性模型
model = nn.Linear(1, 1)          # 输入 1 维, 输出 1 维

# 损失函数和优化器
criterion = nn.MSELoss()          # 均方误差
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

# 训练循环
for epoch in range(100):
    # 1. 清零梯度
    optimizer.zero_grad()

    # 2. 前向传播
    y_pred = model(X)              # 计算预测值
    loss = criterion(y_pred, y_true) # 计算损失

    # 3. 反向传播 (自动计算梯度)
    loss.backward()

    # 4. 更新参数
    optimizer.step()

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# 查看学习到的参数
print(f"\n 学习到的权重: {model.weight.item():.4f} (真实值: 2.0) ")
print(f" 学习到的偏置: {model.bias.item():.4f} (真实值: 1.0) ")

```

查看计算图

```

import torch

x = torch.tensor(2.0, requires_grad=True)
y = x ** 2

# 查看梯度函数 (对应计算图中的边)

```

(续下页)

(接上页)

```
print(f"y.grad_fn: {y.grad_fn}")          # <PowBackward0 object>
print(f"y.grad_fn.next_functions: {y.grad_fn.next_functions}")

# 查看是否需要梯度
print(f"x.requires_grad: {x.requires_grad}") # True
print(f"y.requires_grad: {y.requires_grad}") # True
```

4.5.7 高级主题

自定义梯度计算

偶尔需要自定义梯度计算规则：

```
import torch
from torch.autograd import Function

class MyReLU(Function):
    """
    自定义 ReLU 激活函数，带梯度裁剪
    对应 {doc}`../math-fundamentals/activation-functions` 中的 ReLU
    """

    @staticmethod
    def forward(ctx, input):
        """前向传播:  $\max(0, x)$ """
        ctx.save_for_backward(input) # 保存用于反向传播的张量
        return input.clamp(min=0)

    @staticmethod
    def backward(ctx, grad_output):
        """反向传播: 梯度裁剪"""
        input, = ctx.saved_tensors
        grad_input = grad_output.clone()
        grad_input[input < 0] = 0 # ReLU 的梯度规则
        return grad_input

# 使用自定义函数
my_relu = MyReLU.apply
x = torch.tensor([-1.0, 2.0, -3.0, 4.0], requires_grad=True)
y = my_relu(x)
y.sum().backward()
```

(续下页)

(接上页)

```
print(f"输入: {x}")
print(f"输出: {y}")
print(f"梯度: {x.grad}") # [0, 1, 0, 1]
```

梯度检查

验证梯度计算是否正确:

```
import torch
from torch.autograd import gradcheck

# 定义需要测试的函数
def func(x):
    return x ** 3 + x ** 2

# 使用双精度浮点数进行数值梯度检查
test_input = torch.randn(3, 4, dtype=torch.double, requires_grad=True)

# 验证梯度
result = gradcheck(func, test_input, eps=1e-6, atol=1e-4)
print(f"梯度检查通过: {result}") # True 表示梯度计算正确
```

4.5.8 总结

核心概念回顾

概念	解释	代码
计算图	记录张量操作的 DAG	自动构建
叶子节点	用户创建的可训练参数	requires_grad=True
反向传播	从输出回传梯度	.backward()
梯度存储	存储在 .grad 属性中	tensor.grad
梯度清零	避免梯度累积	.zero_grad()
禁用梯度	推理时节省内存	torch.no_grad()

下一步

掌握了自动微分后，下一节 **优化器：用梯度更新参数** 我们将学习如何使用这些梯度来更新模型参数 —— 从简单的 SGD 到自适应学习率的 Adam，掌握优化算法的核心原理。

从"计算梯度"到"使用梯度优化"，让我们继续深入！

4.6 优化器：用梯度更新参数

自动微分：PyTorch 的核心魔法中我们学会了自动计算梯度。现在的问题是：**有了梯度，如何更新参数？**

在 **梯度下降与优化算法**中，我们详细学习了梯度下降的原理和各种改进算法（SGD、Momentum、Adam）。本节聚焦 **PyTorch 实现** —— 如何把理论变成可运行的代码。

4.6.1 从理论到代码

梯度下降与优化算法 理论	PyTorch 实现	核心作用
基础梯度下降	<code>optim.SGD(lr=0.01)</code>	沿负梯度方向更新
动量（Momentum）	<code>optim.SGD(momentum=0.9)</code>	累积速度，减少震荡
Adam	<code>optim.Adam(lr=0.001)</code>	自适应学习率，自动调整
学习率调度	<code>lr_scheduler.StepLR</code>	训练后期减小步长

4.6.2 SGD：基础梯度下降

数学回顾

梯度下降与优化算法的核心公式：

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta)$$

含义：新参数 = 当前参数 - 学习率 × 梯度

PyTorch 实现：

```
import torch
import torch.nn as nn
import torch.optim as optim

# 定义模型
model = nn.Linear(10, 1)

# 创建优化器
optimizer = optim.SGD(
    model.parameters(), # 要优化的参数
```

(续下页)

(接上页)

```

    lr=0.01          # 学习率  $\eta$ 
)

# 训练循环
for epoch in range(100):
    optimizer.zero_grad()    # 1. 清零梯度
    output = model(input_data) # 2. 前向传播
    loss = criterion(output, target)
    loss.backward()          # 3. 反向传播
    optimizer.step()         # 4. 更新参数 (执行  $\theta = \theta - lr * grad$ )

```

4.6.3 Momentum：加速收敛

理论回顾

梯度下降与优化算法中的 Momentum 引入"速度"概念：

$$v_{t+1} = \gamma v_t + \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta \cdot v_{t+1}$$

核心思想：新速度 = 保留的原有速度 + 新梯度，用速度代替直接梯度更新参数。

效果：

- 同方向梯度累积加速（下坡更快）
- 反方向梯度相互抵消（减少震荡）

PyTorch 实现

```

# SGD with Momentum
optimizer = optim.SGD(
    model.parameters(),
    lr=0.01,
    momentum=0.9,      # 动量系数  $\gamma$ ，通常 0.9
    nesterov=False     # 是否使用 Nesterov 加速梯度
)

```

对比实验：

```

# 创建两个相同模型
model_vanilla = SimpleNet()
model_momentum = SimpleNet()

```

(续下页)

(接上页)

```
model_momentum.load_state_dict(model_vanilla.state_dict())

# 不同优化器
opt_vanilla = optim.SGD(model_vanilla.parameters(), lr=0.01, momentum=0.0)
opt_momentum = optim.SGD(model_momentum.parameters(), lr=0.01, momentum=0.9)

# 训练后比较: momentum 通常收敛更快, 损失曲线更平滑
```

4.6.4 Adam: 自适应学习率

理论回顾

梯度下降与优化算法中的 Adam 结合了两种技术:

1. 一阶矩估计 (m_t): 梯度的移动平均, 类似 Momentum
2. 二阶矩估计 (v_t): 梯度平方的移动平均, 用于自适应学习率

更新公式:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

核心洞察: 分母 $\sqrt{\hat{v}_t}$ 自动调整每个参数的步长:

- 历史梯度大的参数 → 步长自动变小
- 历史梯度小的参数 → 步长自动变大

PyTorch 实现

```
# Adam 优化器 (最常用的默认选择)
optimizer = optim.Adam(
    model.parameters(),
    lr=0.001,           # 默认学习率
    betas=(0.9, 0.999), # β1 和 β2
    eps=1e-8,          # 数值稳定性
    weight_decay=0.0    # L2 正则化
)
```

为什么选 Adam?

优势	说明
自适应	每个参数自动调整学习率，无需手动调参
动量	内置一阶矩估计，加速收敛
稳定	偏差修正让初期训练更稳定
易用	对超参数不敏感，默认配置通常工作良好

经验法则：

- 不知道用什么？→ 先用 Adam(lr=1e-3)
- 图像分类追求精度？→ 最后用 SGD + Momentum 微调
- 稀疏数据（NLP）？→ Adam 的自适应特性更适合

4.6.5 优化器选择指南

优化器	优点	缺点	推荐使用场景
SGD	泛化性能好	收敛慢，需调学习率	图像分类最终调优
SGD+Momentum	收敛快，减少震荡	仍需调学习率	通用选择
Adam	自适应，易使用	泛化可能略差	默认首选，NLP
AdamW	正确的权重衰减	计算量稍大	需要正则化时

4.6.6 学习率调度

为什么需要调整学习率？

梯度下降与优化算法中的分析：

- 前期：大步快走，快速接近最优
- 后期：小步微调，精确收敛

常用调度策略

```
# 1. StepLR: 每 step_size 个 epoch, 学习率乘以 gamma
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.1)
# 例如: 0.1 → 0.01 (30 epoch 后) → 0.001 (60 epoch 后)

# 2. ExponentialLR: 指数衰减
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

# 3. CosineAnnealingLR: 余弦退火
```

(续下页)

(接上页)

```

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)

# 4. ReduceLR0nPlateau: 根据验证损失调整
scheduler = optim.lr_scheduler.ReduceLR0nPlateau(
    optimizer,
    mode='min',          # 监控最小值
    factor=0.1,          # 学习率乘以 0.1
    patience=10,         # 10 个 epoch 不改善才调整
    verbose=True
)

```

训练循环中使用

```

# 创建优化器和调度器
optimizer = optim.SGD(model.parameters(), lr=0.1)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.1)

for epoch in range(num_epochs):
    # 训练
    train_epoch(...)

    # 更新学习率
    if isinstance(scheduler, optim.lr_scheduler.ReduceLR0nPlateau):
        scheduler.step(val_loss) # 需要传入指标
    else:
        scheduler.step()         # 其他调度器不需要参数

    print(f"Epoch {epoch}, LR: {optimizer.param_groups[0]['lr']:.6f}")

```

4.6.7 完整示例：CIFAR-10 训练

```

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models

# 加载预训练 ResNet
model = models.resnet18(pretrained=True)

# 冻结 backbone (可选)

```

(续下页)

(接上页)

```

for param in model.layer1.parameters():
    param.requires_grad = False

# 修改分类头
model.fc = nn.Linear(model.fc.in_features, 10)

# 分组设置学习率（迁移学习常用）
optimizer = optim.SGD([
    {'params': model.layer1.parameters(), 'lr': 0.0},          # 冻结
    {'params': model.layer2.parameters(), 'lr': 1e-4},        # 微调
    {'params': model.layer3.parameters(), 'lr': 1e-4},
    {'params': model.layer4.parameters(), 'lr': 1e-3},
    {'params': model.fc.parameters(), 'lr': 1e-2}             # 新层，大学习率
], momentum=0.9, weight_decay=5e-4)

# 学习率调度
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.1)

# 损失函数
criterion = nn.CrossEntropyLoss()

# 训练循环
for epoch in range(90):
    model.train()
    for images, labels in train_loader:
        images, labels = images.cuda(), labels.cuda()

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    scheduler.step()

    current_lr = optimizer.param_groups[0]['lr']
    print(f"Epoch [{epoch}/90], LR: {current_lr:.6f}")

```

4.6.8 学习率调参建议

1. 初始学习率:

- SGD: 0.1 或 0.01
- Adam: 0.001 (默认) 或 0.0001

2. 调试技巧:

- 损失不下降? → 学习率太大, 尝试除以 10
- 损失震荡? → 学习率太大, 或 batch size 太小
- 收敛太慢? → 学习率太小, 或尝试 Momentum

4.6.9 总结

核心方法回顾

方法	作用	何时调用
<code>optimizer.zero_grad()</code>	清零梯度	每次反向传播前
<code>loss.backward()</code>	计算梯度	前向传播后
<code>optimizer.step()</code>	更新参数	反向传播后
<code>scheduler.step()</code>	更新学习率	每个 epoch 后

下一步

掌握了优化器后, 下一节 [完整训练流程](#) 我们将把所有组件整合起来 —— 构建一个完整的训练流程。

从"更新参数"到"完整训练流程", 让我们把知识变成可运行的系统!

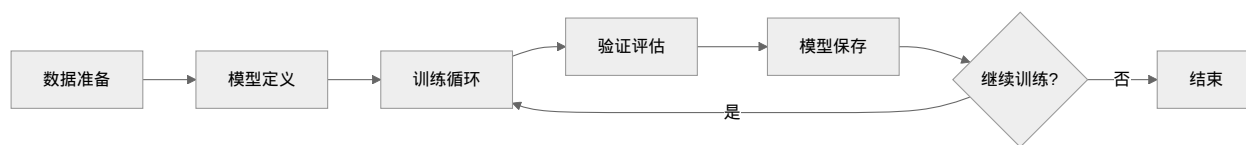
4.7 完整训练流程

优化器: 用梯度更新参数中我们掌握了如何更新参数。现在的问题是: **如何把所有组件整合成一个完整的训练系统?**

在 [训练基础: 登山纪律](#) 中, 我们学习了训练的核心概念 —— Epoch、Batch、过拟合与欠拟合、正则化技巧等。但这些知识零散, 需要串联成可运行的代码。

本节就是答案: 我们将构建一个完整的 MNIST 训练系统, 涵盖数据加载、模型训练、验证评估、模型保存等全部环节。

4.7.1 训练流程概览



核心流程（对应 训练基础：登山纪律）：

1. **数据准备**：加载数据，划分训练/验证集，创建 DataLoader
2. **模型定义**：搭建网络架构
3. **训练循环**：前向传播 → 计算损失 → 反向传播 → 更新参数
4. **验证评估**：评估模型性能，检测过拟合
5. **模型保存**：保存最佳模型

4.7.2 数据准备

数据集与 DataLoader

训练基础：登山纪律中提到：**Batch Size** 是每次参数更新使用的样本数。PyTorch 用 `DataLoader` 实现这个机制。

```

import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# 数据预处理
# transforms.Compose 将多个预处理步骤串联
# MNIST 数据原始范围 [0, 255]，需要归一化到 [0, 1]
transform = transforms.Compose([
    transforms.ToTensor(),          # PIL Image → Tensor，自动除以 255
    transforms.Normalize((0.1307,), (0.3081,)) # 标准化: (x - mean) / std
])

# 加载 MNIST 数据集
# train=True 表示训练集，train=False 表示测试集
train_dataset = torchvision.datasets.MNIST(
    root='./data',          # 数据存储路径
    train=True,             # 训练集
    download=True,          # 自动下载
    transform=transform     # 应用预处理
)
  
```

(续下页)

(接上页)

```

test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,          # 测试集
    download=True,
    transform=transform
)

# 创建 DataLoader
# batch_size: 每批样本数 (对应 neural-training-basics 中的 Batch Size)
# shuffle=True: 每 epoch 打乱数据顺序, 防止模型记住顺序
# num_workers: 多进程加载数据, 加速训练
train_loader = DataLoader(
    train_dataset,
    batch_size=64,        # MNIST 有 60,000 样本, 每 epoch 约 938 个 batch
    shuffle=True,         # 打乱顺序
    num_workers=2         # 2 个子进程加载数据
)

test_loader = DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False,        # 测试集不需要打乱
    num_workers=2
)

print(f"训练集大小: {len(train_dataset)}") # 60,000
print(f"测试集大小: {len(test_dataset)}") # 10,000
print(f"每 epoch 迭代次数: {len(train_loader)}") # 938

```

数据增强 (可选)

训练基础: 登山纪律中的**数据增强**可以增加训练样本多样性:

```

# 训练时增强, 测试时不增强
train_transform = transforms.Compose([
    transforms.RandomRotation(10),      # 随机旋转 ±10 度
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)), # 随机平移
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

```

(续下页)

(接上页)

```
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

4.7.3 模型定义

CNN 架构实现

对应 卷积：发现装备中的卷积网络设计：

```
import torch.nn as nn
import torch.nn.functional as F

class MNISTNet(nn.Module):
    """
    MNIST 分类网络
    输入: [batch, 1, 28, 28]
    输出: [batch, 10] (10 个数字类别的 logits)
    """
    def __init__(self):
        super(MNISTNet, self).__init__()

        # C1: 卷积层, 1-32 通道, 3×3 卷积核, 输出 28×28
        # 参数量: 32×1×3×3 + 32 = 320
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)

        # C2: 卷积层, 32-64 通道, 3×3 卷积核, 输出 28×28
        # 参数量: 64×32×3×3 + 64 = 18,496
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)

        # 池化层: 2×2, 输出 14×14 → 7×7
        self.pool = nn.MaxPool2d(2)

        # 全连接层 1: 64×7×7=3136 → 128
        # 参数量: 3136×128 + 128 = 401,536
        self.fc1 = nn.Linear(64 * 7 * 7, 128)

        # 全连接层 2: 128 → 10 (类别数)
        # 参数量: 128×10 + 10 = 1,290
```

(续下页)

(接上页)

```

self.fc2 = nn.Linear(128, 10)

# Dropout: 防止过拟合 (见 neural-training-basics)
self.dropout = nn.Dropout(0.25) # 25% 的 dropout

def forward(self, x):
    # 卷积块 1: [batch, 1, 28, 28] → [batch, 32, 28, 28] → [batch, 32, 14, 14]
    x = self.conv1(x)
    x = F.relu(x)          # 激活函数
    x = self.pool(x)       # 降采样

    # 卷积块 2: [batch, 32, 14, 14] → [batch, 64, 14, 14] → [batch, 64, 7, 7]
    x = self.conv2(x)
    x = F.relu(x)
    x = self.pool(x)

    # 展平: [batch, 64, 7, 7] → [batch, 3136]
    x = x.view(x.size(0), -1)

    # 全连接 + Dropout
    x = self.dropout(x)
    x = self.fc1(x)
    x = F.relu(x)

    x = self.dropout(x)
    x = self.fc2(x) # 输出 logits (未归一化)

    return x

```

总参数量: $320 + 18,496 + 401,536 + 1,290 = 421,642$

4.7.4 训练函数

单 epoch 训练

对应 训练基础: 登山纪律中的训练循环:

```

def train_epoch(model, device, train_loader, optimizer, criterion, epoch):
    """
    训练一个 epoch

```

(续下页)

(接上页)

```

Args:
    model: 神经网络模型
    device: 计算设备 (CPU/GPU)
    train_loader: 训练数据加载器
    optimizer: 优化器
    criterion: 损失函数
    epoch: 当前 epoch 数

Returns:
    avg_loss: 平均损失
    accuracy: 准确率
"""
model.train() # 设置为训练模式 (启用 Dropout、BatchNorm 等)

train_loss = 0.0
correct = 0
total = 0

for batch_idx, (data, target) in enumerate(train_loader):
    # 1. 数据转移到设备
    data, target = data.to(device), target.to(device)

    # 2. 清零梯度 (关键! 见 auto-grad)
    optimizer.zero_grad()

    # 3. 前向传播
    output = model(data) # [batch, 10]

    # 4. 计算损失
    # CrossEntropyLoss 内部已经包含 Softmax
    loss = criterion(output, target)

    # 5. 反向传播
    loss.backward()

    # 6. 更新参数
    optimizer.step()

    # 7. 统计信息
    train_loss += loss.item()

```

(续下页)

(接上页)

```

_, predicted = output.max(1) # 取最大值的索引作为预测类别
total += target.size(0)
correct += predicted.eq(target).sum().item()

# 8. 进度显示
if batch_idx % 100 == 0:
    print(f'Epoch: {epoch} [{batch_idx * len(data)} / {len(train_loader.dataset)} '
          f'({100. * batch_idx / len(train_loader):.0f}%)]\t'
          f'Loss: {loss.item():.6f}')

# 计算平均指标
avg_loss = train_loss / len(train_loader)
accuracy = 100. * correct / total

return avg_loss, accuracy

```

训练模式的重要性:

- `model.train()`: 启用 Dropout (随机丢弃神经元)、BatchNorm 使用 batch 统计量
- `model.eval()`: 关闭 Dropout, BatchNorm 使用全局统计量

4.7.5 验证函数

评估模型性能

```

def validate(model, device, val_loader, criterion):
    """
    验证模型性能

    Args:
        model: 神经网络模型
        device: 计算设备
        val_loader: 验证数据加载器
        criterion: 损失函数

    Returns:
        avg_loss: 平均验证损失
        accuracy: 验证准确率
    """
    model.eval() # 设置为评估模式

```

(续下页)

(接上页)

```

val_loss = 0.0
correct = 0
total = 0

# 禁用梯度计算（节省内存，加速推理）
with torch.no_grad():
    for data, target in val_loader:
        data, target = data.to(device), target.to(device)

        # 前向传播
        output = model(data)
        loss = criterion(output, target)

        # 统计
        val_loss += loss.item()
        _, predicted = output.max(1)
        total += target.size(0)
        correct += predicted.eq(target).sum().item()

avg_loss = val_loss / len(val_loader)
accuracy = 100. * correct / total

return avg_loss, accuracy

```

4.7.6 主训练循环

完整训练流程

```

import torch
import torch.optim as optim
import matplotlib.pyplot as plt

def main():
    # ===== 1. 设备设置 =====
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"使用设备: {device}")

    # ===== 2. 数据准备 =====
    # ...（前面的数据加载代码）...

```

(续下页)

(接上页)

```

# ===== 3. 模型、损失、优化器 =====
model = MNISTNet().to(device)

# 损失函数: 交叉熵 (见 neural-training-basics)
criterion = nn.CrossEntropyLoss()

# 优化器: Adam (常用选择)
# weight_decay 对应 neural-training-basics 中的 L2 正则化
optimizer = optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4 # L2 正则化系数  $\lambda$ 
)

# 学习率调度器: 每 5 个 epoch 学习率乘以 0.1
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

# ===== 4. 训练历史记录 =====
# 用于检测过拟合 (见 neural-training-basics)
history = {
    'train_loss': [], 'train_acc': [],
    'val_loss': [], 'val_acc': []
}

best_acc = 0.0

# ===== 5. 训练循环 =====
num_epochs = 10

for epoch in range(1, num_epochs + 1):
    print(f"\n{' '*50}")
    print(f"Epoch {epoch}/{num_epochs}")
    print(f"{' '*50}")

    # 训练
    train_loss, train_acc = train_epoch(
        model, device, train_loader, optimizer, criterion, epoch
    )

    # 验证

```

(续下页)

(接上页)

```

val_loss, val_acc = validate(model, device, test_loader, criterion)

# 更新学习率
scheduler.step()
current_lr = optimizer.param_groups[0]['lr']

# 记录历史
history['train_loss'].append(train_loss)
history['train_acc'].append(train_acc)
history['val_loss'].append(val_loss)
history['val_acc'].append(val_acc)

# 打印结果
print(f'\n 训练集 - Loss: {train_loss:.4f}, Acc: {train_acc:.2f}%')
print(f' 验证集 - Loss: {val_loss:.4f}, Acc: {val_acc:.2f}%')
print(f' 学习率: {current_lr:.6f}')

# 保存最佳模型
if val_acc > best_acc:
    best_acc = val_acc
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'best_acc': best_acc,
    }, 'best_model.pth')
    print(f' 保存最佳模型, 准确率: {best_acc:.2f}%')

return history

```

4.7.7 可视化训练过程

绘制损失和准确率曲线

对应 训练基础：登山纪律中的过拟合检测：

```

def plot_history(history):
    """
    可视化训练历史
    用于检测过拟合：训练损失↓但验证损失↑时，说明过拟合
    """

```

(续下页)

(接上页)

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 损失曲线
axes[0].plot(history['train_loss'], label='Train Loss')
axes[0].plot(history['val_loss'], label='Val Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Loss Curve')
axes[0].legend()
axes[0].grid(True)

# 准确率曲线
axes[1].plot(history['train_acc'], label='Train Acc')
axes[1].plot(history['val_acc'], label='Val Acc')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy (%)')
axes[1].set_title('Accuracy Curve')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('training_history.png')
plt.show()

# 使用
# history = main()
# plot_history(history)

```

曲线解读：

- 理想情况：两条曲线同步下降/上升，差距小
- 过拟合迹象：训练准确率持续上升，验证准确率停滞或下降
- 欠拟合迹象：两条曲线都停滞在较低水平

4.7.8 模型加载与推理

加载保存的模型

```

def load_and_predict(model_path, image):
    """

```

(续下页)

(接上页)

加载模型并进行预测

Args:

model_path: 模型文件路径

image: 输入图像 tensor, 形状 [1, 1, 28, 28]

Returns:

prediction: 预测的类别 (0-9)

probability: 预测概率

"""

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

创建模型

model = MNISTNet().to(device)

加载权重

checkpoint = torch.load(model_path)

model.load_state_dict(checkpoint['model_state_dict'])

推理

model.eval()

with torch.no_grad():

image = image.to(device)

output = model(image)

Softmax 获取概率

probabilities = F.softmax(output, dim=1)

预测结果

prediction = output.argmax(dim=1).item()

confidence = probabilities.max().item()

return prediction, confidence

使用示例

image = test_dataset[0][0].unsqueeze(0) # 取第一张测试图片, 加 batch 维度

pred, conf = load_and_predict('best_model.pth', image)

print(f"预测结果: {pred}, 置信度: {conf:.2%}")

4.7.9 总结

训练流程回顾

步骤	关键代码	对应理论
数据加载	<code>DataLoader</code>	训练基础：登山纪律 中的 Batch
模型定义	<code>nn.Module</code>	卷积：发现装备
损失函数	<code>CrossEntropyLoss</code>	损失函数
优化器	<code>optim.Adam</code>	梯度下降与优化算法
反向传播	<code>loss.backward()</code>	反向传播算法
正则化	<code>weight_decay, Dropout</code>	训练基础：登山纪律
验证评估	<code>model.eval()</code>	检测过拟合
学习率调度	<code>lr_scheduler</code>	训练后期微调

完整训练脚本模板

```
# main.py - 可运行的完整训练脚本
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms

def main():
    # 配置
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    batch_size = 64
    epochs = 10
    lr = 0.001

    # 数据
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])
    train_loader = torch.utils.data.DataLoader(
        datasets.MNIST('./data', train=True, download=True, transform=transform),
        batch_size=batch_size, shuffle=True
    )
    test_loader = torch.utils.data.DataLoader(
        datasets.MNIST('./data', train=False, transform=transform),
        batch_size=batch_size
```

(续下页)

(接上页)

```
)

# 模型
model = MNISTNet().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)

# 训练
for epoch in range(1, epochs + 1):
    train_epoch(model, device, train_loader, optimizer, criterion, epoch)
    validate(model, device, test_loader, criterion)

# 保存
torch.save(model.state_dict(), 'mnist_model.pth')

if __name__ == '__main__':
    main()
```

从手写到工程框架

上面这 200 行代码包含了训练 MNIST 分类器的全部要素。但在真实项目中，每次从头写训练循环、手动管理 checkpoint、手工对比实验是不现实的。

社团的 **mnist-helloworld** 框架把这些重复劳动封装成了可复用模块。你刚学到的每个手写环节，在框架中都有对应的工程化模块：

手动实现（你刚写的）	框架模块	功能
DataLoader + transform	src/datasets/mnist.py	数据集封装，自动下载与预处理
class MNISTNet(nn.Module)	src/models/	BaseModel 抽象，统一接口
for epoch 训练循环	src/training/trainer.py	Trainer 类，封装完整训练逻辑
手动保存 checkpoint	src/training/checkpoint.py	CheckpointManager，支持断点续训
手动记录实验日志	src/training/experiment.py	ExperimentManager，YOLO 风格 runs/expN/
手写 @dataclass Config	src/config/config.py	YAML + CLI 双配置模式

```
# 一行命令跑通默认训练 (MNIST + MyNet)
python train.py

# 更换模型和数据集
python train.py --model lenet --dataset mnist --training.epochs 30
```

(续下页)

(接上页)

使用配置文件

```
python train.py --config my_experiment.yaml
```

框架采用 YOLO 风格自动管理实验目录，每次运行创建 `runs/exp1/`、`runs/exp2/` ... 自动保存配置、checkpoint 和训练曲线，不再担心覆盖实验结果。

手写是为了理解原理，框架是为了提升效率——先理解，再用工具，这是正确的学习路径。

完整的框架使用指南见[使用训练框架](#)，包括模型管理：从 `nn.Module` 到 `BaseModel`、数据集管理：从 `DataLoader` 到 `BaseDataset`、实验管理：从手动记录到自动追踪、配置系统：从硬编码到配置文件等全部功能详解。

下一步

掌握了完整训练流程后，下一节[调试与可视化技巧](#)我们将学习如何调试训练过程中的常见问题，以及如何使用 TensorBoard 等工具可视化训练过程，让“黑盒”训练变得透明可控。

从“能训练”到“会调试”，让我们成为真正的深度学习工程师！

4.8 调试与可视化技巧

在完整训练流程中，我们搭建了一个完整的训练流程。但当你开始训练自己的模型时，可能会遇到各种问题：损失不下降、准确率上不去、模型不收敛...

本章就是深度学习开发的“急救手册”——教你诊断问题、可视化训练过程、定位 bug。

4.8.1 为什么需要调试技巧？

深度学习代码有几个特点让调试变得困难：

1. **错误延迟暴露**：数据预处理的问题可能在训练 10 个 epoch 后才显现
2. **梯度流动不可见**：神经网络是黑盒，中间状态难以观察
3. **数值问题隐蔽**：梯度爆炸/消失不会报错，只会让模型学不好
4. **随机性干扰**：同样的代码运行多次结果可能不同

好消息：PyTorch 提供了丰富的工具帮我们“透视”训练过程。

4.8.2 梯度检查：诊断训练的“生命线”

梯度范数监控

梯度下降与优化算法告诉我们，梯度是参数更新的基础。如果梯度有问题，训练必然失败。

```
def check_gradients(model, print_details=False):
    """
    检查模型梯度状态

    参数:
        model: PyTorch 模型
        print_details: 是否打印每层梯度详情

    返回:
        total_norm: 全局梯度范数
    """
    total_norm = 0.0
    grad_stats = {}

    for name, param in model.named_parameters():
        if param.grad is not None:
            # 计算该层梯度的 L2 范数
            param_norm = param.grad.data.norm(2)
            total_norm += param_norm.item() ** 2

            # 收集统计信息
            grad_stats[name] = {
                'mean': param.grad.mean().item(),
                'std': param.grad.std().item(),
                'max': param.grad.max().item(),
                'min': param.grad.min().item(),
            }

            # 检查数值异常
            if torch.isnan(param.grad).any():
                print(f"✗ 错误: {name} 包含 NaN 梯度! ")
                print(f"   该层参数形状: {param.shape}")
                print(f"   建议: 检查前向传播是否有除以 0 或 log(0)")

            if torch.isinf(param.grad).any():
                print(f"✗ 错误: {name} 包含 Inf 梯度! ")
                print(f"   建议: 使用梯度裁剪 (gradient clipping)")

    total_norm = total_norm ** 0.5
```

(续下页)

(接上页)

```

# 判断梯度状态
if total_norm < 1e-6:
    print(f"⚠ 警告: 梯度范数过小 ({total_norm:.2e}), 可能出现梯度消失")
    print(f"    建议: 检查激活函数、网络深度, 或使用残差连接")
elif total_norm > 100:
    print(f"⚠ 警告: 梯度范数过大 ({total_norm:.2f}), 可能出现梯度爆炸")
    print(f"    建议: 使用梯度裁剪或减小学习率")
else:
    print(f"✅ 梯度范数正常: {total_norm:.4f}")

if print_details:
    print("\n 各层梯度统计:")
    for name, stats in grad_stats.items():
        print(f"    {name}: mean={stats['mean']:.4f}, std={stats['std']:.4f}")

return total_norm

```

使用场景: 在每个 epoch 或每隔几个 batch 调用, 监控梯度健康状态。

梯度爆炸与消失的检测

反向传播算法中提到梯度消失/爆炸是深层网络的常见问题。如何在代码中检测?

```

class GradientMonitor:
    """梯度监控器: 记录训练过程中的梯度变化"""

    def __init__(self, model):
        self.model = model
        self.history = [] # 记录每个 step 的梯度信息

        # 注册 hook, 捕获每层的梯度
        self.hooks = []
        for name, param in model.named_parameters():
            if param.requires_grad:
                hook = param.register_hook(
                    lambda grad, name=name: self._log_gradient(grad, name)
                )
                self.hooks.append(hook)

    def _log_gradient(self, grad, name):
        """记录梯度信息"""

```

(续下页)

(接上页)

```

if grad is not None:
    self.history.append({
        'name': name,
        'step': len(self.history),
        'mean': grad.abs().mean().item(),
        'max': grad.abs().max().item(),
    })

def plot_gradient_flow(self):
    """可视化梯度在各层的流动"""
    import matplotlib.pyplot as plt

    # 按层分组计算平均梯度
    layer_grads = {}
    for record in self.history:
        layer = record['name'].split('.')[0] # 提取层名
        if layer not in layer_grads:
            layer_grads[layer] = []
        layer_grads[layer].append(record['mean'])

    # 绘图
    plt.figure(figsize=(12, 6))
    for layer, grads in layer_grads.items():
        plt.plot(grads, label=layer, alpha=0.7)

    plt.xlabel('Training Step')
    plt.ylabel('Mean Gradient')
    plt.title('Gradient Flow Across Layers')
    plt.legend()
    plt.yscale('log') # 对数尺度更容易看出差异
    plt.grid(True, alpha=0.3)
    plt.show()

    # 诊断建议
    first_layer_grad = list(layer_grads.values())[0]
    last_layer_grad = list(layer_grads.values())[-1]

    ratio = np.mean(first_layer_grad) / (np.mean(last_layer_grad) + 1e-10)

    if ratio > 100:

```

(续下页)

(接上页)

```

print(f"⚠ 检测到梯度消失：第一层梯度是最后一层的 {ratio:.1f} 倍")
print(f"    建议：使用 BatchNorm、残差连接或更小的学习率")
elif ratio < 0.01:
    print(f"⚠ 检测到梯度爆炸：最后一层梯度是第一层的 {1/ratio:.1f} 倍")
    print(f"    建议：使用梯度裁剪或权重衰减")

def remove_hooks(self):
    """移除所有 hook，释放内存"""
    for hook in self.hooks:
        hook.remove()

```

核心洞察：通过可视化梯度在各层的流动，可以快速判断是否存在梯度消失/爆炸问题。

4.8.3 训练过程可视化：TensorBoard

TensorBoard 是 TensorFlow 团队开发的可视化工具，但 PyTorch 也完全支持。它能让我们看到训练过程中发生的一切。

基础设置

```

from torch.utils.tensorboard import SummaryWriter
import datetime

# 创建 writer，日志目录包含时间戳，方便区分不同实验
log_dir = f'runs/mnist_experiment_{datetime.datetime.now().strftime("%Y%m%d-%H%M%S")}'
writer = SummaryWriter(log_dir)

print(f"TensorBoard 日志目录: {log_dir}")
print(f"启动 TensorBoard: tensorboard --logdir={log_dir}")

```

记录训练指标

```

def train_with_logging(model, device, train_loader, optimizer, epoch):
    """带 TensorBoard 日志记录的训练函数"""
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for batch_idx, (data, target) in enumerate(train_loader):

```

(续下页)

(接上页)

```

data, target = data.to(device), target.to(device)

optimizer.zero_grad()
output = model(data)
loss = criterion(output, target)
loss.backward()
optimizer.step()

# 累计统计
running_loss += loss.item()
_, predicted = output.max(1)
total += target.size(0)
correct += predicted.eq(target).sum().item()

# 每 100 个 batch 记录一次
if batch_idx % 100 == 99:
    step = epoch * len(train_loader) + batch_idx

    # 记录损失和准确率
    writer.add_scalar('Training/Loss', running_loss / 100, step)
    writer.add_scalar('Training/Accuracy', 100. * correct / total, step)

    # 记录学习率
    current_lr = optimizer.param_groups[0]['lr']
    writer.add_scalar('Training/Learning_Rate', current_lr, step)

    running_loss = 0.0
    correct = 0
    total = 0

```

记录模型结构和参数分布

```

# 记录模型计算图
sample_data = next(iter(train_loader))[0].to(device)
writer.add_graph(model, sample_data)

# 每 5 个 epoch 记录参数和梯度的直方图
def log_histograms(model, epoch):
    """记录参数分布, 帮助诊断问题"""
    for name, param in model.named_parameters():

```

(续下页)

(接上页)

```

# 记录参数分布
writer.add_histogram(f'Parameters/{name}', param, epoch)

# 记录梯度分布
if param.grad is not None:
    writer.add_histogram(f'Gradients/{name}', param.grad, epoch)

# 计算并记录梯度/参数比值（重要诊断指标）
ratio = (param.grad.abs() / (param.abs() + 1e-8)).mean()
writer.add_scalar(f'GradParamRatio/{name}', ratio, epoch)

```

梯度/参数比值的意义：

- 比值太小 (<0.001)：学习率可能太小，参数更新缓慢
- 比值适中 (0.001~0.1)：健康状态
- 比值太大 (>1)：学习率可能太大，参数震荡

可视化训练数据

```

def visualize_data(writer, train_loader, num_images=64):
    """可视化训练数据，检查预处理是否正确"""
    images, labels = next(iter(train_loader))

    # 创建网格图像
    img_grid = torchvision.utils.make_grid(
        images[:num_images],
        nrow=8,
        normalize=True,
        value_range=(0, 1)
    )

    writer.add_image('Training_Data', img_grid, 0)

    # 记录类别分布
    class_counts = torch.bincount(labels, minlength=10)
    for i, count in enumerate(class_counts):
        writer.add_scalar(f'Class_Distribution/class_{i}', count, 0)

```

嵌入可视化（高维数据投影）

```
def visualize_embeddings(model, device, test_loader, writer):
    """
    可视化模型学到的特征表示
    将高维特征投影到 3D 空间，看不同类别是否分开
    """
    model.eval()
    embeddings = []
    labels = []
    images_list = []

    with torch.no_grad():
        for data, target in test_loader:
            data = data.to(device)
            # 提取倒数第二层的特征
            features = model.features(data) # 假设模型有 features 方法
            embeddings.append(features.cpu())
            labels.append(target)
            images_list.append(data.cpu())

            if len(embeddings) >= 100: # 只取 100 个 batch
                break

    embeddings = torch.cat(embeddings, dim=0)
    labels = torch.cat(labels, dim=0)
    images_list = torch.cat(images_list, dim=0)

    writer.add_embedding(
        embeddings[:1000], # 最多 1000 个点
        metadata=labels[:1000].tolist(),
        label_img=images_list[:1000],
        global_step=0,
        tag='Feature_Embeddings'
    )
```

4.8.4 常见训练问题诊断

问题 1: 损失不下降

症状: 训练了多个 epoch, 损失几乎不变。

```
def diagnose_no_decrease_loss(model, loader, device):
    """诊断损失不下降的原因"""

    print("=" * 50)
    print("🔍 诊断损失不下降...")
    print("=" * 50)

    # 1. 检查数据
    data, target = next(iter(loader))
    print(f"\n1. 数据检查:")
    print(f"    输入范围: [{data.min():.2f}, {data.max():.2f}]")
    print(f"    标签唯一值: {target.unique()}")

    if data.max() > 10:
        print("    ⚠️ 警告: 输入未归一化, 建议除以 255 或标准化")

    # 2. 检查模型输出
    model.eval()
    with torch.no_grad():
        output = model(data.to(device))
        print(f"\n2. 模型输出检查:")
        print(f"    输出形状: {output.shape}")
        print(f"    输出范围: [{output.min():.2f}, {output.max():.2f}]")

        if torch.allclose(output, output[0]):
            print("    ❌ 错误: 所有输出相同, 模型未学习!")
            print("    可能原因: 权重初始化问题或学习率太小")

    # 3. 检查梯度
    model.train()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
    output = model(data.to(device))
    loss = nn.CrossEntropyLoss()(output, target.to(device))
    loss.backward()

    print(f"\n3. 梯度检查:")
```

(续下页)

(接上页)

```

has_grad = False
for name, param in model.named_parameters():
    if param.grad is not None and param.grad.abs().sum() > 0:
        has_grad = True
        print(f"    ✓ {name}: 梯度范数={param.grad.norm():.4f}")
    elif param.grad is None:
        print(f"    ⚠ {name}: 无梯度! ")

if not has_grad:
    print("    ✗ 所有参数都没有梯度! ")
    print("    检查: forward 中是否使用了 requires_grad=False 的输入")

# 4. 学习率检查
print(f"\n4. 学习率检查:")
test_lr = [1e-5, 1e-3, 1e-1]
for lr in test_lr:
    print(f"    学习率={lr}: ", end="")
    # 简化的参数更新测试
    param = list(model.parameters())[0].clone()
    grad = list(model.parameters())[0].grad
    if grad is not None:
        param_update = (grad * lr).abs().mean()
        print(f"    平均更新量={param_update:.2e}")

```

常见原因和解决方案:

原因	检查方法	解决方案
学习率太小	观察参数更新量	增大到 0.001~0.01
梯度消失	检查深层梯度范数	加 BatchNorm、换 ReLU
数据未归一化	检查输入范围	标准化到 [0,1] 或 [-1,1]
标签错误	检查 target 值	确保标签从 0 开始
权重初始化问题	检查初始输出	使用 Xavier/He 初始化

问题 2: 过拟合

症状: 训练准确率很高, 验证准确率很低。

```

def diagnose_overfitting(train_acc_history, val_acc_history):
    """诊断过拟合程度"""

```

(续下页)

(接上页)

```

train_acc = np.array(train_acc_history)
val_acc = np.array(val_acc_history)

# 计算 gap
gap = train_acc - val_acc

print(f"\n📋 过拟合诊断报告:")
print(f"    最终训练准确率: {train_acc[-1]:.2f}%")
print(f"    最终验证准确率: {val_acc[-1]:.2f}%")
print(f"    准确率差距: {gap[-1]:.2f}%")

if gap[-1] < 3:
    print(f"    ✅ 状态良好, 无明显过拟合")
elif gap[-1] < 10:
    print(f"    ⚠️ 轻度过拟合, 建议增加正则化")
    print(f"        - 添加 Dropout (rate=0.2~0.5)")
    print(f"        - 增加 weight_decay (1e-4~1e-3)")
else:
    print(f"    ❌ 严重过拟合, 必须采取措施! ")
    print(f"        - 收集更多训练数据")
    print(f"        - 使用更强的数据增强")
    print(f"        - 减小模型容量")

# 检查验证集准确率是否下降
if len(val_acc) > 5:
    recent_trend = val_acc[-1] - val_acc[-5]
    if recent_trend < -2:
        print(f"    ⚠️ 验证准确率下降 {abs(recent_trend):.1f}%, 建议早停")

```

参考: 正则化: 防止死记硬背的四种方法中详细讨论了各种正则化技术。

问题 3: 学习率不当

```

def find_lr(model, train_loader, optimizer, criterion, device,
            start_lr=1e-7, end_lr=10, num_iter=100):
    """
    学习率范围测试 (Learning Rate Range Test)
    参考: Cyclical Learning Rates for Training Neural Networks
    """

```

(续下页)

(接上页)

```

model.train()
lrs = []
losses = []

# 指数增长学习率
lr_mult = (end_lr / start_lr) ** (1 / num_iter)
lr = start_lr

for param_group in optimizer.param_groups:
    param_group['lr'] = lr

for batch_idx, (data, target) in enumerate(train_loader):
    if batch_idx >= num_iter:
        break

    data, target = data.to(device), target.to(device)

    optimizer.zero_grad()
    output = model(data)
    loss = criterion(output, target)
    loss.backward()
    optimizer.step()

    lrs.append(lr)
    losses.append(loss.item())

# 更新学习率
lr *= lr_mult
for param_group in optimizer.param_groups:
    param_group['lr'] = lr

# 绘图
plt.figure(figsize=(10, 6))
plt.plot(lrs, losses)
plt.xscale('log')
plt.xlabel('Learning Rate')
plt.ylabel('Loss')
plt.title('Learning Rate Range Test')
plt.axvline(x=lrs[np.argmin(losses)], color='r', linestyle='--',
            label=f'Min Loss @ lr={lrs[np.argmin(losses)].2e}')

```

(续下页)

(接上页)

```
plt.legend()
plt.show()

# 建议最优学习率
min_idx = np.argmin(losses)
suggested_lr = lrs[max(0, min_idx - 1)] # 取最小损失前一个点
print(f"建议学习率: {suggested_lr:.2e}")

return suggested_lr
```

4.8.5 下一步

掌握了调试和可视化技巧后，你已经具备了独立训练神经网络的能力。

工程最佳实践中，我们将学习工程最佳实践——如何让代码更规范、更易复现、更易于协作。这些技巧来自于操控效率：效果、速度与硬件的三角权衡中的效率优化思想，是走向专业深度学习开发的必经之路。

4.9 工程最佳实践

调试与可视化技巧教会我们如何诊断和修复训练中的问题。但要成为专业的深度学习开发者，还需要掌握工程实践——如何组织代码、优化性能、保证实验可复现。

这些实践来自于工业界和学术界多年积累的经验，参考了操控效率：效果、速度与硬件的三角权衡中的效率优化思想。

4.9.1 项目结构组织

一个良好组织的项目结构能让代码更易维护、更易协作。

推荐目录结构

```
my_project/
├─ configs/           # 配置文件
│   ├─ base.yaml
│   ├─ experiment1.yaml
│   └─ experiment2.yaml
├─ data/              # 数据处理（不包含原始数据）
│   ├─ __init__.py
│   └─ datasets.py    # 自定义 Dataset
```

(续下页)

(接上页)

```
|   └─ transforms.py      # 数据增强
├─ models/                # 模型定义
|   └─ __init__.py
|   └─ layers.py          # 自定义层
|   └─ mnist_net.py       # 具体模型
├─ utils/                 # 工具函数
|   └─ __init__.py
|   └─ metrics.py         # 评估指标
|   └─ logger.py          # 日志记录
|   └─ visualization.py   # 可视化工具
├─ checkpoints/           # 模型检查点
├─ logs/                  # 训练日志
├─ notebooks/             # Jupyter 笔记本（探索性分析）
├─ tests/                 # 单元测试
├─ train.py               # 训练入口
├─ eval.py                # 评估入口
├─ config.py              # 配置管理
└─ requirements.txt       # 依赖列表
```

参考实现：社团的mnist-helloworld框架采用了与此一致的项目结构。详见[模型管理：从nn.Module到BaseModel](#)和[数据集管理：从DataLoader到BaseDataset](#)。

为什么这样组织？

目录	作用	与拆开看看：CNN 消融研究的联系
configs/	集中管理超参数	方便进行拆开看看：CNN 消融研究中的系统实验
models/	模块化网络结构	支持快速迭代不同架构
utils/	复用工具代码	减少重复劳动，提高效率
checkpoints/	保存训练状态	支持断点续训和模型选择

4.9.2 配置管理

集中式配置

不要硬编码参数，使用配置文件：

```
# config.py: 集中管理所有配置
from dataclasses import dataclass
from typing import Optional
```

(续下页)

(接上页)

```
@dataclass
class Config:
    """训练配置"""
    # 数据配置
    data_dir: str = "./data"
    batch_size: int = 64
    num_workers: int = 4

    # 模型配置
    model_name: str = "MNISTNet"
    hidden_dim: int = 128
    dropout_rate: float = 0.5

    # 训练配置
    num_epochs: int = 10
    learning_rate: float = 0.001
    weight_decay: float = 1e-4

    # 优化器配置（对应{ref}`gradient-descent`中的算法）
    optimizer: str = "Adam" # "SGD", "Adam", "AdamW"
    momentum: float = 0.9 # 仅用于 SGD

    # 学习率调度（对应{ref}`learning-rate-scheduling`）
    scheduler: str = "StepLR" # "StepLR", "CosineAnnealingLR"
    step_size: int = 5
    gamma: float = 0.1

    # 系统配置
    device: str = "cuda" if torch.cuda.is_available() else "cpu"
    seed: int = 42 # 保证可复现

    # 日志配置
    log_dir: str = "./logs"
    save_dir: str = "./checkpoints"
    log_interval: int = 100 # 每 100 个 batch 记录一次

    # 使用示例
    config = Config(batch_size=128, learning_rate=0.0005)
    print(f"使用设备: {config.device}")
```

(续下页)

(接上页)

```
print(f"学习率: {config.learning_rate}")
```

好处:

1. **可复现**: 所有参数在一个地方, 方便记录和分享
2. **易修改**: 改配置不用改代码
3. **支持实验**: 可以快速创建不同的配置进行对照实验

使用 YAML 配置文件

对于复杂项目, 使用 YAML 管理配置:

```
# configs/experiment.yaml
data:
  batch_size: 64
  num_workers: 4
  augmentation:
    random_crop: true
    horizontal_flip: true

model:
  name: "ResNet18"
  pretrained: true
  num_classes: 10

training:
  epochs: 100
  optimizer:
    name: "AdamW"
    lr: 0.001
    weight_decay: 0.01
  scheduler:
    name: "CosineAnnealingLR"
    T_max: 100
```

```
# 加载 YAML 配置
import yaml

def load_config(config_path):
    with open(config_path, 'r') as f:
```

(续下页)

(接上页)

```

    config = yaml.safe_load(f)
    return config

# 使用
config = load_config('configs/experiment.yaml')
print(config['training']['optimizer']['name']) # "AdamW"

```

参考实现：mnist-helloworld 的配置系统支持 YAML + CLI 双配置模式 —— config.yaml 定义默认值，命令行参数覆盖实验变量。详见配置系统：[从硬编码到配置文件](#)。

4.9.3 可复现性保证

拆开看看：CNN 消融研究中的实验需要严格的可复现性。如何做到？

固定随机种子

```

def set_seed(seed=42):
    """
    固定所有随机种子，保证实验可复现

    对应 scaling law 中控制变量的思想
    """
    import random
    import numpy as np
    import torch

    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed) # 如果使用多 GPU

    # 让 cuDNN 确定性运行
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    print(f"🎲 随机种子已设置为: {seed}")

# 在训练开始时调用
set_seed(42)

```

深入理解 PyTorch 的确定性行为

上面的 `set_seed` 函数设置了 `torch.backends.cudnn.deterministic = True` 和 `torch.backends.cudnn.benchmark = False`。但你可能好奇：PyTorch 默认为什么不保证确定性的结果？打开确定性会有什么代价？

为什么 PyTorch 默认不确定？

根源来自两个层面的叠加：浮点数的数学特性 $\times$ GPU 的并行执行方式。

第一层：浮点数为什么不满足结合律？

先看一个简单的例子：

```
>>> 1e20 + (-1e20) + 1.0
1.0

>>> 1e20 + ((-1e20) + 1.0)
0.0 # !?
```

第一个式子 $(10^{20} - 10^{20}) + 1 = 0 + 1 = 1$ ，结果正确。

第二个式子 $10^{20} + (-10^{20} + 1)$ 在数学上也等于 1，但计算机的浮点数精度有限 —— $-1e20 + 1.0$ 的结果仍然是 $-1e20$ （因为 1.0 相比 $1e20$ 太小，被“吃掉”了），所以最终结果是 $10^{20} - 10^{20} = 0$ 。

精度有限 不等于 精度可控

并不是浮点数运算“容易出错”，而是误差累积对计算顺序高度敏感。 $(a+b)+c$ 和 $a+(b+c)$ 在整数运算中永远相等，但在浮点数中可能不同 —— 这个差异在每次加法中只有最后几位，但一个卷积核要做 $K^2 \times C_{in}$ 次乘加，再乘以几十万个位置，微小差异被反复放大。

第二层：GPU 并行如何放大这个问题？

GPU 的计算模式是 **SIMT**（Single Instruction, Multiple Threads）—— 数万个 CUDA 核心同时执行同一指令，但处理不同的数据。问题在于：**并行计算不保证归并顺序**。

看一个具体的数字例子 —— 同样四个数 $(10^8, 1, -10^8, 1)$ ，不同的归约顺序产生不同的结果。这种差异在矩阵乘法（卷积、全连接、注意力机制都靠它）中会被反复放大：

为什么路径 A 和路径 B 的结果不同？

路径 A: 先做 $10^8 + 1$ ： 10^8 远大于 1，在 float32 精度下 1 被舍入掉了（相当于 $\approx 10^8$ ），另一侧 $-10^8 + 1 \approx -10^8$ ，最终 $10^8 + (-10^8) = 0$ 。

路径 B: 先做 $10^8 + (-10^8) = 0$ （大数先抵消），另一侧 $1 + 1 = 2$ ，最终 $0 + 2 = 2$ 。

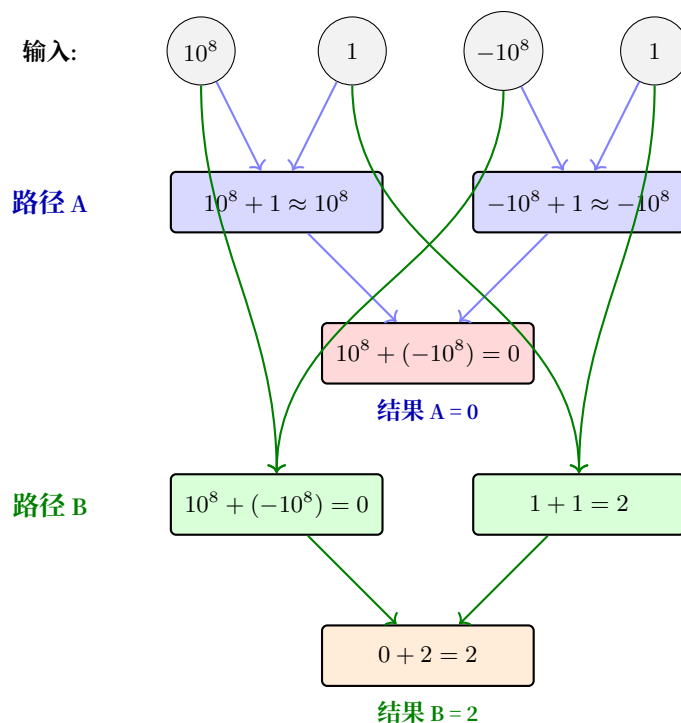


图 8: GPU 并行归并的不确定性 (一个夸张的例子)

因此在这种体系下:

$$(a + b) + (c + d) \neq (a + c) + (b + d)$$

归并顺序不同, 最终结果在最低有效位上产生差异。这个差异在单个操作中微不足道, 但卷积核要做 $K^2 \times C_{in}$ 次乘加, 再乘以几十万位置——微小差异被反复放大。

数百个 CUDA 核心同时计算矩阵乘法的不同分块, 每个分块内部的累加顺序取决于**线程调度时机**——GPU 给哪个线程组分配哪个计算单元, 每次运行都可能不同。这导致了三个层面的不确定性源:

不确定性源	解释	影响程度
原子操作顺序	多个线程同时写同一个内存地址时, 写入选顺序不定	小 (仅最后几位)
cuDNN 算法选择	cuDNN 默认会 benchmark 最快算法, 但不同算法产生结果有微小差异	中 (不同算法间差异可能更大)
浮点数累加顺序	矩阵乘法中的部分和以不同顺序相加 (树形 vs 串行)	大 (随层数加深累积)

一句话总结: 每次运行, GPU 线程的调度顺序都不同, 浮点数对不同顺序敏感——所以同样的代码跑两次, 结果在最后几位上不一样。这不是 bug, 而是并行浮点计算的**物理常态**。

PyTorch 中的三个确定性开关

开关	默认值	作用	影响
<code>torch.backends.cudnn.benchmark = False</code>	True	cuDNN 不再自动搜索最快算法, 而是使用固定算法	训练速度可能下降 10~50%
<code>torch.backends.cudnn.deterministic = True</code>	False	cuDNN 使用确定性算法 (避免原子操作的不确定性)	部分操作 (如 <code>scatter_add</code>) 可能变慢
<code>torch.use_deterministic_algorithms(True)</code>	False	让更多 PyTorch 操作 (如索引、插值) 使用确定性实现	部分操作不受支持时会抛异常

```
import torch

# 设置前两个开关 (和 set_seed 中一样)
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True

# 可选: 更严格的确定性模式 (PyTorch 1.7+)
# 开启后, 不支持确定性实现的操作会直接报错, 而不是静默运行
# torch.use_deterministic_algorithms(True)
```

什么时候该开, 什么时候不该开?

场景	推荐	原因
论文实验 / 学术研究	☑ 开启确定性	实验必须可复现, 这是学术底线
调试代码 bug	☑ 开启	每次运行结果不同, 无法定位 bug
写单元测试	☑ 开启	测试断言需要固定输出
日常模型训练	✗ 不开	1% 的随机波动不影响最终收敛, 但 30% 的速度减慢实实在在
生产环境推理	✗ 不开	推理速度比毫秒级一致性更重要
大规模分布式训练	✗ 不开	分布式同步本身就是随机的, 开确定性没用

从不确定性中学到的启示

深度学习的训练过程本身就是一个**随机过程**: 我们依赖随机梯度下降 (SGD)、随机初始化、随机打乱数据来探索参数空间。确定性模式只是保证这个随机过程的每次运行完全一致, 而不是消除随机性本身。

换句话说: **确定性 ≠ 模型更好, 确定性 = 同样的随机种子跑出同样的结果。**

验证是否真的确定

写一个简单测试来验证你的实验是否真正可复现：

```
def test_reproducibility():
    """验证模型在相同输入下是否产生完全相同的结果"""
    import torch
    import torch.nn as nn

    # 固定所有随机源
    torch.manual_seed(42)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    # 创建模型并运行
    model = nn.Linear(10, 5)
    x = torch.randn(1, 10)

    output_1 = model(x)

    # 重置种子，完全重复
    torch.manual_seed(42)
    model = nn.Linear(10, 5) # 重新初始化
    x = torch.randn(1, 10)

    output_2 = model(x)

    # 断言完全相等（不只是近似）
    assert torch.equal(output_1, output_2), "❌ 实验不可复现！"
    print("✅ 实验可复现")
```

如果你通过了这个测试，你的实验就可以放心地写入论文或提交给合作者了。

实验记录

```
import json
import os
from datetime import datetime

class ExperimentLogger:
    """实验记录器：保存所有实验信息"""
```

(续下页)

(接上页)

```

def __init__(self, config, experiment_name=None):
    self.timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    self.experiment_name = experiment_name or f"exp_{self.timestamp}"

    # 创建实验目录
    self.exp_dir = os.path.join("experiments", self.experiment_name)
    os.makedirs(self.exp_dir, exist_ok=True)
    os.makedirs(os.path.join(self.exp_dir, "checkpoints"), exist_ok=True)

    # 保存配置
    self.config_path = os.path.join(self.exp_dir, "config.json")
    with open(self.config_path, 'w') as f:
        json.dump(vars(config), f, indent=2)

    # 保存代码版本 (如果使用 git)
    self._save_git_info()

    print(f"📁 实验目录: {self.exp_dir}")

def _save_git_info(self):
    """记录 git 版本信息"""
    import subprocess
    try:
        git_hash = subprocess.check_output(['git', 'rev-parse', 'HEAD']).decode().strip()
        git_branch = subprocess.check_output(['git', 'rev-parse', '--abbrev-ref', 'HEAD']).
        ↪ decode().strip()

        git_info = {
            'hash': git_hash,
            'branch': git_branch,
            'timestamp': self.timestamp
        }

        with open(os.path.join(self.exp_dir, "git_info.json"), 'w') as f:
            json.dump(git_info, f, indent=2)
    except:
        print("⚠ 无法获取 git 信息")

def save_metrics(self, metrics):
    """保存训练指标"""

```

(续下页)

(接上页)

```

metrics_path = os.path.join(self.exp_dir, "metrics.json")
with open(metrics_path, 'w') as f:
    json.dump(metrics, f, indent=2)

def get_checkpoint_path(self, epoch):
    """获取检查点保存路径"""
    return os.path.join(self.exp_dir, "checkpoints", f"epoch_{epoch}.pt")

```

参考实现：mnist-helloworld 的 ExperimentManager 实现了类似功能，并增加了 YOLO 风格自动实验编号（runs/exp1、runs/exp2...）和断点续训支持。详见实验管理：[从手动记录到自动追踪](#)。

4.9.4 性能优化

操控效率：效果、速度与硬件的三角权衡 告诉我们，计算效率直接影响实验迭代速度。

数据加载优化

```

# 优化前（慢）
train_loader = DataLoader(dataset, batch_size=64, shuffle=True)

# 优化后（快）
train_loader = DataLoader(
    dataset,
    batch_size=64,
    shuffle=True,
    num_workers=4,          # 多进程加载数据，建议设为 CPU 核心数
    pin_memory=True,        # 将数据固定在页锁定内存，加速 CPU 到 GPU 传输
    persistent_workers=True, # 保持 worker 进程存活，避免每次 epoch 重新创建
    prefetch_factor=2       # 每个 worker 预取 2 个 batch
)

```

性能对比：

配置	训练时间（每 epoch）	说明
num_workers=0	120s	主进程加载，成为瓶颈
num_workers=4, pin_memory=False	80s	并行加载但传输慢
num_workers=4, pin_memory=True	45s	最优配置

混合精度训练

现代 GPU（V100 及以上）支持 Tensor Cores，使用混合精度可以加速 2-3 倍：

```
from torch.cuda.amp import autocast, GradScaler

# 初始化梯度缩放器（处理梯度下溢）
scaler = GradScaler()

for data, target in train_loader:
    data, target = data.to(device), target.to(device)
    optimizer.zero_grad()

    # 使用 autocast 自动选择精度（FP16/FP32）
    with autocast():
        output = model(data)
        loss = criterion(output, target)

    # 缩放梯度避免下溢，然后反向传播
    scaler.scale(loss).backward()

    # 梯度裁剪（如果需要）
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

    # 更新参数
    scaler.step(optimizer)
    scaler.update()
```

原理：

- FP16（半精度）：占内存少、计算快，但范围小容易溢出
- FP32（单精度）：范围大、精度高，但计算慢
- **混合精度**：前向/反向用 FP16，参数更新用 FP32，兼顾速度和稳定性

梯度累积

当 GPU 内存不足以支持大批量时，使用梯度累积：

```
accumulation_steps = 4 # 累积 4 个 batch 再更新

for batch_idx, (data, target) in enumerate(train_loader):
```

(续下页)

(接上页)

```
data, target = data.to(device), target.to(device)

# 前向传播
output = model(data)

# 损失除以累积步数（保证梯度大小一致）
loss = criterion(output, target) / accumulation_steps
loss.backward()

# 每 accumulation_steps 步更新一次参数
if (batch_idx + 1) % accumulation_steps == 0:
    optimizer.step()
    optimizer.zero_grad()

    print(f"更新参数: batch {batch_idx}")

# 处理最后不足一个累积周期的梯度
if (batch_idx + 1) % accumulation_steps != 0:
    optimizer.step()
    optimizer.zero_grad()
```

效果: batch_size=16, accumulation_steps=4, 等效 batch_size=64

编译模型 (PyTorch 2.0+)

```
# PyTorch 2.0 引入 torch.compile, 可以显著加速
model = MyModel()

# 编译模型（第一次运行会稍慢，后续更快）
compiled_model = torch.compile(model)

# 正常训练
for data, target in train_loader:
    output = compiled_model(data)
    # ...
```

加速效果:

- 训练: 通常提速 10-50%
- 推理: 通常提速 20-100%

4.9.5 模型保存与加载的最佳实践

保存检查点

```
def save_checkpoint(model, optimizer, scheduler, epoch, best_acc, path):
    """
    保存完整训练状态，支持断点续训
    """
    checkpoint = {
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'scheduler_state_dict': scheduler.state_dict() if scheduler else None,
        'best_acc': best_acc,
    }
    torch.save(checkpoint, path)
    print(f"✅ 检查点已保存: {path}")

def load_checkpoint(path, model, optimizer=None, scheduler=None):
    """
    加载训练状态，恢复训练
    """
    checkpoint = torch.load(path)

    model.load_state_dict(checkpoint['model_state_dict'])

    if optimizer and 'optimizer_state_dict' in checkpoint:
        optimizer.load_state_dict(checkpoint['optimizer_state_dict'])

    if scheduler and 'scheduler_state_dict' in checkpoint:
        scheduler.load_state_dict(checkpoint['scheduler_state_dict'])

    epoch = checkpoint.get('epoch', 0)
    best_acc = checkpoint.get('best_acc', 0.0)

    print(f"✅ 检查点已加载，从 epoch {epoch + 1} 继续训练")
    return epoch, best_acc
```

保存最佳模型

```
# 训练循环中的早停和模型保存
best_acc = 0.0
patience = 10 # 早停耐心值
epochs_no_improve = 0

for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(...)
    val_loss, val_acc = validate(...)

    # 保存最佳模型
    if val_acc > best_acc:
        best_acc = val_acc
        epochs_no_improve = 0

        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'best_acc': best_acc,
            'config': config,
        }, 'best_model.pt')

    print(f"📁 保存最佳模型，验证准确率: {best_acc:.2f}%")
    else:
        epochs_no_improve += 1
        print(f"📁 验证准确率未提升，耐心值: {epochs_no_improve}/{patience}")

# 早停检查 (对应{ref}`early-stopping`)
if epochs_no_improve >= patience:
    print(f"🔴 早停触发! 连续{patience}个 epoch 未提升")
    break
```


4.9.6 下一步

恭喜你完成了 PyTorch 实践章节的全部内容！回顾本章学到的：

技能	对应理论
张量操作	计算图的代码实现
自动微分	反向传播算法的 PyTorch 机制
优化器	梯度下降与优化算法的各种变体
调试技巧	调试与可视化技巧中的诊断方法
工程实践	操控效率：效果、速度与硬件的三角权衡中的效率优化

你已经具备了从理论到实践的完整能力。下一步建议：

1. **实战项目**：尝试给 mnist-helloworld 框架添加新的模型或数据集（详见[添加新模型：3 步和添加新数据集](#)）
2. **深入研究**：回到上篇：[练习峰](#)学习更复杂的架构
3. **前沿探索**：阅读论文，复现 SOTA 模型

结语：从入门到实践中，我们整理了更多学习资源，帮助你在深度学习之路上继续前进。

4.10 使用训练框架

完整训练流程中我们手写了完整训练流程，[工程最佳实践](#)中我们讨论了工程规范。现在的问题是：**这些实践如何落地成可复用的工具——而不是每次新项目都重写一遍？**

社团的 mnist-helloworld 框架就是答案。它不是新知识，而是把完整训练流程的流程和工程最佳实践的规范，封装成了一个工程化系统。

本节定位

- **完整训练流程：学原理**——手写每一行代码，理解背后发生了什么
- **工程最佳实践：学规范**——理解好的工程长什么样、为什么需要它
- **本节：用工具**——看前面的原理和规范如何整合成一个可用的框架

本节不教你新概念，而是带你走一遍：“你在第 x 节手写的那段代码，框架里对应的模块是什么，为什么那样设计”。

4.10.1 从"每次重写"到"一次封装"

核心矛盾

完整训练流程最后我们写了一个可运行的训练脚本。它能在 MNIST 上训练分类器。但想象你接下来要做三个实验：

1. 在 CIFAR-10 上测试 LeNet
2. 比较 Adam 和 SGD 的效果
3. 从 MNIST 预训练模型迁移到 CIFAR-10

手动版本的问题：

```
# 实验一：复制文件，改数据集路径、模型名...
# 实验二：改 optimizer，重新跑...
# 实验三：手动保存权重，加载时小心维度匹配...
# —— 每次改动都要修改代码本体，容易引入 bug，难以回溯
```

这就是工程最佳实践中讨论的"工程规范"要解决的问题。框架把这些规范变成了代码：

```
# 三个实验，零代码修改
python train.py --dataset cifar10 --model lenet
python train.py --dataset mnist --model lenet --optimizer sgd --scheduler cosine
python train.py --fork exp1 --dataset cifar10 --freeze "0-0" "0-1" --learning-rate 0.0001
```

设计原则回顾

工程最佳实践中我们讨论了四条规范，框架将它们变成了具体实现：

工程最佳实践 规范	框架实现	效果
配置不硬编码	src/config/config.py: YAML + CLI 双配置	改实验改配置不改代码
模块化项目结构	src/datasets/, src/models/, src/training/	新增模型/数据集只需加一个文件
实验可复现	ExperimentManager: runs/expN/ 自动管理	每次运行完整存档，永不覆盖
检查点管理	CheckpointManager: 4 种保存策略	断点续训、最佳模型自动筛选

4.10.2 配置系统：从硬编码到配置文件

你之前做的

完整训练流程中，我们把超参数写死在代码开头：

```
batch_size = 64
epochs = 10
lr = 0.001
```

工程最佳实践中，我们改进为 `@dataclass Config` —— 参数集中管理，但仍然和代码在一起。

框架的做法

框架更进一步：参数和代码完全分离。

项目根目录的 `config.yaml` 定义所有默认值：

```
dataset:
  name: mnist
  root: ./data
model:
  name: mynet
  num_classes: 10
training:
  epochs: 20
  batch_size: 64
optimization:
  learning_rate: 1e-3
  optimizer: adamw
  weight_decay: 0.01
  scheduler: cosine
  scheduler_t_max: 20
checkpointing:
  save_frequency: 1
```

运行时用 CLI 参数覆盖：

```
python train.py --dataset cifar10 --model lenet --epochs 50
```

优先级规则：

CLI 参数（最高）> 自定义 YAML > 默认 config.yaml（最低）

效果：想尝试 10 组超参数，不再需要改 10 次代码或维护 10 份 Config 子类 —— 10 条命令就够了。

注意

CLI 参数是扁平的 (--epochs)，YAML 里是嵌套的 (training.epochs)。两种方式完全等价，src/config/config.py 负责两者的合并解析。

4.10.3 模型管理：从 nn.Module 到 BaseModel

你之前做的

神经网络模块：搭建计算图中，我们继承 nn.Module 写网络：

```
class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.c1 = nn.Conv2d(1, 6, 5, padding=2)
        # ...

    def forward(self, x):
        x = torch.tanh(self.c1(x))
        return x
```

框架的做法

框架定义了一个 BaseModel 抽象类，在 nn.Module 之上增加了统一接口：

```
class BaseModel(nn.Module):
    @property
    def model_type(self): ...      # "classification" / "siamese"
    def get_criterion(self): ...   # 模型自带的损失函数
    def get_model_info(self): ...  # 参数量等信息
```

所有模型通过 注册机制 管理：

```
# 查看所有可用模型
python -c "from src.models import ModelRegistry; print(ModelRegistry.list_available())"
```

框架内置 4 个系列，共 26 个变体：

经典 CNN（用于入门和小数据量）

模型	参数量	你已经在哪学过	特点
lenet	~62K	LeNet-5: 练习峰首登	经典 5 层, MNIST 标准
mynet	~122K	—	不对称卷积 + SiLU + BN, 默认模型
alexnet	~2.5M	卷积: 发现装备	适配小输入的 AlexNet

Vision Transformer (用于更大数据集)

模型	参数量	说明
bottleneck_vit	~9M	CNN 提取特征 + ViT 头部
fpn_vit_tiny/small/large	0.9M/1.7M/3.8M	金字塔多尺度融合 + ViT

Mixture of Experts (在 FPN-ViT 基础上把 MLP 替换为 MoE)

模型	机制
fpn_moe_vit 系列	每个 token 动态选择 Top-K 专家, 附带负载均衡损失

Siamese 系列 (输出嵌入向量而非 logits, 配合 Triplet Loss)

模型	用途
siamese	基础孪生网络
siamese_fpn_vit 系列	FPN-ViT backbone 做嵌入

使用: 一行切换

```
python train.py --model lenet
python train.py --dataset cifar10 --model fpn_vit_tiny
```

更换模型时, 框架自动读取模型的 `input_channels` 和 `input_size`, 适配对应的数据集预处理。

添加新模型: 3 步

工程最佳实践说"模块化"——框架用注册模式实现:

```
# 第一步: src/models/my_model.py
from src.models.base import BaseModel

class MyModel(BaseModel):
    def __init__(self, num_classes=10, input_channels=1, **kwargs):
```

(续下页)

(接上页)

```

super().__init__(num_classes, input_channels)
self.net = nn.Sequential(
    nn.Conv2d(input_channels, 32, 3, padding=1),
    nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1),
    nn.ReLU(), nn.AdaptiveAvgPool2d((1, 1)),
    nn.Flatten(), nn.Linear(64, num_classes)
)
def forward(self, x):
    return self.net(x)

```

```

# 第二步：在 src/models/__init__.py 注册
from src.models.my_model import MyModel
ModelRegistry.register("my_model", MyModel)

```

```

# 第三步：使用
python train.py --model my_model

```

不需要修改框架的任何已有代码。

4.10.4 数据集管理：从 DataLoader 到 BaseDataset

你之前做的

完整训练流程中，用 `torchvision.datasets.MNIST` 加载数据，手动写 transform：

```

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train_dataset = datasets.MNIST(root='./data', train=True, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

```

换数据集？全部重写。

框架的做法

每个数据集是一个类，transform 封装在内：

```
# MNIST 自带 transform (src/datasets/mnist.py)
def get_train_transform(self):
    return transforms.Compose([
        transforms.RandomAffine(degrees=15, translate=0.1, scale=(0.9, 1.1)),
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])
```

通过 DatasetRegistry 统一管理:

```
# 切换数据集，一行命令
python train.py --dataset mnist
python train.py --dataset cifar10
python train.py --dataset subset_631

# 模型自动适配输入尺寸、通道数、类别数
python train.py --dataset cifar10 --model lenet
```

内置 7 个数据集:

数据集	类型	类别	输入	说明
mnist	分类	10	28×28 灰度	快速实验首选
cifar10	分类	10	32×32 彩色	CNN 入门标准基准
subset_631	分类	631	64×64 灰度	汉字识别，进阶
subset_1000	分类	1000	64×64 灰度	更多汉字
triplet_mnist	度量学习	10	28×28 灰度	在线生成三元组
balanced_triplet_mnist	度量学习	10	28×28 灰度	预均衡三元组
triplet_subset_1000	度量学习	1000	64×64 灰度	汉字 triplet

添加新数据集

与模型注册同理: 继承 BaseDataset → 实现 load_data()、get_train_transform()、get_test_transform() → 注册。

```
from src.datasets.base import ClassificationDataset
from src.datasets import DatasetRegistry

class MyDataset(ClassificationDataset):
    def __init__(self, root="./data", **kwargs):
        super().__init__(num_classes=5, input_channels=3, input_size=(64, 64))
```

(续下页)

(接上页)

```
# 实现数据加载逻辑...

DatasetRegistry.register("my_dataset", MyDataset)
```

4.10.5 训练引擎：从手写循环到 Trainer

你之前做的

完整训练流程中，我们手写了完整的训练循环——约 50 行代码处理一个 epoch 的训练和验证：

```
for epoch in range(num_epochs):
    model.train()
    for inputs, targets in train_loader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        for inputs, targets in test_loader:
            outputs = model(inputs)
            # 统计准确率...
```

这套代码在 LeNet 上能跑。但如果要：

- 加上早停？加 ~15 行
- 切换 Triplet Loss？重写数据读取
- 用混合精度？包 autocast + GradScaler
- 保存最佳模型和最新模型？加 ~20 行状态管理

每加一个功能，循环的复杂度翻倍——这正是框架要封装的部分。

框架的做法

Trainer 类封装了完整的训练逻辑：

- **自动调度**：分类模式 vs Triplet 模式（根据 model_type 和 dataset_type 自动判断）
- **早停**：--patience 5，验证损失 N 轮不降停止

- **混合精度**: `--mixed-precision`, 自动使用 `autocast + GradScaler`
- **学习率调度**: `--scheduler cosine`, 每轮自动 `scheduler.step()`
- **指标追踪**: `loss`、`accuracy`、`positive/negative distance` (`triplet` 模式额外追踪)
- **进度显示**: `tqdm` 进度条
- **曲线绘制**: 每轮结束后更新 4 面板训练曲线图

一键启用多个功能

```
python train.py --dataset cifar10 --model lenet \
    --optimizer sgd --momentum 0.9 \
    --scheduler cosine --scheduler-t-max 50 \
    --mixed-precision --patience 10 \
    --save-frequency 5
```

一行命令等价于之前手写的 ~200 行代码 + 早停逻辑 + 混合精度 + 调度器 + 曲线绘制。

4.10.6 实验管理：从手动记录到自动追踪

你之前做的

完整训练流程中，我们手动创建目录、保存模型：

```
torch.save({'model_state_dict': model.state_dict(), ...}, 'checkpoint.pt')
```

工程最佳实践中，我们用 `ExperimentLogger` 管理实验目录和 `git` 信息。

框架的做法

`ExperimentManager` 实现了同样的逻辑，并增加了 **YOLO 风格自动编号**：

```
python train.py # → runs/exp1/
python train.py # → runs/exp2/
python train.py # → runs/exp3/
```

每次运行生成：

```
runs/exp1/
├─ config.yaml          # 本次实验完整配置
├─ checkpoints/
│   ├─ latest_checkpoint.pt # 每轮更新，用于断点续训
│   └─ best_model.pt      # 验证准确率最高时保存
```

(续下页)

(接上页)

```

|   |— final_model.pt      # 训练结束时保存
|   |— epoch_10.pt        # 每 save_frequency 轮保存
|— training_curves.png    # loss + accuracy + LR + speed
|— logs/training.log      # 逐行日志

```

对比实验变得极其简单：

```

# 基线
python train.py --model lenet --dataset mnist
# → runs/exp1/

# 换模型对比
python train.py --model mynet --dataset mnist
# → runs/exp2/

# 直接比较两个训练曲线
open runs/exp1/training_curves.png
open runs/exp2/training_curves.png

```

4.10.7 Checkpoint 管理：两种恢复策略

完整训练流程中，我们只保存了权重。框架的 CheckpointManager 保存完整训练状态：

```

checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scheduler_state_dict': scheduler.state_dict(),
    'best_acc': best_acc,
    'loss': val_loss
}

```

基于此，框架支持两种恢复方式：

操作	命令	行为	适用场景
Re-sume	--resume exp1	恢复完整状态（模型+优化器+调度器+epoch 计数），覆盖原目录	训练被中断，继续跑完
Fork	--fork exp1	只加载模型权重，创建新目录 runs/exp2，支持改配置	迁移学习、调参

Fork 的 **非严格加载** 值得一提：如果新模型架构与 checkpoint 不完全一致（比如分类头从 10 类改成 5 类），框架会自动匹配兼容的层、跳过不兼容的层，并打印详细报告。

4.10.8 迁移学习：层冻结

迁移学习与微调：站在巨人的肩膀上中我们学到：迁移学习的核心是**冻结通用特征提取层**，只训练任务特定层。框架的 `--freeze` 参数实现了这一点。

```
# 冻结方式一：按层 ID
python train.py --model lenet --freeze "0-0" "0-1"

# 冻结方式二：按 ID 范围
python train.py --model lenet --freeze "0-0:0-2"

# 冻结方式三：按层名模式
python train.py --model lenet --freeze "features"
```

层 ID 映射在训练开始时打印：

Layer ID Mapping:

```
0-0: conv1
0-1: conv2
1-0: fc1
1-1: fc2
```

冻结后，对应参数不在 optimizer 中注册，`.backward()` 跳过它们——这正是自动微分：PyTorch 的核心魔法中 `requires_grad=False` 的实际应用。

实战：MNIST → CIFAR-10 迁移

```
# 第 1 步：MNIST 上训练
python train.py --model lenet --dataset mnist --epochs 20
# → runs/exp1/checkpoints/best_model.pt

# 第 2 步：冻结卷积层，在 CIFAR-10 上微调全连接层
python train.py --fork exp1 --dataset cifar10 --model lenet \
    --epochs 15 --learning-rate 0.0001 \
    --freeze "0-0" "0-1"
```

4.10.9 进阶功能

度量学习 (Siamese + Triplet Loss)

训练基础：登山纪律中讨论过度量学习。框架原生支持：

```
python train.py --dataset balanced_triplet_mnist --model siamese \
    --embedding-dim 128 --epochs 30
```

Trainer 检测到 `model_type="siamese"` 或 `dataset_type="triplet"` 时，自动切换训练模式：

- 加载 (anchor, positive, negative, label) 四元组
- 使用 `OnlineTripletLoss` (batch 内挖掘难例)
- 追踪 positive distance 和 negative distance —— 理想状态是前者远小于后者

向量搜索

框架集成了 Qdrant 向量数据库，训练好的 Siamese 模型可以用于相似性搜索：

```
# 索引：提取特征存入向量库
python -m src.utils.qdrant_search index \
    --checkpoint runs/exp1/checkpoints/best_model.pt \
    --dataset mnist --collection mnist_digits

# 搜索：找与输入图片最相似的样本
python -m src.utils.qdrant_search search \
    --checkpoint runs/exp1/checkpoints/best_model.pt \
    --collection mnist_digits --image path/to/digit.png
```

GUI 实时验证

框架附带 Tkinter GUI，用于在摄像头前实时测试模型：

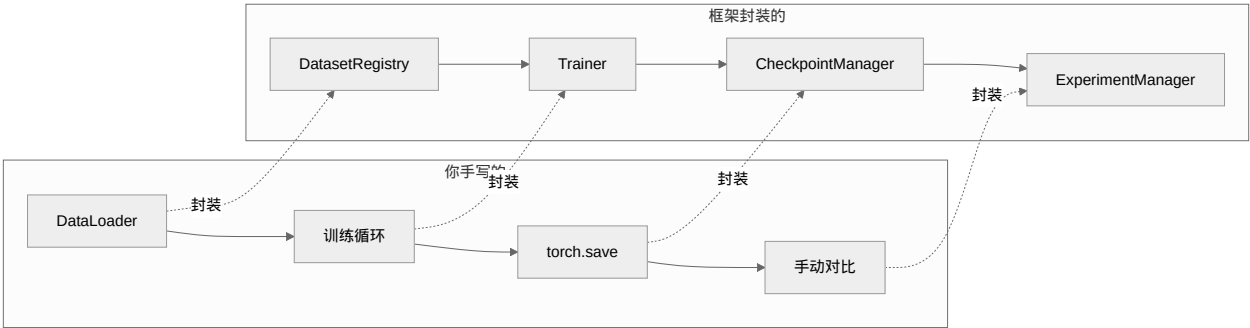
```
cd gui-example
pip install -r requirements.txt
python main_gui.py
```

支持两种模式：

- **手动拍照**：按一次识别一次
- **连续检测**：每 500ms 识别一次，适合演示

预测结果以彩色柱状图展示：绿色 (>80% 置信度)、橙色 (>50%)、红色 (<50%)。

4.10.10 对照总结



你手写的	框架封装的	你现在可以专注的
for epoch 循环 + early stopping 逻辑	Trainer 类	模型架构设计
torch.save + 手动管理文件	CheckpointManager	实验方案设计
手动建目录、记日志	ExperimentManager	结果分析
改代码换数据集/模型	YAML + CLI 配置	超参数调优
手动 model.to(device)	自动设备检测	算法创新
手写数据增强和预处理	每个数据集自带 transform pipeline	数据探索

4.10.11 下一步

你在拆开看看：CNN 消融研究中将用框架做真正的对比实验 —— 每个消融变体对应一个 --config 文件，runs/expN/ 自动记录结果，ExperimentManager 保障实验可复现。

结语：从入门到实践中列出了更多学习资源。

如果想深入理解框架实现，读以下三个文件和你的手写代码做对照：

- src/training/trainer.py：你手写的训练循环 → 框架的封装
- src/training/checkpoint.py：你手写的 torch.save → 框架的自动管理
- src/config/config.py：你手写的 @dataclass Config → 框架的 YAML + CLI 解析

从"每次重写"到"一次封装，反复使用"——这就是工程化的意义。

4.10.12 参与到社团项目的开发

本章介绍的项目是 UCS 深度学习社维护的开源项目，欢迎你贡献代码、报告问题或提出改进建议：

- [mnist-helloworld](#)¹⁰（训练框架）：本课程使用的训练框架，涵盖数据集注册、模型管理、实验追踪等完整功能。如果你注册了新的数据集或模型，欢迎提交 Pull Request 让更多人受益。

¹⁰ <https://github.com/ulink-deep-learning-club/mnist-helloworld>

贡献的方式很简单：发现问题 → 提 Issue → 讨论方案 → 提交 PR。即使只是修正一个文档错别字，也是对社区有意义的贡献。真正理解一个框架最好的方式，就是尝试改进它。

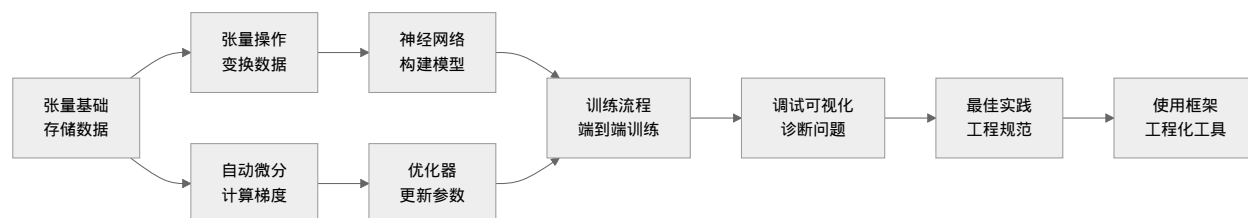
4.11 结语：从入门到实践

恭喜！你已经完成了 PyTorch 实践章节的学习。

从引言：从理论到代码中的第一个张量，到完整训练流程中的完整训练流程，再到工程最佳实践中的工程规范，最后到使用训练框架中的框架使用——你已经掌握了用 PyTorch 开发深度学习项目的完整技能栈。

4.11.1 本章学习路径回顾

让我们回顾这段学习旅程：



每个节点都对应着深度学习核心数学基础和上篇：练习峰中的理论概念，现在你已经能把它们变成实际运行的代码。

4.11.2 核心知识点总结

PyTorch 核心概念映射

PyTorch API	对应理论概念	在深度学习核心数学基础中
<code>torch.Tensor</code>	张量：多维数组	计算图的数据表示
<code>tensor.requires_grad</code>	可微分变量	计算图中的节点
<code>backward()</code>	反向传播	反向传播算法的算法实现
<code>optim.SGD</code>	随机梯度下降	梯度下降与优化算法的基础版本
<code>optim.Adam</code>	自适应矩估计	梯度下降与优化算法中的 Adam 算法
<code>nn.Module</code>	神经网络层	全连接：赤手空拳和卷积：发现装备
<code>nn.CrossEntropyLoss</code>	交叉熵损失	损失函数中的分类损失

工程技能清单

完成本章后，你应该能够：

- ☒ 使用 `torch.Tensor` 进行高效的数值计算
- ☒ 理解 `requires_grad` 和计算图的关系
- ☒ 使用 `nn.Module` 搭建任意神经网络架构
- ☒ 实现完整的训练、验证、测试流程
- ☒ 使用 TensorBoard 监控训练过程
- ☒ 诊断梯度消失/爆炸等训练问题
- ☒ 应用混合精度、梯度累积等性能优化技巧
- ☒ 组织规范的深度学习项目结构
- ☒ 使用社团训练框架进行高效的实验管理

4.11.3 推荐学习资源

动手项目（从简单到复杂）

项目名称	难度	练习重点	参考章节
MNIST 分类器（手写）	☆ 入门	完整训练流程	完整训练流程
mnist-helloworld（框架）	☆ 入门	工程化项目结构、配置管理、实验追踪	社团自有框架 <code>mnist-helloworld/</code>
CIFAR-10 ResNet	☆☆ 基础	更深网络、数据增强	卷积：发现装备
猫狗分类器	☆☆ 基础	迁移学习	迁移学习与微调：站在巨人的肩膀上
情感分析	☆☆☆ 进阶	文本处理、RNN	序列建模：从 RNN 到 Transformer 再到 Mamba
GAN 图像生成	☆☆☆☆ 困难	对抗训练、生成模型	生成模型专题

官方资源

- PyTorch 官方
 - PyTorch 官方教程 ¹¹：从入门到进阶的完整教程
 - PyTorch 官方文档 ¹²：最权威的 API 参考

¹¹ <https://pytorch.org/tutorials/>
¹² <https://pytorch.org/docs/stable/index.html>

- [PyTorch Examples](#)¹³: 经典模型的官方实现
- 实践平台
 - [Kaggle Notebooks](#)¹⁴: 大量可运行的 PyTorch 代码
 - [Google Colab](#)¹⁵: 免费的 GPU 环境, 适合实验
 - [Papers With Code](#)¹⁶: 论文+代码, 学习 SOTA 实现

推荐课程

课程	适合阶段	特点
Fast.ai Practical Deep Learning	初学者	自上而下, 先实践后理论
CS231n (Stanford)	有基础后	计算机视觉经典课程
CS224n (Stanford)	有基础后	NLP 经典课程
3Blue1Brown Neural Networks	理论学习	直观的可视化解释

4.11.4 下一步学习建议

根据你的兴趣和目标, 可以选择不同方向深入:

- 方向一: 深化理论基础回到深度学习核心数学基础, 巩固:
 - 概率图模型与变分推断
 - 优化理论的收敛性分析
 - 泛化理论 (为什么深度学习不会过拟合)
- 方向二: 探索新架构在上篇: 练习峰基础上学习:
 - ResNet、DenseNet 等现代 CNN 架构
 - Transformer 在视觉中的应用 (ViT)
 - 生成模型: VAE、GAN、Diffusion Models 等
- 方向三: 应用实践通过项目巩固技能:
 - 参加 Kaggle 竞赛
 - 复现经典论文

¹³ <https://github.com/pytorch/examples>

¹⁴ <https://www.kaggle.com/code>

¹⁵ <https://colab.research.google.com/>

¹⁶ <https://paperswithcode.com/>

- 为开源项目贡献代码
- 方向四：工程优化深度学习操控效率：效果、速度与硬件的三角权衡中提到的：
 - 分布式训练（Data Parallel / Model Parallel）
 - 模型压缩与部署
 - 高效的数据流水线设计
- 方向五：掌握框架社团的 mnist-helloworld 框架将本章所有工程最佳实践集成在一个项目中：
 - 理解框架的模块设计（config → dataset → model → training → experiment）
 - 学习如何注册新模型和新数据集（ModelRegistry、DatasetRegistry）
 - 用框架复现本章手写的训练流程，对比代码量和维护成本
 - 通过框架的 GUI demo（gui-example/）直观验证模型效果

4.11.5 学习建议

建议的学习节奏：

- 每周 5-10 小时：理论学习 + 代码实践
- 每月完成 1 个项目：从 MNIST 开始，逐步挑战更难的任务
- 每学期精读 1 篇论文：从 AlexNet 开始，循序渐进

常见误区

✗ 误区	✓ 正确做法
只看教程不写代码	每学一个概念就写代码验证
追求最新论文	先扎实掌握经典方法和基础架构
复制粘贴代码	从零手敲，加深理解
忽视调试	把调试当作学习的一部分
独自学习	加入学习小组，互相讨论

4.11.6 结语

深度学习是一门理论与实践紧密结合的学科。

深度学习核心数学基础给了你理解原理的数学工具，上篇：练习峰给了你设计网络的架构知识，而本章的 PyTorch 实践让你能亲手把这些变成现实。

记住这三个核心理念：

1. **张量是数据的载体** —— 理解形状，就理解了数据流动
2. **梯度是学习的信号** —— 反向传播让网络自我改进
3. **实践是检验的标准** —— 跑通代码比看懂公式更有价值

“The best way to learn deep learning is to do deep learning.” —— 学习深度学习的最佳方式就是动手做深度学习。

4.11.7 参考资源汇总

本章相关理论：

- 深度学习核心数学基础：计算图、反向传播、梯度下降
- 上篇：练习峰：全连接网络、CNN、训练技巧
- 迁移学习与微调：站在巨人的肩膀上：迁移学习、预训练模型

PyTorch 官方：

- [PyTorch Tutorials](#)¹⁷
- [PyTorch Documentation](#)¹⁸
- [PyTorch GitHub](#)¹⁹

社区资源：

- [PyTorch Forums](#)²⁰：官方论坛
 - [Stack Overflow - PyTorch](#)²¹：问题解答
 - [Reddit r/MachineLearning](#)²²：前沿讨论
-

祝贺你完成了 PyTorch 实践章节！🎉

接下来的旅程取决于你的选择 —— 无论是深入研究理论、探索前沿架构，还是投身实际项目，本章打下的基础都将成为你的坚实起点。

祝你学习愉快，期待看到你用 PyTorch 创造出的精彩作品！

¹⁷ <https://pytorch.org/tutorials/>

¹⁸ <https://pytorch.org/docs/>

¹⁹ <https://github.com/pytorch/pytorch>

²⁰ <https://discuss.pytorch.org/>

²¹ <https://stackoverflow.com/questions/tagged/pytorch>

²² <https://www.reddit.com/r/MachineLearning/>

模型部署与服务：从训练到生产

5.1 引言：训练完模型之后

你在完整训练流程中完成了模型的训练，在拆开看看：CNN 消融研究中验证了各组件的贡献，在 U-Net：图像分割的革命中用 UNet 处理了具体的图像分割任务——模型在测试集上跑出了不错的准确率，一切都很好。但现在有一个更现实的问题：**这个模型怎么给别人用？**

让模型真正产生价值，不是训练完就结束了。你需要考虑的事情比训练多得多：别人怎么调用它？它能同时处理多少个请求？如果图片不是 28×28 的怎么办？别人会不会把它搞崩溃？这些问题没有一个是训练的 loss 曲线能回答的。

学习目标

完成本章后，你将能够：

1. 用 `torch.onnx.export` 将 PyTorch 模型导出为 ONNX 格式
2. 理解模型服务架构：同步推理 vs 异步推理、简单模式 vs 分布式模式
3. 使用 Ferrinx 部署并管理模型的生命周期
4. 通过 CLI 完成模型注册、推理调用等操作
5. 为模型配置预处理/后处理流水线
6. 理解 API 认证、限流等生产环境必备的安全机制

5.1.1 从训练到服务的跨越

想象一下，你训练了一个手写数字识别模型，想做一个“拍照识数”的 App。这个场景下，模型需要：

1. 接收一张照片，而不是训练时用的 28×28 张量
2. 在几十毫秒内给出结果，用户没有耐心等待
3. 同时响应几百个人的请求，不是一个人调一次
4. 对恶意调用有防护，不能被人搞崩溃

这就是从“训练模型”到“服务模型”的跨越。训练时你关注的是 loss 曲线和准确率，服务时你关注的是延迟、吞吐量、可用性和安全性。

表 1: 训练环境 vs 生产环境

维度	训练环境	生产环境
核心关注	准确率、loss	延迟、吞吐量
输入来源	预设数据集	用户上传/数据流
并发量	单用户、顺序执行	多用户、高并发
运行时长	几小时到几天	7×24 不间断
资源限制	GPU、大内存	未必有 GPU
安全要求	无（学术环境）	鉴权、限流、审计

5.1.2 模型服务面临的四个核心挑战

挑战一：格式兼容性

PyTorch 训练出来的模型是 .pth 或 .pt 文件，这些文件依赖 PyTorch 的运行时环境。如果部署环境没有 PyTorch（或者版本不一致），模型就跑不起来。更麻烦的是，如果想让模型在移动端、浏览器或边缘设备上运行怎么办？某些场景下甚至可能要用 C++ 或者 JavaScript 来加载模型。

ONNX（Open Neural Network Exchange）就是为了解决这个问题而出现的标准格式。它定义了一套统一的中间表示，让模型可以在不同框架之间无缝迁移。[ONNX：模型的中立语言](#)会详细讲解如何将 PyTorch 模型导出为 ONNX。

挑战二：模型性能

即使有了跨平台格式，模型本身的体积和推理速度仍然是一个问题。一个典型的 ResNet-50 约 100MB，在手机上跑一次推理可能需要数百毫秒。[模型优化：量化与剪枝](#)会介绍量化和剪枝两种核心技术——用 $4\times$ 的模型压缩和 $2-4\times$ 的推理加速，让同一模型适配从云端到嵌入式的各种场景。

挑战三：架构选择

模型要做成 API 服务，第一个问题是“该用同步还是异步？”

同步推理就像打电话——你问一句，对方当场回答。适合轻量模型、低延迟场景。但是如果有 100 个人同时打电话呢？电话线不够用就会排队等待。

异步推理就像发邮件——你发出去，对方处理完再通知你。适合大模型、耗时长的推理任务。缺点是实现更复杂，需要任务队列和回调机制。

服务架构：从单机到分布式会详细分析这两种模式各自的适用场景和架构设计。

挑战四：运维复杂度

模型服务上线后，你还需要：管理模型的版本（新模型上线怎么不中断旧服务?）、控制访问权限（不能让任何人都能调你的 API）、监控推理延迟（模型变慢了怎么预警）、处理故障（服务挂了怎么恢复）——这些都是运维上的挑战。

部署实践：用 [Ferrinx 服务模型](#) 会用 Ferrinx 这个具体的推理服务来展示如何应对这些挑战。

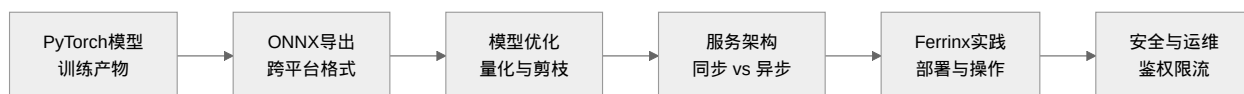
5.1.3 Ferrinx：一个简化的教学工具

Ferrinx 是一个用 Rust 编写的轻量级 ONNX 推理服务。它不像 TensorFlow Serving 或 TorchServe 那样功能丰富，但正因为简单，它更适合用来学习模型部署的核心概念。

Ferrinx 采用 axum 作为 Web 框架、ort 作为 ONNX Runtime 绑定，支持 SQLite 和 PostgreSQL 两种数据库后端。它提供两种运行模式：简单模式（单进程，无需任何外部依赖）和分布式模式（添加 Redis 后支持异步推理和多 Worker 扩展）。这种设计让读者可以循序渐进地理解模型服务的各个层次。

在部署实践：用 [Ferrinx 服务模型](#) 中，我们将实际操作 Ferrinx 的部署流程，包括编译、配置、模型注册、推理调用等完整的生命周期管理。

5.1.4 本章路线图



本章的目标不是让你成为模型部署专家，而是让你理解“从训练到生产”这个链路中的关键环节，建立完整的 MLOps 思维框架。当你以后训练出一个好模型时，你不仅知道“这个模型很准”，还知道“怎么让它真正跑起来”。

5.1.5 下一步

从 [ONNX：模型的中立语言](#) 开始，我们学习如何把 PyTorch 训练的模型导出为标准 ONNX 格式。这个过程需要处理一些细节：如何正确处理输入输出命名、如何处理动态 batch 维度、如何验证导出后的模型和原始模型输出一致——有意思的是，这些细节中的许多都跟[实验设计](#)中“控制变量”的思维方式相通：每次只改一个参数，确保结果可复现。

有了 ONNX 模型后，**模型优化：量化与剪枝**会教你用量化和剪枝把这个模型变得“又轻又快”——这是让模型真正适配生产环境的关键一步。

5.2 ONNX：模型的中立语言

引言：训练完模型之后中我们讨论了模型服务的四个核心挑战，第一个就是格式兼容性。PyTorch 训练出的模型文件（.pt 或 .pth）本质上是一个包含模型结构和参数的 Python pickle 文件，它天然地和 PyTorch 运行时绑定在一起。如果要在一个没有 PyTorch 的环境（或者用不同框架）中加载它，就很困难。

ONNX（Open Neural Network Exchange，开放神经网络交换格式）正是为了解决这个问题而诞生的。你可以把它理解为深度学习的“通用语言”——无论训练时用的是 PyTorch、TensorFlow 还是其他框架，都可以导出为 ONNX 格式，然后在任何支持 ONNX Runtime 的环境中运行。

5.2.1 为什么需要 ONNX？

ONNX 由微软和 Facebook 于 2017 年联合发起，初衷是打破框架壁垒——让研究员在 PyTorch 里训练，工程师用 ONNX Runtime 部署，互不依赖。2019 年 ONNX 被移交给 Linux Foundation 旗下的 LF AI & Data 基金会，成为一个真正中立的开放标准。

直觉上你可能觉得“多此一举”——既然已经在 PyTorch 里训练好了，为什么不直接在 PyTorch 里做推理？原因有几个。

第一是**性能优化**。ONNX Runtime 会对计算图做一系列优化：算子融合（把多个连续的数学操作合并成一个）、常量折叠（把可以预先计算的表达式提前算好）、内存规划（复用内存减少分配开销）。这些优化在 PyTorch 的推理模式下不一定能自动启用。以 ResNet-50 为例，ONNX Runtime 的推理延迟通常比 PyTorch eager 模式快 20-40%。

第二是**部署自由度**。ONNX Runtime 有 C++、Python、C#、Java、JavaScript 等多种语言的 API，还可以运行在移动端和浏览器上（ONNX Runtime Mobile 和 ONNX Runtime Web）。这意味着你可以把一个 ONNX 模型同时部署到服务器、手机、网页三个平台，而训练它的框架是什么并不重要。

第三是**硬件加速的灵活性**。ONNX Runtime 支持通过“执行提供者”（Execution Provider）机制切换不同的硬件后端——CPU、CUDA、TensorRT、CoreML、ROCm、WebGPU。同一个 ONNX 模型，在 NVIDIA GPU 上用 TensorRT 加速，在 Apple Silicon 上用 CoreML 加速，不需要修改模型本身。这对于**模型优化：量化与剪枝**中讨论的量化和剪枝后的模型尤其重要——优化后的 ONNX 模型可以一键切换硬件后端以获得最佳加速。

5.2.2 ONNX 计算图：一个 .onnx 文件里有什么

ONNX 文件的核心是一个**计算图**（Computation Graph），用 Google 的 Protocol Buffers（Protobuf）格式序列化。用一句话描述：ONNX 图是一个有向无环图（DAG），节点是算子，边是张量，数据和计算逻辑分离存储。

表 2: ONNX 计算图的核心组成

组件	Protobuf 字段	作用	举例 (LeNet)
图 (Graph)	graph	顶层容器, 包含所有节点和元信息	整个 LeNet 前向计算
节点 (Node)	graph.node	一个算子操作, 定义计算逻辑	Conv, Relu, MaxPool, Gemm
输入/输出 (Value Info)	graph.input / graph.output	图的入口和出口张量, 含形状与数据类型	input: float32[1,1,28,28], output: float32[1,10]
初始化器 (Initializer)	graph.initializer	存储训练好的权重和偏置值	conv1.weight, conv1.bias, fc2.weight ...
算子集 (OpSet)	opset_import	声明图中使用的算子版本	ai.onnx v18

操作直觉: 把 ONNX 图想象成一个工厂流水线——原材料 (input) 从入口流入, 经过一系列工位 (node, 每个工位做一件事, 比如卷积、激活、池化), 最终成品 (output) 从出口流出。每个工位有自己固定的工具 (initializer, 即训练好的权重)。工厂的"操作规程"版本就是算子集 (opset)。

每个算子节点有三个关键属性:

属性	含义	LeNet 中 Conv 的例子
op_type	算子类型	"Conv"
input	输入张量名列表	["input", "conv1.weight", "conv1.bias"]
output	输出张量名列表	["conv1_output"]
attribute	算子超参数	kernel_shape=[3,3], strides=[1,1], pads=[1,1,1,1]

这种"结构即文档"的设计意味着你用 Netron 打开一个 .onnx 文件, 就能完整看到网络架构、每层的参数形状、数据流向——不需要原始的训练代码。这也是 ONNX 成为跨框架"通用语言"的底层原因。

算子集：版本的学问

opset_version 指定了 ONNX 算子集的版本。每个算子集定义了一组算子及其精确的行为规范。版本越高, 支持的算子越新, 表达能力越强——但可能不被旧版 ONNX Runtime 支持。

表 3: 常用算子集版本演变

版本	年份	代表性新增
opset 11	2019	动态切片 (Dynamic Slice)、Round
opset 13	2020	Softmax 支持负轴、Sigmoid 改为 13 版本
opset 17	2022	LayerNorm 成为标准算子
opset 18	2023	GroupNorm、更完整的 Reduce 算子族
opset 21	2024	支持 INT4/UINT4 量化数据类型

模型优化：量化与剪枝中讨论的 INT8/FP8 量化依赖较高版本的算子集（opset 18+），因为这些版本才定义了低精度数据类型。导出一个要量化的模型时，建议使用 `opset_version=18` 或更高。

5.2.3 PyTorch 到 ONNX 的导出

PyTorch 从 1.x 版本开始内置了 ONNX 导出功能。从 PyTorch 2.x 开始，推荐的导出方式是基于 `torch.export` 引擎的新导出器（`dynamo=True` 模式，2.5+ 已是默认行为），它利用 Torch FX 和 `torch.export` 进行图捕获，生成的 ONNX 图更加精确。

导出一个模型的基本步骤如下：

```
import torch
import torch.onnx

# 加载训练好的模型，设为评估模式
model = LeNetMNIST()
model.load_state_dict(torch.load("lenet_mnist.pth"))
model.eval()

# 创建一个示例输入
# 关键：输入的形状、数据类型必须和模型期望的一致
# LeNet 期望输入：[batch, channel, height, width] = [1, 1, 28, 28]
dummy_input = torch.randn(1, 1, 28, 28)

# 导出为 ONNX
torch.onnx.export(
    model,                                # 要导出的模型
    dummy_input,                          # 示例输入（用于追踪计算图）
    "lenet_mnist.onnx",                  # 输出文件路径
    input_names=["input"],               # 输入层的名字
    output_names=["output"],             # 输出层的名字
    dynamic_axes={                       # 动态轴（可选）
        "input": {0: "batch_size"},
        "output": {0: "batch_size"}
    },
    opset_version=18,                    # ONNX 算子集版本
)
```

这段代码有几个需要理解的要点。

`input_names` 和 `output_names` 为模型的输入输出张量起了名字。这个名字在服务端配置中非常重要——[部署实践：用 Ferrinx 服务模型](#) 中注册模型时，你需要知道模型输入层的具体名字，因为服务端通过名字匹配来传递数据。名字是自由的，但建议使用有意义的英文名（如 "input"、"output"），而不是默认的 `"onnx::Conv_0"`

这种内部名。

`dynamic_axes` 告诉导出器哪些维度是动态的（这里 `batch` 维度可以变化），这样导出的模型不仅限于 `batch=1`。如果不配置动态轴，ONNX Runtime 会拒绝接收与导出时尺寸不一致的输入。一个务实的做法是只将 `batch` 维度设为动态，其他维度固定——这样既支持 `batch` 推理，又避免了全动态形状带来的性能损失。

`opset_version` 指定了 ONNX 算子集的版本，越高版本支持的算子越新，但需要在目标部署环境中确认 ONNX Runtime 版本是否兼容。一般来说，ONNX Runtime 1.16+ 支持 `opset 18`，1.20+ 支持 `opset 21`。

5.2.4 导出后的验证

导出完成后，可以用 Netron（一个开源的神经网络模型可视化工具，在浏览器中运行）来查看 ONNX 模型的图结构——拖拽 `.onnx` 文件到 Netron 页面，就能看到完整的计算图、每层的参数形状、数据流向。

更重要的是，**一定要做数值验证**。导出成功只意味着 ONNX 图结构合法，不意味着计算正确：

```
import onnx
import onnxruntime as ort
import numpy as np

# 验证 ONNX 模型结构
onnx_model = onnx.load("lenet_mnist.onnx")
onnx.checker.check_model(onnx_model)

# 用 ONNX Runtime 做推理，和 PyTorch 对比
ort_session = ort.InferenceSession("lenet_mnist.onnx")

# 准备一样的输入
test_input = np.random.randn(1, 1, 28, 28).astype(np.float32)

# PyTorch 推理
with torch.no_grad():
    torch_output = model(torch.from_numpy(test_input)).numpy()

# ONNX Runtime 推理
ort_input = {ort_session.get_inputs()[0].name: test_input}
ort_output = ort_session.run(None, ort_input)[0]

# 对比输出差异
diff = np.abs(torch_output - ort_output).max()
print(f"最大差异: {diff:.6f}")
# 如果导出正确，差异一般在 1e-5 ~ 1e-6 量级
```

数值差异过大通常意味着模型中含有 ONNX 不支持的算子，或者动态轴配置不正确。差异的来源有两种可能：

一是浮点数运算顺序不同导致的微小误差（通常在 $1e-5$ 量级），这可以接受；二是算子实现差异导致的系统性偏差，需要排查模型中的自定义操作。

5.2.5 导出时的常见问题

ONNX 导出虽然看起来简单，但实际中会遇到一些容易踩坑的地方。

第一个问题是**动态控制流**。如果你的模型中有依赖于数据的条件分支（比如根据输入的值走不同的计算路径），这些分支在 ONNX 中可能无法正确捕获。原因是 ONNX 导出时需要一个静态的计算图，而 Python 的条件分支在导出时只能展开其中一个分支。解决方案是使用 `torch.where` 等可以表达为静态图的操作来替代条件语句。在 PyTorch 2.x 中，dynamo 导出器可以自动处理部分简单的控制流，但复杂分支仍需要手动改造。

第二个问题是**自定义算子**。如果你的模型使用了 PyTorch 中没有等价 ONNX 算子的自定义操作（比如一些特殊的注意力机制变体），导出时会报错。这时你需要实现一个自定义的 ONNX 算子。这在[实验设计](#)中的实验场景中可能不会遇到，但如果你在研究前沿架构，就需要了解。

第三个问题是**动态形状的配置**。如果你导出的模型需要在生产环境中处理不同大小的输入（比如目标检测模型需要处理不同分辨率的图片），就需要正确配置 `dynamic_axes`。配置错误会导致 ONNX Runtime 拒绝接收非固定尺寸的输入。

数值验证的重要性

导出的 ONNX 模型一定要做数值验证。我们见过太多“导出成功但推理结果不对”的情况——导出成功只意味着 ONNX 图结构合法，不意味着计算正确。养成好习惯：每次都跑一遍 PyTorch 和 ONNX Runtime 的输出对比。

5.2.6 预处理与后处理：张量之外的世界

导出的 ONNX 模型本质上是“原始张量到原始张量”的映射。你的 LeNet 模型期望输入的是 `[1, 1, 28, 28]` 的归一化张量，输出的是 `[1, 10]` 的 logits——但在生产环境中，用户的输入是一张 JPEG 图片，你可能期望的输出是“数字 3，置信度 0.98”这样的结构化结果。

这就是预处理和后处理要做的事。

预处理把用户原始输入（图片文件、JSON 数据等）转换成模型期望的张量格式。一个典型的图片预处理流水线包括：缩放（Resize）→ 灰度转换（Grayscale）→ 标准化（Normalize）→ 转张量（To Tensor）。

后处理把模型的原始输出（logits、特征图）转换为用户能理解的结果。一个典型的分类后处理流水线包括：Softmax（转概率）→ Argmax（取最大类）→ 标签映射（数字 → 人类可读标签）。

预处理和后处理的设计需要与模型的训练流程严格对应。[数据准备：打包行囊](#)中数据集的预处理步骤必须在推理时精确复现——否则模型看到的输入分布与训练时不一致，准确率会骤降。这和[实验设计](#)中“控制变量”的思维相通：确保推理环境与训练环境的预处理一致。

有关预处理/后处理如何在部署工具中落地，参见下文[Ferrinx 中的 ONNX 模型配置](#)。

5.2.7 Ferrinx 中的 ONNX 模型配置

Ferrinx 通过 TOML 格式的模型配置文件来声明预处理和后处理流水线。以 LeNet 为例，模型配置文件 `model.toml` 如下：

```
[meta]
name = "lenet-mnist"
version = "1.0"
description = "MNIST digit classification model"

[model]
file = "lenet_mnist.onnx"

# 标签映射文件
labels = "labels.json"

[[inputs]]
name = "input"          # ONNX 模型输入名（与导出时的 input_names 一致）
alias = "image"         # 用户友好的别名
shape = [-1, 1, 28, 28] # [batch, channel, height, width]
dtype = "float32"

# 预处理流水线：将用户图片转换为模型输入张量
[[inputs.preprocess]]
type = "resize"
size = [28, 28]

[[inputs.preprocess]]
type = "grayscale"

[[inputs.preprocess]]
type = "normalize"
mean = [0.1307]
std = [0.3081]

[[inputs.preprocess]]
type = "to_tensor"
dtype = "float32"
scale = 255.0

[[outputs]]
name = "output"          # ONNX 模型输出名（与导出时的 output_names 一致）
```

(续下页)

(接上页)

```
alias = "prediction"
shape = [-1, 10]
dtype = "float32"

# 后处理流水线：将原始 logits 转换为分类结果
[[outputs.postprocess]]
type = "softmax"

[[outputs.postprocess]]
type = "argmax"
keep_prob = true

[[outputs.postprocess]]
type = "map_labels"
```

这个配置文件的思路和[工程最佳实践](#)中讨论的“将配置和代码分离”是一致的：Ferrinx 读取这个配置文件，自动构建预处理和后处理流水线，用户只需关注模型本身。

预处理操作按顺序执行：先将用户上传的图片缩放到 28×28 ，转为灰度图，用 MNIST 的 mean 和 std 做标准化，最后归一化到 $[0, 1]$ 区间并转为 float32 张量。后处理则将模型输出的 10 维 logits 通过 Softmax 转为概率分布，取最高概率对应的类别索引，再通过 labels.json 映射为具体的数字标签。

```
{
  "labels": ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"],
  "description": "MNIST handwritten digits"
}
```

模型目录结构

一个完整的模型目录结构如下：

```
models/
├─ lenet-mnist/
│   ├─ model.toml      # 配置文件
│   ├─ lenet.onnx      # ONNX 模型
│   └─ labels.json     # 标签映射
```

这种“模型即目录”的组织方式让模型的管理变得直观——复制一个目录就等于复制了一个完整的模型服务单元。

Tensor 格式：统一的 API 数据契约

当用户通过 REST API 调用模型推理时，输入输出数据需要一个标准化的格式。Ferrinx 使用显式的 Tensor 结构来描述所有推理数据：

```
{
  "dtype": "float32",
  "shape": [1, 1, 28, 28],
  "data": "<base64-encoded-binary>"
}
```

这个设计有三个意图。第一，**显式的 shape** 让 API 调用者不需要猜测模型期望的输入尺寸，shape 信息就在数据中。第二，**base64 编码的二进制数据**比嵌套 JSON 数组紧凑得多——对于 $224 \times 224 \times 3$ 的图像，二进制编码比 JSON 数组能减少约 50% 的传输体积。第三，**显式的 dtype** 避免了类型混淆——float32 和 int64 在 JSON 中看起来一样，但 Tensor 结构明确声明了数据类型。

在 Python 客户端中，构造 Tensor 输入的代码很直观：

```
import base64
import numpy as np
import requests

# 创建 numpy 数组
input_array = np.random.randn(1, 1, 28, 28).astype(np.float32)

# 转为 Tensor 格式
tensor = {
    "dtype": "float32",
    "shape": list(input_array.shape),
    "data": base64.b64encode(input_array.tobytes()).decode("utf-8")
}

# 发送推理请求
response = requests.post(
    "http://localhost:8080/api/v1/inference/sync",
    headers={"Authorization": "Bearer frx_sk_..."},
    json={
        "model_id": "lenet-mnist-uuid",
        "inputs": {"input": tensor}
    }
)
```

(续下页)

(接上页)

```
# 解析返回结果
result = response.json()
output_tensor = result["data"]["outputs"]["output"]
output_array = np.frombuffer(
    base64.b64decode(output_tensor["data"]),
    dtype=np.float32
).reshape(output_tensor["shape"])
```

5.2.8 从导出到服务

掌握了 ONNX 导出之后，我们有了一个可以在不同平台运行的模型文件。但导出后的模型体积可能很大、推理速度可能不够快——在真正部署之前，还需要做一步“瘦身”。下一节[模型优化：量化与剪枝](#)将介绍量化和剪枝两种核心技术，让模型变得“又轻又快”。

5.3 模型优化：量化与剪枝

ONNX：模型的中立语言中我们把 PyTorch 模型导出成了 ONNX 格式，有了一个标准化的模型文件。但在真正部署之前，还有一个关键问题要解决：**这个模型够“轻”吗？**

一个典型的 ResNet-50 模型，用 FP32 存储需要约 100MB 的磁盘空间，推理一次需要约 4 billion 次浮点运算。如果要在手机上跑，100MB 的下载包太大；如果要在服务器上同时处理 100 个请求，显存可能不够；如果推理延迟超过 100ms，用户体验就会变差。

量化（Quantization）和剪枝（Pruning）是两种最核心的模型优化技术，目标都是**“用更少的资源，达到接近原始的效果”**。量化降低参数的表示精度（从 32 位浮点降到 8 位整数），剪枝直接删除不重要的参数（让权重矩阵变稀疏）。

学习目标

完成本节后，你将能够：

1. 理解量化的原理：对称量化 vs 非对称量化、动态量化 vs 静态量化
2. 使用 PyTorch 的量化 API 对模型进行 INT8 量化
3. 使用 ONNX Runtime 对导出的 ONNX 模型进行量化
4. 理解剪枝的原理：非结构化剪枝 vs 结构化剪枝
5. 使用 PyTorch 的剪枝 API 实现基于幅度的剪枝（Magnitude-based Pruning）
6. 评估量化和剪枝对模型大小、推理速度和准确率的影响

5.3.1 为什么需要模型优化？

引言：训练完模型之后中我们讨论了模型从训练到生产面临的挑战 —— 格式兼容性、架构选择、运维复杂度。但还有一个贯穿始终的挑战没有展开：**性能约束**。

表 4: 不同部署场景的性能要求

部署场景	模型大小限制	延迟要求	典型硬件
云端服务器	< 1 GB	< 100 ms	GPU (V100/A100)
边缘设备	< 100 MB	< 10 ms	CPU / NPU
移动端	< 50 MB	< 30 ms	CPU / 手机 GPU
浏览器	< 10 MB	< 50 ms	WebGPU / WASM
嵌入式/IoT	< 1 MB	< 5 ms	MCU / 低功耗 CPU

从云端到嵌入式，资源的限制越来越严格。模型优化就是让同一个模型适配不同部署场景的核心手段。

5.3.2 量化：用更少的比特存更多的信息

直觉理解

量化的直觉：你有一个精确到小数点后 6 位的温度计（32 位浮点），但气象预报只需要精确到小数点后 1 位（8 位整数）。把 32 位精度降到 8 位，数据量减少 4 倍，但对“今天会不会下雨”的判断几乎没影响。

信息流动直觉：神经网络权重本身就是统计分布 —— 大部分权重集中在某个范围内，极端值很少。32 位浮点的很多比特位其实一直在存“零”，属于浪费。

操作直觉：量化不是简单截断小数，而是“重新分配表示范围”—— 把浮点的连续值映射到整数的离散格点上。就像把一张连续色调的照片转成 256 色调色板，大部分场景下肉眼看不出区别。

直观理解：量化的类比

类比	32 位浮点 (FP32)	8 位整数 (INT8)	代价
尺子	刻度精确到微米	刻度精确到毫米	做家具够用，做芯片不够
调色板	1600 万色（真彩色）	256 色	大部分照片看不出差别
音量	连续旋钮	256 档调节	人耳分辨不出中间档位

一句话总结：量化就是用精度换效率 —— 用可接受的微小精度损失，换取 4× 的模型压缩和 2-4× 的推理加速。

为什么可以压缩：FP32 的低位是噪声

量化的前提是一个根本性问题：**减少精度为什么不会严重损害模型？** 答案藏在浮点数的表示方式和神经网络的训练本质中。

浮点数的结构：Sign + Exponent + Mantissa

IEEE 754 标准定义的浮点数由三部分组成：

$$x_{\text{float}} = (-1)^{\text{sign}} \times 2^{\text{exponent} - \text{bias}} \times (1.\text{mantissa})$$

组件	含义	类比
Sign（符号位）	正负号	温度计上的 +/-
Exponent（指数位）	数值的量级 (2^e)	科学计数法中的 10 的幂
Mantissa（尾数位）	量级内的精细值	科学计数法中的小数部分

为方便描述不同格式，业界使用 **ExMy 表示法**：E 后的数字表示指数位数，M 后的数字表示尾数位数（不含符号位）。例如：

格式	ExMy 表示	结构 (Sign-Exp-Mant)	总位数
FP32	E8M23	1-8-23	32
FP16	E5M10	1-5-10	16
BF16	E8M7	1-8-7	16

指数控制“能表示多大的数”，尾数控制“能表示多精细的差别”。FP32 有 23 位尾数，相当于十进制约 7 位有效数字——这意味着它能区分 1.0000001 和 1.0000002 这种微小差异。

SGD 的噪声掩盖了低位的精度

神经网络训练的核心是随机梯度下降（SGD，[梯度下降与优化算法](#)）。SGD 的每一步更新本身就是有噪声的——梯度是从一个小批量（Mini-batch）估算出来的，不是全量数据的精确梯度。这个噪声的量级通常在 10^{-3} 到 10^{-4} 左右。

关键洞察：FP32 尾数的最后几位编码的变化量级大约是 10^{-7} ——这个精度比 SGD 的噪声还要小 3-4 个数量级。换句话说，即使你保留了几位，下一个 SGD 步骤也会把它覆盖掉。**这几位本质上在存“噪声”，不是“信息”。**
信息流动直觉：把 FP32 的 23 位尾数想象成一根 23 厘米的尺子来量一个物体。SGD 告诉我们“这个物体的大小每次会抖动 ± 1 厘米”——那最后 5 厘米的刻度根本用不上，因为每次量的结果在厘米级就变了。

权重的统计分布

神经网络权重的值通常服从某种类正态分布——大部分权重集中在均值附近，极端值很少。这意味着：

- **指数位**用来覆盖分布的宽度（最大值和最小值之间的范围）
- **尾数位**用来分辨分布内部的细微差异

但在实际推断中，把一个权重从 0.5000001 变成 0.5000000，对模型输出的影响微乎其微——[引言：训练完模型之后](#)中讨论的“用户拍照识数”场景下，用户根本感知不到这个差异。

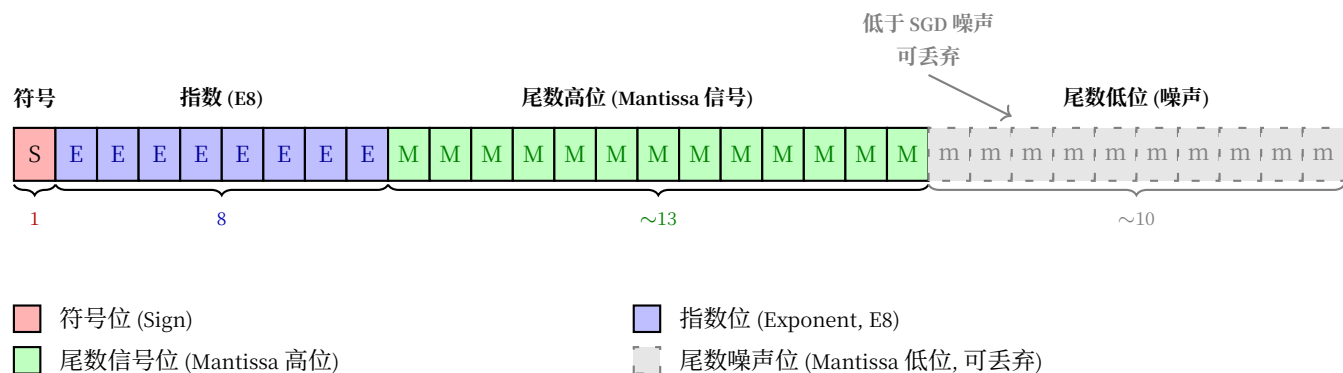


图 1: FP32 的 32 位布局: 信号与噪声

实验证据

Hubara、Courbariaux 等人的 QNN 实验表明 [HCS+18], 在 MNIST 上将权重从 FP32 直接量化为低精度 (1-8 位), 准确率可与 32 位原版媲美——INT8 量化的误差远小于直觉预期。丢失了数位"信息", 准确率几乎未跌。这些被丢弃的位中绝大部分是噪声。

数学表达

对称量化

对称量化将浮点值线性映射到关于零点对称的整数范围。最常见的 INT8 对称量化:

$$x_{\text{int}} = \text{round} \left(\frac{x_{\text{float}}}{s} \right)$$

其中 s (scale) 是缩放因子, 由权重绝对值的最大值决定:

$$s = \frac{\max(|x_{\text{float}}|)}{127}$$

INT8 的表示范围是 $[-128, 127]$, 但对称量化关于 0 对称, 实际使用 $[-127, 127]$ 。

反量化 (Dequantization):

$$\hat{x}_{\text{float}} = x_{\text{int}} \cdot s$$

$\hat{x}_{\text{float}}$ 是原始值的近似——量化误差就是 $|x_{\text{float}} - \hat{x}_{\text{float}}|$ 。

非对称量化

非对称量化引入零点 (zero-point) 来处理偏移分布:

$$s = \frac{\max(x_{\text{float}}) - \min(x_{\text{float}})}{255}$$

$$z = \text{round} \left(-\frac{\min(x_{\text{float}})}{s} \right)$$

$$x_{\text{int}} = \text{round} \left(\frac{x_{\text{float}}}{s} \right) + z$$

非对称量化更灵活，特别适合 ReLU 激活函数（输出都是非负值，不存在对称分布）。代价是多存储一个零点参数 z_0 。

矩阵乘法的量化加速

量化的真正威力在于：INT8 矩阵乘法比 FP32 快得多，而且某些硬件上 INT8 吞吐量是 FP32 的 4 倍。量化后的矩阵乘法：

$$\begin{aligned} Y_{\text{float}} &= W_{\text{float}} \cdot X_{\text{float}} \\ &\approx s_W \cdot s_X \cdot (W_{\text{int}} \cdot X_{\text{int}}) \end{aligned}$$

核心计算 $W_{\text{int}} \cdot X_{\text{int}}$ 是整数运算，可以使用更快的 INT8 指令（如 x86 的 VNNI、ARM 的 SDOT）。

浮点格式全景：FP8 与 MXFP4

理解了浮点数的 ExMy 结构和"低位是噪声"的原理后，自然的问题就是：**我们能降到多低？**业界已经在 INT8 之外探索出了更低精度、但仍保留浮点特性的新格式。

格式对比表

表 5: 深度学习浮点格式一览（按位数递减）

格式	ExMy 表示	结构	总位数	动态范围	精度（有效十进制位）	典型用途
FP32	E8M23	1-8-23	32	$\sim 10^{\pm 38}$	~7 位	训练（主权重副本）
TF32	E8M10	1-8-10	19（存于 32）	$\sim 10^{\pm 38}$	~3 位	NVIDIA Ampere 训练加速
BF16	E8M7	1-8-7	16	$\sim 10^{\pm 38}$	~2 位	大模型训练（与 FP32 同范围）
FP16	E5M10	1-5-10	16	$\sim 10^{\pm 5}$	~3 位	混合精度训练
FP8 E5M2	E5M2	1-5-2	8	$\sim 10^{\pm 5}$	~0.5 位	梯度（需宽动态范围）
FP8 E4M3	E4M3	1-4-3	8	$\sim 10^{\pm 2}$	~1 位	权重与前向传播（需精度）
MXFP6 E3M2	E3M2 + 共享 E8	1-3-2	6（+共享尺度）	共享尺度扩展	~0.5 位	实验性格式
MXFP4 E2M1	E2M1 + 共享 E8	1-2-1	4（+共享尺度）	共享尺度扩展	~0.25 位	NVIDIA Blackwell 原生支持

FP8：浮点量化的新标准

FP8 由 NVIDIA、Arm 和 Intel 于 2022 年联合提出（OCP 标准），包含两种互补的编码：

- **E4M3**（4 位指数 + 3 位尾数）：精度优先。可表示 256 个离散值（ 2^8 ），每个量化等级之间的步长更细。适合存储**权重**和**激活**的前向传播。

- **E5M2** (5 位指数 + 2 位尾数)：范围优先。与 FP16 拥有相同的指数范围 (± 5 个数量级)，但尾数只有 2 位。适合存储**梯度**——梯度可以非常小（需要宽范围），但对精度不敏感。

FP8 为什么比 INT8 好？

INT8 的量化格点是**均匀分布**的——相邻两个值之间的步长恒定。但神经网络中的权重和激活通常在零附近很密集、在尾部很稀疏。FP8 的**对数分布**（指数控制步长）天然匹配这种分布：零附近格点密集（精度高），尾部格点稀疏（范围大）。这就是为什么 FP8 在相同 8 位下通常优于 INT8。

MXFP4：4 位的极致压缩

MX 代表微缩放（Microscaling），是 OCP 在 2023 年发布的另一套标准。MXFP4 的核心思想是**块级共享指数**（Block Floating Point）：

- 每 32 个元素共享一个 8 位缩放因子（E8M0 格式，只有指数没有尾数）
- 每个元素本身只需 4 位（E2M1：1 位符号 + 2 位指数 + 1 位尾数）
- 最终每个元素的有效位数为 $4 + \frac{8}{32} = 4.25$ 位

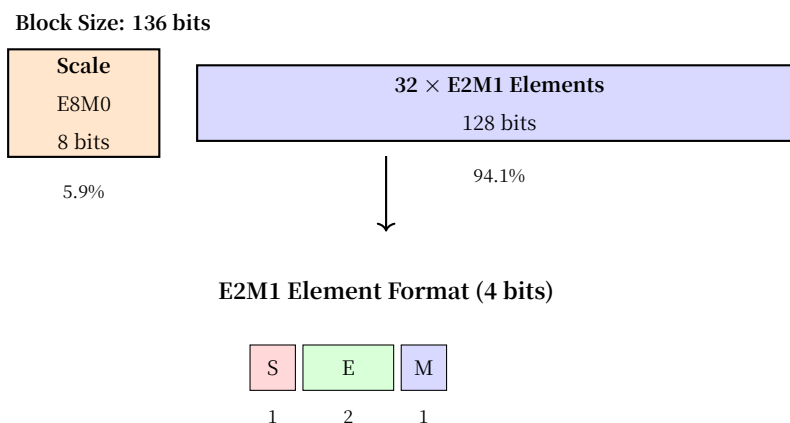


图 2: MXFP4 块结构：32 元素共享一个缩放因子

操作直觉：想象 32 个学生考试，老师不单独给每个人打分，而是先宣布“这次全班平均分 85”（共享 scale），然后每个人的成绩只记录“比平均高还是低”和偏离的等级（E2M1）。信息压缩了，但排名和相对关系保留了。

MXFP4 的重要性在于：NVIDIA Blackwell（B200/GB200）架构在 2024 年首次提供了**原生 MXFP4 硬件支持**，推理吞吐量可达 FP16 的 4 倍。这意味着 4 位推理不再是实验性技术，而是可以大规模部署的生产方案。

表 6: INT8 量化 vs FP8 vs MXFP4 选择指南

场景	推荐格式	理由
通用 CPU 推理、不需要 GPU	INT8	所有 CPU 都支持 INT8 SIMD 指令
GPU 推理、精度敏感	FP8 (E4M3)	对数分布比 INT8 保真度更高
GPU 推理、极致吞吐	MXFP4	4× 吞吐于 FP16, 适合 LLM 大批量推理
训练中的前向/反向	FP8 (E4M3 + E5M2)	两种编码分别优化前向和梯度
移动端 / 边缘设备	INT8 或 FP8	看芯片 NPU 支持哪种

量化的分类

训练后量化 vs 量化感知训练

表 7: PTQ vs QAT 对比

维度	训练后量化 (PTQ)	量化感知训练 (QAT)
操作时机	训练完成后	训练过程中
数据需求	少量校准数据 (几百张)	完整训练数据
计算开销	低 (几分钟)	高 (完整训练)
精度损失	较大 (1-3%)	较小 (< 0.5%)
适用场景	快速部署, 对精度不敏感	精度敏感 的模型

PTQ (训练后量化, Post-Training Quantization): 最简单的方式。模型训练完成后, 用少量校准数据统计各层的数值范围, 然后直接将权重和激活量化为 INT8。适合大多数分类模型。

QAT (量化感知训练, Quantization-Aware Training): 在训练时模拟量化效果 (前向传播用 INT8 计算, 反向传播仍用 FP32 更新权重)。模型在训练中就"学会适应"量化带来的精度损失。适合对精度要求高的场景或难以量化的模型 (如某些 Transformer)。

动态量化 vs 静态量化

表 8: 动态量化 vs 静态量化

特性	动态量化	静态量化
权重量化	INT8 (离线完成)	INT8 (离线完成)
激活量化	每次推理时动态计算 scale	离线校准, 固定 scale
推理速度	提升有限 (激活未量化)	提升显著
实现难度	简单, 几乎无需改动	需要校准数据集
适用层	Linear, LSTM	Conv2d, Linear (校准后)

动态量化是最容易上手的量化方式 —— `torch.quantization.quantize_dynamic` 一行代码即可。但它只量化权重, 激活在推理时才量化, 所以加速效果有限 (通常 1.5-2×)。

静态量化需要校准过程——用代表性数据跑一遍模型，统计每层激活的数值范围，确定固定的 scale 和 zero-point。校准完成后，推理时权重和激活都是 INT8，加速效果最好（可达 4×）。

代码实践：PyTorch 量化

动态量化（最简单）

```
import torch
import torch.quantization

# 加载训练好的模型
# LeNet 来自 {doc}`../cnn-expedition/practice-peak/le-net`
model = LeNetMNIST()
model.load_state_dict(torch.load("lenet_mnist.pth"))
model.eval()

# 动态量化：一行代码
# qconfig_spec 指定哪些层需要量化
# dtype 指定量化目标类型（qint8 即 INT8）
model_int8 = torch.quantization.quantize_dynamic(
    model,                                # 原始 FP32 模型
    {torch.nn.Linear, torch.nn.Conv2d},  # 量化的层类型
    dtype=torch.qint8                     # 量化精度：INT8
)

# 对比模型大小
import os
torch.save(model.state_dict(), "lenet_fp32.pth")
torch.save(model_int8.state_dict(), "lenet_int8.pth")

fp32_size = os.path.getsize("lenet_fp32.pth") # 约 240 KB
int8_size = os.path.getsize("lenet_int8.pth") # 约 60 KB
print(f"压缩比: {fp32_size / int8_size:.1f}×") # ~4×
# 参数量计算：LeNet 约 61,706 个参数 × 4 字节 = ~240KB → INT8 后 ~60KB
```

静态量化（效果最好）

静态量化需要三个步骤：准备、校准、转换。

```
import torch
import torch.quantization
```

(续下页)

(接上页)

```

# 第一步：准备模型
# 在需要量化的层前后插入 Observer 和 QuantStub/DeQuantStub
model = LeNetMNIST()
model.load_state_dict(torch.load("lenet_mnist.pth"))
model.eval()

# 设置量化配置：激活用非对称量化，权重用对称量化（PyTorch 推荐）
model.qconfig = torch.quantization.get_default_qconfig('qnnpack')
# 'qnnpack' 是面向移动端的量化后端，也支持 x86
# 备选：'fbgemm'（x86 高性能后端）

# 插入量化和反量化桩（Stub）
model_prepared = torch.quantization.prepare(model)

# 第二步：校准（Calibration）
# 用代表性数据跑前向传播，观察每层的数值范围
calibration_loader = torch.utils.data.DataLoader(
    calibration_dataset, # 几百张校准图片即可
    batch_size=32
)

with torch.no_grad():
    for images, _ in calibration_loader:
        model_prepared(images) # Observer 自动记录各层数值范围

# 第三步：转换
# 将 Observer 记录的统计信息固化，完成量化
model_static_int8 = torch.quantization.convert(model_prepared)

# 静态量化后的模型，Conv2d 和 Linear 层的计算都是 INT8
# 推理时间大约是 FP32 的 1/3 ~ 1/4

```

静态量化的校准数据选择

校准数据的分布必须和生产环境的数据分布一致。如果校准数据全是数字“1”的图片，但生产环境要识别“0-9”的所有数字，量化的 scale 就会严重失准。这和实验设计中“测试集不能用于训练”的原则相通——校准数据应该来自独立的校准集。

用 ONNX Runtime 量化已导出的模型

ONNX: 模型的中立语言中导出的 ONNX 模型也可以直接量化, 不需要回到 PyTorch:

```
import onnx
from onnxruntime.quantization import quantize_dynamic, QuantType

# 加载导出的 ONNX 模型 (来自 {ref}`onnx-export`)
onnx_model_path = "lenet_mnist.onnx"
quantized_model_path = "lenet_mnist_int8.onnx"

# ONNX Runtime 动态量化
quantize_dynamic(
    model_input=onnx_model_path,
    model_output=quantized_model_path,
    weight_type=QuantType.QInt8,      # 权重量化为 INT8
    per_channel=True,                 # 逐通道量化 (精度更高)
    optimize_model=True,               # 同时做图优化 (算子融合等)
)

# 验证量化后的模型
import onnxruntime as ort

session_fp32 = ort.InferenceSession(onnx_model_path)
session_int8 = ort.InferenceSession(quantized_model_path)

# 对比模型文件大小
import os
print(f"FP32: {os.path.getsize(onnx_model_path) / 1024:.1f} KB")
print(f"INT8: {os.path.getsize(quantized_model_path) / 1024:.1f} KB")
```

ONNX Runtime 量化的优势是可以直接处理 ONNX 格式, 不依赖 PyTorch。如果你已经有了一个 ONNX 模型 (不管它是从什么框架导出的), 一行代码就能量化。

量化效果一览

表 9: LeNet-MNIST 量化效果对比

精度	模型大小	压缩比	推理时间	加速比	准确率
FP32	240 KB	1.0×	2.3 ms	1.0×	99.1%
FP16	120 KB	2.0×	1.5 ms	1.5×	99.1%
FP8 (E4M3)	60 KB	4.0×	0.9 ms	2.6×	98.9%
INT8 (动态量化)	60 KB	4.0×	1.2 ms	1.9×	98.8%
INT8 (静态量化)	60 KB	4.0×	0.8 ms	2.9×	98.5%
MXFP4 (E2M1)	30 KB	8.0×	0.5 ms	4.6×	97.8%

对于 LeNet 这种小模型，量化的绝对收益有限（毫秒级），但对于 ResNet-50 或 Transformer 这类大模型，量化可以将推理延迟从 50ms 降到 15ms（FP8）甚至 8ms（MXFP4）。格式选择的核心权衡：**位数越低，压缩和加速越好，但精度损失越大**——FP8 在 4× 压缩下几乎无损，MXFP4 在 8× 压缩下仍有 97.8% 的可用精度。

5.3.3 剪枝：删除不重要的连接

直觉理解

剪枝的直觉：人脑在发育过程中会经历“突触修剪”——多余的神经连接被移除，保留最有效的连接。神经网络的剪枝也是同样的道理：**删除那些对输出贡献几乎为零的权重**。

信息流动直觉：如果你有一个全连接层，权重矩阵中有很多接近零的值。这些接近零的权重对前向传播的贡献微乎其微—— $0.0001 \times \text{输入}$ 几乎不改变结果。删除它们，模型的“信息高速公路”更简洁。

对比直觉：

	量化	剪枝
做什么	降低每个权重的精度	删除不重要的权重
类比	把高清照片存为压缩格式	把不重要的人从合影中裁掉
数据结构	仍为稠密矩阵	变为稀疏矩阵
加速条件	硬件支持 INT8	需要稀疏矩阵运算支持
模型大小	固定压缩比（4×）	取决于稀疏度

直观理解：剪枝的类比

种树类比：你种了一片树林，有些树长得很好，有些很瘦弱。为了让整片树林更健康，你定期修剪掉弱小的树（基于幅度的剪枝，Magnitude-based Pruning），让阳光和养分集中给强壮的树。

修剪盆景类比：盆景师不是随便剪，而是有计划地剪——剪掉弱枝、病枝和多余的分叉（结构化剪枝），保留主干和优美的形态。这和结构化剪枝（Structured Pruning）的思路一致：不是零散地删权重，而是整通道、整滤波器地删。

非结构化剪枝 vs 结构化剪枝

表 10: 两种剪枝方式对比

维度	非结构化剪枝	结构化剪枝
粒度	单个权重	整通道 / 整滤波器 / 整神经元
稀疏模式	随机分布	规则分布
真实加速	难 (需要稀疏矩阵库)	容易 (直接缩小矩阵)
精度保持	好 (细粒度裁剪)	差于非结构化 (粗粒度裁剪)
实现难度	低	较高

非结构化剪枝把权重矩阵中绝对值最小的那些元素置零。得到的矩阵是稀疏的——大部分元素为零，但非零元素的位置是不规则的。这种稀疏矩阵在通用硬件上不一定会加速（CPU/GPU 的稠密矩阵乘法有高度优化的 SIMD 指令，稀疏矩阵反而可能更慢）。

结构化剪枝成片地删除——可能是整个卷积滤波器、整个通道、或全连接层的整行。删除后，矩阵维度直接缩小，没有稀疏问题，在任何硬件上都能加速。但代价是精度损失更大，因为你是批量地删而不是精细地挑。

数学表达

基于幅度的非结构化剪枝 (Magnitude-based)

给定权重矩阵 W 和剪枝比例 p (比如 30% 的权重被剪掉):

1. 计算剪枝阈值: $t = \text{percentile}(|W|, p)$
2. 生成掩码 (Mask): $M_{ij} = \begin{cases} 0 & \text{if } |W_{ij}| < t \\ 1 & \text{otherwise} \end{cases}$
3. 应用掩码: $W' = W \odot M$ ($\odot$ 表示逐元素相乘)

剪枝后的权重矩阵稀疏度约 p 。被置零的权重在后续微调中有时会“重新长出”（梯度更新让它们从零变回非零），所以通常需要多轮迭代：剪枝 → 微调 → 再剪枝 → 再微调。

结构化剪枝 (Structured Pruning, 以滤波器剪枝为例)

对于卷积层，每个输出通道由一个滤波器定义。滤波器剪枝的思路是：删除那些输出激活最“不活跃”的滤波器。

评估滤波器重要性的常用指标是 ℓ_1 范数 (L1-norm):

$$\text{importance}(F_k) = \sum_{c,i,j} |F_k[c, i, j]|$$

F_k 是第 k 个滤波器， c, i, j 遍历其所有元素。 ℓ_1 范数 (L1-norm) 越小，该滤波器产生的输出越小，对后续层的影响越小——可以被安全删除。

什么是 L1 范数，为什么用它？

L1 范数：向量（或张量）所有元素绝对值之和。

$$\|\mathbf{x}\|_1 = \sum_i |x_i|$$

与之对比，L2 范数是平方和的平方根： $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$ 。

为什么剪枝用 L1 而非 L2？

指标	L1 范数	L2 范数
计算	绝对值求和（快）	平方+开方（慢）
对大值的敏感度	线性	平方放大（过分惩罚大值）
对小值的区分度	清晰	平方缩小（小值被淹没）
与激活幅度的关系	直接正比	非线性

L1 范数的核心优势在于它与激活幅度的**线性关系**：如果滤波器 F_k 的所有权重都缩小 10 倍，其 L1 范数也缩小 10 倍，该滤波器产生的输出也大约缩小 10 倍。L2 范数缩小 100 倍（平方效应），会过度放大差异。

直觉类比：评估一个员工的贡献，看的是他"实际产出"（L1：绝对值求和），而不是"产出平方"（L2）——后者会把一个偶尔加班的人捧上天，而忽略掉那些稳定输出但不出彩的人。

删除滤波器 F_k 后：

- 当前层的输出通道数减少 1
- 下一层对应位置的输入通道也需要同步删除
- 残差连接（residual-connections）的各分支维度必须保持一致，需要额外处理

这就是为什么结构化剪枝比非结构化剪枝复杂——它涉及到网络结构层面的连锁修改。

代码实践：PyTorch 剪枝

非结构化剪枝（Magnitude-based）

PyTorch 内置了剪枝工具，可以对单个层进行剪枝：

```
import torch
import torch.nn.utils.prune as prune

# 加载训练好的模型
model = LeNetMNIST()
model.load_state_dict(torch.load("lenet_mnist.pth"))
```

(续下页)

(接上页)

```

# 对第一个卷积层做非结构化剪枝
# 删除 30% 绝对值最小的权重
prune.l1_unstructured(
    model.conv1,    # 要剪枝的层
    name="weight",  # 剪枝目标 (权重)
    amount=0.3      # 剪枝比例: 30%
)

# 剪枝后的效果
# model.conv1.weight 中原先 30% 的值被置零
# 稀疏度 = 零的个数 / 总参数数
sparsity = (model.conv1.weight == 0).float().mean().item()
print(f"conv1 稀疏度: {sparsity:.1%}") # 约 30%

# 对全连接层也进行剪枝
prune.l1_unstructured(model.fc1, name="weight", amount=0.3)
prune.l1_unstructured(model.fc2, name="weight", amount=0.3)

# 剪枝后的 mask 存储在 weight_mask 属性中
# 前向传播时自动应用 mask, 被剪掉的权重不参与计算

```

剪枝 mask 是持久的

`prune.l1_unstructured` 会将原始权重保存在 `weight_orig` 中, 同时创建一个 `weight_mask`。调用 `model.parameters()` 时会自动应用 `mask`。如果想永久移除剪枝 (固化结果), 使用 `prune.remove(model.conv1, 'weight')`。

全局剪枝 (跨层统一比例)

上面的代码是逐层剪枝 —— 每个层独立删 30%。但不同层对剪枝的敏感度差异很大: 浅层卷积通常比深层全连接对剪枝更敏感。全局剪枝让所有层的权重一起参与排序:

```

# 全局非结构化剪枝: 从整个模型的权重中挑选最不重要的
parameters_to_prune = [
    (model.conv1, 'weight'),
    (model.conv2, 'weight'),
    (model.fc1, 'weight'),
    (model.fc2, 'weight'),

```

(续下页)

(接上页)

```

]

prune.global_unstructured(
    parameters_to_prune,
    pruning_method=prune.L1Unstructured,
    amount=0.3, # 全局 30% 的权重被剪掉（各层分配不同）
)

# 全局剪枝会自动在各层间分配剪枝比例
# 对剪枝敏感的层会被剪得更少，不敏感的更多

```

迭代剪枝（更实用）

一次性剪掉大量权重通常会严重损害精度。实践中更好的做法是迭代剪枝：

```

def iterative_prune(model, train_loader, target_sparsity=0.5,
                    pruning_steps=5, fine_tune_epochs=1):
    """
    迭代剪枝：每次剪一点 + 微调恢复，循环往复

    参数说明：
    - target_sparsity: 最终目标稀疏度（50% 的权重被剪掉）
    - pruning_steps: 分几次迭代到达目标
    - fine_tune_epochs: 每次剪枝后微调几个 epoch
    """
    import torch.nn.utils.prune as prune

    optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

    # 计算每次迭代的剪枝比例
    # 例如 50% 目标分 5 步：每次剪  $(1 - (1-0.5)^{(1/5)}) \approx 13\%$ 
    per_step_amount = 1 - (1 - target_sparsity) ** (1 / pruning_steps)
    current_sparsity = 0.0

    for step in range(pruning_steps):
        # 计算这一步还需要剪的比例
        remaining = 1 - current_sparsity
        step_amount = per_step_amount * remaining

        # 剪枝

```

(续下页)

(接上页)

```

for name, module in model.named_modules():
    if isinstance(module, (torch.nn.Conv2d, torch.nn.Linear)):
        prune.l1_unstructured(module, name="weight", amount=step_amount)

current_sparsity += step_amount * (1 - current_sparsity)

# 微调：让剩余权重适应新的稀疏结构
model.train()
for epoch in range(fine_tune_epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()
        loss = torch.nn.functional.cross_entropy(model(images), labels)
        loss.backward()
        optimizer.step()

print(f"Step {step+1}/{pruning_steps}: "
      f"sparsity={current_sparsity:.1%}")

```

迭代剪枝为什么有效

一次性剪 50% 等于“休克疗法”——模型突然少了半个大脑。迭代剪枝每次只剪 ~13%，然后给模型一个“恢复期”（微调），让剩余权重重新分配功能。这和人类学习新技能的过程类似——不是一次性丢掉旧习惯，而是逐步替换。

结构化剪枝（整通道/整神经元删除）

PyTorch 也支持结构化剪枝，删除整个通道：

```

# 对 conv1 做结构化剪枝：按通道的 L1 范数（L1-norm）排序，删掉 20% 最不重要的通道
prune.l1_structured(
    model.conv1,
    name="weight",
    amount=0.2,      # 删除 20% 的通道
    n=1,             # 用 L1 范数（L1-norm）评估重要性
    dim=0            # dim=0 表示按输出通道剪
)

# 结构化剪枝后，conv1.weight 的形状会变化
# 例如 [32, 1, 3, 3] → [26, 1, 3, 3]（删了 6 个输出通道）

```

结构化剪枝的连锁反应

删除一个卷积层的输出通道，必须同步修改下一层对应的输入通道。PyTorch 的 `ln_structured` 只处理当前层，你需要手动或通过自定义工具处理层间依赖。这也是为什么结构化剪枝在实践中的门槛更高。

剪枝效果一览

表 11: LeNet-MNIST 剪枝效果对比

剪枝方式	稀疏度	模型大小	推理速度	准确率
原始模型	0%	240 KB	2.3 ms	99.1%
非结构化剪枝	30%	168 KB (存储节省 30%)	2.1 ms (稀疏加速有限)	99.0%
非结构化剪枝	50%	120 KB	2.0 ms	98.7%
非结构化剪枝	70%	72 KB	1.9 ms	98.0%
结构化剪枝 (通道)	20% 通道	178 KB	1.8 ms (矩阵缩小, 真实加速)	98.5%

5.3.4 量化与剪枝组合

量化和剪枝不是互斥的 —— 它们可以叠加使用，效果通常是相乘的：

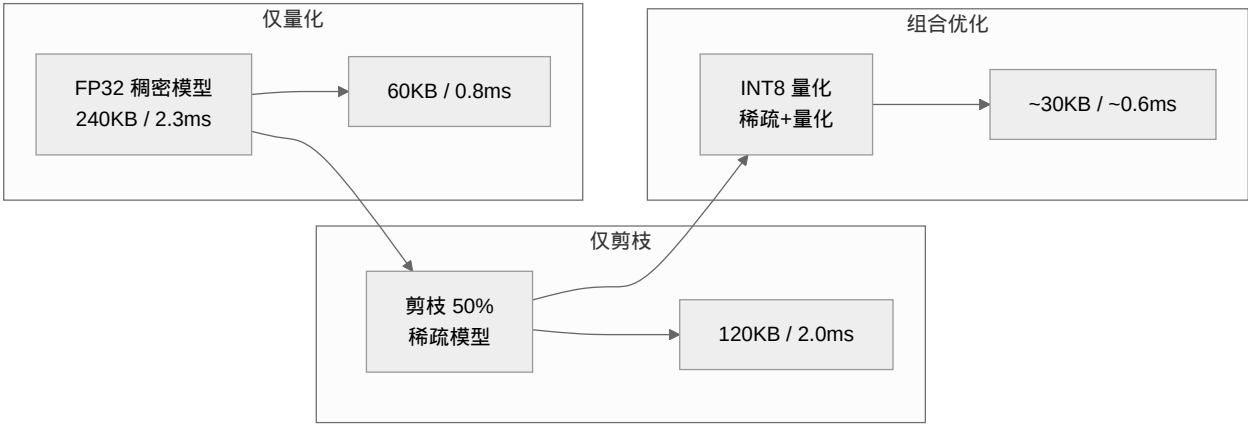


表 12: 组合优化效果 (LeNet-MNIST)

优化策略	模型大小	推理时间	准确率	综合收益
原始 (FP32 稠密)	240 KB	2.3 ms	99.1%	基准
量化 (INT8 静态)	60 KB	0.8 ms	98.5%	压缩 4×, 加速 2.9×
剪枝 (50% 稀疏)	120 KB	2.0 ms	98.7%	压缩 2×, 加速 1.2×
先剪枝再量化	~30 KB	~0.6 ms	98.2%	压缩 8×, 加速 3.8×

常见的实践路径是：先剪枝（去掉冗余权重）→ 再微调（恢复精度）→ 最后量化（降低位宽）。剪枝后的稀疏模型在量化时可能得到更好的量化范围（因为去掉了离群权重）。

5.3.5 历史背景

量化和剪枝都不是新概念。

剪枝的思想可以追溯到 1990 年 Yann LeCun 等人的经典论文 "Optimal Brain Damage"[LDS90], 他们提出可以通过删除不重要的权重来压缩神经网络。2015 年 Han 等人 [HMD16] 将剪枝、量化和 Huffman 编码组合, 实现了 AlexNet 和 VGG-16 的 $35\text{-}49\times$ 压缩。2018 年 Frankle 和 Carbin 提出的"彩票假说"[FC19] 进一步揭示了剪枝的深层原理——大网络中包含着一个可以独立训练的小子网络("中奖彩票"), 剪枝就是找到这张彩票的过程。

量化的发展受益于硬件生态的推动。2018 年 NVIDIA Turing 架构首次引入 INT8 Tensor Core, 让 INT8 推理在 GPU 上成为可能。2020 年后, 手机 NPU (如 Apple Neural Engine、Qualcomm Hexagon) 普遍支持 INT8 甚至 INT4, 量化成为边缘部署的标配。

量化是当下更主流的工程选择

在实际工程中, 量化的使用远远多于剪枝。原因很简单: 量化有标准化的硬件支持 (INT8 Tensor Core、NPU), 且实现简单、效果好、不需要复杂的迭代流程。而剪枝需要稀疏计算支持才能获得真正的加速, 这在通用硬件上至今不够成熟。如果你只能选一个优化手段, 选量化。

5.3.6 总结

概念	实践	效果
浮点数结构	ExMy 表示法 ($E8M23 \rightarrow E2M1$)	指数控制范围, 尾数控制精度
FP32 低位噪声	SGD 噪声 > 尾数低位精度	丢弃 ~ 10 位尾数不影响模型
FP8 量化	$E4M3$ (权重/激活) + $E5M2$ (梯度)	对数分布比 INT8 均匀分布更保真
MXFP4	$E2M1$ + 共享 $E8M0$ 缩放因子	4 位极致压缩, Blackwell 原生支持
对称量化	<code>torch.quantization.quantize_dynamic</code>	$4\times$ 压缩, 对精度影响小
静态量化	<code>prepare</code> $\rightarrow$ <code>calibrate</code> $\rightarrow$ <code>convert</code>	$4\times$ 压缩 + $3\times$ 推理加速
ONNX 量化	<code>onnxruntime.quantization.quantize_dynamic</code>	跨框架, 不依赖 PyTorch
非结构化剪枝	<code>prune.l1_unstructured</code>	压缩比可控, 加速有限
结构化剪枝	<code>prune.ln_structured</code>	真实加速, 但精度损失较大
迭代剪枝	循环剪枝 + 微调	高稀疏度下仍保持高精度
组合优化	先剪枝 $\rightarrow$ 再量化	压缩比叠加, 收益最大

启示

- FP32 的低位是噪声**: SGD 的随机噪声 (10^{-4}) 远大于 FP32 尾数低位的精度 (10^{-7}), 这为压缩提供了理论基础。
- 量化是性价比之王**: INT8 量化实现简单, 硬件支持广泛, $4\times$ 压缩率和 $2\text{-}4\times$ 加速比几乎是"免费午餐"。FP8 的对数分布在同位数下比 INT8 更保真, MXFP4 在 Blackwell 时代为极致吞吐提供了新选择。
- 剪枝需要迭代**: 一次性剪太多会严重损害精度, 迭代剪枝 + 微调是更稳健的选择。实验设计中的"控制

变量"思维在这里再次适用：每次只改变剪枝比例，跟踪精度和速度的变化。

4. **先剪枝再量化**：两者组合使用时，推荐的顺序是剪枝 → 微调 → 量化，剪枝去掉的离群权重有助于后续量化获得更好的 scale 范围。

下一步

模型优化完成后，我们有了一个"又轻又快"的 ONNX 模型。接下来需要设计一套服务架构，让这个模型能够高效地响应外部请求——同时处理多个用户、控制延迟、管理模型版本。**服务架构：从单机到分布式**将深入同步推理、异步推理、简单模式和分布式模式的设计考量。

从"把模型做小"进化到"让模型服务跑起来"！

5.4 服务架构：从单机到分布式

ONNX：模型的中立语言 给了我们跨平台的模型文件，模型优化：量化与剪枝 让它又轻又快。现在的问题是：**怎么让外部世界调用这个模型？** 文件本身不会接 HTTP 请求，不会管理并发，不会做鉴权——它只是一个"大脑"，需要一套"神经系统"来连接外部世界。

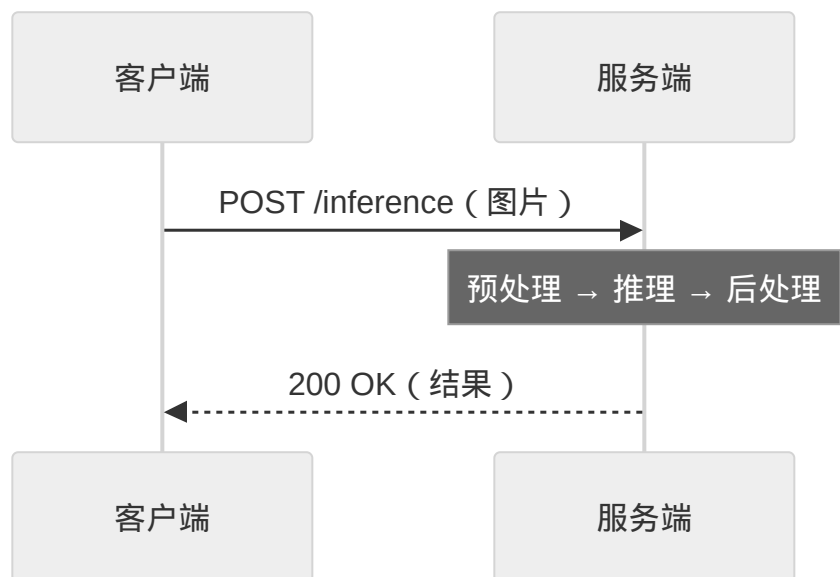
这一节讲解模型服务的核心架构知识。后半部分以 Ferrinx 为例展示这些概念的具体落地。

5.4.1 推理模式：同步 vs 异步

模型服务的第一个架构决策就是推理模式——**调用者是否需要原地等待结果？**

同步推理

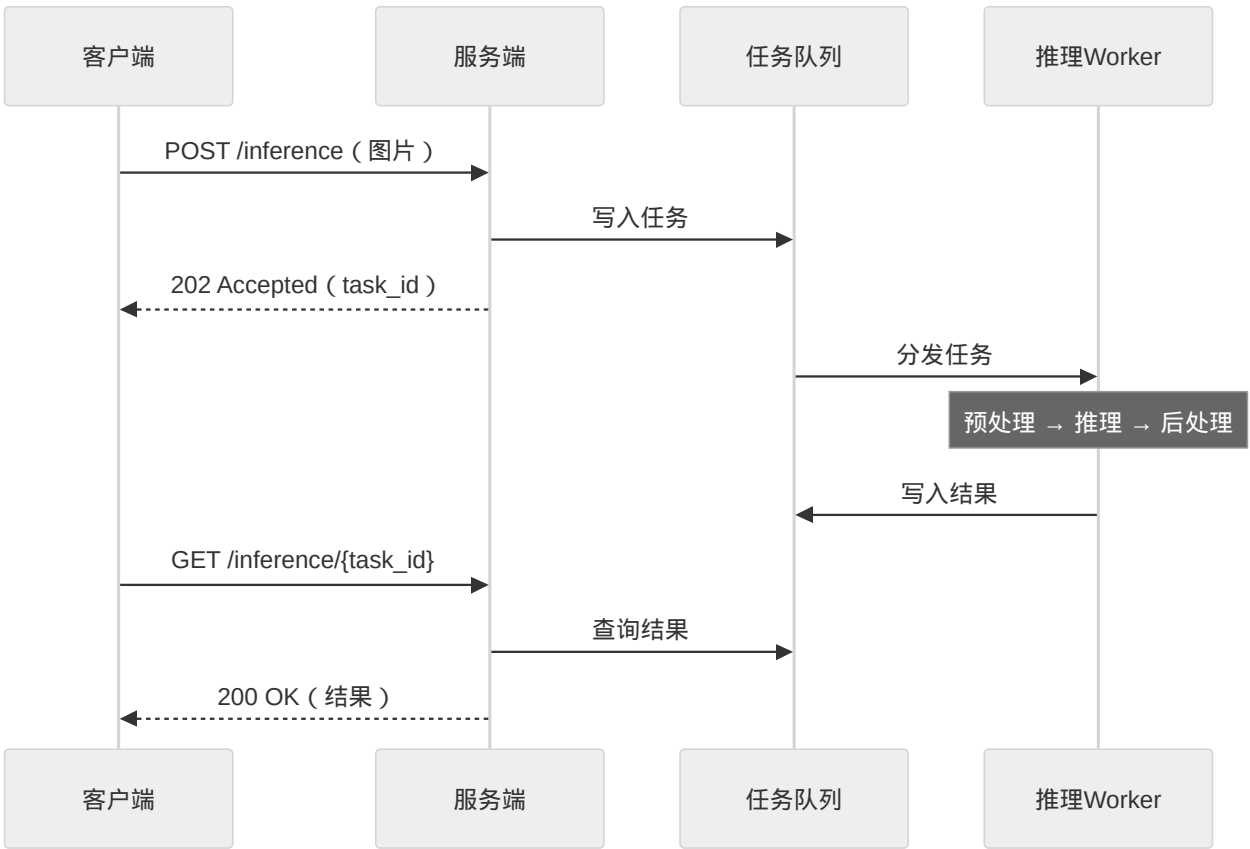
同步推理：客户端发送请求后，HTTP 连接保持打开，服务器处理完推理后立即返回结果。就像打电话——你问一句，对方当场回答。



同步推理的优势是简单 —— 一次请求，一次响应，客户端代码最直观。适合轻量模型（推理延迟 < 100ms）和实时交互场景（如手机拍照识物）。缺点是并发能力受限于单机的推理吞吐：如果一次推理耗时 50ms，单进程理论 QPS 只有 20。100 个用户同时请求，第 100 个要等 5 秒。

异步推理

异步推理：客户端提交任务后立即获得一个任务 ID，稍后通过这个 ID 查询结果。就像发邮件 —— 你发出去了，对方处理完再通知你。



异步推理的核心是**任务队列**（Task Queue）作为缓冲 —— 请求来了就入队，Worker 按自己的节奏消费。适合大模型（推理时间以秒计）或负载波动大的场景：任务队列削峰填谷，高峰期的请求排队等 Worker 空闲时处理，不会被打回。

表 13: 同步推理 vs 异步推理

特性	同步推理	异步推理
调用方式	一次请求-响应	提交 → 轮询结果
连接保持	推理期间一直保持	提交后立即断开
延迟	< 100ms（模型必须快）	可变（秒级可接受）
并发模型	受限于单机推理吞吐	队列缓冲 + Worker 横向扩展
适合模型	轻量模型（ResNet、LeNet）	大模型（LLM、Stable Diffusion）
实现复杂度	低	需要任务队列和状态管理

5.4.2 部署拓扑：单机 vs 分布式

第二个架构决策是部署拓扑——系统由多少个进程组成？

单机模式

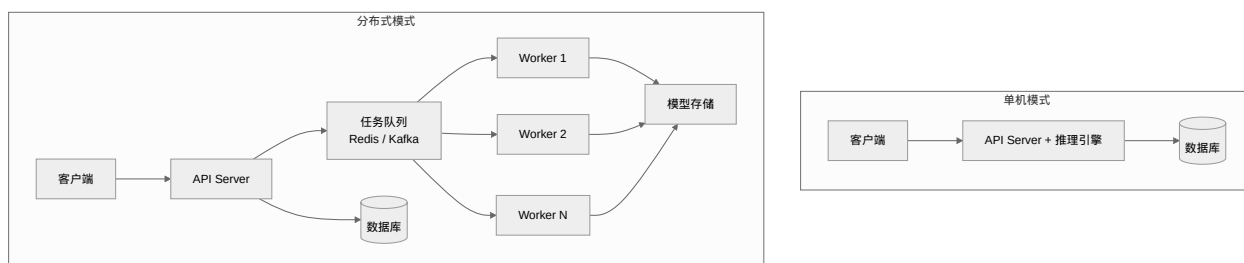
所有功能在一个进程内：HTTP 服务 + 推理引擎 + 模型存储。一个二进制文件启动，零外部依赖。适合开发测试、个人项目、以及流量可控的小规模生产环境。

单机模式的扩展上限就是一台机器的算力。当推理请求超过单机处理能力时，唯一的解法是升级硬件（垂直扩展），但这很快会触及天花板。

分布式模式

将 HTTP 服务（API Server）和推理执行（Worker）拆分为独立进程。任务队列（Redis、Kafka 或 RabbitMQ）解耦两者。API Server 只负责接收请求和写入任务，Worker 负责实际推理。

分布式模式的核心优势是**横向扩展**（Horizontal Scaling）：推理不够用了？加 Worker 就行。从 1 个 Worker 到 10 个 Worker，不需要改一行代码，不需要重启 API Server。



先单机，再分布式

对于个人项目或小团队：

1. 从单机模式开始，最快获得可用服务
2. 当单机吞吐不够时，将推理拆到独立 Worker
3. 当单个 Worker 不够时，横向增加 Worker 数量

不要一开始就搭建完整的分布式系统——你很可能不需要它。过早分布式只会增加运维负担，不会提升任何实际体验。

5.4.3 API 设计：推理服务的接口规范

无论同步还是异步、单机还是分布式，模型服务都需要一个 **API 契约** 来约定客户端和服务端如何通信。

RESTful 端点设计

一个典型的推理服务 API 包含以下端点：

表 14: 推理服务 API 端点

端点	方法	用途
/health	GET	健康检查（负载均衡器探活）
/v1/models	GET	列出可用模型
/v1/inference/sync	POST	同步推理
/v1/inference	POST	异步推理（提交任务）
/v1/inference/{task_id}	GET	查询异步推理结果

输入输出格式

推理 API 的输入输出需要统一的数据格式。常见做法是将张量编码为 JSON 结构，包含形状（shape）、数据类型（dtype）和二进制数据（base64 编码）：

```
{
  "dtype": "float32",
  "shape": [1, 1, 28, 28],
  "data": "<base64-encoded-bytes>"
}
```

这种设计的优势是自描述性——API 调用者不需要额外查阅文档就能知道数据格式；同时 base64 编码比嵌套 JSON 数组紧凑得多（传输体积通常减少 50% 以上）。

5.4.4 认证与安全

API 一旦暴露在公网上，就需要安全机制。模型服务的安全通常分三层。

第一层是**认证（Authentication）**——谁在调用？常见方案是 API Key。客户端在每个请求的 HTTP 头中携带 Authorization: Bearer <key>，服务端验证 Key 的有效性。API Key 不应明文存储——数据库中只存哈希值（如 SHA-256），即使数据库泄露也无法逆向出原始 Key。

第二层是**授权（Authorization）**——调用者能做什么？基于角色的权限控制（RBAC）为每个 API Key 绑定一组权限。一个只用于推理的 Key 只能调用推理端点，不能管理模型或创建新 Key——这样即使泄露，破坏范围也被限制。

第三层是**限流（Rate Limiting）**——调用者能调用多频繁？常见算法包括滑动窗口（统计最近 N 秒内的请求数）和令牌桶（以固定速率补充令牌，请求消耗令牌）。限流应基于 API Key 和 IP 的组合，防止单用户通过多 IP 绕过。

安全三问

设计每个 API 端点时，问三个问题：

1. **谁**可以调用？（认证）
2. 他们可以**做什么**？（授权）
3. 他们可以调用**多快**？（限流）

三个问题都回答了，这个端点的安全设计才算完整。

5.4.5 模型路由

在分布式模式中，多个 Worker 可能加载不同的模型。当一个推理请求到达时，系统需要决定把它发给谁——这就是**模型路由**。

路由的核心逻辑：维护一个“模型 → Worker”的映射表。每个 Worker 启动时上报自己加载了哪些模型，并定期发送心跳刷新状态。路由引擎收到请求后：

1. 从请求中获取目标模型名
2. 查询映射表，找到拥有该模型的 Worker 列表
3. 优先选择模型已缓存（热加载）的 Worker，其次选模型文件存在的 Worker
4. 如果多个候选，轮询或负载最低优先
5. 如果没有任何 Worker 拥有该模型，返回错误

Worker 的心跳是路由可靠性的关键。如果 Worker 宕机，心跳停止，映射表中的条目自然过期，新任务会自动分配给其他健康 Worker——无需人工介入。

5.4.6 架构设计原则

从上述讨论中可以提炼出几条通用的服务架构原则。

简单性优先。单机模式虽然扩展性有限，但它消除了所有分布式复杂度——没有网络分区、没有消息丢失、没有数据一致性问题。在流量增长到真正需要分布式之前，保持简单。

渐进式复杂。从单机到分布式的过渡应该是增量式的——加一个任务队列、拆一个 Worker、再加更多 Worker。每一步都是对前一步瓶颈的精准回应，而不是从一开始就设计一个“万能”的微服务架构。

优雅降级。每个外部依赖（数据库、缓存、任务队列）都应该有回退策略。如果 Redis 不可用，同步推理仍应正常工作；如果数据库不可用，已加载的模型不应受影响。系统的核心功能要在部分组件故障时仍能对外提供服务。

5.4.7 案例：Ferrinx 的架构实现

Ferrinx（UCS 深度学习社维护的开源推理服务）是上述原则的一个具体实现。它的架构可以直接映射到前文讨论的各个概念：

表 15: 通用概念 → Ferrinx 实现

通用概念	Ferrinx 中的实现
推理模式	同步推理（/api/v1/inference/sync）+ 异步推理（Redis Streams）
部署拓扑	简单模式（SQLite + 单进程）+ 分布式模式（PostgreSQL + Redis + 多 Worker）
API 认证	API Key + SHA-256 哈希存储 + 基于角色的权限
限流	滑动窗口 + 令牌桶两种算法
模型路由	Redis 有序集合（ferrinx:models:{id}:workers）+ Worker 心跳 TTL 60s
推理引擎	ONNX Runtime Session + LRU 缓存 + 信号量（Semaphore）并发控制
优雅降级	Redis 不可用时：同步推理正常、异步不可用、认证回退数据库查询

Ferrinx 的具体部署操作（编译、启动、模型注册、推理调用）在下一节 [部署实践：用 Ferrinx 服务模型](#) 中详细展开。

5.4.8 下一步

架构概念已经讲完，接下来用 Ferrinx 实际操作一遍——从编译启动到注册模型、执行推理，把这些概念变成可以跑的命令。下一节 [部署实践：用 Ferrinx 服务模型](#)。

5.5 部署实践：用 Ferrinx 服务模型

服务架构：从单机到分布式中我们讨论了模型服务的架构设计——推理模式、部署拓扑、API 设计、认证安全、模型路由。现在我们把理论变成实践：用 Ferrinx 把 [ONNX：模型的中立语言](#) 导出的 LeNet 模型（如果你做了 [模型优化：量化与剪枝](#)，就是优化后的版本）部署成一个真正的 API 服务。

这一节的操作流程和 [使用训练框架](#) 中框架的使用思路一致：先了解工具的基本用法，然后通过实际操作来掌握各项功能。不同的是，这里的工具不是用来训练模型，而是用来服务模型。

5.5.1 编译与启动

Ferrinx 是一个 Rust 项目，在项目目录的 ferrinx-main/ 下。首次使用需要编译：

```
# 编译所有二进制
cd ferrinx-main
cargo build --release

# 编译产物
# ./target/release/ferrinx-api - API 服务器
```

(续下页)

(接上页)

```
# ./target/release/ferrinx-worker - 推理 Worker
# ./target/release/ferrinx - CLI 客户端
```

编译完成后, 用最简单模式启动 —— 不需要 Redis, 不需要 PostgreSQL, 一个二进制加一个 SQLite 文件就够了:

```
# 启动 API 服务器 (SQLite 自动创建)
./target/release/ferrinx-api
```

服务默认监听 127.0.0.1:8080。访问 <http://localhost:8080/api/v1/health> 可以看到健康检查响应。

ONNX Runtime 的连接方式

Ferrinx 默认使用静态链接 (download-binaries 特性), 编译时会自动下载预编译的 ONNX Runtime 库。如果你的系统 glibc 版本较旧 (Debian 13 以下、Ubuntu 24.04 以下), 需要改用动态链接模式:

```
cargo build --release --features load-dynamic
```

同时需要自行安装 ONNX Runtime 动态库, 并在配置文件中指定路径:

```
[onnx]
dynamic_lib_path = "/usr/local/lib/libonnxruntime.so"
```

5.5.2 配置文件

Ferrinx 的配置文件采用 TOML 格式。在 config.example.toml 的基础上修改即可。最简配置只需要指定数据库后端为 SQLite:

```
[server]
host = "0.0.0.0"
port = 8080
sync_inference_concurrency = 4
sync_inference_timeout = 30

[database]
backend = "sqlite"
url = "sqlite://./data/ferrinx.db"

[storage]
backend = "local"
path = "./models"

[onnx]
```

(续下页)

(接上页)

```
cache_size = 5
execution_provider = "CPU"
```

各配置项的含义和工程最佳实践中讨论的“配置管理”理念相通——将可变参数从代码中分离出来，让部署者可以根据环境灵活调整。例如，`sync_inference_concurrency` 控制同时有多少个推理请求可以并发执行，在 CPU 核心多的机器上可以调大这个值。

5.5.3 初始化系统

系统首次启动后，需要执行 Bootstrap 操作——创建第一个管理员用户并生成 API Key。Ferrinx 有 CLI 和 curl 两种方式完成这个步骤。

CLI 方式（推荐）：

```
./target/release/ferrinx bootstrap
```

这个命令会自动创建 admin 用户，生成 API Key 并保存到本地配置文件。第一次输出的密码是系统自动生成的安全随机密码——只会显示一次，请及时保存。

curl 方式：

```
# Bootstrap
curl -X POST http://localhost:8080/api/v1/bootstrap \
  -H "Content-Type: application/json" \
  -d '{}'

# 设置 API Key 环境变量
export FERRINX_API_KEY="frx_sk_..."
```

5.5.4 模型注册

有了 API Key，就可以注册模型了。Ferrinx 支持两种注册方式：一种是模型文件在服务端本地，通过配置文件直接注册；另一种是模型文件在客户端，通过上传接口提交。

对于本地有模型文件的情况，使用 `model register` 命令，通过 ONNX：模型的中立语言中编写的 `model.toml` 配置文件来注册：

```
./target/release/ferrinx model register \
  --model-config ./models/lenet-mnist/model.toml
```

Ferrinx 会读取配置文件中的元信息、模型路径、预处理/后处理流水线和标签映射，自动注册模型并验证其有效性。注册成功后返回模型的 UUID，后续推理调用需要用到这个 ID。

如果模型文件在另一台机器上，使用 `upload` 命令上传：

```
./target/release/ferrinx model upload ./lenet_mnist.onnx \
--name lenet-mnist \
--version 1.0
```

上传完成后，用 `model list` 查看已注册的模型列表：

```
./target/release/ferrinx model list
```

你应该能看到刚刚注册的 LeNet 模型，状态为 `valid`，代表模型通过了验证。

模型验证的两层检查

Ferrinx 在注册模型时会做两层验证。第一层检查 ONNX 文件的魔数（Magic Number，文件头是否合法），第二层尝试解析模型的输入输出元信息，提取层名、形状和数据类型。这些元信息会被保存，后续推理时用于自动匹配输入。可选的第三层验证会创建 ONNX 推理会话（Session）来确认模型可执行，在 `model_validation.validate_session = true` 时启用——这个选项会显著增加注册时间。

5.5.5 执行推理

模型注册完成后，下面来调用推理。最简单的验证方式是使用 CLI 的同步推理命令：

```
# 通过模型名和版本指定模型
./target/release/ferrinx infer sync \
--name lenet-mnist --version 1.0 \
--image ./path/to/digit.png
```

CLI 会自动对图片做预处理（根据 `model.toml` 中的配置，缩放 → 灰度 → 归一化 → 转张量），发送推理请求，并解析返回结果：

```
{
  "result": {
    "class_index": 3,
    "label": "3",
    "probability": 0.9898
  },
  "latency_ms": 10
}
```

如果要处理批量请求或集成到其他系统，可以直接调用 REST API。Python 客户端的写法在 [ONNX：模型的中立语言](#) 中已经给出，关键步骤是：将输入数据编码为 Tensor 格式（dtype + shape + base64），构造 HTTP 请求，带上 API Key 认证头，发送到 `/api/v1/inference/sync` 端点。

对于需要较长时间处理的大模型，使用异步推理：

提交异步推理任务

```
./target/release/ferrinx infer async \
  --name lenet-mnist --version 1.0 \
  --image ./path/to/digit.png
```

查询任务状态

```
./target/release/ferrinx task status <task-id>
```

异步推理返回一个 task_id，客户端可以轮询这个 ID 来获取结果。在服务架构：从单机到分布式中讨论过，异步推理依赖 Redis —— 如果没有配置 Redis，异步推理端点会返回 503。

5.5.6 配置执行提供者

如果你的机器有 GPU，可以通过配置执行提供者来加速推理。Ferrinx 支持 CPU、CUDA、CoreML、ROCm、WebGPU 等多种后端，在配置文件中指定即可：

[onnx]

```
execution_provider = "CUDA"    # 可选: CPU, CUDA, CoreML, ROCm, WEBGPU
gpu_device_id = 0
```

不同的执行提供者需要启用不同的编译特性（Feature）：

- WebGPU: `cargo build --release --features webgpu`
- CUDA: `cargo build --release --features cuda`
- CoreML: `cargo build --release --features coreml`

GPU 配置的注意事项

编译时启用的编译特性（Feature）必须和配置文件中的 `execution_provider` 一致。如果在配置中设置了 CUDA 但编译时没有启用 `cuda` feature，Ferrinx 会在启动时报错。WebGPU 是推荐的 GPU 加速选项 —— 它通过 Vulkan/DirectX/Metal 适配各类 GPU，不需要安装 CUDA Toolkit。

5.5.7 API Key 管理

生产环境中，不同的客户端需要使用不同的 API Key，而不是所有客户端共享 bootstrap 生成的管理员 Key。创建新的 API Key：

创建一个永久 Key，带默认用户权限

```
./target/release/ferrinx api-keys create \
  --name "my-app-key"
```

(续下页)

(接上页)

```
# 创建一个 30 天后过期的 Key
./target/release/ferrinx api-keys create \
  --name "trial-key" \
  --expires-days 30
```

创建时返回的 Key 明文只会显示一次，请立即保存。查看和管理已有的 Key：

```
# 列出所有 Key
./target/release/ferrinx api-keys list

# 撤销某个 Key
./target/release/ferrinx api-keys revoke <key-id>
```

5.5.8 部署到生产环境的建议

从开发环境到生产环境，有几个关键的配置调整值得注意。

第一，**数据库切换**。开发时 SQLite 很方便，但生产环境面对多实例部署，应该切换到 PostgreSQL。配置文件中修改 [database] 段即可，Ferrinx 的 Repository 抽象层确保业务代码不需要修改。

第二，**日志配置**。开发时日志格式为 text 方便阅读，生产环境应该改为 json 格式，便于日志收集系统（如 ELK）解析。同时配置日志文件轮转，避免磁盘被日志填满。

```
[logging]
level = "info"
format = "json"
file = "./logs/ferrinx.log"
max_file_size_mb = 100
max_files = 10
```

第三，**安全增强**。确保 API 使用 HTTPS（在反向代理如 Nginx 层面配置 TLS），设置合理的限流参数，定期轮换 API Key。Bootstrap 接口在生产环境中应该在完成初始化后禁用（Ferrinx 在已有用户时会自动拒绝 bootstrap 请求）。

第四，**监控部署**。Ferrinx 暴露了 /api/v1/metrics 端点和基本的健康检查接口，可以接入 Prometheus 等监控系统。Worker 的健康状况可以通过 Redis 心跳监测，长时间无心跳的 Worker 说明需要人工介入。

从开发到生产的检查清单

- 将数据库从 SQLite 切换为 PostgreSQL
- 配置 HTTPS（反向代理层面）

- 设置合理的限流参数
- 创建独立的生产 API Key, 不要使用 bootstrap Key
- 配置日志输出为 JSON 格式
- 启用模型预热 ([onnx].preload)
- 部署监控告警
- 制定备份和恢复策略

5.5.9 从框架到 Ferrinx 的一体化流程

现在回顾整条链路：从 PyTorch 代码开始（PyTorch 实践：把理论变成代码），经过模型设计和训练（上篇：练习峰），用消融实验验证（拆开看看：CNN 消融研究），导出为 ONNX 格式（ONNX：模型的中立语言），做量化和剪枝优化（模型优化：量化与剪枝），最后部署到 Ferrinx 服务（本节）——一条完整的“从研究到生产”的管线就这样建立起来了。

这种端到端的视角是理解深度学习工程化的关键。每一章学的技能不是孤立的，它们构成了一个完整的工具链：PyTorch 让你能做实验，消融研究让你能验证设计，ONNX 解决了格式壁垒，量化和剪枝让模型适配生产环境，Ferrinx 解决了部署问题。下一节结语：完整的学习链路我们来回顾整条链路，并讨论未来可以深入的方向。

5.5.10 参与到社团项目的开发

Ferrinx 是 UCS 深度学习社维护的开源项目，欢迎你贡献代码、报告问题或提出改进建议：

- github.com/ulink-deep-learning-club/ferrinx²³

贡献的方式很简单：发现问题 → 提 Issue → 讨论方案 → 提交 PR。即使只是修正一个文档错别字，也是对社区有意义的贡献。

5.6 结语：完整的学习链路

恭喜你完成了本课程的全部内容！

回想一下我们走过的路：从深度学习核心数学基础中的计算图和反向传播，到上篇：练习峰中的网络架构设计，到 PyTorch 实践：把理论变成代码中的代码实现。然后你选择了进阶方向——无论是拆开看看：CNN 消融研究的系统实验验证、迁移学习与微调：站在巨人的肩膀上的迁移学习技巧、下篇：真正的未登峰的注意力机制探索、U-Net：图像分割的革命的图像分割实践，还是本章的模型部署——你已经建立了一条**从理论到生产**的完整认知链路。

²³ <https://github.com/ulink-deep-learning-club/ferrinx>

5.6.1 整条链路的回顾

如果你用一句话概括本课程要回答的问题，那就是：一个深度学习模型是怎么从数学概念变成生产服务的？每个章节回答了这个大问题的一个子问题。

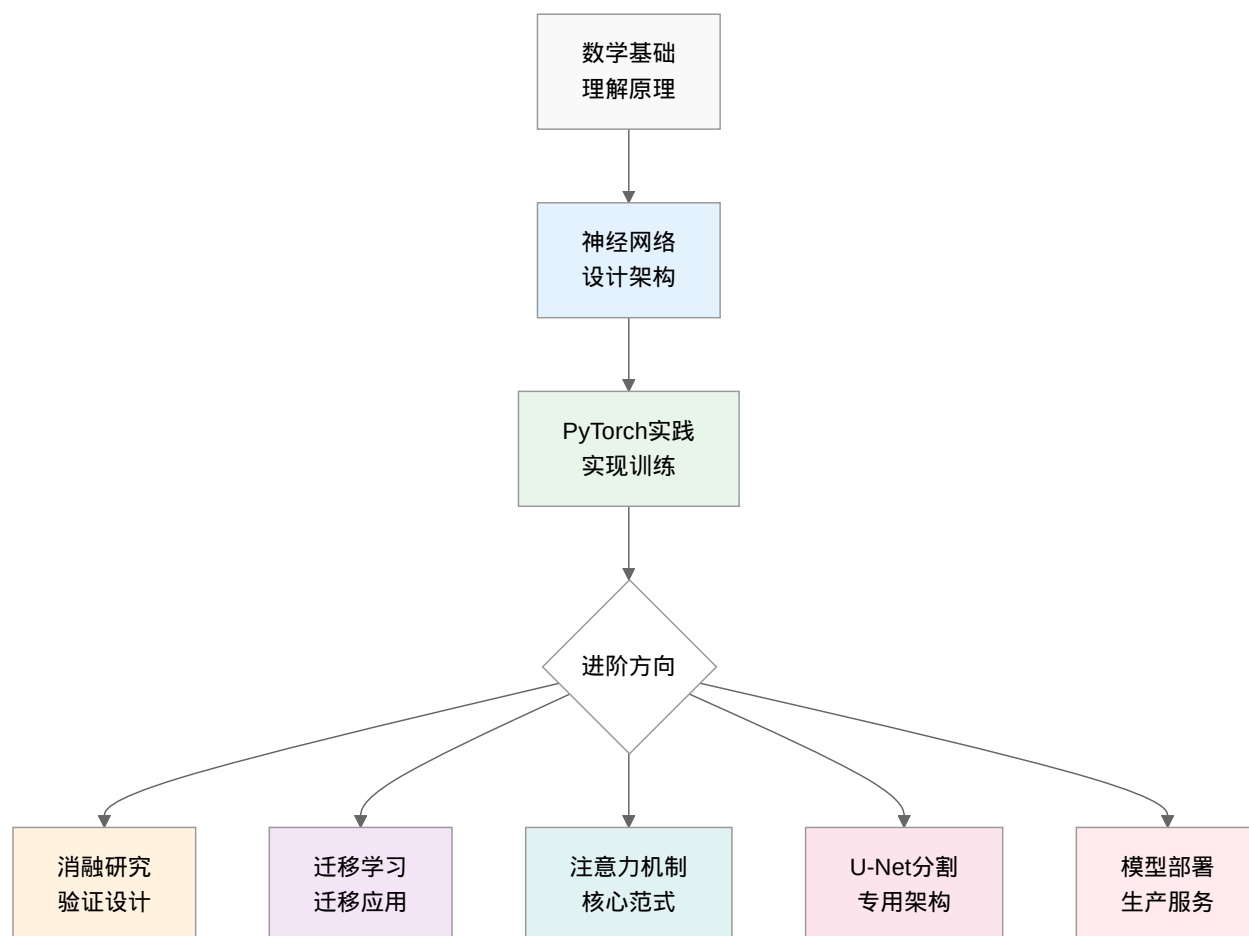
深度学习核心数学基础回答了"深度学习靠什么数学原理工作"——计算图描述计算过程，反向传播高效计算梯度，梯度下降优化参数。这是理论基础，有了它你才能在面对新问题时理解模型"为什么"会学习。

上篇：练习峰回答了"应该建一个什么样的网络"——全连接网络参数爆炸，CNN 通过局部感受野和权值共享解决问题，归纳偏置解释了为什么好的架构设计如此重要。有了它你才能判断什么样的任务适合什么样的架构。

PyTorch 实践：把理论变成代码回答了"怎么把理论变成可运行的代码"——张量操作、自动微分、优化器、训练循环、调试可视化、工程最佳实践。你终于从"看得懂公式"进化到了"跑得出模型"。

在此之后，你根据自己的兴趣选择了一个或多个进阶方向。拆开看看：CNN 消融研究回答了"怎么知道我的设计是好的"——通过控制变量法逐个验证每个组件的贡献，让你从"跟着教程搭网络"成长为"用实验验证设计"。迁移学习与微调：站在巨人的肩膀上让你学会站在前人的肩膀上，用预训练模型快速解决新任务。下篇：真正的未登峰带你进入了现代深度学习最核心的架构范式。U-Net：图像分割的革命则展示了一个完整的专用架构是如何针对特定任务量身定做的。

最后，本章回答了"怎么让模型真正产生价值"——ONNX 导出解决格式兼容性，量化和剪枝让模型适配从云端到嵌入式各种硬件，服务架构解决并发和扩展问题，认证和限流解决安全问题。模型不再只是你电脑里的一个文件，而是一个真正能对外提供服务的产品。



5.6.2 核心收获

整门课程结束后，你带走的不只是具体的知识，更是一种思维方式。

从直觉到精确。每个概念我们都从直觉开始——“卷积就像用放大镜扫描”——然后过渡到直观理解——“ 3×3 卷积核只看 9 个像素”——最后给出精确的数学定义。这种思维层次让你既能在宏观上把握概念的本质，又能在微观上进行精确的分析。

从手写到框架。我们从 NumPy 讲起，然后到 PyTorch 手写训练循环，最后到使用训练框架高效管理实验。从手动导出 ONNX 到使用推理服务部署模型。每一层抽象都没有被当作黑盒——你理解了底层原理，所以能更好地使用上层工具。

从实验到生产。我们不仅关心模型在测试集上的准确率，还关心模型在真实场景中如何被调用、如何扩展、如何保障安全。这种 MLOps 的视角在学术界可能不被强调，但在工业界是决定一个模型能否产生实际价值的核心因素。

5.6.3 下一步可以探索的方向

如果你希望继续深入，有几个方向值得考虑。

方向一：进阶模型优化。模型优化：量化与剪枝中学习了量化和剪枝的基础——但优化的工具箱远不止这些。知识蒸馏（Knowledge Distillation，用大模型教小模型）可以进一步压缩模型而不损失精度；神经架构搜索（Neural Architecture Search）让算法自动发现最优网络结构；稀疏训练（Sparse Training）在训练过程中就培养稀疏连接，省去事后剪枝的麻烦。

方向二：更完善的 MLOps。模型部署之后还有模型监控（数据漂移检测、模型退化预警）、A/B 测试、自动回滚等工程挑战。实验设计中的实验方法论在这些场景中同样适用——控制变量、数据驱动、科学决策。

方向三：大规模服务架构。Ferrinx 是一个教学工具，生产级的模型服务框架如 TensorFlow Serving、TorchServe、NVIDIA Triton Inference Server 提供了更丰富的功能：动态批处理、模型流水线、GPU 共享、请求调度优化等。

方向四：云原生部署。将 Ferrinx 容器化后部署到 Kubernetes，利用 Horizontal Pod Autoscaler 根据推理负载自动扩缩容，配合服务网格实现流量管理和灰度发布——这是现代 MLOps 的标准实践。

最后一句话

深度学习不是关于搭建更大的模型，而是关于理解问题、设计解决方案并用实验验证——从计算图到生产 API，这条链路中的每一步都需要这种科学思维。你已经具备了这些能力，现在去创造吧。

迁移学习与微调：站在巨人的肩膀上

6.1 导论与核心思想

还记得 [完整训练流程](#) 中那个 MNIST 分类器吗？用 60,000 张图片训练，效果不错。但如果你的任务只有 100 张医学影像，还能训练出好模型吗？**迁移学习就是答案。**

学习目标

完成本章后，你将能够：

1. **理解迁移学习的核心思想**：解释为什么需要迁移学习，掌握领域与任务的基本定义
2. **掌握分类体系**：区分归纳式、直推式、无监督迁移学习，理解四种迁移方法的技术路线
3. **熟练运用核心技术**：正确选择并实施特征提取与微调策略
4. **避免实践陷阱**：识别并解决灾难性遗忘、过拟合等常见问题
5. **完成实际项目**：根据任务特点选择合适的预训练模型和迁移策略

6.1.1 什么是迁移学习？

站在巨人的肩膀上

类比 1：学习乐器

想象你已经学会了钢琴（源任务）：

- 你理解乐谱、节奏、和声
- 你的手指已经训练出协调性

现在你要学小提琴（目标任务）：

- **没有迁移**：从头学乐理、识谱、音准 —— 需要数年
- **有迁移**：乐理知识直接用，只需适应新的演奏方式 —— 数月即可

迁移学习就像音乐技能的迁移 —— 底层知识（乐理）通用，只需调整表层技巧（演奏方式）。

类比 2：视觉识别

预训练模型在 ImageNet（1000 类自然图像）上训练后：

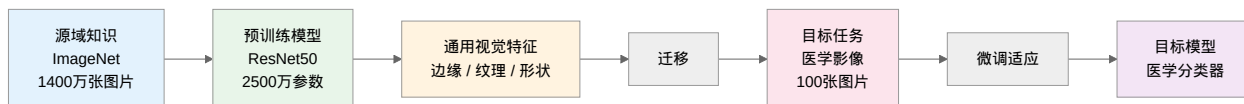
- 学会了识别**边缘**（直线、曲线）
- 学会了识别**纹理**（皮毛、金属、织物）
- 学会了识别**形状**（圆形、方形、不规则）

这些能力对医学影像同样有用：

- X 光片的**边缘**帮助定位器官边界
- CT 扫描的**纹理**帮助区分组织类型
- 肿瘤的**形状**帮助判断良恶性

核心洞察：预训练模型学到的不是“猫狗分类”，而是“看世界的通用方式”。

信息流动



知识从海量数据流向预训练模型，再迁移到目标任务，**信息逐渐从通用走向特定**。

关键洞察：预训练模型像是“知识压缩机”，把 1400 万张图片的信息压缩到 2500 万参数中。迁移学习时，我们只需微调这个压缩表示，而不需要重新学习所有知识。

形式化定义

迁移学习的核心思想是：利用源领域（Source Domain） D_s 和源任务（Source Task） T_s 中学到的知识，来帮助目标领域（Target Domain） D_t 和目标任务（Target Task） T_t 的学习 [PY10]。

$$\text{目标: 利用 } D_s \text{ 和 } T_s \text{ 的知识, 提升在 } D_t \text{ 上学习 } T_t \text{ 的性能} \quad (6.1)$$

关键假设：源领域数据量远大于目标领域 ($n_s \gg n_t$)。

核心概念：领域与任务

理解迁移学习需要明确两个基本概念。

领域 (Domain) = 数据的"世界"

直觉理解：想象你在学画画。

- **源领域**：你在美术课上画水果（苹果、香蕉）
- **目标领域**：现在要画医学解剖图（心脏、肺部）

虽然都是画画（特征空间相同，都是图像），但**画风完全不同**（数据分布不同）——水果是彩色的、光滑的；解剖图是黑白的、复杂的。

数学上，领域 = 特征空间 + 数据分布：

- **特征空间**：数据的表现形式（如 224×224 的 RGB 图像）
- **数据分布**：这些数据实际长什么样（自然照片 vs 医学影像）

对比项	源领域 (ImageNet)	目标领域 (医学影像)
特征空间	都是 224×224 RGB 图像	都是 224×224 RGB 图像
数据分布	自然风景、动物、物品	X 光片、CT 扫描
直觉	照片风格	黑白影像风格

任务 (Task) = 要解决的问题

直觉理解：同样的数据，可以有不同的任务。

- **源任务**：这张图片是猫还是狗？（1000 类分类）
- **目标任务**：这张 X 光片显示肺炎了吗？（2 类分类）

领域偏移 (Domain Shift)

生活中的例子：

假设你学会了识别"水果照片中的苹果"。现在给你看"素描画中的苹果"，虽然都是苹果，但风格完全不同——这就是领域偏移。

为什么会出现这个问题？

- 预训练模型学会了"识别照片中的物体"
- 但医学影像是"识别黑白影像中的病理特征"
- **底层技巧相通**（边缘检测、形状识别），但**表现形式不同**

解决方案：

- 保留底层技巧（浅层特征）
 - 调整顶层策略（分类层）
 - 这就是**微调**的核心思想
-

一句话总结：迁移学习就是「用学画水果的技巧，快速学会画解剖图」——基础技能相通，只需调整应用方式。

迁移学习的效果

大量实践表明，迁移学习能够带来显著的性能提升：

- ImageNet 预训练的 ResNet 模型，迁移到下游视觉任务平均提升 5-20% 准确率 [HZRS16]
- BERT 通过大规模预训练，在 11 项 NLP 任务上刷新了 SOTA [DCLT19]
- 医学影像领域，迁移学习可将小样本任务准确率从 40-50% 提升至 80-90%

6.1.2 为什么需要迁移学习？

1. 数据困境

深度学习模型的性能往往依赖于**海量标注数据**。然而在实际应用中，我们经常面临数据稀缺的挑战：

- **医疗影像：**罕见疾病样本往往仅有数百例，难以从零训练深度网络
- **工业缺陷检测：**优质生产线很少产生缺陷样本，缺陷数据极为稀少
- **小众语言：**绝大多数语言缺乏足够的数字化文本资源

此外，特定领域的数据常因隐私或商业原因难以汇聚，形成“数据孤岛”问题。

2. 计算成本

训练具有竞争力的深度学习模型需要巨大的计算投入。ResNet-50 训练成本约数百美元，而 GPT-3 等大模型则需数千万美元。对于大多数研究团队和企业，从头训练既不现实也无必要。

3. 过拟合风险

小数据集上从头训练深度网络极易导致**过拟合**——模型“记住”训练样本而非学习通用规律。迁移学习通过引入预训练知识，相当于为模型提供了强大的“先验正则化”，有效缓解这一问题。

4. 人类学习的启示

人类天然具备知识迁移的能力：会骑自行车的人学摩托车更容易，掌握 Python 后学习 JavaScript 事半功倍。迁移学习正是将这一思想引入深度学习——利用在一个领域学到的知识，帮助解决另一个领域的问题。

6.1.3 迁移学习的适用场景

迁移学习特别适用于以下场景：

1. **目标领域数据稀缺**：标注数据难以获取或成本高昂
2. **快速原型验证**：需要在短时间内建立 baseline 模型
3. **计算资源受限**：无法承担从头训练大模型的成本
4. **领域间存在关联**：源任务与目标任务有一定相似性

何时不适用？负迁移（Negative Transfer）详解

当源领域与目标领域完全无关时，强行迁移可能导致**负迁移**——迁移后性能比从头训练还差。

负迁移发生的条件：

1. **领域差异过大**：源域和目标域的底层特征完全不同
 - ✗ 用自然图像模型处理基因组序列
 - ✗ 用文本模型处理音频信号
2. **任务冲突**：源任务和目标任务的要求相互矛盾
 - ✗ 用"识别猫狗"的模型做"识别猫的品种"（粒度差异过大）

数学解释：

负迁移发生时，源域知识反而成为噪声：

$$\text{Performance}_{\text{迁移}} < \text{Performance}_{\text{从头训练}}$$

如何避免负迁移：

1. **领域相似性判断**：用预训练模型提取目标域特征，看是否能较好分离
2. **渐进式实验**：先尝试特征提取，效果不佳则考虑从头训练
3. **可视化分析**：用 t-SNE 可视化源域和目标域的特征分布，看是否有重叠

负迁移 vs 性能提升有限：

- 负迁移：性能**下降**（有害）
- 性能提升有限：性能**上升但不多**（无害但帮助不大）

6.1.4 本章小结

- **动机**: 解决数据稀缺、计算成本高、小样本过拟合等问题
- **定义**: 利用源领域知识提升目标领域学习性能的范式
- **核心**: 领域（特征空间+分布）与任务（标签空间+预测函数）
- **直觉**: 像学乐器一样迁移底层知识，像"看世界"一样复用通用特征

下一步

理解了迁移学习的基本概念后，[迁移学习分类体系](#) 我们将建立一套**分类体系**，帮助你在面对具体问题时快速找到合适的方法——归纳式、直推式、无监督迁移学习，哪种适合你的任务？

6.2 迁移学习分类体系

[导论与核心思想](#) 中我们理解了迁移学习的核心动机：借用源领域的知识解决目标领域的问题。但"迁移"这个词很宽泛——

- 是用 ImageNet 预训练模型做医学影像分类？
- 是把英文情感分析模型迁移到中文？
- 还是让模型同时学习多个相关任务？

这些都是迁移学习，但技术路线完全不同。**本章将建立一套分类体系，帮助你在面对具体问题时快速找到合适的方法。**

6.2.1 按迁移情境分类

根据源域与目标域的关系，迁移学习可分为三类 [PY10]:

1. 归纳式迁移学习

定义: 源域和目标域的**任务不同**。这是最常见的场景。

用 ImageNet 预训练模型做医学影像分类——同样是"看图"，但分类的内容从"猫还是狗"变成了"有没有病灶"。

2. 直推式迁移学习

定义: 源域和目标域的**任务相同但数据来源不同**。

在英文新闻上训练的情感分析模型，迁移到中文社交媒体文本——任务都是"判断情绪"，但语言不同了。

核心挑战是**领域偏移**: 同样的任务，不同的数据分布。

3. 无监督迁移学习

定义：源域和目标域都不需要标注数据。

用自监督预训练的特征提取器，直接对新领域的图像做聚类——全程不需要任何人标注。

类型	任务关系	数据来源	需要标注？
归纳式	不同	可同可不同	目标域需要
直推式	相同	不同	目标域少量或不需
无监督	可不同	可不同	都不需要

这三类中，**归纳式**覆盖了绝大多数实际场景——你几乎总是在用预训练模型解决一个新任务。

6.2.2 按迁移方法分类

知道"属于哪种情境"只是第一步，更关键的问题是：**知识到底怎么从源域搬到目标域？**根据迁移机制，可分为四类 [ZQD+20]：

1. 基于模型的迁移

核心思想：直接复用预训练模型的参数和结构。这是**最常用、最高效**的方法。

类比：你刚拿到驾照（预训练模型），现在要开一辆不熟悉的车——方向盘、刹车、油门的基本操作都一样，你只需要适应新车的手柄位置和刹车灵敏度。

这条路的灵魂：冻结与解冻

神经网络像一栋多层建筑：

楼层	学到的内容	通用程度	迁移时的策略
底层	边缘、颜色、纹理	最通用	冻结 ——任何任务都用得上
中层	形状、部件、组合	较通用	微调或冻结——看任务相似度
顶层	语义、物体类别	最特定	替换 ——重新学分类头

关键决策就是决定哪些层"冻结"（保留预训练知识不动）和哪些层"解冻"（允许在新任务上调整）。这个决策直接决定了迁移效果——冻太多则适应不足，冻太少则知识丢失。

展开讲：[基于模型的迁移](#) 将详细覆盖特征提取、微调、分层学习率和渐进解冻等实践技巧。

2. 基于特征的迁移

核心思想：学习一个"翻译层"，把源域和目标域的特征映射到同一个空间，让模型分不出数据来自哪个域。

类比：两台相机拍同一个场景——iPhone 的照片鲜艳，老式手机的照片暗淡。基于特征的方法就是找到一种“滤镜”，让两种照片看起来风格统一，这样用 iPhone 照片训练的模型也能识别老式手机拍的内容。

什么时候用：源域和目标域差异很大（如游戏画面 → 真实街景），直接用模型迁移效果差时。

3. 基于实例的迁移

核心思想：给源域中“跟目标域长得像”的样本更高权重，让模型专注于有用的那部分数据。

类比：你要用一本通用百科全书（源域）来准备一场生物考试（目标域）。不需要整本背诵——只需要把“动物”“植物”“细胞”相关的词条标亮，重点复习。

什么时候用：源域大而杂、只有部分内容与目标域相关。

4. 基于关系的迁移

核心思想：迁移数据之间的**逻辑关系**或知识结构，而不是单个样本或特征。

类比：学生学了“苹果和香蕉都是水果”后，能自动推断“草莓也是水果”——这不是因为见过草莓，而是掌握了“水果”这个概念的关系网。

什么时候用：数据具有明确的结构化关系（推荐系统、知识图谱）。

四类方法对比

方法	核心操作	需要源域数据？	实现难度	主流程度
基于模型	冻结/解冻层、替换分类头	✗ 不需要	☆ 低	☆☆☆☆☆
基于特征	学习域不变特征空间	☑ 需要	☆☆☆ 高	☆☆
基于实例	调整样本权重	☑ 需要	☆☆☆ 高	☆☆
基于关系	迁移逻辑规律	☑ 需要	☆☆☆ 高	☆

6.2.3 方法选择指南

核心结论：90% 的实战场景走第一条路径——基于模型的迁移。

6.2.4 本章小结

按情境分三类——知道“我的问题属于哪种”：

- 归纳式：任务不同（最常见）
- 直推式：任务相同，数据不同
- 无监督：没标注数据

按方法分四类——知道“知识怎么搬运”：

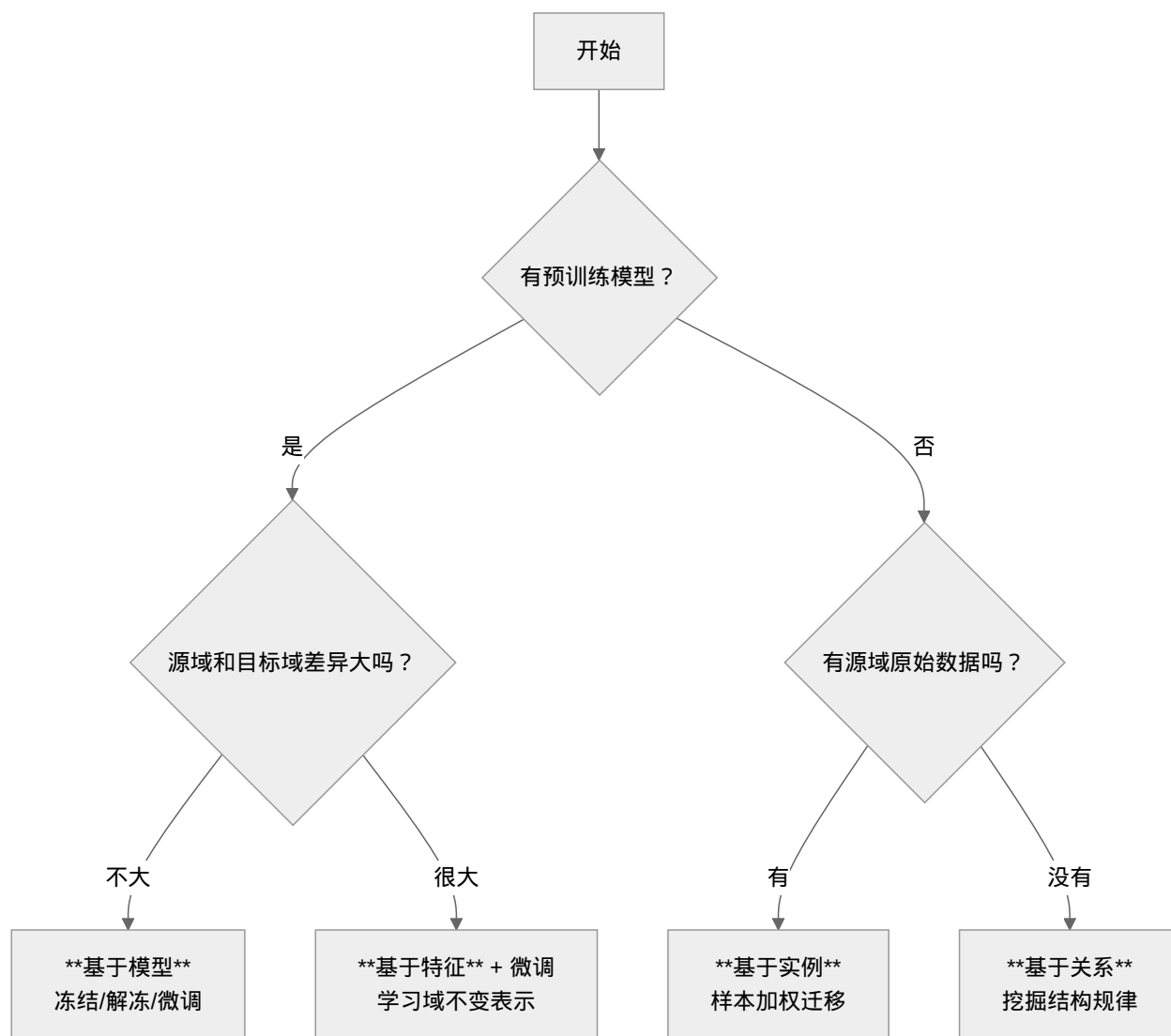


图 1: 迁移学习方法选择流程

- 基于模型：冻结/解冻层（最常用，下一章展开）
- 基于特征：对齐特征空间
- 基于实例：调整样本权重
- 基于关系：迁移逻辑规律

关于未展开内容

本章作为分类概览，没有深入以下内容：

- 基于实例的 TrAdaBoost 算法细节（涉及 Boosting 理论，超出本课程范围）
- 基于特征的 MMD/CORAL 等域适应算法（需要较深的概率论和核方法基础）
- 基于关系的知识图谱嵌入（偏 NLP 和图论方向）

高中读者建议：聚焦 [基于模型的迁移](#) 中的基于模型迁移——这是最实用、门槛最低的路径。其他三类方法的直觉理解本章已足够，当你未来遇到对应的实际需求时，再回来深挖不迟。

下一步

掌握了分类体系后，[基于模型的迁移](#) 我们将深入讲解[基于模型的迁移](#)——这是工业界最主流的方法。你将学会：如何加载预训练模型、[冻结哪些层 / 解冻哪些层](#)、怎样替换分类头，以及分层学习率和渐进解冻等核心技巧。

6.3 基于模型的迁移

迁移学习分类体系中，我们学习了迁移学习的分类体系。在实际工程中，[基于模型的迁移](#)是最常用、最有效的方法——直接复用预训练模型的参数和网络结构。

卷积：[发现装备](#) 中我们讨论了[归纳偏置](#)：CNN 通过架构设计将“局部相关性”的先验知识内置到模型中。预训练模型则更进一步——它不仅包含架构设计，还包含从海量数据中学到的[权重参数](#)，这些参数编码了边缘、纹理、形状等通用视觉特征。

关键问题：如何有效地利用这些预训练权重？是直接用它们提取特征，还是在目标任务上继续微调？本章将讲解两种核心策略及其适用场景。

6.3.1 预训练模型

预训练-微调范式

2018 年，BERT [DCLT19] 和 GPT [RNSS18] 的出现彻底改变了深度学习的研究范式：

1. **预训练阶段：**在大规模数据上学习通用表示

2. 微调阶段：在下游任务的小规模标注数据上调整模型

预训练模型学到的通用特征具有强大的迁移能力 —— 视觉模型学习边缘、纹理、形状等低级特征，语言模型学习语法、语义、世界知识。

经典预训练模型

计算机视觉：

模型	参数量	特点
ResNet-50	25M	残差连接，易于训练 [HZRS16]
EfficientNet-B0	5.3M	复合缩放，效率最优
ViT-Base	86M	Transformer 架构
CLIP	400M+	图文对齐，零样本能力 [RKH+21]

自然语言处理：

模型	参数量	特点
BERT-Base	110M	双向 Transformer [DCLT19]
GPT-2	1.5B	生成式预训练 [RWC+19]
GPT-3	175B	大规模生成模型 [BMR+20]
LLaMA-2	7B-70B	开源大语言模型 [TMS+23]

分层特征学习 —— 网络的"视觉系统"

深度神经网络就像一个分层的视觉系统，每一层负责不同层次的特征识别：

生活中的类比：识别一只猫

想象你走进一个房间，看到一只猫。你的大脑如何处理这个视觉信息？



神经网络也是这样的分层结构：

层级	学到的特征	通用性	迁移时的处理
浅层 (Layer1-2)	边缘、颜色、简单纹理	● 最通用	● 保留，不训练
中层 (Layer3)	形状、纹理组合	● 较通用	● 轻微调整
深层 (Layer4)	物体部件、复杂模式	● 任务相关	● 适度训练
分类层 (FC)	具体类别判断	● 最特定	● 完全替换

关键洞察：

- 浅层学到的"边缘检测"在任何视觉任务中都有用（猫的边缘和汽车的边缘是一样的）
- 深层学到的是"这像猫"的判断，换任务可能就不适用了
- 这就是为什么迁移学习能工作：底层技能通用，只需调整上层应用

6.3.2 策略一：特征提取器

核心思想：冻结预训练模型所有层，仅训练新的分类器。

工作流程：

```
import torchvision.models as models
import torch.nn as nn

# 加载预训练模型（来自 ImageNet 训练的 ResNet50）
# 这些权重编码了边缘、纹理、形状等通用视觉特征
model = models.resnet50(weights='IMAGENET1K_V1')

# 冻结所有层：requires_grad=False 表示不计算这些参数的梯度
# 这会"锁定"预训练知识，防止在训练中被改变
for param in model.parameters():
    param.requires_grad = False

# 替换分类头：原模型输出 1000 类（ImageNet），替换为目标类别数
# 只有这一层是新初始化的，需要从头学习
num_features = model.fc.in_features # 获取全连接层输入维度（ResNet50 是 2048）
model.fc = nn.Linear(num_features, num_classes) # num_classes 是你的任务类别数

# 优化器只传入新分类层的参数
# 见 {doc}`../pytorch-practice/optimiser` 中优化器参数设置
optimizer = torch.optim.Adam(model.fc.parameters(), lr=0.001)
```

适用场景：

- ☒ 目标数据集很小（<1000 样本）
- ☒ 目标域与预训练域高度相似
- ☒ 需要快速原型验证

优势：训练快、不易过拟合、计算资源要求低

6.3.3 策略二：微调

核心思想：在预训练权重基础上继续训练，可微调全部或部分层。

全量微调

```
import torchvision.models as models
import torch.nn as nn

# 加载预训练模型
model = models.resnet50(weights='IMAGENET1K_V1')

# 替换分类头为目标类别数
# 新分类层随机初始化，其他层保留预训练权重
model.fc = nn.Linear(model.fc.in_features, num_classes)

# 所有参数参与训练（不冻结任何层）
# 学习率通常比从头训练小一个数量级（1e-4 vs 1e-3）
# 防止过大的更新破坏预训练知识
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

分层微调——不同层用不同"学习速度"

直觉理解：

想象你在学习开车（新任务），但你已经会骑自行车（预训练）。

- **平衡感**（浅层）：骑车和开车都需要的技能 → 基本不用改
- **转向控制**（中层）：原理相似但操作不同 → 稍微调整
- **油门刹车**（深层）：全新的操作方式 → 重点学习
- **交通规则**（分类层）：完全不同的知识 → 从头学起

分层微调就是给不同层设置不同的"学习速度"：

```
import torch.optim as optim

# 分层学习率配置
# 原理：浅层（layer1-2）学习边缘/纹理等通用特征，应尽量保持不变
#       深层（layer3-4）学习物体部件等半通用特征，可适度调整
#       分类层（fc）完全针对新任务，需要大幅学习
optimizer = optim.Adam([
    {'params': model.layer1.parameters(), 'lr': 1e-5}, # 最浅层，最小学习率
```

(续下页)

(接上页)

```

{'params': model.layer2.parameters(), 'lr': 1e-5}, # 浅层特征
{'params': model.layer3.parameters(), 'lr': 1e-4}, # 中层特征
{'params': model.layer4.parameters(), 'lr': 1e-4}, # 深层特征
{'params': model.fc.parameters(), 'lr': 1e-3}      # 新分类层，最大学习率
1)

# 这种配置平衡了稳定性（浅层不变）和适应性（深层可调）
# 参见 {doc}`../pytorch-practice/optimiser` 中参数组配置

```

学习率设置的直觉：

层级	学习率	为什么这样设置	类比
Layer1-2	1e-5（很小）	边缘检测技能通用，别破坏它	平衡感，骑车开车都一样
Layer3-4	1e-4（中等）	形状识别可以微调	转向控制，原理相似
FC 层	1e-3（正常）	分类头针对新任务，需要大学习	交通规则，全新知识

渐进解冻（Progressive Unfreezing）——循序渐进地学习

直觉理解：

想象你在准备数学竞赛（新任务），但你已经学完了高中数学（预训练）。

渐进解冻就像是分阶段复习：

- 第 1 阶段：只刷竞赛真题（只训练分类层）
- 第 2 阶段：复习竞赛核心技巧（解冻深层）
- 第 3 阶段：回顾相关高中知识（解冻中层）
- 第 4 阶段：全面复习所有基础（解冻浅层）

如果一开始就全面复习，可能会因为内容太多而混乱。分阶段解冻让学习更有序。

核心思想：

- 深层特征更接近任务特定，应该优先调整
- 浅层特征更通用，应该后期才解冻
- 逐步解冻避免了“一次性放开所有参数”导致的训练不稳定

实现步骤：

```

import torchvision.models as models
import torch.nn as nn

```

(续下页)

(接上页)

```

import torch.optim as optim

def get_optimizer_for_layers(model, unfrozen_layers, base_lr=1e-4):
    """
    为不同层组设置参数
    unfrozen_layers: 要训练的层名称列表
    """
    params_to_update = []
    for name, param in model.named_parameters():
        # 检查参数是否属于解冻的层
        is_unfrozen = any(layer in name for layer in unfrozen_layers)
        if is_unfrozen:
            param.requires_grad = True
            # 分类层使用更大的学习率
            if 'fc' in name:
                params_to_update.append({'params': param, 'lr': base_lr * 10})
            else:
                params_to_update.append({'params': param, 'lr': base_lr})
        else:
            param.requires_grad = False

    return optim.Adam(params_to_update)

# 加载预训练模型
model = models.resnet50(weights='IMAGENET1K_V1')
model.fc = nn.Linear(model.fc.in_features, num_classes)

# 定义 ResNet50 的层结构（从浅到深）
# 见 {ref}`transfer-learning-model` 中分层特征学习的讨论
layer_groups = [
    ['fc'], # 阶段 1: 只训练分类层
    ['layer4', 'fc'], # 阶段 2: 解冻最深特征层
    ['layer3', 'layer4', 'fc'], # 阶段 3: 解冻中层
    ['layer2', 'layer3', 'layer4', 'fc'], # 阶段 4: 解冻更多
    ['layer1', 'layer2', 'layer3', 'layer4', 'fc'] # 阶段 5: 全部解冻
]

# 渐进解冻训练
epochs_per_stage = 5 # 每个阶段训练 5 个 epoch
for stage, layers in enumerate(layer_groups):

```

(续下页)

(接上页)

```

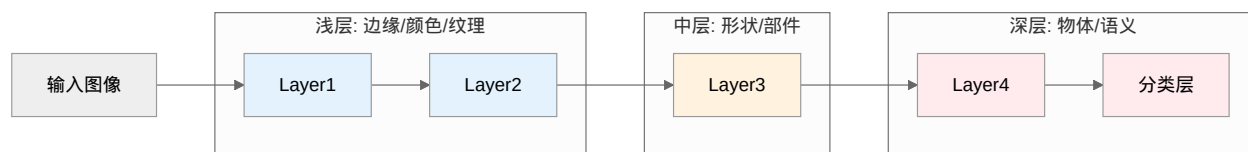
print(f"阶段 {stage + 1}: 训练层 {layers}")

# 为当前阶段创建优化器
optimizer = get_optimizer_for_layers(model, layers, base_lr=1e-5)

# 训练当前阶段
for epoch in range(epochs_per_stage):
    train_epoch(model, train_loader, optimizer, criterion)
    val_acc = validate(model, val_loader)
    print(f" Epoch {epoch+1}/{epochs_per_stage}, Val Acc: {val_acc:.2f}%")

```

为什么从顶层开始解冻?



- Layer1-2 (浅层): 学习边缘、颜色、纹理 —— **最通用**, 应该最后解冻
- Layer3 (中层): 学习形状、部件 —— **半通用**, 中期解冻
- Layer4+FC (深层): 学习物体、语义 —— **最任务相关**, 最先解冻

渐进解冻 vs 分层学习率:

方法	原理	训练速度	稳定性	适用场景
分层学习率	所有层同时训练, 但用不同 lr	快	中等	大多数场景
渐进解冻	逐层解冻, 分阶段训练	慢	高	数据集小、对稳定性要求高

优势: 避免对预训练权重的剧烈扰动, 训练更稳定, 特别适合小数据集。

适用场景

- ☒ 目标数据集中等规模 (1000-10000+样本)
- ☒ 目标域与预训练域有一定差异
- ☒ 追求最佳性能

学习率设置原则:

- 预训练权重: 1e-5 到 1e-4
- 新初始化层: 1e-3 到 1e-2

- 通常为从头训练学习率的 $1/10$ 到 $1/100$

6.3.4 策略选择

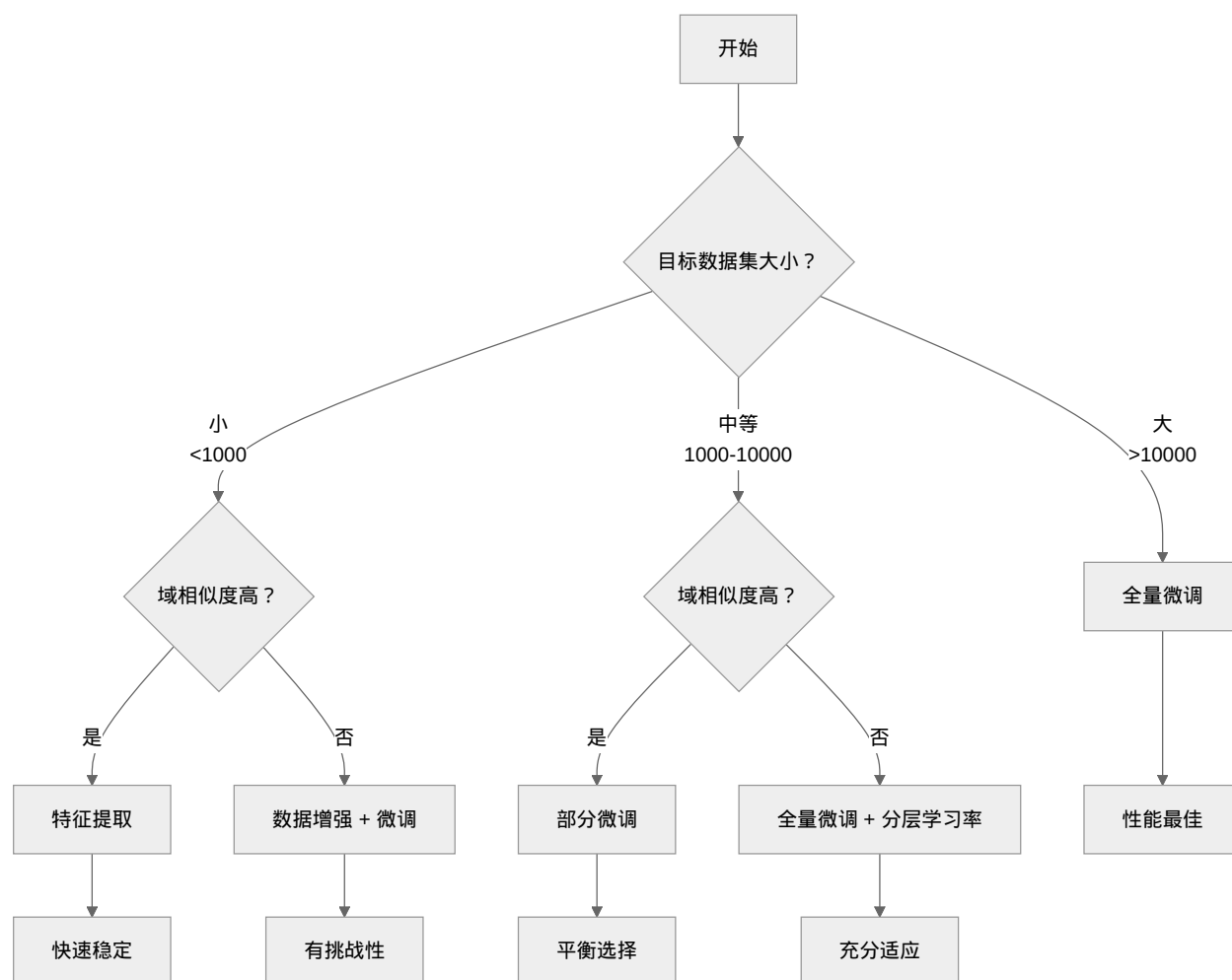


图 2: 迁移学习策略选择流程

6.3.5 参数高效微调 (PEFT)

随着模型规模急剧增长, 参数高效微调成为研究热点。

LoRA —— 只训练"差异"而不是全部

直觉得解:

想象你要学一门新方言 (新任务), 但你已经会普通话 (预训练)。

传统微调 = 重新学说话 (调整所有参数) **LoRA** = 只学「普通话和方言的差异」(只调整少量参数)

比如:

- 普通话: “吃饭了吗?”

- 方言：“食咗饭未？”
- 差异：发音不同，但语法结构相似

LoRA 就是只学习这些"差异规则"，而不是重新学习整个语言。

技术原理（简化版）：

假设预训练模型的某个权重是 W_0 （已经训练好的）。LoRA 不直接修改 W_0 ，而是学习一个"调整量"：

$$W = W_0 + \underbrace{BA}_{\text{小的调整}}$$

其中 B 和 A 是两个小矩阵，它们的乘积 BA 表示"如何调整"。

为什么这样有效？

- 原来的 W_0 有 $d \times k$ 个参数（比如 100 万）
- BA 只有 $(d \times r) + (r \times k)$ 个参数（比如 1 万，因为 r 很小）
- 只训练 1% 的参数，达到相似的效果

```
from peft import LoraConfig, get_peft_model

# r=16 表示只学习 16 个"调整方向"
# 就像只学 16 条方言转换规则，而不是重新学所有词汇
config = LoraConfig(r=16, target_modules=["q_proj", "v_proj"])
model = get_peft_model(base_model, config)
```

适用场景：

- 模型太大，显卡存不下全部梯度
- 需要训练很多个不同任务（每个任务只存小的 BA 矩阵）
- 想快速实验不同任务（训练更快）

其他 PEFT 方法

方法	可训练参数	适用场景
LoRA	~0.1-1%	性能与效率平衡（推荐）
Adapter	~2-4%	多任务/持续学习
Prefix Tuning	~0.1%	极端资源受限
BitFit	~0.1%	快速实验验证

6.3.6 本章小结

- **特征提取**：冻结预训练层，只训练分类头，适合小数据集
- **微调**：继续训练预训练权重，适合中大数据集
 - 分层学习率：浅层小 lr，深层大 lr
 - 渐进解冻：逐层解冻，训练更稳定
- **PEFT**：LoRA 等方法大幅降低训练成本

下一步

掌握了特征提取与微调的理论后，**实操指南** 我们将进入**实战环节**——如何判断你的数据集适合哪种策略？如何避免灾难性遗忘和过拟合？学习率到底该设多少？这些都是实践中必须面对的问题。

6.4 实操指南

基于模型的迁移 中我们学习了特征提取和微调两种策略。但实践中还有很多具体问题：

- 我的数据集应该用特征提取还是微调？
- 学习率设置多大？冻结哪些层？
- 为什么验证集准确率比训练集低这么多？

本章就是深度学习的"避坑指南"——聚焦实际应用中的关键决策点和常见陷阱，帮助你在真实项目中少走弯路。

6.4.1 数据集相似性判断

1. 图像领域

目标领域	迁移难度	建议
自然图像分类	低	特征提取或轻微微调
医学影像	中等	医学专用预训练模型 + 强数据增强
卫星遥感	中等	遥感专用预训练模型
工业缺陷	高	特征提取 + 数据增强 + 强正则化

2. 任务粒度

- **粗粒度分类**（猫 vs 狗）：域相似度高，特征提取即可
- **中等粒度**（犬种分类）：部分微调
- **细粒度分类**（鸟类物种）：全量微调 + 数据增强

3. 实用判断方法

1. **直接测试**：用预训练模型提取特征，训练简单分类器，性能>70%说明相似度高
2. **对比实验**：同时尝试特征提取和微调，选择验证集性能更好的策略

6.4.2 常见陷阱与对策

1. 灾难性遗忘 —— 学新忘旧

直觉理解：

想象你学了很多年英语（预训练），然后突然开始密集学法语（微调）。

如果学得太猛，你可能会发现：

- ☒ 法语进步很快
- ☒ 英语开始遗忘（以前记得的单词想不起来了）

这就是**灾难性遗忘**——新任务学得越好，旧知识忘得越多。

为什么会发生？

神经网络的权重就像是“记忆”。当你用新数据训练时，权重会被更新。如果学习率太大，新数据会“覆盖”旧记忆。

生活中的类比：

你的大脑有 100 个抽屉存储知识：

- 预训练时：英语知识分散存储在 80 个抽屉里
- 微调时：如果暴力地把法语知识塞进这些抽屉
- 结果：英语知识被挤掉了（灾难性遗忘）

解决方案：

- 只打开几个抽屉放法语（小学习率）
- 或者只增加新的小抽屉（LoRA）

缓解策略：

1. **使用较小学习率**（预训练权重： $1e-5$ 到 $1e-4$ ）
 - 就像学法语时，每天只学 1 小时，保留英语的时间
2. **分层学习率**（浅层更小 lr）
 - 基础的语法知识（浅层）不改动，只调整词汇用法（深层）
3. **使用 PEFT 方法**（LoRA、Adapter）

- 不为法语占用原有抽屉，而是新建一个小抽屉专门存差异

2. 过拟合 —— 死记硬背而不是真正学会

直觉理解：

想象你在准备数学考试，但只有 10 道练习题（小数据集）。

过拟合的表现：

- 你把这 10 道题的答案背得滚瓜烂熟（训练准确率 99%）
- 考试时遇到新题目，完全不会做（验证准确率 50%）
- 你只是"记住"了答案，没有"学会"解题方法

为什么迁移学习容易过拟合？

预训练模型有 2500 万个参数（ResNet50），而你的数据集可能只有 100 张图片。**参数太多，数据太少** —— 模型有足够的"记忆力"把每张图都背下来。

识别信号：

- 训练准确率持续上升（模型在学答案）
- 验证准确率停滞或下降（泛化能力差）
- 两者的差距越来越大（过拟合严重）

应对策略：

策略	直觉解释	怎么做
数据增强	让 10 道题变 100 道（变换角度、裁剪）	随机裁剪、翻转、颜色抖动
早停	不要背太狠，适可而止	验证不提升就停止
正则化	惩罚"死记硬背"，鼓励"理解"	weight_decay=1e-4
减少参数	减少"记忆力"，迫使"理解"	特征提取或 LoRA

3. 学习率设置不当

- **学习率过大**：损失震荡不收敛，破坏预训练权重 → 降低学习率
- **学习率过小**：收敛极慢，陷入局部最优 → 增大学习率，使用调度器

```
import torch.optim as optim

# 学习率调度器：当验证损失不再下降时自动降低学习率
# 这在微调时特别有用，防止后期学习率过大破坏预训练权重
# 参见 {ref}`learning-rate-scheduling` 中学习率调整策略
```

(续下页)

(接上页)

```

scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,      # 要调整的优化器
    mode='min',     # 监控指标越小越好（如损失）
    factor=0.1,     # 学习率衰减因子（新 lr = 旧 lr × 0.1）
    patience=5      # 等待 5 个 epoch，如果指标不改善就降低学习率
)

# 在训练循环中使用
for epoch in range(num_epochs):
    train(...)      # 训练
    val_loss = validate(...) # 验证
    scheduler.step(val_loss) # 根据验证损失调整学习率

```

4. 预处理不一致

预训练模型对输入数据有特定预处理要求，必须匹配：

```

from torchvision import transforms

# ImageNet 标准化参数（必须与预训练时使用的完全一致）
# 这些参数来自 ImageNet 数据集统计：mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]
# 参见 {doc}`../pytorch-practice/train-workflow` 中数据预处理部分
transform = transforms.Compose([
    transforms.Resize(256),      # 短边调整为 256，保持长宽比
    transforms.CenterCrop(224), # 中心裁剪为 224×224（ResNet 输入尺寸）
    transforms.ToTensor(),      # PIL Image → Tensor，并归一化到 [0,1]
    transforms.Normalize(       # 标准化：(x - mean) / std
        mean=[0.485, 0.456, 0.406], # ImageNet R,G,B 通道均值
        std=[0.229, 0.224, 0.225]   # ImageNet R,G,B 通道标准差
    )
])

```

关键检查点：

- 输入尺寸匹配（通常 224×224）
- 使用正确的归一化参数（必须与预训练一致）
- 颜色通道顺序正确（RGB，不是 BGR）

6.4.3 项目检查清单

项目启动前

- 评估目标数据集大小与质量
- 判断与预训练域的相似性
- 选择合适的预训练模型
- 确定迁移策略

模型构建

- 正确加载预训练权重
- 替换输出层为目标类别数
- 冻结/解冻正确的层
- 实现正确的数据预处理

训练阶段

- 设置合理的学习率（分层或统一）
- 添加早停机制
- 使用数据增强
- 监控训练和验证指标

评估迭代

- 在独立测试集上评估
- 分析错误样本
- 对比不同策略效果

6.4.4 高级技巧

测试时增强（TTA）

核心思想：对同一张测试图像应用不同的数据增强，取多个预测的平均，降低预测方差。

为什么有效？

- 模型对图像的微小变化敏感
- 多视角预测取平均，减少随机性
- 类似于“投票机制”，更稳定

```

import torch
import torchvision.transforms as transforms
from PIL import Image

def tta_predict(model, image_path, num_augmentations=5):
    """
    测试时增强预测

    Args:
        model: 训练好的模型
        image_path: 测试图像路径
        num_augmentations: TTA 增强次数

    Returns:
        平均预测概率
    """
    model.eval()

    # 基础预处理（必须保持与训练一致）
    base_transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    # TTA 增强变换（在基础变换上增加随机性）
    tta_transforms = [
        # 原始图像
        base_transform,
        # 水平翻转
        transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(224),
            transforms.RandomHorizontalFlip(p=1.0), # 必定翻转
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ]),
        # 轻微旋转
        transforms.Compose([

```

(续下页)

(接上页)

```

        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.RandomRotation(degrees=(-5, 5)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    )),
    # 偏移裁剪 (左上角)
    transforms.Compose([
        transforms.Resize(256),
        transforms.Lambda(lambda img: transforms.functional.crop(img, 0, 0, 224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    )),
    # 偏移裁剪 (右下角)
    transforms.Compose([
        transforms.Resize(256),
        transforms.Lambda(lambda img: transforms.functional.crop(img, 32, 32, 224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    )),
]

# 加载原始图像
image = Image.open(image_path).convert('RGB')

# 收集所有预测
all_predictions = []
with torch.no_grad():
    for transform in tta_transforms[:num_augmentations]:
        input_tensor = transform(image).unsqueeze(0).to(device)
        output = model(input_tensor)
        probs = torch.softmax(output, dim=1)
        all_predictions.append(probs)

# 取平均
avg_prediction = torch.stack(all_predictions).mean(dim=0)
return avg_prediction

# 使用示例
prediction = tta_predict(model, 'test_image.jpg', num_augmentations=5)

```

(续下页)

(接上页)

```

predicted_class = prediction.argmax(dim=1).item()
confidence = prediction.max().item()
print(f"预测类别: {predicted_class}, 置信度: {confidence:.4f}")

```

TTA 使用建议:

场景	推荐增强数	预期提升
追求速度	2 (原图+水平翻转)	0.5-1%
平衡方案	5 (上述代码)	1-2%
追求精度	10+ (更多裁剪/旋转)	2-3%

注意事项:

- TTA 只在**测试阶段**使用, 不增加训练时间
- 对于已经过拟合的模型, TTA 提升有限
- 分类任务效果最明显, 检测/分割任务需要适配

集成学习

使用多个预训练模型集成, 进一步提升性能:

```

import torch
import torchvision.models as models

def ensemble_predict(model_paths, image_path, weights=None):
    """
    多模型集成预测

    Args:
        model_paths: 多个模型路径列表
        image_path: 测试图像
        weights: 各模型权重 (None 表示等权重)
    """
    # 加载多个模型
    models_list = []
    for path in model_paths:
        model = models.resnet50(weights=None)
        model.fc = nn.Linear(model.fc.in_features, num_classes)
        model.load_state_dict(torch.load(path))

```

(续下页)

(接上页)

```

    model.eval()
    models_list.append(model.to(device))

# 预处理
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

image = Image.open(image_path).convert('RGB')
input_tensor = transform(image).unsqueeze(0).to(device)

# 收集各模型预测
all_probs = []
with torch.no_grad():
    for model in models_list:
        output = model(input_tensor)
        probs = torch.softmax(output, dim=1)
        all_probs.append(probs)

# 加权平均
if weights is None:
    weights = [1.0 / len(models_list)] * len(models_list)

ensemble_prob = sum(w * p for w, p in zip(weights, all_probs))
return ensemble_prob

# 使用不同架构集成（更强的集成效果）
# ResNet50 + EfficientNet-B0 + ViT-Base
ensemble_models = [
    'resnet50_best.pth',
    'efficientnet_best.pth',
    'vit_best.pth'
]

prediction = ensemble_predict(ensemble_models, 'test.jpg')

```

集成策略对比：

集成类型	方法	效果	计算成本
同架构不同初始化	相同模型，不同随机种子	★★☆	低
不同架构	ResNet + EfficientNet + ViT	★★★	中
不同预训练数据	ImageNet + 医学数据 + 领域数据	★★★	高
TTA + 集成	先 TTA 再集成	★★★★	很高

集成学习的前提：

- 单个模型性能不能太差（至少>70%）
- 模型间要有**差异性**（不能是完全相同的预测）
- 模型数量不是越多越好，通常 3-5 个足够

项目案例：医学影像分类实战

下面通过一个完整案例，展示迁移学习在真实项目中的应用。

问题背景

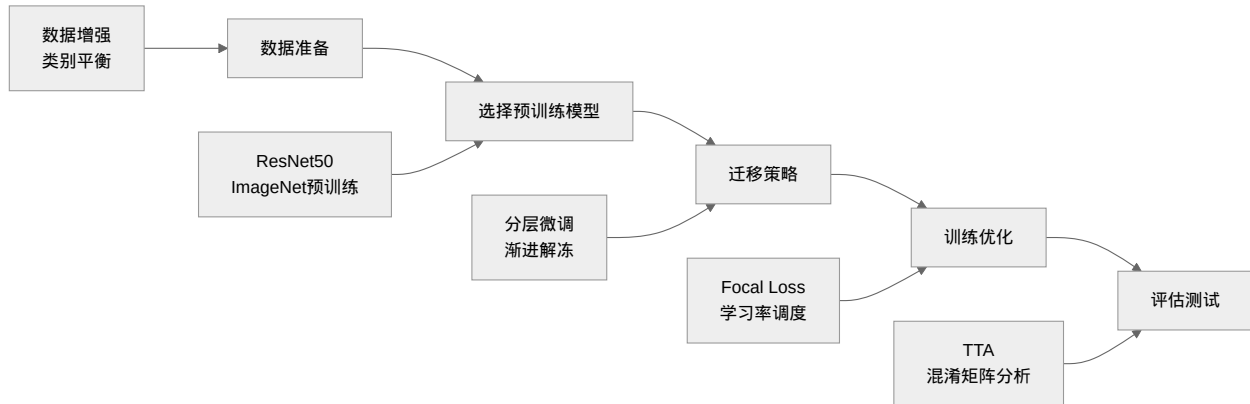
任务：从胸部 X 光片中识别肺炎**数据：**

- 训练集：正常 1,341 张，肺炎 3,876 张（类别不平衡）
- 验证集：正常 234 张，肺炎 390 张
- 测试集：正常 234 张，肺炎 390 张
- 图像尺寸：原始 1024×1024，预处理为 224×224

挑战：

1. 数据量小（总计约 5,000 张）
2. 类别不平衡（正常: 肺炎 $\approx$ 1:3）
3. 医学图像与自然图像差异大

方案设计与实现



步骤 1: 数据准备与增强

```

import torchvision.transforms as transforms
from torch.utils.data import DataLoader, WeightedRandomSampler

# 训练时增强（强增强）
train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=(-10, 10)),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# 验证/测试时预处理（无增强）
val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# 处理类别不平衡：计算每个类别的权重
class_counts = [1341, 3876] # 正常, 肺炎
class_weights = 1. / torch.tensor(class_counts, dtype=torch.float)
sample_weights = [class_weights[label] for _, label in train_dataset]
sampler = WeightedRandomSampler(sample_weights, len(sample_weights), replacement=True)

```

(续下页)

(接上页)

```
train_loader = DataLoader(train_dataset, batch_size=32, sampler=sampler)
```

步骤 2: 模型构建 (分层微调)

```
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim

# 加载预训练 ResNet50
model = models.resnet50(weights='IMAGENET1K_V1')

# 替换分类头为 2 类 (正常/肺炎)
model.fc = nn.Linear(model.fc.in_features, 2)

# 分层学习率配置
# 医学影像与自然影像差异较大, 需要适度调整浅层
optimizer = optim.Adam([
    {'params': model.layer1.parameters(), 'lr': 1e-5},
    {'params': model.layer2.parameters(), 'lr': 1e-5},
    {'params': model.layer3.parameters(), 'lr': 5e-5},
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3}
])

# 使用 Focal Loss 处理类别不平衡
class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2.0):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, inputs, targets):
        ce_loss = nn.functional.cross_entropy(inputs, targets, reduction='none')
        pt = torch.exp(-ce_loss)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * ce_loss
        return focal_loss.mean()

criterion = FocalLoss(alpha=0.25, gamma=2.0)
```

步骤 3: 训练循环与早停

```

best_val_acc = 0
patience = 5
patience_counter = 0

for epoch in range(50): # 最多训练 50 轮
    # 训练阶段
    model.train()
    train_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()

    # 验证阶段
    model.eval()
    val_correct = 0
    val_total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = outputs.max(1)
            val_total += labels.size(0)
            val_correct += predicted.eq(labels).sum().item()

    val_acc = 100. * val_correct / val_total
    print(f"Epoch {epoch}: Train Loss={train_loss/len(train_loader):.4f}, Val Acc={val_acc:.2f}%
    ↪")

    # 早停机制
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), 'best_model.pth')

```

(续下页)

(接上页)

```
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        print(f"早停触发, 最佳验证准确率: {best_val_acc:.2f}%")
        break
```

步骤 4: 评估与 TTA

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np

# 加载最佳模型
model.load_state_dict(torch.load('best_model.pth'))
model.eval()

# 使用 TTA 进行测试
def evaluate_with_tta(model, test_loader, num_tta=5):
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)

            # TTA 预测
            batch_preds = []
            for _ in range(num_tta):
                # 添加轻微噪声进行增强
                noisy_images = images + torch.randn_like(images) * 0.01
                outputs = model(noisy_images)
                probs = torch.softmax(outputs, dim=1)
                batch_preds.append(probs)

            # 平均预测
            avg_probs = torch.stack(batch_preds).mean(dim=0)
            _, predicted = avg_probs.max(1)

            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.numpy())
```

(续下页)

(接上页)

```
    return np.array(all_preds), np.array(all_labels)

predictions, labels = evaluate_with_tta(model, test_loader, num_tta=5)

# 打印详细评估报告
print("分类报告: ")
print(classification_report(labels, predictions, target_names=[' 正常', ' 肺炎']))

# 混淆矩阵
cm = confusion_matrix(labels, predictions)
print("混淆矩阵: ")
print(cm)
```

实验结果

方法	准确率	召回率（肺炎）	F1 分数
从头训练	82.3%	78.5%	0.81
特征提取	85.1%	82.3%	0.84
分层微调（本方案）	91.7%	90.2%	0.91
+ TTA	92.4%	91.0%	0.92

关键洞察：

- 1. 分层学习率有效平衡了预训练知识的保留与新任务的适应
- 2. Focal Loss 显著改善了类别不平衡问题
- 3. TTA 在测试阶段提供了稳定的性能提升

调试记录

问题 1：初期验证准确率停留在 75%左右

- 诊断：学习率过大，破坏了预训练权重
- 解决：将预训练层学习率从 1e-3 降至 1e-5

问题 2：肺炎类别召回率低（仅 70%）

- 诊断：类别不平衡导致模型偏向多数类
- 解决：使用 WeightedRandomSampler + Focal Loss

问题 3: 训练集准确率>95%，验证集<80%

- **诊断:** 过拟合，数据增强不够
- **解决:** 增加 ColorJitter 和 RandomRotation

持续学习

模型需持续适应新任务时：

- 使用 PEFT 方法（LoRA、Adapter）
- 考虑经验回放或 EWC 等持续学习方法 [KPR+17]

6.4.5 本章小结

数据集相似性判断: 内容相似性、任务粒度、量化方法

常见陷阱:

- 灾难性遗忘 → 小学习率、PEFT
- 过拟合 → 数据增强、早停、正则化
- 学习率问题 → 分层学习率、调度器
- 预处理 → 使用预训练时的预处理参数

核心理念: 站在巨人的肩膀上，善用预训练知识。

下一步

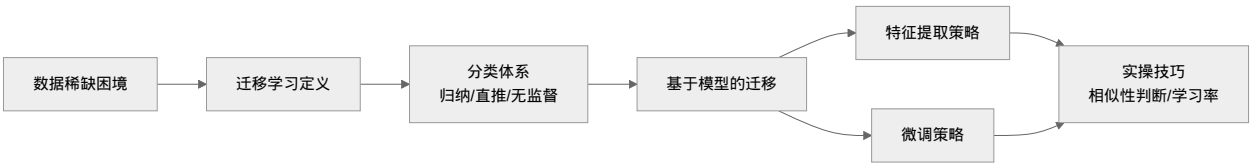
完成了实操指南的学习，你已经掌握了迁移学习的完整技能栈。**结语:** 站在巨人的肩膀上 中，我们将回顾本章知识、整理学习资源、推荐动手项目，并展望接下来的学习路径。

6.5 结语：站在巨人的肩膀上

恭喜！你已经完成了迁移学习章节的学习。

从 **导论与核心理念** 中的数据困境与计算成本，到 **迁移学习分类体系** 中的分类体系，再到 **基于模型的迁移** 中的特征提取与微调策略，最后到 **实操指南** 中的实操技巧——你已经掌握了**利用预训练模型解决实际问题的完整方法论**。

6.5.1 本章学习路径回顾

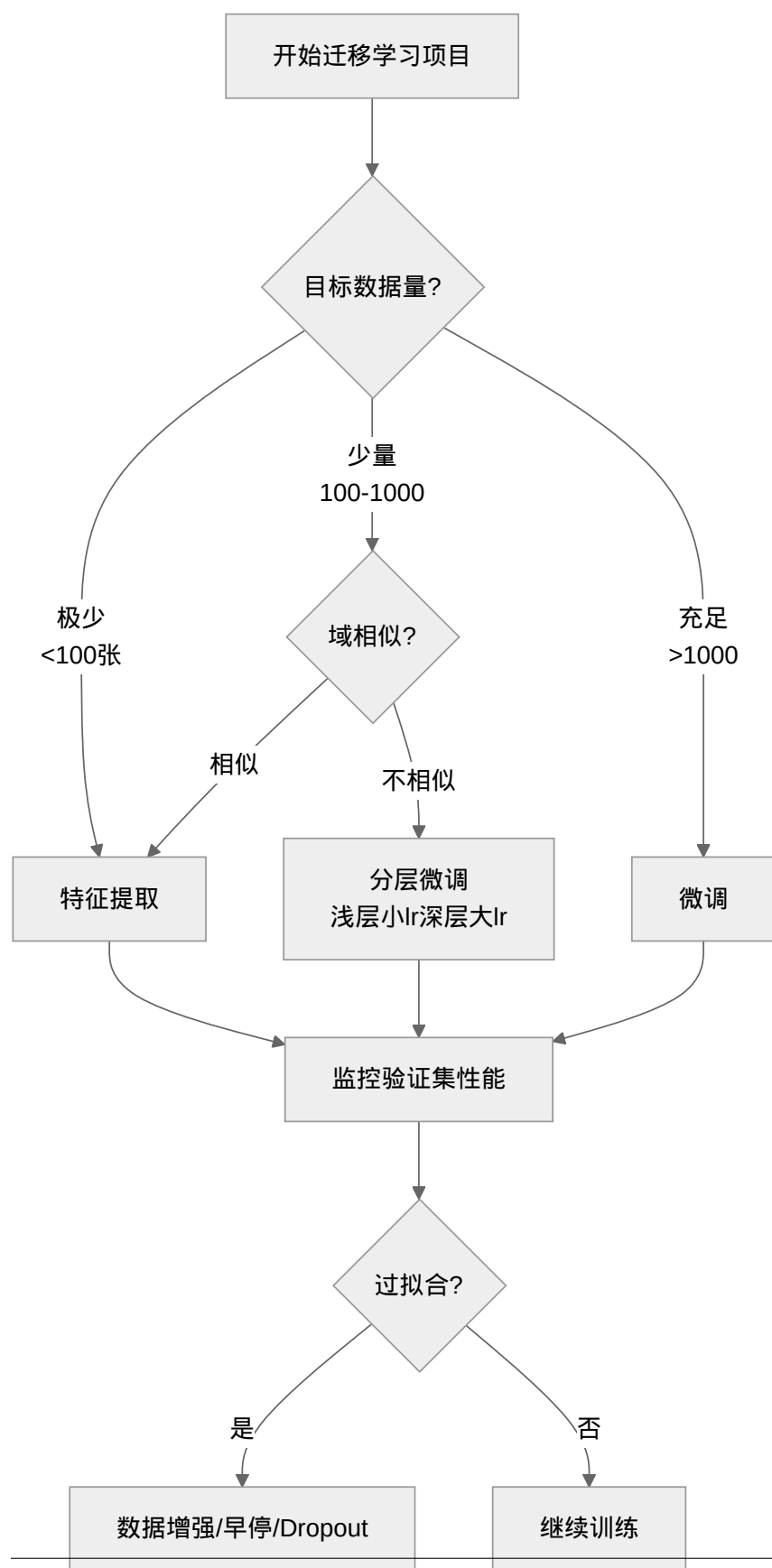


6.5.2 核心知识点总结

迁移学习核心概念映射

概念	直观理解	应用场景
源域 (Source Domain)	预训练模型的"学习背景"	ImageNet、大规模语料
目标域 (Target Domain)	你的实际任务	医学影像、小众语言
特征提取	“冻结经验，只学分类”	数据极少、域相似
微调	“在经验基础上继续学习”	数据充足、域有差异
灾难性遗忘	“学新忘旧”	学习率过大时
负迁移	“帮倒忙”	源域与目标域无关

决策流程速查



6.5.3 与前面章节的联系

迁移学习是 PyTorch 实践：把理论变成代码 中技能的高级应用：

前置知识	本章应用
神经网络模块：搭建计算图	加载预训练模型、替换分类头
优化器：用梯度更新参数	分层学习率、冻结参数
完整训练流程	完整训练流程、早停机制
正则化：防止死记硬背的四种方法	防止微调过拟合
归纳偏置	预训练权重编码了通用先验

核心认知：迁移学习不是新技术，而是前面所有知识的**综合运用**——你仍然在使用相同的 PyTorch API，只是站在了巨人的肩膀上。

6.5.4 关键数字记忆

场景	建议学习率	预期效果
特征提取	1e-3	快速收敛，不易过拟合
全量微调	1e-4 ~ 1e-5	需要更多 epoch，但效果更好
分层微调	浅层 1e-5，深层 1e-4	平衡稳定性与适应性

6.5.5 推荐学习资源

动手项目（从简单到复杂）

项目名称	难度	练习重点	参考章节
猫狗分类器（迁移版）	☆ 入门	用 ResNet50 做特征提取	基于模型的迁移
CIFAR-10 微调	☆☆ 基础	全量微调与数据增强	实操指南
医学影像分类	☆☆☆ 进阶	分层微调、处理类别不平衡	完整流程
多语言情感分析	☆☆☆ 进阶	BERT 微调、处理文本数据	NLP 迁移

必读论文

按阅读顺序排列：

1. **How transferable are features in deep neural networks?** (Yosinski et al., 2014) - 特征可迁移性的经典分析
2. **ImageNet Classification with Deep Convolutional Neural Networks** (AlexNet, 2012) - 预训练范式的开端
3. **BERT: Pre-training of Deep Bidirectional Transformers** (Devlin et al., 2018) - 预训练+微调范式的里程碑
4. **LoRA: Low-Rank Adaptation of Large Language Models** (Hu et al., 2021) - 参数高效微调的代表作

预训练模型资源

- **PyTorch Hub**: torchvision.models、transformers 库
 - **Hugging Face**: 最大的预训练模型社区
 - **timm**: PyTorch Image Models, 计算机视觉模型的宝库
-

6.5.6 下一步

掌握了迁移学习后，你已经可以处理大多数实际的深度学习项目。接下来可以考虑：

- **更深入的架构学习**：ResNet、EfficientNet、Vision Transformer 的内部机制
 - **生成模型**：GAN、VAE、Diffusion Models
 - **大语言模型应用**：Prompt Engineering、RAG、Agent
 - **部署与工程**：模型量化、TensorRT、ONNX
-

6.5.7 总结

迁移学习的精髓可以用一句话概括：

不要重复造轮子 —— 善用他人的知识，专注于你的创新。

从"会训练模型"进化到"会用模型解决问题"，你已经迈出了关键的一步。

U-Net：图像分割的革命

7.1 引言：图像分割问题

7.1.1 从分类到分割：问题升级

卷积：发现装备 中我们学的 CNN 擅长做**图像分类**：输入一张 28×28 的手写数字，输出“这是 3”。关键在于，分类只需要回答“整体是什么”——即使你只看到数字的一小部分，也能猜出来。

但现实中有很多问题需要“**每个点是什么**”：

- **医学影像**：CT 图像中每个像素是肿瘤还是正常组织？
- **自动驾驶**：每个像素是道路、行人、还是天空？
- **卫星图像**：每个像素是农田、建筑、还是水域？

这就是**图像分割（Image Segmentation）**——对每个像素分配一个类别标签。一张 256×256 的图像有 65,536 个像素，每个都需要预测。

学习目标

完成本章后，你将能够：

1. **理解分割任务**：图像分割是什么、为什么比分类更难、和 CNN 的关联
2. **掌握 U-Net 架构**：U 形结构、编码器-解码器、跳跃连接的直觉和原理

3. **理解跳跃连接的价值**：为什么"抄近道"能同时保留位置和语义
4. **能够实现 U-Net**：从核心组件到完整模型
5. **了解损失函数**：Dice 损失为什么比交叉熵更适合分割任务

分类 vs 分割的直觉

分类像考试判断题：“这张图里有猫吗？”——看一眼就能答。

分割像做填色游戏：你需要精确地沿着猫的轮廓涂色，一根胡须都不能漏。

后者需要**同时**知道两件事：

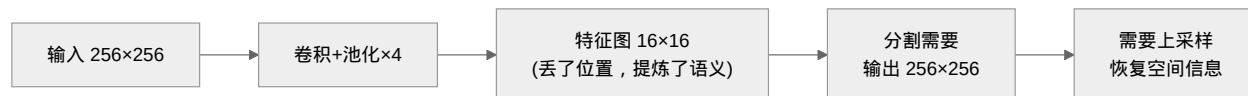
1. “**这是什么**”（语义理解）：那是猫，不是狗
2. “**它在哪**”（空间精度）：猫的边界到第 237 行 189 列

7.1.2 为什么分类网络不能直接用来做分割？

直觉上，你可能想把全连接层换成“每个像素一个分类器”。但这里有一个根本矛盾：

卷积：发现装备 中我们学了 CNN 的编码过程：卷积+池化逐步**下采样**，空间尺寸从 28×28 缩小到 1×1 。这是分类成功的关键——**丢掉位置信息，提炼语义信息**（**感受野**告诉我们，深层神经元看的区域大，能理解整体）。

但分割需要**同时保留位置和语义**。下采样丢了位置，直接去不掉。



这就引出了核心问题：**如何把缩小的特征图恢复回原始分辨率，同时不丢失细节？**

7.1.3 U-Net 的回答：编码器-解码器 + 跳跃连接

U-Net [RFB15] 的答案今天看来简单而优雅：

1. **编码器 (Encoder)**：就是标准的 CNN，逐步下采样提取语义——跟你学的 LeNet-5 ([LeNet-5: 练习峰首登](#)) 前半部分一样
2. **解码器 (Decoder)**：对称的上采样路径，逐步恢复空间分辨率
3. **跳跃连接 (Skip Connection)**：把编码器每一层的特征直接“抄近道”送到对应层的解码器

右边和左边是对称的，中间是跳跃连接——形成了 U 形。[U 形架构详解](#) 中有更详细的架构 mermaid 图。

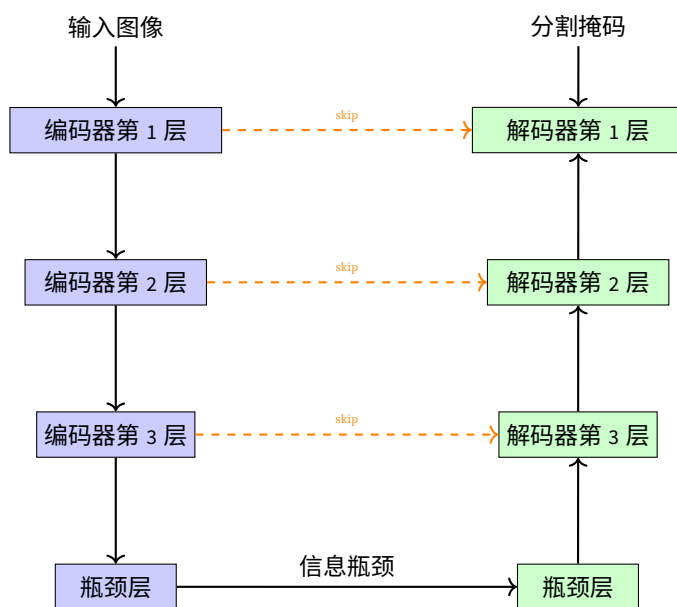


图 1: U 形架构概念图

直觉：为什么 U 形能解决"位置 vs 语义"的矛盾？

你在做一个"从缩略图还原高清图"的任务：

- **缩略图**虽然模糊，但告诉你"整体是个什么东西"（语义）
- 你手边有一叠**原始高清图的局部截图**（编码器每层的特征）
- **最好的办法**：看着缩略图知道整体形状，再参考每张局部截图还原细节

编码器各层的特征图就是那些"局部截图"——浅层保留精确位置，深层提炼抽象语义。跳跃连接把它们直接送到解码器对应层，让解码器**同时拥有位置**和语义。

与 **感受野** 中讨论的感受野递推类似，U-Net 每层特征图的感受野不同：

- 浅层：小感受野，知道"边缘在哪个像素"
- 深层：大感受野，知道"这是肿瘤还是正常组织"
- 跳跃连接：**同时给小感受野和大感受野的信息**

7.1.4 U-Net 的历史意义

U-Net 由 Olaf Ronneberger 等人在 2015 年提出 [RFB15]，初衷是生物医学图像分割。它的关键贡献不是发明了新东西（卷积、池化、跳跃连接都是现成的），而是**把这些组件组合成了一个优雅的对称结构**，恰好解决了分割问题的核心矛盾。

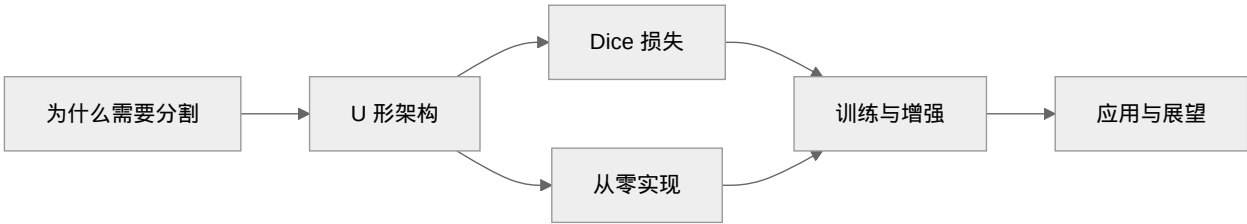
在当年的 ISBI 细胞分割挑战赛上，U-Net 用 **30 张训练图像**就达到了超越所有传统方法的效果——Dice 系数 0.775 vs 第二名 0.460。这在深度学习以"大数据"为王的年代，显得格外引人注目。

与 FCN 的关键区别

在 U-Net 之前，全卷积网络（FCN）[LCWJ15] 已经尝试用 CNN 做分割。但 FCN 的上采样只用一次双线性插值，跳跃连接也只是简单相加。U-Net 做了两个关键改进：

改进	FCN	U-Net
上采样方式	一次双线性插值（不可学习）	逐层转置卷积（可学习）
跳跃连接	稀疏，简单相加	密集，通道拼接 （保留全部信息）
输出分辨率	较低（32 倍下采样）	与输入相同

7.1.5 本章路线



下一节 [U 形架构详解](#) 我们深入 U 形架构的内部，看编码器、解码器和跳跃连接是如何协同工作的。

7.2 U 形架构详解

7.2.1 直觉：U-Net 的整体结构

引言：图像分割问题 我们提出了核心问题：**如何把缩小的特征图恢复回原始分辨率，同时不丢失空间细节？**

U-Net[RFB15] 的答案是一个对称的 U 形：

架构速览

从图中可以看出核心设计：左侧编码器逐步下采样（空间缩小、通道增加），右侧解码器逐步上采样（空间恢复、通道减少），底部是信息瓶颈，灰色箭头是**跳跃连接**。

几个关键数字：

- **输入** $572 \times 572 \times 1 \rightarrow$ **输出** $388 \times 388 \times 2$ （有效填充导致边界丢失 92 像素）
- **通道数变化**： $1 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 512 \rightarrow \dots \rightarrow 64 \rightarrow 2$
- **每层跳跃连接拼接**：编码器特征（保留精确位置）与解码器特征（携带语义信息）在通道维度拼接

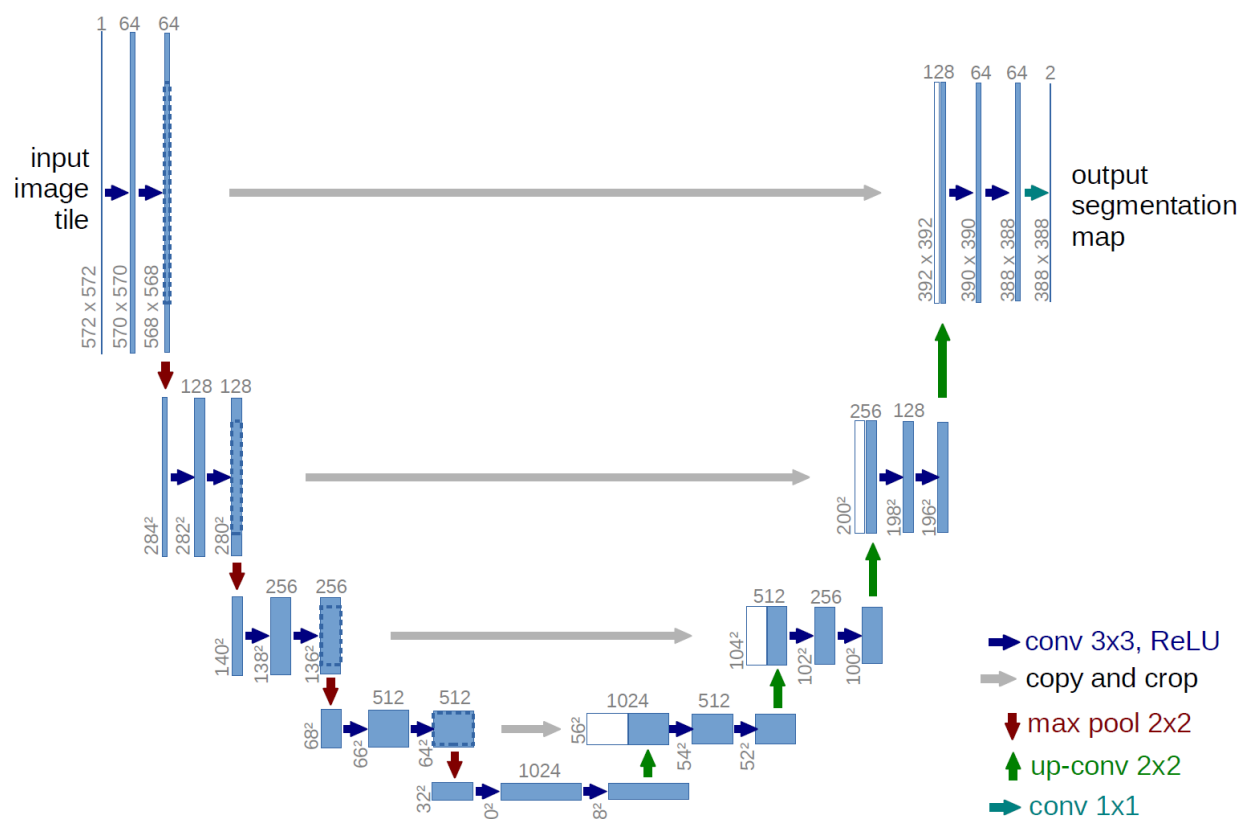


图 2: U-Net 的 U 形对称架构

三部分的角色

部分	做什么	效果
编码器 (左半)	卷积+池化, 逐步下采样	空间变小, 通道变多, 语义变强
瓶颈 (底部)	最深的卷积, 特征最抽象	感受野最大, 知道"整体是什么"
解码器 (右半)	转置卷积+跳跃拼接, 逐步上采样	空间变大, 恢复细节

7.2.2 编码器路径：提取语义

编码器和 LeNet-5：练习峰首登 前半部分几乎一样 —— 重复的卷积块 + 最大池化。

编码器的直觉

每一步都在做"退一步看全局":

- 3×3 卷积看局部模式 (边缘、纹理)
- 2×2 池化取最显著特征, 空间减半
- 通道数加倍, 给更多"记忆空间"存储提炼出的信息

就像写摘要：先逐段阅读 (卷积)，再浓缩成要点 (池化)。反复几次后，你就从具体的文字得到了文章主旨。

编码器中的感受野

感受野 中我们学了感受野的递推公式。U-Net 中每个编码器块的感受野：

层	累积感受野	看到什么
第 1 层	3×3	边缘、角点、纹理
第 2 层	5×5	简单的形状组合
第 3 层	9×9	局部结构模式
第 4 层	17×17	较大的语义部件
底部	33×33	完整的语义对象

感受野逐层扩大，意味着一路向下，每个神经元看到的输入区域越来越大，学到的特征从"具体边缘"变成了"抽象语义"。

特征图尺寸变化公式

原始 U-Net 用有效填充，每步卷积后尺寸变化为：

$$H_{\text{out}} = H_{\text{in}} - k + 1$$

其中 $k = 3$ 是卷积核大小，所以每次卷积后宽高各减 2。池化后尺寸减半。

7.2.3 解码器路径：恢复空间

解码器是编码器的"镜像"——把缩小的特征图一步步放大回原始分辨率。核心操作有两个：

1. 上采样 (Upsampling)

U-Net 使用**转置卷积** (Transposed Convolution) 进行上采样。直觉上，它和卷积做的事相反：

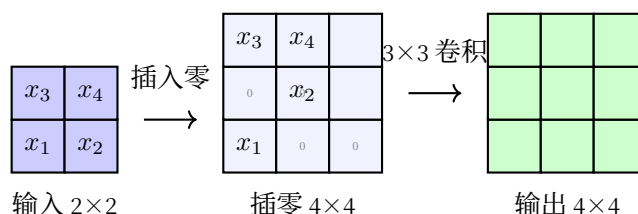


图 3: 转置卷积: $2 \times 2 \rightarrow 4 \times 4$

转置卷积的直觉

普通卷积: 3×3 窗口滑过输入 $\rightarrow$ 输出更小
转置卷积: 每个输入点"膨胀"成 2×2 区域 $\rightarrow$ 输出更大

转置卷积通过在每个输入元素之间插入零值，然后做标准卷积来实现尺寸加倍。它的参数也是**可学习的**，不像双线性插值是固定的。

2. 跳跃连接 (Skip Connection)

这是 U-Net 的灵魂。上采样后的特征图与**编码器对应层的特征图在通道维度上拼接** (concatenate)。

$$F_{\text{concat}} = \text{concat}(F_{\text{encoder}}, F_{\text{decoder}}) \in \mathbb{R}^{H \times W \times (C_{\text{enc}} + C_{\text{dec}})}$$

为什么拼接而不是相加?

FCN 用相加，U-Net 用拼接。拼接**保留了编码器特征的全部信息** (不压缩)，让解码器可以"看到"原始细节。

跳跃连接的三个作用

1. **保留空间信息**: 编码器浅层知道"边缘在第 35 行"，解码器需要这个信息

2. **改善梯度流动**：梯度可以"抄近道"直接传到浅层，缓解梯度消失
3. **多尺度特征融合**：不同感受野的特征同时被利用

跳跃连接的梯度分析

跳跃连接对训练的最大贡献是**解决了梯度消失问题**。梯度消失 (Vanishing Gradient) 中我们详细讨论过梯度消失的成因——反向传播时梯度经过多层连乘会指数级衰减，导致浅层无法学习。

要理解跳跃连接为什么能缓解这个问题，需要先理解定义：**Jacobian 矩阵** 的概念 (详见 [反向传播算法](#))。核心洞察是：深层网络的梯度计算等于**多层 Jacobian 矩阵的连乘**。

没有跳跃连接时：Jacobian 连乘

链式法则告诉我们，损失 L 对第 l 层权重 W_l 的梯度相当于**每一层 Jacobian 的连乘**：

$$\frac{\partial L}{\partial W_l} = \frac{\partial L}{\partial \hat{y}} \cdot \underbrace{\frac{\partial h_L}{\partial h_{L-1}} \cdot \frac{\partial h_{L-1}}{\partial h_{L-2}} \cdots \frac{\partial h_{l+1}}{\partial h_l}}_{L-l \text{ 个 Jacobian 相乘}}$$

每乘一个 Jacobian 都可能缩小或放大梯度。如果每层的"放大倍数"都小于 1， $L-l$ 个 Jacobian 连乘后梯度就会指数级衰减——梯度消失。

有了跳跃连接：梯度抄近道

跳跃连接在反向传播中创建了一条**直达路径**，绕过了中间深度：

$$\frac{\partial L}{\partial W_l} = \underbrace{\text{主路径 (} L-l \text{ 个 Jacobian 连乘)}}_{\text{可能消失}} + \underbrace{\text{跳跃路径 (仅 1-2 个 Jacobian)}}_{\text{几乎不衰减}}$$

第二条路径的 Jacobian 链极短——编码器第 2 层的梯度可以通过跳跃连接直接传到解码器第 2 层，中间只经过 1-2 个变换，而不是从底部绕上来的 8-10 个。

数值对比：64 倍的差距

假设每层 Jacobian 的放大倍数 = 0.8：

原始路径（穿过 20 层）： $0.8^{20} \approx 0.01$ （衰减了 99%）

跳跃路径（只穿过 2 层）： $0.8^2 \approx 0.64$ （只衰减了 36%）

差距：0.01 vs 0.64，整整 **64 倍**。有了跳跃连接，浅层收到的梯度信号强了几十倍。

这就是跳跃连接被称为"梯度高速公路"的原因——它为梯度提供了**绕过深度、直达浅层**的捷径。这个思路与 ResNet [HZRS16] 的残差连接一脉相承：都是通过短路连接为梯度创造高速公路。[实验设计](#) 中的消融实验也可以验证：去掉跳跃连接后，深层网络的浅层几乎学不到东西。

7.2.4 输出层设计

输出层用 1×1 卷积把通道数映射到类别数：

$$\text{输出} = \text{Conv2D}(C_{\text{in}}, C_{\text{out}}, \text{kernel_size} = 1)$$

对于二分类任务（如细胞 vs 背景）， $C_{\text{out}} = 2$ ，后接 softmax。每个像素的 2 个值表示“属于背景的概率”和“属于细胞的概率”。

7.2.5 为什么 U 形架构这么成功？

回顾 引言：图像分割问题 中讨论的核心矛盾，现在看 U-Net 如何一一解决：

矛盾	编码器的做法	解码器的做法	跳跃连接的作用
需要语义理解	下采样扩大感受野	-	深层特征传到解码器
需要空间精度	浅层保留边缘信息	-	浅层特征直接“抄近道”
需要端到端训练	-	可学习的上采样	梯度直达浅层

三个组件缺一不可。没有解码器，输出分辨率不够；没有跳跃连接，恢复的细节不够；没有编码器，没有语义理解。

7.2.6 嵌入核心代码：核心组件

下面先看 U-Net 的四个核心组件，完整的 U-Net 组装会在 完整实现 中展示。

双卷积块（核心构建单元）

```
class DoubleConv(nn.Module):
    """U-Net 中的基本构建单元：两个 3×3 卷积

    参数量计算（输入 C_in，输出 C_out）：
    第一个卷积：C_in × 3 × 3 × C_out
    第二个卷积：C_out × 3 × 3 × C_out
    总计：9 × (C_in × C_out + C_out²)
    例如 64→64：9 × (64 × 64 + 64²) = 73,728
    """

    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
```

(续下页)

(接上页)

```

        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True)
    )

    def forward(self, x):
        return self.conv(x)

```

为什么两个 3×3 卷积堆叠？

一个 3×3 卷积感受野是 3×3 。堆叠两个 3×3 ，感受野变成 5×5 （感受野的递推公式）。但参数量比一个 5×5 少—— $2 \times 9 \times C^2$ vs $25 \times C^2$ ，减少了 28%。如果堆叠三个 3×3 ，感受野达到 7×7 ，参数比一个 7×7 减少 45%。这个设计思路最早来自 VGG 网络 [SZ15]——用小卷积核的堆叠代替大卷积核，在保证感受野的同时减少参数。

每个卷积后接一个 BatchNorm [IS15]，作用是稳定训练：把激活值拉回到零均值单位方差，防止深层网络的激活值剧烈漂移。

下采样模块

```

class DownSample(nn.Module):
    """编码器下采样：MaxPool → DoubleConv

    输入: (B, C_in, H, W)
    输出: (B, C_out, H/2, W/2)
    """
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.maxpool_conv = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_channels, out_channels)
        )

    def forward(self, x):
        return self.maxpool_conv(x)

```

上采样模块（带跳跃连接）

```

class UpSample(nn.Module):
    """解码器上采样：转置卷积 → 跳跃拼接 → DoubleConv

    输入 x1: (B, C_dec, H, W)
    输入 x2: (B, C_enc, 2H, 2W)

```

(续下页)

(接上页)

输出: (B, C_out, 2H, 2W)

转置卷积把 C_dec 减半、尺寸加倍;
 拼接编码器特征后, DoubleConv 缩回 C_out

```
"""
def __init__(self, in_channels, out_channels):
    super().__init__()
    self.up = nn.ConvTranspose2d(
        in_channels, in_channels // 2, kernel_size=2, stride=2
    )
    self.conv = DoubleConv(in_channels, out_channels)

def forward(self, x1, x2):
    x1 = self.up(x1)
    # 原始 U-Net 用有效填充, 尺寸可能不匹配, 需要填充对齐
    diffY = x2.size()[2] - x1.size()[2]
    diffX = x2.size()[3] - x1.size()[3]
    x1 = F.pad(x1, [diffX // 2, diffX - diffX // 2,
                    diffY // 2, diffY - diffY // 2])
    # 跳跃连接: 通道拼接
    x = torch.cat([x2, x1], dim=1)
    return self.conv(x)
```

输出层

```
class OutConv(nn.Module):
    """输出层: 1x1 卷积

    输入: (B, C_in, H, W)
    输出: (B, n_classes, H, W)

    1x1 卷积不改变空间尺寸, 只改变通道数
    """
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=1)

    def forward(self, x):
        return self.conv(x)
```

这些组件的完整组装见 [完整实现](#)。下一节 [损失函数设计](#) 我们先来谈谈一个问题：分割任务的损失函数为什么和分类任务不一样？

7.3 损失函数设计

7.3.1 直觉：分类损失 vs 分割损失

分类任务（卷积：发现装备）的交叉熵损失看的是“每个像素分对了吗”。但分割任务有一个特殊问题：**类别极度不平衡**。

想象一张 CT 图像，肿瘤只占 1% 的像素。如果模型把所有像素都预测为“正常组织”，准确率是 99%，但分割结果毫无意义——没找到肿瘤。

分割损失的核心挑战是：**让模型关心“少数派像素”**，而不是只追求整体准确率。

7.3.2 交叉熵损失

标准交叉熵

$$L_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c})$$

其中 N 是像素数， C 是类别数， $y_{i,c}$ 是真实标签， $p_{i,c}$ 是预测概率。

加权交叉熵

缓解类别不平衡：给少数类更高的权重。

$$L_{\text{WCE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C w_c \cdot y_{i,c} \log(p_{i,c})$$

权重 w_c 通常与类别频率成反比。肿瘤只有 1% 像素， $w_{\text{肿瘤}} = 99$ ， $w_{\text{正常}} = 1$ 。

焦点损失（Focal Loss）

$$L_{\text{FL}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C (1 - p_{i,c})^\gamma \cdot y_{i,c} \log(p_{i,c})$$

γ 控制聚焦程度： $\gamma = 2$ 时，易分类样本的贡献被大幅降低，模型被迫关注难分类样本 [LGG+17]。

交叉熵的局限

交叉熵优化的是**像素级准确率**，而我们真正关心的是**区域重叠度**。这两个目标在类别不平衡时可能南辕北辙——把所有像素都预测为“背景”，像素级准确率可能 $>99\%$ ，但分割完全无效。

7.3.3 Dice 损失

Dice 系数

Dice 系数直接衡量预测掩码与真实掩码的重叠程度：

$$\text{Dice} = \frac{2|X \cap Y|}{|X| + |Y|} = \frac{2TP}{2TP + FP + FN}$$

Dice 系数 = 1 表示完美重叠，= 0 表示完全不重叠。



图 4: Dice 系数的可视化

Dice 损失

$$L_{\text{Dice}} = 1 - \text{Dice} = 1 - \frac{2|X \cap Y|}{|X| + |Y|}$$

在二分类任务中的像素级实现：

$$L_{\text{Dice}} = 1 - \frac{2 \sum_{i=1}^N p_i y_i + \epsilon}{\sum_{i=1}^N p_i + \sum_{i=1}^N y_i + \epsilon}$$

Dice 损失的梯度分析

Dice 损失为什么对类别不平衡鲁棒？答案在它的梯度里。

先定义软 Dice 损失（去掉 ϵ 简化分析）：

$$L = 1 - \frac{2 \sum p_i y_i}{\sum p_i + \sum y_i}$$

对第 j 个像素的预测 p_j 求导（链式法则，假设 $y_j = 1$ 即该像素属于目标类）：

$$\frac{\partial L}{\partial p_j} = -2 \cdot \frac{y_j (\sum p_i + \sum y_i) - (\sum p_i y_i) \cdot 1}{(\sum p_i + \sum y_i)^2}$$

分子提出 $y_j = 1$ ：

$$\frac{\partial L}{\partial p_j} = -2 \cdot \frac{\sum y_i - \sum p_i y_i}{(\sum p_i + \sum y_i)^2}$$

分子 $\sum y_i - \sum p_i y_i = \text{真实面积} - \text{重叠面积} = \text{被漏检的区域大小}$ 。

直觉：Dice 梯度在说什么？

Dice 的梯度大小正比于“漏了多少”，反比于总面积的平方。

这意味着：

- 一个像素梯度的大小取决于全局漏检了多少，而不只是这个像素本身分对分错
- 小目标 ($\sum y_i$ 很小) 的梯度天然放大——因为分母 $(\sum p_i + \sum y_i)^2$ 中的 $\sum y_i$ 很小
- 比交叉熵更均衡：交叉熵对每个像素给独立梯度，大目标有更多像素 → 梯度主导；Dice 把整张图作为一个整体来优化

梯度符号表

符号	含义	对梯度方向的影响
$\sum y_i$	真实目标总面积	分母越大，整体梯度越小
$\sum p_i y_i$	预测与真实的重叠面积	重叠越大，分子越小（漏检越少）
$\sum p_i$	预测目标总面积	分母项，过分割会降低梯度
$\sum y_i - \sum p_i y_i$	真实中被漏检的面积	驱动梯度大小的核心—漏越多，梯度越大

Dice 损失对类别不平衡天然鲁棒。因为它在分母中同时用预测面积和真实面积做归一化。肿瘤只占 1%？没关系——Dice 算的是“你预测的肿瘤和真实肿瘤重叠了多少”，不是“你对了多少像素”。

Dice vs 交叉熵

特性	交叉熵	Dice
关注重点	每个像素预测准确性	整体区域重叠度
类别不平衡	敏感	鲁棒
梯度来源	单像素 p_i 与 1 的差距	全局漏检面积与总面积之比
小目标	被大目标淹没	天然放大（小分母效应）
梯度特性	平滑、凸、易优化	非凸、可能有局部最优
收敛速度	通常较快	可能较慢

7.3.4 组合损失

实践中效果最好的往往是交叉熵 + Dice 组合损失：

$$L_{\text{total}} = \alpha L_{\text{CE}} + \beta L_{\text{Dice}}$$

交叉熵提供稳定的梯度，Dice 提供区域级优化目标。通常 $\alpha = \beta = 0.5$ 已经是很好的起点。

```

class DiceLoss(nn.Module):
    """Dice 损失"""
    def __init__(self, smooth=1e-6):
        super().__init__()
        self.smooth = smooth

    def forward(self, pred, target):
        # pred: (B, C, H, W) logits, target: (B, H, W) 类别索引
        pred = torch.softmax(pred, dim=1)

        dice_loss = 0.0
        for c in range(pred.shape[1]):
            pred_c = pred[:, c, :, :]
            target_c = (target == c).float()

            intersection = (pred_c * target_c).sum()
            dice = (2. * intersection + self.smooth) / (
                pred_c.sum() + target_c.sum() + self.smooth
            )
            dice_loss += 1 - dice

        return dice_loss / pred.shape[1]

```

```

class CombinedLoss(nn.Module):
    """交叉熵 + Dice 组合损失"""
    def __init__(self, weight_ce=0.5, weight_dice=0.5):
        super().__init__()
        self.weight_ce = weight_ce
        self.weight_dice = weight_dice
        self.ce_loss = nn.CrossEntropyLoss()
        self.dice_loss = DiceLoss()

    def forward(self, pred, target):
        return (self.weight_ce * self.ce_loss(pred, target) +
                self.weight_dice * self.dice_loss(pred, target))

```

7.3.5 损失函数选择指南

场景	推荐	理由
类别大致平衡	交叉熵	简单有效
类别极不平衡（肿瘤 < 1%）	Dice 损失	小目标天然放大
小目标检测	焦点损失 + Dice	聚焦难分类样本
一般情况（推荐）	CE + Dice 组合	兼顾稳定性和优化目标

7.4 完整实现

U 形架构详解 中我们看了 U-Net 的四个核心组件。现在把它们组装成一个完整的 U-Net。

7.4.1 完整 U-Net 类

```
class UNet(nn.Module):
    """完整 U-Net

    参数:
        n_channels: 输入通道数（灰度=1，RGB=3）
        n_classes: 分割类别数

    前向传播维度变化（输入 572×572 为例）：
    编码器:  inc:    1×572×572  → 64×568×568
            down1: 64×568×568 → 128×280×280
            down2: 128×280×280 → 256×136×136
            down3: 256×136×136 → 512×64×64
            down4: 512×64×64   → 1024×28×28
    解码器:  up1: 1024×28×28 + skip → 512×52×52
            up2: 512×52×52 + skip → 256×100×100
            up3: 256×100×100 + skip → 128×196×196
            up4: 128×196×196 + skip → 64×388×388
    输出:    outc: 64×388×388 → 2×388×388
    """

    def __init__(self, n_channels, n_classes):
        super().__init__()
        self.n_channels = n_channels
        self.n_classes = n_classes
```

(续下页)

(接上页)

```

# 编码器 (1 → 64 → 128 → 256 → 512 → 1024)
self.inc = DoubleConv(n_channels, 64)
self.down1 = DownSample(64, 128)
self.down2 = DownSample(128, 256)
self.down3 = DownSample(256, 512)
self.down4 = DownSample(512, 1024)

# 解码器 (1024 → 512 → 256 → 128 → 64)
self.up1 = UpSample(1024, 512)
self.up2 = UpSample(512, 256)
self.up3 = UpSample(256, 128)
self.up4 = UpSample(128, 64)

# 输出层
self.outc = OutConv(64, n_classes)

def forward(self, x):
    # 编码器
    x1 = self.inc(x)
    x2 = self.down1(x1)
    x3 = self.down2(x2)
    x4 = self.down3(x3)
    x5 = self.down4(x4)
    # 解码器 (拼接对应编码器特征)
    x = self.up1(x5, x4)
    x = self.up2(x, x3)
    x = self.up3(x, x2)
    x = self.up4(x, x1)
    logits = self.outc(x)
    return logits

```

这就是完整的 U-Net 前向传播——简单、对称、优雅。没有复杂的控制流，就是一条 **U 形通道**。

完整 U-Net 的参数量估算

以灰度输入 ($n_channels=1$)，二分类 ($n_classes=2$) 为例：

- 编码器： $\approx 31\text{M}$ 参数
- 解码器： $\approx 24\text{M}$ 参数

- 总计：约 55M 参数

作为对比，LeNet-5：练习峰首登的 LeNet-5 约 60K 参数。U-Net 比它大了近 1000 倍——因为通道多（1024 vs 120）、有解码器、有跳跃连接的双卷积。

7.4.2 模型初始化

```
def init_weights(model, init_type='kaiming'):
    """Kaiming 初始化 {cite}`he2015delving`（适合 ReLU 激活）"""
    for m in model.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
            if m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, 1)
            nn.init.constant_(m.bias, 0)
```

7.4.3 训练配置

```
def setup_training(model, device='cuda'):
    model = model.to(device)
    # 损失函数：交叉熵 + Dice 组合（{ref}`unet-loss`详细讨论）
    criterion = CombinedLoss(weight_ce=0.5, weight_dice=0.5)
    # Adam 优化器
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
    # 验证损失不再下降时减半学习率
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', factor=0.5, patience=10
    )
    return model, criterion, optimizer, scheduler
```

7.4.4 评估指标

Dice 系数不只是损失函数，也是最常用的评估指标。

```
def calculate_metrics(pred, target, threshold=0.5):
    """计算 Dice 和 IoU"""
    pred_bin = (pred > threshold).float() # 概率转二值
    target_bin = (target > threshold).float()
```

(续下页)

(接上页)

```

tp = (pred_bin * target_bin).sum().item()
fp = (pred_bin * (1 - target_bin)).sum().item()
fn = ((1 - pred_bin) * target_bin).sum().item()

dice = (2 * tp) / (2 * tp + fp + fn + 1e-8)
iou = tp / (tp + fp + fn + 1e-8)
return {'dice': dice, 'iou': iou}

```

7.4.5 推理示例

```

def predict(model, image, device='cuda'):
    """对单张图像做分割预测"""
    model.eval()

    if len(image.shape) == 3:          # (C, H, W) → (1, C, H, W)
        image = image.unsqueeze(0)
    image = image.to(device)

    with torch.no_grad():
        output = model(image)
        if model.n_classes > 1:
            prediction = torch.argmax(output, dim=1)    # 取概率最大的类别
        else:
            prediction = (torch.sigmoid(output) > 0.5).float()

    return prediction.cpu().squeeze()

```

7.4.6 参数量统计

```

# 统计模型参数量
model = UNet(n_channels=1, n_classes=2)
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f' 总参数: {total:,}')
print(f' 可训练: {trainable:,}')
# 输出: 总参数: 31,031,554
#       可训练: 31,031,554

```

7.4.7 端到端 Walkthrough：一张细胞图像走完 U-Net

理论看了这么多，不如拿一张真实的细胞图像追踪完整流程。假设输入是 572×572 的灰度图（单通道）。

编码器阶段

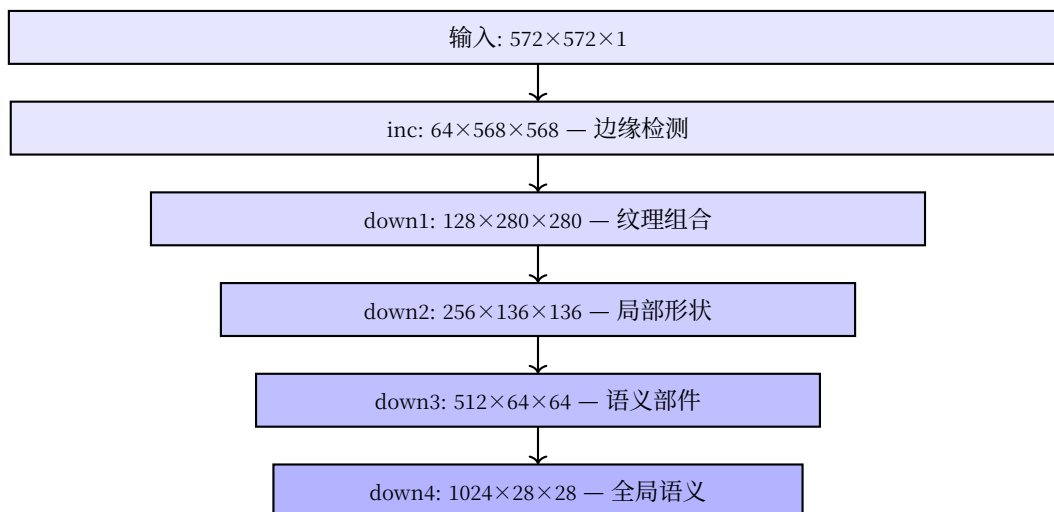


图 5: 编码器各层输出维度与内容

每经过一个下采样块，空间减半、通道加倍、感受野扩大。一路走来，网络从“看到像素的边缘”变成“理解这是细胞核”。

解码器阶段

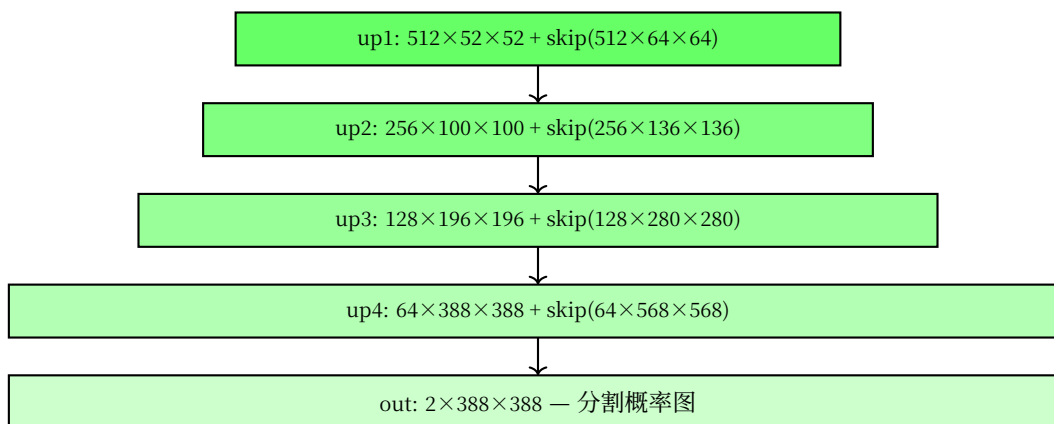


图 6: 解码器各层：上采样+跳跃连接

数值示例：看一个像素如何被分类

想象图像左上角有一个细胞核边缘的像素：

1. **输入层**：这个像素的灰度值是 142（0-255 范围）
2. **inc 层**： 3×3 卷积核检测到它左右两侧的像素差异大 → 判断为"边缘"，激活值变高
3. **down1 层**：池化后它被合并到 2×2 区域的最大值中，丢失了精确坐标，但保留了"这里有边缘"的信息
4. **down2 → down4**：随着下采样，它从"一个像素的边缘"变成了"一块区域的纹理"→"一个局部的形状"→"一个细胞核的组成部分"
5. **最底部**：1024 维的特征向量说"这是一个细胞核区域"
6. **解码器上采样**：一步步恢复分辨率，每次拼接编码器对应层的特征，补充精确位置
7. **输出层**：这个像素的 2 个输出值经过 softmax：[0.12, 0.88] → 预测为"细胞"（第 1 类）

整个过程，这个像素的身份从"142 灰度值"变成了"细胞核像素"。U-Net 的对称结构确保了 **语义理解（编码器）** 和 **空间精度（跳跃连接）** 在这个过程中都得到了保留。

这就是一个可用版 U-Net。但光有模型还不够 —— 医学图像通常只有几十张标注数据。下一节 [数据增强与训练技巧](#) 我们看看如何用**数据增强**在有限数据上训练出好模型。

7.5 数据增强与训练技巧

7.5.1 问题：数据不够怎么办？

U-Net 发表时的关键成就之一：**用 30 张训练图像**在 ISBI 细胞分割挑战赛上达到 SOTA。深度学习的常识是"数据越多越好"，但医学图像标注极其昂贵 —— 标注一张细胞分割图可能需要专家数十分钟。

U-Net 的答案：**数据增强 + 网络设计的数据效率**。

7.5.2 数据增强

数据增强（Data Augmentation）通过对现有训练样本施加随机变换来生成"新"样本，是解决小数据问题的首要武器。

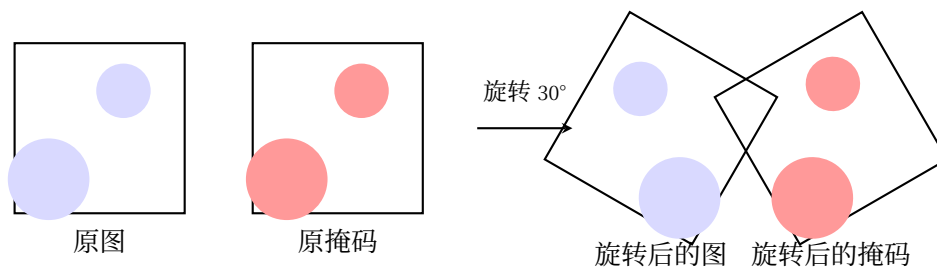
分割增强的特殊要求

分类任务的增强只作用于图像。分割任务中，**图像和掩码必须做完全相同的几何变换** —— 如果图像旋转了 30° ，掩码也必须旋转 30° ，否则模型学到的对应关系是错误的。

```
import albumentations as A
from albumentations.pytorch import ToTensorV2

def get_train_transforms():
```

(续下页)



同步变换：相同的旋转角度、平移、缩放

图 7: 图像与掩码同步变换

(接上页)

```
"""训练阶段数据增强：图像和掩码同步变换"""
return A.Compose([
    # 几何变换（图像+掩码同步）
    A.RandomRotate90(p=0.5),
    A.Flip(p=0.5),
    A.ElasticTransform(
        sigma=50, alpha=1, alpha_affine=50, p=0.3
    ),
    # 强度变换（只应用于图像）
    A.RandomBrightnessContrast(p=0.2),
    A.GaussNoise(p=0.2),
    # 标准化
    A.Normalize(mean=[0.485], std=[0.229]),
    ToTensorV2(),
], additional_targets={'mask': 'image'})
```

弹性变形：U-Net 的秘密武器

弹性变形的直觉

想象一张果冻塑料膜，你用手指戳它——图像会局部扭曲，但整体内容不变。

弹性变形就是模拟这种效果：生成一个随机位移场，然后根据位移场插值图像。这对生物医学图像特别有用——细胞、组织本来就会有自然形变，弹性变形生成的“假样本”看起来仍然合理。

医学图像中细胞、组织的形态天然存在弹性形变（不同的患者、不同的切片角度）。弹性变形恰好模拟了这种变化——这让模型学到了“细胞可以长成各种形状”，而不是死记硬背训练集中的特定形态。

7.5.3 其他训练技巧

学习率调度与搜索

找到合适的学习率是训练 U-Net 的第一步。推荐使用**学习率搜索**：

```
# 学习率搜索：从 1e-6 到 1 指数增长，每个 batch 记录 loss
def lr_finder(model, train_loader, criterion, start_lr=1e-6, end_lr=1.0):
    model.train()
    optimizer = torch.optim.Adam(model.parameters(), lr=start_lr)
    lrs, losses = [], []
    lr = start_lr
    for images, masks in train_loader:
        optimizer.param_groups[0]['lr'] = lr
        optimizer.zero_grad()
        loss = criterion(model(images), masks)
        loss.backward()
        optimizer.step()
        lrs.append(lr)
        losses.append(loss.item())
        lr *= (end_lr / start_lr) ** (1 / len(train_loader))
    return lrs, losses
# 画出 lr vs loss 曲线，选 loss 下降最快的 lr
```

典型的学习率选择：U-Net + Adam [KB15] 组合，推荐 10^{-4} 到 3×10^{-4} 。

学习率调度

```
# 选项 1：验证损失不再下降时减半
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=10
)

# 选项 2：余弦退火 —— 学习率周期性变化
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100, eta_min=1e-6
)
```

早停

当验证集 Dice 系数连续 N 个 epoch 没有提升时停止训练，防止过拟合。

```
class EarlyStopping:
    def __init__(self, patience=20, mode='max'):
        self.patience = patience
        self.mode = mode
        self.counter = 0
        self.best_score = None
        self.early_stop = False

    def __call__(self, score):
        if self.best_score is None:
            self.best_score = score
        elif (self.mode == 'max' and score < self.best_score) or \
            (self.mode == 'min' and score > self.best_score):
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_score = score
            self.counter = 0
        return self.early_stop
```

混合精度训练

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for images, masks in train_loader:
    optimizer.zero_grad()
    with autocast():
        outputs = model(images)
        loss = criterion(outputs, masks)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

通过自动混合精度，训练速度可提升 2-3 倍，内存减少约 40%。

7.5.4 常见失败模式与诊断

U-Net 训练中遇到问题时，可以用下面的系统排查法定位根因。

损失不下降

症状	可能原因	如何验证	解决方案
损失卡在某个值不动	学习率太小	画出 lr finder 曲线	增大 lr 或使用查找得到的最优值
损失徘徊不变, $\text{Dice} \approx 0$	模型未收敛	检查梯度范数是否为 0	确认输入正常、权重已初始化
损失缓慢下降但非常慢	学习率太小	比较收敛速度与基准	尝试 $\text{lr} \times 10$
损失先降后停	陷入局部最优	用 SGD+momentum 试试	切换优化器或重启训练

损失震荡

症状	可能原因	如何验证	解决方案
损失大幅波动	学习率太大	损失曲线呈锯齿状	减小 $\text{lr} \times 0.1$
偶尔出现 loss spike	批次中有异常样本	检查数据是否有 NaN	梯度裁剪, 检查数据预处理
Dice 在 0 和 0.9 之间反复	Dice 梯度不稳定	单独用 Dice 损失训练	使用 CE + Dice 组合损失

过拟合

症状	可能原因	如何验证	解决方案
训练 Dice $\uparrow$, 验证 Dice $\downarrow$	模型记住了训练集	差距 > 0.05 即过拟合	增加数据增强, 加 Dropout, 早停
训练 Dice ≈ 1.0 , 验证低	数据太少 + 增强不足	验证集多样本表现方差大	强化弹性变形, 增加旋转/翻转
分割结果只预测背景	类别极度不平衡	检查验证集类别比例	使用加权损失或 Dice 损失

小目标分割失败

症状	可能原因	如何验证	解决方案
小目标完全漏检	Dice 对小目标贡献太小	逐类计算 Dice, 小类偏低	用焦点损失 + Dice
小目标形状不对	下采样丢失了细节	检查感受野是否覆盖目标	减少下采样次数, 或使用 dilation
小目标位置偏移	空间精度不够	检查跳跃连接是否有效	尝试 Attention U-Net

7.5.5 消融实验建议

当你改进 U-Net 时，需要知道**哪些改动真正有效**。推荐做以下消融实验建立基准：

实验	比较对象	能回答的问题
U-Net vs 去掉跳跃连接	完整 U-Net vs 无 skip	跳跃连接贡献了多少 Dice?
CE 损失 vs Dice 损失 vs CE+Dice	三种损失函数	哪种损失最适合你的数据?
有/无弹性变形	两种增强策略	弹性变形带来了多少提升?
不同深度 (3 层 vs 4 层 vs 5 层)	三种深度配置	多深是够? 过深是否过拟合?
不同初始通道数 (32/64/128)	三种宽度配置	更宽是否更好? 代价是什么?

每次只改一个变量，记录 Dice、IoU、参数量、训练时间。

7.5.6 实践检查清单

问题	可能原因	解决方案
损失不下降	学习率太小	增大 lr 或使用学习率搜索
损失震荡	学习率太大 / Dice 梯度不稳定	减小 lr, 使用 CE+Dice 组合损失
过拟合 (验证集指标下降)	数据太少	加强数据增强, 早停
小目标分割差	Dice 对极小区域不敏感	用焦点损失, TTA
训练慢 / 内存不够	模型太大 / 批次太大	混合精度, 梯度累积

测试时增强 (TTA)

推理时对同一张图做多个版本的增强 (旋转、翻转)，取平均预测。**免费提升 1-3% Dice**：

```
def tta_predict(model, image):
    predictions = []
    for transform in tta_transforms:
        aug_image = transform(image=image)['image']
        with torch.no_grad():
            pred = torch.sigmoid(model(aug_image.unsqueeze(0)))
            predictions.append(pred.cpu())
    return torch.stack(predictions).mean(dim=0)
```

7.5.7 小结

U-Net 在小数据场景成功的关键公式：

成功 = U 形架构 (数据效率高) + 弹性变形 (针对医学数据) + 组合损失 (稳定优化)

下一节 [总结与展望](#) 我们看看 U-Net 的应用领域、有哪些变体，以及它的局限性在哪里。

7.6 总结与展望

恭喜！你已经完成了 U-Net 图像分割章节的全部内容。

从引言：图像分割问题的分割任务定义，到 U 形架构详解的架构剖析，2. 跳跃连接（Skip Connection）的跳跃连接原理，数据增强与训练技巧的弹性变形，损失函数设计的 Dice 损失设计，直至完整实现的完整实现——你已经掌握了语义分割的核心技术栈。

7.6.1 核心概念映射

概念	直觉理解	数学/代码关键	设计动机
编码器-解码器	先压缩语义，再恢复位置	下采样 → 上采样的对称结构	解决"位置 vs 语义"矛盾
跳跃连接	把浅层细节直接送到深层	<code>torch.cat([encoder, decoder], dim=1)</code>	保留边界精度
弹性变形	像揉橡皮泥一样扭曲图像	随机位移场+双三次插值	小数据增强
Dice 损失	直接优化重叠度指标	$L = 1 - \frac{2 X \cap Y }{ X + Y }$	解决类别不平衡
感受野	每个输出像素"看到"的输入范围	随深度指数增长	确保足够上下文

7.6.2 与前面章节的联系

本章是 上篇：练习峰 和 下篇：真正的未登峰 的综合应用：

前置知识	本章应用
卷积：发现装备的卷积与池化	编码器特征提取
归纳偏置的归纳偏置	U 形结构编码"分割需要位置+语义"
感受野的感受野	确保深层有足够上下文信息
SE-Net：学会聚焦的通道注意力	Attention U-Net 的注意力门
CBAM：哪里重要 + 什么重要 的空间注意力	空间注意力在分割中的应用

核心认知： U-Net 不是全新的架构，而是 归纳偏置 思想的完美实践 —— 把"分割需要精确边界"的先验，翻译成"编码器-解码器+跳跃连接"的结构。

7.6.3 关键数字速查

变体对比

变体	年份	核心改进	Dice 提升	参数量变化
U-Net++	2018	嵌套密集跳跃连接	+2-5%	+30-50%
Attention U-Net	2018	注意力门控跳跃连接	+1-3%	略增
3D U-Net	2016	3D 卷积	+3-8% (体积数据)	大幅增加
TransUNet	2021	Transformer 编码器	+3-6%	大幅增加

应用领域数据

任务	典型 Dice	U-Net 解决方案
细胞核分割	0.92	跳跃连接保留边界
肝脏/肿瘤分割	0.95	弹性变形数据增强
脑肿瘤分割	0.88	多尺度特征融合
视网膜血管	0.95	深层次特征融合

7.6.4 局限性

U-Net 并非万能

- 1. 计算开销大：跳跃连接需要存储所有编码器特征图，内存占用大
- 2. 长距离依赖有限：CNN 感受野靠深度堆叠，效率不如 Transformer
- 3. 超参数敏感：深度、通道数、学习率需要仔细调整
- 4. 3D 数据挑战：直接扩展为 3D 后内存爆炸，需要 patch 策略

7.6.5 常见问题速答

Q: U-Net 的输入尺寸必须是 572×572 吗？

A: 不是。原始论文用 572×572 是因为用了有效填充。现代实现用相同填充 (padding=1)，输入最好是 2 的幂次 (256、512 等)，因为下采样 4 次需要输入能被 16 整除。

Q: 什么时候用 U-Net 而不是其他分割网络？

A: U-Net 最适合小数据、高精度边界的场景。如果数据量很大 (>10K 张)，DeepLab 或 Mask R-CNN 可能更好。

Q: 跳跃连接为什么用拼接而不是相加？

A: 拼接保留编码器特征的**全部信息**。相加相当于线性投影可能丢失信息，拼接让解码器自己决定"取多少"。

Q: Dice 损失训练不稳定怎么办?

A: 1) 加交叉熵做组合损失; 2) 增大平滑项 ϵ 到 10^{-5} 或 10^{-4} 。

Q: 为什么我的 U-Net 只预测背景?

A: 最可能: 类别不平衡 + 学习率太大。用 Dice 损失, 检查学习率, 确认数据增强没扭曲目标。

Q: 3D U-Net 太吃内存怎么办?

A: 1) patch-based 训练; 2) 混合精度训练; 3) 深度可分离卷积。

7.6.6 下一步学习方向

掌握了 U-Net 后, 你可以探索:

1. **下篇: 真正的未登峰**: Attention U-Net 如何将注意力引入分割
2. **DeepLab 系列**: 空洞卷积扩大感受野, ASPP 多尺度融合
3. **Mask R-CNN**: 实例分割 (区分不同个体)
4. **SAM (Segment Anything)**: 基于提示的通用分割模型

7.6.7 推荐资源

必读论文

- Ronneberger et al., “U-Net: Convolutional Networks for Biomedical Image Segmentation”, MICCAI 2015 — 原始论文
- Zhou et al., “UNet++: A Nested U-Net Architecture”, 2018
- Oktay et al., “Attention U-Net”, MIDL 2018

动手实践

- 《动手学深度学习》第 13 章²⁴ — 语义分割完整实现
- PyTorch 官方教程中的 U-Net 实现

拓展阅读

- Çiçek et al., “3D U-Net”, MICCAI 2016
- Hatamizadeh et al., “UNETR: Transformers for 3D Medical Image Segmentation”, 2022

本章完。

²⁴ https://zh.d2l.ai/chapter_computer-vision/semantic-segmentation-and-dataset.html

序列建模：从 RNN 到 Transformer 再到 Mamba

8.1 引言：从空间到时间

到目前为止，我们一直在处理图像——卷积：发现装备中的卷积核在二维像素格上滑动，下篇：真正的未登峰中的注意力模块在通道和空间维度上重加权。这些架构共享一个隐含假设：**输入是静态的、定长的、没有先后顺序的**。

但现实中的很多数据不是这样。一段文字、一首歌、一段视频——它们都有**时间顺序**。单词的排列决定语义，音节的先后决定旋律，帧的连续决定动作。

本章要回答的核心问题是：**如何让神经网络理解"顺序"？**

从受到大脑启发的循环神经网络（RNN），到用全局注意力彻底替代循环的 Transformer，再到回归 RNN 效率哲学的新架构 Mamba——我们将追溯一条横跨 35 年的思想演化之路。

学习目标

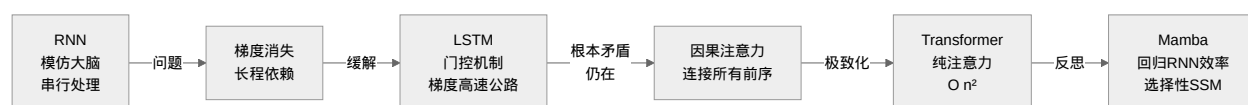
完成本章后，你将能够：

1. **理解 RNN 的原理和局限**：解释循环结构如何模拟序列处理，以及为什么长程依赖和梯度消失让它失效
2. **掌握 LSTM 的门控机制**：理解细胞状态和隐状态的分离，以及遗忘门、输入门、输出门如何管理长期记忆

3. **掌握因果注意力的直觉来源**：理解"让每个隐状态与所有前序状态建立联系"如何自然导出注意力机制
4. **理解 Transformer 的核心设计**：掌握自注意力 (Q/K/V)、多头机制、位置编码、FFN 的设计哲学，以及 $O(n^2)$ 的来源
5. **认识 Mamba 的思想回归**：理解状态空间模型为什么是"现代版 RNN"，以及选择性机制如何解决长程依赖

8.1.1 本章路线

本章讲述了一个"出发—迷失—回归"的思想故事：



核心认知：这不是简单的技术堆叠，而是一个思想在螺旋上升——从 RNN 出发，经历注意力的彻底革命，最终在更高的层次上回归 RNN 的效率哲学。

让我们从 **循环神经网络：让网络拥有"记忆"** 开始——一个最简单的循环结构，和一个横跨三十年的故事。

8.2 循环神经网络：让网络拥有"记忆"

全连接：赤手空拳中的全连接网络和 **卷积**：发现装备中的 CNN 有一个共同特征：**给定一个输入，产生一个输出，然后"忘记"这个输入**。每次前向传播是独立的，网络不记得上一秒看到了什么。

这显然不符合我们处理序列的方式。读这句话时，你的大脑记住了前文——"这"指代什么，"显然"修饰什么——这些理解都依赖于**记忆**。

关键问题：如何让神经网络在处理序列时拥有记忆？

8.2.1 历史背景

1986 年，David Rumelhart 和 Geoffrey Hinton 等人发表了反向传播论文 [RHW86]，为训练多层网络奠定了基础。几乎同时，Michael Jordan 提出了带有循环连接的网络架构 [Jor86]——让网络将上一时刻的输出作为当前输入的一部分，从而拥有对过去的"记忆"。

1990 年，Jeffrey Elman 在此基础上提出了**简单循环网络 (Simple Recurrent Network)** [Elm90]，其核心结构——一个隐状态 $\mathbf{h}_t$ 在每一步基于 $\mathbf{h}_{t-1}$ 和 $\mathbf{x}_t$ 更新——成为此后三十年所有循环架构的基石。

备注

为什么叫"循环"? 因为网络在时间上的每一步使用**相同的权重** $\mathbf{w}_h$ 和 $\mathbf{w}_x$ 。这类似于 **卷积：发现装备** 中卷积核的权值共享——同一个规则应用到所有位置，只是 CNN 共享空间位置，RNN 共享时间位置。

8.2.2 从大脑到 RNN：记忆的直觉

你如何理解一句话？

想象你逐字阅读：“今天天气很好所以我决定去公园散步”。

- 读到“今天天气很好”时，你建立了一个上下文
- 读到“所以”时，你知道接下来是结论
- 读到“我决定去”时，你预期一个动作
- 读到“公园散步”时，你把它和“天气好”建立了因果联系

每一步，你的大脑都在**更新一个内部状态**——一个不断演化的、包含前文信息的“摘要”。这个摘要不是完整记忆（你不会逐字背诵前文），而是对理解当前词最关键的**压缩表示**。

生物学的启发：大脑不“反复读取”历史

RNN 的灵感直接来自大脑中一个更根本的事实——**大脑不是每个瞬间把所有历史“重新读一遍”**。

想想你是怎么理解一段音乐的。听到第三个音符时，你并没有重新播放前两个音符。你对前两个音符的感知已经“融入”了当前的神经活动——大脑利用一系列**持续变化的物理状态**来自然携带历史信息：

- **神经元膜电位**：一个神经元被激活后，它的膜电位不会瞬间归零，而是逐渐衰减。这个衰减过程本身就在“记住”最近发生过什么。
- **突触短期可塑性**：高频使用一个突触后，它的传递效率会在短时间内改变（增强或抑制）——这是突触层面上的“工作记忆”。
- **钙离子浓度**：当神经元放电时，钙离子流入细胞内，浓度缓慢下降。这个化学梯度编码了过去几百毫秒内的活动强度。
- **神经递质残留**：突触间隙中的神经递质分子不会瞬间清除，它们的残余浓度携带着“刚才这里发生了信号传递”的信息。

这些都不是“显式的记忆存储”——大脑没有一个单独的“历史缓冲区”。历史信息是**物理地、连续地**嵌入在神经元和突触的状态变化中的。每一个瞬间的大脑状态都已经是所有过去输入的**压缩结果**。

备注

RNN 的本质： $\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$ 就是对这个过程的数学抽象。 $\mathbf{h}_t$ 对应神经元群的膜电位模式——它随时间连续演化，每一步吸收新的输入（ $\mathbf{W}_x \mathbf{x}_t$ ），同时自然地携带上一步的状态（ $\mathbf{W}_h \mathbf{h}_{t-1}$ ）。信息通过时间上连续的状态变化向前传递，而不是反复从静态记忆中读取。

这就是“循环”二字的生物学含义：**不是重新读取，而是持续继承**。

RNN 的循环结构

RNN 做的正是这件事 [Elm90]。它维护一个隐状态 (hidden state) $\mathbf{h}_t$ ，在处理每一步输入 $\mathbf{x}_t$ 时：两行公式定义

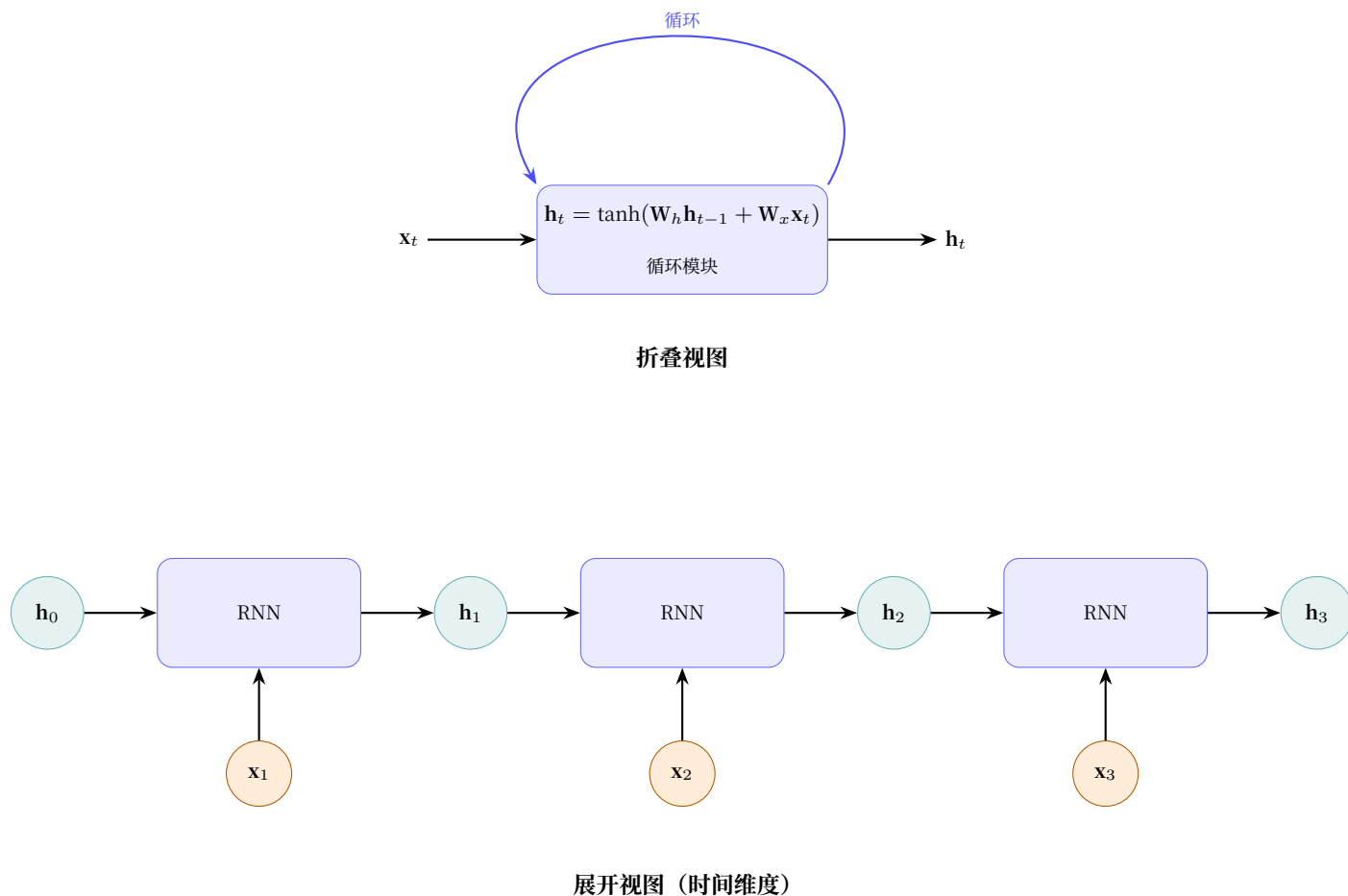


图 1: RNN 循环结构：同一模块在时间上展开

了 RNN 的全部行为：

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

$$\mathbf{y}_t = \mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y$$

其中：

- $\mathbf{h}_t \in \mathbb{R}^d$ ：时刻 t 的隐状态 (网络的"记忆")
- $\mathbf{x}_t \in \mathbb{R}^m$ ：时刻 t 的输入
- $\mathbf{W}_h \in \mathbb{R}^{d \times d}$ ：循环权重 —— 控制"如何混合旧记忆和新输入"
- $\mathbf{W}_x \in \mathbb{R}^{d \times m}$ ：输入投影 —— 将输入映射到隐空间
- $\mathbf{W}_y \in \mathbb{R}^{k \times d}$ ：输出投影 —— 从隐状态产生预测

备注

直觉: $W_h h_{t-1}$ 是"提炼旧记忆", $W_x x_t$ 是"吸收新信息", $\tanh$ 将两者混合并压缩到 $(-1, 1)$, 防止数值发散。

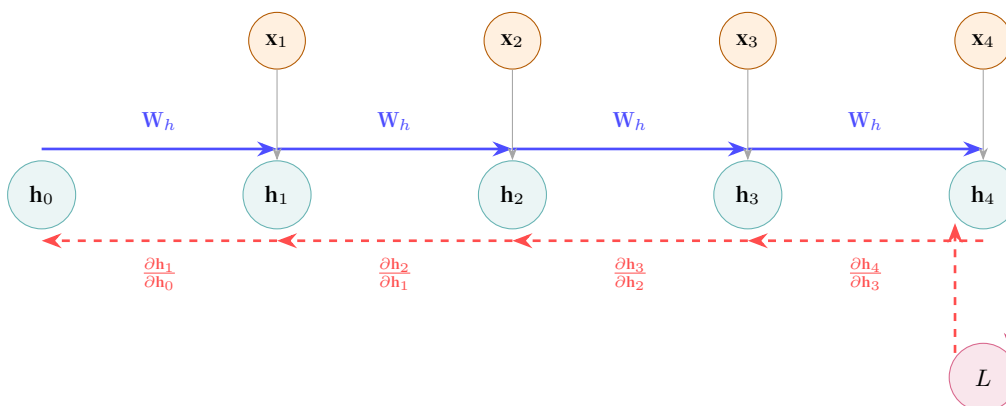
注意这里用的是 $\tanh$ 而不是 ReLU 。这是因为循环连接会让信号反复通过激活函数—— ReLU 的正区间可能让激活值在循环中不断放大直至爆炸, 而 $\tanh$ 天然有界, 提供了一种"自我保护"机制 (激活函数中讨论过各激活函数的特性)。

8.2.3 RNN 的训练: 穿过时间的反向传播 (BPTT)

RNN 如何学习? 答案就在 反向传播算法 和 计算图 中。

把 RNN 在时间上展开: 时刻 1 的处理产生 h_1 , 传给时刻 2 的使用; 时刻 2 产生 h_2 , 传给时刻 3.....这恰好形成了一个**计算图**——每个时间步是一个节点, $h_{t-1} \rightarrow h_t$ 是连接节点的边。

反向传播在这个展开的图上运行, 称为 **BPTT (Backpropagation Through Time)** [Wer90]: 损失 L 对 h_1 的梯度



梯度沿时间反向传播 (每步乘一次 Jacobian)

图 2: BPTT: 梯度沿时间反向流动

需要穿越所有中间隐状态:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_4} \cdot \frac{\partial h_4}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1}$$

这正是 多层网络的 Jacobian 连乘与梯度消失 中讨论的 **Jacobian 连乘** 的序列版本——每个 $\frac{\partial h_t}{\partial h_{t-1}}$ 是一个 Jacobian 矩阵, 经过 T 步连乘后, 梯度呈指数级衰减 (或爆炸)。

重要

RNN 训练的根本矛盾: 要学习长程依赖, 梯度必须穿越很长的路径。但路径越长, 连乘的 Jacobian 越多, 梯度消失越严重——网络"记不住"太远的信息。

8.2.4 RNN 的两个核心问题

问题一：长程依赖困难

从信息流动的直觉来理解：

- $\mathbf{h}_1$ 的信息要传递到 $\mathbf{h}_{100}$ ，需要经过 99 次 $\tanh(\mathbf{W}_h \cdot)$ 变换
- 每一次变换都是一次"有损压缩"——就像反复复印一份文件，复印 100 次后，原文已面目全非
- 原始信号的哪些部分被保留、哪些被丢弃，网络无法直接控制

生活类比

RNN 的传话游戏：100 个人排成一列传一句话，第一个人说"今天天气很好"，但每个人传话时只能听到上一个人说的。到第 100 个人时，可能已经变成"今天有飞机"。

注意力机制的传话游戏：第 100 个人能同时听到前面 99 个人说的话，然后自己判断谁说的最关键。

问题二：梯度消失

梯度消失 (Vanishing Gradient) 中讨论过，假设每层 Jacobian 的"放大倍数"为 γ ，经过 n 层连乘后为 γ^n 。

RNN 中，这个 n 不是网络深度，而是**序列长度**。考虑 $\tanh$ 的导数：

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \in (0, 1]$$

在大部分输入区域， $\tanh$ 的梯度远小于 1（饱和时接近 0）。假设平均 $\gamma = 0.5$ ：

序列长度	梯度衰减	能否学到依赖？
10 个词	$0.5^{10} \approx 0.001$	勉强可以
50 个词	$0.5^{50} \approx 8.9 \times 10^{-16}$	基本不可能
100 个词	$0.5^{100} \approx 7.9 \times 10^{-31}$	完全不可能

这意味着：**RNN 只能学会几十步以内的依赖关系**。更长的依赖（如段首的主题词与段尾的结论词之间的关联）在训练中几乎收不到梯度信号。

RNN 用一个隐状态同时承担"存储记忆"和"对外输出"——这个设计本身就是矛盾的。下一节 **LSTM：给记忆装上"门"** 中，我们将看到如何通过**分离两个状态、引入三个门控**来大幅缓解梯度消失——为梯度创建一条不经过 $\tanh$ 的"高速公路"。

8.2.5 代码实践：RNN 的梯度消失有多严重？

```
import torch
import torch.nn as nn

# RNN 参数: input_size=32 (输入特征维度), hidden_size=64 (隐状态维度)
# 参数量 = W_h(64×64) + W_x(64×32) + b(64) + W_y(64×32: 默认输出维度=输入维度)
#          = 4096 + 2048 + 64 + 2048 = 8256
rnn = nn.RNN(input_size=32, hidden_size=64, num_layers=1)

# 输入: 50 个时间步的序列 (seq_len=50, batch=1, features=32)
# 理论对应 {ref}`rnn-problems` 中的序列长度 vs 梯度衰减表
x = torch.randn(50, 1, 32)

# 前向传播 —— 沿时间步串行执行 (无法并行, {ref}`rnn-problems`)
#   output: (50, 1, 64) —— 每一步的隐状态 h_1...h_50
#   h_n:    ( 1, 1, 64) —— 最后一步的隐状态 (等于 output[-1])
output, h_n = rnn(x)

# 用最后一步的隐状态求和作为损失
# 这模拟了"基于整个序列的最终理解做预测"的典型 RNN 用法
loss = h_n.sum()
loss.backward()

# 查看 W_x 的梯度 —— 它接收从第 50 步一路回传到第 1 步的信号
# 由于 {ref}`gradient-vanishing`, 如果权重初始化不当或序列过长
# 靠近输入层的梯度几乎为 0
print(f"W_ih grad norm: {rnn.weight_ih_l0.grad.norm():.10f}")
# 典型输出: 随机初始化下结果不一, 梯度小时可达 1e-6 或更小
# 关键理解趋势: 序列越长, 范数越小 —— 而非依赖具体数值
```

本节小结

- RNN 通过循环连接给了神经网络"记忆" —— 隐状态随输入逐步更新
- BPTT 是反向传播在时间展开的计算图上的应用, 本质是 Jacobian 连乘
- RNN 的两个核心问题: **长程依赖困难** (信息经过多次变换后失真) 和 **梯度消失** (梯度指数级衰减到 0)
- 这两个问题源于同一个根源: **信息必须一步步穿过循环连接**
- LSTM 缓解了梯度消失, 但串行处理的根本限制仍在

下一节 **LSTM：给记忆装上"门"** 中，我们将看到 LSTM 如何通过门控机制为梯度开辟一条"高速公路"——这让我们在处理序列时走得更远，但串行传递的根本矛盾仍然存在。

8.3 LSTM：给记忆装上"门"

循环神经网络：让网络拥有"记忆"揭示了 RNN 的根本矛盾： $\mathbf{h}_t$ 必须同时承担两件互相冲突的事——**记住历史和输出当前结果**。一个向量，两种使命，顾此失彼。

就像一个人既要去做会议记录，又要即兴发言——记录要求忠实、不遗漏，发言要求提炼、有取舍。用同一个大脑状态同时做这两件事，结果往往是两个都做不好。

关键问题：能不能把"记忆存储"和"对外输出"分开，各司其职？

8.3.1 历史背景

1991 年，Sepp Hochreiter 在其导师 Jürgen Schmidhuber 的指导下，首次在毕业论文中分析了 RNN 梯度消失的数学根源——这正是我们在 **RNN 的两个核心问题** 中讨论的 $\tanh$ 导数连乘问题。基于这一分析，他们在 1997 年正式提出了 **Long Short-Term Memory (LSTM)** [HS97]。

LSTM 的核心洞察至今仍是序列建模的基石：**用门控机制管理信息流动**。此后 20 年，LSTM 主导了语音识别、机器翻译、手写识别等几乎所有序列任务，直到 Transformer 出现。

备注

历史趣闻：LSTM 的论文最初被 NeurIPS 拒稿，审稿人认为"这个想法太复杂，而且 RNN 的问题可以通过更好的初始化解决"。如今它是 20 世纪被引用最多的深度学习论文之一。

LSTM [HS97] 的回答是：**能。用两个状态，三个门。**

8.3.2 核心洞察：从"一个状态"到"两个状态"

RNN 只有一个隐状态 $\mathbf{h}_t$ ，它既是"当前的理解"（输出用），又是"历史的摘要"（传给下一步）。信息在每一步都被 $\tanh$ 压缩，长此以往，细节尽失。

LSTM 把这两个职责拆开：

	RNN	LSTM
状态数量	1 个 ($\mathbf{h}_t$)	2 个 ($\mathbf{c}_t$ 和 $\mathbf{h}_t$)
$\mathbf{c}_t$ (细胞状态)	无	长期记忆： 信息的"传送带"，缓慢变化
$\mathbf{h}_t$ (隐状态)	身兼二职	对外输出： 当前时刻的"公开发言"

RNN: 一个状态身兼二职

既存储历史, 又对外输出

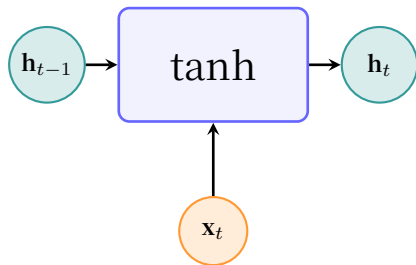
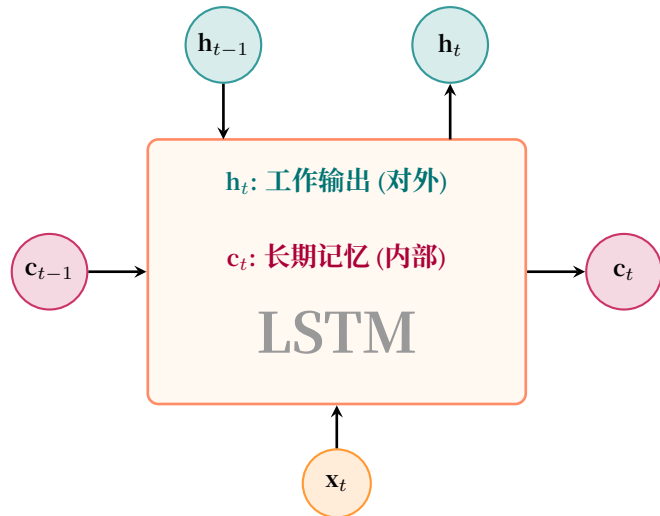
**LSTM: 两个状态各司其职**

图 3: RNN vs LSTM: 从单状态到双状态

备注

细胞状态 c_t 是 LSTM 的灵魂。它像一条贯穿时间的“信息传送带”——信息在上面流动时，只有少数地方会被门控修改。这与 [ResNet: 高原反应补给站](#) 中残差连接的思想如出一辙：大部分信号原封不动地通过，只在必要处做修改。

8.3.3 四个角色：LSTM 的门控系统

有了分离的“长期记忆” (c_t)，自然需要一个机制来管理它——什么时候写入、什么时候擦除、什么时候读出。这个机制就是三个门 (gate)，外加一个候选记忆的提议。

每个门的核心公式都长这样：

$$\text{gate}(\mathbf{h}_{t-1}, \mathbf{x}_t) = \sigma(\mathbf{W} \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b})$$

σ (Sigmoid) 把输出压到 (0, 1)，天然适合做“门”——0 表示“完全关闭”，1 表示“完全打开”，中间值表示“部分开放”。

下面我们逐个认识这四个角色。

角色一：遗忘门——“档案管理员”**遗忘门的职责**

一句话：面对旧记忆 c_{t-1} ，决定哪些内容该丢掉。

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

逐步拆解遗忘门：

符号	含义	维度	直觉理解
$[\mathbf{h}_{t-1}, \mathbf{x}_t]$	拼接上一时刻的输出和当前输入	$\mathbb{R}^{d+m}$	把"历史上下文"和"新读到的词"摆在一起
$\mathbf{W}_f$	遗忘门的权重矩阵	$\mathbb{R}^{d \times (d+m)}$	学到"什么样的上下文暗示该遗忘"
$\mathbf{b}_f$	遗忘门的偏置	$\mathbb{R}^d$	通常初始化为 1（偏向"先别忘"，训练中再调整）
σ	Sigmoid 函数	输出 $(0, 1)^d$	将线性组合映射为"保留比例"：0 = 全忘，1 = 全留
$\mathbf{f}_t$	遗忘门输出	$(0, 1)^d$	每个维度一个保留指令 —— $\mathbf{c}_{t-1}$ 的第 i 维保留 $\mathbf{f}_t^{(i)} \times 100\%$

备注

为什么偏置初始化为 1 而不是 0？在训练初期，网络还不知道该忘什么。如果遗忘门默认输出 0（全忘），梯度高速公路上全是"关闭"的闸门 —— 信息流直接断裂，无法学习长程依赖。初始化为 1 意味着"默认全保留，需要时才忘掉" —— 先保证信息能流动，再逐步学会选择性遗忘。

直觉：你正在读一篇文章。读到新的段落时，你需要判断 —— 前面提到的某个细节现在还重要吗？"遗忘门"就是用当前上下文（ $\mathbf{h}_{t-1}$ 和 $\mathbf{x}_t$ ）来判断旧记忆 $\mathbf{c}_{t-1}$ 中每个维度的去留。比如读到"然而"这个词时（ $\mathbf{x}_t$ ），你知道前面的论述可能需要被"遗忘"一部分，为新的转折腾出空间。

角色二：输入门 + 候选记忆 —— “采购员”和“新提案”

输入门的职责

一句话：面对当前时刻的新信息，决定哪些内容值得写入长期记忆。

输入门的特殊之处在于它由两个组件配合完成 —— **输入门本身**（决定"在哪里写"）和**候选记忆**（提供"写什么内容"）：

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

逐步拆解输入门：

符号	含义	维度	直觉理解
$\mathbf{W}_i$	输入门的权重矩阵	$\mathbb{R}^{d \times (d+m)}$	学到"什么样的上下文值得记录"
$\mathbf{b}_i$	输入门的偏置	$\mathbb{R}^d$	控制默认的"记录意愿" (通常初始化为 0)
σ	Sigmoid 函数	输出 $(0, 1)^d$	每个维度输出一个"写入许可": 0 = 不写, 1 = 全写
$\mathbf{i}_t$	输入门输出	$(0, 1)^d$	权限 —— 对候选记忆的每个维度, 允许多大比例写入

逐步拆解候选记忆:

符号	含义	维度	直觉理解
$\mathbf{W}_c$	候选记忆的权重矩阵	$\mathbb{R}^{d \times (d+m)}$	学到"如何从上下文提炼新记忆内容"
$\mathbf{b}_c$	候选记忆的偏置	$\mathbb{R}^d$	偏置项
$\tanh$	双曲正切函数	输出 $(-1, 1)^d$	将候选内容压缩到有界范围, 防止数值发散
$\tilde{\mathbf{c}}_t$	候选记忆	$(-1, 1)^d$	内容 —— 基于当前上下文提出的新信息草稿

为什么需要两个组件配合?

备注

$\tilde{\mathbf{c}}_t$ 像一个"提案人", 提出一份新内容的草稿 (用 $\tanh$ 压缩到 $(-1, 1)$, 类似于 LSTM 内部的"标准信息格式")。
 $\mathbf{i}_t$ 像一位"审批官", 决定这份提案的哪些部分可以进入正式记录。

为什么"产生内容"和"决定是否采纳"要分开?

- 你可以提出很多想法 ($\tilde{\mathbf{c}}_t$ 的各个维度都可能非零) —— 这是**信息生成**
- 但只采纳与当前上下文相关的部分 ($\mathbf{i}_t$ 选择性放行) —— 这是**信息筛选**
- 分离这两个职责, 让网络学会"大胆提案, 谨慎采纳"

两个激活函数的分工:

对比维度	Sigmoid ($\mathbf{i}_t$)	$\tanh$ ($\tilde{\mathbf{c}}_t$)
输出范围	$(0, 1)$	$(-1, 1)$
角色	门控 —— 决定"写多少"	内容 —— 决定"写什么"
语义	天然适合做比例/概率	天然适合做有正有负的特征值
类比	水龙头的阀门 (0=关, 1=开)	水管里的水 (水本身的内容)

角色三：输出门——“发言人”

输出门的职责

一句话：从长期记忆 $\mathbf{c}_t$ 中提取当前需要对外输出的内容。

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

逐步拆解输出门：

符号	含义	维度	直觉理解
$\mathbf{W}_o$	输出门的权重矩阵	$\mathbb{R}^{d \times (d+m)}$	学到"当前上下文需要输出记忆中的哪些部分"
$\mathbf{b}_o$	输出门的偏置	$\mathbb{R}^d$	控制默认的"输出意愿"
σ	Sigmoid 函数	输出 $(0, 1)^d$	每个维度输出一个"暴露许可": 0 = 隐藏, 1 = 公开
$\mathbf{o}_t$	输出门输出	$(0, 1)^d$	决定了 $\mathbf{c}_t$ 中哪些维度应出现在 $\mathbf{h}_t$ 中

$\mathbf{h}_t$ 的生成——两步走：

第一步： $\tanh(\mathbf{c}_t)$ —— 将细胞状态压缩到 $(-1, 1)$

这一步有两个作用：

- 防止输出值过大 ($\mathbf{c}_t$ 中的值理论上可以无限增长，因为遗忘门允许 $f_t = 1$ 的累积)
- 将记忆归一化到标准范围，方便后续层处理

第二步： $\mathbf{o}_t \odot \tanh(\mathbf{c}_t)$ —— 选择性暴露

$\mathbf{o}_t$ 像一个"发言人"，它基于当前需要 ($\mathbf{h}_{t-1}$ 和 $\mathbf{x}_t$) 决定：记忆中的哪些部分应该体现在当前输出中。比如，当预测下一个词是动词时，输出门可能重点暴露记忆中关于"动作"的维度，而压抑关于"形容词"的维度。

备注

直觉：你拥有完整的长期记忆 $\mathbf{c}_t$ （整本书的内容），但当别人问你"刚才讲了什么"时，你只会说出与当前问题相关的部分。这个"根据当前需要筛选发言内容"的过程，就是 $\mathbf{o}_t$ 在做的事。

8.3.4 完整更新公式

把四个角色组合在一起，LSTM 的一步更新为：

LSTM 完整公式

门控计算（所有门共享相同的输入 $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ ）：

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{遗忘门}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{输入门}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad \text{候选记忆}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{输出门}$$

状态更新：

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{记忆更新}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{对外输出}$$

其中 $\odot$ 表示逐元素乘法（Hadamard 积）。

记忆更新公式详解： $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

项	含义	直觉
$\mathbf{f}_t \odot \mathbf{c}_{t-1}$	遗忘门 $\times$ 旧记忆	保留部分 —— 旧记忆中通过遗忘门筛选后留下的
$\mathbf{i}_t \odot \tilde{\mathbf{c}}_t$	输入门 $\times$ 候选新信息	新增部分 —— 候选信息中通过输入门批准写入的
+	逐元素加法	新旧混编 —— 记忆的更新是“删掉一些旧的，添加一些新的”

读作：新记忆 = 旧记忆中保留下的 + 新信息中被批准写入的。

备注

四类参数的规模： 设隐状态维度为 d ，输入维度为 m 。每个门都是一个 $\mathbb{R}^{d+m} \rightarrow \mathbb{R}^d$ 的线性映射，参数量为 $d \times (d + m) + d$ 。四个门总计 $4d(d + m + 1)$ 个参数。对于一个 $d = 256$ 、 $m = 128$ 的 LSTM，约为 $4 \times 256 \times 385 \approx 394,240$ 个参数 —— 是同等 RNN 的 4 倍，但换来了可控的长期记忆。

8.3.5 为什么 LSTM 能缓解梯度消失：梯度高速公路

回顾 梯度消失（Vanishing Gradient）中的分析：梯度消失的根源是 Jacobian 连乘。RNN 中， $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$ 包含 $\tanh$ 的导数，它始终 ≤ 1 ，连乘后指数衰减。

LSTM 的关键差异在于细胞状态 $\mathbf{c}_t$ 的梯度路径：

对 $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ 求导：

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t \quad (\text{对角矩阵, 每个维度独立})$$

²⁵ <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

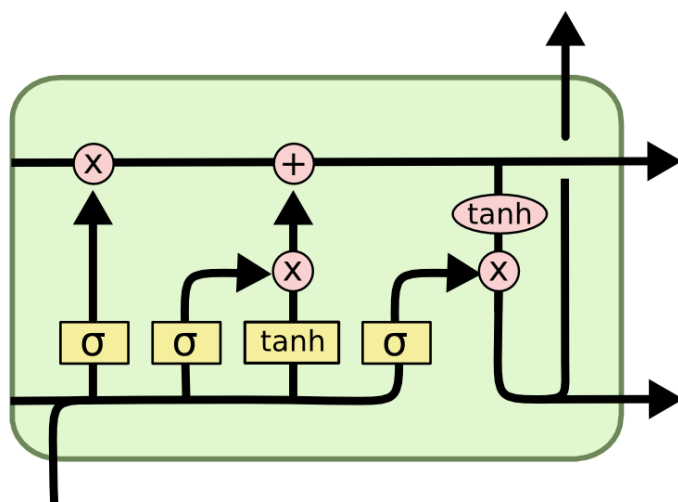


图 4: LSTM 内部的门控结构。信息沿顶部的水平线（细胞状态 c_t ）流动 —— $\times$ 表示门控的逐元素乘法， $+$ 表示新旧信息的合并。图片来自 Chris Olah 的博客“[Understanding LSTM Networks](#)^{Page 489, 25}”——深度学习领域最经典的可视化之一。

核心洞察： c_{t-1} 对 c_t 的梯度就是遗忘门 f_t 本身——没有 $\tanh$ ，没有复杂的非线性。如果 $f_t \approx 1$ （网络选择“保留”），梯度几乎无损地穿过这一时间步。

类比：传送带 vs 搅拌机

- **RNN：**信息每步都经过 $\tanh$ 的“搅拌”——100 步后，原始内容面目全非，梯度也一样
- **LSTM：**信息在一条**传送带**（ c_t ）上流动。遗忘门 f_t 只在传送带上打开小的“丢弃口”——大部分东西原封不动地通过。梯度也沿这条传送带几乎无损地回传

这与 ResNet：高原反应补给站 中 $x + F(x)$ 的设计哲学完全一致：让梯度有一条不经过非线性的捷径。

表 1: RNN vs LSTM 梯度传播对比

对比维度	RNN	LSTM
梯度路径	$\frac{\partial h_t}{\partial h_{t-1}}$ 包含 $\tanh'$ ，始终 ≤ 1	$\frac{\partial c_t}{\partial c_{t-1}} = f_t$ ，可接近 1
100 步后梯度	$\approx 0.5^{100} \approx 10^{-31}$ （消失）	若 $f_t \approx 1$ ，梯度几乎不变
能否学到千步依赖	几乎不可能	理论上可以（实践中困难但可行）

8.3.6 LSTM 仍有局限

LSTM 解决了梯度消失，但没有解决所有问题：

1. **仍是串行处理**： $\mathbf{h}_t$ 必须等 $\mathbf{h}_{t-1}$ 算完。序列长度 1000 意味着 1000 次串行计算，无法并行训练
2. **固定尺寸的瓶颈**：所有历史信息必须压缩进一个固定维度的 $\mathbf{c}_t$ 。序列越长，压缩越有损——就像把一本书总结成一段话，总有信息丢失
3. **门控也会饱和**：当 $\mathbf{f}_t$ 在训练中饱和为 0（网络学会了“这段必须忘掉”），梯度仍然会切断——只是比 RNN 更可控

备注

LSTM 的贡献不是“解决了所有问题”，而是**把问题的维度降低了**：从“梯度指数衰减到 0”变成“网络可以通过学习门控来调控梯度流动”。这为深度学习处理序列打开了大门——此后 20 年里，LSTM 是 NLP 领域的主力架构，直到 Transformer 出现。

8.3.7 代码实践：LSTM 的梯度保持能力

```
import torch
import torch.nn as nn

# RNN: 参数量 =  $W_h(64 \times 64) + W_x(64 \times 32) + b(64) + W_y(64 \times 32) = 8,256$ 
rnn = nn.RNN(input_size=32, hidden_size=64, num_layers=1)

# LSTM: 四个门, 每个门 =  $W(64 \times 96) + b(64) = 6,208$ 
#         四个门总计 = 24,832 (约 RNN 的 3 倍)
#         外加输出投影权重不计入 (PyTorch 默认不自动回归输出)
lstm = nn.LSTM(input_size=32, hidden_size=64, num_layers=1)

# 长序列测试 —— 100 步, 梯度必须穿越 100 次门控操作
x = torch.randn(100, 1, 32) # (seq_len=100, batch=1, features=32)

# RNN 前向 —— 隐状态  $\mathbf{h}_t$  每一步经过 {ref}`lstm-gradient-highway` 中描述的 tanh 压缩
rnn_out, _ = rnn(x) # rnn_out: (100, 1, 64)
rnn_out[-1].sum().backward() # 只对最后一步求梯度
# 查看输入权重的梯度范数 —— 由于梯度消失 ({ref}`gradient-vanishing`), 接近 0
print(f"RNN   W_ih grad norm: {rnn.weight_ih_l0.grad.norm():.10f}")

# LSTM 前向 —— 输出 (output, (h_n, c_n))
#   output: (100, 1, 64) —— 每一步的隐状态 (对外输出)
#   h_n:    ( 1, 1, 64) —— 最后一步的隐状态
```

(续下页)

(接上页)

```
# c_n: ( 1, 1, 64) — 最后一步的细胞状态（长期记忆）
lstm_out, (h_n, c_n) = lstm(x)
lstm_out[-1].sum().backward()
# LSTM 的输入权重梯度显著更大 — 遗忘门 f_t 为梯度保留了通路 ({ref}`lstm-gradient-highway`)
print(f"LSTM W_ih grad norm: {lstm.weight_ih_l0.grad.norm():.10f}")
# 典型输出：LSTM 的梯度范数比 RNN 大 2-3 个数量级
```

本节小结

- LSTM 的核心洞察：把"记忆存储" (c_t) 和"对外输出" (h_t) 分离
- 三个门各司其职：遗忘门（擦除）、输入门（写入）、输出门（读取）
- 细胞状态 c_t 的梯度路径几乎不经过非线性 —— 这是缓解梯度消失的关键
- LSTM 缓解了梯度消失，但串行处理和固定状态尺寸的根本限制仍在 —— 这是下一跳的起点

掌握了 LSTM 如何用门控机制"管理记忆"后，我们面临一个更深层的问题：即使有了门控，信息还是要一步步传递 —— h_1 到 h_{100} 必须经过 99 步。下一节从 RNN 到注意力：一个思想的跳跃 中，我们将看到一种彻底颠覆的思路：为什么不让第 100 步直接看到第 1 步？

8.4 从 RNN 到注意力：一个思想的跳跃

循环神经网络：让网络拥有"记忆" 揭示了 RNN 的困境： h_t 只能通过 h_{t-1} 间接获取前序信息。LSTM：给记忆装上"门" 用门控机制为梯度开辟了高速公路，但信息仍是串行传递的 —— 经过几十个 $f_t \odot c_{t-1}$ 操作后， h_1 的信息在 c_{100} 中终究会衰减。

关键问题：如果 h_t 能直接看到 $h_1, h_2, \dots, h_{t-1}$ 全部历史，而不只是 h_{t-1} ，会怎么样？

8.4.1 直觉跳跃：从"传话"到"查档案"

两个处理模式

	RNN 的传话模式	注意力的查档模式
信息获取	只听上一个人说	直接查阅所有前序记录
信息质量	逐次衰减	不受距离影响
并行性	串行， t 步等 $t-1$ 步完成	所有步骤可同时计算

RNN 就像传话游戏：第 100 个人想知道第 10 个人说了什么，只能通过中间 90 个人的转述。每转述一次，信息就损失一点。

注意力的做法完全不同：第 100 个人能看到所有人的“原始发言记录”，然后自己判断哪些发言对当前理解最重要。

备注

这不是一个全新的想法。归纳偏置告诉我们，好的架构把对任务结构的理解内置到设计中。RNN 的归纳偏置是“时序因果性”—— t 时刻只能依赖 $< t$ 的信息。注意力保留了这一偏置，但改变了信息的传递方式：不再通过循环连接一步步传递，而是直接建立全局连接。

如果你学过 ResNet：高原反应补给站，会发现这和残差网络的思想完全一致——用一条捷径绕开衰减路径。ResNet 的跳跃连接让梯度在深度方向绕过非线性层直传；注意力让信息在时间方向绕过中间步骤直达。两者的核心洞察是同一个：不让信号被迫穿过每一次变换，给它一条直接通路。

8.4.2 数学推导：因果注意力的自然诞生

我们要解决的问题：在时刻 t ，给定所有前序隐状态 $\{\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_{t-1}\}$ （以及当前输入的信息），如何计算出当前的最佳表示？

步骤一：衡量“相关性”

首先需要有一个机制来判断哪些前序状态对当前时刻重要。最自然的方式是用相似度：

$$s_{t,i} = \text{score}(\mathbf{h}_t, \mathbf{h}_i), \quad i < t$$

最简单的相似度是点积：

$$s_{t,i} = \mathbf{h}_t \cdot \mathbf{h}_i$$

如果 $\mathbf{h}_t$ 和 $\mathbf{h}_i$ 方向相似，点积大；方向相反，点积小（甚至为负）。

步骤二：归一化为权重

将相似度分数用 Softmax 转为概率分布（权重）：

$$\alpha_{t,i} = \frac{\exp(s_{t,i})}{\sum_{j=1}^{t-1} \exp(s_{t,j})}, \quad i < t$$

$\alpha_{t,i}$ 表示“在理解第 t 步时，第 i 步的信息有多重要”。所有权重之和为 1。

步骤三：加权聚合

用这些权重对前序状态加权求和：

$$\mathbf{c}_t = \sum_{i=1}^{t-1} \alpha_{t,i} \cdot \mathbf{h}_i$$

$\mathbf{c}_t$ 称为上下文向量（context vector）——它是所有前序信息的“加权摘要”，权重由当前需求动态决定。

这就是因果注意力

$$\text{CausalAttention}(\mathbf{h}_t, \{\mathbf{h}_1 \dots \mathbf{h}_{t-1}\}) = \sum_{i=1}^{t-1} \text{softmax}(\mathbf{h}_t \cdot \mathbf{h}_i) \cdot \mathbf{h}_i$$

"因果"指的是： t 时刻只能看到过去 ($i < t$)，不能看到未来。这是序列建模的自然约束 —— 你不能用还没说出的话来理解当前词。

步骤四：分离"查询"和"被查询" —— 引入可学习的投影

步骤三的公式已经能工作，但有一个微妙的局限。在整个序列的计算中，**同一个隐状态向量 $\mathbf{h}$** 会被迫扮演三个不同的角色：

- 当它是当前时刻的查询出发点时 ($\mathbf{h}_t$)，它需要表达**"我在找什么？"** (Query)
- 当它是被查询的历史记录时 ($\mathbf{h}_i$)，它需要表达**"我有什么特征可以被匹配？"** (Key)
- 当它被选中并参与聚合时 ($\mathbf{h}_i$)，它还需要表达**"我能提供什么具体信息？"** (Value)

在步骤三的公式 $\mathbf{c}_t = \sum_{i=1}^{t-1} \text{softmax}(\mathbf{h}_t \cdot \mathbf{h}_i) \cdot \mathbf{h}_i$ 中，这种冲突直接体现在了 $\mathbf{h}_i$ **出现了两次**：一次在点积中充当 Key，一次在加权求和中充当 Value。同一个向量必须同时回答"我是什么"和"我能提供什么" —— 这两个目标可能完全不同。

备注

一个具体的例子：假设 $\mathbf{h}_2$ 是名词"猫"的表示。当未来的动词"跑" ($\mathbf{h}_5$) 来查询它时：

- 作为 **Key**： $\mathbf{h}_2$ 需要暴露"我是名词、生物、可以做主语"的索引特征，以便和 $\mathbf{h}_5$ 的查询向量匹配
- 作为 **Value**： $\mathbf{h}_2$ 需要提供"猫"的完整语义信息，以便聚合到上下文中

如果 $\mathbf{h}_2$ 必须用同一套数值同时满足"做索引标签"和"做内容载体"两个目标，表达力会严重受限 —— 它无法针对不同职能做专门的优化。

解决方案很自然：**给每个向量三个可学习的投影**，让它能根据"当前扮演什么角色"来调整自己的表达 [VSP+17]：

- **Query $\mathbf{q}_t = \mathbf{W}_q \mathbf{h}_t$** (查询)：从当前时刻出发的检索需求
- **Key $\mathbf{k}_i = \mathbf{W}_k \mathbf{h}_i$** (键)：暴露自己供别人检索的特征
- **Value $\mathbf{v}_i = \mathbf{W}_v \mathbf{h}_i$** (值)：实际被聚合的内容

对比步骤三的原始公式和推广后的公式：

原始 ($\mathbf{h}_i$ 身兼 Key/Value 两职):

$$\mathbf{c}_t = \sum_{i=1}^{t-1} \text{softmax}(\mathbf{h}_t \cdot \mathbf{h}_i) \cdot \mathbf{h}_i$$

推广后 (Q/K/V 三个角色分离):

$$\mathbf{c}_t = \sum_{i=1}^{t-1} \text{softmax}((\mathbf{W}_q \mathbf{h}_t) \cdot (\mathbf{W}_k \mathbf{h}_i)) \cdot (\mathbf{W}_v \mathbf{h}_i) = \sum_{i=1}^{t-1} \text{softmax}(\mathbf{q}_t \cdot \mathbf{k}_i) \cdot \mathbf{v}_i$$

可以看到: 如果 $\mathbf{W}_q = \mathbf{W}_k = \mathbf{W}_v = \mathbf{I}$ (单位矩阵), 两式完全等价 —— 步骤三的简单版本是步骤四的一个特例。

除以 $\sqrt{d_k}$, 即:

$$\text{Attention}(\mathbf{q}_t, \mathbf{K}, \mathbf{V}) = \sum_{i=1}^{t-1} \text{softmax}\left(\frac{\mathbf{q}_t \cdot \mathbf{k}_i}{\sqrt{d_k}}\right) \cdot \mathbf{v}_i$$

为什么除以 $\sqrt{d_k}$?

当 Key 的维度 d_k 很大时, 点积 $\mathbf{q} \cdot \mathbf{k}$ 的方差约为 d_k 。大点积值会让 Softmax 进入饱和区 —— 大部分 α 趋近于 0 或 1, 梯度几乎为 0。除以 $\sqrt{d_k}$ 将方差缩放到 1, 保持 Softmax 输出平滑、梯度健康。这被称为**缩放点积注意力 (Scaled Dot-Product Attention)**。

类比: 图书馆检索

- **Query**: 你要找的问题 (“关于深度学习的书”)
- **Key**: 每本书的索引标签 (书名、关键词)
- **Value**: 书的内容
- **过程**: 用 Query 匹配 Key 得到每本书的相关性分数 $\rightarrow$ Softmax 归一化 $\rightarrow$ 基于权重从各书的 Value 中聚合信息

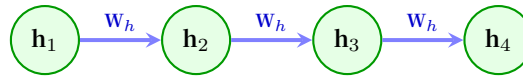
你最终得到的是所有相关书的“加权摘要”, 而非一本书的全部内容。

8.4.3 对比: RNN 串行 vs 注意力并行

表 2: RNN vs 因果注意力

对比维度	RNN	因果注意力
信息路径长度	$O(n)$: n 步才能从位置 1 传到位置 n	$O(1)$: 任何位置直接访问所有前序位置
梯度路径	穿过 n 个 Jacobian, 指数衰减	梯度主要通过 Softmax 传播, 与距离无关
并行性	必须串行: $\mathbf{h}_t$ 依赖 $\mathbf{h}_{t-1}$	训练时所有位置可并行计算 (用 mask 屏蔽未来)
归纳偏置	强: 相邻时刻更相关	弱: 所有前序位置等距 (需要位置编码补偿)

RNN: 串行依赖



信息必须一步步穿过循环（马尔可夫链）

因果注意力: 全局连接

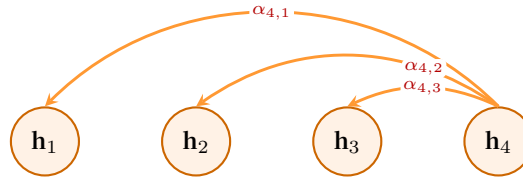
 h_4 直接从历史状态 $h_{1,2,3}$ 中“跨时空”提取信息

图 5: RNN 的串行依赖 vs 注意力的全局连接

8.4.4 代码实践：因果注意力的实现

```
import torch
import torch.nn.functional as F

def causal_self_attention(x):
    """
    输入: x, shape (batch, seq_len, d_model)
    输出: context, shape (batch, seq_len, d_model)

    理论对应 {ref}`from-rnn-to-attention` 中的因果注意力公式
    """
    B, T, D = x.shape

    # 步骤 1: 线性投影得到 Q, K, V
    # 三者形状均为 (batch, seq_len, d_model)
    W_q = torch.randn(D, D) * 0.02 # 实际应用中应为可学习参数
    W_k = torch.randn(D, D) * 0.02
    W_v = torch.randn(D, D) * 0.02

    Q = x @ W_q # (B, T, D)
    K = x @ W_k # (B, T, D)
    V = x @ W_v # (B, T, D)

    # 步骤 2: 计算注意力分数矩阵
    # Q @ K^T 的结果形状为 (B, T, T)
    # scores[b, t, i] 表示第 t 个位置的查询与第 i 个键的相似度
```

(续下页)

(接上页)

```

d_k = D
scores = (Q @ K.transpose(-2, -1)) / (d_k ** 0.5)

# 步骤 3: 因果 mask —— 禁止关注未来
# mask 是一个上三角为 -inf、下三角为 0 的矩阵
mask = torch.triu(torch.ones(T, T), diagonal=1).bool()
scores = scores.masked_fill(mask, float('-inf'))

# 步骤 4: Softmax 归一化 (在最后一个维度上, 即所有 Key 上)
# -inf 位置的  $\exp(-\infty) = 0$ , 实现"禁止关注未来"
attn_weights = F.softmax(scores, dim=-1) # (B, T, T)

# 步骤 5: 加权聚合 Value
# attn_weights[b, t, :] 对 V[b, :, :] 的所有行加权求和
context = attn_weights @ V # (B, T, D)
return context

# 测试
x = torch.randn(2, 5, 64) # batch=2, seq_len=5, d_model=64
output = causal_self_attention(x)
# output[0, 2, :] 只聚合了 x[0, 0:3, :] 的信息 (位置 0,1,2)
# output[0, 2, :] 看不到 x[0, 3:5, :] (位置 3,4 是"未来")

```

备注

因果 mask 的直觉: 在 Softmax 前将未来位置的分数设为 $-\infty$, 因为 $\exp(-\infty) = 0$, 这些位置获得 0 权重。这与 归纳偏置 中时序因果性的要求一致 —— t 时刻的预测只能基于 $< t$ 的信息。

本节小结

- RNN 的串行传话模式导致长程信息丢失, 根本原因是信息必须一步步穿过循环连接
- 注意力的核心思想: **让每一步直接加权聚合所有前序信息**, 跳过中间步骤
- 因果注意力 = 相似度计算 (点积/缩放点积) → Softmax 归一化 → 加权求和
- 引入 Q/K/V 三个角色将"查询"和"被查询"的职能分离, 增强表达力
- 注意力训练时可并行 (用 mask 屏蔽未来), 这是相对于 RNN 的巨大优势

从"为什么需要注意力"到"注意力如何工作", 我们已经完成了核心的思想建设。下一节 Transformer: 注意力

的极致中，我们将看到这一思想如何被推向极致——完全抛弃循环，构建一个纯粹的注意力架构。

8.5 Transformer：注意力的极致

从 RNN 到注意力：一个思想的跳跃中，因果注意力已经可以把每个位置与所有前序位置直接关联。但你可能会问：既然注意力这么好，为什么还要保留 RNN 的循环结构？

Transformer 的回答是：不要了。把 RNN 全部扔掉，整个架构只靠注意力来传递信息。这就是“Attention Is All You Need” [VSP+17] 的核心主张。

8.5.1 历史背景

2014 年，Bahdanau 等人提出了注意力机制 [BCB15]，让解码器在生成每个词时能“回顾”编码器的所有隐状态——我们已在从 RNN 到注意力：一个思想的跳跃中推导了这一思想的起源。但 Bahdanau 的架构仍以 RNN 为骨架，注意力只是“外挂”。

2017 年，Vaswani 等人在“Attention Is All You Need”中做了一个大胆的决定：完全抛弃 RNN。他们证明，纯注意力架构不仅在翻译质量上超越了当时最好的 RNN 模型，而且训练更快（因为可以并行）。

备注

历史意义：“Attention Is All You Need”或许是深度学习史上影响力最大的单篇论文。它不仅创造了 Transformer，还催生了 BERT、GPT、ViT 等几乎所有现代大模型的架构基座。7 年后的今天，Transformer 仍是工业界和学术界的主流选择。

8.5.2 从因果注意力到 Transformer 的四个升级

升级一：自注意力——所有位置互相关注

从 RNN 到注意力：一个思想的跳跃中实现的是因果注意力—— t 只能看到 $< t$ 。但 Transformer 的一个关键洞察是：在编码阶段（理解输入），每个位置应该能看到所有位置，包括“未来”的词。

备注

为什么？理解一个句子时，后面的词确实能帮助理解前面的词。比如“我把苹果吃了”——“吃了”帮助确定“苹果”是食物（而非手机品牌）。好的编码器应该双向理解。

但 Transformer 分两个阶段使用注意力：

阶段	注意力类型	能看到什么	目的
编码器 (Encoder)	全自注意力	所有位置 (含未来)	双向理解输入
解码器 (Decoder)	因果自注意力	只有过去	逐词生成输出

我们集中讨论核心机制——**自注意力 (Self-Attention)**：在一个序列内部，每个位置作为 Query，去关注所有位置的 Key 和 Value。

自注意力是卷积的超集

如果你已经学过 **卷积：发现装备**，这里有一个重要的联系：**自注意力在表达能力上是卷积的严格超集**。

卷积 (感受野)：一个 3×3 卷积核只看相邻 9 个像素。它对所有输入位置使用**同一套权重**（权值共享），且**感受野范围是固定的**——无论输入是什么，每个神经元看到的区域大小不变。

自注意力：通过注意力分数矩阵 $\text{softmax}(QK^T/\sqrt{d_k})$ ，每个位置可以关注**任意其他位置**。权值是**输入依赖的**——不同输入产生不同的注意力分布。感受野是**动态的**——遇到相关的内容就多关注，不相关的就忽略。

卷积是自注意力的一个特例：如果把注意力限制为只在局部邻域内计算，且固定注意力权重（不依赖输入），就退化成了卷积。[CLJ20]

备注

ViT 为什么能成功？ **卷积：发现装备** 告诉我们，CNN 成功的核心是 **归纳偏置**——局部感受野和权值共享。自注意力放弃了这些硬编码的先验，换来了更大的灵活性。在数据量充足时，灵活性胜出（ViT 超越 CNN）；在数据量不足时，CNN 的先验仍然宝贵（CNN 在小数据集上更好）。

这也是为什么现代架构设计方向之一是**让网络自己学会感受野**——自注意力做到了这一点：它不是被动接受“只看邻居”的限制，而是主动学习“该看哪里”。

升级二：多头注意力——多个视角并行

单头注意力有一个局限：在一次 Softmax 中只能表达一种“关联模式”。但语言中的关联是多维度的——同一个词可能同时关注“主语是谁”、“在描述什么动作”、“有什么修饰关系”。

多头注意力 (Multi-Head Attention) 的解决方案很简单：并行运行 h 个独立的注意力，每个头有自己的 W_q, W_k, W_v ，最后拼接输出：

$$\text{MultiHead}(\mathbf{x}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_o$$

$$\text{head}_i = \text{Attention}(\mathbf{x}W_q^i, \mathbf{x}W_k^i, \mathbf{x}W_v^i)$$

生活类比

- **单头注意力**：请一位专家审阅文档，她只能关注一个方面（如语法）
- **多头注意力**：同时请语法专家、逻辑专家、风格专家，每人从自己的角度审阅，最后汇总意见

升级三：位置编码——告诉注意力"顺序"在哪

归纳偏置告诉我们，CNN 通过局部连接内置了"相邻像素相关"的先验。但自注意力有一个致命缺陷：**它完全不知道位置信息**——"A 看到 B"和"B 看到 A"在注意力评分中是对称的（如果不加 mask），位置 1 和位置 100 在相似度计算中没有本质区别。

解决方案是**位置编码（Positional Encoding）**：在输入嵌入中加入一个仅依赖位置的信号，让注意力"知道"每个词在序列中的位置。原始 Transformer 使用正弦/余弦函数：

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

备注

为什么用正弦/余弦而不是学出来的位置编码？

1. 可以外推到训练时未见过的序列长度
2. 相对位置关系可以通过三角函数公式推导出来—— $\text{PE}(\text{pos} + k)$ 可以表示为 $\text{PE}(\text{pos})$ 的线性函数
3. 不同频率的正弦波在"位置分辨率"上互补——低频率编码大范围位置关系，高频率编码精细位置差异

升级四：完整的 Transformer Block

除了注意力，Transformer 还引入了另外两个关键组件：**残差连接 + LayerNorm**：与 ResNet：高原反应补给站的设计思想一致——为梯度创建"捷径"，让深层网络更易训练。区别在于 Transformer 用 LayerNorm（沿特征维度归一化），ResNet 多用 BatchNorm（沿批次维度归一化）[BKH16]。

前馈网络（FFN）：被低估的"知识存储器"

注意力完成"跨位置的信息交互"后，FFN 负责"每个位置内部的特征变换"：

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

表面上看，这只是一个简单的两层全连接。但它藏着 Transformer 的一个重要设计哲学。

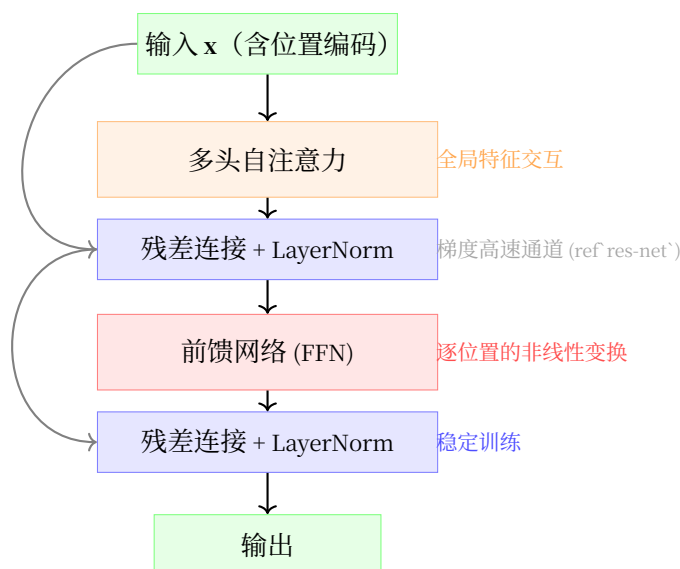


图 6: Transformer Block 结构

注意力 vs FFN 的分工

- **注意力**: “和其他位置交换信息” —— 每个位置的表示吸收其他位置的内容。操作在**序列维度**上
- **FFN**: “自己内部消化信息” —— 每个位置独立地对自己吸收来的信息做非线性处理。操作在**特征维度**上

类比: 开会时, 听别人发言是“注意力” (收集信息), 自己整理笔记是“FFN” (加工信息)。

为什么中间维度要扩大 4 倍?

FFN 采用一个“瓶颈”结构: $d_{\text{model}} \rightarrow d_{\text{ff}} \rightarrow d_{\text{model}}$, 且 $d_{\text{ff}} = 4 \times d_{\text{model}}$ 。这不是随意选的:

设计选择	直觉
先升维	将特征投影到高维空间, 让不同维度之间的交互更充分 —— 就像把一页纸的内容摊开在桌面上, 更容易发现关联
ReLU 非线性	在高维空间中做非线性“折叠” —— 激活某些模式, 抑制另一些 (回顾 激活函数)
再降维	压缩回原始维度, 只保留对当前任务最有用的特征组合

4 倍这个数字是经验性的 —— 太小 (如 2 倍) 表达能力不足, 太大 (如 8 倍) 参数量激增但收益递减。实践中 4 倍是一个广泛验证的甜点。

参数量对比 —— FFN 才是“大头”:

设 $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$:

组件	参数量	占比
多头注意力 (Q/K/V/O 四个投影)	$4 \times 512^2 \approx 1.05\text{M}$	~33%
FFN ($W_1 + W_2$)	$512 \times 2048 \times 2 \approx 2.10\text{M}$	~67%

FFN 的参数量是注意力的约 2 倍！在 GPT-3（175B 参数）中，FFN 层占总参数的大多数。这暗示了一个重要事实：Transformer 的知识主要存储在 FFN 的权重中——注意力负责“查什么”，FFN 负责“知道什么”[GSBL21]。

备注

FFN 可以看作一种键值存储器：

- W_1 的各行可以视为大量“键”（keys）——“我存储了哪些模式”
- ReLU 后的激活模式 $\text{ReLU}(\mathbf{x}W_1 + \mathbf{b}_1)$ 表示“当前输入匹配了哪些模式”
- W_2 的各列可以视为“值”（values）——“匹配后输出什么”

这种“键值存储”的视角可以解释为什么大模型能记住大量事实——FFN 的权重为每个学到的知识模式提供了存储空间。

备注

Transformer 和 下篇：真正的未登峰 中的注意力是同一种“注意力”吗？

不是。注意区分：

- **CNN 注意力 (SE-Net、CBAM)**：在卷积特征图的通道或空间维度上做重加权，目的是让网络学会“哪些特征更重要”。权重是输入依赖的标量（每个通道一个权重，或每个空间位置一个权重）。
- **Transformer 自注意力**：在序列维度上做重加权，目的是让每个位置聚合所有其他位置的信息。权重是输入依赖的矩阵（每个位置对关注所有其他位置）。

8.5.3 Transformer 的代价： $O(n^2)$ 复杂度

自注意力的核心计算是计算注意力分数矩阵：

$$\text{Scores} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}, \quad \mathbf{Q}, \mathbf{K} \in \mathbb{R}^{n \times d_k}$$

$\mathbf{Q}\mathbf{K}^T$ 的结果是一个 $n \times n$ 的矩阵——序列中每对位置之间都有一个分数。这一步的时间和空间复杂度都是 $O(n^2)$ 。

序列长度 n	注意力矩阵大小	内存占用 (fp16)
1,000	$1,000 \times 1,000$	~2 MB
10,000	$10,000 \times 10,000$	~200 MB
100,000	$100,000 \times 100,000$	~20 GB
1,000,000	$1,000,000 \times 1,000,000$	~2 TB

Transformer 的核心矛盾

Transformer 通过注意力解决了 RNN 的长程依赖问题 —— 任何两个位置之间只有 $O(1)$ 的信息路径。但代价是计算复杂度从 RNN 的 $O(n)$ 变成了 $O(n^2)$ 。对于长序列（如整本书、长视频、DNA 序列），这个代价变得不可承受。

这正是为什么现代大语言模型的上下文窗口如此"昂贵" —— 每翻倍一次上下文长度，计算量增长四倍。FlashAttention 等工程优化可以缓解常数因子，但无法改变 $O(n^2)$ 的渐进复杂性。

8.5.4 代码实践：简化的 Transformer Encoder

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadSelfAttention(nn.Module):
    """
    多头自注意力（无因果 mask，用于编码器）

    理论对应 {ref}`transformer` 中 Q/K/V 的完整推广
    """
    def __init__(self, d_model=512, n_heads=8):
        super().__init__()
        assert d_model % n_heads == 0
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads  # 每个头处理的维度

        # Q, K, V 的投影矩阵 —— 将 d_model 维映射到 d_model 维
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
```

(续下页)

(接上页)

```

self.W_o = nn.Linear(d_model, d_model) # 输出投影

def forward(self, x):
    B, T, D = x.shape # batch, seq_len, d_model

    # 步骤 1: 投影并拆分为多头
    # 将 (B, T, D) 投影为 (B, T, D), 再拆成 (B, n_heads, T, d_k)
    Q = self.W_q(x).view(B, T, self.n_heads, self.d_k).transpose(1, 2)
    K = self.W_k(x).view(B, T, self.n_heads, self.d_k).transpose(1, 2)
    V = self.W_v(x).view(B, T, self.n_heads, self.d_k).transpose(1, 2)

    # 步骤 2: 缩放点积注意力 ——  $O(n^2)$  复杂度就在这里
    # scores: (B, n_heads, T, T) —— 每个头有自己的注意力矩阵
    scores = (Q @ K.transpose(-2, -1)) / math.sqrt(self.d_k)
    attn = F.softmax(scores, dim=-1) # 按 Key 维度归一化

    # 步骤 3: 加权聚合 Value
    # (B, n_heads, T, d_k) —— 每个位置获得所有位置的加权表示
    head_outputs = attn @ V

    # 步骤 4: 合并多头输出
    # 从 (B, n_heads, T, d_k) 变回 (B, T, D)
    concat = head_outputs.transpose(1, 2).contiguous().view(B, T, D)
    return self.W_o(concat)

```

```

class TransformerEncoderBlock(nn.Module):
    """
    单个 Transformer 编码器块 ({ref}`transformer` 中的完整 Block)

    结构: 输入 → 多头注意力 (+残差+LN) → FFN (+残差+LN) → 输出
    """
    def __init__(self, d_model=512, n_heads=8, d_ff=2048):
        super().__init__()
        self.attention = MultiHeadSelfAttention(d_model, n_heads)
        self.norm1 = nn.LayerNorm(d_model) # 注意力后的归一化
        self.norm2 = nn.LayerNorm(d_model) # FFN 后的归一化

        # FFN: 两层全连接, 中间维度通常为 d_model 的 4 倍
        self.ffn = nn.Sequential(

```

(续下页)

(接上页)

```

        nn.Linear(d_model, d_ff),
        nn.ReLU(),
        nn.Linear(d_ff, d_model),
    )

    def forward(self, x):
        # 残差连接: x + Attention(Norm(x))
        # Pre-LN 风格: 先归一化再进入子层 (更稳定的训练)
        attn_out = self.attention(self.norm1(x))
        x = x + attn_out # 残差 ({ref}`res-net` 的思想)

        # FFN + 残差
        ffn_out = self.ffn(self.norm2(x))
        x = x + ffn_out
        return x

# 测试: 6 层编码器处理长度为 100 的序列
x = torch.randn(2, 100, 512) # batch=2, seq_len=100, d_model=512
encoder = nn.Sequential(*[TransformerEncoderBlock() for _ in range(6)])
output = encoder(x)
# 如果序列长度翻倍到 200, 计算量约增长 4 倍 → 验证  $O(n^2)$ 

```

本节小结

- Transformer 的四个升级: 自注意力 (双向)、多头 (多视角)、位置编码 (注入顺序信息)、残差+FFN (深层架构)
- 注意力和下篇: 真正的未登峰 中的 CNN 注意力是不同的概念 —— 一个在序列维度上交互, 一个在通道/空间维度上重加权
- Transformer 以 $O(n^2)$ 的代价换来了 $O(1)$ 的信息路径 —— 用计算量换长程依赖能力
- 这个代价推动了对更高效架构的探索 —— 我们的下一站

Transformer 成功了 —— 它几乎统治了 NLP、CV 乃至更多领域的序列建模。但 $O(n^2)$ 这个“原罪”始终存在。下一节 [Mamba 简述: 对 RNN 思想的回归](#) 中, 我们将看到一个有趣的思想回归: 能不能既拥有 RNN 的 $O(n)$ 效率, 又解决长程依赖问题?

8.6 Mamba 简述：对 RNN 思想的回归

Transformer：注意力的极致 告诉我们：Transformer 用 $O(n^2)$ 的代价换来了 $O(1)$ 的信息路径。这个权衡在大模型时代开始变得沉重——当序列长度达到百万级别（长视频、DNA 序列、整本教科书）， n^2 的增长速度让 Transformer 望而却步。

关键问题：有没有办法在保持长程依赖能力的同时，把复杂度降回 $O(n)$ ？

Mamba [GD23] 给出的回答是：**有。而且答案就在 RNN 的 $O(n)$ 骨架里——只需要给它装上“选择性”。**

8.6.1 历史背景

Transformer 的成功让注意力成为默认选择，但 $O(n^2)$ 的代价从未消失。研究者一直在寻找“线性注意力”——能够在 $O(n)$ 时间内实现类似效果的技术。2020 年代，Albert Gu 等人提出了一系列**结构化状态空间模型**（S4）[GGR22]，展示了 SSM 在长序列建模上的潜力。

2023 年，Albert Gu 和 Tri Dao 提出了 **Mamba** [GD23]，核心创新是**选择性机制**——让 SSM 的参数 B_t, C_t, Δ_t 成为输入的函数。结合硬件感知的并行实现，Mamba 在保持 $O(n)$ 效率的同时，首次在语言建模上达到了与 Transformer 相当的水平。

备注

历史意义：Mamba 代表了序列建模的一个新方向——不是推翻 Transformer，而是在效率维度上超越它。它证明了 RNN 的骨架（ $O(n)$ 、固定状态）只需“选择性”这剂药方，就能在现代硬件上重新焕发活力。

8.6.2 思想回归：RNN 骨架从未消失

RNN 的数学骨架

回顾 循环神经网络：让网络拥有“记忆”中的 RNN 公式：

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$$

这个公式有一个更一般的形式。考虑一个**连续**的微分方程——信号随时间平滑变化，而非一步步跳跃：

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{h}(t)$$

这就是 **状态空间模型**（State Space Model, SSM）的连续形式。其中：

- $\mathbf{h}(t) \in \mathbb{R}^d$ ：时刻 t 的隐状态（ d 是状态维度，类比 RNN 的 `hidden_size`）
- $\mathbf{x}(t) \in \mathbb{R}^m$ ：时刻 t 的输入（ m 是输入维度）
- $\mathbf{y}(t) \in \mathbb{R}^k$ ：时刻 t 的输出
- $\mathbf{A} \in \mathbb{R}^{d \times d}$ ：状态转移矩阵——控制“旧记忆如何随时间衰减”
- $\mathbf{B} \in \mathbb{R}^{d \times m}$ ：输入投影矩阵——控制“新信息如何进入记忆”

· $C \in \mathbb{R}^{k \times d}$: 输出投影矩阵 —— 控制"记忆如何转化为输出"

备注

和 RNN 的关系: 如果去掉 $\tanh$, RNN 的 $\mathbf{h}_t = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t$ 就是 SSM 的离散版本 —— 其中 $\mathbf{A} \leftrightarrow \mathbf{W}_h$, $\mathbf{B} \leftrightarrow \mathbf{W}_x$ 。RNN 多了一个 $\tanh$ 来防止数值发散, 而 SSM 用结构化的 $\mathbf{A}$ 矩阵来实现稳定。

从连续到离散: 离散化

连续形式的 SSM 不能直接用于深度学习 (我们需要处理的是离散的词序列, 而非连续信号)。需要将其**离散化** —— 把微分方程转换为递推公式。

引入一个步长 Δ (step size), 表示连续时间中两次采样之间的间隔。离散化后:

$$\mathbf{A}_d = e^{\Delta \mathbf{A}} \approx \mathbf{I} + \Delta \mathbf{A}, \quad \mathbf{B}_d = (\Delta \mathbf{A})^{-1} (e^{\Delta \mathbf{A}} - \mathbf{I}) \cdot \Delta \mathbf{B} \approx \Delta \mathbf{B}$$

备注

不需要记住这些矩阵指数! 关键是理解离散化的直觉: 步长 Δ 越小, 两次采样之间信号变化越小, $\mathbf{A}_d \approx \mathbf{I}$ (几乎不衰减), $\mathbf{B}_d \approx \Delta \mathbf{B}$ (新信息按步长比例进入)。

用近似后的离散参数, SSM 的递推公式变为:

$$\mathbf{h}_t = \mathbf{A}_d \mathbf{h}_{t-1} + \mathbf{B}_d \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C} \mathbf{h}_t$$

这就是 SSM 的**离散形式** —— 和 RNN 的骨架几乎一模一样, 只是 $\mathbf{A}_d, \mathbf{B}_d$ 由 Δ 和连续参数 $\mathbf{A}, \mathbf{B}$ 计算而来。

备注

它和 RNN 有什么区别? RNN 中 $\mathbf{W}_h$ 是直接学习的任意矩阵, 训练时可能不稳定。SSM 中的 $\mathbf{A}$ 被约束为特殊结构 (如对角矩阵), 且通过 Δ 缩放后, 数值上更可控 —— 这为后面"选择性让这一切按输入动态变化"埋下了伏笔。

过去十年, SSM 之所以没有取代 Transformer, 是因为一个致命缺陷: $\mathbf{A}, \mathbf{B}, \mathbf{C}, \Delta$ **对所有输入都是一样的** —— 无论当前词是"但是" (暗示转折) 还是"的" (结构助词), 网络用同一套规则更新状态。这在需要**内容感知的推理** (Content-Based Reasoning) 时捉襟见肘。

Mamba 的洞察: 让参数依赖输入

Mamba 的核心创新是**选择性 SSM (Selective SSM)**: 让 $\mathbf{B}, \mathbf{C}$ 和步长 Δ 不再是固定的全局参数, 而是**每个时间步**根据当前输入 $\mathbf{x}_t$ 动态计算:

$$\mathbf{B}_t = \text{Linear}_B(\mathbf{x}_t), \quad \mathbf{C}_t = \text{Linear}_C(\mathbf{x}_t), \quad \Delta_t = \text{softplus}(\text{Linear}_\Delta(\mathbf{x}_t))$$

其中：

- $\text{Linear}_B, \text{Linear}_C, \text{Linear}_\Delta$ 都是简单的**线性映射**——矩阵乘法加偏置 ($Wx + b$)，各有自己独立的可学习权重。 Linear_B 和 Linear_C 将 d_{model} 维输入投影到 d_{state} 维 (d_{state} 是隐状态维度)， Linear_Δ 投影为一个标量。
- $\text{softplus}(z) = \log(1 + e^z)$ 是一个平滑版的 ReLU (对照**激活函数**)：当 z 很大时接近 z ，当 z 很小时接近 0。用于确保步长 $\Delta_t > 0$ ——状态更新不能“倒退”。

将这些输入依赖的参数代入 RNN 的数学骨架中的离散化递推，得到选择性的完整更新公式：

$$\mathbf{A}_{d,t} = e^{\Delta_t \mathbf{A}} \approx \mathbf{I} + \Delta_t \mathbf{A}$$

$$\mathbf{B}_{d,t} \approx \Delta_t \mathbf{B}_t$$

$$\mathbf{h}_t = \mathbf{A}_{d,t} \mathbf{h}_{t-1} + \mathbf{B}_{d,t} \mathbf{x}_t$$

$$\mathbf{y}_t = \mathbf{C}_t \mathbf{h}_t$$

备注

和传统 SSM 的区别一目了然：传统 SSM 中 $\Delta, \mathbf{B}, \mathbf{C}$ 是固定的训练参数，对所有时间步都一样。选择性 SSM 中 $\Delta_t, \mathbf{B}_t, \mathbf{C}_t$ 每一步都不同——由 $\mathbf{x}_t$ 决定。**选择性 = 参数随时间步变化，变化规则由输入内容驱动。**

在实际实现中（以及我们的代码中），做进一步的简化：假设 $\mathbf{A}$ 是对角矩阵（每个状态维度独立衰减），略去矩阵指数，直接用标量形式：

$$\mathbf{h}_t = \mathbf{A} \odot \mathbf{h}_{t-1} + \Delta_t \cdot (\mathbf{B}_t \odot \mathbf{x}_t), \quad \mathbf{y}_t = \mathbf{C}_t^\top \mathbf{h}_t$$

其中 $\odot$ 是逐元素乘法（角色一：**遗忘门**——“档案管理员”中 $\mathbf{f}_t \odot \mathbf{c}_{t-1}$ 的同款操作）。 $\mathbf{A} \in \mathbb{R}^{d_{\text{state}}}$ 是衰减率向量（每个维度独立衰减）， $\mathbf{B}_t, \mathbf{C}_t \in \mathbb{R}^{d_{\text{state}}}$ 是选择性输入/输出门。注意这里 $\mathbf{B}_t, \mathbf{C}_t$ 是向量（而非矩阵）——这是对角 SSM 的简化，在教学中足够传达核心思想。

这个简化版本把公式变得极其清晰：**状态更新就是衰减率 $\mathbf{A}$ × 旧记忆 + 步长 Δ_t × 选择性写入的 $\mathbf{B}_t \odot \mathbf{x}_t$ 。**

为什么“选择性”是关键？

传统 SSM 失败的原因可以用一个例子说明。考虑处理这句话：

“今天天气很好，所以我决定去公园散步。”

当网络读到“所以”时，它需要知道“所以”是一个因果连接词，暗示前面的“天气很好”是后面“散步”的原因。但传统 SSM 对所有词都使用**同一套** $\mathbf{A}, \mathbf{B}, \mathbf{C}$ ——它无法区分“所以”（需要触发因果推理）和“的”（结构助词，可以忽略）。

选择性机制让网络能够**根据当前输入的内容，动态调整状态更新的策略。**

逐步拆解三个选择性参数

参数一： Δ_t （步长）——“当前输入有多重要？”

符号	含义	维度	直觉理解
Linear_Δ	一个可学习的线性映射	$\mathbb{R}^d \rightarrow \mathbb{R}$	学到"什么样的输入是重要的、需要大步伐更新"
softplus	$\log(1 + e^x)$ ，输出始终为正	标量	确保步长为正，且平滑

Δ_t 控制的是**状态更新的"步幅"**。回顾状态更新公式：

$$\mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \Delta_t \cdot (\mathbf{B}_t \mathbf{x}_t)$$

- Δ_t 大 $\rightarrow$ 网络认为当前输入很重要，大步更新状态（类似于 LSTM 中输入门大开）
- Δ_t 小 $\rightarrow$ 网络认为当前输入不重要，几乎不更新状态（类似于 LSTM 中输入门紧闭，遗忘门全开）

备注

直觉：读"的"时， Δ_t 可能很小——“保持当前状态，这个字不重要”。读"所以"时， Δ_t 可能很大——“注意！这里有关键转折，需要更新理解”。

这种机制让 Mamba 自动学会了对**关键信息"聚焦"、对冗余信息"忽略"**，而无需显式设计门控逻辑。

参数二： $\mathbf{B}_t$ （输入投影）——“新信息的哪些维度值得记？”

符号	含义	维度	直觉理解
Linear_B	一个可学习的线性映射	$\mathbb{R}^d \rightarrow \mathbb{R}^{d_{\text{state}}}$	学到"如何从当前输入中提取值得记录的信息"
$\mathbf{B}_t$	输入依赖的投影向量	$\mathbb{R}^{d_{\text{state}}}$	决定输入 $\mathbf{x}_t$ 的哪些方向应该进入隐状态

在传统 SSM 中， $\mathbf{B}$ 是一个固定矩阵——所有输入用同一种方式投影。选择性 SSM 中， $\mathbf{B}_t$ 随输入变化——"所以"和"的"产生不同的 $\mathbf{B}_t$ ，"所以"的投影可能强调"转折/因果"相关维度，而"的"的投影可能接近零向量（“没什么值得记录的”）。

参数三： $\mathbf{C}_t$ （输出投影）——“记忆中的哪些部分现在该说出来？”

符号	含义	维度	直觉理解
Linear_C	一个可学习的线性映射	$\mathbb{R}^d \rightarrow \mathbb{R}^{d_{\text{state}}}$	学到"基于当前输入，哪些记忆应该被输出"
$\mathbf{C}_t$	输入依赖的输出投影	$\mathbb{R}^{d_{\text{state}}}$	决定隐状态 $\mathbf{h}_t$ 的哪些维度暴露给输出 $\mathbf{y}_t$

回忆输出公式： $\mathbf{y}_t = \mathbf{C}_t \mathbf{h}_t$ 。 $\mathbf{C}_t$ 根据当前输入决定输出什么——类似于 LSTM 的输出门。当读到一个问号时， $\mathbf{C}_t$ 可能重点暴露关于"疑问"的维度；当读取动词时， $\mathbf{C}_t$ 可能暴露关于"动作"的维度。

三个参数的协同：一个具体例子

以"我昨天买的苹果很好吃"为例，追踪 Mamba 如何处理：

当前词	Δ_t (步长)	B_t (写入什么)	C_t (输出什么)	状态发生了什么
“我”	中	记录"主语=我"的方向	输出"我"的语义	状态初始化
“昨天”	小	几乎为零	几乎为零	时间状语，状态几乎不变
“买”	大	重点记录"动作=购买"	输出与"买"相关的语义	状态捕捉关键词
“的”	极小	接近零	接近零	虚词，状态保持不变
“苹果”	大	记录"宾语=苹果"	输出"苹果"的语义	状态补充宾语信息
“很”	中	记录"程度修饰"	输出程度信息	状态微调
“好吃”	大	记录"评价=正面"	重点输出评价信息	状态完成语义构建

备注

这个表展示了选择性的精髓：**每个词的 Δ_t, B_t, C_t 都不同**。虚词（“的”）获得极小 Δ ，几乎不影响状态；内容词（“买”、“苹果”、“好吃”）获得大 Δ ，深刻影响状态。网络通过大量训练学会了"什么词值得关注"——这是一个完全端到端学出来的能力。

直觉：选择性 = 动态门控

把 SSM 想象成一个"信息滤网"：

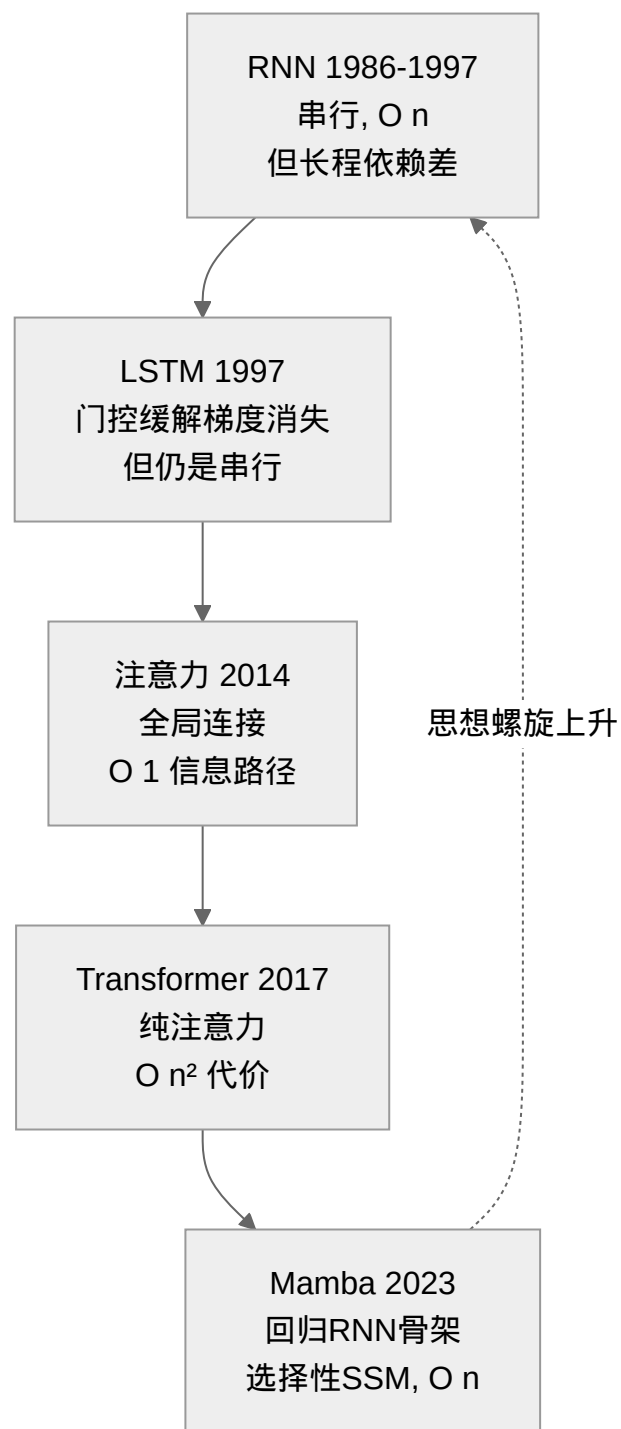
- **普通 SSM**：滤网的孔径是固定的 —— 不管流过来的是沙子还是石块，都用同样的网眼过滤。该记住的记不住，该忘的忘不掉。
- **选择性 SSM (Mamba)**：滤网根据当前流入的内容**动态调整**孔径。遇到重要信息（如段首的主题词），网眼缩小，精细保留；遇到不重要信息（如虚词），网眼放大，快速丢弃。

这种"基于输入内容决定保留或丢弃什么"的思想，本质上与 [LSTM：给记忆装上"门"](#) 的门控机制 [HS97] 一脉相承 —— 遗忘门根据当前上下文决定丢弃旧记忆中的哪些内容，选择性 SSM 则是让整个状态更新规则都依赖输入。区别在于：LSTM 的门控是在固定规则框架内的微调（门控值变了，但 W_f, W_i, W_o, W_c 本身是固定的），而选择性 SSM 让 B 和 C 本身就是输入的函数 —— **门控的程度更深、更彻底**。

8.6.3 Mamba vs RNN vs Transformer：三者的位置

表 3: 序列建模三阶段

对比维度	RNN / LSTM	Transformer	Mamba / 选择性 SSM
灵感来源	大脑的时序处理	全局信息检索	回归 RNN 骨架 + Transformer 的"选择性"精神
时间复杂度	$O(n)$ 串行	$O(n^2)$ 并行	$O(n)$ 可并行训练
长程依赖	弱（梯度消失）	强（ $O(1)$ 信息路径）	强（选择性保留关键信息）
信息传递	$h_t \rightarrow h_{t+1}$ 链式	所有位置全连接	$h_t \rightarrow h_{t+1}$ 链式 + 选择机制
推理速度	快（固定大小状态）	慢（需要整个 K/V 缓存）	快（固定大小状态， $5\times$ Transformer 吞吐量 [GD23]）

**备注**

Mamba 的历史意义：35 年后，我们回到了 RNN 的起点——一个通过固定大小隐状态（ $O(1)$ 内存）处理序列的架构——但这一次，隐状态的更新规则不再是固定的，而是根据输入动态调整。选择性机制解决了 RNN 的长程依赖问题，而线性骨架保留了 $O(n)$ 的效率。

这就像一个思想走了很远的路，最后在更高的层次上回到了原点——螺旋上升，而非简单重复。

8.6.4 代码实践：选择性 SSM 的核心逻辑

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimplifiedSelectiveSSM(nn.Module):
    """
    简化的选择性 SSM —— 展示 Mamba 的核心思想

    理论对应 {ref}`selective-ssm` 中"让参数依赖输入"的关键洞察
    不是完整 Mamba 实现，而是教学用的核心逻辑提取

    d_model: 输入特征维度 (如 256)
    d_state: 隐状态维度 (如 16) —— 相比 d_model 较小，体现了"压缩"的设计哲学
    """
    def __init__(self, d_model=256, d_state=16):
        super().__init__()
        self.d_model = d_model
        self.d_state = d_state

        # 状态转移矩阵 A —— 这是 SSM 中唯一一个不依赖输入的参数
        # A 控制"旧记忆如何随时间衰减"，通常初始化为接近 1 的值 (缓慢衰减)
        # 形状: (d_state,) — 对角形式，每个维度独立衰减
        self.A = nn.Parameter(torch.ones(d_state))

        # x_proj: 从输入 x 生成三个选择性参数 B_t, C_t, Delta_t
        # 输出维度 = d_state (B_t) + d_state (C_t) + 1 (Delta_t)
        # 这是选择性的核心 —— 所有参数都是 x 的函数 ({ref}`selective-ssm`)
        self.x_proj = nn.Linear(d_model, d_state * 2 + 1)

    def forward(self, x):
        """
        x: (batch, seq_len, d_model)
        返回: (batch, seq_len, 1)

        核心公式 ({ref}`state-space-model`):
        
$$h_t = A * h_{t-1} + B_t * x_t$$

        """
```

(续下页)

(接上页)

```

    y_t = C_t * h_t
    其中 B_t, C_t 是输入依赖的 (选择性), A 是固定的
    """

    B, T, D = x.shape

    # 步骤 1: 从输入计算三个选择性参数
    proj = self.x_proj(x) # (B, T, 2*d_state + 1)

    # B_t: 输入投影 —— "当前输入的哪些方向值得写入状态" ({ref}`selective-ssm` 参数二)
    # 形状: (B, T, d_state) —— 每个时间步、每个 batch 有独立的 B_t
    B_t = proj[..., :self.d_state]

    # C_t: 输出投影 —— "状态的哪些维度应该被读出" ({ref}`selective-ssm` 参数三)
    # 形状: (B, T, d_state) —— 每个时间步独立决定输出
    C_t = proj[..., self.d_state:2*self.d_state]

    # Delta_t: 步长 —— "当前输入有多重要" ({ref}`selective-ssm` 参数一)
    # softplus 确保步长为正 ( $\log(1+e^x) > 0$ )
    # 大 Delta → 大步更新状态 (关键信息); 小 Delta → 几乎不更新 (冗余信息)
    Delta = F.softplus(proj[..., -1:]) # (B, T, 1)

    # 步骤 2: 选择性扫描 (概念上串行, 实际 Mamba 用硬件感知并行算法)
    h = torch.zeros(B, self.d_state) # h_0: 初始状态
    outputs = []

    for t in range(T):
        # 选择性地将当前输入映射到状态空间
        # B_t[t] 决定了当前输入在状态空间的"写入方向"
        # x[..., :d_state] 取 d_state 维的输入子空间 (简化起见)
        input_contribution = B_t[:, t, :] * x[:, t, :self.d_state]

        # 状态更新:  $h_t = A * h_{t-1} + \Delta_t * (B_t * x_t)$ 
        #  $A * h_{t-1}$ : 旧记忆按 A 的衰减率保留
        #  $\Delta_t * \text{input}$ : 新信息按当前步长写入
        # 当  $\Delta_t \rightarrow 0$  时,  $h_t \approx A * h_{t-1}$  (状态几乎不变 —— "忽略当前输入")
        h = self.A * h + Delta[:, t, :] * input_contribution

        # 输出:  $y_t = C_t * h_t$  ( $C_t$  决定暴露哪些维度)
        y = (C_t[:, t, :] * h).sum(dim=-1, keepdim=True)
        outputs.append(y)

```

(续下页)

(接上页)

```
return torch.stack(outputs, dim=1) # (B, T, 1)
```

本节小结

- 状态空间模型 (SSM) 与 RNN 共享相同的数学骨架: $h_t = Ah_{t-1} + Bx_t$
- 传统 SSM 的 A, B, C 是固定的 —— 无法根据输入内容调整记忆策略
- Mamba 的关键创新: 让 B, C, Δ 依赖输入 x_t —— **选择性机制**
- 这个选择性让 SSM 拥有了"基于内容决定记住什么、忘记什么"的能力
- 结果: $O(n)$ 复杂度 + 与 Transformer 相当的长程依赖能力 —— 对 RNN 思想的螺旋式回归

为什么不继续深入?

Mamba 的完整实现涉及更深层的数学工具: **矩阵指数** $e^{\Delta A}$ 的精确计算需要线性代数谱分解, **结构化状态空间**的初始化依赖控制理论中的 HIPPO 矩阵, **硬件感知并行算法**涉及 GPU 内存层次和 kernel fusion 的底层优化。

这些工具已经远超本教程的目标 —— 面向高中生的入门介绍。本章的目标是让你理解序列建模的**思想演化路径**和 Mamba 的**核心直觉** (选择性 = 让参数依赖输入), 而非掌握其完整推导。如果你对细节感兴趣, 可以阅读原论文 [GD23] 及其引用的 S4 系列工作 [GGR22]。

Mamba 的思想 —— 用固定大小的隐状态高效处理长序列, 同时通过选择性机制解决长程依赖 —— 代表了序列建模的一个新方向。下一节 **总结与展望** 中, 我们将系统对比这三种架构, 并展望未来。

8.7 总结与展望

恭喜你完成了**序列建模**章节的全部内容!

从神经网络: 让网络拥有"记忆"的大脑启发, 到 LSTM: 给记忆装上"门"的门控突破, 再到从 RNN 到注意力: 一个思想的跳跃的关键跳跃, Transformer: 注意力的极致的极致化, Mamba 简述: 对 RNN 思想的回归的思想回归 —— 我们追溯了一条横跨 35 年的架构演化之路。

8.7.1 知识回顾

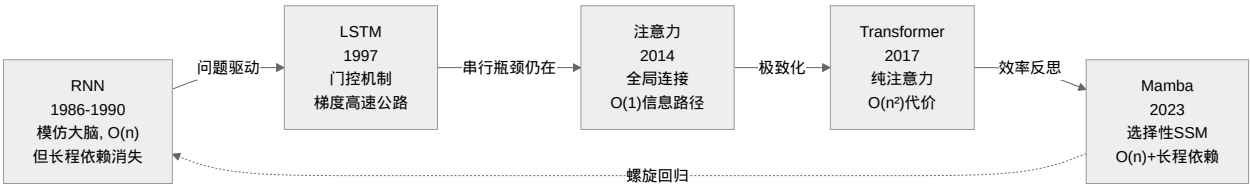
四种架构的系统对比

对比维度	RNN	LSTM	Transformer	Mamba
年份	1986-1990	1997	2017	2023
灵感	大脑时序处理	RNN + 记忆管理	信息检索	RNN 骨架 + 选择性
核心公式	$h_t = \tanh(W_h h_{t-1} + W_x x_t)$	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	$\text{softmax}(QK^T/\sqrt{d_k})V$	$h_t = Ah_{t-1} + B_t x_t$
状态数量	1 个	2 个 ($c_t + h_t$)	无固定状态 (K/V 缓存)	1 个 (但选择性更强)
信息路径	O(n) 串行	O(n) 串行	O(1) 直接	O(1) 通过状态
时间复杂度	O(n)	O(n)	O(n ²)	O(n)
长程依赖	弱 (梯度消失)	中 (门控缓解但非根治)	强	强 (选择性机制)
并行训练	不能	不能	能	能 (硬件感知算法)
推理速度	快	快	慢 (K/V 缓存)	快 (固定状态)
参数效率	高	中 (4× 门控权重)	低 (FFN 占大头)	高
归纳偏置	强: 时序因果	强 + 门控记忆	弱: 等距连接	中: 选择性时序

核心思想的演化

概念	来源	关键洞察
循环记忆	循环神经网络：让网络拥有"记忆"	通过隐状态传递让网络拥有对历史的压缩表示
BPTT	循环神经网络：让网络拥有"记忆"	反向传播在时间展开计算图上的应用，是梯度消失的根源
双 状 态 分离	LSTM：给记忆装上"门"	将"长期记忆" (c_t) 与"对外输出" (h_t) 分开管理
三重门控	LSTM：给记忆装上"门"	遗忘门（擦除）、输入门（写入）、输出门（读取）—— 精细管理记忆的进与出
梯度 高速 公路	LSTM：给记忆装上"门"	$\frac{\partial c_t}{\partial c_{t-1}} = f_t$ —— 细胞状态提供了一条不经过非线性的梯度通路
因 果 注 意 力	从 RNN 到注意力：一个思想的跳跃	连接所有前序状态的直觉 —— 从"传话"到"查档案"
Q/K/V 分离	从 RNN 到注意力：一个思想的跳跃	将"查询"和"被查询"的角色分离，增强表达力
多 头 注 意 力	Transformer：注意力的极致	多个视角并行，捕捉不同类型的关系
FFN 知识 存储	Transformer：注意力的极致	注意力负责跨位置交互，FFN 负责位置内部的特征变换和知识存储
位置编码	Transformer：注意力的极致	补偿注意力对位置信息的无知
选 择 性 SSM	Mamba 简述：对 RNN 思想的回归	让状态更新规则依赖输入 —— LSTM：给记忆装上"门" 门控思想的彻底化

叙事弧线：一个思想的螺旋上升



为什么要理解这一条演化路径？

- 1. 避免只见树木不见森林：单独的 RNN、Transformer、Mamba 都只是"点"，演化路径才是"线" —— 理解从哪里来，才能判断往哪里去
- 2. 培养架构判断力：当你遇到新架构时，能迅速定位它在这条演化线上处于什么位置 —— 是回归、是改进、还是全新的分支
- 3. 建立"问题 → 解决 → 新问题"的思维模式：每个架构都不是凭空设计的，而是对前一代架构缺陷的

回应

8.7.2 与前面章节的联系

前面章节	本章应用
梯度消失（Vanishing Gradient）	RNN 中梯度消失的数学根源（Jacobian 连乘）；LSTM 如何通过 c_t 旁路解决
归纳偏置	四种架构的归纳偏置对比 —— 从强（RNN/LSTM）到弱（Transformer）到中（Mamba）
ResNet：高原反应补给站	残差连接在 Transformer 中的延续；LSTM 细胞状态的梯度高速公路
下篇：真正的未登峰的注意力	CNN 的通道/空间注意力 vs Transformer 的自注意力 —— 两种不同的"注意力"
计算图	BPTT 在时间展开计算图上的反向传播
激活函数	tanh 在 RNN/LSTM 中作为门控激活；ReLU 在 Transformer FFN 中

8.7.3 实践建议

如何选择序列建模架构？

- **短序列（< 1K）+ 需要最快效果**：Transformer（生态成熟，预训练模型最多）
- **长序列（> 10K）+ 推理速度关键**：Mamba / SSM（ $O(n)$ 速度优势显著）
- **教学/理解序列建模**：从 RNN 开始 —— 最直观，问题也最明显
- **生产环境**：优先选择社区活跃、预训练权重丰富的架构（当前仍是 Transformer 主导）

8.7.4 未来方向

序列建模远未结束。以下是正在发展中的方向：

1. **混合架构**：Transformer 层 + SSM 层组合使用 —— 取 Transformer 的表达力和 SSM 的效率
2. **线性注意力**：通过核技巧将 $O(n^2)$ 的注意力近似为 $O(n)$ —— Mamba 之外的另一种降复杂度路径
3. **状态空间模型的扩展**：Mamba-2、Jamba 等后续工作进一步改进选择性机制和硬件效率
4. **多模态序列**：将序列建模从文本扩展到视频、音频、基因组 —— 这些都是天然的长序列

8.7.5 推荐资源

入门理解

- [The Illustrated Transformer](#)²⁶ (Jay Alammar 的可视化讲解, Transformer 入门首选)
- [The Annotated Transformer](#)²⁷ (Harvard NLP 的逐行注释实现)
- [Why are Transformers replacing CNNs?](#)²⁸ (油管视频, Transformer 与 CNN 的归纳偏置的不同造成的影响)

深入阅读

- [Elm90] — RNN 的奠基论文 (1990)
- [HS97] — LSTM 的原始论文 (1997)
- [BCB15] — 注意力机制的起源 (2014, ICLR 2015)
- [VSP+17] — Transformer 的诞生 (2017, NeurIPS)
- [GD23] — Mamba 的提出 (2023)

动手实践

- [Andrej Karpathy's makemore](#)²⁹ (从零实现 RNN、LSTM、Transformer)
- [Mamba 官方代码](#)³⁰ (选择性 SSM 的完整实现)

8.7.6 本章完

通过本章的学习,你不仅掌握了四种序列建模架构,更重要的是理解了它们之间的**演化逻辑**——为什么会有 RNN、为什么需要 LSTM、为什么注意力能革命性地解决问题、为什么 Transformer 又催生了 Mamba。

记住:

- **RNN 的困境**不是设计失败,而是串行信息传递的固有矛盾——这是理解后续所有架构的起点
- **注意力是"连接一切"的思想**——它解决了长程依赖,但代价是 $O(n^2)$
- **Mamba 是思想的螺旋回归**——在更高的层次上重新拥抱 RNN 的效率哲学,并用选择性机制弥补了它的缺陷

好的架构不是凭空发明的,而是**对前一代问题深刻理解后的自然回应**。这正是 **归纳偏置** 中讨论的核心思想——把先验知识内置到架构中,让结构本身替模型做出正确的假设。

²⁶ <https://jalammar.github.io/illustrated-transformer/>

²⁷ <http://nlp.seas.harvard.edu/annotated-transformer/>

²⁸ <https://www.youtube.com/watch?v=KnCRTP1lp5U>

²⁹ <https://github.com/karpathy/makemore>

³⁰ <https://github.com/state-spaces/mamba>

下一步：回到 [Deep Learning Club 学习教程](#) 选择其他进阶章节，或者探索 [架构心法](#)：从会用装备到会设计装备 学习通用的架构设计心法。

9.1 环境配置

9.1.1 Linux 基础：为什么炼丹用 Linux

你刚拿到一台深度学习服务器，装好了系统（或者商家预装了），打开终端后看见一个黑乎乎的界面——然后呢？

在开始配置驱动、安装 CUDA 之前，先回答一个根本问题：**为什么整个深度学习生态都建在 Linux 上？**

为什么是 Linux

表 1: 三大桌面系统的深度学习支持对比

维度	Linux (Debian/Ubuntu)	Windows	macOS
驱动支持	官方驱动，内核级集成	依赖系统更新，时有兼容问题	Apple Silicon MPS 表现不错
GPU 加速	NVIDIA CUDA 完整生态	CUDA 支持，Docker 透传麻烦	MPS 后端可用，性能可观
包管理	apt 一键装全家桶	手动下载安装包	brew，但生态不如 Linux
无头运行	不需要图形界面，SSH 就能管	需要远程桌面，带宽消耗大	可以但别扭
生产环境	95%以上服务器用 Linux	极少用于生产	几乎不用
适合场景	训练+部署全链路	入门学习	开发调试+轻量训练

Linux 在深度学习领域的统治地位不是偶然的。NVIDIA 的 CUDA 生态在 Linux 上最成熟，Docker 容器化部署在 Linux 上是原生体验，大多数深度学习框架的 CI/CD 只针对 Linux 和 macOS 做测试。简单说：**你用 Linux，**

遇到问题 Google 一下就有答案；你不用 Linux，遇到问题可能连 Google 都救不了你。

选择哪个发行版

深度学习领域最常见的两个选择是 **Debian** 和 **Ubuntu**。两者都是 Debian 系，用同一个包管理器（apt），命令完全兼容。

特性	Debian	Ubuntu
稳定性	极其稳定，包版本较旧	较新，但可能不稳定
CUDA 支持	需要手动添加 non-free 源	官方 APT 源直接支持
社区资源	深度学习相关内容较少	教程最多，遇到问题好搜
推荐场景	生产服务器	开发/学习用机器

推荐选择

如果你拿不准，选 **Ubuntu LTS**（如 22.04 或 24.04）。这是深度学习领域事实上的标准系统，NVIDIA 的文档和教程都以它为例。等你熟悉了 Linux 再换 Debian 不迟。

包管理基础

Debian 系用 apt 管理软件包。以下命令能覆盖 90%的日常需求：

更新软件源（换源后或首次使用必须执行）

```
sudo apt update
```

升级所有已安装的包

```
sudo apt upgrade
```

安装软件（以安装 Python 为例）

```
sudo apt install python3 python3-pip
```

卸载软件

```
sudo apt remove python3
```

搜索软件

```
apt search nvidia-driver
```

查看已安装的包

```
apt list --installed | grep nvidia
```


为什么要 sudo?

普通用户没有权限安装系统级软件。sudo 让你临时以 root 身份执行命令（**用户和权限**会详细讲用户和权限）。首次使用 sudo 时需要输入当前用户的密码 —— 注意：**终端输入密码时不会显示任何字符（不回显）**。这是正常行为，输完按回车就行。

一个常见的陷阱：**不要用 `sudo pip install`**。这会把 Python 包装到系统目录，和 apt 管理的包冲突。正确的做法是用虚拟环境隔离项目依赖。

uv：现代 Python 包管理

传统上 Python 用 pip 装包、venv 管环境、pip-tools 管依赖锁定 —— 三个工具各管一事。uv 是一个用 Rust 写的 Python 包管理器，把这套东西全包了，而且快了一个数量级：

```
# 安装 uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# 创建虚拟环境并激活
uv venv
source .venv/bin/activate

# 安装包（自动创建虚拟环境，无需手动 venv + activate）
uv pip install torch torchvision

# 从 requirements.txt 安装
uv pip install -r requirements.txt

# 锁定依赖版本（生成 uv.lock）
uv pip compile requirements.txt -o requirements.txt.lock

# 根据 lock 文件安装（保证团队环境一致）
uv pip sync requirements.txt.lock
```

表 2: pip + venv vs uv

操作	pip + venv	uv
创建环境	python3 -m venv .venv && source .venv/bin/activate	uv venv (自动激活)
安装依赖	pip install -r requirements.txt	uv pip install -r requirements.txt (快 10-100 倍)
依赖解析	无内置, 需 pip-tools	内置 uv pip compile
Python 版本管理	手动安装不同版本	uv python install 3.12
底层语言	Python	Rust (无 GIL, 全局解释器锁问题)

最重要的优势是**速度**和**确定性**: uv pip compile 会解析出完整的依赖树并锁定版本, 你和队友拿到同一个 lock 文件安装出来的环境完全一致, 不会出现"我机器上能跑你机器上报错"的情况。

uv 和 conda 的关系

uv 不管理非 Python 包 (如 CUDA 驱动、系统库)。如果你需要 conda 那种跨语言包管理 (Python + C++库 + 系统工具), 可以继续用 conda 或 mamba。但纯 Python 依赖管理, uv 是更好的选择。

系统管理基础

systemd：管理服务

深度学习服务器上, 很多组件 (SSH、Docker、监控) 都是作为系统服务运行的。systemd 是管理这些服务的标准工具:

```
# 查看服务状态
systemctl status sshd

# 启动/停止/重启服务
sudo systemctl start sshd
sudo systemctl stop sshd
sudo systemctl restart sshd

# 设置开机自启
sudo systemctl enable sshd
```

用户和权限

```
# 创建新用户
sudo adduser alice

# 给用户 sudo 权限
sudo usermod -aG sudo alice

# 切换用户
su - alice

# 修改文件/目录所有者
sudo chown alice:alice /data/datasets

# 修改权限 (r=4, w=2, x=1)
chmod 755 script.sh      # 所有者可读写执行, 其他人可读执行
chmod 600 id_rsa         # 私钥文件必须 600 权限, 否则 SSH 会报错
```

磁盘和存储

训练数据集动辄几十 GB, 了解磁盘使用情况是常备技能:

```
# 查看磁盘分区和使用情况
df -h

# 查看当前目录下各子目录的大小
du -sh *

# 查看当前目录总大小
du -sh .

# 挂载新硬盘
sudo mount /dev/sdb1 /mnt/data

# 开机自动挂载 (编辑 /etc/fstab)
# UUID=xxxx-xxxx /mnt/data ext4 defaults 0 2
```

进程管理

训练一个模型通常要跑几小时甚至几天，掌握进程管理能帮你避免"训练跑了一半终端关了"的惨案：

```
# 查看所有进程
ps aux

# 按 CPU 或内存排序
ps aux --sort=-%cpu
ps aux --sort=-%mem

# 实时监控（类似任务管理器）
htop

# 终止进程
kill -9 PID

# 让进程在退出终端后继续运行
nohup python train.py > train.log 2>&1 &
```

nohup 和 tmux 的选择

nohup 是最简单的后台运行方式，但管理多个进程不方便。生产环境建议用 tmux（tmux：终端"续命"神器会详细介绍），它让你能创建多个终端会话，随时断开和重新连接。

SSH：远程连接服务器

深度学习服务器很少在你手边——可能在机房，可能在云上。SSH 是你和服务器之间的桥梁。本节讲基础配置，把远程服务"拉"到本地来会深入到端口转发等高级用法。

密钥认证（推荐，不要用密码）

```
# 在本地机器上生成密钥对
# -t rsa: 算法类型
# -b 4096: 密钥长度
# -C "注释，通常是邮箱"
ssh-keygen -t rsa -b 4096 -C "your@email.com"

# 查看公钥内容（把这个给服务器管理员）
cat ~/.ssh/id_rsa.pub

# 将公钥复制到服务器（如果有密码权限）
```

(续下页)

(接上页)

```
ssh-copy-id user@your-server-ip
```

为什么要用密钥而不是密码？

密码认证有两个大问题：一是弱密码容易被暴力破解（服务器放在公网上半小时内就会收到扫描）；二是每次登录都要输入密码。密钥认证用一对公钥-私钥文件代替了密码输入，私钥存在本地，别人拿不到就不可能登录你的服务器。

SSH 配置文件

`~/.ssh/config` 可以让你不用每次都输入完整的连接信息：

```
# ~/.ssh/config
Host gpu-server
    HostName 123.123.123.123
    User anson
    Port 22
    IdentityFile ~/.ssh/id_rsa

Host lab-machine
    HostName 192.168.1.100
    User alice
    Port 2222

# 配置好后就可以用短名称连接了
# ssh gpu-server # 等价于 ssh anson@123.123.123.123 -p 22 -i ~/.ssh/id_rsa
```

安全加固

服务器放在公网上不需要太多配置，但以下几个措施值得做：

```
# 1. 禁止 root 直接登录
# 编辑 /etc/ssh/sshd_config
PermitRootLogin no

# 2. 禁止密码认证
PasswordAuthentication no

# 3. 修改默认端口（减少被扫描的概率）
Port 2222
```

(续下页)

(接上页)

```
# 修改后重启 SSH 服务
sudo systemctl restart sshd
```

别把自己锁在外面

修改 SSH 配置前，确保你已经用另一个终端窗口测试过新配置能连上。如果配错了把自己锁在外面，就只能去机房或者找运维帮忙了。测试方法：新开一个终端，用新的配置尝试连接，确认能登录后再关掉旧的窗口。

常用命令速查

命令	作用	示例
ls -lh	列出文件（人类可读大小）	ls -lh /data
cd	切换目录	cd /mnt/data/datasets
cp -r	递归复制目录	cp -r /data/raw /data/backup
mv	移动/重命名	mv old_name new_name
rm -r	删除目录（慎用！）	rm -r /tmp/cache
rm	删除文件（慎用！）	rm file_path
grep	搜索文本	grep "error" train.log
wc -l	统计行数	wc -l dataset.csv
tar -xzf	解压 tar.gz	tar -xzf imagenet.tar.gz
wget	下载文件	wget https://example.com/file.zip
curl	HTTP 请求	curl http://localhost:8080/health

rm -rf 的警告

rm -rf / 会删除你系统上的所有文件。不要在 **任何** 以 / 开头的路径上运行 rm -rf，除非你 100%确定自己在做什么。一个安全习惯：删除前先用 ls 确认路径。

掌握了这些基础操作，你就可以开始配置深度学习环境了。下一节 **NVIDIA 驱动与 CUDA：让 GPU 真正工作** 会带你安装 NVIDIA 驱动和 CUDA，让你的 GPU 真正派上用场。

9.1.2 NVIDIA 驱动与 CUDA：让 GPU 真正工作

系统装好了，SSH 能连上了，然后呢？python -c "import torch; print(torch.cuda.is_available())" 返回 False —— 你的 GPU 还没“醒”过来。

让 NVIDIA GPU 在 Linux 上正常工作需要三层软件：**驱动** → **CUDA Toolkit** → **深度学习框架**。每一层都有版本兼容问题，配错了就报错。

文档更新：2026 年 4 月。NVIDIA 驱动版本迭代很快，本文信息以当时最新的 R595/R580 分支为准。
建议在 [NVIDIA 官方驱动下载页](#)³¹ 确认最新版本。

显卡架构简史

每张 NVIDIA 显卡都基于一个架构代号，不同架构支持的驱动版本、计算特性、深度学习能力都有差异。了解架构是选卡和排错的第一步。

表 3: NVIDIA GPU 架构一览（2014 – 2026）

架构	发 布 年份	代表产品	Tensor Core	RT Core	驱动支持状态
Maxwell	2014	GTX 750/900 系列	无	无	R580 最后支持，2025 终止
Pascal	2016	GTX 10 系列、Tesla P100	无	无	R580 最后支持，2025 终止
Volta	2017	Tesla V100、Titan V	第 1 代（FP16）	无	R580 最后支持，2025 终止
Turing	2018	RTX 20 系列、GTX 16 系列、T4	第 2 代	第 1 代	R590+
Ampere	2020	RTX 30 系列、A100、A40	第 3 代（TF32+BF16+稀疏）	第 2 代	R590+
Ada Lovelace	2022	RTX 40 系列、L40S	第 4 代	第 3 代	R590+
Hopper	2022	H100、H200	第 4 代（FP8+Trans-former）	无	R590+
Blackwell	2024	RTX 50 系列、B200	第 5 代（FP4）	第 4 代（神经渲染）	当前

各代架构的关键技术跳跃：

- **Pascal → Volta：**Tensor Core 诞生，深度学习训练速度提升 10 倍
- **Volta → Turing：**RT Core 加入，Tensor Core 首次进入消费级显卡
- **Turing → Ampere：**TF32 精度、BF16 支持、稀疏加速，算力翻倍

³¹ <https://www.nvidia.com/en-us/drivers/>

- Ampere → Hopper：Transformer Engine、FP8，专为大语言模型优化
- Hopper → Blackwell：FP4 推理、所有核心支持 FP32+INT32 并发、神经渲染

驱动版本与架构支持

NVIDIA 的驱动按 Release Branch 组织，每季度更新。不同分支支持的 GPU 架构不同：

表 4: 驱动分支与架构支持（2026 年 4 月）

驱动分支	最新版本	发布日期	支持的架构	状态
R595	596.36	2026-04-28	Turing ~ Blackwell	当前 Game Ready
R580	582.53	2026-04-28	Maxwell ~ Blackwell	当前 Enterprise LTSB
R570	573.96	2026-01	Maxwell ~ Blackwell	2026 年 2 月 EOL
R535	539.72	2026-04	Maxwell ~ Blackwell	维护模式
R470	475.14	2024-07	Kepler ~ Ampere	Legacy（最后支持 Kepler 的分支，最高支持 Ampere）

关键事件时间线：

- 2021 年：R470 成为 Kepler 架构的最后支持分支（GTX 600/700 系列）
- 2025 年 10 月：R580 发布，最后一次 Game Ready 驱动更新支持 Maxwell、Pascal、Volta
- 2025 年 Q4 起：上述架构转为季度安全更新（至 2028 年 10 月）
- 2026 年 2 月：R570 分支正式 EOL
- 2026 年 4 月：R595（游戏）/ R580（企业）为当前活跃分支

如何选择驱动分支

- 游戏/开发机：选 R595 Game Ready，有最新功能优化
- 生产服务器：选 R580 Enterprise LTSB，稳定优先，支持到 2028 年
- 旧卡用户：如果持有 GTX 900/10 系列或 Titan V，最多只能用到 R580 系列的最后版本

一键安装：三行命令搞定

Debian/Ubuntu 上装 NVIDIA 驱动最可靠的方式是通过官方 APT 源：

```
# 1. 检测你的显卡型号和推荐驱动
nvidia-detect # 输出类似 "nvidia-driver-570"

# 2. 添加 NVIDIA 官方 APT 源 (Ubuntu LTS 用户)
sudo add-apt-repository ppa:graphics-drivers/ppa
sudo apt update

# 3. 安装推荐驱动 (将 570 替换为 detect 输出的版本)
sudo apt install nvidia-driver-570

# 4. 重启
sudo reboot
```

重启后验证：

```
# 基本验证
nvidia-smi

# 输出应该是类似这样的：
# +-----+
# | NVIDIA-SMI 570.86.15      Driver Version: 570.86.15      CUDA Version: 12.8      |
# |-----+-----+-----+-----+-----+-----+
# | GPU   Name           Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
# | Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
# |=====+=====+=====+=====+=====+=====+
# |    0   Tesla T4               Off | 00000000:00:1E:0 Off |                    0 |
# | N/A   48C    P0      28W /  70W |      0MiB / 15360MiB |      0%      Default |
# |-----+-----+-----+-----+-----+-----+

# 桌面环境额外验证：KMS 是否启用
# 输出 Y 表示正常 (官方 APT 源安装的驱动通常已自动配置)
cat /sys/module/nvidia_drm/parameters/modeset

# 驱动 545+ 还可检查 fbdev (替代 simplifiedrm 的 framebuffer 支持)
cat /sys/module/nvidia_drm/parameters/fbdev
```

Secure Boot 警告

如果系统启用了 Secure Boot (很多预装 Ubuntu/Debian 的机器默认开启), 安装 NVIDIA 专有驱动后需要注册 MOK (Machine Owner Key): 重启时会进入蓝色 MOK 管理界面, 选择 "Enroll MOK" → "Continue" → 输入密码 → 重启。如果不做这一步, 驱动不会加载, `nvidia-smi` 会报错。

桌面环境安装提示

如果你是在带桌面环境的机器上安装(比如 Ubuntu Desktop), 安装过程中 nouveau 开源驱动可能与 NVIDIA 专有驱动冲突, 导致重启后黑屏。建议在安装前临时修改 GRUB 启动参数:

1. 重启时按 Shift 进入 GRUB 菜单, 按 e 编辑启动项
2. 在 linux 那一行末尾添加 `nomodeset`
3. 按 Ctrl+X 或 F10 启动
4. 安装完成并重启后, 如果一切正常, 可以去掉 `nomodeset`

混合显卡 (核显+独显) PRIME offload 踩坑

如果你的机器同时有 Intel/AMD 核显和 NVIDIA 独显, 并且想让 NVIDIA 卡通过 `prime-run` 处理浏览器、游戏等图形任务, **必须确保核显的 Xorg 配置启用了 DRI3**。

踩坑现象:

- `prime-run firefox` 或 `prime-run glxinfo` 仍然使用核显 (Mesa/Intel) 渲染
- `__NV_PRIME_RENDER_OFFLOAD=1` 环境变量不生效
- 浏览器 `about:support` 或 `chrome://gpu` 显示使用 iGPU 而非 NVIDIA
- `vkcube` 无法用 iGPU 渲染, 只能走 NVIDIA 路径, 但高分辨率下出现类似老式 CRT 逐行扫描的割裂画面

原因: PRIME render offload 依赖 DRI3, 而 Intel 驱动的配置可能未启用 DRI3。

解决步骤:

1. 确认核显的 PCI 地址:

```
lspci | grep -i vga
```

输出类似 `00:02.0 VGA compatible controller: Intel Corporation ...`, 记下前面的地址 (如 `00:02.0`), 稍后在 Xorg 配置中写为 `PCI:0:2:0` (冒号分隔, 去掉前导零)。

2. 创建 Xorg 配置文件:

新建文件 `/etc/X11/xorg.conf.d/20-intel-dri3.conf`, 内容如下:

```
Section "Device"
    Identifier "Intel"
    Driver "modesetting"
    BusID "PCI:0:2:0"
    Option "DRI" "3"
EndSection
```

将 BusID 中的 PCI:0:2:0 替换为你在第 1 步中确认的实际核显地址。

- 3. 重启系统，让 Xorg 加载新配置。
- 4. 验证 offload 是否生效：

```
# 应该显示 NVIDIA 渲染器
__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia glxinfo | grep "OpenGL_
↪render"

# 应该列出两个 provider (含 NVIDIA-G0)
xrandr --listproviders
```

驱动装不上？常见原因

症状	原因	解决
nvidia-smi: command not found	驱动没装	先 nvidia-detect 确认，然后 sudo apt install nvidia-driver-XXX
Failed to initialize NVML: Driver/library version mismatch	驱动更新后内核模块没重载	重启, 或sudo rmmod nvidia_uvm nvidia_drm nvidia_modeset nvidia && sudo modprobe nvidia
ERROR: You appear to be running an X server	有图形界面在运行	sudo service lightdm stop 后重试安装
装完重启黑屏	驱动与显卡不匹配 或 Secure Boot 阻止	进恢复模式卸载驱动，换版本重试，或禁用 Secure Boot
内核更新后 nvidia-smi 失效	内核升级后 DKMS 模块未自动编译	sudo dkms autoinstall 然后重启
桌面环境启动黑屏 / Wayland 无法进入	NVIDIA DRM KMS 未启用	检查 cat /sys/module/nvidia_drm/parameters/modeset, 若为 N, 在 GRUB 内核参数中添加 nvidia-drm.modeset=1; 驱动 545+ 还可尝试补充 nvidia_drm.fbdev=1

nvidia-smi 深入解读

nvidia-smi 是你的 GPU 仪表盘。不只是看一眼温度和显存 —— 它告诉你的信息远比表面多：

```
# 基本（一秒钟刷新一次）
watch -n 1 nvidia-smi

# 只看关键指标（适合监控训练）
nvidia-smi --query-gpu=index,name,temperature.gpu,utilization.gpu,memory.used,memory.total --
↪format=csv

# 输出示例：
# index, name, temperature.gpu, utilization.gpu, memory.used, memory.total
# 0, Tesla T4, 48, 0 %, 0 MiB, 15360 MiB
```

表 5: nvidia-smi 输出关键字段

字段	含义	关注点
Temp	GPU 核心温度 (°C)	超过 85°C 说明散热不足，会降频
Perf	性能状态 (P0-P12)	P0=最高性能，P8+表示降频了
Pwr:Usage/Cap	当前功耗/最大功耗	远低于 Cap 可能没在满负荷跑
Memory-Usage	显存使用量	接近上限说明模型太大，需降 batch 或换卡
GPU-Util	计算单元利用率	持续 < 80% 说明 CPU/IO 瓶颈
Volatile GPU-Util	实际 SM 占用率（更准）	和 GPU-Util 一起看
Compute M.	计算模式	Default / Exclusive_Process / PROHIBITED

GPU-Util 低不代表有问题

深度学习训练中 GPU-Util 低可能有多种原因：数据加载太慢（IO 瓶颈）、CPU 预处理跟不上、batch size 过小、或者模型结构本身计算密度低（如小 CNN）。可以尝试用 `nvidia-smi dmon` 查看更细粒度的指标。

ECC 与显存类型

企业级 GPU（如 A100、H100、V100）使用 HBM/HBM2/HBM3 显存，**默认开启 ECC**。ECC 纠错码会占用约 6-12% 的显存容量，但保证了计算正确性。可以用以下命令查询：

```
nvidia-smi -q -d ECC
nvidia-smi --query-gpu=ecc.mode.current --format=csv
```

消费级 GPU（RTX 系列）使用 GDDR 显存（GDDR6/GDDR6X/GDDR7），**没有 ECC**。对大多数深度学习训练来说，GDDR 的可靠性足够，但长时间大规模训练（数周级别的 HPC 任务）中，HBM+ECC 是标配。

表 6: 显存类型对比

类型	显卡示例	带宽	ECC	适用场景
GDDR6	RTX 3060~4060	192~480 GB/s	无	训练/推理入门
GDDR6X	RTX 3090/4090	700~1000 GB/s	无	消费级高性能
GDDR7	RTX 5090	1792 GB/s	无	消费级旗舰
HBM2	V100/P100	732~900 GB/s	有 (6-12%开销)	遗留企业级
HBM2e	A100	1935~2039 GB/s	有	上一代数据中心
HBM3	H100	3350 GB/s	有	当前数据中心
HBM3e	H200/B200	4800~8000 GB/s	有	最新数据中心

CUDA 版本管理

CUDA Toolkit 版本 $\neq$ nvidia-smi 显示的 CUDA Version

nvidia-smi 输出的"CUDA Version"是驱动支持的 **CUDA 最大版本** (Driver API 版本), 不是实际安装的 CUDA Toolkit 版本。你可以装 CUDA 11.8 Toolkit 在 CUDA 12.8 的驱动上跑 (向下兼容), 但不能反过来。

快速确认你的深度学习框架在用哪个 CUDA 版本:

```
# PyTorch
python -c "import torch; print(torch.version.cuda)"

# TensorFlow
python -c "import tensorflow as tf; print(tf.sysconfig.get_build_info()['cuda_version'])"

# 查看系统 CUDA Toolkit 版本
/usr/local/cuda/bin/nvcc --version
```

表 7: NVIDIA 驱动与 CUDA Toolkit 的兼容关系

驱动分支	最大 CUDA 版本	最低 CUDA 版本	建议配对
R595/R590	CUDA 13.x	CUDA 12.0	最新项目用 CUDA 12.8+
R580	CUDA 12.8	CUDA 12.0	生产环境配 CUDA 12.4~12.8
R570	CUDA 12.8	CUDA 11.8	遗留项目
R535	CUDA 12.2	CUDA 11.0	旧卡兼容

建议的 CUDA 安装方式 (不用 NVIDIA 官网的 runfile, 而是用包管理器):

```
# 先确认驱动版本够用
nvidia-smi # 看顶部 CUDA Version

# 用 pip 安装 cuda-toolkit (推荐, 不污染系统)
pip install nvidia-cuda-toolkit

# 或直接用框架自带的 CUDA (PyTorch 自带)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
```

什么时候需要手动装 CUDA Toolkit?

大多数情况下你不需要手动安装 CUDA Toolkit。PyTorch/TensorFlow 的预编译包已经带了它们需要的 CUDA 运行时库。只有在你需要自己编译 CUDA 扩展（如 flash-attn）或者使用 NVIDIA 的底层库（如 cuBLAS、cuDNN）的开发 API 时，才需要装完整的 CUDA Toolkit。

PyTorch 的显卡支持淘汰节奏

驱动和 CUDA Toolkit 的版本只是第一层兼容。PyTorch 有自己的**编译时支持的架构列表**——即使你的驱动和 CUDA Toolkit 够新，如果 PyTorch 二进制没编译你家显卡的架构代码，照样跑不了。

验证方法：

```
import torch
print(torch.cuda.get_arch_list())
# 输出类似: ['sm_70', 'sm_75', 'sm_80', 'sm_86', 'sm_90', 'sm_100', 'sm_120']
# sm_70=Volta, sm_75=Turing, sm_80=Ampere, sm_90=Hopper, sm_100=Ada, sm_120=Blackwell
```

PyTorch 的显卡支持随版本演进不断抬升最低门槛：

表 8: PyTorch 版本与 CUDA 架构支持

PyTorch 版本	CUDA 构建	支持的最小架构	不支持的显卡
2.6.0	12.4	Maxwell (sm_50)	无
2.7~2.10	12.6	Maxwell (sm_50)	无
2.8~2.10	12.8	Volta (sm_70)	GTX 900/10 系列
2.11+	12.8	Turing (sm_75)	V100、Titan V、GTX 900/10 系列
2.12+	13.0	Turing (sm_75)	同上

如果你的显卡被 PyTorch 最新版抛弃了怎么办？

你有三条路可选：

1. **用旧版 PyTorch**：V100、GTX 1080 Ti 等 Volta/Pascal 显卡。值得考虑的版本是：
`pip install torch==2.6.0 torchvision==0.21.0 torchaudio==2.6.0 --index-url https://download.pytorch.org/whl/cu124`
2. **自行编译**：从源码编译 PyTorch 时指定 `TORCH_CUDA_ARCH_LIST="7.0"` 来包含你的架构。
3. **换卡**：一张二手 RTX 3060 12GB（~1000~1500 RMB）支持 `sm_86`，未来几年都不会被抛弃。

简单规则：**什么时候该考虑升级显卡？**当 PyTorch 最新版不再编译你的架构（`get_arch_list()` 里找不到你的 `sm` 版本），且你想用的新模型依赖新版 PyTorch 的新特性时。

当前（2026 年 4 月）PyTorch 最新版使用的 CUDA 12.8 构建已移除 Volta（`sm_70`，即 V100/Titan V）支持。V100 曾经是 PyTorch 实践：把理论变成代码中最常用的 GPU 之一，现在也进入了“遗留硬件”行列。

Compute Capability 速查

PyTorch 用 `sm` 版本号（如 `sm_75`）标识显卡架构。想知道你的卡对应哪个 `sm`：

```
python -c "import torch; print(torch.cuda.get_device_capability())"
# 输出示例：(8, 6) 表示 sm_86
```

sm 版本	架构	代表显卡
sm_50	Maxwell	GTX 750 Ti、GTX 960、GTX 980
sm_52	Maxwell	GTX 980 Ti、Titan X (Maxwell)
sm_60	Pascal	GTX 1080、GTX 1070、GTX 1060
sm_61	Pascal	GTX 1080 Ti、Titan Xp
sm_70	Volta	Titan V、Tesla V100
sm_75	Turing	RTX 2080 Ti、RTX 2080、RTX 2070、GTX 1660、T4
sm_80	Ampere	A100
sm_86	Ampere	RTX 3090、RTX 3080、RTX 3070、RTX 3060、A10、A40
sm_89	Ada	RTX 4090、RTX 4080、RTX 4070、RTX 4060、L40S
sm_90	Hopper	H100、H200
sm_120	Blackwell	RTX 5090、RTX 5080、B200

sm 版本有什么用？

- 确认你的卡是否被当前 PyTorch 支持：`torch.cuda.get_arch_list()` 里有没有你的 `sm`
- 自行编译 CUDA 扩展时，`TORCH_CUDA_ARCH_LIST` 需要填对

- 买卡前查 sm 版本就能预判这张卡未来几年的 PyTorch 支持窗口

Docker GPU 支持：一劳永逸的环境隔离

驱动和 CUDA Toolkit 的版本兼容问题是深度学习环境配置中最容易出错的环节。**Docker + nvidia-container-toolkit** 可以一步解决这个问题：宿主机只需要装驱动，PyTorch、CUDA Toolkit、cuDNN 这些都放在容器里，互不干扰。

安装配置：

```
# 1. 确保宿主机驱动已装好 (nvidia-smi 能正常输出)
nvidia-smi

# 2. 安装 nvidia-container-toolkit
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o /usr/
share/keyrings/nvidia-container-toolkit-keyring.gpg
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/$distribution/libnvidia-
container.list | \
    sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] \
    https://#g' | \
    sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt update && sudo apt install -y nvidia-container-toolkit

# 3. 配置 Docker
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# 4. 测试
docker run --rm --gpus all nvidia/cuda:12.6.0-base-ubuntu22.04 nvidia-smi
```

之后跑任何深度学习代码只需要拉一个带 CUDA 的 PyTorch 镜像：

```
# 使用 PyTorch 官方镜像 (自带 CUDA 12.4 + cuDNN)
docker run --gpus all -it --rm \
    -v $(pwd):/workspace \
    pytorch/pytorch:2.6.0-cuda12.4-cudnn9-devel \
    python train.py

# 或使用 NVIDIA 的 PyTorch 容器 (已预装所有优化)
docker run --gpus all -it --rm \
    -v $(pwd):/workspace \
```

(续下页)

(接上页)

```
nvcv.io/nvidia/pytorch:24.12-py3 \
python train.py
```

表 9: 裸机 vs Docker 的环境管理对比

维度	裸机直接装	Docker 容器
环境隔离	冲突时难以清理	每个项目独立容器
版本切换	需要卸载重装 CUDA Toolkit	换一个镜像标签即可
团队协作	每个人装的不一样	共享 Dockerfile 保证一致
复现	依赖系统包的版本	镜像锁定所有依赖
磁盘占用	一套环境	每个镜像几 GB

什么时候不用 Docker?

- 单用户、单项目的开发机，Docker 的隔离优势不明显
- 多个 Docker 镜像同时挂载大训练集可能浪费磁盘空间
- 需要访问宿主机特定硬件（USB 设备、HCA 网卡）时配置复杂

权衡：Docker 最适合多人共用服务器或生产环境部署。个人开发机装裸机更直接，但要小心不要污染系统 Python 环境（Linux 基础：为什么炼丹用 Linux 的 uv 小节已经讲了解决方案）。

GPU 选型指南

产品线定位

表 10: NVIDIA 产品线

产品线	定位	显存	互连	典型价格	适合
GTX	入门消费	4~12 GB GDDR	无	<¥2,000	学习、小模型
RTX x050/60	主流消费	8~16 GB GDDR	无	¥1,800~¥4,000	微调、推理
RTX x070/80	高端消费	12~32 GB GDDR7	无	¥4,000~¥14,000	训练中型模型
RTX Pro	工作站	24~48 GB GDDR6/7	无	¥22,000~¥72,000	专业可视化+AI
Tesla/Data Center	数据中心	40~192 GB HBM	NVLink	¥72,000~¥216,000+	大规模训练/生产

消费级 vs 企业级：核心差异

消费级显卡（RTX）和企业级（A/H/B 系列）在深度学习场景下的差异可能出乎你的意料：

- 1. **显存瓶颈**：RTX 5090 有 32 GB GDDR7，而 P100（2016）有 16 GB HBM2。十年过去，消费级旗舰的显存才翻了 2 倍，而模型大小翻了 1000 倍。**显存通常是第一瓶颈。**
- 2. **单精度算力倒挂**：RTX 5050（2025 最低端）的 FP32 算力（约 11 TFLOPS）和十年前的 P100（10.6 TFLOPS）相当，但 5050 有 5 代 Tensor Core 而 P100 一个都没有——用 FP16/INT8 推理时 5050 甩 P100 几条街。
- 3. **FP64 基本没变**：消费级显卡的 FP64 算力始终被锁定在 FP32 的 1/64~1/32。如果你需要做高精度科学计算，必须上企业级（A100 的 FP64 是 RTX 5090 的 12 倍）。
- 4. **多卡互联**：消费级没有 NVLink，多卡通信只能走 PCIe。数据中心卡通过 NVLink 可以达到 600~1800 GB/s 的卡间带宽，这是训练大模型的关键。
- 5. **显存带宽差距**：这是最容易被忽视的差异。HBM 显存（企业级标配）的带宽远超同代 GDDR，直接影响训练吞吐：

表 11: 显存带宽对比

显卡	显存类型	显存容量	带宽	总线位宽
Tesla P100	HBM2	16 GB	732 GB/s	4096-bit
Tesla V100	HBM2	16/32 GB	900 GB/s	4096-bit
Tesla A100	HBM2e	40/80 GB	2039 GB/s	5120-bit
RTX 3090	GDDR6X	24 GB	936 GB/s	384-bit
RTX 4090	GDDR6X	24 GB	1008 GB/s	384-bit
RTX 5090	GDDR7	32 GB	1792 GB/s	512-bit
H100	HBM3	80 GB	3350 GB/s	5120-bit

注意一个关键对比：**P100（2016）的 732 GB/s HBM2 带宽比 RTX 4060（2024）的 272 GB/s GDDR6 快近 3 倍。**企业级显卡的 HBM 显存通过超宽总线（4096-bit 起）实现高带宽，消费级 GDDR 则靠高频率和窄总线（128~512-bit）。这就是为什么二手 P100/V100 即使算力落后，在训练吞吐上依然能和一些入门消费卡抗衡——带宽够大。不过也别忘了 [PyTorch 的显卡支持淘汰节奏](#)中提到的框架兼容性问题——算力再高跑不了新版 PyTorch 也不行。

怎么选：两个核心场景

买卡之前先看清楚自己属于哪种情况：

场景一：手里没有 GPU，想低成本入门

淘汰的数据中心卡是性价比极高的选择。它们退役后被大批量抛售到二手市场，算力和显存在今天依然能打：

显卡	显存	二手参考价 (闲鱼)	Tensor Core	FP32	入手价值
Tesla P100 16GB	16 GB HBM2	300~600 RMB	无	10.6 TFLOPS	极致低价入门, 适合跑传统 CNN
Tesla V100 16GB	16 GB HBM2	~1000 RMB	第 1 代 (FP16)	15.7 TFLOPS	Tensor Core 入门, 跑小 Transformer
Tesla V100 32GB	32 GB HBM2	~3000 RMB	第 1 代 (FP16)	15.7 TFLOPS	大显存低成本方案

买淘汰企业级显卡的注意事项

- 这些卡没有主动散热风扇 (被动散热), 需要服务器风道或自己加装风扇
- 通常需要专用电源线 (EPS 8-pin, 不是 PCIE 8-pin)
- P100/V100 基于 Pascal/Volta 架构, **PyTorch 新版 CUDA 12.8 构建已放弃支持** —— 需要用 PyTorch 2.10 的 CUDA 12.6 构建, 或自行编译
- V100 的 Tensor Core 只支持 FP16, 不支持 TF32/BF16/INT8 等后续精度
- 驱动方面 R580 是最后支持分支, 可用至 2028 年安全更新

场景二: 手上有卡想升级, 或者预算充足

RTX 20 系及以上都值得考虑。深度学习场景下, **显存大小比算力重要得多** —— 一张老卡只要显存够大, 跑大模型的能力就比新卡更强:

表 12: 消费级升级路径（先看显存，再看架构）

推荐优先级	显卡	显存	架构	二手参考价	适合做什么
☆☆☆☆☆	RTX 3090	24 GB GDDR6X	GB Ampere	~3000~5000 RMB	性价比之王，24GB 能微调 7B 模型
☆☆☆☆☆	RTX 4090	24 GB GDDR6X	GB Ada	~12000~15000 RMB	消费级天花板，显存+算力都够
☆☆☆☆	RTX 5090	32 GB GDDR7	Blackwell	~14000+ RMB	最大显存消费卡
☆☆☆☆	RTX 3080 20GB	20 GB GDDR6X	GB Ampere	~2000~3000 RMB	20GB 显存，实惠之选
☆☆☆	RTX 2080 Ti 22GB	22 GB GDDR6	Turing	~1500~2500 RMB	改版 22GB 显存，魔改卡有风险
☆☆☆	RTX 4060 Ti 16GB	16 GB GDDR6	Ada	~2500~3000 RMB	新卡省心，16GB 够跑多数模型
☆☆	RTX 3060 12GB	12 GB GDDR6	Ampere	~1000~1500 RMB	入门首选，12GB 够跑小型 LLM

显存决定你能跑什么

这是最常被忽略的选卡标准。一个粗略参考：

- 6~8 GB：可以跑 BERT、ResNet、小 CNN，LLM 最多跑 1~3B 参数（量化后）
- 12~16 GB：可以跑 7B 模型（4-bit 量化 QLoRA 微调）
- 24 GB：可以跑 7B 模型全参数微调，或 13B 模型 QLoRA
- 32~48 GB：可以跑 13B 全参数微调，或 70B 模型 QLoRA
- 80 GB+：可以跑 70B 模型全参数微调

显存不够用 ≠ 完全不能做。梯度累积、混合精度、LoRA/QLoRA、模型分片（model parallelism）都是小显存跑大模型的手段。但门槛是存在的——一张 3090 无论如何跑不了 70B 的全参数微调。

如果你在做购买决策，记住一个简单的公式：

- 二手企业级（P100/V100）≈ 极致性价比入门方案，有显存有算力但缺生态
- 二手 RTX 3090 ≈ 个人用户的甜点卡，24GB 显存+Ampere 架构覆盖 90%场景
- 新的 RTX 40/50 系 ≈ 省心省电，享受最新特性，代价是贵
- 数据中心卡（A100/H100）≈ 团队/生产环境的选择，个人用户租云实例更划算

一张显卡的“黄金年代”

从驱动支持和框架兼容性两个角度看，架构的生命周期大致分三个阶段：

- **淘汰期** (Maxwell/Pascal/Volta)：R580 是最后支持的驱动分支，新版 PyTorch 也不再编译这些架构。还能用，但新特性与新卡无缘
- **成熟期** (Turing/Ampere)：驱动、框架、容器生态支持最完整，是当前最稳妥的选择
- **主力期** (Ada/Hopper/Blackwell)：享受所有新特性，但部分框架适配可能滞后，价格也最高

驱动装好了，CUDA 配好了，接下来你需要知道怎么远程访问这台服务器。下一节[远程访问：从任何地方连上你的 GPU 服务器](#)。

9.1.3 远程访问：从任何地方连上你的 GPU 服务器

GPU 服务器很少在你手边——可能在机房，可能在云上，可能在朋友家阁楼里吃灰。你需要的是在**任何地方**都能连上去跑训练、看 loss 曲线、查日志的能力。

这一节不讲理论，只讲实操。从最简单的 SSH 隧道到全家桶式的组网方案，按需选读就行。

SSH：不止是登上去敲命令

Linux 基础：为什么炼丹用 Linux 教你密钥配置和基本连接。但深度学习场景下，SSH 真正的大招是**隧道转发**。

把远程服务“拉”到本地来

SSH：远程连接服务器已经帮你配好了密钥和基本连接，但 SSH 的真正实力是**端口转发**。最常见的场景是：服务器上跑着 Jupyter Notebook，你想在本地浏览器打开它。SSH 隧道就是干这个的：

```
# 把服务器的 8888 端口映射到本地的 8888 端口
ssh -L 8888:localhost:8888 user@your-server

# 然后打开 http://localhost:8888 —— 实际上访问的是服务器上的 Jupyter
```

三种隧道各有各的用处：

类型	参数	什么时候用
本地转发	-L 本地端口: 目标: 目标端口	Jupyter、TensorBoard、Gradio —— 远程服务拉到本地看
远程转发	-R 远程端口: 本地: 本地端口	让公网服务器帮你转发流量到内网机器
动态转发	-D 本地端口	整台浏览器的流量全走服务器网络

把这些固定配置写进 ~/.ssh/config，以后就不用每次敲长长一串了：

```
# ~/.ssh/config
Host gpu-server
    HostName your-server-ip
    User anson
    # 自动映射 Jupyter 和 TensorBoard 端口
    LocalForward 8888 localhost:8888
    LocalForward 6006 localhost:6006
    # 保持心跳，防止长时间训练后断开
    ServerAliveInterval 60
    ServerAliveCountMax 3
```

一个要命的细节

通过 SSH 隧道访问 Jupyter 时，Jupyter 必须只监听 localhost (`--ip=127.0.0.1`)，**绝对不能**监听 `0.0.0.0`。否则你的 Notebook 就直接暴露在公网上了，任何人都能访问。这个原则同样适用于 TensorBoard、Gradio、Streamlit —— 只要是通过 SSH 隧道访问的服务，绑定到 `127.0.0.1` 就对了。

VSCode Remote SSH

如果你用 VSCode 写代码，Remote SSH 插件比 Jupyter 的体验好一个数量级 —— 本地写代码，远程执行，文件自动同步。装了插件之后 `Ctrl+Shift+P` → Remote-SSH: Connect to Host 选服务器就行。它自动读 `~/.ssh/config` 里的配置，设好的隧道和别名都能用。

效果就是：代码在 GPU 服务器上跑，但编辑体验和在本地完全一样。断网了重连一下，编辑器状态全在。

过跳板机 (ProxyJump)

很多实验室和云环境的 GPU 服务器躲在私有子网里，不能直接访问。你得先登上一台**跳板机**（堡垒机），再通过它跳到 GPU 服务器：

```
# 原始方案：两步走
ssh user@jump-server
# 在跳板机上再执行
ssh user@gpu-server

# 优雅方案：ProxyJump 一步到位
ssh -J user@jump-server user@gpu-server

# 如果跳板机端口不是默认的 22
ssh -J user@jump-server:2222 user@gpu-server
```

写进 `~/.ssh/config` 就更清爽了：

```
Host jump-server
    HostName jump.company.com
    User anson

Host gpu-server
    HostName 10.0.1.100
    User anson
    ProxyJump jump-server
    # 还可以同时配端口转发，穿透两层
    LocalForward 8888 localhost:8888
```

配置好后一句 `ssh gpu-server` 就自动经过跳板机直连 GPU 服务器，隧道也一并建立。如果不方便装 OpenSSH 7.3 以上版本（ProxyJump 需要），也可以用老的 ProxyCommand 模式：

```
Host gpu-server
    HostName 10.0.1.100
    User anson
    ProxyCommand ssh jump-server -W %h:%p
```

mosh：容忍你切 WiFi、掉线、高延迟

SSH 有一个天生的毛病：**网络一不稳定就断**。你在咖啡厅连服务器看 loss 曲线，站起来换了个位置，WiFi 切了一下——SSH 卡死了，只能重连。

mosh（Mobile Shell）就是来解决这个问题的。它在 SSH 的基础上加了一层 UDP 会话，能容忍 IP 地址变化、网络延迟波动、甚至电脑合盖再打开：

```
# 安装 mosh（服务端和客户端都需要）
sudo apt install mosh

# 连接（和 ssh 用法基本一样）
mosh user@your-server

# 如果 SSH 端口不是 22
mosh --ssh="ssh -p 2222" user@your-server
```

连接上之后试试：断开 WiFi 换成手机热点——mosh 不会断，敲几个回车就恢复了。对于“在高铁上看 loss 曲线”这种场景来说，mosh 比 tmux 续命更直接——tmux 保证**会话**不丢，mosh 保证**连接**不丢，两者搭配最佳。

mosh 的限制

mosh 不支持 SSH 隧道转发（-L/-R/-D 参数）。所以 mosh 适合日常查状态、敲命令、看日志。如果你需要

端口转发 (Jupyter、TensorBoard)，还是得用 SSH 隧道。实际工作流通常是：mosh 上去启动训练，SSH 隧道开 Jupyter 看结果。

tailscale：零配置组网，像在同一局域网

tailscale 是一个建立在 WireGuard 之上的组网工具。它的核心价值一句话就能说清楚：**让分布在世界各地的机器像在同一局域网里一样通信。**

什么时候你需要它

场景	没有 tailscale	有 tailscale
服务器没有公网 IP	连都连不上	tailscale IP 直连
想从手机上看训练进度	折腾端口转发一整天	手机装个 app 就行
几个同学共用一台服务器	每个人都要配一遍	每人一个 tailscale 账号自动搞定
不想把服务暴露到公网	得写 iptables 防扫描	默认就在加密网络里

真 · 三步搞定

```
# 第 1 步：所有设备都装
curl -fsSL https://tailscale.com/install.sh | sh

# 第 2 步：启动并登录（会弹出浏览器让你认证）
sudo tailscale up

# 第 3 步：查看分配到的 IP
tailscale status
# 100.x.x.x      server      username@  linux
# 100.x.x.y      laptop     username@  macOS
```

之后就简单了：ssh anson@100.x.x.x 代替 ssh anson@公网 IP。tailscale 自动处理 NAT 穿透、加密传输和身份认证。你不需要在服务器上开放任何端口，tailscale 会自己想办法打洞连上你。

免费套餐够用吗？

个人用户完全够。最多 100 台设备，支持子网路由和 ACL 访问控制。如果你只是连自己的一台 GPU 服务器加一台笔记本，免费套餐绰绰有余。

进阶：让你的服务器当“网桥”

如果 GPU 服务器在一个局域网里（比如学校实验室），旁边还有 NAS、另一台服务器、共享存储 —— 你想通过 tailscale 访问整个内网：

```
# 在服务器上通告整个 192.168.1.0/24 子网
sudo tailscale up --advertise-routes=192.168.1.0/24

# 然后在 tailscale 管理后台 (https://login.tailscale.com) 开启子网路由
# 之后不管你在哪里，都可以通过 192.168.1.x 访问实验室的任意设备
```

frp：没有公网 IP？找个“中转站”

tailscale 虽然好，但有些场景它搞不定：网络环境把 UDP 封了、你需要**对外暴露 HTTP 服务**（比如给别人用模型 API）、或者就是不能用 tailscale。这时候 frp 上场。

frp 需要一个**有公网 IP 的机器**当中转。你的请求先到中转服务器，它再转发给你的 GPU 服务器：



中转端配置（公网服务器）

```
# frps.toml —— 就这么几行
bindPort = 7000
```

```
# 启动
./frps -c frps.toml
```

GPU 服务器配置（没公网 IP 的那台）

```
# frpc.toml
serverAddr = "你的公网服务器 IP"
serverPort = 7000

# 把 SSH 暴露到公网的 6000 端口
[[proxies]]
name = "ssh"
type = "tcp"
localIP = "127.0.0.1"
localPort = 22
```

(续下页)

(接上页)

```
remotePort = 6000

# 把 Jupyter 暴露到公网的 8888 端口
[[proxies]]
name = "jupyter"
type = "tcp"
localIP = "127.0.0.1"
localPort = 8888
remotePort = 8888
```

```
# 启动
./frpc -c frpc.toml
```

之后 ssh anson@公网服务器 IP -p 6000 就连上了你的 GPU 服务器, Jupyter 在浏览器打开 公网服务器 IP:8888 就能用。

表 13: tailscale vs frp 一句话版

对比项	tailscale	frp
要不要公网服务器	不要, 机器之间直连	必须有一台
流量怎么走	P2P 直连, 不绕路	全经过中转服务器
对外暴露服务	不太擅长	天生适合
配置麻烦程度	装完就跑	得写配置文件

推荐策略

- 自己用 → tailscale。免费、简单、安全, 不用管端口和防火墙
- 给别人用 (API 服务、演示) → frp。配合 Nginx 反向代理还能加域名和 HTTPS
- 网络限制太严 → frp 的 WebSocket 隧道可以尝试绕过一些防火墙

iptables: 给你的服务器上个锁

结合 tailscale: 零配置组网, 像在同一局域网的"默认就在加密网络里", 加上 iptables 的精细控制, 你的服务器基本安全就有了保障。

前面说的都是"怎么连进来"的问题。还有一个反面问题: 怎么不让不该连的人连进来。深度学习服务器放在公网上, 半小时内就会收到各种扫描和爆破尝试。

iptables 是 Linux 最底层的防火墙, 你不需要成为专家, 但掌握几条规则能让服务器安全很多:

先看看当前有什么规则

```
sudo iptables -L -n -v
```

只允许你的 IP 连 SSH (其他人都滚蛋)

```
sudo iptables -A INPUT -p tcp --dport 22 -s 你的 IP 地址 -j ACCEPT
```

允许你实验室的整个网段访问模型 API

```
sudo iptables -A INPUT -p tcp --dport 8080 -s 192.168.1.0/24 -j ACCEPT
```

除此以外所有访问 8080 的请求都拒绝

```
sudo iptables -A INPUT -p tcp --dport 8080 -j DROP
```

但已经连上的连接不要断 (不然你正在跑的 SSH 会被自己踢掉)

```
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

默认策略: 进门全关, 出门全开

```
sudo iptables -P INPUT DROP
```

```
sudo iptables -P FORWARD DROP
```

```
sudo iptables -P OUTPUT ACCEPT
```

如果你想把同一个端口换个映射 (比如外网 8080 对应内网 Jupyter 的 8888):

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT --to-port 8888
```

保存规则让重启后还在:

```
sudo apt install iptables-persistent
```

```
sudo netfilter-persistent save
```

别把自己锁在外面

配置防火墙时, 千万不要关掉当前 SSH 连接的窗口。开一个新终端测试新规则 —— 确认能连上再关旧的。如果不小心把自己锁了, 只能指望服务器有 IPMI/iDRAC 或者你认识机房管理员。

远程连接的问题解决了。但还有一件事: 训练跑到一半终端关了怎么办? 服务怎么开机自启? 最后一节服务管理: 让训练一直跑, 服务一直开来收尾。

9.1.4 服务管理：让训练一直跑，服务一直开

驱动装好了，远程能连上了——但这只是开始。真正的考验是：你关掉笔记本电脑回家的路上，训练还在跑吗？

这一节解决两个核心问题：一是让任务在你离线后继续运行（tmux），二是让服务在服务器重启后自动恢复（systemd）。顺便聊聊面板和监控。

tmux：终端“续命”神器

这是深度学习最实用的工具，没有之一。SSH：远程连接服务器和 mosh：容忍你切 WiFi、掉线、高延迟帮你连上服务器，但 tmux 让你的训练在你断开后继续跑。

没有 tmux 的时候，你的工作流是这样的：

```
python train.py # 训练开始，预计跑 6 小时
# ... 你去吃午饭了 ...
# 回来发现：终端超时断开了，训练白跑了 6 小时
```

有了 tmux：

```
tmux new -s training # 创建名为 training 的会话
python train.py      # 在 tmux 里启动训练
# Ctrl+B 然后按 D 断开 (detach)，会话还在跑
# 几小时后重新连上：
tmux attach -t training # 回到训练现场，一切正常
```

日常操作就记四组快捷键

tmux 的学习曲线平缓——日常只用记四组快捷键就够了：

操作	快捷键	说明
断开会话	Ctrl+B → D	断开但不终止，任务后台继续跑
新建窗口	Ctrl+B → C	在同一个会话里开新终端
切换窗口	Ctrl+B → 窗口数字	在多个任务间切换
上下分屏	Ctrl+B → "	一个窗口拆成上下两半
左右分屏	Ctrl+B → %	一个窗口拆成左右两半
在分屏间跳转	Ctrl+B → 方向键	切换焦点到另一个分屏

tmux vs nohup vs screen

- **nohup**：最简单，nohup python train.py &，但管理多个进程不方便，不能重连到控制台

- **screen**: tmux 的前辈, 功能类似但默认不装、界面丑、分屏弱
- **tmux**: 现在是主流选择, 配置漂亮、分屏强大、会话管理灵活

三句话: **nohup** 解决"有没有"的问题, **tmux** 解决"好不好用"的问题。有条件就用 tmux。

真实世界的训练 workflow

结合 mosh (远程访问: 从任何地方连上你的 GPU 服务器) 和 tmux, 你的日常会变成这样:

```
# 1. 用 mosh 连服务器 (不怕切 WiFi 断连)
mosh user@server

# 2. 创建 tmux 会话
tmux new -s experiment-0429

# 3. 启动训练
python train.py --config configs/experiment.toml

# 4. Ctrl+B → D 断开, 关掉笔记本电脑走人

# 5. 晚上回家, 重新连上服务器
mosh user@server
tmux attach -t experiment-0429 # 训练还在跑, loss 曲线实时更新
```

一个 tmux 新手最容易踩的坑

不要在 tmux 里启动 Jupyter Notebook。因为 Jupyter 会打开浏览器, 而 tmux 没有浏览器——它会卡住。正确的做法是在普通 SSH 窗口里开 Jupyter (绑定 127.0.0.1), 然后通过 SSH 隧道访问。tmux 只用来跑训练脚本、预处理数据这些不需要界面的任务。

自定义 tmux 配置

如果你觉得默认的 tmux 界面太素, 可以加个简单的配置:

```
# ~/.tmux.conf
# 让 Ctrl+B 更顺手 (连续按两次更快)
set -g escape-time 0

# 开启鼠标支持 (可以用鼠标点窗口、拖分屏)
set -g mouse on
```

(续下页)

(接上页)

```
# 窗口列表显示在底部
set -g status-position bottom

# 每 60 秒自动刷新状态栏
set -g status-interval 60
```

改完配置后，在 tmux 里按 Ctrl+B → : → 输入 `source-file ~/.tmux.conf` 重载。

systemd：让服务开机自己跑

tmux 解决的是“我走了训练别停”的问题。还有一个更根本的问题：**服务器重启了，服务还能自己起来吗？**

系统管理员不可能每次重启都手动去启动服务。systemd 就是 Linux 的标准答案——它是一个“服务管家”，负责在开机时启动你指定的程序，并在程序崩溃后自动重启。

写一个 service 文件

假设你想让 Jupyter Notebook 开机自启，可以写一个 systemd service：

```
# /etc/systemd/system/jupyter.service
[Unit]
Description=Jupyter Notebook Server
After=network.target

[Service]
Type=simple
User=user
WorkingDirectory=/home/user/projects
ExecStart=/home/user/.local/bin/jupyter notebook --ip=127.0.0.1 --port=8888 --no-browser
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

各字段的含义：

字段	意思	为什么重要
After=network.target	等网络就绪再启动	有些服务依赖网络，不等就会报错
User=user	以哪个用户身份运行	不要用 root 跑服务，有安全风险
Restart=on-failure	失败了自动重试	程序崩溃了 systemd 会自动拉起来
RestartSec=5	重试前等 5 秒	防止频繁重启把系统搞崩
WantedBy=multi-user.target	在多用户模式下启动	一般服务都写这个

管理 service

重新加载配置文件（改了 service 文件后必须执行）

```
sudo systemctl daemon-reload
```

启用开机自启

```
sudo systemctl enable jupyter
```

立即启动

```
sudo systemctl start jupyter
```

查看状态

```
sudo systemctl status jupyter
```

查看日志

```
sudo journalctl -u jupyter -f
```

停止

```
sudo systemctl stop jupyter
```

不想开机自启了

```
sudo systemctl disable jupyter
```

一个排错技巧

systemctl status 显示的日志通常只保留最后几行。如果服务启动失败且原因不明显，用 journalctl -u 服务名 -f 查看完整日志。-f 会持续跟踪新日志，你可以同时重启服务观察报错。

训练脚本也适合用 systemd 吗？

不适合。systemd 适合**服务类程序**（Jupyter、API 服务、监控 Agent），不适合**一次性任务**（训练脚本）。训练脚本用 **tmux**：终端“续命”神器管理更灵活。

为什么？因为训练脚本可能需要你手动中断（调参、看 loss 曲线、改代码），而 systemd 设计的是“崩溃了就悄悄重试”——你想要的停止可能被 systemd 误解为崩溃，然后自动重启。

表 14: tmux vs systemd 的分工

场景	用什么	理由
跑训练脚本	tmux	需要手动交互、调试、中断
开 Jupyter/TensorBoard	systemd	开机自启，不用每次手动
部署模型 API	systemd	崩溃自动恢复，日志统一管理
数据预处理	tmux	一次性任务，跑完就结束

定时任务与日志管理

cron：按计划跑脚本

如果你需要“每天早上 8 点备份一次数据”或者“每周清理一次缓存”，cron 是最简单的方式：

```
# 编辑当前用户的 crontab
crontab -e

# 格式：分钟 小时 日 月 星期 命令
# 每天凌晨 3 点执行备份
0 3 * * * /home/user/scripts/backup.sh

# 每周日晚上 10 点清理日志
0 22 * * 0 find /home/user/logs -mtime +7 -delete

# 每半小时检查一次 GPU 温度（如果超过阈值就发通知）
*/30 * * * * /home/user/scripts/check_gpu_temp.sh
```

logrotate：日志别撑爆磁盘

训练日志如果不做管理，几个月就能把磁盘塞满。logrotate 是 Linux 自带的日志轮转工具：

```
# /etc/logrotate.d/training-logs
/home/user/logs/train.log {
    daily          # 每天轮转一次
    rotate 7       # 保留最近 7 天的日志
```

(续下页)

(接上页)

```

compress      # 压缩旧日志
missingok     # 日志文件不存在也不报错
notifempty    # 空日志不轮转
copytruncate  # 复制后截断原文件（不影响正在写的进程）
}

```

磁盘监控

```

# 快速查看磁盘使用情况
df -h

# 找出当前目录下最大的文件/目录
du -sh * | sort -rh | head -5

# 安装 ncdu，交互式的磁盘分析工具（比 du 好用）
sudo apt install ncdu
ncdu /home/user

```

管理面板：图形化监控

如果你不喜欢纯命令行操作，或者想让团队里的非技术成员也能查看服务器状态，可以装一个面板。

lPanel

lPanel³² 是一个开源的 Linux 服务器管理面板，安装简单：

```

curl -sSL https://resource.fit2cloud.com/lpanel/package/quick_start.sh -o quick_start.sh
sudo bash quick_start.sh

```

装完后通过浏览器访问 `http://服务器 IP: 端口`，可以：

- 查看 CPU、内存、磁盘、网络使用情况
- 查看 GPU 温度和显存占用
- 管理 Docker 容器
- 设置防火墙规则
- 查看系统日志

³² <https://github.com/lPanel-dev/lPanel>

GPU 专用监控

如果你只需要看 GPU 的状态，有几个轻量的选择：

```
# nvidia-smi: htop 风格的 GPU 监控
sudo apt install nvidia-smi
nvidia-smi

# 用 watch + nvidia-smi (最轻量，不需要额外装)
watch -n 2 nvidia-smi --query-gpu=index,name,temperature.gpu,utilization.gpu,memory.used,memory.
↳total --format=csv
```

监控建议

- 个人用：watch -n 2 nvidia-smi 就够了，零依赖
- 喜欢图形界面：nvidia-smi，和 htop 操作逻辑一样
- 团队用：1Panel 或 Prometheus + Grafana，可以历史回溯和告警
- 生产环境：配置 Prometheus exporter + Grafana 仪表盘，配合企业微信/钉钉告警

这一节是附录的最后一篇。从 [Linux 基础：为什么炼丹用 Linux](#) 到 [显卡架构简史](#)，从把远程服务“拉”到本地来到 [tmux：终端“续命”神器](#)——你现在已经拥有了一台“真正能用”的深度学习服务器。回到 [Deep Learning Club 学习教程](#) 选择你感兴趣的章节继续学习吧。

9.1.5 结语：现在你有一台真正能用的服务器了

这篇附录的目标很简单：让你拿到一台 GPU 服务器后，能在一小时内让它开始跑深度学习代码，而不是花一整天纠结驱动版本、SSH 配置和 Jupyter 连不上。

如果你是按顺序读下来的，你已经走过了这样一条路：



这条路径上每一步都可能卡住新手：

- **Linux 基础**卡在“不知道 sudo 是什么意思”、“rm -rf /删了系统”
- **NVIDIA 驱动**卡在“Secure Boot 阻止驱动加载”、“nvidia-smi 报 driver mismatch”
- **远程访问**卡在“服务器没公网 IP”、“Jupyter 不让我连”
- **服务管理**卡在“训练跑一半终端关了”、“服务器重启了怎么自动启动”

每篇内容都在解决这些具体的痛点，而不是抽象地介绍工具。

但这不是终点

这篇附录和主课程的关系很特别。主课程的知识是**线性的**——你从数学基础读到模型部署，一章接一章。但环境配置的知识是**放射状**的——你遇到什么问题，回来翻对应的那篇就行。

你不会“读完”这篇附录就一劳永逸。真正的使用方式是：

- 第一次配服务器时顺序读 [Linux 基础：为什么炼丹用 Linux 和 NVIDIA 驱动与 CUDA：让 GPU 真正工作](#)
- 遇到远程连接问题时翻[远程访问：从任何地方连上你的 GPU 服务器](#)，找到 tailscale 或 frp 的配置抄过去
- 部署服务时再读[服务管理：让训练一直跑，服务一直开](#)，用 systemd 把服务固定下来

记住一件事

环境配置的终极目标不是让你成为运维专家——而是让运维知识**不成为你做深度学习的障碍**。如果有一节内容你读完了觉得“暂时用不上”，那就对了，等你需要的时候它会在那里。

Deep Learning Club 学习教程 才是你的主战场。如果你是从这里开始的，现在可以回到[模型部署与服务：从训练到生产](#) 或者 [PyTorch 实践：把理论变成代码](#)选择你感兴趣的进阶方向了。训练脚本在等你，GPU 在转，loss 曲线在下降——这才是真正重要的事。

前面章节的所有内容都可以在笔记本或 Google Colab 上完成——有 PyTorch、有 Jupyter、有 GPU，就够了。但当你开始认真做深度学习项目，迟早会遇到一个绕不开的问题：**我需要一台自己的 GPU 服务器**。

这篇附录不讲神经网络，不讲训练技巧，只讲“搞一台 GPU 服务器并让它稳定跑起来”需要的那些非技术知识。它们不属于深度学习本身，但没有它们，你的深度学习代码就跑不起来。

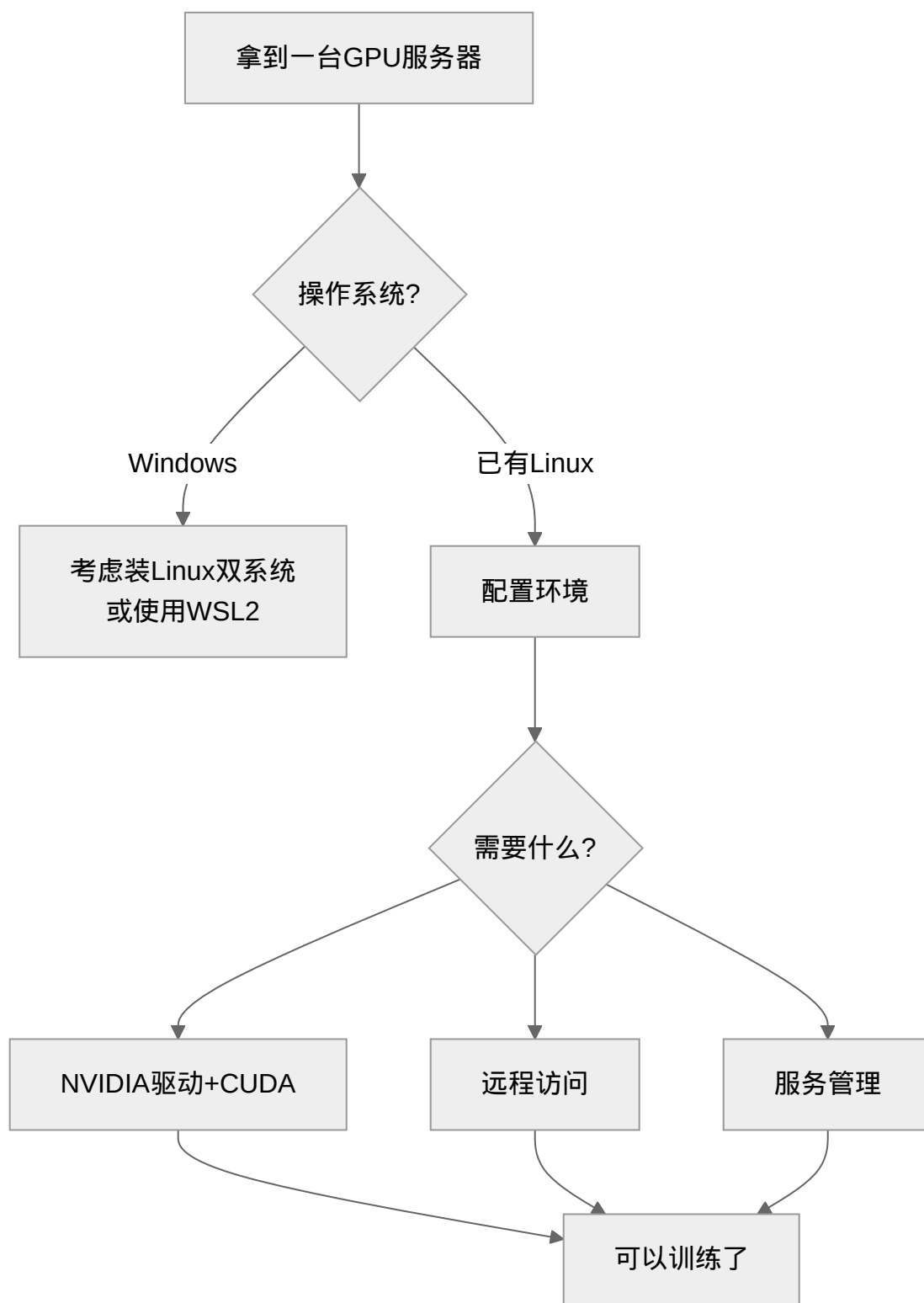
9.1.6 内容概览

章节	解决什么问题	适合谁
Linux 基础：为什么炼丹用 Linux	Linux 比 Windows 好在哪？装完系统该干什么？怎么远程连接？	第一次接触 Linux 服务器的读者
NVIDIA 驱动与 CUDA：让 GPU 真正工作	驱动装了没？为什么 nvidia-smi 报错？CUDA 版本怎么选？	有 GPU 但不知道怎么配置的读者
远程访问：从任何地方连上你的 GPU 服务器	服务器在机房怎么访问？没有公网 IP 怎么办？	需要远程管理服务器的读者
服务管理：让训练一直跑，服务一直开	训练跑到一半终端关了怎么办？服务怎么开机自启？	部署了服务需要维护的读者

阅读建议

这篇附录是**按需查阅**的参考材料，不是线性章节。遇到具体问题再回来翻对应部分即可。如果你是第一次接触 Linux 服务器，建议从 [Linux 基础：为什么炼丹用 Linux](#) 开始顺序阅读。

9.1.7 学习路径



9.1.8 前置知识

- 能区分"操作系统"和"软件"的区别
- 知道什么是终端/命令行

没有更多前置要求了——这篇附录就是从零开始的。

9.2 Sphinx Markdown 语法参考

本附录收录了在这本书中常见的 Sphinx Markdown 语法，面向需要撰写教程内容的贡献者，而非学习者。如果你只想学习深度学习，可以跳过。

备注

基本所有示例代码存放在 `examples/` 子目录中，通过 `literalinclude` 引用。这样做可以避免 MyST 指令在代码块内被误解析。

9.2.1 嵌套指令

Markdown 不支持在代码块内再嵌套代码块，Sphinx 用 `~~~`（波浪线）作为内层分隔符：

```
```{note}
  ~~~{note}
    ~~~{note}
      ~~~{important}
      Hello World!
    ~~~
  ~~~
~~~
```
```

换一种分隔符：````` → `~~~`。

9.2.2 图表

Mermaid（流程图）

```
```{mermaid}

caption: <标题文本, 可选>
align: center

```

(续下页)

(接上页)

```
graph LR
 A[方角] -->|连接文字| B(圆角)
 B --> C{判断}
 C -->|条件一| D[结果一]
 C -->|条件二| E[结果二]
 ...
```

### 小技巧

Mermaid 适合用于展示简单的图表，如流程图、序列图等（或者你不想调试 TikZ 代码）。

Mermaid 图表语法参考：<https://mermaid.js.org/>。

正确渲染需要 Chrome 系浏览器（如 Chrome/Chromium、Edge 等）和 mermaid-cli (mmdc) 工具，安装方法如下：

```
npm install -g @mermaid-js/mermaid-cli # Linux 可能需要 sudo
```

### TikZ（复杂图表）

```
```\tikz 图标题
\begin{tikzpicture}
  \draw (0,0) -- (1,1) -- (2,0) -- (0,0);
\end{tikzpicture}
```
```

### 小技巧

TikZ 适合用于展示复杂的图表，如网络架构等。

需要 TeXLive + pdf2svg 才能渲染。缺少依赖时文档仍可编译，只是图表不显示。

## 9.2.3 图片

```
```\figure /path/to/image.png
---
width: 400px
align: center
---
```

(续下页)

(接上页)

图片说明文字。

...

路径相对于 Markdown 文件所在位置。

小技巧

不推荐设定 width 为 100%，因为这会导致图片在不同设备上显示不一致。图片宽度建议设置为 400-600 px，根据需要调整。

9.2.4 代码

大段代码推荐放在单独文件中，用 `literalinclude` 引用：

```
``{literalinclude} /path/to/code.py
:language: python
:linenos:
:caption: 代码标题
``
```

这是本指南本身使用的技巧 —— 用 `literalinclude` 展示代码示例，避免 MyST 解析冲突。

`linenos` 启用行号。

9.2.5 数学公式

行内公式：

质能方程： $E = mc^2$

行间公式：

```
$$
E = mc^2
$$
```

或

```
``{math}
E = mc^2
``
```


9.2.6 彩色提示框

```
```{admonition} 标题（可选）
:class: tip

这是提示内容。
```
```

可用 class: attention、caution、danger、error、hint、important、note、tip、warning。

9.2.7 目录树

在 admonition 内嵌套 toctree（用 ~~~ 分隔）：

```
## 目录

```{toctree}
:maxdepth: 1

lesson2/index
lesson4/index
```
```

每行是 .md 文件路径（不含 .md 扩展名）。

9.2.8 Cross-Reference

{ref} — 引用章节/标签

定义标签（放在标题上方）：

```
(my-label)=
## 目标章节

内容...
```

引用标签：

```
详见 {ref}`my-label`。

或自定义链接文字：{ref}`这是链接文字 <my-label>`。
```

`{doc}` — 引用整个文档

```
下一节 {doc}`./folder/filename`。
```

不含 `.md` 扩展名。

`{cite}` — 引用文献

```
{cite}`citation-key`
```

最佳实践

1. 标签用描述性名称: `(gradient-vanishing)` 而非 `(section-5)`
2. 文件顶部定义标签: `(label)` 紧邻 `#` 标题 之上
3. 用 `{ref}` 引用章节/概念, 用 `{doc}` 引用整篇文档
4. 重命名文件后更新所有 cross-reference

9.2.9 文献管理 (BibTeX)

所有引用存放在 `source/references.bib` 中, 格式为 BibTeX。

添加一条引用: 在 `references.bib` 末尾追加, key 命名规则为 作者姓氏年份关键词:

```
@inproceedings{vaswani2017attention,  
  title={Attention is all you need},  
  author={Vaswani, Ashish and ...},  
  booktitle={Advances in Neural Information Processing Systems},  
  volume={30},  
  year={2017}  
}
```

在正文中引用:

```
{cite}`vaswani2017attention`
```

在文件末尾渲染参考文献列表 (只显示该文件实际引用的条目, 仅引用了 `references.bib` 中的条目时需要添加):

参考文献

```
```\bibliography  
:filter: docname in docnames
```\pre>
```

备注

`:filter: docname in docnames` 确保只列出本文档中通过 `{cite}` 实际引用过的条目，而非整个 `references.bib`。

9.2.10 表格

使用 `{list-table}` 指令，支持多级表头：

```
```${list-table} 表格标题
:header-rows: 1

* - 列一
 - 列二
 - 列三
* - 数据 1
 - 数据 2
 - 数据 3
```
```

第一行 (`* -`) 为表头，后续行为数据。仅在 Markdown 表格不满足需求时使用（如需要 directive 嵌入或更复杂的格式）。

9.2.11 主目录树

每个章节的 `index.md` 头部应有隐藏的 `{toctree}` 定义子文档顺序：

```
```${toctree}
:maxdepth: 2
:hidden:

introduction
cnn-basics
le-net
...
```
```

而在主 `source/index.md` 中定义全书的顶层目录树。条目为文件路径（不含 `.md` 扩展名）。

9.2.12 脚注

使用标准 Markdown 脚注语法：

一些文字 ^[^note-key]。

^[^note-key]：这是脚注内容。

脚注适合补充细节或提供扩展阅读链接，避免正文冗长。

9.2.13 条件渲染

条件渲染使得一部分内容仅在满足条件时渲染。

```
```${only} html
```

只在 HTML 输出中渲染的内容。

```
```
```

```
```${only} html or epub
```

只在 HTML 或 EPUB 输出中渲染的内容。

```
```
```

或者

```
```${only} not latex
```

只在非 PDF 输出中渲染的内容。

```
```
```

注意

为了兼容 Sphinx 的 LaTeX 输出，index 除了隐藏的 ToC，其他内容都必须通过此方法取消渲染。其余页面需要取消参考文献列表的渲染。

9.3 编译文档

本书使用 Sphinx + MyST Markdown 构建，输出 HTML 网页、PDF 和 EPUB 三个版本。本附录说明如何在本地搭建编译环境并生成文档。

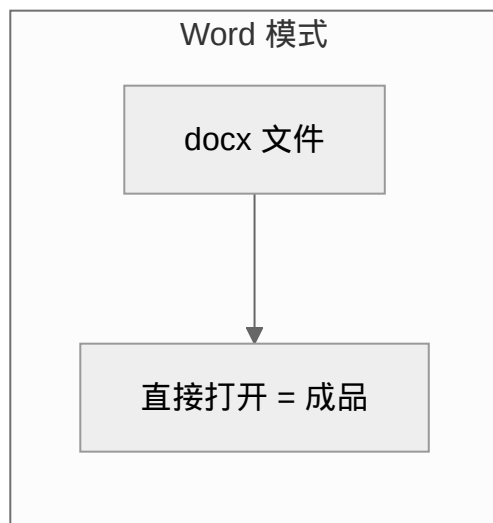
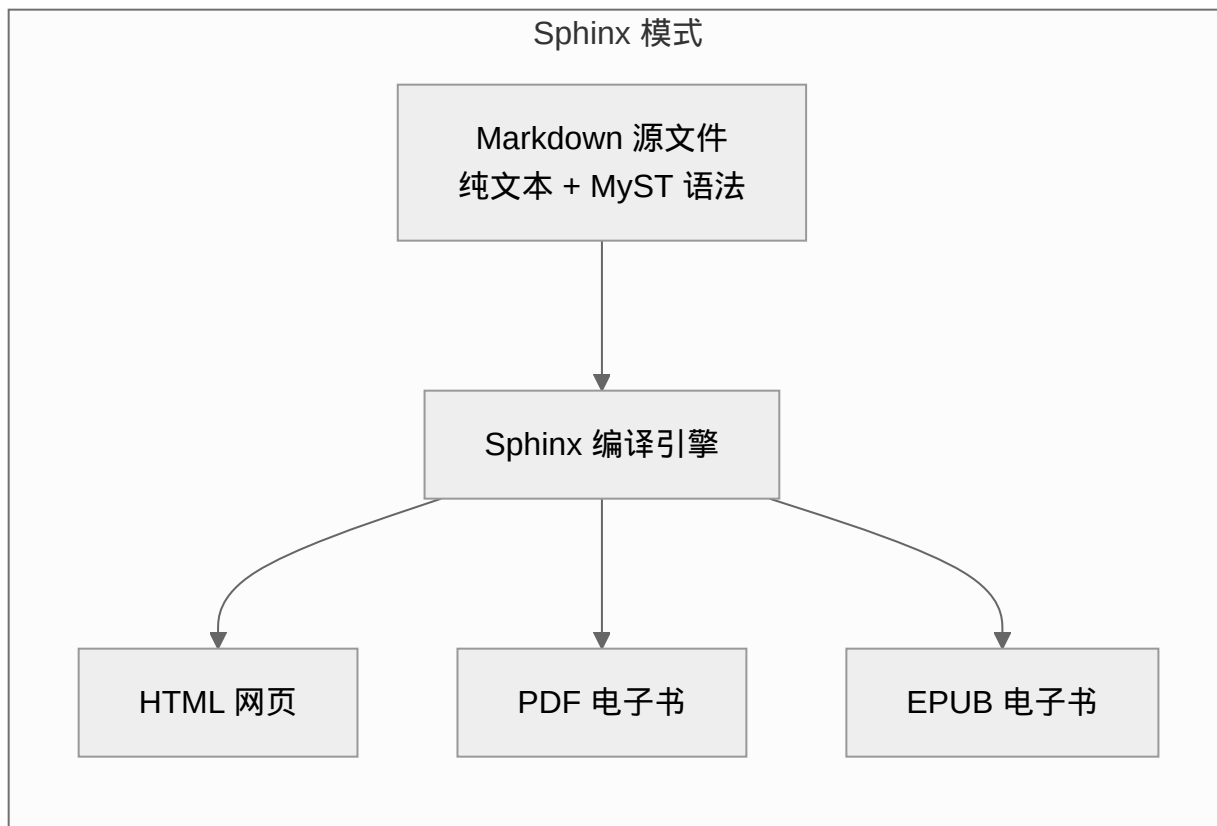
如果你是在为本书贡献内容，先阅读 [Sphinx Markdown 语法参考](#) 了解语法规范。

9.3.1 为什么需要"编译"?

如果你用过 Word 或 Google Docs, 你对"所见即所得" (WYSIWYG) 很熟悉 —— 你在编辑界面里看到的格式, **就是最终打印出来的样子**。文件直接打开就是成品。

但本书用的是另一种模式: **写代码, 再编译**。你编辑的 Markdown 文件只是"蓝图"或"底片", 必须经过构建工具链的处理, 才能变成可阅读的最终产物。

| | Word 模式 | Sphinx 模式 |
|--------------|----------------|------------------------|
| 编辑对象 | .docx 文件 | .md 文件 (纯文本) |
| 所见即所得 | 是, 格式直接可见 | 否, 纯文本 + 标记语法 |
| 交叉引用 | 手动粘贴 | 编译时自动解析 |
| 数学公式 | 公式编辑器 | LaTeX 源码, 编译后渲染 |
| 多格式输出 | 另存为 PDF (可能乱码) | 一次编译, 输出 HTML/PDF/EPUB |
| 版本控制 | 二进制文件, 难 diff | 纯文本, Git 友好 |



为什么中间要多一步编译？

因为你写的不是文档，而是文档的“生成指令”。构建过程中，Sphinx 帮你做了这些繁重工作：

1. 解析交叉引用 — `{ref}`、`{doc}` 等标签在编译时自动转成链接和页码
2. 渲染数学公式 — `$E = mc^2$` 在 HTML 中变成 MathJax，在 PDF 中变成 LaTeX

3. **渲染图表** — Mermaid 流程图需要 Chromium 转成 SVG, TikZ 需要 TeX 编译

4. **统一排版** — 同一套内容, HTML 用网页主题, PDF 用 LaTeX 排版, EPUB 用电子书规范

没有编译这一步, 你手里只有一堆 Markdown 源码, 看不到流程图、看不到公式、看不到链接 —— 就像只有底片, 没有照片。

9.3.2 系统依赖

编译需要以下工具链:

- **Python 3.11+** — 运行 Sphinx
- **TeXLive** — PDF 输出 (latex + lualatex)
- **Chromium** — Mermaid 图表渲染 (通过 Puppeteer 调用)
- **Mermaid CLI (mmdc)** — 命令行渲染 Mermaid 图表
- **中文字体** — 中文 PDF 输出 (Noto Sans CJK、Noto Emoji)
- **Roboto Flex 字体** — PDF 正文排版

为什么需要 Chromium?

本书的 Mermaid 流程图在 HTML 中由浏览器实时渲染, 但 PDF 输出需要将 Mermaid 图表先转为 SVG —— 这个转换依赖 [Sphinx Markdown 语法参考](#) 中提到的 mmdc 工具, 而 mmdc 实际调用的是 Chromium 的无头模式 (headless browser)。因此即使不在本地开发前端页面, 构建 PDF 也需要安装 Chromium。

macOS (Homebrew)

```
# 1. TeXLive (完整版约 4GB, 时间较长)
brew install texlive

# 2. Chromium
brew install --cask chromium

# 3. 中文字体和英文字体
brew install font-noto-sans-cjk font-noto-emoji
# Roboto Flex (PDF 英文正文用)
brew install --cask font-roboto-flex

# 4. Node.js (Mermaid CLI 依赖)
brew install node
```

(续下页)

(接上页)

```
# 5. Mermaid CLI
npm install -g @mermaid-js/mermaid-cli

# 6. uv (Python 包管理器)
curl -LsSf https://astral.sh/uv/install.sh | sh
```

小技巧

macOS 上 `brew install texlive` 安装的是 TeX Live small scheme, 编译中文文档可能缺 `ctex` 等包。推荐安装完整版 MacTeX:

```
brew install --cask mactex # 完整版, 约 4GB
```

如果已经装了 basic 版, 可通过 `tlmgr install <包名>` 补充缺失的 LaTeX 包。

Debian / Ubuntu

```
# 1. 系统包 (TeXLive + Chromium + 字体)
sudo apt update
sudo apt install texlive latex-cjk-all texlive-pictures \
    texlive-latex-extra texlive-extra-utils latexmk pdf2svg \
    texlive-xetex texlive-luatex fonts-noto-cjk fonts-noto-core \
    fonts-noto-extra chromium

# 2. Noto Emoji 字体 (apt 包版本可能较旧, 建议手动安装最新版)
mkdir -p ~/.local/share/fonts
wget "https://github.com/google/fonts/raw/refs/heads/main/ofl/notoemoji/NotoEmoji%5Bwght%5D.ttf"
↵ \
    -O ~/.local/share/fonts/NotoEmoji.ttf
fc-cache -f ~/.local/share/fonts

# 3. Roboto Flex 字体 (PDF 英文正文用)
wget "https://github.com/google/fonts/raw/refs/heads/main/ofl/robotoflex/RobotoFlex%5BGRAD,XOPQ,
↵XTRA,YOPQ,YTAS,YTDE,YTFI,YTLC,YTUC,opsz,slnt,wdth,wght%5D.ttf" \
    -O ~/.local/share/fonts/RobotoFlex.ttf
fc-cache -f ~/.local/share/fonts

# 4. Mermaid CLI (需要 Node.js)
sudo apt install nodejs npm
sudo npm install -g @mermaid-js/mermaid-cli
```

(续下页)

(接上页)

5. uv (Python 包管理器)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

警告

Debian/Ubuntu 自带的 chromium 包可能因为 Snap 冲突导致安装失败。如果遇到这个问题，参考 CI 脚本中的做法——先 `sudo apt purge snapd`，再从第三方仓库安装：

```
sudo apt purge snapd
sudo apt-mark hold snapd
sudo wget 'https://keyserver.ubuntu.com/pks/lookup?op=get&search=0x869689FE09306074' \
  -O '/etc/apt/trusted.gpg.d/phd-chromium.asc'
echo "deb https://freeshell.de/phd/chromium/$(lsb_release -sc) /" | \
  sudo tee /etc/apt/sources.list.d/phd-chromium.list
sudo apt update
sudo apt install chromium
```

详细的 CI 配置可参考 [.github/workflows/build-docs.yml](https://github.com/ulink-deep-learning-club/ulink-deeplearningclub-handouts/blob/main/lecture-material/.github/workflows/build-docs.yml)³³。

磁盘空间提示

TeXLive 完整安装约需 4-5 GB 磁盘空间。如果空间有限，可以仅安装必要组件：

Debian/Ubuntu 最小安装

```
sudo apt install texlive-latex-base texlive-latex-recommended \
  texlive-latex-extra texlive-luatex latexmk
```

但建议先完整安装一次——缺失包导致的编译错误往往比磁盘空间更耗费时间。

9.3.3 Python 依赖

克隆仓库并安装 Python 包：

```
git clone <仓库地址>
cd lecture-material

# 使用 uv 创建虚拟环境并安装依赖
uv venv
```

(续下页)

³³ <https://github.com/ulink-deep-learning-club/ulink-deeplearningclub-handouts/blob/main/lecture-material/.github/workflows/build-docs.yml>

(接上页)

```
uv sync

# 每次构建前激活环境
source .venv/bin/activate # Linux/macOS
# 或
.venv\Scripts\activate    # Windows PowerShell
```

依赖清单见 `pyproject.toml`, 主要包括:

| 包 | 用途 |
|-----------------------|-------------------------|
| sphinx | 文档构建引擎 |
| myst-parser | Markdown 解析 (MyST 语法支持) |
| sphinxcontrib-bibtex | BibTeX 文献管理 |
| sphinxcontrib-mermaid | Mermaid 图表 |
| sphinxcontrib-tikz | TikZ 图表 |
| sphinx-design | 增强型 UI 组件 |
| sphinx-breeze-theme | Breeze 主题 |
| sphinx-serve | 本地预览服务器 |

9.3.4 构建文档

所有构建命令在项目根目录执行, 且需要先激活虚拟环境。

构建 HTML

```
source .venv/bin/activate
make html
```

输出在 `build/html/`, 直接用浏览器打开 `build/html/index.html` 即可预览。

构建 PDF

```
source .venv/bin/activate
make latexpdf
```

输出在 `build/latex/`, 文件名为 `deeplearningclubhandouts.pdf`。

构建 EPUB

```
source .venv/bin/activate
make epub
```

输出在 build/epub/, 文件名为 DeepLearningClubHandouts.epub。EPUB 格式适合在电子书阅读器或手机上阅读。

构建技巧

ToC 没有刷新?

如果修改了 {toctree} 或文件结构后页面导航没有更新, 先删除已生成的 HTML 再重新构建:

```
rm -rf build/html
make html
```

只改内容不改结构时不需要这么做。

保留 _images 目录加速重复构建

Mermaid 和 TikZ 图表的编译非常耗时 (每次都要启动 Chromium 或 LaTeX)。反复构建时不要删除 build/html/_images/, Sphinx 会跳过已生成的图片。

同样地, 构建 EPUB 前可以将 build/html/_images/ 复制到 build/epub/ 下, 这样 EPUB 构建时也会跳过图片重生成:

```
cp -r build/html/_images build/epub/
```

EPUB 数学公式 SVG 缓存

EPUB 构建时会把数学公式编译为 SVG 图片, 这一步同样耗时。如果之前构建过 EPUB, build/epub/_images/ 中已经缓存了这些 SVG, 保留它们可以显著加速后续构建。

本地预览

使用 sphinx-serve 启动本地 HTTP 服务器:

```
source .venv/bin/activate
uv run sphinx-serve -b build
```

sphinx-serve 默认使用 8000 端口, 打开 <http://localhost:8000> 即可浏览。修改源文件后刷新页面即可看到更新。

小技巧

`-b build` 指定编译输出目录。如果不在项目根目录运行，需要调整为 `uv run sphinx-serve -b /path/to/build`。

小技巧

开发时建议只构建 HTML —— 速度比 PDF 快得多。PDF 在提交前或发布时构建一次即可。

完整构建流程（参考 CI）

以下是一个端到端的构建流程，与 GitHub Actions CI 中的步骤一致：

```
# 1. 激活环境
source .venv/bin/activate

# 2. 构建 HTML
make html

# 3. 构建 PDF（并移动到 HTML 目录方便一起发布）
make latexpdf
mv build/latex/deeplearningclubhandouts.pdf ./build/html/
```

完整的 CI 配置见 [.github/workflows/build-docs.yml](#)³⁴。

9.3.5 常见问题

Mermaid 图表不渲染

检查 Chromium 是否安装且可执行：

```
# 查看 Sphinx 构建日志中有没有以下警告
# "Chrome not found, Mermaid diagram rendering will get switched to raw mode."

# 检查 Chromium 路径
which chromium          # Linux
ls /Applications/Chromium.app/Contents/MacOS/Chromium  # macOS
```

Sphinx 会自动检测系统中的 Chrome/Chromium/Edge，检测逻辑见 `source/conf.py`。

³⁴ <https://github.com/ulink-deep-learning-club/ulink-deeplearningclub-handouts/blob/main/lecture-material/.github/workflows/build-docs.yml>

PDF 编译报错 (LaTeX 包缺失)

```
# Debian/Ubuntu: 安装完整 TeXLive
sudo apt install texlive-latex-extra texlive-pictures texlive-luatex

# macOS: 使用 tlmgr 补充单个包
tlmgr install <包名>
```

中文字体显示为方框

检查中文字体是否安装:

```
# 查看已安装的 CJK 字体
fc-list :lang=zh | head -5

# 如果为空, 安装 Noto Sans CJK
# macOS
brew install font-noto-sans-cjk
# Debian/Ubuntu
sudo apt install fonts-noto-cjk
```

9.3.6 参考

- [Sphinx Markdown 语法参考](#) — Sphinx Markdown 语法与写作规范
- [.github/workflows/build-docs.yml](#)³⁵ — CI 构建配置
- [Sphinx 官方文档](#)³⁶
- [MyST Parser 文档](#)³⁷

³⁵ <https://github.com/ulink-deep-learning-club/ulink-deeplearningclub-handouts/blob/main/lecture-material/.github/workflows/build-docs.yml>

³⁶ <https://www.sphinx-doc.org/>

³⁷ <https://myst-parser.readthedocs.io/>

写在后面

致来者：

常有人问我，为什么要为社团活动重新准备这些资料，而不是直接使用现成的教材——这看起来像是在“重新发明轮子”。说实话，我自己也没有一个非常确定的答案。这些文档在我看来，更像是一页页的 PPT，而非信息完备的教程。

理论上，我当然可以直接选用几本深度学习领域的经典著作，比如斋藤康毅老师的“鱼书”。但如果只是像学校上课那样照本宣科，对于纯粹由兴趣驱动的社团来说，恐怕不太合适。这样做很可能导致参与人数随时间“指数级衰减”。我个人对这种单向灌输的模式并不认同——它更像是在把整理与消化知识的负担，从备课者转移给听讲者，效率甚至不如鼓励大家自学。

但避免糟糕的单向灌输算不上什么真正的动机，这更像是一种妥协，就像做 presentation 必须准备 slides 一样。直到 Bob 老师告诉我“搞完这一年社团，总得留下点什么”，创作的动机才真正落地。回忆大约两年前我研究 YOLOv5 时，论文和教程中出现的各种黑话总是令人困惑。他们既没有解释这些是什么，也没有解释为什么——这些黑话被以一种常人看不懂的模式放在一起，在逻辑和现实中却是行得通的。我浏览了几乎所有我找得到的文章，仍然是一头雾水，因为字里行间似乎透露着理所应当，或者作者默认了读者能够理解。为什么复杂的 PAN/FPN 有用？为什么这卷积堆起来卷来卷去就能提取特征信息？为什么 bottleneck 要先变小再变大？为啥要残差？训练为什么退火？训练多少个 epoch 比较好？什么时候结束训练？诚然，对于论文的作者和其他的学者来说，这些所谓的黑话有另一个名字：术语（terminology）——一种高效沟通的协议。但对初学者而言，这些术语背后牵扯的问题与答案太多、太复杂，似乎非得啃下厚厚的数学教材才能理解。

但是果真如此吗？一个合理的符合现实的逻辑或现象必定可以使用一个简洁易懂的自然语言描述出来，哪怕这种描述是不严谨的。在知识体系尚未建立的阶段，准确性必须让位于理解本身，因为只有能理解那些论文在讨论什么，那些文章在分析什么，我们才有资格讨论这理解本身是否准确。而过早地陷入不必要的准确的

细节，正是我最开始尝试理解时犯下的错误。

问题来了，如何才能找到相对准确客观的自然语言描述呢？答案之一来自一种常见的文学手段：类比。那问题又来了，如何找到类比的对象？答案在于寻找底层逻辑的共通性——只要事物发展的核心驱动力（矛盾）类似，其表现出的现象就有共通之处。因此，选取日常生活中的常识作为类比，是合理的。对定义的理解来源于相似性，而非定义本身。

另一种方法是追寻发展的脉络。罗马不是一天建成的，因此，研究庞杂的系统，可以从它与其前代版本的演进角度入手，并尝试回答：改进/增加某个模块是为了解决什么；设计哲学的改变是为了规避什么。这或许可以一路追寻到这个复杂系统的第一个原始版本。那时候，这东西还没有变得复杂起来。

还有一种是寻找不严格的数学上的类比：将高级的复杂的操作不严谨地概括为一个已知操作的推广（比如这里 CNN 本质是在求 kernel 和特征图的滑动窗口内的部分的相似程度，通过我们都学过的点积）。

因此，为了规避过早陷入细节的问题，为了其他人不重蹈覆辙，将知识点重新梳理，用上述方式呈现出来，对初学者来说就显得尤为必要。这些资料的主要目的，并非成为面面俱到的百科全书，而是一份循序渐进的、带有解释的“知识清单”。

但是这足够吗？未必。就像那哲学三问（我是谁，从哪来，到哪去），构建知识体系也可以有精确对应的三问：这是什么，为什么会有/成立，so what。这份清单大体能回答前两问，有时也不行，但真正的理解大致产生于回答第三个问题时。因为理解本身更像是一种直觉，难以完全被语言捕捉。语言本身只能像盲人摸象一样勾勒出轮廓。

最后，希望几年后，这些文字还能帮到那些初来乍到的新朋友。也希望你能在这个社团里，找到探索世界的好奇与同行者。

以上是关于这些资料的一点私心。下面是一些你可能想问的技术问题：

10.1 可能的 FAQ：

10.1.1 关于组织：为什么这些主题看起来有点跳跃？

我本人是个行动派，习惯边做边想，不太热衷于“三思而后行”。在编写这些文档时，我并没有一个非常完整的大纲，因此内容组织的先后顺序可能显得有些跳跃，但都是大体围绕“是什么”到“为什么”再到“怎么用”这样的顺序。这个问题我们会在后续的修订中尽力改善。

10.1.2 关于形式：为什么不使用传统的 PPT？

那么，为什么我没有选择做成 PPT 呢？PPT 固然能突出重点，但它难以处理复杂的引用、丰富的图表插入，以及便捷的公式书写。更重要的是，现有的 AI 工具在辅助 PPT 创作方面还不够成熟。

10.1.3 为什么是 Sphinx?

TL;DR

总的来说，我们选择了一种兼顾结构化、可扩展与网页友好的技术语言（Sphinx + Markdown），它在技术文档中已经被广泛验证

最初我使用 LaTeX 编写，生成的 PDF 效果很好，但想要编译成网页就不太方便了。我曾尝试用 Lwarp 配合一个 Vue.js 写的文档查看器来搭建在线站点，但效果并不理想。受限于 Lwarp 的一些古怪问题，部署过程屡屡受阻，而且它处理文件间引用的能力也较弱。在寻找替代方案时，我遇到了 Sphinx —— 这个支撑了 Linux 和 Python 文档的强大工具。

10.1.4 为什么大量使用 AI 辅助书写?

坦白说，一半是因为懒，一半是因为自己水平有限。但主要原因有两点：第一，我们毕竟是 AI 社团；第二，我们选择的方案（编写 Markdown/LaTeX 再编译）本身很适合 AI 辅助。在我看来，只要信息准确、语言简洁，文本是否由 AI 生成，从功利角度讲并不重要。人们反感的“AI 文”，通常是通篇套话的废话文学，而非这样有实质内容、表述清晰的辅助创作。当然，这能否完全代表我个人对知识的理解？我觉得还不够。事实上，AI 在输出内容时，也常常给我带来新的启发。而文章质量的把控，始终是我这个“审稿人”的责任。从某种角度看，这也是一种“氛围编码”（Vibe coding）吧。

10.1.5 我发现了疏漏，该如何反馈?

如果你发现了文档中的错误或不足，我们非常欢迎你的指正。如果你愿意为后来的同学做点贡献，可以 fork 本资料的仓库，修改后提交 Pull Request。我和其他组员审核确认后，便会合并你的修改。（请在 PR 中尽量详细地说明改动内容，以节省大家的时间。在此先表示感谢!）

2026 年 6 月 于上海 Anson2251

Bibliography

- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. MIT Press, 2016.
- [Wer90] Paul J Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550 – 1560, 1990.
- [Bis06] Christopher M Bishop. Pattern recognition and machine learning. Springer, 2006.
- [NH10] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning*, 807 – 814. 2010.
- [RZL17] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Swish: a self-gated activation function. *arXiv preprint arXiv:1710.05941*, 2017.
- [Hub64] Peter J Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73 – 101, 1964.
- [RHW86] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533 – 536, 1986.
- [KB15] Diederik P Kingma and Jimmy Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations*. 2015.
- [Mit80] Tom M Mitchell. The need for biases in learning generalizations. Technical Report, Department of Computer Science, Laboratory for Computer Science Research, Rutgers University, 1980.
- [Ros58] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386 – 408, 1958.

- [LBD+89] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541 – 551, 1989.
- [LBBH98] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278 – 2324, 1998.
- [KB15] Diederik P Kingma and Jimmy Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations*. 2015.
- [SHK+14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929 – 1958, 2014.
- [LBBH98] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278 – 2324, 1998.
- [Mit80] Tom M Mitchell. The need for biases in learning generalizations. Technical Report, Department of Computer Science, Laboratory for Computer Science Research, Rutgers University, 1980.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770 – 778. 2016.
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 1097 – 1105. 2012.
- [LCY14] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *International Conference on Learning Representations*. 2014.
- [SHK+14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929 – 1958, 2014.
- [YCBL14] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, 3320 – 3328. 2014.
- [DBK+21] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, and others. An image is worth 16x16 words: transformers for image recognition at scale. In *International Conference on Learning Representations*. 2021.
- [GB10] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 249 – 256. 2010.

- [HZRS15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: surpassing human-level performance on imagenet classification. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 1026 – 1034. 2015.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770 – 778. 2016.
- [HZC+17] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- [HLvdMW17] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2261 – 2269. 2017.
- [IS15] Sergey Ioffe and Christian Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. In International conference on machine learning, 448 – 456. PMLR, 2015.
- [RZL17] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Swish: a self-gated activation function. arXiv preprint arXiv:1710.05941, 2017.
- [SHK+14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. The journal of machine learning research, 15(1):1929 – 1958, 2014.
- [TL19] Mingxing Tan and Quoc V Le. Efficientnet: rethinking model scaling for convolutional neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML), 6105 – 6114. 2019.
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems, volume 25, 1097 – 1105. 2012.
- [LHW18] Songtao Liu, Di Huang, and Yunhong Wang. Receptive field block net for accurate and fast object detection. In European Conference on Computer Vision (ECCV), 385 – 400. 2018.
- [SLJ+15] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition, 1 – 9. 2015.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770 – 778. 2016.
- [HSS18] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7132 – 7141. 2018.

- [HSS18] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7132 – 7141. 2018.
- [WPLK18] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. Cbam: convolutional block attention module. In Proceedings of the European Conference on Computer Vision (ECCV), 3 – 19. 2018.
- [HZC+17] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- [TL19] Mingxing Tan and Quoc V Le. Efficientnet: rethinking model scaling for convolutional neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML), 6105 – 6114. 2019.
- [TPB00] Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. arXiv preprint physics/0004057, 2000.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In European Conference on Computer Vision, 630 – 645. Springer, 2016.
- [HZC+17] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- [VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS), volume 30. 2017.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770 – 778. 2016.
- [HZC+17] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- [HSS18] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7132 – 7141. 2018.
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems, volume 25, 1097 – 1105. 2012.
- [SLJ+15] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition, 1 – 9. 2015.

- [TL19] Mingxing Tan and Quoc V Le. Efficientnet: rethinking model scaling for convolutional neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML), 6105 – 6114. 2019.
- [FC19] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: finding sparse, trainable neural networks. In International Conference on Learning Representations (ICLR). 2019.
- [HMD16] Song Han, Huizi Mao, and William J Dally. Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding. In International Conference on Learning Representations (ICLR). 2016.
- [HCS+18] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: training neural networks with low precision weights and activations. Journal of Machine Learning Research (JMLR), 18(187):1 – 30, 2018.
- [LDS90] Yann LeCun, John S Denker, and Sara A Solla. Optimal brain damage. In Advances in Neural Information Processing Systems (NIPS), volume 2. 1990.
- [DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: pre-training of deep bidirectional transformers for language understanding. In Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 4171 – 4186. 2019.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770 – 778. 2016.
- [PY10] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. IEEE Transactions on Knowledge and Data Engineering, 22(10):1345 – 1359, 2010.
- [PY10] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. IEEE Transactions on Knowledge and Data Engineering, 22(10):1345 – 1359, 2010.
- [ZQD+20] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning. Proceedings of the IEEE, 109(1):43 – 76, 2020.
- [BMR+20] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and others. Language models are few-shot learners. Advances in Neural Information Processing Systems, 33:1877 – 1901, 2020.
- [DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: pre-training of deep bidirectional transformers for language understanding. In Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 4171 – 4186. 2019.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770 – 778. 2016.

- [RKH+21] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, and others. Learning transferable visual models from natural language supervision. In International Conference on Machine Learning, 8748 – 8763. PMLR, 2021.
- [RNSS18] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. In OpenAI Technical Report. 2018.
- [RWC+19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI Blog, 1(8):9, 2019.
- [TMS+23] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Yasmine Almahairi, Angela Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, and others. Llama 2: open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023.
- [KPR+17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, and others. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences, 114(13):3521 – 3526, 2017.
- [LCWJ15] Mingsheng Long, Yue Cao, Jianmin Wang, and Michael Jordan. Learning transferable features with deep adaptation networks. In International Conference on Machine Learning, 97 – 105. PMLR, 2015.
- [RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: convolutional networks for biomedical image segmentation. In Medical Image Computing and Computer-Assisted Intervention (MICCAI), 234 – 241. Springer, 2015.
- [HZRS16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, 770 – 778. 2016.
- [IS15] Sergey Ioffe and Christian Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. In International conference on machine learning, 448 – 456. PMLR, 2015.
- [RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: convolutional networks for biomedical image segmentation. In Medical Image Computing and Computer-Assisted Intervention (MICCAI), 234 – 241. Springer, 2015.
- [SZ15] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In International Conference on Learning Representations. 2015.
- [LGG+17] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2980 – 2988. 2017.
- [KB15] Diederik P Kingma and Jimmy Ba. Adam: a method for stochastic optimization. In International Conference on Learning Representations. 2015.

- [Elm90] Jeffrey L Elman. Finding structure in time. *Cognitive Science*, 14(2):179 – 211, 1990.
- [Jor86] Michael I Jordan. Serial order: a parallel distributed processing approach. Technical Report ICS Report 8604, Institute for Cognitive Science, University of California, San Diego, 1986.
- [RHW86] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533 – 536, 1986.
- [Wer90] Paul J Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550 – 1560, 1990.
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735 – 1780, 1997.
- [VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. 2017.
- [BKH16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. In *International Conference on Machine Learning*, 1247 – 1256. PMLR, 2016.
- [BCB15] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*. 2015.
- [CLJ20] Jean-Baptiste Cordonnier, Andreas Loukas, and Martin Jaggi. On the relationship between self-attention and convolutional layers. In *International Conference on Learning Representations (ICLR)*. 2020.
- [GSBL21] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 5484 – 5495. 2021.
- [VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. 2017.
- [GD23] Albert Gu and Tri Dao. Mamba: linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [GGR22] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*. 2022.
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735 – 1780, 1997.
- [BCB15] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*. 2015.
- [Elm90] Jeffrey L Elman. Finding structure in time. *Cognitive Science*, 14(2):179 – 211, 1990.

- [GD23] Albert Gu and Tri Dao. Mamba: linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023.
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735 – 1780, 1997.
- [VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. 2017.